BİLGİSAYAR MÜHENDİSLİĞİ · 1. SINIF

```java
public class Merhaba {
    public static void main(String[] args) {
        System.out.println("Merhaba, Java!");
    }
}
```

Java ile Programlamaya Giriş

Konu anlatımı, sınav tuzakları ve çözümlü soru bankası

17	208	4	2
bölüm	soru	mini proje	deneme sınavı

HAZIRLAYAN

Ahmet Şerif Kart · BARÜ – Bartın Üniversitesi

Java 25 LTS · VS Code · 2026

İçindekiler

Bu Kitap Nasıl Kullanılır? . . . 8

1 Programlamaya ve Java'ya Giriş . . . 11
- 1.1 Program ve programlama dili . . . 11
- 1.2 Java nasıl çalışır? . . . 11
- 1.3 Kurulum: Java 25 ve VS Code . . . 12
- 1.4 İlk program: Merhaba Dünya . . . 13
- 1.5 Terminalden derleme ve çalıştırma . . . 14
- 1.6 print, println ve kaçış dizileri . . . 14
- 1.7 Yorum satırları . . . 16
- 1.8 Hata türleri . . . 16
- 1.9 Java 25 ile gelen kısa yazım . . . 18

2 Değişkenler, Veri Tipleri ve Operatörler . . . 24
- 2.1 Değişken nedir? . . . 24
- 2.2 İsimlendirme kuralları . . . 25
- 2.3 İlkel veri tipleri . . . 26
- 2.4 Sabitler (final) ve var . . . 27
- 2.5 Aritmetik operatörler . . . 29
- 2.6 Tip dönüşümü (casting) . . . 30
- 2.7 Atama ve bileşik atama operatörleri . . . 32
- 2.8 Artırma ve azaltma: x++ ve ++x . . . 33
- 2.9 Math sınıfı . . . 34
- 2.10 Taşma ve ondalık hassasiyet . . . 36
- 2.11 Bit operatörleri ve sayı yazımları . . . 38

3 Klavyeden Girdi ve Biçimli Çıktı . . . 47
- 3.1 Scanner ile klavyeden okuma . . . 47
- 3.2 next() ve nextLine() farkı . . . 48
- 3.3 nextInt() + nextLine() tuzağı . . . 49
- 3.4 printf ile biçimli çıktı . . . 51
- 3.5 String.format . . . 53
- 3.6 Türkçe bilgisayarlarda ondalık virgül . . . 54
- 3.7 Örnek uygulama: Vücut kitle indeksi . . . 54

4 Koşullu İfadeler . . . 63
- 4.1 İlişkisel operatörler ve boolean ifadeler . . . 63
- 4.2 if ifadesi . . . 64
- 4.3 if-else . . . 66
- 4.4 else-if merdiveni . . . 67
- 4.5 İç içe if ve sarkan else . . . 67
- 4.6 Mantıksal operatörler . . . 69
- 4.7 Operatör önceliği . . . 71
- 4.8 Üçlü operatör (?:) . . . 72
- 4.9 switch ifadesi . . . 73
- 4.10 Yeni switch: ok (->) sözdizimi . . . 76
- 4.11 String ve double karşılaştırma . . . 77
- 4.12 Örnek uygulamalar . . . 78

5 Döngüler . . . 86
5.1 Neden döngü? . . . 86
5.2 while döngüsü . . . 86
5.3 do-while döngüsü . . . 87
5.4 for döngüsü . . . 88
5.5 Sonsuz döngü, break ve continue . . . 90
5.6 İç içe döngüler . . . 91
5.7 Yıldız desenleri . . . 92
5.8 Döngü izleme tablosu ve off-by-one . . . 94
5.9 Klasik döngü algoritmaları . . . 95
5.10 Kullanıcı girdisiyle döngüler . . . 99

6 Metotlar . . . 109
6.1 Neden metot? . . . 109
6.2 Metodun anatomisi . . . 110
6.3 void ve değer döndüren metotlar . . . 111
6.4 return ile erken çıkış . . . 112
6.5 Parametre ve argüman . . . 114
6.6 Metot çağrıldığında ne olur? . . . 114
6.7 Aşırı yükleme (overloading) . . . 116
6.8 Değer ile geçirme (pass-by-value) . . . 119
6.9 Kapsam (scope) ve yerel değişkenler . . . 120
6.10 Neden şimdilik static? . . . 122
6.11 Javadoc yorumları . . . 122
6.12 Algoritmaları metotlara dönüştürmek . . . 123
6.13 Hazır metotlar ve Math sınıfı . . . 124
6.14 Özyinelemeye kısa bir bakış . . . 125

7 Diziler . . . 134
7.1 Neden dizi? . . . 134
7.2 Dizi tanımlama ve oluşturma . . . 134
7.3 length ve indeksler . . . 136
7.4 Diziyi dolaşmak . . . 137
7.5 Temel dizi algoritmaları . . . 138
7.6 Scanner ile dizi doldurma . . . 140
7.7 Diziler referans tipidir . . . 140
7.8 Diziyi kopyalama . . . 141
7.9 Arrays sınıfı . . . 142
7.10 Diziler ve metotlar . . . 143
7.11 Ters çevirme ve kaydırma . . . 145
7.12 İki boyutlu diziler . . . 146
7.13 Komut satırı argümanları: String[] args . . . 149

8 String ve Karakter İşlemleri . . . 158
8.1 String nedir? . . . 158
8.2 String değiştirilemez (immutable) . . . 160
8.3 Temel String metotları . . . 161
8.4 Büyük/küçük harf ve Türkçe I sorunu . . . 166
8.5 String karşılaştırma: == ve equals . . . 166
8.6 Sıralama karşılaştırması: compareTo . . . 168
8.7 Karakter karakter dolaşma ve Character sınıfı . . . 169
8.8 StringBuilder . . . 171
8.9 Dönüşümler: String, sayı ve karakter . . . 172
8.10 Klasik String algoritmaları . . . 174

P1 Öğrenci Not Sistemi . . . 185

P2 XOX (Tic-Tac-Toe) Oyunu . . . 194

S Deneme Vize Sınavı . . . 201

9 Sınıflar ve Nesneler . . . 217
9.1 Nesneye yönelik programlama fikri . . . 217
9.2 Sınıf ve nesne . . . 218
9.3 Alanlar ve metotlar . . . 219
9.4 Referans değişkenler: stack ve heap . . . 221
9.5 Yapıcılar (constructor) . . . 223
9.6 this anahtar kelimesi . . . 226
9.7 Kapsülleme (encapsulation) . . . 227
9.8 Erişim belirteçleri . . . 229
9.9 static alanlar ve metotlar . . . 229
9.10 toString metodu . . . 231
9.11 null ve NullPointerException . . . 232
9.12 Nesne dizileri . . . 233
9.13 Nesneleri == ile karşılaştırmak . . . 234
9.14 UML sınıf diyagramı . . . 235

10 Kalıtım . . . 247
10.1 Kod tekrarı problemi . . . 247
10.2 extends ile kalıtım . . . 247
10.3 Neler kalıtılır? . . . 249
10.4 super(...) ile üst sınıf yapıcısını çağırma . . . 250
10.5 Yapıcıların çalışma sırası . . . 252
10.6 Metot ezme (overriding) . . . 253
10.7 super.metot() ile üst sınıfın metodunu çağırma . . . 256
10.8 Overloading ve overriding farkı . . . 257
10.9 Object sınıfı: her sınıfın atası . . . 258
10.10 protected erişim . . . 261
10.11 final: ezilemez, genişletilemez . . . 262
10.12 Java'da tekli kalıtım . . . 263
10.13 is-a ve has-a: kalıtım mı, kompozisyon mu? . . . 264
10.14 Çok seviyeli kalıtım örneği . . . 265

11 Polimorfizm, Soyut Sınıflar ve Arayüzler . . . 280

11.1 Polimorfizm nedir? . . . 280
11.2 Yukarı dönüşüm (upcasting) . . . 281
11.3 Dinamik bağlama: hangi metot çalışır? . . . 281
11.4 Aşağı dönüşüm (downcasting) . . . 284
11.5 instanceof ve pattern matching . . . 285
11.6 Polimorfik diziler . . . 286
11.7 Soyut sınıflar (abstract class) . . . 288
11.8 Arayüzler (interface) . . . 289
11.9 Soyut sınıf mı, arayüz mü? . . . 292
11.10 Comparable'a kısa bir bakış . . . 293
11.11 enum: sabit değer kümeleri . . . 294
11.12 record: veri taşıyan kısa sınıflar . . . 296
11.13 sealed sınıflar ve switch ile desen eşleme . . . 296
11.14 Polimorfik olmayanlar: static, final ve alanlar . . . 299

12 İstisna (Exception) Yönetimi . . . 310

12.1 Exception nedir? . . . 310
12.2 İstisna hiyerarşisi . . . 311
12.3 Checked ve unchecked exception'lar . . . 312
12.4 try-catch ile yakalama . . . 313
12.5 Birden çok catch . . . 314
12.6 finally: her durumda çalışan blok . . . 315
12.7 throw: exception fırlatmak . . . 316
12.8 throws: checked exception'ı bildirmek . . . 317
12.9 Kendi exception sınıfını yazmak . . . 318
12.10 Exception bilgilerini okumak . . . 319
12.11 try-with-resources . . . 321
12.12 Kullanıcı girdisini doğrulamak . . . 322
12.13 İyi uygulamalar . . . 323

P3 Banka Hesabı Simülasyonu . . . 334

13 Koleksiyonlar ve Generics . . . 344

13.1 Dizilerin sınırı: koleksiyonlar . . . 344
13.2 Sarmalayıcı sınıflar ve autoboxing . . . 345
13.3 ArrayList . . . 347
13.4 Dolaşırken eleman silmek . . . 349
13.5 List arayüzü, List.of ve LinkedList . . . 350
13.6 HashMap: anahtar–değer çiftleri . . . 353
13.7 Set: HashSet ve TreeSet . . . 356
13.8 Generics . . . 357
13.9 Collections sınıfı ve Comparable . . . 361
13.10 Lambda ifadeleri ve Comparator . . . 363
13.11 Stream API'ye giriş . . . 366

14 Dosya İşlemleri . . . 376

14.1 Neden dosya? . . . 376
14.2 Dosya yolları ve çalışma klasörü . . . 377
14.3 Files ile okuma ve yazma . . . 378
14.4 IOException: kontrol edilen istisna . . . 380
14.5 BufferedReader ve BufferedWriter . . . 381
14.6 Scanner ile dosya okuma . . . 383
14.7 PrintWriter ile biçimli yazma . . . 384
14.8 Karakter kodlaması: UTF-8 ve Türkçe karakterler . . . 386
14.9 CSV dosyası işleme . . . 387

P4 Kütüphane Yönetim Sistemi . . . 398

15 Özyineleme (Recursion) . . . 408

15.1 Özyineleme nedir? . . . 408
15.2 Temel durum ve özyinelemeli durum . . . 409
15.3 Çağrı yığını (call stack) . . . 410
15.4 Sayılarla özyineleme . . . 412
15.5 Fibonacci ve tekrar eden hesaplar . . . 414
15.6 String ve dizilerle özyineleme . . . 415
15.7 Hanoi kuleleri . . . 417
15.8 Özyinelemeli ikili arama . . . 418
15.9 Temel durum unutulursa: StackOverflowError . . . 419
15.10 Özyineleme mi, döngü mü? . . . 420
15.11 Çağrıdan önce mi, sonra mı? . . . 421

16 Arama, Sıralama ve Big-O . . . 430

16.1 Algoritma ve verimlilik . . . 430
16.2 Doğrusal arama (linear search) . . . 430
16.3 İkili arama (binary search) . . . 431
16.4 Kabarcık sıralaması (bubble sort) . . . 434
16.5 Seçmeli sıralama (selection sort) . . . 435
16.6 Eklemeli sıralama (insertion sort) . . . 436
16.7 Algoritmaları karşılaştırma . . . 437
16.8 Java'nın hazır sıralama metotları . . . 439
16.9 Big-O gösterimi . . . 440
16.10 Döngülerden karmaşıklık çıkarma . . . 441
16.11 Deneysel karşılaştırma . . . 442

17 Swing ile Grafik Arayüze Giriş . . . 453

17.1 Konsoldan pencereye . . . 453
17.2 İlk pencere: JFrame . . . 454
17.3 Temel bileşenler . . . 455
17.4 Yerleşim yöneticileri (layout managers) . . . 456
17.5 Olaylar ve ActionListener . . . 458
17.6 Metin kutusundan sayı okumak . . . 461
17.7 EDT ve SwingUtilities.invokeLater . . . 463
17.8 JOptionPane ile diyalog kutuları . . . 463
17.9 Uygulama 1: Tıklama sayacı . . . 465
17.10 Uygulama 2: Sıcaklık dönüştürücü . . . 466
17.11 Uygulama 3: Mini hesap makinesi . . . 467

S Deneme Final Sınavı 480
A Java Cheat Sheet 502
B Terimler Sözlüğü 507
C Sık Karşılaşılan Hata Mesajları 515

Bu Kitap Nasıl Kullanılır?

Bu kitap, bilgisayar mühendisliğinin ilk yılında Java dersini alan ve daha önce hiç kod yazmamış öğrenci için yazıldı. Bilgisayarın ne olduğunu, derleyicinin ne işe yaradığını kabaca biliyorsun; ama değişken, döngü, sınıf gibi kavramlarla henüz tanışmadın. Hedef iki tane: **Java'yı gerçekten öğrenmen** ve **vize/finalde soru kaçırmaman**.

Kitabın yapısı

Kitap, tipik bir iki dönemlik "Programlamaya Giriş" dersinin sırasını takip eder:

Kısım	Bölümler	Ne öğreneceksin
Temeller	1–8	Değişken, koşul, döngü, metot, dizi, String
Projeler 1–2 + Vize denemesi	—	İlk sekiz bölümü birleştiren uygulamalar
Nesneye yönelik programlama	9–12	Sınıf, nesne, kalıtım, polimorfizm, arayüz, exception
Veri ve dosya	13–14	ArrayList, HashMap, generics, dosya okuma/yazma
Algoritmalar	15–16	Özyineleme, arama, sıralama, Big-O
Grafik arayüz	17	Swing ile pencere, buton, olay
Projeler 3–4 + Final denemesi	—	Tüm dönemi birleştiren uygulamalar

Her bölüm aynı düzende ilerler: önce **"Bu bölümde öğreneceklerin"** listesi, sonra konu anlatımı ve örnekler, ardından **Bölüm Özeti**, en sonda **Sorular** ve yeni bir sayfada **Çözümler**.

Kutular

Metnin içinde dört tür renkli kutu göreceksin. Hepsinin bir görevi var:

> **İpucu**
>
> İşini kolaylaştıran kısa yollar, VS Code numaraları, akılda tutma yöntemleri.

> **Sık Yapılan Hata**
>
> Öğrencilerin en sık düştüğü tuzaklar. Sınavda puan kaybettiren hataların çoğu bu kutulardan çıkar.

> **Sınavda Çıkar**
>
> Hocaların sormayı çok sevdiği konular. Vize/final öncesi sadece bu kutuları okuyarak bile hızlı tekrar yapabilirsin.

> **Java 25 Notu**
>
> Java 25 ile gelen yeni ve kolaylaştırıcı özellikler. Bilmek iyidir, ama sınavda klasik yazımı kullan.

Kod ve çıktılar

Kitaptaki **her program JDK 25 ile derlendi ve çalıştırıldı**; çıktı kutularındaki yazılar elle yazılmadı, programın gerçek çıktısı olarak kitaba eklendi. Yani bir örneği kendi bilgisayarında çalıştırdığında aynı sonucu görmelisin. Tek fark ondalık ayırıcıda olabilir: Çıktılar İngilizce bölge ayarıyla (`Locale.US`) üretildi; bilgisayarın Türkçe ayardaysa `3.5` yerine `3,5` görebilirsin (nedeni ve çözümü Bölüm 3'te).

```java
public class Ornek {
    public static void main(String[] args) {
        System.out.println("Kod kutusu böyle görünür.");
    }
}
```

Çıktı

```
Kod kutusu böyle görünür.
```

Programın klavyeden bir şey okuduğu örneklerde, senin yazdığın kısım çıktı kutusunda **turuncu** renkte gösterilir; tıpkı terminalde gördüğün gibi:

```java
import java.util.Scanner;

public class Selam {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Adın: ");
        String ad = sc.nextLine();
        System.out.println("Selam " + ad + "!");
    }
}
```

Çıktı

```
Adın: Ayşe
Selam Ayşe!
```

> **Not: Ondalık sayılar ve virgül**
>
> Kitaptaki çıktılar, ders kitaplarında ve sınavlarda olduğu gibi **ondalık nokta** ile verilmiştir (`3.75`). Bilgisayarının dili Türkçe ise `printf` bazen `3,75` yazdırabilir; bunun nedenini ve çözümünü Bölüm 3'te göreceksin.

Sorular nasıl çözülmeli?

Her bölümün sonunda üç tür soru var:

- **Çoktan seçmeli (5 soru):** Kavramları ölçer. Önce cevabı kendin bul, sonra şıklara bak.
- **Çıktı tahmini (3 soru):** "Bu kod ne yazdırır?" Sınavların vazgeçilmezi. Kodu **kâğıt üzerinde, satır satır** izle; değişkenlerin değerini bir tabloya yaz. Bilgisayarda çalıştırmadan önce cevabını mutlaka kâğıda yaz.
- **Kod yazma (2 soru):** Küçük programlar. Önce kendin yaz, VS Code'da çalıştır, sonra çözümle karşılaştır. Çözümdeki kod tek doğru cevap değildir; senin kodun da doğru çıktı veriyorsa doğrudur.

İpucu: Çalışma önerisi

Bölümü oku → örnekleri **elle yazarak** (kopyala-yapıştır yok!) çalıştır → soruları çöz → yanlışlarını çözümlerden kontrol et → bir hafta sonra sadece kutuları ve özeti tekrar oku. Programlama izleyerek değil, yazarak öğrenilir.

BÖLÜM 1

Programlamaya ve Java'ya Giriş

Bu Bölümde Öğreneceklerin

- Java programının yazıldıktan sonra nasıl çalıştığını (`javac` → bytecode → JVM)
- JDK, JRE ve JVM arasındaki farkı
- Java 25 ve VS Code kurulumunu
- İlk programını yazıp hem VS Code'dan hem terminalden çalıştırmayı
- `print` ile `println` farkını ve kaçış dizilerini (`\n`, `\t`, `\"`)
- Derleme hatası, çalışma zamanı hatası ve mantık hatası arasındaki farkı

1.1 Program ve programlama dili

Program, bilgisayara ne yapacağını adım adım anlatan komutlar dizisidir. İşlemci (CPU) sadece kendi **makine dilini** (0 ve 1'lerden oluşan komutlar) anlar. İnsanların makine diliyle program yazması çok zor olduğu için **yüksek seviyeli programlama dilleri** (high-level language) geliştirilmiştir: Java, C, Python bunlardan birkaçı.

Yüksek seviyeli dilde yazdığımız metne **kaynak kod** (source code) denir. Kaynak kodun makinenin anlayacağı hale gelmesi için çevrilmesi gerekir. Bunun iki klasik yolu vardır:

- **Derleyici** (compiler): Bütün programı bir kerede çevirir, sonra çevrilmiş hali çalıştırılır. (C, C++)
- **Yorumlayıcı** (interpreter): Kodu satır satır okuyup o anda çalıştırır. (klasik Python)

Java ikisinin ortasında bir yol izler. Bunu bir sonraki başlıkta göreceğiz.

1.2 Java nasıl çalışır?

Java programı iki aşamada çalışır:

1. **Derleme:** `javac` derleyicisi, `Merhaba.java` dosyasındaki kaynak kodu **bytecode** denen ara bir dile çevirir ve `Merhaba.class` dosyasını üretir. Bytecode ne insan dilidir ne de işlemcinin dili; Java'ya özel bir ara koddur.
2. **Çalıştırma:** `java` komutu **JVM**'i (Java Virtual Machine, Java Sanal Makinesi) başlatır. JVM, `.class` dosyasındaki bytecode'u o bilgisayarın işlemcisinin anlayacağı komutlara çevirerek çalıştırır.

Adım	Komut	Girdi	Çıktı
1. Derleme	`javac Merhaba.java`	`Merhaba.java` (kaynak kod)	`Merhaba.class` (bytecode)
2. Çalıştırma	`java Merhaba`	`Merhaba.class`	Programın çıktısı

Bu yapının büyük avantajı şudur: Aynı `.class` dosyası Windows'ta, macOS'ta, Linux'ta değiştirilmeden çalışır. Yeter ki o cihazda bir JVM olsun. (Android telefonlarda standart JVM yoktur; oradaki uygulamalar bytecode'un başka bir biçime dönüştürülmesiyle çalışır.) Java'nın ünlü sloganı buradan gelir: **"Bir kere yaz, her yerde çalıştır"** (*Write Once, Run Anywhere*). Buna **platform bağımsızlığı** denir.

JDK, JRE ve JVM

Bu üç kısaltma birbirine çok karıştırılır:

Kısaltma	Açılımı	Ne işe yarar?	İçinde ne var?
JVM	Java Virtual Machine	Bytecode'u çalıştırır	Sadece çalıştırma motoru
JRE	Java Runtime Environment	Hazır Java programlarını çalıştırmaya yeter	JVM + standart kütüphaneler
JDK	Java Development Kit	Program **geliştirmek** için gerekir	JRE + `javac` derleyici + araçlar

Kısaca: **JDK** ⊃ **JRE** ⊃ **JVM**. Program yazacağın için sana **JDK** lazım.

> **Not: Güncel Java'da JRE**
>
> Java 11'den itibaren ayrı bir JRE kurulum paketi dağıtılmaz; yalnız JDK indirilir (uygulamaya özel çalışma ortamı gerekirse JDK'daki `jlink` aracıyla üretilir). Yine de JDK–JRE–JVM ayrımı kavram olarak sınavlarda sorulmaya devam eder.

> **Sınavda Çıkar**
>
> "JDK, JRE ve JVM arasındaki fark nedir?" ve "Java neden platform bağımsızdır?" klasik sorulardır. Cevabın anahtar kelimeleri: **bytecode** ve **JVM**. Java kaynak kodu doğrudan makine koduna değil, bytecode'a derlenir; bytecode'u her platformdaki JVM çalıştırır.

> **Sık Yapılan Hata**
>
> "Java yorumlanan bir dildir" ya da "Java derlenen bir dildir" demek tek başına eksik kalır. Doğrusu: Java önce **derlenir** (bytecode'a), sonra bytecode JVM tarafından **çalıştırılır**. Modern JVM'ler sık çalışan kodu **JIT** (Just-In-Time) derleyici ile makine koduna da çevirir; bu yüzden Java hızlıdır.

1.3 Kurulum: Java 25 ve VS Code

1. JDK 25'i kur

Java 25, Eylül 2025'te çıkan **LTS** (Long-Term Support, uzun süreli destek) sürümüdür; yıllarca güncelleme alacağı için kitap boyunca bunu kullanıyoruz.

- **Windows / macOS / Linux:** adoptium.net adresinden **Eclipse Temurin JDK 25** kurulum dosyasını indir ve kur. (Oracle'ın sitesindeki JDK 25 de olur.)
- macOS'ta Homebrew kullanıyorsan: `brew install openjdk@25`

Kurulumdan sonra bir terminal (Windows'ta *Komut İstemi* veya *PowerShell*) aç ve şunu yaz:

```
java –version
javac –version
```

İkisi de `25` ile başlayan bir sürüm numarası göstermeli. "komut bulunamadı" (*not recognized*) hatası alırsan, kurulumda **"Add to PATH"** / **"Set JAVA_HOME"** seçeneklerini işaretleyerek tekrar kur.

2. VS Code ve Java eklentileri

1. code.visualstudio.com adresinden VS Code'u kur.

2. Sol menüden **Extensions** (Ctrl+Shift+X) bölümüne gir, **"Extension Pack for Java"** (Microsoft) paketini kur. Bu paket; kod tamamlama, hata gösterme, çalıştırma ve hata ayıklama (debug) özelliklerini getirir.
3. Bilgisayarda bir klasör aç (ör. `JavaDersleri`) ve VS Code'da **File → Open Folder** ile bu klasörü aç.
4. Klasörde `Merhaba.java` adında yeni bir dosya oluştur.

> **İpucu**
>
> VS Code'da `main` metodunun hemen üstünde küçük **Run | Debug** yazıları çıkar. **Run**'a tıklamak, derleme ve çalıştırmayı tek seferde yapar. Çıktı alttaki **Terminal** panelinde görünür.

1.4 İlk program: Merhaba Dünya

Programlama geleneğidir: Her yeni dilde ilk program ekrana bir selam yazar.

Merhaba.java

```java
public class Merhaba {
    public static void main(String[] args) {
        System.out.println("Merhaba Dünya!");
    }
}
```

Çıktı

```
Merhaba Dünya!
```

Beş satırlık bu program, Java'nın neredeyse bütün temel kurallarını içinde barındırır. Satır satır inceleyelim:

Kod	Anlamı
`public class Merhaba`	`Merhaba` adında bir **sınıf** (class) tanımlıyoruz. Java'da her kod bir sınıfın içinde olmak zorundadır.
`{ ... }`	Süslü parantezler bir **blok** oluşturur: sınıfın veya metodun gövdesi. Her açılan `{` kapanmalıdır.
`public static void main(String[] args)`	**main metodu**: Programın başladığı yer. JVM programı çalıştırırken ilk olarak burayı arar.
`System.out.println(...)`	Parantez içindekini ekrana yazar ve **alt satıra geçer**.
`"Merhaba Dünya!"`	Çift tırnak içindeki metin bir **String**'dir (karakter dizisi).
`;`	Her **komut** (statement) noktalı virgülle biter.

`main` metodunun başındaki kelimelerin (`public`, `static`, `void`, `String[] args`) tam anlamını Bölüm 6 ve 9'da öğreneceksin. Şimdilik bu satırı **kalıp olarak ezberle**; her programda aynen yazacaksın.

> **Sık Yapılan Hata: Dosya adı ve sınıf adı**
>
> `public class Merhaba` yazdıysan dosyanın adı **tam olarak** `Merhaba.java` olmalıdır. `merhaba.java` ya da `Merhaba1.java` olursa derleyici şu hatayı verir: *class Merhaba is public, should be declared in a file named Merhaba.java*.

● Sık Yapılan Hata: Java büyük/küçük harf duyarlıdır

Java **case-sensitive** bir dildir: `System` ile `system`, `Main` ile `main` farklı şeylerdir. `system.out.println(...)` yazarsan program derlenmez. Kural: `String` ve `System` gibi sınıf adları büyük harfle başlar.

1.5 Terminalden derleme ve çalıştırma

VS Code'un **Run** düğmesi işini kolaylaştırır; ama sınavlarda ve mülakatlarda arka planda ne olduğunu bilmen beklenir. Terminalde, `Merhaba.java` dosyasının bulunduğu klasördeyken:

```
javac Merhaba.java      ← derle: Merhaba.class oluşur
java Merhaba            ← çalıştır: .class uzantısı YAZILMAZ
Merhaba Dünya!
```

İkinci komutta `java Merhaba.class` değil `java Merhaba` yazdığına dikkat et: `java` komutuna dosya adı değil, **sınıf adı** verilir.

● İpucu: Tek dosyalık programlar için kısa yol

Java 11'den beri, tek dosyalık bir programı `javac` olmadan doğrudan çalıştırabilirsin: `java Merhaba.java`. Java dosyayı arka planda derleyip hemen çalıştırır (`.class` dosyası oluşmaz). Hızlı denemeler için çok kullanışlıdır.

1.6 print, println ve kaçış dizileri

Ekrana yazmanın iki temel yolu vardır:

- `System.out.println(x)` → `x`'i yazar, sonra **alt satıra geçer** (*print line*).
- `System.out.print(x)` → `x`'i yazar, **aynı satırda kalır**.

PrintFarki.java

```java
public class PrintFarki {
    public static void main(String[] args) {
        System.out.print("Bir ");
        System.out.print("İki ");
        System.out.println("Üç");
        System.out.println("Dört");
        System.out.println();
        System.out.println("Beş");
    }
}
```

Çıktı

```
Bir İki Üç
Dört

Beş
```

Parametresiz `System.out.println()` sadece alt satıra geçer, yani boş bir satır bırakır.

Bir String'in içine yazılamayan ya da özel anlamı olan karakterler için **kaçış dizileri** (escape sequence) kullanılır. Hepsi ters bölü `\` ile başlar:

Kaçış dizisi	Anlamı
`\n`	Yeni satır (alt satıra geç)
`\t`	Sekme (tab), genelde birkaç boşluk kadar ileri atlar
`\"`	Çift tırnak karakteri
`\'`	Tek tırnak karakteri
`\\`	Ters bölü karakterinin kendisi

Kacis.java

```java
public class Kacis {
    public static void main(String[] args) {
        System.out.println("Ad\tSoyad\tNot");
        System.out.println("Ali\tYılmaz\t85");
        System.out.println("Hoca \"çok kolay\" dedi.");
        System.out.println("Dosya: C:\\java\\Merhaba.java");
        System.out.print("Satır 1\nSatır 2\n");
    }
}
```

Çıktı

```
Ad      Soyad   Not
Ali     Yılmaz  85
Hoca "çok kolay" dedi.
Dosya: C:\java\Merhaba.java
Satır 1
Satır 2
```

> **Sınavda Çıkar**
>
> Çıktı sorularında `print` / `println` karışımı ve `\n` çok sık kullanılır. Kural basit: Ekrandaki "imleç" `println`'den ve `\n`'den sonra alt satıra iner, `print`'ten sonra olduğu yerde kalır. Soruyu çözerken imlecin nerede olduğunu takip et.

String birleştirme

`+` operatörü iki String'i yan yana ekler (*concatenation*). Sayılarla kullanıldığında ise toplama yapar. İkisi bir arada olduğunda işlem **soldan sağa** yapılır ve bir String ile karşılaşıldığı andan itibaren her şey metne dönüşür:

Birlestirme.java

```java
public class Birlestirme {
    public static void main(String[] args) {
        System.out.println("Java " + "25");
        System.out.println(1 + 2 + " elma");
        System.out.println("elma " + 1 + 2);
        System.out.println("elma " + (1 + 2));
    }
}
```

Çıktı

```
Java 25
3 elma
elma 12
elma 3
```

İkinci satırda önce `1 + 2 = 3` hesaplanır, sonra `"3 elma"` olur. Üçüncü satırda ise önce `"elma " + 1` → `"elma 1"`, sonra `"elma 1" + 2` → `"elma 12"` olur. Parantez kullanırsan toplama önce yapılır.

> **Sık Yapılan Hata**
>
> `"Toplam: " + 5 + 3` ifadesi `Toplam: 8` değil **`Toplam: 53`** yazdırır. Sayıları toplamak istiyorsan parantez kullan: `"Toplam: " + (5 + 3)`. Bu, vize sınavlarının en sevilen tuzağıdır.

1.7 Yorum satırları

Yorumlar (comments), derleyicinin tamamen görmezden geldiği, kodu okuyan insana yönelik notlardır:

Yorumlar.java

```java
/**
 * Javadoc yorumu: Sınıfın ne yaptığını belgeler.
 * Bu programın amacı yorum türlerini göstermektir.
 */
public class Yorumlar {
    public static void main(String[] args) {
        // Tek satırlık yorum: satırın sonuna kadar geçerlidir
        System.out.println("Merhaba"); // kodun yanına da yazılabilir

        /* Çok satırlı yorum:
           Bu iki satır arasındaki her şey yok sayılır.
        System.out.println("Bu satır çalışmaz"); */
    }
}
```

Çıktı

```
Merhaba
```

Yorum türü	Sözdizimi	Kullanım
Tek satır	`// ...`	Kısa açıklamalar
Çok satır	`/* ... */`	Uzun açıklamalar, kod parçasını geçici olarak devre dışı bırakma
Javadoc	`/** ... */`	Sınıf ve metot belgeleri (otomatik dokümantasyon üretilir)

> **İpucu**
>
> VS Code'da bir veya birkaç satırı seçip **Ctrl+/** (macOS'ta **Cmd+/**) tuşlarına basarsan satırlar yoruma alınır; tekrar basarsan yorumdan çıkar.

1.8 Hata türleri

Program yazarken üç farklı türde hata ile karşılaşırsın. Hangisi olduğunu anlamak, hatayı çözmenin yarısıdır.

1. Derleme hatası (compile-time error)

Kod Java'nın **yazım kurallarına** (sözdizimi, *syntax*) uymuyorsa `javac` programı derlemez, `.class` dosyası oluşmaz. VS Code bu hataların altını **kırmızı** çizer. En klasik örnek, unutulan noktalı virgüldür:

NoktaliVirgul.java

```java
public class NoktaliVirgul {
    public static void main(String[] args) {
        System.out.println("Merhaba")
    }
}
```

Derleyici çıktısı – program derlenmez

```
NoktaliVirgul.java:3: error: ';' expected
        System.out.println("Merhaba")
                                    ^
1 error
```

Derleyici hatanın **dosya adını, satır numarasını** ve ne beklediğini (`';' expected`) söyler. Hata mesajlarını okumayı alışkanlık haline getir; Ek C'de en sık görülen mesajların Türkçe açıklamalarını bulabilirsin.

2. Çalışma zamanı hatası (runtime error)

Kod kurallara uygundur ve derlenir; ama **çalışırken** yapılamayacak bir işlemle karşılaşılır ve program çöker. Java'da bunlara **exception** (istisna) denir:

SifiraBolme.java

```java
public class SifiraBolme {
    public static void main(String[] args) {
        System.out.println("Hesap başlıyor...");
        System.out.println(10 / 0);
        System.out.println("Bu satır hiç çalışmaz.");
    }
}
```

Çıktı – program hata ile durur

```
Hesap başlıyor...
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at SifiraBolme.main(SifiraBolme.java:4)
```

İlk satır yazdırıldı, ama tam sayıyı sıfıra bölmeye çalışınca `ArithmeticException` oluştu ve program orada durdu. Mesajın son satırı hatanın **hangi satırda** olduğunu gösterir: `SifiraBolme.java:4`. Exception'ları nasıl yakalayacağımızı Bölüm 12'de öğreneceğiz.

3. Mantık hatası (logic error)

En sinsi hata budur: Program derlenir, çalışır, çökmez; ama **yanlış sonuç** üretir. Bilgisayar sana bir şey söylemez, hatayı sen bulmak zorundasın.

Ortalama.java

```java
public class Ortalama {
    public static void main(String[] args) {
        // İki notun ortalamasını hesaplamak istiyoruz: (70 + 90) / 2 = 80
        System.out.println(70 + 90 / 2);
        System.out.println((70 + 90) / 2);
    }
}
```

Çıktı
```
115
80
```

İlk satır 80 yerine 115 yazdırdı, çünkü bölme toplamadan **önce** yapılır: `70 + 45 = 115`. Doğru sonuç için parantez gerekir.

Hata türü	Ne zaman ortaya çıkar?	Program çalışır mı?	Örnek
Derleme hatası	`javac` aşamasında	Hayır, `.class` oluşmaz	Eksik `;`, yanlış yazılmış `System`
Çalışma zamanı hatası	Program çalışırken	Başlar ama yarıda çöker	Sıfıra bölme
Mantık hatası	Sonuçlara bakınca	Evet, ama yanlış sonuç verir	Unutulan parantez

Sınavda Çıkar

"Aşağıdaki hata hangi türdendir?" sorusu çok sık çıkar. İpucu: Eksik `;`, kapanmayan `}`, tanımsız değişken → **derleme**. Sıfıra bölme, dizinin dışına çıkma → **çalışma zamanı**. Yanlış formül → **mantık**.

1.9 Java 25 ile gelen kısa yazım

Java 25 Notu: Kompakt kaynak dosyası ve kısa main

Java 25 ile (JEP 512) yeni başlayanlar için daha kısa bir yazım resmîleşti: **kompakt kaynak dosyası**. Sınıf tanımı, `public static` ve `String[] args` gerekmeden şöyle yazılabilir:

```
void main() {
    IO.println("Merhaba Dünya!");
}
```

Çıktı
```
Merhaba Dünya!
```

	Klasik yazım (kitap ve sınav)	Java 25 kısa yazım
Sınıf	`public class X { ... }`	Yazılmaz (derleyici gizli sınıfı kendisi üretir)
Başlangıç metodu	`public static void main(String[] args)`	`void main()`
Ekrana yazma	`System.out.println(...)`	`IO.println(...)`, `IO.print(...)`
Klavyeden okuma	`Scanner`	`IO.readln(...)` (Bölüm 3)
Çalıştırma	`javac X.java` ve `java X`	`java X.java`

`IO` sınıfı hazırdır, `import` istemez; `IO.print` alt satıra geçmez, `IO.println` geçer. Kompakt dosyada `List`, `Scanner`, `Random` gibi `java.base` sınıfları `import` yazmadan da kullanılabilir. Tablodaki son satır ayrıca önemlidir: `java` komutuna `.class` yerine doğrudan `.java` dosyasını verirsen JDK dosyayı bellekte derleyip çalıştırır; `.class` dosyası oluşmaz:

```
java KisaMain.java
```

> Bu tek komut klasik yazılmış dosyalarda da çalışır (Java 11'den beri). Hızlı denemeler için harikadır. Ama **üniversite derslerinde, sınavlarda ve gerçek projelerde klasik yazım kullanılır**, bu kitap da klasik yazımla ilerleyecek; kısa yazımı sınavda kullanmadan önce mutlaka hocana sor.

Bölüm Özeti – Bilmen Gerekenler

- Java kaynak kodu (`.java`) `javac` ile **bytecode**'a (`.class`) derlenir, bytecode'u **JVM** çalıştırır. Platform bağımsızlığı buradan gelir.
- **JDK** = JRE + derleyici ve araçlar; **JRE** = JVM + kütüphaneler. Geliştirici için JDK gerekir.
- Her program bir **sınıf** içinde yazılır; program `public static void main(String[] args)` metodundan başlar.
- `public class X` varsa dosya adı `X.java` olmalıdır. Java **büyük/küçük harf duyarlıdır**.
- Her komut `;` ile biter; her `{` bir `}` ile kapanır.
- `println` yazıp alt satıra geçer, `print` aynı satırda kalır. `\n` yeni satır, `\t` sekme, `\"` tırnak, `\\` ters bölü.
- `+` String ile kullanılınca birleştirir; işlem **soldan sağa** yapılır: `1 + 2 + "a"` → `"3a"`, `"a" + 1 + 2` → `"a12"`.
- Hata türleri: **derleme** (kural ihlali), **çalışma zamanı** (çökme, exception), **mantık** (yanlış sonuç).
- Java 25'te `void main()` ve `IO.println` ile sınıfsız kısa yazım vardır; `java Dosya.java` tek komutla derleyip çalıştırır. Kitap ve sınavlar **klasik** yazımı bekler.

Sorular

A) Çoktan Seçmeli

1 Java kaynak kodu `javac` ile derlendiğinde hangi dosya türü oluşur?

(A) `.exe` dosyası (makine kodu)
(B) `.java` dosyası
(C) `.class` dosyası (bytecode)
(D) `.jvm` dosyası
(E) `.txt` dosyası

2 Aşağıdakilerden hangisi Java'nın "platform bağımsız" olmasının asıl nedenidir?

(A) Java'nın ücretsiz olması
(B) Bytecode'un her platformda bulunan JVM tarafından çalıştırılması
(C) Java programlarının sadece Windows'ta çalışması
(D) Java'nın yorum satırlarını desteklemesi
(E) Java derleyicisinin doğrudan makine kodu üretmesi

3 `public class Hesap` ile başlayan bir program hangi dosya adıyla kaydedilmelidir?

(A) `Hesap.java` (B) `Hesap.class` (C) `hesap.java` (D) `HESAP.java` (E) `Main.java`

4 Aşağıdaki hatalardan hangisi bir **çalışma zamanı** (runtime) hatasıdır?

- (A) Komutun sonundaki `;` işaretinin unutulması
- (B) `System` kelimesinin `system` diye yazılması
- (C) Kapanmayan bir süslü parantez `{`
- (D) Bir tam sayının sıfıra bölünmesi
- (E) Ortalama hesaplanırken parantezin unutulması

5 `System.out.println("A" + 2 + 3);` komutu ekrana ne yazdırır?

- (A) `A5`
- (B) `5A`
- (C) `A 2 3`
- (D) Derleme hatası verir
- (E) `A23`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru6 {
    public static void main(String[] args) {
        System.out.print("Ja");
        System.out.println("va");
        System.out.print("2");
        System.out.print("5\n");
        System.out.println("L\tT\tS");
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru7 {
    public static void main(String[] args) {
        System.out.println(3 + 4 + "7");
        System.out.println("7" + 3 + 4);
        System.out.println("7" + (3 + 4));
        System.out.println(3 + 4 * 2);
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru8 {
    public static void main(String[] args) {
        // System.out.println("Bir");
        System.out.println("İki"); /* System.out.println("Üç"); */
        System.out.println("\"Dört\"");
        System.out.println("Beş\\Altı");
    }
}
```

C) Kod Yazma

9 Ekrana aşağıdaki **kimlik kartını** yazdıran `Kimlik` adlı programı yaz. Her satır için ayrı bir `println` kullan.

```
************************
* Ad    : Elif Demir    *
* Bölüm : Bilgisayar M. *
* Sınıf : 1             *
************************
```

10 **Tek bir** `System.out.println` komutu ve kaçış dizileri kullanarak aşağıdaki çıktıyı üreten `TekSatir` programını yaz:

```
Ders    Kredi
"Java"  6
C:\ders\java
```

(İlk iki satırda sütunlar arasında **tab** karakteri vardır.)

Çözümler

Cevap anahtarı

1 C	2 B	3 A	4 D	5 E

1 Cevap: C

`javac`, `.java` uzantılı kaynak kodu derleyip **bytecode** içeren `.class` dosyasını üretir. Java derleyicisi `.exe` gibi doğrudan makine kodu üretmez.

2 Cevap: B

Platform bağımsızlığın anahtarı bytecode + JVM ikilisidir. Aynı `.class` dosyası, JVM'i olan her işletim sisteminde çalışır. (E) şıkkı yanlıştır; Java derleyicisi makine kodu değil bytecode üretir.

3 Cevap: A

`public` bir sınıfın adı ile dosya adı **harfi harfine** aynı olmalıdır ve uzantı `.java`'dır. Java büyük/küçük harf duyarlı olduğu için `hesap.java` ve `HESAP.java` yanlıştır.

4 Cevap: D

Sıfıra bölme kod derlendikten sonra, **program çalışırken** ortaya çıkar (`ArithmeticException`). A, B, C derleme hatası; E mantık hatasıdır.

5 Cevap: E

İşlem soldan sağa yapılır: `"A" + 2` → `"A2"`, sonra `"A2" + 3` → `"A23"`. String ile başlayan bir ifadede sonraki `+` işlemleri birleştirme yapar.

6

`print("Ja")` aynı satırda kalır, `println("va")` yazıp alt satıra geçer. `"2"` ve `"5\n"` yan yana yazılır, `\n` alt satıra geçirir. Son satırda harfler arasında tab boşluğu vardır.

Çıktı

```
Java
25
L	T	S
```

7

1. satır: `3 + 4 = 7`, sonra `7 + "7"` → `"77"`.
2. satır: `"7" + 3` → `"73"`, sonra `"734"`.
3. satır: Parantez önce: `3 + 4 = 7` → `"77"`.
4. satır: Çarpma önce: `4 * 2 = 8`, `3 + 8 = 11`.

Çıktı
```
77
734
77
11
```

8

`//` ile başlayan satır ve `/* ... */` arasındaki kısım yorumdur, çalışmaz. `\"` tırnak karakterini, `\\` tek bir ters bölüyü yazdırır.

Çıktı
```
İki
"Dört"
Beş\Altı
```

9

Her satırı ayrı bir String olarak yazdırmak yeterli. Kenarların hizalı olması için boşlukları dikkatle say.

Kimlik.java
```java
public class Kimlik {
    public static void main(String[] args) {
        System.out.println("************************");
        System.out.println("* Ad    : Elif Demir    *");
        System.out.println("* Bölüm : Bilgisayar M. *");
        System.out.println("* Sınıf : 1             *");
        System.out.println("************************");
    }
}
```

Çıktı
```
************************
* Ad    : Elif Demir    *
* Bölüm : Bilgisayar M. *
* Sınıf : 1             *
************************
```

10

Satır sonları için `\n`, tab için `\t`, tırnak için `\"`, ters bölü için `\\` kullanılır.

TekSatir.java
```java
public class TekSatir {
    public static void main(String[] args) {
        System.out.println("Ders\tKredi\n\"Java\"\t6\nC:\\ders\\java");
    }
}
```

Çıktı
```
Ders    Kredi
"Java"  6
C:\ders\java
```

BÖLÜM 2

Değişkenler, Veri Tipleri ve Operatörler

Bu Bölümde Öğreneceklerin

- Değişken tanımlamayı, değer atamayı ve isimlendirme kurallarını
- Java'nın 8 ilkel (primitive) veri tipini ve hangisinin ne zaman kullanılacağını
- `final` ile sabit, `var` ile tip çıkarımı yapmayı
- Aritmetik operatörleri, **tam sayı bölmesini** ve `%` (mod) operatörünü
- Otomatik ve zorunlu tip dönüşümünü (casting)
- `x++` ile `++x` arasındaki farkı (sınavların bir numaralı tuzağı)
- `Math` sınıfını, taşmayı (overflow) ve ondalık sayı hassasiyetini

2.1 Değişken nedir?

Değişken (variable), bellekte bir değeri saklamak için ayrılmış, **adı** ve **tipi** olan bir kutudur. Kutunun tipi, içine ne tür bir değer konabileceğini belirler: tam sayı, ondalıklı sayı, karakter...

Java'da bir değişkeni kullanmadan önce **tanımlaman** (declare) gerekir:

```
tip  ad  =  değer;
int  yas =  19;
```

DegiskenOrnek.java

```
public class DegiskenOrnek {
    public static void main(String[] args) {
        int yas = 19;              // tanımlama + ilk değer verme
        double boy;                // sadece tanımlama
        boy = 1.78;                // sonradan atama
        String ad = "Can";

        System.out.println(ad + " " + yas + " yaşında, boyu " + boy);

        yas = 20;                  // değeri değişti, eski değer kayboldu
        yas = yas + 1;             // sağ taraf önce hesaplanır: 20 + 1
        System.out.println("Yeni yaş: " + yas);
    }
}
```

Çıktı

```
Can 19 yaşında, boyu 1.78
Yeni yaş: 21
```

`yas = yas + 1;` satırı matematikte saçma görünür ama programlamada `=` **eşittir değil, atamadır** (assignment): "Sağdaki ifadeyi hesapla, sonucu soldaki değişkene koy." Önce `yas + 1` yani `21` hesaplanır, sonra `yas` değişkenine yazılır.

Sık Yapılan Hata: İlk değer verilmemiş değişken

Bir metot içinde tanımlanan değişkene (yerel değişken, *local variable*) değer vermeden onu okuyamazsın. Java bu durumda programı derlemez:

```
public class IlkDegerYok {
    public static void main(String[] args) {
        int sayi;
        System.out.println(sayi);
    }
}
```

Derleyici çıktısı – program derlenmez

```
IlkDegerYok.java:4: error: variable sayi might not have been initialized
        System.out.println(sayi);
                           ^
1 error
```

Aynı blok içinde aynı isimle ikinci bir değişken **tanımlayamazsın** (`int x = 5; int x = 6;` derleme hatası). Ama mevcut değişkene istediğin kadar yeni değer **atayabilirsin** (`x = 6;`).

2.2 İsimlendirme kuralları

Değişken, metot ve sınıf adlarına **tanımlayıcı** (identifier) denir. Kurallar:

1. Harf, rakam, alt çizgi `_` ve dolar `$` içerebilir.
2. **Rakamla başlayamaz:** `2sayi` geçersiz, `sayi2` geçerli.
3. Boşluk ve `-`, `+`, `.` gibi özel karakterler içeremez: `ogrenci no`, `ogrenci-no` geçersiz.
4. Java'nın **anahtar kelimeleri** (keyword) kullanılamaz: `int`, `class`, `public`, `if`, `for`, `new`…
5. Büyük/küçük harf duyarlıdır: `sayi`, `Sayi` ve `SAYI` üç farklı isimdir.

Kurallara ek olarak, herkesin uyduğu **isimlendirme gelenekleri** (naming conventions) vardır. Uymazsan program çalışır ama kodun "acemi" görünür:

Ne için?	Gelenek	Örnek
Değişken ve metot	**camelCase**: küçük harfle başla, sonraki kelimeler büyük	`ogrenciSayisi`, `ortalamaHesapla`
Sınıf	**PascalCase**: her kelime büyük harfle	`Ogrenci`, `BankaHesabi`
Sabit (`final`)	**BÜYÜK_HARF** ve alt çizgi	`PI`, `MAX_OGRENCI`

İpucu

Java, değişken adlarında `ö`, `ş`, `ğ` gibi Türkçe karakterlere izin verir; ama farklı bilgisayarlarda karakter kodlaması sorunu çıkabileceği için **kullanmaman önerilir**. `ogrenciSayisi` yaz, `öğrenciSayısı` yazma.

> **Sınavda Çıkar**
>
> "Aşağıdakilerden hangisi geçerli bir değişken adıdır?" sorusunda sırayla kontrol et: Rakamla başlıyor mu? Boşluk veya `-` var mı? Anahtar kelime mi (`class`, `int`, `new`...)? `_toplam`, `$fiyat`, `sayi2` **geçerlidir**; `2sayi`, `ogrenci no`, `class` **geçersizdir**.

2.3 İlkel veri tipleri

Java'da 8 tane **ilkel veri tipi** (primitive type) vardır. Bunlar dilin en temel yapı taşlarıdır:

Tip	Boyut	Değer aralığı / örnek	Ne için?
`byte`	1 bayt	−128 … 127	Çok küçük tam sayılar
`short`	2 bayt	−32 768 … 32 767	Küçük tam sayılar (nadiren kullanılır)
`int`	4 bayt	yaklaşık ±2,1 milyar	**Varsayılan tam sayı tipi**
`long`	8 bayt	yaklaşık $\pm 9{,}2 \times 10^{18}$	Çok büyük tam sayılar: `8000000000L`
`float`	4 bayt	~7 basamak hassasiyet	Ondalıklı sayı: `3.14f`
`double`	8 bayt	~15 basamak hassasiyet	**Varsayılan ondalıklı tip**
`char`	2 bayt	Tek karakter: `'A'`, `'7'`, `'ş'`	Karakter (Unicode)
`boolean`	—	Sadece `true` veya `false`	Doğru/yanlış bilgisi

Pratikte en çok kullanacakların: tam sayı için **`int`**, ondalıklı sayı için **`double`**, karakter için **`char`**, doğru/yanlış için **`boolean`**.

VeriTipleri.java

```
public class VeriTipleri {
    public static void main(String[] args) {
        byte kat = 12;
        short yil = 2026;
        int ogrenciSayisi = 1450;
        long dunyaNufusu = 8_100_000_000L;   // _ okunabilirlik için
        float sicaklik = 23.5f;
        double pi = 3.141592653589793;
        char harf = 'J';
        boolean gectiMi = true;

        System.out.println(kat + " " + yil + " " + ogrenciSayisi);
        System.out.println(dunyaNufusu);
        System.out.println(sicaklik + " " + pi);
        System.out.println(harf + " " + gectiMi);
    }
}
```

Çıktı

```
12 2026 1450
8100000000
23.5 3.141592653589793
J true
```

Sayıların içine yazılan alt çizgi `_` sadece okunabilirlik içindir; `8_100_000_000` ile `8100000000` aynı sayıdır.

Sabit değerlerin (literal) tipi

Koda doğrudan yazdığın değerlere **literal** denir ve onların da bir tipi vardır:

- Ondalık noktası olmayan sayılar (`42`) → **`int`**
- Ondalık noktası olan sayılar (`3.14`) → **`double`**
- Sonuna `L` eklenmiş tam sayılar (`42L`) → `long`
- Sonuna `f` eklenmiş ondalıklılar (`3.14f`) → `float`
- Tek tırnak (`'A'`) → `char`, çift tırnak (`"A"`) → `String`

Bu yüzden şu iki program derlenmez:

```
public class LongHata {
    public static void main(String[] args) {
        long nufus = 8100000000;
    }
}
```

Derleyici çıktısı – program derlenmez

```
LongHata.java:3: error: integer number too large
        long nufus = 8100000000;
                     ^
1 error
```

```
public class FloatHata {
    public static void main(String[] args) {
        float oran = 0.75;
    }
}
```

Derleyici çıktısı – program derlenmez

```
FloatHata.java:3: error: incompatible types: possible lossy conversion from double to float
        float oran = 0.75;
                     ^
1 error
```

`8100000000` bir `int` literal'idir ama `int`'in sınırını (~2,1 milyar) aşar; sonuna `L` eklemek gerekir. `0.75` ise `double`'dır; `float` değişkene koymak için `0.75f` yazılmalıdır.

> **Sık Yapılan Hata: `char` ve `String` karıştırma**
>
> `char c = "A";` derlenmez; `char` tek tırnak ister: `char c = 'A';`. Tersine `String s = 'Merhaba';` da derlenmez. Kural: **tek karakter → tek tırnak**, **metin → çift tırnak**.

> **Not: String ilkel tip değildir**
>
> `String` 8 ilkel tipten biri **değildir**; bir **sınıftır** (bu yüzden büyük harfle başlar). Şimdilik ilkel tipler gibi kullanacağız; String'in ayrıntılarını Bölüm 8'de, sınıfların ne olduğunu Bölüm 9'da göreceksin.

2.4 Sabitler (final) ve var

Değeri program boyunca **hiç değişmeyecek** bir değişken için `final` kullanılır. `final` bir değişkene ikinci kez değer atamaya çalışırsan derleme hatası alırsın:

SabitOrnek.java

```java
public class SabitOrnek {
    public static void main(String[] args) {
        final double PI = 3.14159;
        final int HAFTALIK_DERS = 4;
        double yaricap = 2.0;

        System.out.println("Alan: " + PI * yaricap * yaricap);
        System.out.println("Dönemlik ders: " + HAFTALIK_DERS * 14);
    }
}
```

Çıktı

```
Alan: 12.56636
Dönemlik ders: 56
```

```java
public class FinalHata {
    public static void main(String[] args) {
        final int MAX = 100;
        MAX = 200;
    }
}
```

Derleyici çıktısı – program derlenmez

```
FinalHata.java:4: error: cannot assign a value to final variable MAX
        MAX = 200;
        ^
1 error
```

`var` anahtar kelimesi (Java 10+) ile tipi yazmadan değişken tanımlayabilirsin; derleyici tipi sağdaki değerden **çıkarır** (type inference):

VarOrnek.java

```java
public class VarOrnek {
    public static void main(String[] args) {
        var sayi = 10;        // int
        var oran = 2.5;       // double
        var mesaj = "Selam";  // String
        System.out.println(sayi * oran + " " + mesaj);
    }
}
```

Çıktı

```
25.0 Selam
```

`var` ile tanımlanan değişkenin tipi yine sabittir: `var sayi = 10;` dedikten sonra `sayi = "on";` yazamazsın. Ayrıca `var x;` gibi değersiz tanımlama yapılamaz. Yeni başlayan biri olarak tipleri **açıkça yazmanı** öneririz; tipleri düşünmek, öğrenmenin parçası.

2.5 Aritmetik operatörler

Operatör	İşlem	Örnek	Sonuç
`+`	Toplama	`7 + 2`	`9`
`-`	Çıkarma	`7 - 2`	`5`
`*`	Çarpma	`7 * 2`	`14`
`/`	Bölme	`7 / 2`	`3` (!)
`%`	Kalan (mod)	`7 % 2`	`1`

Tam sayı bölmesi

Bu bölümün **en önemli kuralı**: İki tam sayı birbirine bölünürse sonuç da **tam sayıdır**; ondalık kısım **atılır** (yuvarlanmaz, kesilir). Sonucun ondalıklı olması için işlenenlerden **en az birinin** ondalıklı olması gerekir:

Bolme.java

```java
public class Bolme {
    public static void main(String[] args) {
        System.out.println(7 / 2);
        System.out.println(7.0 / 2);
        System.out.println(7 / 2.0);
        System.out.println(9 / 10);
        System.out.println(-7 / 2);
        System.out.println(7 % 2);
        System.out.println(10 % 3);
        System.out.println(3 % 10);
        System.out.println(-7 % 3);
    }
}
```

Çıktı

```
3
3.5
3.5
0
-3
1
1
3
-1
```

- `9 / 10` → `0.9` değil, **`0`**. Ondalık kısım kesildi.
- `-7 / 2` → `-3.5`'in ondalığı kesilir: **`-3`** (sıfıra doğru kesme; `-4` değil).
- `%` operatörü bölmeden kalanı verir. `3 % 10` → `3`, çünkü 3, 10'a 0 kere bölünür ve 3 kalır.
- Negatif sayıda kalanın işareti **soldaki sayının** işaretiyle aynıdır: `-7 % 3` → `-1`.

> **Sınavda Çıkar: `%` operatörünün klasik kullanımları**
>
> - **Tek/çift kontrolü:** `sayi % 2 == 0` ise çift.
> - **Bölünebilme:** `sayi % 5 == 0` ise 5'e tam bölünür.
> - **Son basamak:** `sayi % 10` → `1234 % 10 = 4`.
> - **Son basamağı atma:** `sayi / 10` → `1234 / 10 = 123`.
>
> Son iki madde, "bir sayının basamaklarını topla / ters çevir" sorularının temelidir (Bölüm 5).

İşlem önceliği

Matematikteki gibi çarpma, bölme ve mod (`*`, `/`, `%`) toplama ve çıkarmadan (`+`, `-`) **önce** yapılır. Aynı öncelikteki işlemler **soldan sağa** yapılır. Parantez her şeyden önce gelir.

Oncelik.java

```
public class Oncelik {
    public static void main(String[] args) {
        System.out.println(2 + 3 * 4);
        System.out.println((2 + 3) * 4);
        System.out.println(10 - 4 - 3);
        System.out.println(20 / 4 * 2);
        System.out.println(2 + 10 % 4);
        System.out.println(17 % 5 * 2 + 1);
    }
}
```

Çıktı

```
14
20
3
10
4
5
```

`20 / 4 * 2` ifadesinde `/` ve `*` aynı öncelikte olduğu için soldan sağa gidilir: `(20 / 4) * 2 = 10`. `20 / 8` değil!

2.6 Tip dönüşümü (casting)

Otomatik (genişleyen) dönüşüm

Küçük bir tipi değer aralığı **daha geniş** bir tipe koyarken Java dönüşümü kendisi yapar (*widening*):

`byte → short → int → long → float → double`

`char → int → long → float → double`

```
int a = 5;
double d = a;      // d = 5.0  (otomatik)
```

Bir işlemde farklı tipler varsa, küçük olan büyüğe dönüştürülür: `int + double` → `double`. Tam sayı bölmesi kuralının nedeni de budur: `7 / 2.0` işleminde `7`, `7.0`'a dönüştürülür.

Not: Otomatik dönüşüm her zaman kayıpsız değildir

`int → float`, `long → float` ve `long → double` dönüşümleri de otomatiktir, ama `float` yaklaşık 7, `double` yaklaşık 15–16 anlamlı basamak tutabildiği için büyük sayıların son basamakları kaybolabilir. Hedef tipin **aralığı** daha geniştir, **duyarlılığı** değil:

```
public class GenislemeKaybi {
    public static void main(String[] args) {
        int a = 123456789;
        float f = a;                    // otomatik, cast yok
        System.out.println((long) f);
    }
}
```

Çıktı

```
123456792
```

Zorunlu (daralan) dönüşüm

Büyük bir tipi **daha küçük** bir tipe koymak veri kaybına yol açabileceği için Java bunu kendiliğinden yapmaz; senin açıkça istemen gerekir. Buna **casting** denir ve parantez içinde hedef tip yazılır:

Donusum.java

```
public class Donusum {
    public static void main(String[] args) {
        double d = 3.99;
        int i = (int) d;                      // ondalık kısım KESİLİR
        System.out.println(i);
        System.out.println((int) -3.99);
        System.out.println(Math.round(3.99));  // yuvarlama böyle yapılır

        int toplam = 7, adet = 2;
        System.out.println(toplam / adet);
        System.out.println((double) toplam / adet);
        System.out.println((double) (toplam / adet));
    }
}
```

Çıktı

```
3
-3
4
3
3.5
3.0
```

Son iki satır çok kritik:

- `(double) toplam / adet`: Cast işlemi bölmeden **önce** yapılır. `toplam` → `7.0` olur, `7.0 / 2 = 3.5`.
- `(double) (toplam / adet)`: Önce parantez içi: `7 / 2 = 3` (tam sayı bölmesi!), sonra `3` → `3.0`. Ondalık kısım çoktan kaybolmuştur.

● Sık Yapılan Hata: Ortalama hesabında tam sayı bölmesi

Notları `int` olan bir öğrencinin ortalamasını `int toplam = 85 + 90 + 72; double ort = toplam / 3;` diye hesaplarsan sonuç `82.33` değil **`82.0`** olur. Bölme `int / int` olarak yapılıp ondalık atıldıktan sonra `double`'a çevrilir. Doğrusu: `toplam / 3.0` veya `(double) toplam / 3`.

Daralan dönüşüm cast olmadan yazılırsa derlenmez:

```
public class DaralmaHata {
    public static void main(String[] args) {
        int x = 3.5;
    }
}
```

Derleyici çıktısı – program derlenmez

```
DaralmaHata.java:3: error: incompatible types: possible lossy conversion from double to int
        int x = 3.5;
                ^
1 error
```

char ve sayılar

`char` aslında bir sayıdır: Her karakterin bir **Unicode** (ve ilk 128 karakter için **ASCII**) numarası vardır. `'A'` = 65, `'a'` = 97, `'0'` = 48. Bu yüzden karakterlerle aritmetik yapılabilir:

CharSayi.java

```
public class CharSayi {
    public static void main(String[] args) {
        char c = 'A';
        System.out.println(c + 1);           // char + int → int
        System.out.println((char) (c + 1));  // tekrar karaktere çevir
        System.out.println((int) 'a');
        c++;                                 // c artık 'B'
        System.out.println(c);
        System.out.println('7' - '0');       // rakam karakterini sayıya çevirme
    }
}
```

Çıktı

```
66
B
97
B
7
```

● İpucu: Bilmen gereken üç ASCII değeri

`'A'` = **65**, `'a'` = **97**, `'0'` = **48**. Gerisini bunlardan hesaplarsın: `'C'` = 67, `'z'` = 122. Büyük-küçük harf farkı her zaman **32**'dir.

2.7 Atama ve bileşik atama operatörleri

Bir değişkenin değerini kendisine bağlı olarak değiştirmek çok yaygın olduğu için kısa yazımlar vardır:

Kısa yazım	Uzun hali
`x += 5`	`x = x + 5`
`x -= 5`	`x = x - 5`
`x *= 5`	`x = x * 5`
`x /= 5`	`x = x / 5`
`x %= 5`	`x = x % 5`

BilesikAtama.java

```java
public class BilesikAtama {
    public static void main(String[] args) {
        int x = 10;
        x += 5;     // 15
        x *= 2;     // 30
        x -= 4;     // 26
        x /= 4;     // 6  (tam sayı bölmesi)
        x %= 4;     // 2
        System.out.println(x);

        double d = 10;
        d /= 4;
        System.out.println(d);
    }
}
```

Çıktı

```
2
2.5
```

> **Sınavda Çıkar: Bileşik atamanın gizli cast'i**
>
> `byte b = 10; b = b + 5;` **derlenmez**, çünkü `b + 5` işleminin sonucu `int`'tir ve `int` `byte`'a otomatik dönüşmez. Ama `b += 5;` **derlenir**! Bileşik atama operatörleri, sonucu otomatik olarak sol taraftaki tipe cast eder. Yani `b += 5` aslında `b = (byte) (b + 5)` demektir.

2.8 Artırma ve azaltma: x++ ve ++x

Bir değişkeni 1 artırmak (`x = x + 1`) için `x++` veya `++x`, 1 azaltmak için `x--` veya `--x` yazılır. Tek başına bir satırda kullanıldıklarında **ikisi arasında fark yoktur**. Fark, bir **ifadenin içinde** kullanıldıklarında ortaya çıkar:

- **Son ek** (postfix) `x++`: Önce `x`'in **mevcut değerini kullan**, sonra artır.
- **Ön ek** (prefix) `++x`: Önce **artır**, sonra yeni değeri kullan.

Artirma.java

```java
public class Artirma {
    public static void main(String[] args) {
        int a = 5;
        int b = a++;     // b = 5 (eski değer), sonra a = 6
        System.out.println("a=" + a + " b=" + b);

        int c = 5;
        int d = ++c;     // önce c = 6, sonra d = 6
        System.out.println("c=" + c + " d=" + d);

        int x = 3;
        System.out.println(x++);
        System.out.println(x);
        System.out.println(++x);

        int y = 10;
        int z = y-- - --y;   // 10 - 8
        System.out.println("y=" + y + " z=" + z);
    }
}
```

Çıktı

```
a=6 b=5
c=6 d=6
3
4
5
y=8 z=2
```

Son örneği adım adım çözelim: `y-- - --y`

1. `y--`: Değer olarak **10** kullanılır, sonra `y` = 9 olur.
2. `--y`: Önce `y` = 8 olur, değer olarak **8** kullanılır.
3. `z = 10 - 8 = 2`, `y`'nin son değeri 8.

Sınavda Çıkar

`++`/`--` soruları her sınavda vardır. Çözerken bir **değişken tablosu** tut ve her `++`'dan sonra değeri güncelle. Altın kural: **"Artı işareti değişkenin neresindeyse, artırma o zaman olur."** Önde (`++x`) → önce artır. Arkada (`x++`) → kullandıktan sonra artır.

Sık Yapılan Hata

`x = x++;` satırı `x`'i **artırmaz**! Önce eski değer (diyelim 5) bir kenara alınır, `x` 6 olur, sonra eski değer (5) `x`'e geri atanır. Sonuç: `x` hâlâ 5. Bu satırı gerçek kodda asla yazma; ama sınavda görürsen tuzağı tanı.

2.9 Math sınıfı

Java'nın hazır `Math` sınıfı sık kullanılan matematik işlemlerini sunar. `import` gerekmez:

Metot	Ne yapar?	Örnek → Sonuç
`Math.pow(a, b)`	a^b (üs alma), sonuç `double`	`Math.pow(2, 10)` → `1024.0`
`Math.sqrt(x)`	Karekök, sonuç `double`	`Math.sqrt(81)` → `9.0`
`Math.abs(x)`	Mutlak değer	`Math.abs(-7)` → `7`
`Math.max(a, b)`	Büyük olan	`Math.max(3, 8)` → `8`
`Math.min(a, b)`	Küçük olan	`Math.min(3, 8)` → `3`
`Math.round(x)`	En yakın tam sayıya yuvarla (`double` için `long`, `float` için `int` döner)	`Math.round(2.5)` → `3`
`Math.ceil(x)`	Yukarı yuvarla, `double` döner	`Math.ceil(2.1)` → `3.0`
`Math.floor(x)`	Aşağı yuvarla, `double` döner	`Math.floor(2.9)` → `2.0`
`Math.random()`	0.0 (dahil) ile 1.0 (hariç) arası rastgele `double`	`0.7310...`
`Math.PI`	π sabiti	`3.141592653589793`

MathOrnek.java

```java
public class MathOrnek {
    public static void main(String[] args) {
        System.out.println(Math.pow(2, 10));
        System.out.println(Math.sqrt(2));
        System.out.println(Math.abs(-4.5));
        System.out.println(Math.max(10, Math.min(25, 17)));
        System.out.println(Math.round(4.5) + " " + Math.round(-4.5));
        System.out.println(Math.ceil(-2.5) + " " + Math.floor(-2.5));

        // Hipotenüs: a = 3, b = 4
        double c = Math.sqrt(Math.pow(3, 2) + Math.pow(4, 2));
        System.out.println("Hipotenüs: " + c);
    }
}
```

Çıktı

```
1024.0
1.4142135623730951
4.5
17
5 -4
-2.0 -3.0
Hipotenüs: 5.0
```

Dikkat: `Math.round(-4.5)` sonucu `-4`'tür, çünkü `Math.round` yarımları **yukarı** (pozitif sonsuza doğru) yuvarlar.

İpucu: Belirli aralıkta rastgele tam sayı

1 ile 6 arasında (zar) rastgele bir tam sayı: `int zar = (int) (Math.random() * 6) + 1;` Genel formül: `(int) (Math.random() * (ust - alt + 1)) + alt`. `Math.random() * 6` → 0.0 ile 5.99… arası; `(int)` ile 0–5; `+ 1` ile 1–6.

2.10 Taşma ve ondalık hassasiyet

Her tipin bir sınırı vardır. `int`'in en büyük değerine 1 eklersen, sayı bir saatin 12'den 1'e dönmesi gibi **en küçük değere döner**. Buna **taşma** (overflow) denir ve Java bu durumda hata **vermez**:

Tasma.java

```
public class Tasma {
    public static void main(String[] args) {
        int enBuyuk = Integer.MAX_VALUE;
        System.out.println(enBuyuk);
        System.out.println(enBuyuk + 1);

        int saniye = 100_000 * 100_000;          // taşar!
        long dogru = 100_000L * 100_000;
        System.out.println(saniye + " / " + dogru);
    }
}
```

Çıktı

```
2147483647
-2147483648
1410065408 / 10000000000
```

`Integer.MAX_VALUE`, `int`'in en büyük değerini veren hazır bir sabittir (`Integer.MIN_VALUE` en küçüğü). `Integer` sınıfını Bölüm 13'te tanıyacaksın; şimdilik bu iki sabiti ezberlemen yeterli.

`100_000 * 100_000` işleminde iki `int` çarpılır; sonuç `int` sınırını aşar ve anlamsız bir sayı çıkar. Sonucu `long` bir değişkene koymak yetmez, çarpmanın kendisinin `long` olması için bir tarafa `L` ekleyerek işleme `long` olarak başlamak gerekir.

Ondalıklı sayılar da bilgisayarda ikilik (binary) sistemde tutulduğu için bazı değerler **tam olarak** saklanamaz:

Hassasiyet.java

```
public class Hassasiyet {
    public static void main(String[] args) {
        System.out.println(0.1 + 0.2);
        System.out.println(0.1 + 0.2 == 0.3);
        System.out.println(1.0 - 0.9);
    }
}
```

Çıktı

```
0.30000000000000004
false
0.09999999999999998
```

> **Sık Yapılan Hata: Ondalıklı sayıları == ile karşılaştırma**
>
> `0.1 + 0.2 == 0.3` sonucu `false`'tur! İki `double`'ı karşılaştırırken aralarındaki farkın çok küçük olup olmadığına bak: `Math.abs(a - b) < 1e-9`. Para gibi kesin hesap gereken yerlerde profesyoneller `BigDecimal` sınıfını kullanır.

Taşma neden tersine döner? İkinin tümleyeni

`Integer.MAX_VALUE + 1` neden tam `Integer.MIN_VALUE` çıkıyor, `byte` neden −128 … 127 tutuyor? Cevap, bilgisayarın negatif sayıları nasıl sakladığında gizli: **ikinin tümleyeni** (two's complement).

Bir `byte` 8 bittir, yani $2^8 = 256$ farklı bit deseni tutar. Bu desenlerin yarısı negatif, yarısı sıfır ve pozitif sayılara ayrılır:

- En soldaki bit **işaret bitidir**: `0` ise sayı sıfır ya da pozitif, `1` ise negatiftir.
- Pozitif sayılar normal ikilik yazılır: 5 → `0000 0101`.
- Bir sayının negatifini bulmak için **bütün bitleri ters çevir, sonuca 1 ekle**: 5 → `0000 0101` → ters `1111 1010` → `+ 1` → `1111 1011` = −5.

8 bitlik desen	Değeri
`0000 0000`	0
`0111 1111`	127 (en büyük)
`1000 0000`	−128 (en küçük)
`1111 1111`	−1

Sıfır "pozitif tarafta" sayıldığı için pozitif tarafta 0 … 127 (128 desen), negatif tarafta −1 … −128 (128 desen) vardır; negatif taraf bir fazladır. n bitlik işaretli tam sayı $-2^{n-1} \ldots 2^{n-1} - 1$ aralığını tutar; `int` için 32 bit, yani $-2^{31} \ldots 2^{31} - 1$.

Taşma da bu tablodan okunur: `0111 1111` (127) sayısına 1 eklersen `1000 0000` olur ve bu desen −128'dir. En büyük pozitiften en küçük negatife geçersin. `Integer.toBinaryString` hazır metodu bir `int`'in bitlerini yazı olarak verir; `int` için aynı olayı görelim:

IkininTumleyeni.java

```
public class IkininTumleyeni {
    public static void main(String[] args) {
        int enBuyuk = Integer.MAX_VALUE;
        int tasan = enBuyuk + 1;
        System.out.println("5       = " + Integer.toBinaryString(5));
        System.out.println("-5      = " + Integer.toBinaryString(-5));
        System.out.println("MAX     = " + Integer.toBinaryString(enBuyuk));
        System.out.println("MAX + 1 = " + Integer.toBinaryString(tasan));
        System.out.println(tasan == Integer.MIN_VALUE);

        byte b = 127;
        b++;                                        // 0111 1111 + 1
        System.out.println(b);
        System.out.println((byte) 128 + " " + (byte) 255 + " " + (byte) 256);
        System.out.println(Integer.toBinaryString(200) + " -> " + (byte) 200);

        System.out.println(Math.abs(Integer.MIN_VALUE) + " " + -Integer.MIN_VALUE);
    }
}
```

Çıktı

```
5       = 101
-5      = 11111111111111111111111111111011
MAX     = 11111111111111111111111111111111
MAX + 1 = 10000000000000000000000000000000
true
-128
-128 -1 0
11001000 -> -56
-2147483648 -2147483648
```

`Integer.toBinaryString` baştaki sıfırları yazmaz: `MAX` 32 değil 31 bittir (işaret biti `0`'dır, yazılmadı). `MAX + 1` ise 32 bittir: `1` ve ardından 31 tane `0`, yani `MIN_VALUE`. Negatif sayıların başı hep `1` olduğu için `-5` tam 32 bit görünür. `(byte) 200` daralan dönüşümdür: `200` = `1100 1000` zaten 8 bittir, ama işaret biti `1` olduğu için `byte` onu −56 diye okur.

> **Sınavda Çıkar: `Math.abs` her zaman pozitif değildir**
>
> Negatif tarafın bir fazla olması bir tuzak doğurur: `+2 147 483 648` sayısı `int`'e sığmaz. Bu yüzden `-Integer.MIN_VALUE` ve `Math.abs(Integer.MIN_VALUE)` işlemleri sayıyı **değiştirmez**, ikisi de yine `-2147483648` verir. "Mutlak değer hep ≥ 0" varsayımı sınavda burada çöker.

2.11 Bit operatörleri ve sayı yazımları

Şimdiye kadar sayıları hep onluk tabanda yazdık. Java aynı sayıyı ikilik, onaltılık ve sekizlik tabanda da yazmana izin verir. Bit operatörleri ise sayıların **bitlerine** tek tek dokunur; bunlar bayrak (flag) tutma, şifreleme, donanım ve "2'nin kuvveti" sorularında karşına çıkar.

Sayı yazımları

Yazım	Taban	Örnek	Değeri
(önek yok)	onluk	`31`	31
`0b`	ikilik (binary)	`0b1010`	10
`0x`	onaltılık (hexadecimal)	`0x1F`	31
`0` (başta sıfır)	sekizlik (octal)	`017`	15

Onaltılık tabanda rakamlar `0–9` ve `A–F`'dir (`A` = 10, `F` = 15): `0x1F` = 1 × 16 + 15 = 31. Her onaltılık rakam tam 4 bittir, bu yüzden bit desenlerini kısa yazmanın yoludur: `0xFF` = `1111 1111` = 255. Alt çizgi `_` ayırıcısı her tabanda kullanılabilir.

SayiYazimlari.java

```
public class SayiYazimlari {
    public static void main(String[] args) {
        int ikilik = 0b1010;
        int onaltilik = 0x1F;
        int sekizlik = 017;
        int maske = 0b1111_0000;
        System.out.println(ikilik + " " + onaltilik + " " + sekizlik);
        System.out.println(maske + " " + 0xFF + " " + 1_000_000);
        System.out.println(0x7FFFFFFF == Integer.MAX_VALUE);
        System.out.println(0x80000000 == Integer.MIN_VALUE);
    }
}
```

Çıktı

```
10 31 15
240 255 1000000
true
true
```

> **Sık Yapılan Hata: Başa sıfır koyma: `017` 17 değildir**
>
> Java, başında `0` olan tam sayıyı **sekizlik** sayar. `int kod = 010;` satırı `kod`'a 10 değil **8** koyar ve derleyici uyarmaz; `017` yukarıda gördüğün gibi 15'tir. Sayının başına süs için sıfır ekleme (`007`, `0100`). `int ay = 08;` gibi sekizlikte olmayan rakam (8, 9) içeren yazım ise derlenmez (`illegal digit in an octal literal`).

Bit operatörleri

Bit operatörleri iki sayının **aynı sıradaki bitlerini ayrı ayrı** işler ve yalnız tam sayılarla çalışır. Örnekte `a = 12` (`1100`) ve `b = 10` (`1010`):

Operatör	Adı	Ne yapar?	Sonuç
`a & b`	VE (AND)	İkisinde de 1 olan bit 1 olur	`1000` (8)
`a | b`	VEYA (OR)	En az birinde 1 olan bit 1 olur	`1110` (14)
`a ^ b`	ÖZEL VEYA (XOR)	Yalnız birinde 1 olan bit 1 olur	`0110` (6)
`~a`	DEĞİL (NOT)	32 bitin hepsini ters çevirir (tekli)	−13
`a << n`	Sola kaydırma	Bitleri `n` basamak sola iter, sağdan `0` gelir	`110000` (48)
`a >> n`	İşaretli sağa kaydırma	Sağa iter, soldan **işaret biti** kopyalanır	`11` (3)
`a >>> n`	İşaretsiz sağa kaydırma	Sağa iter, soldan her zaman `0` gelir	`11` (3)

BitOperatorleri.java

```java
public class BitOperatorleri {
    public static void main(String[] args) {
        int a = 0b1100;                      // 12
        int b = 0b1010;                      // 10
        System.out.println("a & b   = " + (a & b));
        System.out.println("a | b   = " + (a | b));
        System.out.println("a ^ b   = " + (a ^ b));
        System.out.println("~a      = " + ~a);
        System.out.println("a << 2  = " + (a << 2));
        System.out.println("a >> 2  = " + (a >> 2));
        System.out.println("a >>> 2 = " + (a >>> 2));

        int x = 13;                          // 1101
        System.out.println("x & 1   = " + (x & 1));
        x = x | (1 << 1);                    // 1. biti aç
        System.out.println("açıldı  = " + Integer.toBinaryString(x));
        x = x & ~(1 << 3);                   // 3. biti kapat
        System.out.println("kapandı = " + Integer.toBinaryString(x));
    }
}
```

Çıktı

```
a & b   = 8
a | b   = 14
a ^ b   = 6
~a      = -13
a << 2  = 48
a >> 2  = 3
a >>> 2 = 3
x & 1   = 1
açıldı  = 1111
kapandı = 111
```

Son üç satır tipik işlerdir. `x & 1` sayının son bitidir: `1` ise sayı tektir (pozitif sayılarda `x % 2` ile aynı). `1 << n`, yalnız `n`. biti 1 olan sayıdır (**bit maskesi**, 2^n'e eşittir). `x | maske` biti açar, `x & ~maske` kapatır, `x ^ maske` tersler. `<<` (taşma olmazsa) sayıyı 2^n ile çarpar, `>>` ise 2^n'e bölüp **aşağı yuvarlar**. Bileşik atama biçimleri de vardır: `&=`, `|=`, `^=`, `<<=`, `>>=`, `>>>=`.

`>>` ile `>>>` farkı

Pozitif sayılarda `>>` ile `>>>` aynı sonucu verir; fark **negatif sayılarda** başlar. `>>` soldan işaret bitini kopyalar, sayı negatif kalır; `>>>` soldan `0` doldurur, sonuç pozitif olur. **`>>>` Java'ya özgüdür**, C ve C++'ta yoktur; sınavda "`>>` ile `>>>` farkı nedir?" sorusu çıkar.

KaydirmaFarki.java

```java
public class KaydirmaFarki {
    public static void main(String[] args) {
        int n = -8;
        System.out.println(n >> 1);
        System.out.println(n >>> 1);
        System.out.println(Integer.toBinaryString(n >> 1));
        System.out.println(Integer.toBinaryString(n >>> 1));
        System.out.println((-5 / 2) + " " + (-5 >> 1));
        System.out.println((1 << 33) + " " + (1L << 33));

        byte k = -1;
        System.out.println(k >>> 4);
        System.out.println((byte) (k >>> 4));
    }
}
```

Çıktı

```
-4
2147483644
11111111111111111111111111111100
11111111111111111111111111111100
-2 -3
2 8589934592
268435455
-1
```

`-8 >> 1` sonucu −4'tür (yaklaşık 2'ye bölme); `-8 >>> 1` ise baş bit `0` olduğu için dev bir pozitif sayıdır. `-5 / 2` sıfıra doğru keser (−2), `-5 >> 1` aşağı yuvarlar (−3); negatif sayılarda `>> 1` ile `/ 2` aynı değildir. Kaydırma miktarı `int` için 32'ye göre mod alınır: `1 << 33` aslında `1 << 1` olur (C'de bu davranış tanımsızdır). `long` için miktar 64'e göre alınır, `1L << 33` beklediğin 2^{33}'tür.

> **Sınavda Çıkar: `byte` üzerinde kaydırma**
>
> `byte`, `short` ve `char` bit işlemlerinden önce **`int`'e yükseltilir**. `byte k = -1;` (hepsi `1`) için `k >>> 4` önce 32 bitlik `int`'te kaydırılır ve 268435455 çıkar; sonucu `(byte)` ile daraltınca üst bitler atılıp tekrar −1 olur. Aynı nedenle `byte b = 5; b = ~b;` derlenmez (sonuç `int`), `b = (byte) ~b;` gerekir.

Sık Yapılan Hata: `&` ve `==` önceliği

Java'da `==` karşılaştırması `&`, `^` ve `|`'dan **önce** yapılır. Bu yüzden `x & 1 == 0` ifadesi `x & (1 == 0)` diye okunur ve `int & boolean` olduğu için derlenmez (C'de derlenir ama yanlış çalışır). Parantez koy: `(x & 1) == 0`.

```java
public class BitOncelik {
    public static void main(String[] args) {
        int x = 6;
        if (x & 1 == 0) {
            System.out.println("çift");
        }
    }
}
```

Derleyici çıktısı – program derlenmez

```
BitOncelik.java:4: error: bad operand types for binary operator '&'
        if (x & 1 == 0) {
             ^
  first type:  int
  second type: boolean
1 error
```

Not: `&` ve `|` ikinci bir iş daha yapar

`&` ve `|`, iki tarafı `boolean` olunca mantıksal VE/VEYA yapar ve `&&`, `||` gibi kısa devre **yapmaz** (Bölüm 4'te göreceksin). İki tarafı tam sayı olunca bit düzeyinde çalışırlar. `&&` ve `||` ise yalnız `boolean` ister: C'deki gibi `12 && 10` yazarsan `bad operand types for binary operator '&&'` hatası alırsın.

Bölüm Özeti – Bilmen Gerekenler

- Değişken = ad + tip + değer. Kullanmadan önce **tanımla**; yerel değişkeni değer vermeden okuyamazsın.
- İsim rakamla başlayamaz, boşluk ve anahtar kelime içeremez. Değişken `camelCase`, sınıf `PascalCase`, sabit `BUYUK_HARF`.
- 8 ilkel tip: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`. Varsayılanlar: tam sayı → `int`, ondalık → `double`. `String` ilkel değildir.
- `long` için `L`, `float` için `f` eki. `char` tek tırnak, `String` çift tırnak.
- **`int / int` = `int`**: `7 / 2 = 3`, `9 / 10 = 0`. Ondalık sonuç için birini `double` yap.
- `%` kalan verir: tek/çift, bölünebilme, son basamak (`n % 10`).
- Genişleyen dönüşüm otomatik, daralan dönüşüm cast ister: `(int) 3.99 = 3` (kesme). `(double) a / b` ile `(double) (a / b)` farklıdır.
- `x++` önce kullanır sonra artırır; `++x` önce artırır sonra kullanır.
- `b += 5` gizli cast içerir; `b = b + 5` (b `byte` ise) derlenmez.
- `int` taşınca hata vermez, tersine döner. `double`'lar `==` ile karşılaştırılmaz.
- Negatif tam sayılar **ikinin tümleyeniyle** tutulur: `byte` −128 … 127'dir, `MAX_VALUE + 1 = MIN_VALUE` olur (en büyük pozitif + 1 → işaret biti 1).
- Bit operatörleri: `&` `|` `^` `~` `<<` `>>` `>>>`. `>>` işaret bitini korur, `>>>` (yalnız Java'da) soldan `0` doldurur. `byte`/`short` bit işleminden önce `int` olur.
- Sayı yazımları: `0b1010` ikilik, `0x1F` onaltılık, **`017` sekizliktir (15)**, `1_000` alt çizgi ayırıcı. `x & 1 == 0` derlenmez; `(x & 1) == 0` yaz.

Sorular

A) Çoktan Seçmeli

1 Aşağıdakilerden hangisi **geçerli** bir Java değişken adıdır?

(A) `_ortalama2` (B) `2ortalama` (C) `ogrenci no` (D) `class` (E) `not-ortalamasi`

2 `double sonuc = 15 / 4;` satırından sonra `sonuc` değişkeninin değeri nedir?

(A) `3.75`
(B) `4.0`
(C) `3`
(D) Derleme hatası verir
(E) `3.0`

3 Aşağıdaki satırlardan hangisi **derlenmez**?

(A) `double d = 10;` (B) `long l = 100;` (C) `char c = 65;` (D) `float f = 2.5;` (E) `int i = 'A';`

4 `int a = 4; int b = a++ + ++a;` satırları çalıştıktan sonra `a` ve `b`'nin değerleri nedir?

(A) `a = 6, b = 8` (B) `a = 5, b = 9` (C) `a = 6, b = 10` (D) `a = 6, b = 9` (E) `a = 5, b = 10`

5 `System.out.println((int) 7.9 + 17 % 5 * 2);` komutu ne yazdırır?

(A) `13` (B) `11` (C) `14` (D) `11.9` (E) `15`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru6 {
    public static void main(String[] args) {
        int a = 17, b = 5;
        double c = a / b;
        double d = (double) a / b;
        System.out.println(a / b + " " + a % b);
        System.out.println(c + " " + d);
        System.out.println(a / (double) b * 0);
        System.out.println(b / a + b % a);
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru7 {
    public static void main(String[] args) {
        int x = 5;
        int y = x++ * 2;
        int z = --x + y--;
        System.out.println(x + " " + y + " " + z);
        x += x++ + ++x;
        System.out.println(x);
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru8 {
    public static void main(String[] args) {
        char h = 'a';
        h += 2;
        System.out.println(h);
        System.out.println(h + 1);
        System.out.println((char) (h - 32));
        System.out.println("" + h + 1);
        System.out.println(Math.round(7.5) + Math.abs(-2));
    }
}
```

C) Kod Yazma

9 Santigrat derecesini Fahrenhayt'a çeviren `Sicaklik` programını yaz. Formül: **F = C × 9 / 5 + 32**. Programda `double c = 37.5;` değişkeni olsun ve sonucu `37.5 C = 99.5 F` biçiminde yazdırsın. (İpucu: Formülde tam sayı bölmesine dikkat et!)

10 Saniye cinsinden verilen bir süreyi **saat, dakika ve saniyeye** ayıran `Sure` programını yaz. `int toplamSaniye = 9876;` için çıktı `2 saat 44 dakika 36 saniye` olmalıdır. Sadece `/` ve `%` operatörlerini kullan.

Çözümler

Cevap anahtarı

1	2	3	4	5
A	E	D	C	B

1 Cevap: A

Değişken adı harf, `_` veya `$` ile başlayabilir; içinde rakam olabilir. `2ortalama` rakamla başlıyor, `ogrenci no` boşluk içeriyor, `class` anahtar kelime, `not-ortalamasi` ise `-` içeriyor.

2 Cevap: E

Sağ taraf önce hesaplanır: `15 / 4` iki tam sayının bölümüdür, sonuç `3` olur. Sonra `3`, `double`'a otomatik dönüşür ve `3.0` olur. `3.75` elde etmek için `15 / 4.0` yazılmalıydı.

3 Cevap: D

`2.5` bir `double` literal'idir; `double` → `float` daralan dönüşümdür ve otomatik yapılmaz. `2.5f` yazılmalıydı. C şıkkı derlenir: Sabit bir tam sayı, `char` aralığına sığıyorsa `char`'a atanabilir (`c = 'A'`). E de derlenir: `char` → `int` genişleyen dönüşümdür.

4 Cevap: C

`a++`: Değer olarak **4** kullanılır, `a` = 5 olur. `++a`: `a` = 6 olur, değer olarak **6** kullanılır. `b = 4 + 6 = 10`, `a` = 6.

5 Cevap: B

Öncelik sırası: Cast → `(int) 7.9 = 7`. Sonra `%` ve `*` soldan sağa: `17 % 5 = 2`, `2 * 2 = 4`. En son toplama: `7 + 4 = 11`.

6

- `a / b` = `17 / 5` = 3, `a % b` = 2.
- `c = a / b` → önce tam sayı bölmesi (3), sonra `3.0`. `d` = `17.0 / 5 = 3.4`.
- `17 / 5.0 * 0` = `3.4 * 0` = `0.0` (sonuç `double`).
- `b / a` = `5 / 17` = 0, `b % a` = `5 % 17` = 5 → `0 + 5 = 5`.

Çıktı

```
3 2
3.0 3.4
0.0
5
```

7

Değişken tablosu ile izleyelim:

Satır	İşlem	x	y	z
`int x = 5;`		5		
`y = x++ * 2`	5 kullanılır, x = 6	6	10	
`z = --x + y--`	x = 5 olur, 5 kullanılır; y'nin 10'u kullanılır, y = 9	5	9	15
`x += x++ + ++x`	x = 5 + (5 + 7)	17	9	15

Son satır: `x += ...` ifadesi `x = x + (...)` demektir ve soldaki `x`'in değeri (**5**) en başta alınır. Sonra `x++` → 5 kullanılır, x = 6; `++x` → x = 7, 7 kullanılır. Sonuç `5 + (5 + 7) = 17`.

Çıktı
```
5 9 15
17
```

8

- `h += 2`: `'a'` (97) + 2 = 99 = `'c'`. Bileşik atama cast'i kendisi yapar.
- `h + 1`: `char + int` → `int`: 100.
- `h - 32` = 67 = `'C'` (büyük harf).
- `"" + h + 1`: Önce `""` ile String olur → `"c"`, sonra `"c1"`.
- `Math.round(7.5)` = 8 (long), `Math.abs(-2)` = 2, toplam 10.

Çıktı
```
c
100
C
c1
10
```

9

`9 / 5` yazarsak tam sayı bölmesi `1` verir ve sonuç yanlış çıkar. `9.0 / 5` yazmak ya da `c * 9` ile başlamak (c zaten `double`) sorunu çözer.

Sicaklik.java
```java
public class Sicaklik {
    public static void main(String[] args) {
        double c = 37.5;
        double f = c * 9 / 5 + 32;
        System.out.println(c + " C = " + f + " F");
    }
}
```

Çıktı
```
37.5 C = 99.5 F
```

Not: `c = 36.6` ile denersen sonucu `97.88000000000001` olarak görürsün; bu, "Taşma ve ondalık hassasiyet" başlığındaki `0.1 + 0.2` durumunun aynısıdır. Çıktıyı 2 basamağa yuvarlamayı Bölüm 3'te `printf` ile öğreneceksin.

10

Bir saatte 3600 saniye var. Önce `/ 3600` ile saati bul, `% 3600` ile kalan saniyeyi al; kalanı `/ 60` ile dakikaya böl, `% 60` ile kalan saniyeyi bul.

Sure.java

```
public class Sure {
    public static void main(String[] args) {
        int toplamSaniye = 9876;
        int saat = toplamSaniye / 3600;
        int kalan = toplamSaniye % 3600;
        int dakika = kalan / 60;
        int saniye = kalan % 60;
        System.out.print(saat + " saat " + dakika + " dakika ");
        System.out.println(saniye + " saniye");
    }
}
```

Çıktı

```
2 saat 44 dakika 36 saniye
```

BÖLÜM 3

Klavyeden Girdi ve Biçimli Çıktı

Bu Bölümde Öğreneceklerin

- `Scanner` ile klavyeden sayı, kelime ve satır okumayı
- `next()` ile `nextLine()` arasındaki farkı
- **`nextInt()` + `nextLine()` tuzağını** ve çözümünü
- `printf` ve `String.format` ile çıktıyı biçimlendirmeyi (`%d`, `%.2f`, `%s`, genişlik, hizalama)
- Türkçe bilgisayarlarda ondalık virgül sorununu

3.1 Scanner ile klavyeden okuma

Şimdiye kadar yazdığımız programlar hep aynı şeyi yaptı; değerler koda gömülüydü. Kullanıcıdan veri almak için Java'nın `Scanner` sınıfını kullanırız. `Scanner`, `java.util` paketinde bulunduğu için dosyanın en başında **içe aktarılması** (import) gerekir:

Toplayici.java

```java
import java.util.Scanner;

public class Toplayici {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Birinci sayı: ");
        int a = sc.nextInt();
        System.out.print("İkinci sayı: ");
        int b = sc.nextInt();

        System.out.println("Toplam = " + (a + b));
    }
}
```

Çıktı

```
Birinci sayı: 12
İkinci sayı: 30
Toplam = 42
```

- `import java.util.Scanner;` → Scanner sınıfını kullanacağımızı söyler. `class` satırından **önce** yazılır.
- `Scanner sc = new Scanner(System.in);` → Klavyeden (`System.in`) okuyan bir Scanner nesnesi oluşturur. Programda **bir tane** oluşturman yeterli.
- `sc.nextInt()` → Kullanıcı bir tam sayı yazıp Enter'a basana kadar bekler, sonra o sayıyı döndürür.

İpucu

Okumadan önce `print` ile kullanıcıya ne girmesi gerektiğini söyle (*prompt*). `println` yerine `print` kullanırsan imleç aynı satırda kalır ve kullanıcı cevabını sorunun yanına yazar.

Not: VS Code'daki sarı uyarı: "Resource leak"

Programı VS Code'da yazarken `Scanner sc = new Scanner(System.in);` satırının altında sarı bir dalgalı çizgi ve `Resource leak: 'sc' is never closed` uyarısı görebilirsin. Hata değildir, program çalışır. Uyarı şunu der: `Scanner` bir **kaynak**tır (resource) ve işin bitince `close()` ile kapatılması beklenir. Klavyeyle işin bittiğinde, genellikle `main`'in son satırında kapat:

ScannerKapatma.java

```java
import java.util.Scanner;

public class ScannerKapatma {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Adın: ");
        String ad = sc.nextLine();
        System.out.println("Merhaba " + ad + "!");

        sc.close();    // klavyeyle işimiz bitti; uyarı da kaybolur
    }
}
```

Çıktı

```
Adın: Ece
Merhaba Ece!
```

Kapatmanın bir bedeli vardır: Klavyeyi saran Scanner'ı kapatmak `System.in`'i de kapatır; sonradan başka bir `new Scanner(System.in)` ile okumaya çalışırsan `NoSuchElementException` alırsın. Bu yüzden **ilk okuma bitmeden `close()` yazma**, programda tek Scanner kullan ve kapatmayı en sona bırak. Dosyaları okurken Scanner'ı kapatmak zorunludur; bunu `try-with-resources` ile otomatik yapmayı Bölüm 12'de öğreneceksin.

Scanner'ın okuma metotları

Metot	Ne okur?	Dönüş tipi
`nextInt()`	Bir tam sayı	`int`
`nextLong()`	Büyük tam sayı	`long`
`nextDouble()`	Ondalıklı sayı	`double`
`nextBoolean()`	`true` / `false`	`boolean`
`next()`	**Tek bir kelime** (boşluğa kadar)	`String`
`nextLine()`	**Satırın geri kalanının tamamı** (Enter'a kadar)	`String`

Scanner'da `char` okuyan bir metot yoktur. Tek karakter okumak için kelimeyi okuyup ilk karakterini alırsın: `char c = sc.next().charAt(0);` (`charAt`'i Bölüm 8'de göreceksin.)

3.2 next() ve nextLine() farkı

`next()` boşluk, sekme veya satır sonu görene kadar okur; yani **tek kelime** alır. `nextLine()` ise satır sonuna kadar her şeyi, boşluklar dahil alır:

NextFarki.java

```java
import java.util.Scanner;

public class NextFarki {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Ad soyad (next): ");
        String kelime = sc.next();
        String kalan = sc.nextLine();   // aynı satırın geri kalanı
        System.out.println("next()     -> [" + kelime + "]");
        System.out.println("nextLine() -> [" + kalan + "]");

        System.out.print("Ad soyad (nextLine): ");
        String satir = sc.nextLine();
        System.out.println("nextLine() -> [" + satir + "]");
    }
}
```

Çıktı

```
Ad soyad (next): Ali Yılmaz
next()     -> [Ali]
nextLine() -> [ Yılmaz]
Ad soyad (nextLine): Ayşe Nur Kaya
nextLine() -> [Ayşe Nur Kaya]
```

Köşeli parantezleri, okunan metnin tam olarak nerede başlayıp bittiğini görmek için ekledik. `next()` sadece `Ali`'yi aldı; aynı satırda kalan `Yılmaz` (başındaki boşlukla birlikte) bir sonraki `nextLine()` ile okundu.

3.3 nextInt() + nextLine() tuzağı

Bu, Java öğrenen herkesin en az bir kere düştüğü tuzaktır. Önce bir sayı, sonra bir satır okumaya çalışalım:

Tuzak.java

```java
import java.util.Scanner;

public class Tuzak {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Yaşın: ");
        int yas = sc.nextInt();
        System.out.print("Adın soyadın: ");
        String ad = sc.nextLine();

        System.out.println("Ad: [" + ad + "], yaş: " + yas);
    }
}
```

Çıktı

```
Yaşın: 19
Adın soyadın: Ad: [], yaş: 19
```

Program, ad soyadı **beklemeden** geçti ve `ad` boş kaldı! Neden?

1. Kullanıcı `19` yazıp **Enter**'a bastığında girdi tamponunda `19⏎` vardır (`⏎` satır sonu karakteri `\n`).
2. `nextInt()` sadece `19`'u alır; `⏎` **tamponda kalır**.

3. `nextLine()` "satır sonuna kadar oku" demektir. Tamponda hemen bir `⏎` gördüğü için **boş bir satır** okuyup döner. Kullanıcının yazacağı `Zeynep Ak` hiç okunmaz.

(Kullanıcının yazdığı `Zeynep Ak` yukarıdaki çıktıda görünmüyor, çünkü program onu okumadan bitti.)

Çözüm 1: Artık satır sonunu tüket

`nextInt()`'ten sonra boş bir `sc.nextLine();` çağır. Bu çağrı tamponda kalan `⏎`'yi okuyup atar:

TuzakCozum.java

```java
import java.util.Scanner;

public class TuzakCozum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Yaşın: ");
        int yas = sc.nextInt();
        sc.nextLine();                      // satır sonunu temizle
        System.out.print("Adın soyadın: ");
        String ad = sc.nextLine();

        System.out.println("Ad: [" + ad + "], yaş: " + yas);
    }
}
```

Çıktı

```
Yaşın: 19
Adın soyadın: Zeynep Ak
Ad: [Zeynep Ak], yaş: 19
```

Çözüm 2: Her şeyi nextLine() ile oku

Her girdiyi satır olarak okuyup sayıya kendin çevirirsen tuzak hiç oluşmaz:

TuzakCozum2.java

```java
import java.util.Scanner;

public class TuzakCozum2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Yaşın: ");
        int yas = Integer.parseInt(sc.nextLine());
        System.out.print("Boyun (m): ");
        double boy = Double.parseDouble(sc.nextLine());
        System.out.print("Adın soyadın: ");
        String ad = sc.nextLine();

        System.out.println(ad + ", " + yas + " yaş, " + boy + " m");
    }
}
```

Çıktı

```
Yaşın: 19
Boyun (m): 1.68
Adın soyadın: Zeynep Ak
Zeynep Ak, 19 yaş, 1.68 m
```

`Integer.parseInt("19")` metni `19` sayısına, `Double.parseDouble("1.68")` metni `1.68` sayısına çevirir.

> **Sınavda Çıkar**
>
> "Aşağıdaki programda kullanıcı `25` yazıp Enter'a bastıktan sonra `isim` değişkeninin değeri nedir?" sorusunun cevabı genellikle **boş String** (`""`) olur. Sebep: `nextInt()` satır sonu karakterini tamponda bırakır, ardından gelen `nextLine()` onu okur. Kural: **`nextInt` / `nextDouble` / `next` → `nextLine` geçişlerinde araya bir boş `nextLine()` koy.**

> **Not: Tuzak sadece nextLine'da var**
>
> `nextInt()`'ten sonra yine `nextInt()`, `nextDouble()` veya `next()` gelirse sorun olmaz. Bu metotlar okumaya başlamadan önce baştaki boşlukları ve satır sonlarını atlar. Tuzak, sadece ardından **`nextLine()`** geldiğinde oluşur.

Yanlış tipte girdi

Program sayı beklerken kullanıcı başka bir şey yazarsa `nextInt()` bir `InputMismatchException` fırlatır ve program çöker:

YanlisGirdi.java

```java
import java.util.Scanner;

public class YanlisGirdi {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Yaşın: ");
        int yas = sc.nextInt();
        System.out.println("Gelecek yıl " + (yas + 1) + " olacaksın.");
    }
}
```

Çıktı – program hata ile durur

```
Yaşın: yirmi
Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:977)
        at java.base/java.util.Scanner.next(Scanner.java:1632)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2297)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2251)
        at YanlisGirdi.main(YanlisGirdi.java:7)
```

Kullanıcı girdisini kontrol etmeyi (`hasNextInt()`) Bölüm 5'te, exception'ları yakalamayı Bölüm 12'de öğreneceksin.

3.4 printf ile biçimli çıktı

`println` ile ondalıklı sayı yazdırınca `97.88000000000001` gibi uzun sonuçlar ya da hizasız tablolar çıkabilir. `System.out.printf` (*print formatted*) çıktının **görünüşünü** kontrol etmeni sağlar. İlk parametre bir **biçim metnidir** (format string); içindeki `%` ile başlayan **biçim belirteçlerinin** (format specifier) yerine sıradaki değerler konur:

PrintfGiris.java

```java
public class PrintfGiris {
    public static void main(String[] args) {
        String ad = "Deniz";
        int yas = 20;
        double ort = 3.456789;

        System.out.printf("%s %d yaşında.%n", ad, yas);
        System.out.printf("Ortalama: %.2f%n", ort);
        System.out.printf("%s'in ortalaması %.1f, yaşı %d%n", ad, ort, yas);
    }
}
```

Çıktı

```
Deniz 20 yaşında.
Ortalama: 3.46
Deniz'in ortalaması 3.5, yaşı 20
```

`printf` kendiliğinden alt satıra **geçmez**; satır sonu için `%n` (veya `\n`) eklemen gerekir.

Belirteç	Ne için?	Örnek	Çıktı
`%d`	Tam sayı (`int`, `long`)	`printf("%d", 42)`	`42`
`%f`	Ondalıklı sayı (varsayılan 6 basamak)	`printf("%f", 3.5)`	`3.500000`
`%.2f`	Virgülden sonra 2 basamak (**yuvarlar**)	`printf("%.2f", 3.456)`	`3.46`
`%s`	String (aslında her şey)	`printf("%s", "Java")`	`Java`
`%c`	Karakter	`printf("%c", 'A')`	`A`
`%b`	boolean	`printf("%b", true)`	`true`
`%e`	Bilimsel gösterim	`printf("%e", 12345.6)`	`1.234560e+04`
`%n`	Satır sonu		
`%%`	Yüzde işaretinin kendisi	`printf("%d%%", 85)`	`85%`

Genişlik ve hizalama

`%` ile harf arasına bir sayı yazarsan, o değer **en az o kadar karakterlik** bir alana yerleştirilir. Varsayılan olarak **sağa** yaslanır; `-` işareti eklersen **sola** yaslanır. Tablo yazdırmak için idealdir:

Tablo.java

```java
public class Tablo {
    public static void main(String[] args) {
        System.out.printf("%-10s%6s%8s%n", "Ürün", "Adet", "Fiyat");
        System.out.printf("%-10s%6d%8.2f%n", "Kalem", 3, 12.5);
        System.out.printf("%-10s%6d%8.2f%n", "Defter", 12, 45.0);
        System.out.printf("%-10s%6d%8.2f%n", "Silgi", 125, 7.25);
        System.out.println("------------------------");
        System.out.printf("[%5d] [%-5d] [%05d]%n", 42, 42, 42);
        System.out.printf("[%,d] [%+d] [%8.3f]%n", 1234567, 15, Math.PI);
    }
}
```

```
Çıktı
Ürün        Adet   Fiyat
Kalem          3   12.50
Defter        12   45.00
Silgi        125    7.25
------------------------
[   42] [42   ] [00042]
[1,234,567] [+15] [   3.142]
```

Biçim	Anlamı	Örnek (değer 42)
`%5d`	5 karakterlik alan, sağa yasla	`42`
`%-5d`	5 karakterlik alan, sola yasla	`42`
`%05d`	5 karakter, boşluk yerine **sıfırla** doldur	`00042`
`%,d`	Binlik ayırıcı	`1,234,567`
`%+d`	İşareti her zaman göster	`+15`
`%8.3f`	Toplam 8 karakter, 3'ü ondalık	`3.142`

Sık Yapılan Hata: Belirteç ile değer tipinin uyuşmaması

`printf("%d", 3.5)` derlenir ama **çalışırken** `IllegalFormatConversionException` hatası verir: `%d` tam sayı ister, `3.5` ise `double`'dır. Tersi de geçerlidir: `printf("%.2f", 10)` da hata verir; `10.0` yazmalısın. Bir diğer sık hata: belirteç sayısı kadar değer vermemek (`printf("%d %d", 5)` → `MissingFormatArgumentException`).

Sınavda Çıkar: `%.2f` yuvarlar, cast keser

`printf("%.2f", 2.678)` → `2.68` (**yuvarlama**). `(int) 2.678` → `2` (**kesme**). `Math.round(2.678)` → `3`. Bu üçü sık sık aynı soruda karşılaştırılır.

3.5 String.format

`String.format`, `printf` ile **aynı** biçim kurallarını kullanır; ama sonucu ekrana yazmak yerine bir **String olarak döndürür**. Biçimlendirilmiş metni bir değişkende saklamak veya başka bir metinle birleştirmek istediğinde kullanılır:

FormatOrnek.java

```java
public class FormatOrnek {
    public static void main(String[] args) {
        double fiyat = 1249.9;
        int indirim = 15;
        double yeniFiyat = fiyat * (100 - indirim) / 100;

        String etiket = String.format("%d%% indirimle %.2f TL", indirim, yeniFiyat);
        System.out.println("Etiket: " + etiket);
        System.out.println("Uzunluk: " + etiket.length() + " karakter");
    }
}
```

```
Çıktı
Etiket: 15% indirimle 1062.42 TL
Uzunluk: 24 karakter
```

`etiket.length()` bir String'in kaç karakter olduğunu verir (boşluk ve işaretler dahil). String'in diğer metotlarını Bölüm 8'de göreceksin.

Java 15'ten beri aynı işi `"%d%% indirimle %.2f TL".formatted(indirim, yeniFiyat)` şeklinde de yapabilirsin.

3.6 Türkçe bilgisayarlarda ondalık virgül

`printf`, `String.format` ve `Scanner` sayıları biçimlendirirken bilgisayarın **bölge ayarını** (locale) kullanır. Türkçede ondalık ayırıcı **virgül** olduğu için Türkçe bir Windows veya macOS'ta:

- `System.out.printf("%.2f", 3.75)` → `3,75` yazdırabilir.
- `sc.nextDouble()` kullanıcıdan `3.75` değil **`3,75`** bekler; `3.75` yazılırsa `InputMismatchException` verebilir.

`println` bundan etkilenmez; `System.out.println(3.75)` her zaman `3.75` yazar. Programın her bilgisayarda aynı davranmasını istiyorsan bölgeyi açıkça belirt:

BolgeAyari.java

```java
import java.util.Locale;

public class BolgeAyari {
    public static void main(String[] args) {
        Locale tr = Locale.forLanguageTag("tr-TR");
        double sayi = 1234.5;

        System.out.println(String.format(Locale.US, "%,.2f", sayi));
        System.out.println(String.format(tr, "%,.2f", sayi));
        System.out.printf(Locale.US, "%.1f%n", 2.25);
    }
}
```

```
Çıktı
1,234.50
1.234,50
2.3
```

Scanner için de aynısı geçerlidir: `Scanner sc = new Scanner(System.in).useLocale(Locale.US);` yazarsan ondalık sayılar her bilgisayarda **nokta** ile girilir.

> **İpucu**
>
> Bu kitaptaki tüm çıktılar `Locale.US` ile üretilmiştir, yani ondalık ayırıcı noktadır. Kendi bilgisayarında virgül görürsen program yanlış değildir, sadece bölge ayarın Türkçedir. Ödevlerde ve sınavlarda genellikle nokta beklenir.

3.7 Örnek uygulama: Vücut kitle indeksi

Öğrendiklerimizi birleştirelim. Kullanıcıdan adını, kilosunu ve boyunu alıp **vücut kitle indeksini** (VKİ = kilo / boy²) hesaplayan bir program:

Vki.java

```java
import java.util.Locale;
import java.util.Scanner;

public class Vki {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Adın: ");
        String ad = sc.nextLine();
        System.out.print("Kilon (kg): ");
        double kilo = sc.nextDouble();
        System.out.print("Boyun (m): ");
        double boy = sc.nextDouble();

        double vki = kilo / (boy * boy);
        System.out.printf("%s, VKİ değerin: %.1f%n", ad, vki);
    }
}
```

Çıktı

```
Adın: Mert Can
Kilon (kg): 72.5
Boyun (m): 1.80
Mert Can, VKİ değerin: 22.4
```

Sonucu “zayıf / normal / kilolu” diye yorumlamak için **koşul** gerekir; bu bir sonraki bölümün konusu.

Java 25 Notu: IO.readln ile kısa okuma

Java 25’teki `IO` sınıfında, kullanıcıya bir soru yazıp cevabını satır olarak okuyan `IO.readln` metodu vardır. Bölüm 1’de gördüğün kısa yazımla (`void main()`, sınıfsız) birleşince `Scanner`, `import` ve `new` gerekmez:

```java
void main() {
    String ad = IO.readln("Adın: ");
    int yas = Integer.parseInt(IO.readln("Yaşın: "));
    IO.println("Merhaba " + ad + ", " + (yas + 1) + " yaşına az kaldı!");
}
```

Çıktı

```
Adın: Ece
Yaşın: 18
Merhaba Ece, 19 yaşına az kaldı!
```

- `IO.readln(soru)` önce `soru`’yu yazar (alt satıra geçmeden), sonra Enter’a kadar bütün satırı `String` olarak döndürür. Soru yazmadan `IO.readln()` de çağrılabilir.
- Her şeyi satır olarak okuduğu için `nextInt` + `nextLine` tuzağı yoktur; sayı lazımsa kendin çevirirsin: `Integer.parseInt(...)`, `Double.parseDouble(...)`.
- Terminalde tek komutla çalıştırılır: `java KisaOkuma.java`.

Derslerde ve sınavlarda `Scanner` ile klasik yazım beklenir.

Bölüm Özeti – Bilmen Gerekenler

- Klavyeden okumak için: `import java.util.Scanner;` ve `Scanner sc = new Scanner(System.in);`
- `nextInt()`, `nextDouble()`, `next()` (tek kelime), `nextLine()` (tüm satır). `char` için `sc.next().charAt(0)`.
- **Tuzak:** `nextInt()` satır sonunu tamponda bırakır; hemen ardından gelen `nextLine()` boş String döndürür. Çözüm: araya boş bir `sc.nextLine();` koy veya her şeyi `nextLine()` + `Integer.parseInt` ile oku.
- Yanlış tipte girdi → `InputMismatchException`.
- `printf` biçimleri: `%d` tam sayı, `%f` ondalık, `%.2f` 2 basamak (yuvarlar), `%s` metin, `%c` karakter, `%b` boolean, `%n` satır sonu, `%%` yüzde.
- Genişlik: `%8d` sağa, `%-8s` sola yaslar; `%05d` sıfırla doldurur; `%,d` binlik ayırıcı.
- `printf` alt satıra kendisi geçmez. `String.format` aynı işi yapıp String döndürür.
- Türkçe bölge ayarında `printf` virgül kullanır ve `nextDouble` virgül bekler; `Locale.US` ile sabitlenebilir.
- VS Code'daki `Resource leak` uyarısı: `Scanner` kaynaktır, işin bitince `sc.close()` yaz. Klavye Scanner'ını kapatınca `System.in` de kapanır; bir daha okunamaz.

Sorular

A) Çoktan Seçmeli

1 `Scanner` sınıfını kullanabilmek için dosyanın başına hangi satır yazılmalıdır?

(A) `import java.lang.Scanner;`
(B) `import java.util.Scanner;`
(C) `include Scanner;`
(D) `import java.io.Scanner;`
(E) `using java.util.Scanner;`

2 Kullanıcı klavyeden `Ali Can Demir` yazıp Enter'a bastığında `String s = sc.next();` satırı `s` değişkenine ne atar?

(A) `"Ali Can Demir"` (B) `"Ali Can"` (C) `"Demir"` (D) `""` (boş String) (E) `"Ali"`

3 `System.out.printf("[%6.2f]", 3.14159);` komutunun çıktısı nedir? (Boşluklar ␣ ile gösterilmiştir.)

(A) `[␣␣3.14]` (B) `[3.14␣␣]` (C) `[3.1416]` (D) `[␣␣␣3.1]` (E) `[003.14]`

4 Aşağıdaki kodda kullanıcı önce `20` yazıp Enter'a basıyor. Bunun hemen ardından ne olur?

```
int no = sc.nextInt();
String isim = sc.nextLine();
```

(A) Program kullanıcının ismi yazmasını bekler.
(B) `isim` değişkenine `"20"` atanır.
(C) `InputMismatchException` oluşur.
(D) `isim` değişkenine boş String (`""`) atanır, program beklemez.
(E) Program derlenmez.

5 `printf` ile ekrana `%85 başarı` yazdırmak için hangisi kullanılmalıdır?

(A) `printf("%%d başarı", 85)`
(B) `printf("%d başarı", "%85")`
(C) `printf("%%%d başarı", 85)`
(D) `printf("%85 başarı")`
(E) `printf("\%%d başarı", 85)`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (Satırlardaki boşluk sayısına dikkat et.)

```
public class Soru6 {
    public static void main(String[] args) {
        System.out.printf("|%5s|%-5s|%n", "ab", "cd");
        System.out.printf("|%4d|%-4d|%04d|%n", 7, 7, 7);
        System.out.printf("|%.3f|%.0f|%7.2f|%n", 2.71828, 2.5, -3.14159);
        System.out.printf("%s-%c-%b%n", 10 + 5, 'x', 3 > 2);
    }
}
```

7 Kullanıcı klavyeden aşağıdaki girdiyi yazarsa program ne yazdırır?

```
12 5
Java Programlama
```

```java
import java.util.Scanner;

public class Soru7 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        String s1 = sc.nextLine();
        String s2 = sc.next();
        System.out.println(a / b + "|" + s1 + "|" + s2 + "|");
        System.out.println(a % b * 2.0);
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru8 {
    public static void main(String[] args) {
        int ogr = 1234;
        double ort = 72.456;
        String s = String.format("No:%06d Ort:%.1f", ogr, ort);
        System.out.println(s);
        System.out.println(s.length());
        System.out.printf("%,d TL%n", 2500000);
        System.out.printf("%d%%%n", 7 * 100 / 8);
    }
}
```

C) Kod Yazma

9 Kullanıcıdan iki ondalıklı sayı okuyan ve bu sayıların toplamını, farkını, çarpımını ve bölümünü **virgülden sonra 2 basamakla** aşağıdaki gibi yazdıran `DortIslem` programını yaz. Ondalık ayırıcı her bilgisayarda nokta olsun.

```
Birinci sayı: 7.5
İkinci sayı: 2
7.50 + 2.00 = 9.50
7.50 - 2.00 = 5.50
7.50 * 2.00 = 15.00
7.50 / 2.00 = 3.75
```

10 Kullanıcıdan önce **öğrenci numarasını** (tam sayı), sonra **ad soyadını** (boşluk içerebilir), sonra **vize** ve **final** notlarını (tam sayı) okuyan `NotHesap` programını yaz. Ortalama = vize × 0.4 + final × 0.6. Çıktı örneği:

```
Öğrenci no: 2026104
Ad soyad: Elif Su Demir
Vize: 65
Final: 78
2026104 – Elif Su Demir – Ortalama: 72.80
```

Çözümler

Cevap anahtarı

1	2	3	4	5
B	E	A	D	C

1 Cevap: B

`Scanner` sınıfı `java.util` paketindedir. Java'da `include` veya `using` yoktur. `java.lang` paketi otomatik içe aktarılır ama Scanner orada değildir.

2 Cevap: E

`next()` boşluğa kadar olan **tek bir kelimeyi** okur: `"Ali"`. Kalan `Can Demir` tamponda bekler ve sonraki okuma işlemiyle alınır.

3 Cevap: A

`%6.2f`: Toplam genişlik 6 karakter, virgülden sonra 2 basamak. `3.14` dört karakterdir; 6'ya tamamlamak için **soluna** 2 boşluk eklenir (varsayılan sağa yaslama).

4 Cevap: D

`nextInt()` sadece `20`'yi okur, Enter'ın ürettiği satır sonu tamponda kalır. `nextLine()` bu satır sonunu görünce hemen boş bir String döndürür. Kullanıcıya isim yazma fırsatı kalmaz.

5 Cevap: C

`%%` bir yüzde işareti yazdırır, ardından gelen `%d` 85'i yazdırır: `%%%d` → `%85`. A şıkkı `%d başarı` yazdırır (85 kullanılmaz). E şıkkı derlenmez, çünkü `\%` geçerli bir kaçış dizisi değildir.

6

- `%5s` → `ab`'nin soluna 3 boşluk; `%-5s` → `cd`'nin sağına 3 boşluk.
- `%4d` → `7`; `%-4d` → `7`; `%04d` → `0007`.
- `%.3f` → `2.71828` yuvarlanır: `2.718`. `%.0f` → `2.5` yarım olduğu için yukarı: `3`. `%7.2f` → `-3.14` (5 karakter), 7'ye tamamlamak için başına 2 boşluk.
- `10 + 5` → `15`, `'x'`, `3 > 2` → `true`.

Çıktı

```
|   ab|cd   |
|   7|7   |0007|
|2.718|3|  -3.14|
15-x-true
```

7

- `nextInt()` iki kere: `a = 12`, `b = 5`.
- `nextLine()`: İlk satırda `5`'ten sonra kalan **boş metni** okur → `s1 = ""`.
- `next()`: Yeni satırdan tek kelime → `s2 = "Java"`.

- `12 / 5 = 2` → `2||Java|`. İkinci satır: `12 % 5 = 2`, `2 * 2.0 = 4.0`.

Aşağıdaki kutuda turuncu satırlar klavyeden yazılan girdidir. Programın ekrana yazdırdığı yalnız son iki satırdır; sınavda cevap olarak `2||Java|` ve `4.0` satırlarını yazarsın.

```
Çıktı
12 5
Java Programlama
2||Java|
4.0
```

8

- `%06d` → 1234 altı haneye sıfırla tamamlanır: `001234`. `%.1f` → `72.456` → `72.5`.
- `s` = `No:001234 Ort:72.5` → 18 karakter.
- `%,d` → `2,500,000`.
- `7 * 100 / 8` = `700 / 8` = `87` (tam sayı bölmesi), ardından `%%` → `87%`.

```
Çıktı
No:001234 Ort:72.5
18
2,500,000 TL
87%
```

9

Ondalık ayırıcının her bilgisayarda nokta olması için Scanner'a `useLocale(Locale.US)` uygulanır. `printf` için de `Locale.US` verilebilir; burada kısa olsun diye `Locale.setDefault(Locale.US)` ile programın tamamı için ayarlıyoruz.

DortIslem.java

```java
import java.util.Locale;
import java.util.Scanner;

public class DortIslem {
    public static void main(String[] args) {
        Locale.setDefault(Locale.US);
        Scanner sc = new Scanner(System.in);

        System.out.print("Birinci sayı: ");
        double a = sc.nextDouble();
        System.out.print("İkinci sayı: ");
        double b = sc.nextDouble();

        System.out.printf("%.2f + %.2f = %.2f%n", a, b, a + b);
        System.out.printf("%.2f - %.2f = %.2f%n", a, b, a - b);
        System.out.printf("%.2f * %.2f = %.2f%n", a, b, a * b);
        System.out.printf("%.2f / %.2f = %.2f%n", a, b, a / b);
    }
}
```

```
Çıktı
Birinci sayı: 7.5
İkinci sayı: 2
7.50 + 2.00 = 9.50
7.50 - 2.00 = 5.50
7.50 * 2.00 = 15.00
7.50 / 2.00 = 3.75
```

10

Öğrenci numarası `nextInt()` ile okunduktan sonra ad soyad `nextLine()` ile okunacağı için araya **boş bir `nextLine()`** koymak şart. Notlar `int` olsa da `0.4` ile çarpım `double` verir.

NotHesap.java

```java
import java.util.Scanner;

public class NotHesap {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Öğrenci no: ");
        int no = sc.nextInt();
        sc.nextLine();                      // tuzağa düşme!
        System.out.print("Ad soyad: ");
        String ad = sc.nextLine();
        System.out.print("Vize: ");
        int vize = sc.nextInt();
        System.out.print("Final: ");
        int fin = sc.nextInt();

        double ort = vize * 0.4 + fin * 0.6;
        System.out.printf("%d - %s - Ortalama: %.2f%n", no, ad, ort);
    }
}
```

```
Çıktı
Öğrenci no: 2026104
Ad soyad: Elif Su Demir
Vize: 65
Final: 78
2026104 - Elif Su Demir - Ortalama: 72.80
```

Değişken adı olarak `final` kullanamayacağımıza dikkat et; `final` bir anahtar kelimedir. Bu yüzden `fin` dedik.

BÖLÜM 4

Koşullu İfadeler

Bu Bölümde Öğreneceklerin

- İlişkisel operatörlerle (`==`, `!=`, `<`, `>`, `<=`, `>=`) `boolean` ifadeler kurmayı
- `if`, `if-else` ve `else-if` merdiveniyle programın akışını dallandırmayı
- Süslü parantezsiz `if`, sarkan `else` ve `if (x = 5)` tuzaklarını
- `&&`, `||`, `!` operatörlerini, **kısa devre** davranışını ve De Morgan kurallarını
- Operatör öncelik sırasını ve üçlü operatörü (`?:`)
- Klasik `switch` (`break`, fall-through) ve yeni `switch` ifadesini (`->`)
- String'leri ve ondalıklı sayıları doğru karşılaştırmayı

Şimdiye kadar yazdığın programlar yukarıdan aşağı her satırı sırayla çalıştırdı. Gerçek programlar ise **karar verir**: "Not 50'den büyükse *geçti* yaz, değilse *kaldı* yaz." Bu bölümde programın akışını koşullara göre dallandırmayı öğreneceksin.

4.1 İlişkisel operatörler ve boolean ifadeler

Karar vermek için önce bir soru sormak gerekir: "a, b'den büyük mü?" Bu soruların cevabı her zaman `true` ya da `false`'tur, yani bir `boolean` değerdir (Bölüm 2). Soru sormak için **ilişkisel operatörler** (relational operators) kullanılır:

Operatör	Anlamı	Örnek (`a = 7`, `b = 3`)	Sonuç
`==`	Eşit mi?	`a == b`	`false`
`!=`	Eşit değil mi?	`a != b`	`true`
`>`	Büyük mü?	`a > b`	`true`
`<`	Küçük mü?	`a < b`	`false`
`>=`	Büyük veya eşit mi?	`a >= 7`	`true`
`<=`	Küçük veya eşit mi?	`a <= 6`	`false`

Sonucu `boolean` olan ifadelere **boolean ifade** (boolean expression) ya da kısaca **koşul** (condition) denir. Sonucu bir değişkende saklayabilir veya ekrana yazdırabilirsin:

Iliskisel.java

```java
public class Iliskisel {
    public static void main(String[] args) {
        int a = 7, b = 3;
        System.out.println(a == b);
        System.out.println(a != b);
        System.out.println(a <= 6);

        boolean gecti = a * 10 >= 50;        // önce 7 * 10, sonra >=
        System.out.println("Geçti mi? " + gecti);
        System.out.println('a' < 'b');       // 97 < 98
    }
}
```

Çıktı

```
false
true
false
Geçti mi? true
true
```

Aritmetik işlemler karşılaştırmadan **önce** yapılır: `a * 10 >= 50` ifadesi `70 >= 50` olarak hesaplanır. Karakterler ise Unicode numaralarıyla karşılaştırılır (`'a'` = 97, `'b'` = 98).

> **Sık Yapılan Hata: `=` ile `==` karıştırma**
>
> `=` **atama**, `==` **karşılaştırma** operatörüdür. `a = b` "b'nin değerini a'ya koy", `a == b` ise "a ile b eşit mi?" demektir. Bu karışıklığın `if` içinde neye yol açtığını birazdan göreceksin.

4.2 if ifadesi

En temel karar yapısı `if`'tir (eğer):

```java
if (koşul) {
    // koşul true ise çalışacak komutlar
}
```

Koşul **parantez içinde** yazılır. Koşul `true` ise blok çalışır; `false` ise blok atlanır ve program bloktan sonraki satırdan devam eder.

IfOrnek.java

```java
import java.util.Scanner;

public class IfOrnek {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Sıcaklık: ");
        int derece = sc.nextInt();

        if (derece > 30) {
            System.out.println("Hava çok sıcak, bol su iç!");
        }
        System.out.println("İyi günler.");
    }
}
```

Çıktı
```
Sıcaklık: 34
Hava çok sıcak, bol su iç!
İyi günler.
```

Sıcaklık 30 veya daha düşük girilseydi blok atlanır, sadece `İyi günler.` yazılırdı.

Süslü parantezsiz if

Blokta **tek bir komut** varsa süslü parantezler yazılmayabilir. Ama bu durumda `if`'e sadece **hemen ardından gelen tek komut** bağlıdır. Girinti (indentation) Java için hiçbir anlam taşımaz:

ParantezsizIf.java
```java
public class ParantezsizIf {
    public static void main(String[] args) {
        int bakiye = 50;
        if (bakiye >= 100)
            System.out.println("Ödeme yapıldı.");
            System.out.println("Bakiye düşüldü.");   // if'e dahil DEĞİL!
        System.out.println("Bitti.");
    }
}
```

Çıktı
```
Bakiye düşüldü.
Bitti.
```

`Bakiye düşüldü.` satırı `if`'in içindeymiş gibi girintili yazılmış, ama `if`'e sadece ilk `println` bağlı. Koşul `false` olduğu halde ikinci satır çalıştı.

> **İpucu**
>
> Tek satır bile olsa **her zaman süslü parantez kullan**. Sonradan bloğa ikinci bir satır eklediğinde bu tuzağa düşme ihtimalin kalmaz. VS Code'da **Shift+Alt+F** (macOS: **Shift+Option+F**) kodu otomatik biçimlendirir ve girintiyi gerçek yapıya göre düzeltir; böyle hatalar hemen göze batar.

> **Sık Yapılan Hata: if'ten sonra noktalı virgül**
>
> `if (not >= 50);` satırının sonundaki `;` **boş bir komuttur**: "Koşul doğruysa hiçbir şey yapma." Altına yazdığın `{ ... }` bloğu artık `if`'e bağlı değildir ve koşul ne olursa olsun **her zaman** çalışır. Derleyici hata vermez; bu yüzden bulması zordur.

Koşul boolean olmak zorunda

C ve C++'ta `if (x = 5)` yazmak derlenir. Java ise koşulun **mutlaka** `boolean` olmasını ister. `x = 5` bir atamadır ve değeri `int` (5) olduğu için program derlenmez:

AtamaKosul.java

```java
public class AtamaKosul {
    public static void main(String[] args) {
        int x = 3;
        if (x = 5) {
            System.out.println("x beştir");
        }
    }
}
```

Derleyici çıktısı – program derlenmez

```
AtamaKosul.java:4: error: incompatible types: int cannot be converted to boolean
        if (x = 5) {
              ^
1 error
```

Sınavda Çıkar: `boolean` değişkende atama tuzağı

`if (x = 5)` derlenmez, ama değişken `boolean` ise atama **derlenir**! `boolean bitti = false; if (bitti = true) {...}` yazarsan `bitti = true` atamasının değeri `true` olduğu için blok **her zaman** çalışır, üstelik `bitti` değişkeni de `true` olur. `boolean` bir değişkeni kontrol ederken `== true` yazmana gerek yok: `if (bitti)` ve `if (!bitti)` hem kısa hem güvenlidir.

4.3 if-else

Koşul yanlış olduğunda da bir şey yapmak istiyorsan `else` (değilse) kullanırsın. İki bloktan **tam olarak biri** çalışır:

TekCift.java

```java
import java.util.Scanner;

public class TekCift {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Bir sayı gir: ");
        int sayi = sc.nextInt();

        if (sayi % 2 == 0) {
            System.out.println(sayi + " çift sayıdır.");
        } else {
            System.out.println(sayi + " tek sayıdır.");
        }
    }
}
```

Çıktı

```
Bir sayı gir: 17
17 tek sayıdır.
```

Sık Yapılan Hata: Negatif tek sayılar

Tek sayı kontrolünü `sayi % 2 == 1` diye yazma. Bölüm 2'de gördüğün gibi `%` sonucunun işareti soldaki sayının işaretiyle aynıdır: `-7 % 2` sonucu `-1`'dir. Yani `-7 % 2 == 1` ifadesi **false** olur ve -7 tek sayı sayılmaz. Güvenli yol: Çiftlik için `sayi % 2 == 0`, teklik için `sayi % 2 != 0`.

4.4 else-if merdiveni

İkiden fazla seçenek varsa `else if` ile koşulları zincirlersin. Koşullar **yukarıdan aşağı** sırayla denenir; **ilk doğru olan** koşulun bloğu çalışır ve merdivenin geri kalanı atlanır. Hiçbiri doğru değilse en sondaki `else` çalışır:

HarfNotu.java

```
import java.util.Scanner;

public class HarfNotu {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Puanın (0–100): ");
        int puan = sc.nextInt();
        String harf;

        if (puan >= 90) {
            harf = "AA";
        } else if (puan >= 80) {
            harf = "BB";
        } else if (puan >= 70) {
            harf = "CC";
        } else if (puan >= 60) {
            harf = "DD";
        } else {
            harf = "FF";
        }
        System.out.println("Harf notun: " + harf);
    }
}
```

Çıktı

```
Puanın (0–100): 78
Harf notun: CC
```

78 için `puan >= 90` ve `puan >= 80` yanlış, `puan >= 70` doğru: harf `CC` olur. Alttaki `puan >= 60` da doğrudur ama artık **denenmez**. Bu yüzden `else if (puan >= 70 && puan < 80)` yazmana gerek yok; o satıra gelindiyse puanın 80'den küçük olduğu zaten bellidir. (Gerçek harf aralıkları üniversiteden üniversiteye değişir.)

Merdiven yerine art arda **bağımsız** `if`'ler yazsaydık her koşul ayrı ayrı denenir ve 78 puan için `CC`, ardından `DD` ataması da yapılırdı. "Sadece biri çalışsın" istiyorsan `else if` kullan.

> **Sınavda Çıkar: Koşulların sırası önemli**
>
> Merdiveni ters sırayla yazarsan (`if (puan >= 60) ... else if (puan >= 70) ... else if (puan >= 90)`) program derlenir ama yanlış çalışır: 95 puan ilk koşul olan `puan >= 60`'ı sağladığı için `DD` alır, alttaki koşullara hiç bakılmaz. Kural: `>=` ile kurulan merdivenlerde **en büyük eşikten başla**.

4.5 İç içe if ve sarkan else

Bir `if` bloğunun içine başka bir `if` yazılabilir; buna **iç içe if** (nested if) denir. Klasik örnek, üç sayının en büyüğünü bulmak:

EnBuyuk.java

```
public class EnBuyuk {
    public static void main(String[] args) {
        int a = 12, b = 45, c = 7;
        int enBuyuk;
```

```
        if (a >= b) {
            if (a >= c) {
                enBuyuk = a;
            } else {
                enBuyuk = c;
            }
        } else {
            if (b >= c) {
                enBuyuk = b;
            } else {
                enBuyuk = c;
            }
        }
        System.out.println("En büyük: " + enBuyuk);
    }
}
```

Çıktı

```
En büyük: 45
```

İpucu: Daha kısa yol: "şampiyon" yöntemi

İlk sayıyı şampiyon kabul et; her yeni sayı şampiyondan büyükse şampiyon o olsun: `int enBuyuk = a; if (b > enBuyuk) enBuyuk = b; if (c > enBuyuk) enBuyuk = c;` Bu yöntem sayı adedi arttıkça da işe yarar; Bölüm 5'te döngülerle tekrar kullanacağız. Hazır metot istersen: `Math.max(a, Math.max(b, c))`.

Sarkan else (dangling else)

Süslü parantez kullanmadan iç içe `if` yazarsan "Bu `else` hangi `if`'e ait?" sorusu ortaya çıkar. Java'nın kuralı: **`else`, kendisinden önceki en yakın ve henüz `else`'i olmayan `if`'e aittir.** Girintinin yine hiçbir önemi yoktur:

SarkanElse.java

```
public class SarkanElse {
    public static void main(String[] args) {
        int x = 5, y = 15;

        // Girinti yanıltıcı: else aslında içteki if'e ait
        if (x > 10)
            if (y > 10)
                System.out.println("1: İkisi de büyük");
        else
            System.out.println("1: x küçük");

        // Süslü parantezle niyetimizi açıkça yazdık
        if (x > 10) {
            if (y > 10) {
                System.out.println("2: İkisi de büyük");
            }
        } else {
            System.out.println("2: x küçük");
        }
    }
}
```

```
Çıktı
2: x küçük
```

İlk yapıda `else`, girintiye bakılırsa dıştaki `if`'e aitmiş gibi duruyor; ama Java onu içteki `if (y > 10)`'a bağlar. Dıştaki `x > 10` yanlış olduğu için içteki `if` ve onun `else`'i **tamamen** atlanır, ekrana hiçbir şey yazılmaz. İkinci yapıda süslü parantezler `else`'i dıştaki `if`'e bağladı.

> **Sınavda Çıkar**
>
> Sarkan `else` çıktı sorularının favorisidir. Çözerken **girintiyi tamamen görmezden gel**: Her `else`'i yukarıya doğru giderek en yakın, boşta kalan `if` ile eşleştir, sonra süslü parantezleri kafanda kendin yerleştir.

4.6 Mantıksal operatörler

Birden fazla koşulu birleştirmek için **mantıksal operatörler** (logical operators) kullanılır:

Operatör	Adı	Ne zaman `true`?	Örnek
`&&`	VE (AND)	İki koşul da doğruysa	`yas >= 18 && ehliyetVar`
`||`	VEYA (OR)	En az biri doğruysa	`gun == 6 || gun == 7`
`!`	DEĞİL (NOT)	Koşul yanlışsa (tersine çevirir)	`!bitti`

Doğruluk tablosu (truth table):

`a`	`b`	`a && b`	`a || b`	`!a`
`true`	`true`	`true`	`true`	`false`
`true`	`false`	`false`	`true`	`false`
`false`	`true`	`false`	`true`	`true`
`false`	`false`	`false`	`false`	`true`

Kısaca: `&&` sadece "ikisi de doğru" satırında doğru; `||` sadece "ikisi de yanlış" satırında yanlış.

Klasik bir örnek **artık yıl** (leap year) kontrolüdür. Bir yıl artık yıldır (Şubat 29 gün çeker) ancak şu durumda: **4'e bölünür ve 100'e bölünmez; ya da 400'e bölünür.**

ArtikYil.java

```java
import java.util.Scanner;

public class ArtikYil {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Yıl: ");
        int yil = sc.nextInt();

        boolean artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;

        if (artik) {
            System.out.println(yil + " artık yıldır, Şubat 29 gün.");
        } else {
            System.out.println(yil + " artık yıl değildir, Şubat 28 gün.");
        }
    }
}
```

Çıktı

```
Yıl: 1900
1900 artık yıl değildir, Şubat 28 gün.
```

1900, 4'e bölünür ama 100'e de bölünür ve 400'e bölünmez; bu yüzden artık yıl değildir. 2000 ise 400'e bölündüğü için artık yıldır. Sınavda en çok bu iki yıl sorulur.

> **Sık Yapılan Hata: Matematikteki gibi aralık yazmak**
>
> Matematikte `0 ≤ puan ≤ 100` yazılır, ama Java'da `if (0 <= puan <= 100)` **derlenmez** (*bad operand types for binary operator '<='*). Önce `0 <= puan` hesaplanıp `true` ya da `false` olur, sonra `true <= 100` gibi anlamsız bir karşılaştırma denenir. Doğrusu iki koşulu `&&` ile bağlamaktır: `puan >= 0 && puan <= 100`. Aynı sebeple `a == b == c` de derlenmez; `a == b && b == c` yazmalısın.

Kısa devre değerlendirme

`&&` ve `||` **kısa devre** (short-circuit) operatörleridir: Sonuç sol taraftan belli oluyorsa sağ taraf **hiç hesaplanmaz**.

- `false && ...` → sağ taraf ne olursa olsun sonuç `false`; sağ taraf atlanır.
- `true || ...` → sağ taraf ne olursa olsun sonuç `true`; sağ taraf atlanır.

Bu özellik sayesinde tehlikeli bir işlemi bir koşulla "koruyabilirsin":

KisaDevre.java

```java
public class KisaDevre {
    public static void main(String[] args) {
        int x = 0;
        if (x != 0 && 10 / x > 1) {
            System.out.println("10 / x, 1'den büyük");
        } else {
            System.out.println("Sıfıra bölme yapılmadı.");
        }

        int sayac = 0;
        boolean s1 = (5 > 3) || (++sayac > 0);   // sağ taraf atlanır
        boolean s2 = (5 > 3) | (++sayac > 0);    // iki taraf da hesaplanır
        System.out.println(s1 + " " + s2 + " sayac=" + sayac);
    }
}
```

Çıktı

```
Sıfıra bölme yapılmadı.
true true sayac=1
```

`x != 0` yanlış olduğu için `10 / x` hiç hesaplanmadı ve program çökmedi. Koşulların sırasını ters yazsaydın (`10 / x > 1 && x != 0`), önce bölme yapılır ve `ArithmeticException` oluşurdu. **Koruyucu koşul her zaman solda olmalı.**

Tek karakterli `&` ve `|` de `boolean`'larla VE/VEYA yapar ama kısa devre **yapmaz**; iki tarafı da her zaman hesaplar. Yukarıda `sayac` sadece bir kez arttı: `||`'nin sağ tarafı atlandı, `|`'nün sağ tarafı çalıştı. Aynı korumayı `x != 0 & 10 / x > 1` diye tek `&` ile yazsaydın, bölme yine yapılır ve program `ArithmeticException` ile çökerdi.

> **Sınavda Çıkar**
>
> Kısa devre soruları genelde koşulun içine `++` gizler: `if (a > 5 && ++b > 0)`. Önce sol tarafı hesapla; sonuç belli oluyorsa sağdaki `++` **hiç çalışmaz** ve `b` artmaz. `&&` ve `||` kısa devre yapar; `&` ve `|` yapmaz.

De Morgan kuralları

Bir koşulun **tersini** yazman gerektiğinde **De Morgan kuralları** kullanılır:

- `!(a && b)` ≡ `!a || !b`
- `!(a || b)` ≡ `!a && !b`

Yani `!` parantezin içine girerken her koşulu tersine çevirir ve `&&` ile `||` **yer değiştirir**. Karşılaştırmaların tersleri: `<` ↔ `>=`, `>` ↔ `<=`, `==` ↔ `!=`. Örneğin "puan geçerli değil" koşulu `!(puan >= 0 && puan <= 100)` yerine `puan < 0 || puan > 100` diye yazılabilir.

> **Sık Yapılan Hata: Tersini alırken && kalırsa**
>
> "Puan geçersiz" koşulunu `puan < 0 && puan > 100` diye yazmak çok sık yapılan bir hatadır. Bir sayı aynı anda hem 0'dan küçük hem 100'den büyük olamayacağı için bu koşul **hiçbir zaman** doğru olmaz. Karşılaştırmaları çevirirken `&&`'yi de `||`'ye çevirmeyi unutma.

4.7 Operatör önceliği

Bölüm 2'de aritmetik operatörlerin önceliğini gördün. Karşılaştırma ve mantıksal operatörler eklenince tablo şöyle olur (yukarıdan aşağı, öncelik azalır):

Öncelik	Operatörler	Açıklama
1	`()`	Parantez
2	`!`, `++`, `--`, tekli `-`, `(tip)`	Tekli (unary) operatörler
3	`*`, `/`, `%`	Çarpma, bölme, mod
4	`+`, `-`	Toplama, çıkarma, String birleştirme
5	`<`, `>`, `<=`, `>=`	Büyüklük karşılaştırma
6	`==`, `!=`	Eşitlik
7	`&&`	Mantıksal VE
8	`||`	Mantıksal VEYA
9	`?:`	Üçlü operatör
10	`=`, `+=`, `-=`, ...	Atama (en son)

Tek karakterli `&` ve `|` bu tabloda `==` ile `&&` arasında yer alır. Ezber cümlesi: **"Tekli, aritmetik, karşılaştırma, mantık, atama."** En önemli ayrıntı: **`&&`, `||`'den önce gelir** (çarpmanın toplamadan önce gelmesi gibi).

OncelikMantik.java

```java
public class OncelikMantik {
    public static void main(String[] args) {
        int a = 5, b = 10;
        System.out.println(a + 5 > b - 1);            // 10 > 9
        System.out.println(true || false && false);   // && önce
        System.out.println((true || false) && false); // parantez önce
        System.out.println(!(a > 3) || b == 10);
        System.out.println(a > 3 == b > 3);           // true == true
    }
}
```

Çıktı

```
true
true
false
true
true
```

İkinci satırda önce `false && false` → `false` hesaplanır, sonra `true || false` → `true`. Parantez sırayı değiştirince sonuç `false` oldu.

> **Sık Yapılan Hata: String birleştirme ile karşılaştırma**
>
> `System.out.println("Sonuç: " + a > b);` **derlenmez**. `+` operatörü `>`'den önce geldiği için önce `"Sonuç: " + a` hesaplanır ve bir String olur; sonra String ile sayı `>` ile karşılaştırılmaya çalışılır (*bad operand types for binary operator '>'*). Çözüm parantez: `"Sonuç: " + (a > b)`.

4.8 Üçlü operatör (?:)

Bir koşula göre iki değerden birini seçmenin kısa yolu **üçlü operatördür** (ternary / conditional operator). Java'nın üç işlenen alan tek operatörüdür:

```
koşul ? doğruysaDeğer : yanlışsaDeğer
```

Uclu.java

```
public class Uclu {
    public static void main(String[] args) {
        int a = 8, b = 13;
        int buyuk = (a > b) ? a : b;
        System.out.println("Büyük olan: " + buyuk);

        int x = -6;
        int mutlak = x < 0 ? -x : x;
        System.out.println("|" + x + "| = " + mutlak);

        int puan = 64;
        System.out.println("Durum: " + (puan >= 50 ? "Geçti" : "Kaldı"));
        System.out.println(true ? 1 : 2.0);       // 1 mi, 1.0 mı?
    }
}
```

Çıktı

```
Büyük olan: 13
|-6| = 6
Durum: Geçti
1.0
```

`int buyuk = (a > b) ? a : b;` satırı, dört satırlık bir `if-else`'in kısa halidir. Üçlü operatör bir **değer** ürettiği için doğrudan atamada veya `println` içinde kullanılabilir. `println` içinde parantez şarttır, çünkü `+` operatörü `?:`'den önce gelir.

> **Sınavda Çıkar: Üçlü operatörde tip yükseltme**
>
> İki seçenek farklı sayısal tipteyse sonuç **büyük olan tipe** dönüştürülür. Bu yüzden yukarıdaki programın son satırı `1` değil `1.0` yazdırdı: `int` ile `double` karşılaşınca sonuç `double` olur. Ayrıca iç içe üçlü operatör (`a > b ? (a > c ? a : c) : ...`) okunması zor koddur; birden fazla koşul varsa `if-else` tercih et.

4.9 switch ifadesi

Bir değişkenin **belirli değerlerine** göre farklı işler yapacaksan uzun bir else-if merdiveni yerine `switch` kullanabilirsin:

Menu.java

```java
public class Menu {
    public static void main(String[] args) {
        int secim = 2;
        switch (secim) {
            case 1:
                System.out.println("Bakiye görüntüleniyor");
                break;
            case 2:
                System.out.println("Para yatırma ekranı");
                break;
            case 3:
                System.out.println("Çıkış yapılıyor");
                break;
            default:
                System.out.println("Geçersiz seçim!");
        }
        System.out.println("İşlem tamam.");
    }
}
```

Çıktı

```
Para yatırma ekranı
İşlem tamam.
```

Çalışma mantığı:

1. Parantezdeki değer hesaplanır (`secim` → 2).
2. Değeri eşleşen `case` etiketine atlanır (`case 2:`).
3. Oradan başlayarak komutlar çalışır; **`break`** görülünce switch'ten çıkılır.
4. Hiçbir `case` eşleşmezse `default` çalışır. `default` zorunlu değildir; genelde en sona yazılır.

break unutulursa: fall-through

`break` yazmazsan Java eşleşen case'ten sonra **durmaz**; alttaki case'lerin komutlarını da etiketlere bakmadan çalıştırmaya devam eder. Buna **fall-through** (aşağı düşme) denir:

```java
switch (secim) {                    // break'ler silindi!
    case 1:
        System.out.println("Bakiye görüntüleniyor");
    case 2:
        System.out.println("Para yatırma ekranı");
    case 3:
        System.out.println("Çıkış yapılıyor");
    default:
        System.out.println("Geçersiz seçim!");
}
```

Çıktı

```
Para yatırma ekranı
Çıkış yapılıyor
Geçersiz seçim!
```

> **Sınavda Çıkar**
>
> "switch çıktısı" sorularında önce eşleşen case'i bul, sonra **ilk `break`'e veya switch'in sonuna kadar** her satırı çalıştır. Eşleşen case yoksa `default`'tan başla; `default` ortadaysa ve altında `break` yoksa onun altındaki case'ler de çalışır.

Örnek: switch ile hesap makinesi

Fall-through bazen işe de yarar: Birkaç case aynı işi yapacaksa onları **alt alta** yazıp gruplarsın. Aşağıdaki hesap makinesinde kullanıcı `15 / 4` gibi bir işlem yazıyor; `switch` bu kez bir `char` ile çalışıyor. Tek karakter okumak için Bölüm 3'teki `sc.next().charAt(0)` kalıbını kullanıyoruz:

HesapMakinesi.java

```
import java.util.Locale;
import java.util.Scanner;

public class HesapMakinesi {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);
        System.out.print("İşlem (ör. 12 * 4): ");
        double a = sc.nextDouble();
        char op = sc.next().charAt(0);
        double b = sc.nextDouble();

        switch (op) {
            case '+': System.out.printf("Sonuç: %.2f%n", a + b); break;
            case '-': System.out.printf("Sonuç: %.2f%n", a - b); break;
            case '*':
            case 'x': System.out.printf("Sonuç: %.2f%n", a * b); break;
            case '/':
                if (b == 0) {
                    System.out.println("Sıfıra bölünemez!");
                } else {
                    System.out.printf("Sonuç: %.2f%n", a / b);
                }
                break;
            default: System.out.println("Bilinmeyen işlem: " + op);
        }
    }
}
```

Çıktı

```
İşlem (ör. 12 * 4): 15 / 4
Sonuç: 3.75
```

`case '*':` ve `case 'x':` gruplandığı için kullanıcı çarpma için iki karakterden birini yazabilir. Kısa case'ler tek satırda yazılabilir; `case '/'`'de olduğu gibi bir case'in içine `if` de yazılabilir. `switch` **String** ile de çalışır (`case "Pazar":`); karşılaştırma büyük/küçük harf duyarlıdır.

> **Not: switch'te hangi tipler kullanılabilir?**
>
> `int`, `char`, `byte`, `short`, bunların sarmalayıcıları `Integer`, `Character`, `Byte`, `Short` (Bölüm 13), `String` ve `enum` (Bölüm 11). `long`, `float`, `double` ve `boolean` ile klasik `switch` **derlenmez**. Ayrıca `case` etiketleri **sabit** olmalıdır (literal ya da `final` sabit; sıradan bir değişken olamaz) ve aynı değer iki kez yazılamaz. Aralık kontrolü (`puan >= 90`) gerekiyorsa `switch` değil `if` kullan.

4.10 Yeni switch: ok (->) sözdizimi

Java 14'ten beri `switch`'in daha güvenli bir yazımı var. Etiketten sonra `:` yerine `->` kullanılır:

- **Fall-through yoktur**, `break` gerekmez; sadece eşleşen case çalışır.
- Bir case'e birden fazla değer virgülle yazılabilir: `case 6, 7 ->`.
- `switch` bir **değer üretebilir** ve bu değer bir değişkene atanabilir (*switch expression*).

YeniSwitch.java

```
public class YeniSwitch {
    public static void main(String[] args) {
        int ay = 4;
        String mevsim = switch (ay) {
            case 12, 1, 2 -> "Kış";
            case 3, 4, 5 -> "İlkbahar";
            case 6, 7, 8 -> "Yaz";
            case 9, 10, 11 -> "Sonbahar";
            default -> "Geçersiz ay";
        };
        System.out.println(ay + ". ay: " + mevsim);

        char harf = 'B';
        int katsayi = switch (harf) {
            case 'A' -> 4;
            case 'B' -> {
                System.out.println("B için hesap yapılıyor...");
                yield 3;
            }
            default -> 0;
        };
        System.out.println("Katsayı: " + katsayi);
    }
}
```

Çıktı

```
4. ay: İlkbahar
B için hesap yapılıyor...
Katsayı: 3
```

- `String mevsim = switch (ay) { ... };` bir atama komutudur; bu yüzden kapanan süslü parantezden sonra `;` gelir.
- Bir case'te birden fazla komut gerekiyorsa süslü parantezli blok yazılır ve üretilecek değer **`yield`** ile verilir (`return` değil).

	Klasik `case X:`	Yeni `case X ->`
Fall-through	Var, `break` gerekir	Yok
Çoklu değer	`case 1: case 2:`	`case 1, 2 ->`
Değer üretme	Hayır	Evet (blok içinde `yield`)
Tüm değerleri kapsama	Zorunlu değil	Değer üretiyorsa zorunlu

> **Sınavda Çıkar: switch ifadesi her durumu kapsamalı**
>
> Değer üreten bir `switch` **her olası değer** için bir sonuç vermek zorundadır. `int` veya `String` ile kullanılıyorsa `default` yazmak şarttır. Yukarıdaki `mevsim` örneğinden `default` satırını silersen program derlenmez: *the switch expression does not cover all possible input values*.

4.11 String ve double karşılaştırma

String'ler == ile karşılaştırılmaz

`==` ilkel tiplerde **değerleri** karşılaştırır. String ise bir sınıftır (Bölüm 2'deki nota bak) ve `==` iki String'in **aynı nesne** olup olmadığına bakar, içeriklerine değil. İçeriği karşılaştırmak için `equals` metodu kullanılır:

StringEsitlik.java

```
import java.util.Scanner;

public class StringEsitlik {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Devam edilsin mi? ");
        String cevap = sc.next();

        System.out.println("==               : " + (cevap == "evet"));
        System.out.println("equals           : " + cevap.equals("evet"));
        System.out.println("equalsIgnoreCase : " + cevap.equalsIgnoreCase("EVET"));
    }
}
```

Çıktı

```
Devam edilsin mi? evet
==               : false
equals           : true
equalsIgnoreCase : true
```

Kullanıcı tam olarak `evet` yazdığı halde `==` **false** verdi. `equalsIgnoreCase` ise büyük/küçük harf farkını önemsemez. Bunun nedenini ve String'in diğer metotlarını Bölüm 8'de ayrıntılı göreceksin. Şimdilik kural: **String'leri her zaman `equals` ile karşılaştır.**

> **Sık Yapılan Hata: Bazen çalışan yanlış kod**
>
> `if (sifre == "1234")` gibi bir kod, iki metin de koda doğrudan yazıldığında doğru çalışıyormuş gibi görünebilir; ama kullanıcıdan okunan metinlerde başarısız olur. Bu tutarsızlık hatayı daha da sinsi yapar. Doğrusu: `sifre.equals("1234")`.

Ondalıklı sayılar

Bölüm 2'de `0.1 + 0.2 == 0.3` ifadesinin `false` olduğunu gördün. Bu yüzden hesaplama sonucu olan `double`'ları `if` içinde `==` ile karşılaştırma; aralarındaki farkın çok küçük bir **tolerans** değerinden (epsilon) küçük olup olmadığına bak:

```
if (Math.abs(toplam - 0.3) < 1e-9) { ... }   // == yerine
```

Hesap makinesi örneğindeki `b == 0` kontrolü ise sorun değildir: Kullanıcının yazdığı `0` tam olarak `0.0` olarak okunur. Sorun, **hesaplama sonucunda** oluşan küçük yuvarlama farklarındadır.

4.12 Örnek uygulamalar

VKİ yorumu

Bölüm 3'teki vücut kitle indeksi programını tamamlayalım. Dünya Sağlık Örgütü'nün sınıflandırması: 18.5'in altı **zayıf**, 25'in altı **normal**, 30'un altı **fazla kilolu**, 30 ve üzeri **obez**. Okuma kısmı Bölüm 3'tekiyle aynı olduğu için değerleri doğrudan yazıyoruz:

VkiYorum.java

```
public class VkiYorum {
    public static void main(String[] args) {
        double kilo = 84, boy = 1.75;
        double vki = kilo / (boy * boy);
        String durum;
        if (vki < 18.5) {
            durum = "zayıf";
        } else if (vki < 25) {
            durum = "normal";
        } else if (vki < 30) {
            durum = "fazla kilolu";
        } else {
            durum = "obez";
        }
        System.out.printf("VKİ: %.1f (%s)%n", vki, durum);
    }
}
```

Çıktı

```
VKİ: 27.4 (fazla kilolu)
```

Girdi doğrulama: üçgen türü

Kullanıcıya güvenme; girilen değerin anlamlı olup olmadığını **önce** kontrol et. Üç kenar uzunluğundan üçgen olabilmesi için her kenar, diğer iki kenarın toplamından küçük olmalıdır (üçgen eşitsizliği):

Ucgen.java

```
import java.util.Scanner;

public class Ucgen {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Üç kenar: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = sc.nextInt();

        if (a <= 0 || b <= 0 || c <= 0) {
            System.out.println("Hata: Kenarlar pozitif olmalı.");
        } else if (a + b <= c || a + c <= b || b + c <= a) {
            System.out.println("Bu kenarlarla üçgen çizilemez.");
        } else if (a == b && b == c) {
            System.out.println("Eşkenar üçgen");
        } else if (a == b || a == c || b == c) {
            System.out.println("İkizkenar üçgen");
        } else {
```

```
            System.out.println("Çeşitkenar üçgen");
        }
    }
}
```

Çıktı

```
Üç kenar: 5 5 8
İkizkenar üçgen
```

Merdivenin sırasına dikkat et: Önce **geçersiz** durumlar elenir, sonra asıl sınıflandırma yapılır. Eşkenar kontrolü ikizkenardan önce gelmelidir; aksi halde eşkenar üçgen de "ikizkenar" diye yazılırdı. Kullanıcı sayı yerine harf yazarsa ne yapacağımızı (`hasNextInt()`) Bölüm 5'te döngülerle birlikte göreceksin.

Bölüm Özeti – Bilmen Gerekenler

- İlişkisel operatörler (`==`, `!=`, `<`, `>`, `<=`, `>=`) `boolean` sonuç üretir. `=` atama, `==` karşılaştırmadır.
- `if` koşulu **mutlaka** `boolean` olmalıdır: `if (x = 5)` derlenmez (x `int` ise). `boolean` değişkende `if (b = true)` derlenir ve hep doğrudur.
- Süslü parantez yoksa `if`'e sadece **bir** komut bağlıdır; girinti önemsizdir. `if (...);` boş komuttur.
- else-if merdiveninde **ilk doğru** koşul çalışır; `>=` merdivenlerinde en büyük eşikten başla.
- **Sarkan else:** `else`, en yakın boşta kalan `if`'e aittir.
- `&&` (VE), `||` (VEYA), `!` (DEĞİL). `&&` ve `||` **kısa devre** yapar; `&` ve `|` iki tarafı da hesaplar. Koruyucu koşul solda: `x != 0 && 10 / x > 1`.
- De Morgan: `!(a && b)` = `!a || !b`, `!(a || b)` = `!a && !b`.
- Öncelik: `!` > aritmetik > `< > <= >=` > `== !=` > `&&` > `||` > `?:` > `=`.
- `koşul ? a : b` bir değer üretir; farklı sayısal tiplerde büyük tipe yükseltilir (`true ? 1 : 2.0` → `1.0`).
- Klasik `switch`'te `break` unutulursa **fall-through** olur. `long`, `double`, `boolean` ile kullanılamaz; case'ler sabit olmalı.
- Yeni `switch` (`->`): fall-through yok, `case 1, 2 ->`, değer üretebilir (`yield`); değer üretiyorsa `default` şart.
- String'ler `equals` ile, `double`'lar toleransla (`Math.abs(a - b) < 1e-9`) karşılaştırılır.

Sorular

A) Çoktan Seçmeli

1 `!(x > 5 && y <= 10)` ifadesine **denk** olan ifade hangisidir?

(A) `x <= 5 && y > 10`

(B) `x < 5 || y >= 10`

(C) `x > 5 || y <= 10`

(D) `x <= 5 || y > 10`

(E) `x < 5 && y > 10`

2 Aşağıdaki kod parçası çalıştığında ekrana ne yazdırılır?

```
int a = 3, b = 0;
if (a > 5 && ++b > 0) {
    a++;
}
if (a < 5 || ++b > 0) {
    a--;
}
System.out.println(a + " " + b);
```

(A) `2 0` (B) `2 1` (C) `4 1` (D) `3 0` (E) `2 2`

3 `int n` değişkeninin **tek** sayı olup olmadığını, negatif sayılar dahil her zaman doğru kontrol eden koşul hangisidir?

(A) `n % 2 == 1` (B) `n / 2 == 1` (C) `n % 2 != 0` (D) `n % 2 == -1` (E) `n % 2 > 0`

4 Aşağıdaki kod parçası çalıştığında ekrana ne yazdırılır?

```
int k = 2;
switch (k) {
    case 1: System.out.print("A");
    case 2: System.out.print("B");
    case 3: System.out.print("C"); break;
    case 4: System.out.print("D");
    default: System.out.print("E");
}
```

(A) `B` (B) `BCDE` (C) `ABC` (D) `BCE` (E) `BC`

5 `int x = 3;` tanımından sonra yazılan aşağıdaki satırlardan hangisi **derleme hatası** verir?

(A) `if (x == 5) x++;`

(B) `if (x = 5) x++;`

(C) `boolean b = x > 3 && x < 9;`

(D) `String s = x > 0 ? "pozitif" : "değil";`

(E) `if (x > 0); else x = 0;`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (Girintiye güvenme!)

```java
public class Soru6 {
    public static void main(String[] args) {
        int a = 4, b = 9;
        if (a > b)
            if (a > 0)
                System.out.println("bir");
        else
            System.out.println("iki");
        if (b % a == 1)
            System.out.println("üç");
            System.out.println("dört");
        if (a > 5);
            System.out.println("beş");
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru7 {
    public static void main(String[] args) {
        int x = 5, y = 10;
        boolean p = x > 3 || ++y > 10;
        boolean q = x > 3 | ++y > 10;
        boolean r = !(x == 5) && y++ > 0;
        System.out.println(p + " " + q + " " + r + " " + y);
        System.out.println(true || false && false);
        System.out.println(x > y ? x : y * 2);
        System.out.println(x == 5 ? 1 : 2.0);
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru8 {
    public static void main(String[] args) {
        String s = "b";
        switch (s) {
            case "a": System.out.print("1");
            case "b": System.out.print("2");
            case "c": System.out.print("3");
            default:  System.out.print("4");
        }
        System.out.println();

        int n = 7;
        String t = switch (n % 3) {
            case 0 -> "sıfır";
            case 1 -> "bir";
            default -> "iki";
        };
        System.out.println(t);
    }
}
```

C) Kod Yazma

9 **İkinci dereceden denklemin** ($ax^2 + bx + c = 0$) köklerini bulan `Denklem` programını yaz. Kullanıcıdan `a`, `b`, `c` katsayılarını ondalıklı sayı olarak oku. $\Delta = b^2 - 4ac$ olmak üzere: $\Delta > 0$ ise iki kök, $\Delta = 0$ ise çift kat kök, $\Delta < 0$ ise "gerçek kök yok" yazdır. `a = 0` ise uyarı ver. Kökler 2 basamakla yazılsın:

```
a: 1
b: -5
c: 6
İki kök var: x1 = 3.00, x2 = 2.00
```

10 Kullanıcıdan bir **ay** (1–12) ve **yıl** okuyup o ayın kaç gün çektiğini yazdıran `AyGun` programını yaz. Şubat için artık yıl kontrolü yap. Ay geçersizse uyarı ver. (İpucu: Yeni `switch` ifadesi ile çok kısa yazılır.)

```
Ay (1-12): 2
Yıl: 2024
2024 yılının 2. ayı 29 gün çeker.
```

Çözümler

Cevap anahtarı

1	2	3	4	5
D	A	C	E	B

1 Cevap: D

De Morgan: `!(A && B)` = `!A || !B`. `x > 5`'in tersi `x <= 5`, `y <= 10`'un tersi `y > 10`; `&&` ise `||` olur. A şıkkı `&&`'yi değiştirmeyi unutmuş. B ve E'de karşılaştırmaların tersi yanlış alınmış (`>`'nin tersi `<` değil `<=`'dir). C ise ifadeyi hiç tersine çevirmemiş.

2 Cevap: A

İlk `if`: `a > 5` yanlış → `&&` kısa devre yapar, `++b` **çalışmaz**; blok da çalışmaz. İkinci `if`: `a < 5` doğru → `||` kısa devre yapar, `++b` yine **çalışmaz**; blok çalışır ve `a` 2 olur. `b` hiç artmadı.

Çıktı
```
2 0
```

3 Cevap: C

Negatif tek sayılarda kalan **-1** olur: `-7 % 2` = `-1`. Bu yüzden `n % 2 == 1` (A) ve `n % 2 > 0` (E) negatif tek sayılarda `false` verir; `n % 2 == -1` (D) ise pozitif tek sayılarda `false` verir. B şıkkı bölmeyle ilgilidir, teklikle ilgisi yoktur. `n % 2 != 0` ise kalan 1 de olsa -1 de olsa doğru çalışır. `n = -7` için ilk satır kalanı, ikinci satır A, C, E şıklarını; üçüncü satır `n = 7` için D ve C şıklarını gösteriyor:

Çıktı
```
-1
false true false
false true
```

4 Cevap: E

`k = 2` olduğu için `case 2`'ye atlanır ve `B` yazılır. `break` olmadığı için `case 3`'e düşülür (fall-through), `C` yazılır. Ardından gelen `break` switch'ten çıkarır; `D` ve `E` yazılmaz.

Çıktı
```
BC
```

5 Cevap: B

`x = 5` bir atamadır ve değeri `int`'tir; `if` ise `boolean` ister. E şıkkı garip görünse de derlenir: `if (x > 0);` boş komut çalıştırır, `else` kısmı da geçerlidir. C ve D sıradan, geçerli ifadelerdir.

6

- İlk yapı: `else`, girintiye rağmen içteki `if (a > 0)`'a aittir. Dıştaki `a > b` (4 > 9) yanlış olduğu için hiçbir şey yazılmaz.
- `b % a` = `9 % 4` = 1 → koşul doğru, `üç` yazılır. `dört` satırı `if`'e bağlı değildir, her durumda yazılır.

- `if (a > 5);` boş komuttur; `beş` her durumda yazılır.

Çıktı
```
üç
dört
beş
```

7

- `p`: `x > 3` doğru → `||` kısa devre, `++y` çalışmaz. `p = true`, `y = 10`.
- `q`: `|` kısa devre yapmaz → `++y` çalışır, `y = 11`, `11 > 10` doğru. `q = true`.
- `r`: `!(x == 5)` = `false` → `&&` kısa devre, `y++` çalışmaz. `r = false`, `y = 11`.
- `true || false && false`: `&&` önce → `true || false` = `true`.
- `x > y` → `5 > 11` yanlış → `y * 2` = 22.
- `x == 5 ? 1 : 2.0` → `int` ve `double` karışık olduğu için sonuç `double`: `1.0`.

Çıktı
```
true true false 11
true
22
1.0
```

8

- Klasik switch: `"b"` eşleşir, `break` olmadığı için `2`, `3` ve `4` yazılır.
- `7 % 3` = 1 → `case 1` eşleşir ve `"bir"` üretilir. `->` ile yazılan switch'te fall-through olmadığı için `default`'a düşülmez.

Çıktı
```
234
bir
```

9

Önce `a = 0` durumunu ele, sonra deltayı hesapla. Kökler: x = (−b ± √Δ) / 2a. Δ ondalıklı sayılarla hesaplandığı için "Δ = 0" kontrolünü toleransla yapıyoruz.

Denklem.java
```
import java.util.Locale;
import java.util.Scanner;

public class Denklem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);
        System.out.print("a: ");
        double a = sc.nextDouble();
        System.out.print("b: ");
        double b = sc.nextDouble();
        System.out.print("c: ");
        double c = sc.nextDouble();

        if (a == 0) {
            System.out.println("a = 0 ise denklem ikinci dereceden değildir.");
        } else {
            double delta = b * b - 4 * a * c;
```

```
            if (Math.abs(delta) < 1e-9) {
                System.out.printf("Çift kat kök: x = %.2f%n", -b / (2 * a));
            } else if (delta > 0) {
                double x1 = (-b + Math.sqrt(delta)) / (2 * a);
                double x2 = (-b - Math.sqrt(delta)) / (2 * a);
                System.out.printf("İki kök var: x1 = %.2f, x2 = %.2f%n", x1, x2);
            } else {
                System.out.println("Gerçek kök yok (delta < 0).");
            }
        }
    }
}
```

Çıktı

```
a: 1
b: -5
c: 6
İki kök var: x1 = 3.00, x2 = 2.00
```

10

Artık yıl koşulunu bölümdeki gibi tek bir `boolean` ifadeyle hesapla. Yeni `switch` ifadesinde 31 ve 30 çeken ayları virgülle grupla; Şubat için üçlü operatör kullan. Geçersiz ay için `default` 0 üretsin, sonra bunu `if` ile kontrol et.

AyGun.java

```
import java.util.Scanner;

public class AyGun {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Ay (1–12): ");
        int ay = sc.nextInt();
        System.out.print("Yıl: ");
        int yil = sc.nextInt();

        boolean artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
        int gun = switch (ay) {
            case 1, 3, 5, 7, 8, 10, 12 -> 31;
            case 4, 6, 9, 11 -> 30;
            case 2 -> artik ? 29 : 28;
            default -> 0;
        };

        if (gun == 0) {
            System.out.println("Geçersiz ay: " + ay);
        } else {
            System.out.printf("%d yılının %d. ayı %d gün çeker.%n", yil, ay, gun);
        }
    }
}
```

Çıktı

```
Ay (1–12): 2
Yıl: 2024
2024 yılının 2. ayı 29 gün çeker.
```

BÖLÜM 5

Döngüler

Bu Bölümde Öğreneceklerin

- `while`, `do-while` ve `for` döngülerini ve hangisinin ne zaman kullanılacağını
- Sonsuz döngüyü, `break` ve `continue` ile döngü akışını kontrol etmeyi
- İç içe döngülerle tablo ve **yıldız deseni** yazdırmayı (sınav klasiği)
- **Döngü izleme tablosu** (trace table) ile çıktı tahmin etmeyi ve off-by-one hatasını
- Klasik algoritmaları: basamak toplamı, ters çevirme, asal sayı, faktöriyel, Fibonacci, EBOB, mükemmel sayı
- Nöbetçi değer (sentinel) ve `hasNextInt()` ile kullanıcı girdisini döngüyle okumayı

5.1 Neden döngü?

Ekrana 1'den 5'e kadar sayı yazdırmak için beş tane `println` yazabilirsin. Peki 1'den 1000'e kadar? Ya da kullanıcı "bitti" diyene kadar not okumak? Kaç satır yazacağını bilemezsin bile. Bir işi **tekrar tekrar** yaptırmak için **döngü** (loop) kullanılır. Döngünün her bir turuna **iterasyon** (iteration) denir.

Java'da üç döngü vardır: `while`, `do-while` ve `for`. Üçü de aynı işi yapabilir; aralarındaki fark, hangi durumda daha okunaklı oldukları.

5.2 while döngüsü

`while` (... olduğu sürece), koşul `true` olduğu sürece gövdesini tekrar eder. Koşul her turdan **önce** kontrol edilir:

```
while (koşul) {
    // koşul true olduğu sürece tekrar edilecek komutlar
}
```

Bir sayaçlı döngünün üç parçası vardır: **başlangıç**, **koşul** ve **güncelleme**. Bunlardan biri eksikse döngü ya hiç çalışmaz ya da hiç bitmez:

WhileOrnek.java

```java
public class WhileOrnek {
    public static void main(String[] args) {
        int i = 1;                    // 1. başlangıç
        while (i <= 5) {              // 2. koşul
            System.out.print(i + " ");
            i++;                      // 3. güncelleme
        }
        System.out.println();
        System.out.println("Döngüden sonra i = " + i);

        int toplam = 0;
        int sayi = 1;
        while (sayi <= 100) {
            toplam += sayi;           // toplama ekle
            sayi++;
        }
        System.out.println("1'den 100'e toplam: " + toplam);
    }
}
```

Çıktı

```
1 2 3 4 5
Döngüden sonra i = 6
1'den 100'e toplam: 5050
```

Döngü bittiğinde `i` 5 değil **6**'dır: Koşulu bozan değer 6'dır ve döngü tam o anda durur. İkinci döngü, "toplam" değişkeni kullanan klasik **biriktirme** (accumulator) kalıbıdır: Toplam döngüden **önce** 0 yapılır, her turda üzerine eklenir.

> **Sık Yapılan Hata: Toplamı döngünün içinde sıfırlamak**
>
> `int toplam = 0;` satırını döngünün **içine** yazarsan toplam her turda sıfırlanır ve sonunda sadece son sayıyı tutar. Biriktirilecek değişken (toplam, sayaç, çarpım) her zaman döngüden **önce** başlatılır. Çarpımda başlangıç değeri 0 değil **1** olmalıdır.

5.3 do-while döngüsü

`do-while`'da koşul gövdeden **sonra** kontrol edilir. Bu yüzden gövde, koşul baştan yanlış olsa bile **en az bir kez** çalışır:

```
do {
    // en az bir kez çalışacak komutlar
} while (koşul);      ← noktalı virgül şart!
```

En tipik kullanımı **menüdür**: Menü önce bir kez gösterilmeli, sonra kullanıcı "çıkış" diyene kadar tekrar edilmelidir:

MenuDongu.java

```java
import java.util.Scanner;

public class MenuDongu {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int bakiye = 100;
        int secim;                    // do'nun DIŞINDA tanımlanmalı
        do {
```

```
            System.out.print("1) Bakiye 2) Yatır 0) Çıkış > ");
            secim = sc.nextInt();
            switch (secim) {
                case 1 -> System.out.println("Bakiye: " + bakiye + " TL");
                case 2 -> {
                    System.out.print("Miktar: ");
                    bakiye += sc.nextInt();
                }
                case 0 -> System.out.println("Güle güle!");
                default -> System.out.println("Geçersiz seçim.");
            }
        } while (secim != 0);
    }
}
```

Çıktı

```
1) Bakiye 2) Yatır 0) Çıkış > 2
Miktar: 50
1) Bakiye 2) Yatır 0) Çıkış > 1
Bakiye: 150 TL
1) Bakiye 2) Yatır 0) Çıkış > 0
Güle güle!
```

Sık Yapılan Hata: do-while'da değişkenin kapsamı

`secim` değişkenini `do { int secim = sc.nextInt(); ... } while (secim != 0);` diye bloğun **içinde** tanımlarsan program derlenmez: Bir blok içinde tanımlanan değişken sadece o blokta yaşar ve `while (...)` koşulu bloğun dışındadır. Ayrıca `while (koşul)`'dan sonraki `;`'yi unutma.

5.4 for döngüsü

Tekrar sayısı **belli** olduğunda en okunaklı döngü `for`'dur. Başlangıç, koşul ve güncelleme tek satırda toplanır:

```
for (başlangıç; koşul; güncelleme) {
    // gövde
}
```

Çalışma sırası: **1)** başlangıç (sadece bir kez) → **2)** koşul kontrolü → `true` ise **3)** gövde → **4)** güncelleme → tekrar **2)**'ye. Koşul `false` olunca döngü biter.

ForOrnek.java

```java
public class ForOrnek {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.print(i + " ");
        }
        System.out.println();

        for (int i = 10; i > 0; i -= 3) {     // geriye doğru, 3'er 3'er
            System.out.print(i + " ");
        }
        System.out.println();

        for (char c = 'a'; c <= 'e'; c++) {    // char da sayaç olabilir
            System.out.print(c);
        }
        System.out.println();
    }
}
```

Çıktı

```
1 2 3 4 5
10 7 4 1
abcde
```

Sayaç değişkeninin kapsamı

`for` parantezinde tanımlanan değişken (`int i`) **sadece döngünün içinde** yaşar. Döngüden sonra kullanmaya çalışırsan derleyici onu tanımaz:

ForKapsam.java

```java
public class ForKapsam {
    public static void main(String[] args) {
        for (int i = 0; i < 3; i++) {
            System.out.println(i);
        }
        System.out.println("Son değer: " + i);
    }
}
```

Derleyici çıktısı – program derlenmez

```
ForKapsam.java:6: error: cannot find symbol
        System.out.println("Son değer: " + i);
                                           ^
  symbol:   variable i
  location: class ForKapsam
1 error
```

Döngüden sonra da sayaca ihtiyacın varsa onu döngüden **önce** tanımla: `int i; for (i = 0; i < 3; i++) {...}`. Bu kural sayesinde aynı metotta art arda birden çok `for (int i = ...)` yazabilirsin; her biri kendi `i`'sini kullanır.

for mu, while mı?

Her `for` döngüsü bir `while` döngüsüne çevrilebilir (tersi de):

```
for (int i = 1; i <= 5; i++) {      int i = 1;
    gövde                           while (i <= 5) {
}                                       gövde
                                        i++;
                                    }
```

	`while`	`do-while`	`for`
Koşul ne zaman kontrol edilir?	Turdan önce	Turdan sonra	Turdan önce
En az kaç kez çalışır?	0	**1**	0
Ne zaman tercih edilir?	Tekrar sayısı belli değil (“-1 girilene kadar”)	Gövde en az bir kez çalışmalı (menü, girdi)	Tekrar sayısı belli (“10 kez”, “1’den n’e”)

5.5 Sonsuz döngü, break ve continue

Koşulu hiçbir zaman `false` olmayan döngüye **sonsuz döngü** (infinite loop) denir. Program donmuş gibi görünür, hiç bitmez. En sık sebepler:

```java
int i = 1;
while (i <= 10) {
    System.out.println(i);          // i++ unutuldu: i hep 1
}

// ...

for (int k = 0; k < 10; k--) {      // k azalıyor, hep 10'dan küçük
    System.out.println(k);
}

// ...

int x = 1;
while (x != 10) {                   // x hep tek sayı: asla 10 olmaz
    x += 2;
}
```

İkinci döngü, `k` taşma (Bölüm 2) ile pozitife dönene kadar milyarlarca tur atar; pratikte sonsuzdur. Üçüncüsünde `!=` yerine `<` kullanmak (`while (x < 10)`) hatayı önlerdi: Sayaçlı döngülerde **`!=` yerine `<` veya `<=` tercih et.**

> **İpucu**
>
> Program sonsuz döngüye girerse VS Code terminalinde **Ctrl+C** ile durdurabilirsin. Döngünün ne yaptığını görmek için satır numarasının soluna tıklayıp **kesme noktası** (breakpoint) koy ve **Debug** ile çalıştır; değişkenlerin her turdaki değerlerini sol panelde izleyebilirsin.

Bazen sonsuz döngü **bilerek** yazılır: `while (true)` ile başlayıp, çıkış koşulu sağlanınca **`break`** ile döngüden çıkılır. `break`, içinde bulunduğu döngüyü **anında** bitirir. **`continue`** ise döngüyü bitirmez; o turun geri kalanını atlayıp **sonraki tura** geçer (`for`’da önce güncelleme yapılır):

BreakContinue.java

```java
public class BreakContinue {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) {
```

```
                continue;            // çiftleri atla
            }
            if (i > 7) {
                break;               // 7'den büyük ilk tekte dur
            }
            System.out.print(i + " ");
        }
        System.out.println();

        int n = 1;
        while (true) {
            n *= 2;
            if (n > 100) {
                break;
            }
        }
        System.out.println("100'ü geçen ilk 2'nin kuvveti: " + n);
    }
}
```

Çıktı

```
1 3 5 7
100'ü geçen ilk 2'nin kuvveti: 128
```

Sık Yapılan Hata: while'da continue tuzağı

`while` döngüsünde güncelleme gövdenin sonundaysa `continue` onu da atlar: `int i = 0; while (i < 10) { if (i % 2 == 0) continue; ...; i++; }` sonsuz döngüye girer, çünkü `i` 0'da takılı kalır. `for`'da bu sorun yoktur; `continue`'dan sonra güncelleme (`i++`) yine çalışır.

5.6 İç içe döngüler

Bir döngünün gövdesine başka bir döngü yazılabilir. **Dış döngünün her turu için iç döngü baştan sona çalışır.** Dış döngü 5, iç döngü 5 tur atıyorsa iç gövde toplam $5 \times 5 = 25$ kez çalışır. Klasik örnek çarpım tablosu; hizalı sütunlar için Bölüm 3'teki `printf` genişliğini kullanıyoruz:

CarpimTablosu.java

```
public class CarpimTablosu {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {          // satırlar
            for (int j = 1; j <= 5; j++) {      // sütunlar
                System.out.printf("%4d", i * j);
            }
            System.out.println();               // satır bitti
        }
    }
}
```

Çıktı

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

İç içe döngüde `break` sadece **içinde bulunduğu** (en içteki) döngüyü bitirir; dış döngü devam eder. İki döngüden birden çıkmak için **etiketli break** (labeled break) kullanılır: Dış döngünün önüne `etiket:` yazılır, `break etiket;` o döngüyü bitirir:

EtiketliBreak.java

```java
public class EtiketliBreak {
    public static void main(String[] args) {
        dis:
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 3; j++) {
                if (i * j == 4) {
                    System.out.println("Bulundu: " + i + " x " + j);
                    break dis;              // iki döngüden de çık
                }
                System.out.println(i + " x " + j + " = " + i * j);
            }
        }
    }
}
```

Çıktı

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
2 x 1 = 2
Bulundu: 2 x 2
```

Sadece `break;` yazsaydık yalnızca iç döngü biterdi ve `i = 3` için tarama devam ederdi. Aynı şekilde `continue etiket;` dış döngünün sonraki turuna geçer. Etiketler nadiren kullanılır; sınavda görürsen tanıman yeterli.

5.7 Yıldız desenleri

“n verildiğinde şu deseni yazdır” soruları sınavların klasiğidir. Hepsinin tarifi aynı:

1. **Dış döngü satırları** sayar (`i = 1 ... n`).
2. **İç döngü(ler)** o satırdaki karakterleri basar. Kaç karakter basılacağı **satır numarası `i` cinsinden** yazılır.
3. Satır bitince `println()` ile alt satıra geçilir.

Desenler1.java

```java
public class Desenler1 {
    public static void main(String[] args) {
        int n = 4;
        // Dik üçgen: i. satırda i yıldız
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        System.out.println();
        // Ters üçgen: dış döngü n'den 1'e iner
        for (int i = n; i >= 1; i--) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Çıktı

```
*
**
***
****

****
***
**
*
```

Piramit için her satırda önce boşluk, sonra yıldız basılır. Önce bir tablo yapıp formülü bul (n = 4):

Satır i	Boşluk	Yıldız	Formül
1	3	1	boşluk = `n - i`
2	2	3	yıldız = `2 * i - 1`
3	1	5	
4	0	7	

Desenler2.java

```java
public class Desenler2 {
    public static void main(String[] args) {
        int n = 4;
        // Piramit
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }
            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        System.out.println();
        // Sayı üçgeni: i. satırda 1'den i'ye kadar sayılar
```

```
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
```

Çıktı

```
   *
  ***
 *****
*******

1
1 2
1 2 3
1 2 3 4
```

> **Sınavda Çıkar: Desen sorusunu çözme yöntemi**
>
> Kodu yazmadan önce kâğıda **satır | boşluk | karakter** tablosunu çiz ve her sütunu `i` ile ifade et. Sayı desenlerinde de aynı: "i. satırda ne yazıyor?" sorusunu cevapla. Sayı üçgeninde `print(j)` yerine `print(i)` yazsaydın her satır kendi numarasını tekrar ederdi (`1`, `2 2`, `3 3 3`...). Hocalar bu iki varyantı sık sık karşılaştırır.

5.8 Döngü izleme tablosu ve off-by-one

İzleme tablosu (trace table)

Döngülü bir kodun çıktısını tahmin etmenin en güvenli yolu, kâğıda bir **izleme tablosu** (trace table) çizmektir: Her değişken için bir sütun açılır, her turda değişen değerler yeni bir satıra yazılır. Kafadan yapmaya çalışmak hataya davetiyedir.

Izleme.java

```
public class Izleme {
    public static void main(String[] args) {
        int a = 1, b = 0;
        while (a < 20) {
            b += a;
            a *= 3;
        }
        System.out.println(a + " " + b);
    }
}
```

Tur	Koşul `a < 20`	`b += a`	`a *= 3`
başlangıç		`b = 0`	`a = 1`
1	`1 < 20` ✓	`b = 1`	`a = 3`
2	`3 < 20` ✓	`b = 4`	`a = 9`
3	`9 < 20` ✓	`b = 13`	`a = 27`
—	`27 < 20` ✗	döngü biter	

Çıktı
```
27 13
```

> **Sınavda Çıkar**
>
> Çıktı sorularında izleme tablosu çizmek zaman kaybı değildir, **puan kazancıdır**. Özellikle koşulun **her turun başında** kontrol edildiğine ve döngüden çıkıldığında değişkenlerin **son** değerlerinin yazdırıldığına dikkat et: Yukarıda `a`'nın son değeri 27'dir, 9 değil.

Off-by-one hatası

Döngünün **bir tur fazla veya eksik** çalışmasına **off-by-one** (bir kayma) hatası denir. Programlamada en sık yapılan hatalardan biridir; sebebi genelde `<` ile `<=` karışıklığı ya da sayacın 0'dan mı 1'den mi başladığıdır. Tur sayısını şu formüllerle hızlıca bulabilirsin:

Döngü	Tur sayısı	Örnek
`for (i = a; i <= b; i++)`	`b - a + 1`	`i = 1; i <= 10` → 10 tur
`for (i = a; i < b; i++)`	`b - a`	`i = 0; i < 10` → 10 tur
`for (i = 0; i <= 10; i++)`	`11`	0'dan 10'a, **11** tur!
`for (i = a; i < b; i += k)`	`(b - a) / k`, yukarı yuvarlanır	`i = 0; i < 10; i += 3` → 0, 3, 6, 9 → 4 tur

Emin olamadığında küçük bir değerle (n = 1 veya n = 2) döngüyü kafanda çalıştır ve tur sayısını say.

5.9 Klasik döngü algoritmaları

Bu başlıktaki algoritmalar hem sınavlarda hem ödevlerde tekrar tekrar karşına çıkacak. Her birini anlayarak **ezberle**.

Basamak toplamı ve sayıyı ters çevirme

Bölüm 2'deki iki kural burada döngüye dönüşür: `n % 10` **son basamağı verir**, `n / 10` **son basamağı atar**. Sayı 0 olana kadar bu ikisini tekrarlarsan tüm basamakları sağdan sola gezersin:

Basamaklar.java
```java
import java.util.Scanner;

public class Basamaklar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Bir sayı gir: ");
        int sayi = sc.nextInt();
```

```
        int n = Math.abs(sayi);     // kopya; eksi işaretini at
        int adet = 0, toplam = 0, ters = 0;
        do {                        // 0 da bir basamaklıdır: en az bir tur
            int basamak = n % 10;   // son basamak
            adet++;
            toplam += basamak;
            ters = ters * 10 + basamak;
            n /= 10;                // son basamağı at
        } while (n > 0);
        System.out.println("Basamak sayısı: " + adet);
        System.out.println("Basamak toplamı: " + toplam);
        System.out.println("Tersi: " + ters);
        System.out.println("Palindrom mu? " + (sayi == ters));
    }
}
```

Çıktı

```
Bir sayı gir: 4728
Basamak sayısı: 4
Basamak toplamı: 21
Tersi: 8274
Palindrom mu? false
```

Döngü `do-while` çünkü `0` girilirse de bir tur dönmeli: `0` bir basamaklı bir sayıdır (`while (n > 0)` yazsaydık hiç dönmez ve "0 basamak" derdi). `Math.abs` ile eksi işaretini atıyoruz; böylece `-472` için de basamaklar doğru sayılır. `ters = ters * 10 + basamak` satırı, eldeki sayıyı bir basamak sola kaydırıp yeni basamağı sağa ekler: 0 → 8 → 82 → 827 → 8274. Tersi kendisine eşit olan sayıya **palindrom** denir (12321 gibi).

Asal sayı kontrolü

Bir sayı **asal** ise 1 ve kendisinden başka böleni yoktur; 1 asal değildir, en küçük asal 2'dir. 2'den başlayıp bölen ararız; bir bölen bulduğumuz anda cevap kesinleştiği için `break` ile çıkarız:

AsalMi.java

```
import java.util.Scanner;

public class AsalMi {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Sayı: ");
        int n = sc.nextInt();

        boolean asal = n >= 2;              // 0 ve 1 asal değil
        for (int i = 2; i * i <= n; i++) {  // karekökü kadar dene
            if (n % i == 0) {
                System.out.println(i + " bir böleni.");
                asal = false;
                break;
            }
        }
        System.out.println(n + (asal ? " asaldır." : " asal değildir."));
    }
}
```

Çıktı

```
Sayı: 91
7 bir böleni.
91 asal değildir.
```

`i * i <= n` koşulu önemli bir hızlandırmadır: Bir sayının karekökünden büyük bir böleni varsa, karekökünden küçük bir eşi de vardır ($91 = 7 \times 13$). Bu yüzden `n - 1`'e kadar değil, `√n`'e kadar denemek yeter. Bu kalıbın adı **bayrak** (flag) değişkenidir: `asal` önce `true` kabul edilir, aksini gösteren durum bulununca `false` yapılır.

Faktöriyel ve Fibonacci

Faktöriyel $n! = 1 \times 2 \times \ldots \times n$, bir **çarpım biriktirme** örneğidir. Sonuç çok hızlı büyüdüğü için `long` kullanılmalıdır:

Faktoriyel.java

```
public class Faktoriyel {
    public static void main(String[] args) {
        int n = 20;
        long f = 1;                  // çarpımda başlangıç 1!
        int fInt = 1;
        for (int i = 1; i <= n; i++) {
            f *= i;
            fInt *= i;
        }
        System.out.println(n + "! (long) = " + f);
        System.out.println(n + "! (int)  = " + fInt);
    }
}
```

Çıktı

```
20! (long) = 2432902008176640000
20! (int)  = -2102132736
```

`int` versiyonu sessizce **taştı** (Bölüm 2) ve anlamsız bir sonuç verdi: `int` en fazla 12!'yi, `long` en fazla 20!'yi doğru tutabilir.

Fibonacci dizisinde her terim, kendinden önceki iki terimin toplamıdır: 0, 1, 1, 2, 3, 5, 8... Sadece son iki terimi tutan iki değişken yeterlidir:

Fibonacci.java

```
public class Fibonacci {
    public static void main(String[] args) {
        int a = 0, b = 1;
        for (int i = 1; i <= 12; i++) {
            System.out.print(a + " ");
            int sonraki = a + b;
            a = b;
            b = sonraki;
        }
        System.out.println();
    }
}
```

Çıktı

```
0 1 1 2 3 5 8 13 21 34 55 89
```

`a = b; b = a + b;` yazmak yanlış olurdu: `a`'nın eski değeri ilk satırda kaybolur. Bu yüzden toplam önce geçici bir değişkene (`sonraki`) alınır.

EBOB (Öklid algoritması)

İki sayının **en büyük ortak böleni** (EBOB, GCD) için 2300 yıllık **Öklid algoritması**: `(a, b)` çiftini `(b, a % b)` ile değiştir; `b` 0 olunca `a` EBOB'dur. **En küçük ortak kat** (EKOK) da EBOB'dan bulunur: `a * b / EBOB`.

Ebob.java

```
public class Ebob {
    public static void main(String[] args) {
        int a = 48, b = 18;
        int x = a, y = b;
        while (y != 0) {
            int kalan = x % y;
            x = y;
            y = kalan;
        }
        System.out.println("EBOB(" + a + ", " + b + ") = " + x);
        System.out.println("EKOK(" + a + ", " + b + ") = " + a / x * b);
    }
}
```

Çıktı

```
EBOB(48, 18) = 6
EKOK(48, 18) = 144
```

İzleme: (48, 18) → (18, 12) → (12, 6) → (6, 0). `a / x * b` sırası bilinçli: Önce bölmek, `a * b`'nin taşma ihtimalini azaltır.

Mükemmel sayı

Kendisi hariç pozitif bölenlerinin toplamı kendisine eşit olan sayıya **mükemmel sayı** denir: 6 = 1 + 2 + 3. Her sayı için bölenleri toplayan **iç içe** bir döngü gerekir:

Mukemmel.java

```
public class Mukemmel {
    public static void main(String[] args) {
        for (int n = 2; n <= 10000; n++) {
            int toplam = 0;                  // her n için sıfırdan!
            for (int b = 1; b <= n / 2; b++) {
                if (n % b == 0) {
                    toplam += b;
                }
            }
            if (toplam == n) {
                System.out.print(n + " ");
            }
        }
        System.out.println();
    }
}
```

Çıktı

```
6 28 496 8128
```

Burada `toplam = 0` dış döngünün **içinde** olmalıdır, çünkü her sayının bölen toplamı ayrı hesaplanır. "Toplamı döngüden önce başlat" kuralı, toplamın **hangi döngüye** ait olduğuna göre uygulanır. Bir sayının kendisi hariç en büyük böleni en fazla yarısı olabileceği için iç döngü `n / 2`'de durur.

5.10 Kullanıcı girdisiyle döngüler

Nöbetçi değer (sentinel)

Kullanıcının kaç değer gireceğini bilmiyorsan, girişin bittiğini haber veren özel bir değer belirlersin: **nöbetçi değer** (sentinel). Bu değer gerçek veri olamayacak bir değer olmalıdır; notlar için `-1` gibi:

Nobetci.java

```
import java.util.Scanner;

public class Nobetci {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int toplam = 0, adet = 0;

        System.out.print("Not gir (bitirmek için -1): ");
        int not = sc.nextInt();
        while (not != -1) {
            toplam += not;
            adet++;
            System.out.print("Not gir (bitirmek için -1): ");
            not = sc.nextInt();
        }

        if (adet > 0) {
            double ort = (double) toplam / adet;
            System.out.printf("%d not, ortalama %.2f%n", adet, ort);
        } else {
            System.out.println("Hiç not girilmedi.");
        }
    }
}
```

Çıktı

```
Not gir (bitirmek için -1): 70
Not gir (bitirmek için -1): 85
Not gir (bitirmek için -1): 92
Not gir (bitirmek için -1): -1
3 not, ortalama 82.33
```

Kalıba dikkat: İlk değer döngüden **önce** okunur (*priming read*), sonraki değerler gövdenin **sonunda** okunur. Böylece `-1` hiçbir zaman toplama eklenmez. `(double) toplam / adet` ise Bölüm 2'deki tam sayı bölmesi tuzağından korur; `adet > 0` kontrolü de sıfıra bölmeyi engeller.

hasNextInt() ile girdi doğrulama

Bölüm 3'te kullanıcı sayı yerine harf yazınca `nextInt()`'in `InputMismatchException` ile çöktüğünü gördün. **`hasNextInt()`**, sıradaki girdinin bir tam sayı olup olmadığını **okumadan** kontrol eder ve `true`/`false` döner. Bir döngüyle birleştirince kullanıcı geçerli bir değer girene kadar tekrar sorabilirsin:

GuvenliOkuma.java

```
import java.util.Scanner;

public class GuvenliOkuma {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int yas;
        while (true) {
```

```
            System.out.print("Yaşın (0–120): ");
            if (sc.hasNextInt()) {
                yas = sc.nextInt();
                if (yas >= 0 && yas <= 120) {
                    break;                          // geçerli: döngüden çık
                }
                System.out.println("Aralık dışında, tekrar dene.");
            } else {
                String yanlis = sc.next();          // hatalı kelimeyi TÜKET
                System.out.println("'" + yanlis + "' bir sayı değil!");
            }
        }
        System.out.println("Kaydedildi: " + yas);
    }
}
```

Çıktı

```
Yaşın (0–120): yirmi
'yirmi' bir sayı değil!
Yaşın (0–120): 150
Aralık dışında, tekrar dene.
Yaşın (0–120): 19
Kaydedildi: 19
```

İki tür doğrulama iç içe: **tip** kontrolü (`hasNextInt()`) ve **aralık** kontrolü (`0–120`). Aynı kalıp `hasNextDouble()` ve `hasNext()` ile de çalışır.

> **Sık Yapılan Hata: Hatalı girdiyi tüketmemek**
>
> `hasNextInt()` girdiyi **okumaz**, sadece bakar. `else` kısmındaki `sc.next()` satırını silersen `yirmi` kelimesi tamponda kalır; döngü her turda aynı kelimeye bakıp `false` alır ve program **sonsuz döngüye** girer. Hatalı girdiyi mutlaka `next()` (veya `nextLine()`) ile okuyup at.

Bölüm Özeti – Bilmen Gerekenler

- `while`: koşul turdan **önce** kontrol edilir, 0 kez de çalışabilir. `do-while`: koşul turdan **sonra**, gövde **en az bir kez** çalışır; sonunda `;` var.
- `for (başlangıç; koşul; güncelleme)`: tekrar sayısı belliyse. Parantezde tanımlanan sayaç **sadece döngü içinde** geçerlidir.
- Sayaçlı döngünün üç parçası: başlangıç, koşul, güncelleme. Güncelleme unutulursa **sonsuz döngü**. Sayaçta `!=` yerine `</<=` kullan.
- `break` döngüyü bitirir, `continue` sonraki tura geçer. İç içe döngüde ikisi de **en içteki** döngüyü etkiler; dış döngü için etiket: `break dis;`.
- İç içe döngü: dış döngünün **her turu** için iç döngü baştan sona çalışır ($5 \times 5 = 25$ tur).
- Desenler: dış döngü **satır**, iç döngü **sütun**; karakter sayısını `i` ile ifade et (piramit: `n - i` boşluk, `2i - 1` yıldız).
- Tur sayısı: `i = a; i <= b` → `b - a + 1`; `i = a; i < b` → `b - a`. Çıktı sorularında **izleme tablosu** çiz.
- Basamaklar: `n % 10` son basamak, `n /= 10` son basamağı at. Ters: `ters = ters * 10 + basamak`.
- Asal: 2'den `√n`'e kadar bölen ara, bulunca `break`. Faktöriyel `long` ile (int 12!'de, long 20!'de sınıra ulaşır). Fibonacci: geçici değişken. EBOB: `(a, b) → (b, a % b)`.
- Toplam/sayaç döngüden **önce** başlatılır (çarpımda 1 ile). İç içe döngüde hangi döngüye ait olduğuna dikkat.
- Nöbetçi değer: ilk değeri döngüden önce oku, sonrakileri gövde sonunda. `hasNextInt()` okumaz; hatalı girdiyi `next()` ile tüket.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod parçasında `*` karakteri toplam kaç kez yazdırılır?

```
for (int i = 1; i <= 5; i++) {
    for (int j = i; j <= 5; j++) {
        System.out.print("*");
    }
}
```

(A) 25 (B) 15 (C) 5 (D) 10 (E) 20

2 Aşağıdaki kod parçası ekrana ne yazdırır?

```
int x = 10;
do {
    System.out.print(x + " ");
    x++;
} while (x < 5);
```

(A) Hiçbir şey yazdırmaz.
(B) `5 6 7 8 9`
(C) Sonsuz döngüye girer.
(D) `10`
(E) Derleme hatası verir.

3 Aşağıdaki döngülerden hangisi gövdesini **tam 5 kez** çalıştırır?

(A) `for (int i = 5; i > 0; i--)`
(B) `for (int i = 0; i <= 5; i++)`
(C) `for (int i = 1; i < 5; i++)`
(D) `for (int i = 0; i < 10; i += 3)`
(E) `for (int i = 10; i >= 0; i -= 2)`

4 Aşağıdaki kod parçası ekrana ne yazdırır?

```
int toplam = 0;
for (int i = 1; i <= 10; i++) {
    if (i % 3 == 0) {
        continue;
    }
    if (i == 8) {
        break;
    }
    toplam += i;
}
System.out.println(toplam);
```

(A) 27 (B) 37 (C) 22 (D) 12 (E) 19

5 Aşağıdakilerden hangisi **derleme hatası** verir?

(A) `for (;;) { break; }`
(B) `int i = 0; while (i < 3) i++;`
(C) `for (int i = 0; i < 3; i++) { } System.out.println(i);`
(D) `do { } while (false);`
(E) `for (int i = 0, j = 10; i < j; i++, j--) { }`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru6 {
    public static void main(String[] args) {
        int i = 0;
        while (i++ < 3) {
            System.out.print(i + " ");
        }
        System.out.println("| " + i);

        int j = 0;
        while (++j < 3) {
            System.out.print(j + " ");
        }
        System.out.println("| " + j);

        int k;
        for (k = 0; k < 3; k++);
        {
            System.out.println("k = " + k);
        }
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru7 {
    public static void main(String[] args) {
        int sayac = 0;
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print((i + j) % 2 == 0 ? "1" : "0");
                sayac++;
            }
            System.out.println();
        }
        System.out.println("sayac = " + sayac);
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru8 {
    public static void main(String[] args) {
        int n = 1203, sonuc = 0;
        while (n > 0) {
            int b = n % 10;
            n /= 10;
            if (b == 0) {
                continue;
            }
            sonuc = sonuc * 10 + b;
        }
        System.out.println(sonuc);

        dis:
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 3; j++) {
                if (i == 3) {
                    break dis;
                }
                if (j > i) {
                    break;
                }
                System.out.print(i + "" + j + " ");
            }
        }
        System.out.println();
    }
}
```

C) Kod Yazma

9 Kullanıcıdan bir `n` sayısı okuyup `2n − 1` satırlık **elmas** (baklava) desenini yazdıran `Elmas` programını yaz. `n = 4` için çıktı:

```
   *
  ***
 *****
*******
 *****
  ***
   *
```

10 **Sayı tahmin oyunu:** Program 1 ile 100 arasında bir sayı tutsun; kullanıcı doğru tahmin edene kadar “Daha büyük!” / “Daha küçük!” ipucu versin ve sonunda kaç denemede bildiğini yazsın. Sayıyı `java.util.Random` ile üret: `new Random(42).nextInt(100) + 1`. (`nextInt(100)` 0–99 arası bir sayı üretir; sabit tohum `42`, her çalıştırmada aynı sayının tutulmasını sağlar.) `TahminOyunu` adlı programı `do-while` ile yaz.

Çözümler

Cevap anahtarı

1 B	2 D	3 A	4 E	5 C

1 Cevap: B

İç döngü `j = i`'den 5'e kadar döner, yani `5 - i + 1` tur atar. `i = 1` için 5, `i = 2` için 4, ... `i = 5` için 1 tur: 5 + 4 + 3 + 2 + 1 = **15**. A şıkkı (25), iç döngünün her seferinde 1'den başladığını sananların cevabıdır.

Çıktı
```
***************
```

2 Cevap: D

`do-while` koşulu **sonradan** kontrol eder; gövde koşula bakılmadan bir kez çalışır ve `10` yazılır. Sonra `x = 11` olur, `11 < 5` yanlış olduğu için döngü biter. Aynı kod `while` ile yazılsaydı hiçbir şey yazdırmazdı (A).

Çıktı
```
10
```

3 Cevap: A

`i` 5, 4, 3, 2, 1 değerlerini alır: 5 tur. Diğerleri: B → 0'dan 5'e **6** tur (off-by-one); C → 1'den 4'e **4** tur; D → 0, 3, 6, 9: **4** tur; E → 10, 8, 6, 4, 2, 0: **6** tur. Beş döngünün tur sayıları sırasıyla:

Çıktı
```
5 6 4 4 6
```

4 Cevap: E

`i = 3` ve `i = 6` `continue` ile atlanır. `i = 8` olunca `break` ile döngü biter (`9` ve `10` hiç gelmez). Toplanan sayılar: 1 + 2 + 4 + 5 + 7 = **19**.

Çıktı
```
19
```

5 Cevap: C

`for` parantezinde tanımlanan `i` sadece döngünün içinde geçerlidir; döngüden sonra `println(i)` *cannot find symbol* hatası verir. A şıkkı geçerli bir sonsuz döngüdür ve `break` ile hemen çıkar; B gövdesi tek komut olan bir `while`'dır; D gövdesi boş bir `do-while`'dır; E ise virgülle iki değişkenli geçerli bir `for`'dur.

6

- `while (i++ < 3)`: Karşılaştırma `i`'nin **eski** değeriyle yapılır, sonra `i` artar. 0 < 3 → `i = 1`, yazılır; 1 < 3 → 2; 2 < 3 → 3; 3 < 3 yanlış, ama `i` yine artar → **4**.
- `while (++j < 3)`: Önce artırılır. `j = 1` ✓, `j = 2` ✓, `j = 3` → 3 < 3 yanlış. Sadece 1 ve 2 yazılır, `j = 3`.

- `for (...);` sonundaki `;` boş gövdedir: Döngü hiçbir şey yapmadan `k`'yı 3'e çıkarır. Alttaki blok döngüye ait değildir, **bir kez** çalışır.

Çıktı
```
1 2 3 | 4
1 2 | 3
k = 3
```

7

`i + j` çiftse `1`, tekse `0` yazılır. İzleme: `i = 1`: j=1 → 2 çift → `1`. `i = 2`: j=1 → 3 → `0`, j=2 → 4 → `1`. `i = 3`: `1`, `0`, `1`. `i = 4`: `0`, `1`, `0`, `1`. İç gövde 1 + 2 + 3 + 4 = 10 kez çalışır.

Çıktı
```
1
01
101
0101
sayac = 10
```

8

- İlk döngü basamakları **sağdan sola** gezer: 3, 0, 2, 1. `0` `continue` ile atlanır; diğerleri ters çevirme formülüyle eklenir: 3 → 32 → 321. `n /= 10` satırı `continue`'dan **önce** olduğu için sonsuz döngü olmaz.
- İkinci döngüde `break` (etiketsiz) sadece iç döngüyü bitirir: `i = 1` için `11`, `i = 2` için `21 22` yazılır. `i = 3` olunca `break dis` iki döngüden de çıkar.

Çıktı
```
321
11 21 22
```

9

Elmas, bir piramit ile onun ters çevrilmiş halinin birleşimidir. Üst kısım `i = 1 ... n` için piramit formülüyle (`n − i` boşluk, `2i − 1` yıldız); alt kısım aynı formülle ama `i = n − 1`'den 1'e **azalarak** yazılır. Ortadaki en geniş satır iki kez yazılmasın diye alt kısım `n − 1`'den başlar.

Elmas.java
```java
import java.util.Scanner;

public class Elmas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("n: ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {            // üst yarı
            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }
            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        for (int i = n - 1; i >= 1; i--) {        // alt yarı
```

```
            for (int j = 1; j <= n - i; j++) {
                System.out.print(" ");
            }
            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Çıktı

```
n: 4
   *
  ***
 *****
*******
 *****
  ***
   *
```

10

Kullanıcı en az bir tahmin yapacağı için `do-while` en doğal seçimdir. `tahmin` ve `deneme` değişkenleri koşulda kullanılacağı için döngüden **önce** tanımlanır. Her turda deneme sayısı artırılır, tahmin tutulan sayıyla karşılaştırılır.

TahminOyunu.java

```
import java.util.Random;
import java.util.Scanner;

public class TahminOyunu {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int tutulan = new Random(42).nextInt(100) + 1;
        int tahmin;
        int deneme = 0;

        System.out.println("1-100 arası bir sayı tuttum.");
        do {
            System.out.print("Tahmin: ");
            tahmin = sc.nextInt();
            deneme++;
            if (tahmin < tutulan) {
                System.out.println("Daha büyük!");
            } else if (tahmin > tutulan) {
                System.out.println("Daha küçük!");
            }
        } while (tahmin != tutulan);

        System.out.println("Bildin! " + deneme + " denemede buldun.");
    }
}
```

Çıktı

```
1–100 arası bir sayı tuttum.
Tahmin: 50
Daha küçük!
Tahmin: 25
Daha büyük!
Tahmin: 37
Daha küçük!
Tahmin: 31
Bildin! 4 denemede buldun.
```

Her seferinde aralığın **ortasını** tahmin etmek (50 → 25 → 37 → 31) en hızlı stratejidir; buna ikili arama (binary search) denir ve Bölüm 16'da algoritma olarak göreceksin.

BÖLÜM 6

Metotlar

Bu Bölümde Öğreneceklerin

- Kodu neden metotlara böldüğümüzü (tekrarı önleme, okunabilirlik)
- Bir metodun parçalarını: erişim belirteci, `static`, dönüş tipi, ad, parametre listesi, gövde
- `void` metotlar ile değer döndüren metotların farkını ve `return` kullanımını
- Parametre ile argüman farkını ve bir metot çağrıldığında neler olduğunu (çağrı yığını)
- Aşırı yüklemeyi (overloading) ve metot imzasını
- Java'nın **değer ile geçirme** kuralını: `swap` metodu neden çalışmaz?
- Değişkenlerin kapsamını (scope) ve Javadoc ile metot belgelemeyi

6.1 Neden metot?

Şimdiye kadar bütün kodu `main`'in içine yazdık. Program büyüdükçe bu iki soruna yol açar: Aynı kod parçasını **tekrar tekrar** yazarsın ve `main` okunamayacak kadar uzar. **Metot** (method), bir işi yapan kod parçasına bir **ad** verip onu istediğin yerden, istediğin kadar **çağırmanı** (call) sağlar. Başka dillerde buna **fonksiyon** (function) denir.

CizgiMetodu.java

```
public class CizgiMetodu {
    public static void cizgiCiz() {
        for (int i = 0; i < 24; i++) {
            System.out.print("=");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        cizgiCiz();
        System.out.println("  2025-2026 GÜZ DÖNEMİ");
        cizgiCiz();
        System.out.println("  Programlamaya Giriş");
        cizgiCiz();
    }
}
```

Çıktı

```
========================
  2025-2026 GÜZ DÖNEMİ
========================
  Programlamaya Giriş
========================
```

Döngüyü bir kere yazdık, üç kere çağırdık. Çizginin uzunluğunu 30 yapmak istersen tek bir yeri değiştirmen yeter. Metot kullanmanın faydaları:

- **Tekrar yok:** Kod bir kere yazılır, çok kere kullanılır (*DRY: Don't Repeat Yourself*).

- **Okunabilirlik:** `cizgiCiz();` satırı ne yaptığını adıyla anlatır; içindeki döngüyü okumana gerek kalmaz.
- **Kolay test ve hata ayıklama:** Her metodu ayrı ayrı deneyebilirsin; hata varsa nerede arayacağını bilirsin.

Aslında metotları baştan beri kullanıyorsun: `main` bir metottur; `System.out.println`, `Math.sqrt`, `sc.nextInt` de Java'nın hazır metotlarıdır. Bu bölümde **kendi** metotlarını yazacaksın.

6.2 Metodun anatomisi

Bir metot tanımı (method declaration) altı parçadan oluşur:

```
public  static  int     topla  (int a, int b)   {  return a + b;  }
------  ------  -----   -----  --------------   ------------------
erişim  static  dönüş   ad     parametre        gövde
                tipi           listesi
```

Parça	Örnekte	Anlamı
Erişim belirteci (access modifier)	`public`	Metoda kimlerin erişebileceği. Ayrıntısı Bölüm 9'da; şimdilik hep `public`.
`static`	`static`	Metot, nesne oluşturmadan çağrılabilir. Şimdilik hep yazacağız (nedeni bu bölümün sonlarında).
Dönüş tipi (return type)	`int`	Metodun geri verdiği değerin tipi. Hiçbir şey döndürmüyorsa **`void`**.
Metot adı	`topla`	camelCase, genellikle fiil: `hesapla`, `yazdir`, `asalMi`.
Parametre listesi	`(int a, int b)`	Metodun dışarıdan aldığı değerler; her biri **tip + ad**. Boş olabilir: `()`.
Gövde (body)	`{ return a + b; }`	Metot çağrıldığında çalışan komutlar.

Metot, sınıfın süslü parantezlerinin **içine**, ama başka bir metodun **dışına** yazılır; Java'da metot içinde metot tanımlanamaz. `main`'den önce ya da sonra olması fark etmez.

IlkMetot.java

```java
public class IlkMetot {
    public static int topla(int a, int b) {
        int sonuc = a + b;
        return sonuc;
    }

    public static void main(String[] args) {
        int x = topla(3, 4);
        System.out.println(x);
        System.out.println(topla(10, 20) * 2);
        System.out.println(topla(topla(1, 2), 3));
    }
}
```

Çıktı

```
7
60
6
```

`topla(3, 4)` çağrısında `a` = 3, `b` = 4 olur, gövde çalışır ve `return` ile 7 değeri çağrının yapıldığı yere döner. Yani `int x = topla(3, 4);` satırı, metot bittikten sonra `int x = 7;` gibi davranır. Değer döndüren bir çağrıyı, o tipte bir değerin kullanılabileceği her yerde kullanabilirsin: bir ifadenin içinde (`topla(10, 20) * 2`), `println`'in içinde, hatta başka bir çağrının argümanı olarak (`topla(topla(1, 2), 3)`: önce içteki çağrı 3 döndürür, sonra `topla(3, 3)` çalışır).

> **Not: `main` metodunu artık okuyabilirsin**
>
> `public static void main(String[] args)`: Herkese açık (`public`), nesne gerektirmeyen (`static`), değer döndürmeyen (`void`), adı `main` olan ve parametre olarak bir String dizisi (`String[] args`, Bölüm 7) alan bir metot. JVM programı başlatırken tam olarak bu imzaya sahip metodu arar.

6.3 void ve değer döndüren metotlar

Metotlar iki gruba ayrılır:

- `void` **metotlar** bir **iş yapar** (ekrana yazmak gibi), ama geriye değer **vermez**. Çağrısı tek başına bir komuttur: `cizgiCiz();`
- **Değer döndüren metotlar** bir **hesap yapar** ve sonucu `return` ile çağırana **geri verir**. Dönüş tipi, döndürülen değerin tipidir.

DonusTipleri.java

```java
public class DonusTipleri {
    public static void selamla(String ad) {
        System.out.println("Merhaba " + ad + "!");
    }

    public static int kare(int n) {
        return n * n;
    }

    public static double ortalama(int a, int b) {
        return (a + b) / 2.0;
    }

    public static void main(String[] args) {
        selamla("Ece");
        int k = kare(5);
        kare(9);                        // dönen 81 kullanılmadı, kayboldu
        System.out.println(k + " " + kare(k));
        System.out.println(ortalama(7, 8));
    }
}
```

Çıktı

```
Merhaba Ece!
25 625
7.5
```

- `kare(9);` derlenir ve çalışır, ama dönen değer hiçbir yerde kullanılmadığı için **kaybolur**. Java bunu hata saymaz.
- `ortalama`'da `2.0` yerine `2` yazsaydık tam sayı bölmesi yüzünden `7.0` dönerdi. Dönüş tipinin `double` olması bölmeyi ondalıklı **yapmaz**.
- `return` ifadesinin tipi dönüş tipine uymalıdır: Genişleyen dönüşüm otomatiktir (`double` metotta `return 5;` olur), daralan değildir (`int` metotta `return 2.5;` derlenmez).

Sık Yapılan Hata: void metodun "değerini" kullanmak

`void` metot değer üretmediği için bir ifadenin içinde kullanılamaz. `int x = selamla("Ali");` ya da `println` içinde çağırmak derleme hatasıdır:

```
public class VoidHata {
    public static void selamla() {
        System.out.println("Selam");
    }

    public static void main(String[] args) {
        System.out.println(selamla());
    }
}
```

Derleyici çıktısı – program derlenmez

```
VoidHata.java:7: error: 'void' type not allowed here
        System.out.println(selamla());
                                   ^
1 error
```

Sınavda Çıkar: Yazdırmak mı, döndürmek mi?

Soruda "... değerini **döndüren** bir metot yaz" deniyorsa metodun içinde `println` değil `return` olmalı; yazdırma işini `main` yapar. Değer döndüren metot daha esnektir: Sonucu yazdırabilir, saklayabilir ya da başka bir hesapta kullanabilirsin. Metot sadece `println` yaparsa sonuç ekrana gider ve program onu bir daha kullanamaz.

6.4 return ile erken çıkış

`return` çalıştığı anda metot **biter**; altındaki satırlar çalışmaz ve kontrol çağırana döner. Bu yüzden bir metotta birden fazla `return` olabilir. `void` metottan da değersiz `return;` ile erken çıkılır:

ErkenCikis.java

```
public class ErkenCikis {
    public static String harfNotu(int puan) {
        if (puan < 0 || puan > 100) {
            return "Geçersiz";
        }
        if (puan >= 85) {
            return "AA";
        }
        if (puan >= 60) {
            return "CC";
        }
        return "FF";
    }

    public static void bolumuYaz(int a, int b) {
        if (b == 0) {
            System.out.println("Sıfıra bölünemez!");
            return;                    // void metottan erken çıkış
        }
        System.out.println(a + " / " + b + " = " + (a / b));
    }

    public static void main(String[] args) {
```

```
        System.out.println(harfNotu(92) + " " + harfNotu(70) + " " + harfNotu(150));
        bolumuYaz(17, 5);
        bolumuYaz(17, 0);
    }
}
```

Çıktı

```
AA CC Geçersiz
17 / 5 = 3
Sıfıra bölünemez!
```

`harfNotu(92)` çağrısında ikinci `if` doğru olur ve `"AA"` döner; `puan >= 60` kontrolü **hiç çalışmaz**. Her `return` metodu bitirdiği için `else` yazmaya gerek kalmadı. `bolumuYaz(17, 0)` çağrısında da `return;` sayesinde bölme satırına hiç gelinmedi. `return`'den hemen sonra, aynı blokta bir komut yazarsan derleyici ona asla ulaşılamayacağını fark eder: *unreachable statement* hatası.

"missing return statement" hatası

Dönüş tipi `void` olmayan bir metotta derleyici, metodun **her olası yoldan** bir değer döndürdüğünden emin olmak ister. Bir yol bile `return`'süz biterse program derlenmez. İşin ilginç yanı, derleyici koşulların değerini **hesaplamaz**:

EksikReturn.java

```
public class EksikReturn {
    public static String isaret(int n) {
        if (n >= 0) {
            return "pozitif veya sıfır";
        }
        if (n < 0) {
            return "negatif";
        }
    }

    public static void main(String[] args) {
        System.out.println(isaret(-3));
    }
}
```

Derleyici çıktısı – program derlenmez

```
EksikReturn.java:9: error: missing return statement
    }
    ^
1 error
```

Sen biliyorsun ki iki koşuldan biri mutlaka doğrudur. Derleyici ise iki ayrı `if` görür ve "ikisi de yanlış olursa ne dönecek?" diye sorar. Çözüm: İkinci `if`'i kaldırıp metodu düz bir `return "negatif";` ile bitirmek ya da `if-else` kullanmak; `else` dalı olan bir `if-else`'te iki daldan birinin mutlaka çalışacağını derleyici bilir.

> **Sınavda Çıkar**
>
> "Aşağıdaki metot derlenir mi?" sorularında metodun **son satırına** bak. Son komut `return` değilse ve son `if`'in `else` dalı yoksa cevap büyük ihtimalle *missing return statement*'tır. Döngünün içindeki `return` de yetmez: Döngü hiç dönmeyebilir, döngüden sonra da bir `return` gerekir (aşağıdaki `asalMi`'ye bak).

6.5 Parametre ve argüman

Bu iki kelime sık karıştırılır, ama sınavda ayrımı sorulur:

	Parametre (parameter)	Argüman (argument)
Nerede?	Metot **tanımında**	Metot **çağrısında**
Nedir?	Değeri bekleyen bir değişken	Gönderilen gerçek değer
Örnek	`cizgi(char karakter, int uzunluk)`	`cizgi('*', 10)`

Çağrıda argümanlar **sırayla** parametrelere atanır: birinci argüman birinci parametreye, ikinci argüman ikinci parametreye... Bu yüzden argümanların **sayısı**, **sırası** ve **tipi** parametrelerle uyumlu olmalıdır. Argüman bir değişken, bir sabit ya da `n + 1` gibi bir ifade olabilir; önce değeri hesaplanır, sonra parametreye konur.

Parametreler.java

```java
public class Parametreler {
    public static void cizgi(char karakter, int uzunluk) {
        for (int i = 0; i < uzunluk; i++) {
            System.out.print(karakter);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        cizgi('*', 10);
        int n = 3;
        cizgi('#', n + 1);
        System.out.println(Math.sqrt(16));    // int argüman, double parametre
    }
}
```

Çıktı

```
**********
####
4.0
```

`cizgi('#', n + 1)` çağrısında `main`'deki değişkenin adı `n`, metottaki parametrenin adı `uzunluk`: Parametre adlarının argümanların adlarıyla hiçbir ilgisi yoktur. `Math.sqrt(16)` çağrısında `int` argüman `double` parametreye **otomatik** dönüştürüldü (genişleyen dönüşüm). Tersi olmaz: `int` parametreli bir metoda `2.5` gönderirsen *possible lossy conversion*, eksik ya da fazla argüman gönderirsen *cannot be applied to given types* derleme hatası alırsın.

6.6 Metot çağrıldığında ne olur?

Bir metot çağrıldığında sırasıyla şunlar olur:

1. Argümanların değerleri hesaplanır ve metodun parametrelerine **kopyalanır**.
2. Çağıran metot (ör. `main`) olduğu yerde **bekler**; çağrılan metodun gövdesi çalışır.
3. `return` ile (ya da `void` metotta gövdenin sonuna gelince) kontrol, çağrının yapıldığı **noktaya** geri döner. Dönen değer varsa çağrı ifadesinin yerine geçer.

Metotlar başka metotları da çağırabilir. Akışı görmek için her metoda mesaj yazdıralım:

CagriAkisi.java

```java
public class CagriAkisi {
    public static int kare(int n) {
        System.out.println("    kare(" + n + ") başladı");
        return n * n;
    }

    public static int kareToplami(int a, int b) {
        System.out.println("  kareToplami(" + a + ", " + b + ") başladı");
        int t = kare(a) + kare(b);
        System.out.println("  kareToplami " + t + " döndürüyor");
        return t;
    }

    public static void main(String[] args) {
        System.out.println("main başladı");
        int s = kareToplami(3, 4);
        System.out.println("main: sonuç = " + s);
    }
}
```

Çıktı

```
main başladı
  kareToplami(3, 4) başladı
    kare(3) başladı
    kare(4) başladı
  kareToplami 25 döndürüyor
main: sonuç = 25
```

Çağrı yığını

Java, hangi metodun hangisini çağırdığını **çağrı yığını** (call stack) denen bir yapıda tutar. Her çağrıda yığının en üstüne o metoda ait bir **çerçeve** (stack frame) eklenir; çerçevede metodun parametreleri ve yerel değişkenleri bulunur. Metot bitince çerçevesi atılır ve içindeki değişkenler **yok olur**. Çalışan metot her zaman en üsttekidir. `kare(3)` çalışırken yığın şöyledir:

```
+---------------------------+
| kare         n=3          |   <- en üst: şu an çalışan
+---------------------------+
| kareToplami  a=3, b=4     |   <- kare'nin bitmesini bekliyor
+---------------------------+
| main         args         |   <- kareToplami'nin bitmesini bekliyor
+---------------------------+
```

Bu yığını aslında Bölüm 1'den beri görüyorsun: Bir exception oluştuğunda Java o anki çağrı yığınını ekrana döker (*stack trace*):

YiginIzi.java

```java
public class YiginIzi {
    public static int bol(int a, int b) {
        return a / b;
    }

    public static int ortalama(int toplam, int adet) {
        return bol(toplam, adet);
    }

    public static void main(String[] args) {
        System.out.println(ortalama(30, 3));
        System.out.println(ortalama(30, 0));
    }
}
```

Çıktı – program hata ile durur

```
10
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at YiginIzi.bol(YiginIzi.java:3)
        at YiginIzi.ortalama(YiginIzi.java:7)
        at YiginIzi.main(YiginIzi.java:12)
```

İpucu: Yığın izini okumak

`at` satırlarını **yukarıdan aşağıya** oku: En üstteki satır hatanın **oluştuğu** yer (`bol`, satır 3), altındakiler sırayla onu **çağıranlar** (`ortalama` satır 7, `main` satır 12). Hatalı argümanın hangi çağrıdan geldiğini bu satırlardan bulursun.

6.7 Aşırı yükleme (overloading)

Aynı sınıfta, **aynı adı** taşıyan ama **parametre listeleri farklı** olan birden fazla metot tanımlayabilirsin. Buna **aşırı yükleme** (overloading) denir. Derleyici, argümanlara bakarak hangi sürümün çalışacağına **derleme zamanında** karar verir:

AsiriYukleme.java

```java
public class AsiriYukleme {
    public static int topla(int a, int b) {
        System.out.print("[int,int] ");
        return a + b;
    }

    public static int topla(int a, int b, int c) {
        System.out.print("[int,int,int] ");
        return a + b + c;
    }

    public static double topla(double a, double b) {
        System.out.print("[double,double] ");
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(topla(2, 3));
        System.out.println(topla(2, 3, 4));
```

```
        System.out.println(topla(2.5, 1.5));
        System.out.println(topla(2, 3.5));
    }
}
```

Çıktı

```
[int,int] 5
[int,int,int] 9
[double,double] 4.0
[double,double] 5.5
```

`topla(2, 3.5)` için `(int, double)` alan bir sürüm yok. Derleyici önce **birebir uyan** sürümü arar; bulamazsa **genişleyen dönüşümle** uyanı seçer: `2` → `2.0` yapılır ve `topla(double, double)` çağrılır.

Hangi metotların birlikte var olabileceğini **metot imzası** (method signature) belirler: İmza = **metodun adı + parametrelerin tipleri** (sayısı ve sırasıyla). **Dönüş tipi, parametre adları ve `public`/`static` gibi belirteçler imzaya dahil değildir.** Bir sınıfta aynı imzalı iki metot olamaz.

Birinci metot	İkinci metot	Geçerli aşırı yükleme mi?
`int f(int a)`	`int f(int a, int b)`	Evet: parametre **sayısı** farklı
`int f(int a)`	`int f(double a)`	Evet: parametre **tipi** farklı
`void f(int a, String s)`	`void f(String s, int a)`	Evet: parametre **sırası** farklı
`int f(int a)`	`int f(int b)`	**Hayır**: sadece parametre adı farklı
`int f(int a)`	`double f(int a)`	**Hayır**: sadece dönüş tipi farklı

ImzaHata.java

```
public class ImzaHata {
    public static int hesapla(int x) {
        return x * 2;
    }

    public static double hesapla(int x) {
        return x * 2.5;
    }

    public static void main(String[] args) {
        hesapla(4);
    }
}
```

Derleyici çıktısı – program derlenmez

```
ImzaHata.java:6: error: method hesapla(int) is already defined in class ImzaHata
    public static double hesapla(int x) {
                         ^
1 error
```

Dönüş tipinin imzaya dahil olmamasının sebebi `main`'deki satırda: `hesapla(4);` çağrısında dönen değer kullanılmıyor, dolayısıyla hangi metodun kastedildiği anlaşılamaz.

Sınavda Çıkar

- **"Hangisi geçerli bir aşırı yüklemedir?"** Sadece parametre listesine bak: sayı, tip veya sıra farklıysa geçerli. Dönüş tipi ya da parametre adı farkı **yetmez**.
- **"Hangi sürüm çağrılır?"** Önce birebir uyan; yoksa genişleyen dönüşümle uyan (`int` → `long` → `double`). `char` argüman, `int` parametreli sürüme gider.

`System.out.println` her tip için ayrı yazılmış (`println(int)`, `println(double)`, `println(String)`...) aşırı yüklenmiş bir metottur. Bölüm 11'deki **override** (geçersiz kılma) ile karıştırma; o bambaşka bir kavram.

Değişken sayıda argüman (varargs)

`topla(a, b)` ve `topla(a, b, c)` yazdık; ya 5 ya da 10 sayıyı toplamak istersen? Her adet için ayrı sürüm yazmak yerine parametre tipinden sonra üç nokta (`...`) yazabilirsin: Metot **istediğin sayıda** argüman alır. Buna **değişken sayıda argüman** (varargs, *variable arguments*) denir:

DegiskenArguman.java

```
public class DegiskenArguman {
    public static int topla(int... sayilar) {
        int toplam = 0;
        for (int i = 0; i < sayilar.length; i++) {
            toplam += sayilar[i];
        }
        System.out.print("[" + sayilar.length + " argüman] ");
        return toplam;
    }

    public static int topla(int a, int b) {
        System.out.print("[tam iki] ");
        return a + b;
    }

    public static void ortalama(String ad, int... notlar) {
        if (notlar.length == 0) {
            System.out.println(ad + ": not yok");
            return;
        }
        int toplam = 0;
        for (int i = 0; i < notlar.length; i++) {
            toplam += notlar[i];
        }
        System.out.printf("%s: %d not, ortalama %.1f%n",
                ad, notlar.length, (double) toplam / notlar.length);
    }

    public static void main(String[] args) {
        System.out.println(topla());
        System.out.println(topla(5));
        System.out.println(topla(2, 3));
        System.out.println(topla(2, 3, 4));
        System.out.println(topla(1, 2, 3, 4, 5, 6));
        ortalama("Ece", 80, 90, 70);
        ortalama("Can");
    }
}
```

Çıktı

```
[0 argüman] 0
[1 argüman] 5
[tam iki] 5
[3 argüman] 9
[6 argüman] 21
Ece: 3 not, ortalama 80.0
Can: not yok
```

- Metodun içinde `sayilar` sıradan bir **dizidir** (`int[]`): `sayilar.length` eleman sayısı, `sayilar[0]` ilk elemandır. Dizileri Bölüm 7'de ayrıntılı göreceksin; şimdilik "numaralı kutular" diye düşün. Hiç argüman verilmezse dizi boştur (uzunluk 0): `topla()` sonucu 0.
- Varargs parametresi **yalnız son parametre** olabilir ve bir metotta **en fazla bir tane** bulunur. `ortalama(String ad, int... notlar)` geçerlidir; `(int... notlar, String ad)` geçersizdir.
- `topla(int a, int b)` ile `topla(int... sayilar)` birlikte varsa `topla(2, 3)` **sabit sayılı** sürümü seçer (`[tam iki]` satırı); varargs sürümü en son çaredir.
- `printf` ve `String.format` (Bölüm 3) de böyle yazılmıştır: `printf(String format, Object... args)`. Biçim metninden sonra istediğin kadar değer verebilmenin sebebi budur.

Sık Yapılan Hata: Varargs son parametre olmalı

Üç noktalı parametreyi başa ya da ortaya yazarsan metot derlenmez:

```
public class VarargsHata {
    public static void yaz(int... sayilar, String ad) {
    }

    public static void main(String[] args) {
    }
}
```

Derleyici çıktısı – program derlenmez

```
VarargsHata.java:2: error: varargs parameter must be the last parameter
    public static void yaz(int... sayilar, String ad) {
                                           ^
1 error
```

6.8 Değer ile geçirme (pass-by-value)

Java'da argümanlar **her zaman değer ile** geçirilir (*pass-by-value*): Metoda değişkenin kendisi değil, değerinin bir **kopyası** gider. Parametre, metodun kendi yerel değişkenidir. Metot içinde parametreyi değiştirmek çağırandaki değişkeni **etkilemez**:

DegerIleGecirme.java

```
public class DegerIleGecirme {
    public static void artir(int n) {
        n = n + 10;
        System.out.println("artir içinde n = " + n);
    }

    public static void swap(int a, int b) {
        int gecici = a;
        a = b;
        b = gecici;
        System.out.println("swap içinde a = " + a + ", b = " + b);
```

```
    }

    public static void main(String[] args) {
        int sayi = 5;
        artir(sayi);
        System.out.println("main'de sayi = " + sayi);

        int x = 1, y = 2;
        swap(x, y);
        System.out.println("main'de x = " + x + ", y = " + y);
    }
}
```

Çıktı

```
artir içinde n = 15
main'de sayi = 5
swap içinde a = 2, b = 1
main'de x = 1, y = 2
```

`swap` kendi içinde değerleri değiştirdi, ama `main`'deki `x` ve `y` hiç etkilenmedi:

An	`main`: x	`main`: y	`swap`: a	`swap`: b
`swap(x, y)` çağrıldı, değerler kopyalandı	1	2	1	2
`a` ile `b` yer değiştirdi	1	2	2	1
`swap` bitti, çerçevesi silindi	1	2	—	—

Sonucu çağırana iletmenin doğru yolu `return`'dür: `artir` metodunu `public static int artir(int n) { return n + 10; }` biçiminde yazıp çağıran tarafta dönen değeri geri atarsın: `sayi = artir(sayi);`.

> **Sınavda Çıkar: Java'da pass-by-reference yoktur**
>
> "Java'da parametreler nasıl geçirilir?" sorusunun cevabı her zaman **değer ile** (pass-by-value) olur. İki `int`'i yer değiştiren bir `swap(int a, int b)` metodu Java'da **yazılamaz**. Bölüm 7'de diziyi metoda gönderdiğinde metottaki değişikliklerin dışarıda göründüğünü fark edeceksin. Kural yine aynıdır; sadece kopyalanan değer bir **referanstır**.

> **Sık Yapılan Hata: Aynı isim, aynı değişken değildir**
>
> `artir` metodunun parametresinin adını `sayi` yapsan bile sonuç değişmez: Metottaki `sayi` ile `main`'deki `sayi` **iki ayrı değişkendir**, sadece adları aynıdır. Metot, kendi kopyasını değiştirir.

6.9 Kapsam (scope) ve yerel değişkenler

Bir değişkenin programın neresinden kullanılabildiğine **kapsam** (scope) denir. Metodun içinde tanımlanan değişkenlere **yerel değişken** (local variable) denir. Kurallar:

1. Yerel değişken, tanımlandığı satırdan itibaren içinde bulunduğu `{ }` bloğunun **sonuna kadar** geçerlidir. Parametreler, metodun tamamında geçerli yerel değişkenlerdir.
2. `for (int i = ...)` başlığında tanımlanan `i` sadece döngünün içinde yaşar.
3. Farklı metotlardaki aynı isimli değişkenler birbirinden **tamamen bağımsızdır**.
4. Bir değişken hâlâ kapsamdayken, onun adıyla iç blokta yeni bir değişken tanımlanamaz.

Kapsam.java

```java
public class Kapsam {
    public static void yazdir() {
        int x = 100;                  // main'deki x ile ilgisi yok
        System.out.println("yazdir: x = " + x);
    }

    public static void main(String[] args) {
        int x = 1;
        yazdir();
        System.out.println("main: x = " + x);
        for (int i = 1; i <= 3; i++) {
            int kare = i * i;         // her turda yeniden oluşur
            System.out.print(kare + " ");
        }
        System.out.println();         // burada i ve kare artık yok
    }
}
```

Çıktı

```
yazdir: x = 100
main: x = 1
1 4 9
```

Kapsam dışındaki bir değişkeni kullanmak ya da kapsamdaki bir değişkeni yeniden tanımlamak derleme hatasıdır:

KapsamHata.java

```java
public class KapsamHata {
    public static void main(String[] args) {
        int x = 5;
        for (int i = 0; i < 3; i++) {
            int x = i;
        }
        System.out.println(i);
    }
}
```

Derleyici çıktısı – program derlenmez

```
KapsamHata.java:5: error: variable x is already defined in method main(String[])
            int x = i;
                ^
KapsamHata.java:7: error: cannot find symbol
        System.out.println(i);
                           ^
  symbol:   variable i
  location: class KapsamHata
2 errors
```

Sık Yapılan Hata: Döngüde hesaplanan değeri döngüden sonra kullanmak

`for` içinde `int toplam = 0;` tanımlayıp döngüden sonra `toplam`'ı yazdırmaya çalışırsan *cannot find symbol* alırsın. Üstelik değişken her turda yeniden 0 yapılır. Döngüden sonra kullanacağın değişkeni **döngüden önce** tanımla.

Kapsam kuralı, metotlar arasında veri aktarmanın yolunun parametre ve `return` olduğu anlamına gelir: `yazdir` metodu `main`'in yerel değişkenlerini göremez. (Sınıf düzeyindeki değişkenleri, yani alanları, Bölüm 9'da göreceksin.)

6.10 Neden şimdilik static?

Bu bölümdeki bütün metotlar `static`. `static` bir metot **nesne oluşturmadan** çağrılabilir. `main` de `static` olduğu için, `main`'in içinden aynı sınıftaki bir metodu doğrudan çağırabilmek için o metodun da `static` olması gerekir. `static` yazmayı unutursan:

```
public class StaticHata {
    public int kare(int n) {
        return n * n;
    }

    public static void main(String[] args) {
        System.out.println(kare(6));
    }
}
```

Derleyici çıktısı – program derlenmez

```
StaticHata.java:7: error: non-static method kare(int) cannot be referenced from a static context
        System.out.println(kare(6));
                           ^
1 error
```

`static` olmayan metotlar **nesneye ait** metotlardır ve bir nesne üzerinden çağrılır (`sc.nextInt()` gibi; `sc` bir Scanner nesnesidir). Nesneleri ve `static`'in tam anlamını Bölüm 9'da öğreneceksin. O zamana kadar kural basit: **Yazdığın her metodun başına `public static` koy.**

6.11 Javadoc yorumları

Bölüm 1'de gördüğün `/** ... */` yorumları metotları belgelemek için kullanılır. Metodun **hemen üstüne** yazılır; her parametre için bir `@param`, dönen değer için bir `@return` etiketi eklenir (`void` metotta `@return` yazılmaz):

JavadocOrnek.java

```
public class JavadocOrnek {
    /**
     * Bir sayının belirtilen kuvvetini tam sayı olarak hesaplar.
     *
     * @param taban kuvveti alınacak sayı
     * @param us    negatif olmayan üs
     * @return taban üzeri us (ör. usAl(2, 10) = 1024)
     */
    public static long usAl(int taban, int us) {
        long sonuc = 1;
        for (int i = 0; i < us; i++) {
            sonuc *= taban;
        }
        return sonuc;
    }

    public static void main(String[] args) {
        System.out.println(usAl(2, 10) + " " + usAl(7, 0));
    }
}
```

Çıktı

```
1024 1
```

VS Code'da metodun üstüne `/**` yazıp Enter'a basarsan `@param` ve `@return` satırları otomatik oluşur; metodun adının üzerine fareyle geldiğinde de açıklaman görünür. Hocalar ödevlerde Javadoc'u sık sık puanlar.

6.12 Algoritmaları metotlara dönüştürmek

Bölüm 5'te döngülerle yazdığın asal sayı, faktöriyel, EBOB ve basamak toplamı algoritmaları metot olmaya biçilmiş kaftandır: Her biri bir girdi alır, bir sonuç üretir.

Algoritmalar.java

```
public class Algoritmalar {
    public static boolean asalMi(int n) {
        if (n < 2) {
            return false;
        }
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                return false;          // bölen bulundu, hemen çık
            }
        }
        return true;
    }

    public static long faktoriyel(int n) {
        long sonuc = 1;
        for (int i = 2; i <= n; i++) {
            sonuc *= i;
        }
        return sonuc;
    }

    public static int ebob(int a, int b) {
        while (b != 0) {               // Öklid algoritması
            int kalan = a % b;
            a = b;
            b = kalan;
        }
        return Math.abs(a);            // negatif girdide de pozitif
    }

    public static int basamakToplami(int n) {
        int toplam = 0;
        while (n > 0) {
            toplam += n % 10;
            n /= 10;
        }
        return toplam;
    }

    public static void main(String[] args) {
        System.out.print("40'a kadar asallar: ");
        for (int i = 1; i <= 40; i++) {
            if (asalMi(i)) {
                System.out.print(i + " ");
            }
        }
        System.out.println();
        System.out.println("20! = " + faktoriyel(20));
        int x = 48, y = 36;
        int e = ebob(x, y);
        System.out.println("ebob = " + e + ", ekok = " + (x / e * y));
```

```
        System.out.println("x = " + x + ", y = " + y);
        System.out.println("2026 basamak toplamı: " + basamakToplami(2026));
    }
}
```

Çıktı

```
40'a kadar asallar: 2 3 5 7 11 13 17 19 23 29 31 37
20! = 2432902008176640000
ebob = 12, ekok = 144
x = 48, y = 36
2026 basamak toplamı: 10
```

- `asalMi`'de `return false;` döngünün içinde: Bölen bulunur bulunmaz metot biter; `break`'e ve bayrak değişkenine gerek kalmaz. Döngü bölen bulamadan biterse sondaki `return true;` çalışır.
- `faktoriyel` `long` döndürür, çünkü 13! `int`'e sığmaz. 20! `long`'a sığan en büyük faktöriyeldir; 21! taşar (Bölüm 2).
- `ebob` parametreleri `a` ve `b`'yi değiştiriyor; ama değer ile geçirme sayesinde `main`'deki `x` ve `y` aynen kaldı.

boolean döndüren metotlar

`asalMi` gibi "evet/hayır" sorusuna cevap veren metotlar `boolean` döndürür. Adlarını bir soru gibi koymak gelenektir: `asalMi`, `ciftMi`, `gecerliMi`. Bu metotlar doğrudan `if`'in ya da `while`'ın koşuluna yazılır (`if (asalMi(i))`) ve gövdeleri çoğu zaman tek bir `return` ile yazılabilir:

```
public static boolean artikYilMi(int yil) {
    return (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
}
```

`artikYilMi(2024)` `true`, `artikYilMi(1900)` ise `false` döndürür (1900, 100'e bölünüp 400'e bölünmediği için artık yıl değildir).

> **İpucu: Acemi boolean kalıpları**
>
> `if (n % 2 == 0) { return true; } else { return false; }` yerine doğrudan `return n % 2 == 0;` yaz; karşılaştırmanın kendisi zaten `true` ya da `false`'tur. Aynı şekilde `if (asalMi(n) == true)` yerine `if (asalMi(n))`, `if (asalMi(n) == false)` yerine `if (!asalMi(n))` yaz.

6.13 Hazır metotlar ve Math sınıfı

Bölüm 2'de kullandığın `Math.sqrt`, `Math.pow`, `Math.max` gibi metotlar senin yazdıklarından farksızdır: `Math` adlı sınıfın içinde tanımlanmış `public static` metotlardır. Başka bir sınıftaki `static` metodu `SinifAdi.metotAdi(...)` biçiminde çağırırsın; kendi sınıfındaki metot için sınıf adı gerekmez. Bir metodu doğru kullanmak için imzasını, özellikle **dönüş tipini** bilmelisin:

Metodun imzası (sadeleştirilmiş)	Dikkat
`static double sqrt(double a)`	`Math.sqrt(16)` → `4.0`; `int` versen de `double` döner
`static double pow(double a, double b)`	`Math.pow(2, 3)` → `8.0`
`static long round(double a)`	`long` döner, `int`'e atamak için cast gerekir
`int abs(int a)`, `double abs(double a)` …	Aşırı yüklenmiş: `Math.abs(-7)` → `7`, `Math.abs(-7.5)` → `7.5`
`int max(int a, int b)`, `double max(double a, double b)` …	`Math.max(3, 8.0)` → `8.0` (genişleyen dönüşüm)

Sınavda Çıkar: Dönüş tipini bilmek zorundasın

`int k = Math.sqrt(16);` ve `int r = Math.round(2.7);` satırları **derlenmez**: `sqrt` `double`, `round` ise `long` döndürür ve ikisi de `int`'e otomatik daralmaz. Doğrusu `int k = (int) Math.sqrt(16);` ve `int r = (int) Math.round(2.7);`.

6.14 Özyinelemeye kısa bir bakış

Bir metot başka metotları çağırabildiği gibi **kendini de** çağırabilir. Buna **özyineleme** (recursion) denir. Faktöriyel şöyle de tanımlanabilir: $n! = n \times (n-1)!$ ve $1! = 1$.

OzyinelemeOnBakis.java

```
public class OzyinelemeOnBakis {
    public static long faktoriyel(int n) {
        if (n <= 1) {
            return 1;                  // temel durum: burada dur
        }
        return n * faktoriyel(n - 1);  // kendini daha küçük n ile çağır
    }

    public static void main(String[] args) {
        System.out.println(faktoriyel(5));
    }
}
```

Çıktı

```
120
```

`faktoriyel(5)` = `5 * faktoriyel(4)` = `5 * 4 * faktoriyel(3)` = … = `5 * 4 * 3 * 2 * 1`. Her çağrı yığına **yeni bir çerçeve** ekler; en derinde aynı anda beş `faktoriyel` çerçevesi bulunur, her birinin kendi `n`'i vardır. **Temel durum** (base case, burada `n <= 1`) unutulursa metot kendini durmadan çağırır, yığın dolar ve program `StackOverflowError` ile çöker. Ayrıntılar Bölüm 15'te.

Bölüm Özeti – Bilmen Gerekenler

- **Metot**, bir işi yapan ve adıyla çağrılan kod bloğudur. Tekrarı önler, kodu okunur ve test edilebilir yapar.
- Anatomi: `public static dönüşTipi ad(parametreler) { gövde }`. Metotlar sınıfın içine, başka metodun dışına yazılır.
- `void` metot değer döndürmez ve ifade içinde kullanılamaz. Değer döndüren metot `return ifade;` ile sonucu çağırana verir; kullanılmayan dönüş değeri kaybolur.

- `return` metodu **hemen** bitirir. `void` metotta `return;` erken çıkış sağlar.
- Değer döndüren metodun **her yolu** `return` ile bitmeli; yoksa *missing return statement*. Derleyici `if` koşullarının değerini hesaplamaz.
- **Parametre** tanımdaki değişken, **argüman** çağrıdaki değer. Sayı, sıra ve tip uyumlu olmalı; genişleyen dönüşüm otomatik.
- Her çağrı **çağrı yığınına** bir çerçeve ekler; metot bitince yerel değişkenleri yok olur. Stack trace'te en üst satır hatanın oluştuğu metottur.
- **Aşırı yükleme:** aynı ad, farklı parametre listesi. **İmza** = ad + parametre tipleri; dönüş tipi ve parametre adları imzaya dahil **değil**.
- **Varargs:** `int... sayilar` metoda istenen sayıda argüman verir; içeride dizi gibi davranır (`sayilar.length`). Yalnız **son** parametre olabilir; sabit sayılı sürüm varsa o önce seçilir.
- Java **her zaman değer ile geçirir**: Parametreyi değiştirmek çağırandaki değişkeni etkilemez; `swap(int, int)` çalışmaz. Sonucu `return` ile ilet.
- Yerel değişken tanımlandığı bloğun sonuna kadar yaşar. Farklı metotlardaki aynı isimli değişkenler bağımsızdır.
- Şimdilik her metot `public static`. Başka sınıftaki static metot `SinifAdi.metot()` ile çağrılır (`Math.sqrt`).

Sorular

A) Çoktan Seçmeli

1 Aşağıdakilerden hangisi bir metodun **imzasına** (signature) dahil **değildir**?

(A) Metodun adı
(B) Parametrelerin sayısı
(C) Parametrelerin tipleri
(D) Parametrelerin sırası
(E) Metodun dönüş tipi

2 Aşağıdaki metodun **derlenebilmesi** için boşluğa hangisi yazılmalıdır?

```java
public static ______ ortalama(int a, int b) {
    return (a + b) / 2.0;
}
```

(A) `double` (B) `int` (C) `void` (D) `long` (E) `String`

3 Aşağıdaki program ekrana ne yazdırır?

```
public class Test3 {
    public static void ikiKat(int n) {
        n = n * 2;
    }

    public static void main(String[] args) {
        int a = 5;
        ikiKat(a);
        ikiKat(a);
        System.out.println(a);
    }
}
```

(A) `20`
(B) `5`
(C) `10`
(D) `0`
(E) Derleme hatası verir

4 Aşağıdaki metot çiftlerinden hangisi **aynı sınıfta birlikte** tanımlanabilir?

(A) `int f(int a)` ve `double f(int a)`
(B) `int f(int a)` ve `int f(int b)`
(C) `static void f(int a)` ve `void f(int a)`
(D) `void f(int a, double b)` ve `void f(double a, int b)`
(E) `void f(int a)` ve `int f(int x)`

5 Aşağıdaki metot için hangisi doğrudur?

```
public static int enBuyuk(int a, int b) {
    if (a >= b) {
        return a;
    }
    if (a < b) {
        return b;
    }
}
```

(A) Her zaman büyük olan sayıyı döndürür.
(B) `a` ile `b` eşitse 0 döndürür.
(C) *missing return statement* hatası verir, derlenmez.
(D) Derlenir, ama çalışırken exception fırlatır.
(E) Her zaman `b`'yi döndürür.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru6 {
    public static int f(int x) {
        System.out.print("f" + x + " ");
        return x * 2;
    }

    public static void main(String[] args) {
        int a = f(1) + f(2);
        System.out.println();
        System.out.println(a);
        System.out.println(f(f(3)));
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru7 {
    public static int hesapla(int a, int b) {
        a = a + b;
        b = a - b;
        return a * b;
    }

    public static void main(String[] args) {
        int a = 3, b = 4;
        int c = hesapla(b, a);
        System.out.println(a + " " + b + " " + c);
        System.out.println(hesapla(a, a) + a);
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru8 {
    public static void yaz(int x) { System.out.println("int: " + x); }
    public static void yaz(double x) { System.out.println("double: " + x); }
    public static void yaz(String s) { System.out.println("String: " + s); }
    public static void yaz(int a, int b) { System.out.println("iki: " + (a + b)); }

    public static void main(String[] args) {
        yaz(5 / 2);
        yaz(5 / 2.0);
        yaz("5" + 2);
        yaz('A');
        yaz(5L);
        yaz(2, 3);
    }
}
```

C) Kod Yazma

9 Bir sayının **kendisi hariç** pozitif bölenlerinin toplamını döndüren `bolenToplami(int n)` ve bu toplam sayının kendisine eşitse `true` döndüren `mukemmelMi(int n)` metotlarını yaz. `MukemmelSayilar` programı bu metotları kullanarak 1 ile 10000 arasındaki **mükemmel sayıları** aynı satıra yazdırsın. (Örnek: 28'in bölenleri 1 + 2 + 4 + 7 + 14 = 28 olduğu için 28 mükemmel sayıdır.)

```
1–10000 arası mükemmel sayılar: 6 28 496 8128
```

10 Bir tam sayının basamaklarını ters çevirip döndüren `tersCevir(int n)` ve sayı tersine eşitse `true` döndüren `palindromMu(int n)` metotlarını yaz. `Palindrom` programı kullanıcı **0 girene kadar** sayı okusun ve her sayı için sonucu yazdırsın:

```
Sayı (çıkış için 0): 12321
12321 -> tersi 12321, palindrom
Sayı (çıkış için 0): 1230
1230 -> tersi 321, palindrom değil
Sayı (çıkış için 0): 0
```

Çözümler

Cevap anahtarı

1	2	3	4	5
E	A	B	D	C

1 Cevap: E

Metot imzası **ad + parametre tipleri** (sayısı ve sırasıyla birlikte) demektir. Dönüş tipi imzaya dahil değildir; bu yüzden yalnızca dönüş tipi farklı olan iki metot aynı sınıfta bulunamaz.

2 Cevap: A

`(a + b) / 2.0` ifadesinin tipi `double`'dır. `int` ve `long` için `double` → tam sayı daralan dönüşüm gerekir, otomatik yapılmaz (*possible lossy conversion*). `void` metot değer döndüremez, `String` ise `double` ile uyumsuzdur.

3 Cevap: B

Java değer ile geçirir: `ikiKat` her çağrıda `a`'nın **kopyasını** alır ve kendi `n`'ini 10 yapar. `main`'deki `a` hiç değişmez; iki kez çağırmak da bir şey değiştirmez.

Çıktı

```
5
```

4 Cevap: D

D'deki metotların parametre **sırası** farklı (`int, double` ve `double, int`), yani imzaları farklıdır; geçerli bir aşırı yüklemedir. A ve E'de sadece dönüş tipi, B'de sadece parametre adı, C'de sadece `static` belirteci farklı; imzalar aynı olduğu için bunlar derlenmez.

5 Cevap: C

Derleyici `if` koşullarının değerini hesaplamaz; "iki `if` de yanlış olursa ne döner?" diye bakar ve metodun sonunda `return` olmadığı için derlemez. Mantıken bu yol imkânsız olsa da kural böyledir. İkinci `if` yerine düz `return b;` yazılmalıydı.

Derleyici çıktısı – program derlenmez

```
Test5.java:9: error: missing return statement
    }
    ^
1 error
```

6

- `f(1) + f(2)`: Soldan sağa önce `f(1)` çalışır, `f1` yazar ve 2 döndürür; sonra `f(2)`, `f2` yazar ve 4 döndürür. `a = 6`. Bu satırda `println` olmadığı için imleç aynı satırda kalır; ardından gelen `println()` alt satıra geçirir.
- `f(f(3))`: Önce içteki `f(3)`: `f3` yazar, 6 döndürür. Sonra `f(6)`: `f6` yazar, 12 döndürür. `println` en son, argümanı hesaplandıktan **sonra** 12'yi yazar.

Çıktı
```
f1 f2
6
f3 f6 12
```

7

- `hesapla(b, a)` çağrısında argümanların **sırası** ters: metodun `a`'sı 4, `b`'si 3 olur. `a = 4 + 3 = 7`, `b = 7 - 3 = 4`, dönen değer `7 * 4 = 28`.
- Metot kendi kopyalarını değiştirdi; `main`'deki `a` = 3 ve `b` = 4 aynen kaldı.
- `hesapla(3, 3)`: `a = 6`, `b = 6 - 3 = 3`, dönen `18`. `main`'deki `a` (3) eklenir: `21`.

Çıktı
```
3 4 28
21
```

8

- `5 / 2` tam sayı bölmesi → `2` (`int`) → `int` sürümü. `5 / 2.0` → `2.5` → `double` sürümü. `"5" + 2` → `"52"` → `String` sürümü.
- `'A'` bir `char`'dır ve `char` alan sürüm yok. En yakın genişleyen dönüşüm `char` → `int` olduğu için `int` sürümü çağrılır ve `'A'`'nın kodu **65** yazılır.
- `5L` bir `long`'dur; `long` → `int` daraltma olduğundan `int` sürümü uygun değil. `long` → `double` genişlemesiyle `double` sürümü seçilir: `5.0`.
- `yaz(2, 3)` iki parametreli tek sürüme gider: `2 + 3 = 5`.

Çıktı
```
int: 2
double: 2.5
String: 52
int: 65
double: 5.0
iki: 5
```

9

`bolenToplami` 1'den `n / 2`'ye kadar dolaşır (bir sayının kendisi hariç hiçbir böleni yarısından büyük olamaz). `mukemmelMi` tek satırdır. `main` sadece 1–10000 arasını dolaşıp `mukemmelMi`'ye sorar.

MukemmelSayilar.java
```java
public class MukemmelSayilar {
    public static int bolenToplami(int n) {
        int toplam = 0;
        for (int i = 1; i <= n / 2; i++) {
            if (n % i == 0) {
                toplam += i;
            }
        }
        return toplam;
    }

    public static boolean mukemmelMi(int n) {
        return n > 1 && bolenToplami(n) == n;
    }
```

```
    public static void main(String[] args) {
        System.out.print("1-10000 arası mükemmel sayılar:");
        for (int i = 1; i <= 10000; i++) {
            if (mukemmelMi(i)) {
                System.out.print(" " + i);
            }
        }
        System.out.println();
    }
}
```

Çıktı

```
1-10000 arası mükemmel sayılar: 6 28 496 8128
```

10

`tersCevir`, Bölüm 2'deki `% 10` (son basamak) ve `/ 10` (son basamağı at) ikilisini kullanır: Her turda `ters = ters * 10 + son basamak`. `palindromMu` ise `tersCevir`'i çağırır. `main`'deki `while (true)` döngüsü 0 gelince `break` ile biter. Döngü koşulu `n > 0` değil `n != 0`: Java'da `%` ve `/` eksi sayıda işareti korur (`-123 % 10` = `-3`), bu yüzden `-123`'ün tersi `-321` olarak doğru bulunur.

Palindrom.java

```
import java.util.Scanner;

public class Palindrom {
    public static int tersCevir(int n) {
        int ters = 0;
        while (n != 0) {               // n > 0 olsaydı eksi sayıda hiç dönmezdi
            ters = ters * 10 + n % 10;
            n /= 10;
        }
        return ters;
    }

    public static boolean palindromMu(int n) {
        return n == tersCevir(n);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.print("Sayı (çıkış için 0): ");
            int sayi = sc.nextInt();
            if (sayi == 0) {
                break;
            }
            System.out.print(sayi + " -> tersi " + tersCevir(sayi) + ", ");
            if (palindromMu(sayi)) {
                System.out.println("palindrom");
            } else {
                System.out.println("palindrom değil");
            }
        }
    }
}
```

Çıktı

```
Sayı (çıkış için 0): 12321
12321 -> tersi 12321, palindrom
Sayı (çıkış için 0): 1230
1230 -> tersi 321, palindrom değil
Sayı (çıkış için 0): -121
-121 -> tersi -121, palindrom
Sayı (çıkış için 0): 0
```

`tersCevir(1230)` sonucu `0321` değil `321`'dir; tam sayının başındaki sıfır yazılmaz.

BÖLÜM 7

Diziler

Bu Bölümde Öğreneceklerin

- Dizi tanımlamayı, oluşturmayı ve ilk değer vermeyi; varsayılan eleman değerlerini
- İndeks mantığını (0 … `length - 1`) ve `ArrayIndexOutOfBoundsException` hatasını
- Dizileri `for` ve for-each ile dolaşmayı; toplam, ortalama, en büyük, arama, sayma algoritmalarını
- Dizilerin **referans** tipi olduğunu: `b = a`, `==` tuzağı ve kopyalama
- `Arrays` sınıfının hazır metotlarını (`toString`, `sort`, `fill`, `equals`, `copyOf`)
- Diziyi metoda göndermeyi ve metottan dizi döndürmeyi
- İki boyutlu dizileri (matris) ve düzensiz dizileri

7.1 Neden dizi?

Bir sınıftaki 40 öğrencinin notunu saklamak istediğini düşün. `not1`, `not2`, ..., `not40` diye 40 değişken tanımlamak hem zahmetli hem de işe yaramaz: Bu değişkenleri bir döngüyle dolaşamazsın, çünkü `not + i` diye bir değişken adı üretemezsin.

Dizi (array), **aynı tipteki** sabit sayıda değeri **tek bir ad** altında, yan yana tutan yapıdır. Her elemana bir **indeks** (index) numarasıyla ulaşılır ve indeksler **0'dan** başlar:

```
indeks:     0    1    2    3    4
          +----+----+----+----+----+
notlar:   | 85 | 70 | 92 | 64 | 78 |
          +----+----+----+----+----+
```

`notlar[2]` → 92. İndeks bir değişken de olabilir: `notlar[i]`. Dizileri güçlü yapan da budur: `i`'yi döngüyle değiştirerek bütün elemanlara sırayla ulaşırsın.

7.2 Dizi tanımlama ve oluşturma

```
tip[]  ad     = new tip[boyut];
int[]  notlar = new int[5];
```

- `int[]` "int dizisi" anlamına gelen bir tiptir. (`int notlar[]` yazımı da geçerlidir ama C dilinden kalmadır; Java'da `int[] notlar` tercih edilir.)
- `new int[5]` bellekte 5 `int`'lik yer ayırır. Boyut bir kere belirlenince **değişmez**. Boyut bir değişken de olabilir: `new int[n]`.
- Elemanları baştan biliyorsan süslü parantezle **ilk değer** verebilirsin: `int[] a = {85, 70, 92};` Boyutu Java sayar (burada 3).

DiziOlusturma.java

```java
public class DiziOlusturma {
    public static void main(String[] args) {
        int[] notlar = new int[5];          // 5 elemanlı, hepsi 0
        notlar[0] = 85;
        notlar[1] = 70;
        notlar[4] = notlar[0] + 5;
        System.out.println(notlar[0] + " " + notlar[1] + " " + notlar[2]);
        System.out.println("Son eleman: " + notlar[4]);

        String[] gunler = {"Pzt", "Sal", "Çar", "Per", "Cum"};
        double[] fiyatlar = {12.5, 8.0, 99.9};
        System.out.println(gunler[2] + " " + fiyatlar[1]);
        System.out.print(notlar.length + " " + gunler.length);
        System.out.println(" " + fiyatlar.length);
    }
}
```

Çıktı

```
85 70 0
Son eleman: 90
Çar 8.0
5 5 3
```

`{...}` ile ilk değer verme **sadece tanımlama satırında** yapılabilir. Önceden tanımlanmış bir diziye `a = {1, 2, 3};` yazamazsın (derlenmez); bunun yerine `a = new int[] {1, 2, 3};` yazılır.

Varsayılan değerler

`new` ile oluşturulan bir dizinin elemanları, sen değer vermesen bile otomatik olarak **varsayılan değer** (default value) alır. Bu, Bölüm 2'deki yerel değişkenlerden farklıdır: Yerel değişkeni değer vermeden okuyamazsın, ama dizi elemanını okuyabilirsin.

Eleman tipi	Varsayılan değer
`int`, `long`, `short`, `byte`	`0`
`double`, `float`	`0.0`
`boolean`	`false`
`char`	`'\u0000'` (kodu 0 olan boş karakter)
`String` ve diğer bütün nesneler	`null` (hiçbir nesneyi göstermiyor)

Varsayilan.java

```java
public class Varsayilan {
    public static void main(String[] args) {
        int[] a = new int[3];
        double[] b = new double[3];
        boolean[] c = new boolean[3];
        char[] d = new char[3];
        String[] e = new String[3];
        System.out.println(a[0] + " " + b[1] + " " + c[2] + " " + e[0]);
        System.out.println((int) d[0]);
    }
}
```

```
Çıktı
0 0.0 false null
0
```

`null` bir String değil, "burada henüz nesne yok" demektir. `e[0].length()` gibi bir `null` üzerinden metot çağırmak `NullPointerException` ile çöker (Bölüm 9 ve 12).

7.3 length ve indeksler

Her dizinin eleman sayısını tutan bir **`length`** alanı (field) vardır. Geçerli indeksler **0'dan `length - 1`'e** kadardır; son elemana `a[a.length - 1]` ile ulaşılır.

● Sınavda Çıkar: `length` mi, `length()` mi?

Dizide `length` bir **alandır**, parantez almaz: `a.length`. String'de ise `length()` bir **metottur**, parantez ister: `s.length()`. İkisini karıştırmak derleme hatasıdır ve sınavda en sık sorulan ayrıntılardan biridir:

```java
public class LengthHata {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        String s = "Java";
        System.out.println(a.length());
        System.out.println(s.length);
    }
}
```

```
Derleyici çıktısı – program derlenmez
LengthHata.java:5: error: cannot find symbol
        System.out.println(a.length());
                           ^
  symbol:   method length()
  location: variable a of type int[]
LengthHata.java:6: error: cannot find symbol
        System.out.println(s.length);
                           ^
  symbol:   variable length
  location: variable s of type String
2 errors
```

ArrayIndexOutOfBoundsException

Geçersiz bir indeks kullanırsan (negatif ya da `length`'e eşit veya büyük) program **derlenir**, ama çalışırken `ArrayIndexOutOfBoundsException` ile çöker. Derleyici indeksin değerini bilemez; kontrol çalışma zamanında yapılır:

SinirDisi.java

```java
public class SinirDisi {
    public static void main(String[] args) {
        int[] a = {10, 20, 30};
        System.out.println(a[0] + " " + a[a.length - 1]);
        for (int i = 0; i <= a.length; i++) {      // <= : HATA!
            System.out.println(i + ": " + a[i]);
        }
    }
}
```

Çıktı – program hata ile durur

```
10 30
0: 10
1: 20
2: 30
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for
length 3
        at SinirDisi.main(SinirDisi.java:6)
```

Sık Yapılan Hata: `<=` ile bir fazla dönmek

3 elemanlı dizinin indeksleri 0, 1, 2'dir; `a[3]` yoktur. Döngü koşulu **`i < a.length`** olmalı, `i <= a.length` değil. Bu "bir fazla/bir eksik" hatasına *off-by-one* denir. Mesajdaki *Index 3 out of bounds for length 3* ifadesi neyin yanlış gittiğini tam olarak söyler.

7.4 Diziyi dolaşmak

Diziyi dolaşmanın iki yolu vardır: indeksli klasik `for` ve **for-each** (gelişmiş for, *enhanced for*). For-each döngüsü `for (tip eleman : dizi)` biçiminde yazılır ve "dizideki her eleman için" diye okunur:

Dolasma.java

```java
public class Dolasma {
    public static void main(String[] args) {
        int[] notlar = {85, 70, 92, 64, 78};
        for (int i = 0; i < notlar.length; i++) {
            System.out.print("[" + i + "]=" + notlar[i] + " ");
        }
        System.out.println();
        for (int i = notlar.length - 1; i >= 0; i--) {     // tersten
            System.out.print(notlar[i] + " ");
        }
        System.out.println();
        for (int not : notlar) {                           // for-each
            System.out.print(not + " ");
        }
        System.out.println();
    }
}
```

Çıktı

```
[0]=85 [1]=70 [2]=92 [3]=64 [4]=78
78 64 92 70 85
85 70 92 64 78
```

Sık Yapılan Hata: Diziyi `println` ile yazdırmak

`System.out.println(notlar)` yazarsan elemanları değil `[I@4617c264` gibi anlamsız bir metin görürsün: dizinin tipi (`[I` = int dizisi) ve bellekteki kimliği. Bu sayı her çalıştırmada değişebilir. Elemanları görmek için ya döngü kur ya da `java.util.Arrays` sınıfının **`Arrays.toString(dizi)`** metodunu kullan; `[85, 70, 92, 64, 78]` yazar. Bundan sonra bu metodu sık kullanacağız.

for-each ile dizi değiştirilemez

For-each döngüsündeki değişken, elemanın **kopyasıdır** (Bölüm 6'daki değer ile geçirmenin aynısı). Ona değer atamak dizinin kendisini değiştirmez:

ForEachTuzak.java

```java
import java.util.Arrays;

public class ForEachTuzak {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        for (int x : a) {
            x = x * 10;                // sadece kopyayı değiştirir
        }
        System.out.println(Arrays.toString(a));

        for (int i = 0; i < a.length; i++) {
            a[i] = a[i] * 10;          // dizinin kendisini değiştirir
        }
        System.out.println(Arrays.toString(a));
    }
}
```

Çıktı

```
[1, 2, 3]
[10, 20, 30]
```

	Klasik `for`	for-each
İndekse erişim	Var (`i`)	Yok
Elemanı değiştirme	Olur: `a[i] = ...`	**Olmaz**
Tersten / atlayarak dolaşma	Olur	Olmaz, hep baştan sona
Ne zaman?	İndeks gerekiyorsa veya dizi değişecekse	Sadece okumak yetiyorsa (toplam, arama)

7.5 Temel dizi algoritmaları

Toplam, ortalama, en büyük, en küçük

Istatistik.java

```java
public class Istatistik {
    public static void main(String[] args) {
        int[] d = {12, -5, 47, 8, 47, -20, 3};
        int toplam = 0;
        for (int x : d) {
            toplam += x;
        }
        double ortalama = (double) toplam / d.length;

        int enBuyuk = d[0], enKucuk = d[0], enBuyukIndeks = 0;
        for (int i = 1; i < d.length; i++) {
            if (d[i] > enBuyuk) {
                enBuyuk = d[i];
                enBuyukIndeks = i;
            }
            if (d[i] < enKucuk) {
                enKucuk = d[i];
            }
        }
        System.out.println("Toplam: " + toplam);
        System.out.printf("Ortalama: %.2f%n", ortalama);
        System.out.print("En büyük: " + enBuyuk);
```

```
        System.out.println(" (indeks " + enBuyukIndeks + ")");
        System.out.println("En küçük: " + enKucuk);
    }
}
```

Çıktı

```
Toplam: 92
Ortalama: 13.14
En büyük: 47 (indeks 2)
En küçük: -20
```

- Ortalamada `(double)` cast'i şart; yoksa `92 / 7` tam sayı bölmesiyle `13` olurdu.
- 47 dizide iki kere var (indeks 2 ve 4). `>` kullandığımız için **ilk** bulunanın indeksi kaldı. `>=` yazsaydık sonuncusu (4) kalırdı. Sınavda bu fark sorulur.

> **Sık Yapılan Hata: En büyüğe 0 ile başlamak**
>
> `int enBuyuk = 0;` ile başlarsan, elemanların hepsi negatif olan bir dizide (ör. `{-7, -3, -9}`) sonuç `-3` yerine **0** çıkar; 0 dizide yok bile! Aynı şekilde `enKucuk = 0` pozitif dizilerde yanlış sonuç verir. Başlangıç değeri her zaman dizinin **kendi elemanı** olmalı: `d[0]`.

Arama ve sayma

Bir değeri dizide baştan sona tek tek bakarak aramaya **doğrusal arama** (linear search) denir. Bulunamazsa geçersiz bir indeks olan **-1** döndürmek gelenektir. Daha hızlı olan ikili aramayı (binary search) Bölüm 16'da göreceksin.

AramaSayma.java

```
public class AramaSayma {
    public static void main(String[] args) {
        int[] d = {4, 9, 2, 9, 7, 9, 1};
        int aranan = 9;

        int konum = -1;                    // -1: bulunamadı
        for (int i = 0; i < d.length; i++) {
            if (d[i] == aranan) {
                konum = i;
                break;                     // ilkini bulunca dur
            }
        }
        System.out.println(aranan + " ilk olarak " + konum + ". indekste");

        int adet = 0;
        for (int x : d) {
            if (x == aranan) {
                adet++;
            }
        }
        System.out.println(aranan + " dizide " + adet + " kez geçiyor");
    }
}
```

Çıktı

```
9 ilk olarak 1. indekste
9 dizide 3 kez geçiyor
```

7.6 Scanner ile dizi doldurma

Dizinin boyutunu ve elemanlarını kullanıcıdan alabilirsin. Aşağıdaki program, dizisiz yazılamayacak bir iş yapıyor: Ortalamanın üstündeki notları saymak için notları **iki kez** dolaşmak gerekir; ilk turda ortalama henüz bilinmez.

NotGirisi.java

```
import java.util.Scanner;

public class NotGirisi {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Kaç öğrenci? ");
        int n = sc.nextInt();
        int[] notlar = new int[n];          // boyut çalışırken belli oldu

        for (int i = 0; i < n; i++) {
            System.out.print((i + 1) + ". öğrencinin notu: ");
            notlar[i] = sc.nextInt();
        }

        int toplam = 0;
        for (int not : notlar) {
            toplam += not;
        }
        double ort = (double) toplam / n;

        int ustunde = 0;
        for (int not : notlar) {
            if (not > ort) {
                ustunde++;
            }
        }
        System.out.printf("Ortalama: %.1f%n", ort);
        System.out.println("Ortalamanın üstünde: " + ustunde + " öğrenci");
    }
}
```

Çıktı

```
Kaç öğrenci? 5
1. öğrencinin notu: 85
2. öğrencinin notu: 70
3. öğrencinin notu: 92
4. öğrencinin notu: 64
5. öğrencinin notu: 78
Ortalama: 77.8
Ortalamanın üstünde: 3 öğrenci
```

Kullanıcıya gösterilen sıra numarası `i + 1`, dizideki indeks ise `i`. İnsanlar saymaya 1'den, diziler 0'dan başlar.

7.7 Diziler referans tipidir

Bölüm 2'deki ilkel tipler (`int`, `double`…) değişkenin içinde **değerin kendisini** tutar. Dizi değişkeni ise dizinin kendisini değil, bellekteki dizinin **adresini** tutar; buna **referans** (reference) denir. Dizinin kendisi belleğin *heap* denen bölgesinde durur.

TakmaAd.java

```java
import java.util.Arrays;

public class TakmaAd {
    public static void main(String[] args) {
        int x = 5;
        int y = x;          // değer kopyalandı
        y = 99;
        System.out.println("x = " + x);

        int[] a = {1, 2, 3};
        int[] b = a;        // referans kopyalandı: aynı dizi!
        b[0] = 99;
        System.out.println("a = " + Arrays.toString(a));

        int[] c = {99, 2, 3};
        System.out.println(a == b);
        System.out.println(a == c);
        System.out.println(Arrays.equals(a, c));
    }
}
```

Çıktı

```
x = 5
a = [99, 2, 3]
true
false
true
```

`int[] b = a;` satırı yeni bir dizi **oluşturmaz**; sadece aynı diziyi gösteren ikinci bir ad (takma ad, *alias*) oluşturur. `b` üzerinden yapılan değişiklik `a`'dan da görünür, çünkü ortada **tek bir dizi** vardır:

```
a ---+
     +--->  [ 99 | 2 | 3 ]     tek dizi, iki ad
b ---+

c ------->  [ 99 | 2 | 3 ]     ayrı bir dizi, içeriği aynı
```

> **Sınavda Çıkar: `==` içeriği değil, adresi karşılaştırır**
>
> Dizilerde `==` "bu iki değişken **aynı diziyi** mi gösteriyor?" sorusunu sorar. `a == c` içerikleri aynı olsa bile `false`'tur, çünkü ikisi ayrı dizilerdir. İçerik karşılaştırmak için **`Arrays.equals(a, c)`** kullanılır. Aynı tuzak String'lerde de var (Bölüm 8): Referans tiplerinde içerik `==` ile karşılaştırılmaz.

7.8 Diziyi kopyalama

Madem `b = a` kopyalamıyor, gerçek bir kopya (ayrı bir dizi) nasıl oluşturulur? Üç yolu var:

Kopyalama.java

```java
import java.util.Arrays;

public class Kopyalama {
    public static void main(String[] args) {
        int[] a = {5, 10, 15};

        int[] k1 = new int[a.length];            // 1) döngü ile
        for (int i = 0; i < a.length; i++) {
```

```
            k1[i] = a[i];
        }
        int[] k2 = Arrays.copyOf(a, a.length);    // 2) Arrays.copyOf
        int[] k3 = a.clone();                     // 3) clone()

        k1[0] = -1;
        k2[0] = -2;
        k3[0] = -3;
        System.out.println(Arrays.toString(a));   // a etkilenmedi
        System.out.println(k1[0] + " " + k2[0] + " " + k3[0]);

        int[] buyuk = Arrays.copyOf(a, 5);        // fazlası 0 ile dolar
        int[] kucuk = Arrays.copyOf(a, 2);        // fazlası kesilir
        System.out.println(Arrays.toString(buyuk) + " " + Arrays.toString(kucuk));
    }
}
```

Çıktı

```
[5, 10, 15]
-1 -2 -3
[5, 10, 15, 0, 0] [5, 10]
```

`a.clone()`, dizinin aynı uzunlukta yeni bir kopyasını döndürür. `clone` her nesnede bulunan bir metottur; nereden geldiğini Bölüm 10'da (`Object` sınıfı) göreceksin. Şimdilik dizi kopyalamanın kısa yolu olarak bilmen yeterli.

Dizinin boyutu değişmediği için bir diziyi "büyütmenin" tek yolu, daha büyük yeni bir dizi oluşturup elemanları ona kopyalamaktır: `a = Arrays.copyOf(a, a.length * 2);` Bölüm 13'te göreceğin `ArrayList`, arka planda tam olarak bunu yapar.

7.9 Arrays sınıfı

`java.util.Arrays`, dizilerle ilgili hazır `static` metotlar sunar (kullanmak için `import java.util.Arrays;`):

Metot	Ne yapar?
`Arrays.toString(a)`	Elemanları `[1, 2, 3]` biçiminde String olarak döndürür
`Arrays.sort(a)`	Diziyi küçükten büyüğe **yerinde** sıralar, bir şey döndürmez (`void`)
`Arrays.fill(a, x)`	Bütün elemanlara `x` değerini atar
`Arrays.equals(a, b)`	İki dizinin **içeriği** aynı mı? (`boolean`)
`Arrays.copyOf(a, n)`	İlk `n` elemanın kopyası olan **yeni** dizi döndürür
`Arrays.copyOfRange(a, bas, son)`	`bas` (dahil) ile `son` (hariç) arasının kopyası
`Arrays.deepToString(m)`	İki boyutlu diziyi yazdırmak için (`toString` yetmez)

ArraysOrnek.java

```java
import java.util.Arrays;

public class ArraysOrnek {
    public static void main(String[] args) {
        int[] a = {42, 7, 19, 3, 25};
        Arrays.sort(a);
        System.out.println(Arrays.toString(a));

        String[] isimler = {"Zeynep", "ali", "Can", "Ayşe"};
        Arrays.sort(isimler);
        System.out.println(Arrays.toString(isimler));

        int[] b = new int[4];
        Arrays.fill(b, 7);
        int[] c = {7, 7, 7, 7};
        System.out.println(Arrays.toString(b));
        System.out.println(Arrays.equals(b, c) + " " + (b == c));
        System.out.println(Arrays.toString(Arrays.copyOfRange(a, 1, 3)));
    }
}
```

Çıktı

```
[3, 7, 19, 25, 42]
[Ayşe, Can, Zeynep, ali]
[7, 7, 7, 7]
true false
[7, 19]
```

String'ler **karakter kodlarına** göre sıralanır ve büyük harflerin kodları küçüklerden önce gelir (`'A'` = 65, `'a'` = 97). Bu yüzden `"ali"`, `"Zeynep"`'ten sonraya düştü.

> **İpucu**
>
> `Arrays.sort(a)` diziyi **yerinde** değiştirir ve `void`'dir; `int[] s = Arrays.sort(a);` derlenmez. Orijinal sırayı korumak istiyorsan önce kopyala, sonra kopyayı sırala: `int[] s = a.clone(); Arrays.sort(s);`

7.10 Diziler ve metotlar

Diziyi metoda geçirmek

Bölüm 6'da Java'nın her zaman **değer ile geçirdiğini** ve `swap(int a, int b)` metodunun çalışmadığını gördün. Diziyi metoda gönderdiğinde de kural aynıdır; ama kopyalanan değer bu sefer bir **referanstır**. Metot ile çağıran, **aynı diziyi** gösteren iki ayrı değişkene sahip olur:

DiziMetot.java

```java
import java.util.Arrays;

public class DiziMetot {
    public static void ikiyleCarp(int[] d) {
        for (int i = 0; i < d.length; i++) {
            d[i] *= 2;
        }
    }

    public static void yeniDiziAta(int[] d) {
        d = new int[] {0, 0, 0};       // d artık başka bir diziyi gösteriyor
```

```
        d[0] = 100;
    }

    public static void swap(int[] d, int i, int j) {
        int gecici = d[i];
        d[i] = d[j];
        d[j] = gecici;
    }

    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        ikiyleCarp(a);
        System.out.println(Arrays.toString(a));
        yeniDiziAta(a);
        System.out.println(Arrays.toString(a));
        swap(a, 0, 2);
        System.out.println(Arrays.toString(a));
    }
}
```

Çıktı

```
[2, 4, 6]
[2, 4, 6]
[6, 4, 2]
```

- `ikiyleCarp`: `d` ile `a` aynı diziyi gösterir; `d[i]` üzerinden yapılan değişiklik `main`'den görünür.
- `yeniDiziAta`: `d = new int[]...` satırı sadece metodun **kendi kopyası** olan `d`'yi yeni bir diziye yönlendirir; `main`'deki `a` hâlâ eski diziyi gösterir. Yeni dizi metot bitince kaybolur.
- `swap`: Bölüm 6'da `int`'lerle yazılamayan yer değiştirme, dizinin elemanlarıyla yazılabilir.

```
ikiyleCarp(a) çalışırken:            yeniDiziAta içinde d = new ... sonrası:

main:       a ---+                   main:        a ---->  [ 2 | 4 | 6 ]
                 +-->  [ 1 | 2 | 3 ]
ikiyleCarp: d ---+                   yeniDiziAta: d ---->  [ 100 | 0 | 0 ]
```

Sınavda Çıkar: Dizi parametresi: ne dışarıdan görünür?

Metodun içinde yapılan	Çağıranda görünür mü?
Elemanı değiştirmek: `d[0] = 50;`	**Evet**: aynı dizi
Parametreye yeni dizi atamak: `d = new int[3];`	**Hayır**: sadece yerel kopya değişti
`int` parametreyi değiştirmek: `n = 50;` (`f(a[0])` ile çağrılsa bile)	**Hayır**: `a[0]`'ın değeri kopyalandı

"Java'da diziler referans ile geçirilir" demek **yanlıştır**. Doğrusu: Java her şeyi değer ile geçirir; dizi değişkeninin değeri bir referans olduğu için metot aynı diziye erişebilir.

Metottan dizi döndürmek

Dönüş tipi `int[]` olan bir metot, içinde oluşturduğu diziyi `return` ile döndürebilir. Metodun yerel değişkeni (`sonuc`) metot bitince yok olur, ama dizinin kendisi yaşamaya devam eder; çünkü artık çağıran taraftaki değişken onu göstermektedir.

DiziDondurme.java

```java
import java.util.Arrays;

public class DiziDondurme {
    public static int[] kareler(int n) {
        int[] sonuc = new int[n];
        for (int i = 0; i < n; i++) {
            sonuc[i] = (i + 1) * (i + 1);
        }
        return sonuc;
    }

    public static int[] tersKopya(int[] d) {
        int[] t = new int[d.length];
        for (int i = 0; i < d.length; i++) {
            t[i] = d[d.length - 1 - i];
        }
        return t;
    }

    public static void main(String[] args) {
        int[] k = kareler(5);
        int[] t = tersKopya(k);
        System.out.println(Arrays.toString(k) + " " + Arrays.toString(t));
    }
}
```

Çıktı

```
[1, 4, 9, 16, 25] [25, 16, 9, 4, 1]
```

`tersKopya` orijinal diziye dokunmadan **yeni** bir dizi döndürür; `k` olduğu gibi kaldı.

7.11 Ters çevirme ve kaydırma

Diziyi yeni bir dizi oluşturmadan **yerinde** (in-place) ters çevirmek için baştaki ve sondaki elemanlar yer değiştirilerek ortaya doğru ilerlenir. **Kaydırma** (rotation) ise elemanları bir sola ya da sağa taşır; dışarı taşan eleman öbür uca konur:

TersKaydir.java

```java
import java.util.Arrays;

public class TersKaydir {
    public static void tersCevir(int[] d) {
        for (int i = 0; i < d.length / 2; i++) {
            int gecici = d[i];
            d[i] = d[d.length - 1 - i];
            d[d.length - 1 - i] = gecici;
        }
    }

    public static void solaKaydir(int[] d) {
        int ilk = d[0];
        for (int i = 0; i < d.length - 1; i++) {
            d[i] = d[i + 1];
        }
        d[d.length - 1] = ilk;
    }

    public static void sagaKaydir(int[] d) {
```

```
        int son = d[d.length - 1];
        for (int i = d.length - 1; i > 0; i--) {   // sondan başa!
            d[i] = d[i - 1];
        }
        d[0] = son;
    }

    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4, 5};
        int[] b = {1, 2, 3, 4, 5};
        int[] c = {1, 2, 3, 4, 5};
        tersCevir(a);
        solaKaydir(b);
        sagaKaydir(c);
        System.out.println("Ters: " + Arrays.toString(a));
        System.out.println("Sola: " + Arrays.toString(b));
        System.out.println("Sağa: " + Arrays.toString(c));
    }
}
```

Çıktı

```
Ters: [5, 4, 3, 2, 1]
Sola: [2, 3, 4, 5, 1]
Sağa: [5, 1, 2, 3, 4]
```

- **Sınavda Çıkar: İki klasik döngü hatası**
  - **Ters çevirirken** döngü `d.length / 2`'ye kadar gitmeli. Sonuna kadar giderse her çift **iki kez** yer değiştirir ve dizi eski haline döner; "ters çevirme çalışmıyor" sorusunun cevabı budur.
  - **Sağa kaydırırken** döngü **sondan başa** gitmeli. Baştan sona giderse `d[1] = d[0]`, `d[2] = d[1]` ... olur ve ilk eleman bütün diziye kopyalanır: `{1, 2, 3, 4, 5}` → `[5, 1, 1, 1, 1]`.

7.12 İki boyutlu diziler

Tablo ya da **matris** biçimindeki veriler (satır × sütun) için **iki boyutlu dizi** kullanılır. Java'da iki boyutlu dizi aslında **dizilerin dizisidir**: `m[i]`, i. satırı tutan bir `int[]` dizisidir.

```
int[][] m = new int[3][4];      // 3 satır, 4 sütun, hepsi 0

            sütun 0  sütun 1  sütun 2  sütun 3
m[0] ->  [     3        8        1        6    ]
m[1] ->  [     4        0        7        2    ]     m[1][2] = 7
m[2] ->  [     9        5        2        4    ]
```

- `m[i][j]`: i. satırın j. sütunundaki eleman.
- `m.length`: satır sayısı. `m[i].length`: i. satırın sütun sayısı.
- İç içe döngüyle dolaşılır: dıştaki döngü satırları, içteki sütunları gezer.

Matris.java

```
public class Matris {
    public static void main(String[] args) {
        int[][] m = {
            {3, 8, 1, 6},
            {4, 0, 7, 2},
            {9, 5, 2, 4}
        };
        System.out.println("Satır: " + m.length + ", sütun: " + m[0].length);
```

```
        for (int i = 0; i < m.length; i++) {
            int satirToplami = 0;
            for (int j = 0; j < m[i].length; j++) {
                System.out.printf("%4d", m[i][j]);
                satirToplami += m[i][j];
            }
            System.out.println("  | " + satirToplami);
        }
        System.out.println("----------------");
        for (int j = 0; j < m[0].length; j++) {        // sütun toplamları
            int sutunToplami = 0;
            for (int i = 0; i < m.length; i++) {
                sutunToplami += m[i][j];
            }
            System.out.printf("%4d", sutunToplami);
        }
        System.out.println();
    }
}
```

Çıktı

```
Satır: 3, sütun: 4
   3   8   1   6  | 18
   4   0   7   2  | 13
   9   5   2   4  | 20
----------------
  16  13  10  12
```

Satır toplamında **dış** döngü satırı sabit tutar, iç döngü sütunları gezer. Sütun toplamında ise tersi: Dış döngü sütunu (`j`) sabit tutar, iç döngü satırları (`i`) gezer.

> **Sık Yapılan Hata: Satır ve sütun sayısını karıştırmak**
>
> Kare olmayan bir matriste iç döngüyü `j < m.length` diye yazarsan, yukarıdaki 3×4 matriste son sütunu **atlarsın**; 4×3 bir matriste ise `ArrayIndexOutOfBoundsException` alırsın. Kural: Dış döngü **`m.length`**, iç döngü **`m[i].length`**.

Matris toplama ve transpoz

Aynı boyuttaki iki matris eleman eleman toplanır. Bir matrisin **transpozu** (devriği) satırlarla sütunların yer değiştirmesiyle elde edilir: `t[j][i] = a[i][j]`; 2×3 bir matrisin transpozu 3×2'dir.

MatrisIslemleri.java

```
import java.util.Arrays;

public class MatrisIslemleri {
    public static int[][] topla(int[][] a, int[][] b) {
        int[][] c = new int[a.length][a[0].length];
        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[0].length; j++) {
                c[i][j] = a[i][j] + b[i][j];
            }
        }
        return c;
    }

    public static int[][] transpoz(int[][] a) {
        int[][] t = new int[a[0].length][a.length];   // boyutlar yer değiştirir
```

```
        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[0].length; j++) {
                t[j][i] = a[i][j];
            }
        }
        return t;
    }

    public static void main(String[] args) {
        int[][] a = {{1, 2, 3}, {4, 5, 6}};
        int[][] b = {{10, 20, 30}, {40, 50, 60}};
        System.out.println(Arrays.deepToString(topla(a, b)));
        System.out.println(Arrays.deepToString(transpoz(a)));
    }
}
```

Çıktı

```
[[11, 22, 33], [44, 55, 66]]
[[1, 4], [2, 5], [3, 6]]
```

Düzensiz (jagged) diziler

İki boyutlu dizi "dizilerin dizisi" olduğu için satırların **aynı uzunlukta olması gerekmez**. Önce sadece satır sayısı verilir (`new int[5][]`), her satır sonra ayrı ayrı oluşturulur. Pascal üçgeni bunun güzel bir örneği:

Duzensiz.java

```
public class Duzensiz {
    public static void main(String[] args) {
        int[][] pascal = new int[5][];            // sadece satır sayısı belli
        for (int i = 0; i < pascal.length; i++) {
            pascal[i] = new int[i + 1];           // her satır farklı uzunlukta
            pascal[i][0] = 1;
            pascal[i][i] = 1;
            for (int j = 1; j < i; j++) {
                pascal[i][j] = pascal[i - 1][j - 1] + pascal[i - 1][j];
            }
        }
        for (int[] satir : pascal) {
            for (int x : satir) {
                System.out.print(x + " ");
            }
            System.out.println("(" + satir.length + " eleman)");
        }
    }
}
```

Çıktı

```
1 (1 eleman)
1 1 (2 eleman)
1 2 1 (3 eleman)
1 3 3 1 (4 eleman)
1 4 6 4 1 (5 eleman)
```

Dıştaki for-each'in değişkeni `satir` bir `int[]`'dir; içteki for-each onun elemanlarını gezer. Düzensiz dizilerde `m[i].length` kullanmak **zorunludur**, çünkü her satırın uzunluğu farklıdır.

7.13 Komut satırı argümanları: String[] args

`main` metodunun parametresi `String[] args` artık tanıdık: Program **komut satırından** çalıştırılırken sınıf adından sonra yazılan kelimeler bu diziye konur. Hiçbir şey yazılmazsa dizi boştur (`length` = 0):

Argumanlar.java

```
public class Argumanlar {
    public static void main(String[] args) {
        System.out.println("Argüman sayısı: " + args.length);
        for (int i = 0; i < args.length; i++) {
            System.out.println(i + ": " + args[i]);
        }
    }
}
```

Çıktı

```
Argüman sayısı: 0
```

Aynı programı terminalden argümanlarla çalıştırırsan:

```
java Argumanlar Ali 42 "Java Dersi"
Argüman sayısı: 3
0: Ali
1: 42
2: Java Dersi
```

Tırnak içindeki kelimeler tek argüman sayılır. Argümanların hepsi **String**'dir; `42`'yi sayı olarak kullanmak için `Integer.parseInt(args[1])` gerekir (Bölüm 3).

Bölüm Özeti – Bilmen Gerekenler

- Dizi, **aynı tipte** sabit sayıda elemanı tek ad altında tutar: `int[] a = new int[5];` veya `int[] a = {1, 2, 3};`. Boyut sonradan değişmez.
- `new` ile oluşan dizide varsayılan değerler: sayılar `0` / `0.0`, `boolean` `false`, `char` `'\u0000'`, nesneler `null`.
- İndeksler `0` … `a.length - 1`. Dizide **`length`** alan (parantezsiz), String'de **`length()`** metot.
- Geçersiz indeks derlenir ama çalışırken **`ArrayIndexOutOfBoundsException`** verir. Döngü koşulu `i < a.length`.
- for-each sadece okumak içindir; döngü değişkeni elemanın kopyasıdır, diziyi değiştirmez.
- En büyük/en küçük aramaya `d[0]` ile başla, 0 ile değil. Doğrusal arama bulamazsa `-1` döndürür.
- Dizi değişkeni bir **referans** tutar: `b = a` kopya değil takma addır. `==` adres, `Arrays.equals` içerik karşılaştırır.
- Gerçek kopya: döngü, `Arrays.copyOf`, `clone()`. `Arrays.toString`, `sort` (yerinde, `void`), `fill`, `equals`.
- Diziyi metoda göndermek referansın kopyasını gönderir: Elemanlardaki değişiklik **dışarıdan görünür**, parametreye yeni dizi atamak görünmez.
- 2B dizi = dizilerin dizisi. `m.length` satır, `m[i].length` sütun sayısı. Satırlar farklı uzunlukta olabilir (düzensiz dizi).

Sorular

A) Çoktan Seçmeli

1 `double[] d = new double[4];` satırından sonra `d[2]` ve `d.length` değerleri nedir?

(A) `0.0` ve `4`
(B) `null` ve `4`
(C) `0` ve `3`
(D) Değer verilmediği için derleme hatası
(E) `0.0` ve `3`

2 `int[] a` bir dizi, `String s` bir String ise eleman sayısını / uzunluğu veren **doğru** ifade çifti hangisidir?

(A) `a.length()` ve `s.length()`
(B) `a.length` ve `s.length`
(C) `a.size()` ve `s.length()`
(D) `a.length()` ve `s.length`
(E) `a.length` ve `s.length()`

3 Aşağıdaki kod parçası ekrana ne yazdırır?

```
int[] a = {1, 2, 3};
int[] b = a;
b[1] = 20;
a = new int[] {7, 8, 9};
System.out.println(b[1] + a[1]);
```

(A) `10` (B) `28` (C) `16` (D) `22` (E) `40`

4 Aşağıdaki metot ve çağrı için ekrana ne yazılır?

```
public static void f(int[] d, int n) {
    d[0] = 50;
    n = 50;
}
// main içinde:
int[] a = {1, 2};
int x = 1;
f(a, x);
System.out.println(a[0] + " " + x);
```

(A) `1 1` (B) `50 50` (C) `1 50` (D) `50 1` (E) Derleme hatası

5 Aşağıdaki tanıma göre `m.length + m[1].length + m[2][0]` ifadesinin değeri nedir?

```
int[][] m = {{1, 2}, {3, 4, 5}, {6}};
```

A) `9`
B) `11`
C) `12`
D) `6`
E) `ArrayIndexOutOfBoundsException`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
import java.util.Arrays;

public class Soru6 {
    public static void main(String[] args) {
        int[] a = {3, 8, 1, 6, 4};
        int t = 0;
        for (int i = 1; i < a.length; i += 2) {
            t += a[i];
        }
        System.out.println(t);
        for (int i = a.length - 1; i > 0; i--) {
            a[i] = a[i - 1];
        }
        System.out.println(Arrays.toString(a));
        int k = 0;
        for (int x : a) {
            if (x > a[1]) {
                k++;
            }
        }
        System.out.println(k);
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
import java.util.Arrays;

public class Soru7 {
    public static void degistir(int[] d) {
        d[0] = d[d.length - 1];
        d = new int[] {9, 9, 9};
        d[1] = 0;
    }

    public static int[] ikiKat(int[] d) {
        int[] s = d.clone();
        for (int i = 0; i < s.length; i++) {
            s[i] *= 2;
        }
        return s;
    }

    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        degistir(a);
        int[] b = ikiKat(a);
        int[] c = a;
        c[1] = 7;
        System.out.println(Arrays.toString(a));
        System.out.println(Arrays.toString(b));
        System.out.println((a == c) + " " + Arrays.equals(a, b));
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru8 {
    public static void main(String[] args) {
        int[][] m = {{2, 0, 1}, {4, 5, 3}, {1, 7, 6}};
        int kosegen = 0, ters = 0;
        for (int i = 0; i < m.length; i++) {
            kosegen += m[i][i];
            ters += m[i][m.length - 1 - i];
        }
        System.out.println(kosegen + " " + ters);
        for (int[] satir : m) {
            System.out.print(satir[satir.length - 1] + " ");
        }
        System.out.println();
        int[] ikinci = m[1];
        ikinci[0] = 0;
        System.out.println(m[1][0] + m.length * m[2][0]);
    }
}
```

C) Kod Yazma

9 Bir `int` dizisindeki **en büyük ikinci farklı** değeri döndüren `ikinciEnBuyuk(int[] d)` metodunu yaz. Diziyi sıralamadan, **tek bir döngüyle** çöz. Dizide en az iki farklı değer olduğunu varsayabilirsin. `IkinciEnBuyuk` programı metodu iki diziyle denesin:

```
[12, 45, 7, 45, 30] -> 30
[-3, -8, -1] -> -3
```

10 Kullanıcıdan önce matrisin boyutu `n`'yi, sonra `n × n` bir matrisin elemanlarını okuyan `Simetrik` programını yaz. Matrisin **simetrik** olup olmadığını (her `i`, `j` için `m[i][j] == m[j][i]`) `boolean simetrikMi(int[][] m)` metoduyla kontrol etsin:

```
Matris boyutu: 3
Elemanları satır satır gir:
1 2 3
2 5 4
3 4 9
Matris simetrik.
```

Çözümler

Cevap anahtarı

1	2	3	4	5
A	E	B	D	C

1 Cevap: A

`new` ile oluşturulan `double` dizisinin her elemanı `0.0` olur. `length` eleman sayısıdır (4), son indeks değil (3). `null` sadece nesne dizilerinin varsayılanıdır; dizi elemanları yerel değişken gibi "değersiz" kalmadığı için derleme hatası da yoktur.

2 Cevap: E

Dizide `length` bir **alandır** (parantezsiz), String'de `length()` bir **metottur** (parantezli). Dizilerde `size()` diye bir şey yoktur; o, Bölüm 13'teki koleksiyonlara aittir.

3 Cevap: B

`b = a` ile iki değişken **aynı diziyi** gösterir; `b[1] = 20` bu diziyi `{1, 20, 3}` yapar. Sonra `a` yeni bir diziye yönlendirilir, ama `b` hâlâ eski diziyi gösterir. `b[1] + a[1] = 20 + 8 = 28`.

Çıktı
```
28
```

4 Cevap: D

Dizi parametresi `d`, `a` ile aynı diziyi gösterir; `d[0] = 50` dışarıdan görünür. `n` ise `x`'in değerinin kopyasıdır; onu değiştirmek `x`'i etkilemez.

Çıktı
```
50 1
```

5 Cevap: C

`m.length` = 3 (satır sayısı), `m[1].length` = 3 (`{3, 4, 5}`), `m[2][0]` = 6. Toplam `3 + 3 + 6 = 12`. Düzensiz dizide satır uzunlukları farklıdır, ama kullanılan bütün indeksler geçerli olduğu için exception oluşmaz.

Çıktı
```
12
```

6

- İlk döngü `i = 1, 3` için çalışır: `t = a[1] + a[3] = 8 + 6 = 14`.
- İkinci döngü sondan başa her elemanı bir sağa taşır: `a[4] = 6`, `a[3] = 1`, `a[2] = 8`, `a[1] = 3`. `a[0]`'a dokunulmadığı için 3 olarak kalır ve iki kez görünür. Bu bir kaydırma değil, son elemanın kaybolduğu bir **öteleme**dir.
- `a[1]` artık 3: 3'ten büyük elemanlar 8 ve 6 → `k = 2`.

Çıktı
```
14
[3, 3, 8, 1, 6]
2
```

7

- `degistir(a)`: `d[0] = d[2]` → `a` = `{3, 2, 3}`. Sonraki `d = new int[]...` sadece yerel `d`'yi değiştirir; `d[1] = 0` yeni diziye yazılır, `a` etkilenmez.
- `ikiKat(a)`: `clone` ile ayrı bir kopya alınıp ikiyle çarpılır → `b` = `{6, 4, 6}`; `a` değişmez.
- `c = a` takma addır: `c[1] = 7` → `a` = `{3, 7, 3}`.
- `a == c` aynı dizi olduğu için `true`; `a` ile `b`'nin içerikleri farklı, `false`.

Çıktı
```
[3, 7, 3]
[6, 4, 6]
true false
```

8

- Ana köşegen: `m[0][0] + m[1][1] + m[2][2] = 2 + 5 + 6 = 13`. Ters köşegen: `m[0][2] + m[1][1] + m[2][0] = 1 + 5 + 1 = 7`.
- Her satırın son elemanı: `1 3 6` (sonda boşluk var).
- `ikinci = m[1]` yeni bir dizi değil, ikinci **satırın kendisini** gösteren bir referanstır. `ikinci[0] = 0` aslında `m[1][0] = 0` demektir. Sonuç: `0 + 3 * 1 = 3`.

Çıktı
```
13 7
1 3 6
3
```

9

İki değişken tut: `birinci` (en büyük) ve `ikinci`. Yeni eleman `birinci`'den büyükse eski `birinci` ikinciliğe düşer. Değilse ve `ikinci`'den büyükse (ve `birinci`'ye eşit değilse) yeni `ikinci` olur. Eşitlik kontrolü, 45 gibi tekrar eden en büyük değerin ikinci sayılmasını engeller. Başlangıçta `Integer.MIN_VALUE` kullanmak, negatif dizilerde de doğru çalışmayı sağlar.

IkinciEnBuyuk.java
```java
import java.util.Arrays;

public class IkinciEnBuyuk {
    public static int ikinciEnBuyuk(int[] d) {
        int birinci = Integer.MIN_VALUE;
        int ikinci = Integer.MIN_VALUE;
        for (int x : d) {
            if (x > birinci) {
                ikinci = birinci;
                birinci = x;
            } else if (x > ikinci && x != birinci) {
                ikinci = x;
            }
        }
        return ikinci;
```

```
    }

    public static void main(String[] args) {
        int[] a = {12, 45, 7, 45, 30};
        int[] b = {-3, -8, -1};
        System.out.println(Arrays.toString(a) + " -> " + ikinciEnBuyuk(a));
        System.out.println(Arrays.toString(b) + " -> " + ikinciEnBuyuk(b));
    }
}
```

Çıktı

```
[12, 45, 7, 45, 30] -> 30
[-3, -8, -1] -> -3
```

10

Elemanlar iç içe iki döngüyle okunur. `simetrikMi` her `m[i][j]`'yi `m[j][i]` ile karşılaştırır; ilk uyumsuzlukta `false` döndürüp çıkar. İç döngünün `j = i + 1`'den başlaması, köşegeni ve her çifti iki kez kontrol etmeyi önler.

Simetrik.java

```java
import java.util.Scanner;

public class Simetrik {
    public static boolean simetrikMi(int[][] m) {
        for (int i = 0; i < m.length; i++) {
            for (int j = i + 1; j < m.length; j++) {
                if (m[i][j] != m[j][i]) {
                    return false;
                }
            }
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Matris boyutu: ");
        int n = sc.nextInt();
        int[][] m = new int[n][n];
        System.out.println("Elemanları satır satır gir:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                m[i][j] = sc.nextInt();
            }
        }
        if (simetrikMi(m)) {
            System.out.println("Matris simetrik.");
        } else {
            System.out.println("Matris simetrik değil.");
        }
    }
}
```

Çıktı

```
Matris boyutu: 3
Elemanları satır satır gir:
1 2 3
2 5 4
3 4 9
Matris simetrik.
```

BÖLÜM 8

String ve Karakter İşlemleri

Bu Bölümde Öğreneceklerin

- String oluşturmayı ve String'in **değiştirilemez** (immutable) olmasının ne demek olduğunu
- `length`, `charAt`, `indexOf`, `substring`, `split`, `replace` gibi temel metotları
- `substring`'in **"başlangıç dahil, bitiş hariç"** kuralını
- **`==` ile `equals` farkını** (String havuzu) ve `compareTo` ile sözlük sırasını
- Türkçe `I/İ` harf sorununu ve `Locale` ile çözümünü
- `Character` sınıfının metotlarını ve bir String'i karakter karakter dolaşmayı
- `StringBuilder` ile verimli metin oluşturmayı
- String ↔ sayı dönüşümlerini ve `NumberFormatException`'ı
- Palindrom, sesli harf sayma, Sezar şifreleme, harf frekansı gibi klasik algoritmaları

8.1 String nedir?

String, karakterlerden oluşan bir dizidir: bir isim, bir cümle, bir e-posta adresi... Bölüm 2'de gördüğün gibi `String` ilkel bir tip değil, `java.lang` paketindeki bir **sınıftır** (bu yüzden `import` gerekmez). Bir String'in metotlarını, değişken adından sonra nokta koyarak çağırırsın: `ad.length()`.

String oluşturmanın iki yolu vardır:

- **Literal ile:** `String ad = "Java";` → Neredeyse her zaman bunu kullanırsın.
- **`new` ile:** `String ad = new String("Java");` → Gereksizdir; ama sınavda `==` sorularında karşına çıkar (birazdan göreceğiz).

Bölüm 7'deki dizilerde olduğu gibi, String'in karakterleri de **0'dan başlayarak** numaralanır. Bu numaraya **indeks** (index) denir. `"Java 25"` için:

Karakter	J	a	v	a	␣	2	5
İndeks	0	1	2	3	4	5	6

Uzunluk 7'dir, ama son indeks **6**'dır. Kural: **son indeks = `length() - 1`**.

StringGiris.java

```
public class StringGiris {
    public static void main(String[] args) {
        String ad = "Java 25";
        System.out.println(ad.length());
        System.out.println("".length());
        System.out.println(ad.charAt(0) + "|" + ad.charAt(4) + "|" + ad.charAt(6));
        System.out.println(ad.charAt(ad.length() - 1));
    }
}
```

```
Çıktı
7
0
J| |5
5
```

`charAt(i)` metodu `i`. indeksteki karakteri bir `char` olarak döndürür. Boşluk da bir karakterdir; `charAt(4)` boşluğu verdiği için iki `|` arasında boşluk görünüyor. `""` ise **boş String**'dir: hiç karakteri yoktur, uzunluğu 0'dır.

Sık Yapılan Hata: `length` mi, `length()` mi?

Dizilerde uzunluk bir **alandır**, parantez yoktur: `dizi.length`. String'de ise bir **metottur**, parantez şarttır: `s.length()`. Karıştırırsan program derlenmez:

```
public class LengthHata {
    public static void main(String[] args) {
        String s = "Java";
        System.out.println(s.length);
    }
}
```

Derleyici çıktısı – program derlenmez

```
LengthHata.java:4: error: cannot find symbol
        System.out.println(s.length);
                           ^
  symbol:   variable length
  location: variable s of type String
1 error
```

Metin blokları: çok satırlı String (Java 15+)

Birkaç satırlık metni `\n` ve `+` ile yazmak hem okunmaz hem de tırnakları `\"` ile kaçırmayı gerektirir. Java 15'ten beri üç çift tırnakla yazılan **metin bloğu** (text block) bunu çözer. Kurallar kısadır:

- Açılış `"""`'ten sonra **hemen satır sonu** gelmelidir; metin bir alt satırdan başlar.
- Satır sonları kendiliğinden `\n` olur, yazmana gerek yoktur. İçindeki `"` için kaçış da gerekmez.
- **Girinti kuralı:** Derleyici, metin satırlarının ve kapanış `"""`'nin başındaki boşluklardan **en azını** bulur ve bu kadar boşluğu her satırın başından siler. Kodu içeride yazman metni sağa kaydırmaz; kapanış `"""`'yi sola çekersen aradaki fark metinde girinti olarak kalır.
- Kapanış `"""` ayrı bir satırdaysa metin `\n` ile biter; son metin satırının sonuna yazarsan `\n` olmaz.

MetinBlogu.java

```
public class MetinBlogu {
    public static void main(String[] args) {
        String eski = "Ad: Ece\nNot: \"AA\"\nDurum: Geçti";
        String yeni = """
            Ad: Ece
            Not: "AA"
            Durum: Geçti""";
        System.out.println(yeni);
        System.out.println(eski.equals(yeni));

        String a = """
            Merhaba
              Dünya
            """;
```

```
        String b = """
            Merhaba
              Dünya
          """;
        System.out.print(a);
        System.out.println("---");
        System.out.print(b);
        System.out.println("---");
        System.out.println(a.endsWith("\n") + " " + yeni.endsWith("\n"));

        String mektup = """
            Sayın %s,
            Notunuz: %d
            """.formatted("Ece", 85);
        System.out.print(mektup);
    }
}
```

Çıktı

```
Ad: Ece
Not: "AA"
Durum: Geçti
true
Merhaba
  Dünya
---
  Merhaba
    Dünya
---
true false
Sayın Ece,
Notunuz: 85
```

`eski` ile `yeni` aynı içeriğe sahiptir (`equals` → `true`). `a`'nın kapanış `"""`'si metinle aynı hizada olduğu için girintinin tamamı silinir; `b`'de kapanış iki sütun solda olduğundan her satırda **2 boşluk** kalır (ikinci satırda 4). Metin bloğu sıradan bir `String`'dir: `.formatted(...)` (`String.format`'ın metot hali) gibi bütün String metotlarını çağırabilirsin.

8.2 String değiştirilemez (immutable)

String'in en önemli özelliği şudur: Bir String nesnesi bir kere oluşturulduktan sonra **içeriği asla değişmez**. Buna **değiştirilemezlik** (immutability) denir. `toUpperCase()`, `replace()`, `trim()` gibi metotlar mevcut String'i değiştirmez; **yeni bir String üretip döndürür**. Sonucu bir değişkene atamazsan, üretilen yeni String kaybolur:

Degismez.java

```java
public class Degismez {
    public static void main(String[] args) {
        String s = "merhaba";
        s.toUpperCase();                // yeni String üretildi ama atılmadı!
        System.out.println(s);

        String buyuk = s.toUpperCase();  // yeni String'i sakla
        System.out.println(buyuk + " / " + s);

        s = s.replace('a', 'e');         // s artık YENİ nesneyi gösteriyor
        System.out.println(s);

        String t = s;
        s = s + "!";
        System.out.println(s + " / " + t);
    }
}
```

Çıktı

```
merhaba
MERHABA / merhaba
merhebe
merhebe! / merhebe
```

Peki `s = s.replace(...)` satırında `s` değişmedi mi? Değişen **String nesnesi değil, `s` değişkeninin gösterdiği yerdir**. String değişkeni, metnin kendisini değil, bellekteki String nesnesinin **adresini** (referansını, *reference*) tutar:

```
String s = "abc";     s ──► "abc"
String t = s;         t ──► "abc"   (aynı nesne)
s = s + "d";          s ──► "abcd"  (yeni nesne)
                      t ──► "abc"   (eski nesne hiç değişmedi)
```

Son satırda `t`'nin hâlâ `merhebe` yazmasının nedeni budur: `s + "!"` yeni bir nesne oluşturdu, `t` ise eski nesneyi göstermeye devam etti.

> **● Sık Yapılan Hata: Sonucu atanmayan String metodu**
>
> `s.toUpperCase();`, `s.trim();`, `s.replace("a", "b");` satırları tek başına **hiçbir şey yapmaz**. Derleyici seni uyarmaz, program çalışır, ama `s` aynı kalır. Doğrusu: `s = s.toUpperCase();`. Çıktı sorularında bu tuzak çok sık kullanılır.

8.3 Temel String metotları

Aşağıdaki tablo bu bölümün "kopya kâğıdı"dır. Örneklerde `s = "Merhaba"` kabul edilmiştir (M0 e1 r2 h3 a4 b5 a6):

Metot	Ne yapar?	Örnek	Sonuç
`length()`	Karakter sayısı	`s.length()`	`7`
`charAt(i)`	`i`. karakter (`char`)	`s.charAt(1)`	`'e'`
`indexOf(x)`	`x`'in ilk konumu, yoksa `-1`	`s.indexOf('a')`	`4`
`lastIndexOf(x)`	`x`'in son konumu	`s.lastIndexOf('a')`	`6`
`substring(b)`	`b`'den sona kadar	`s.substring(3)`	`"haba"`

Metot	Ne yapar?	Örnek	Sonuç
`substring(b, e)`	`b` dahil, `e` **hariç**	`s.substring(1, 4)`	`"erh"`
`toUpperCase()`	Büyük harfe çevir	`s.toUpperCase()`	`"MERHABA"`
`toLowerCase()`	Küçük harfe çevir	`s.toLowerCase()`	`"merhaba"`
`trim()` / `strip()`	Baştaki ve sondaki boşlukları sil	`" a b ".trim()`	`"a b"`
`replace(a, b)`	Bütün `a`'ları `b` yap	`s.replace('a', 'e')`	`"Merhebe"`
`contains(x)`	`x` içinde geçiyor mu?	`s.contains("hab")`	`true`
`startsWith(x)`	`x` ile başlıyor mu?	`s.startsWith("Me")`	`true`
`endsWith(x)`	`x` ile bitiyor mu?	`s.endsWith("ba")`	`true`
`split(ayrac)`	Parçalara böl → `String[]`	`"a,b,c".split(",")`	`{"a", "b", "c"}`
`String.join(ayrac, ...)`	Parçaları birleştir	`String.join("-", "a", "b")`	`"a-b"`
`repeat(n)`	`n` kez tekrarla	`"ab".repeat(3)`	`"ababab"`
`isEmpty()`	Uzunluk 0 mı?	`"".isEmpty()`	`true`
`isBlank()`	Boş ya da sadece boşluk mu?	`" ".isBlank()`	`true`
`equals(x)`	İçerik aynı mı?	`s.equals("merhaba")`	`false`
`equalsIgnoreCase(x)`	Büyük/küçük harfi önemsemeden	`s.equalsIgnoreCase("MERHABA")`	`true`
`compareTo(x)`	Sözlük sırası farkı (`int`)	`"a".compareTo("c")`	`-2`

Arama: indexOf, lastIndexOf, contains

`indexOf` aranan karakterin ya da metnin **ilk** geçtiği indeksi döndürür; bulamazsa `-1` döndürür. İkinci parametre verirsen aramaya o indeksten başlar. Bu sayede bir kelimenin kaç kez geçtiğini bir döngüyle sayabilirsin:

Arama.java

```
public class Arama {
    public static void main(String[] args) {
        String cumle = "Java zor değil, Java eğlenceli";
        System.out.println(cumle.indexOf("Java"));
        System.out.println(cumle.indexOf("Java", 1));
        System.out.println(cumle.lastIndexOf("Java"));
        System.out.println(cumle.indexOf("Python"));
        System.out.println(cumle.contains("değil"));
        System.out.println(cumle.startsWith("Java") + " " + cumle.endsWith("."));

        // "Java" kaç kez geçiyor?
        int sayac = 0;
        int konum = cumle.indexOf("Java");
        while (konum != -1) {
            sayac++;
            konum = cumle.indexOf("Java", konum + 1);
        }
        System.out.println("Java " + sayac + " kez geçiyor.");
    }
}
```

```
Çıktı
0
16
16
-1
true
true false
Java 2 kez geçiyor.
```

substring: başlangıç dahil, bitiş hariç

`substring(b, e)` metni **`b` indeksinden başlayıp `e` indeksinden hemen önce** biter. `e` indeksindeki karakter **alınmaz**. `substring(b)` ise `b`'den sonuna kadar her şeyi alır. `"Bilgisayar"` üzerinde deneyelim:

Karakter	B	i	l	g	i	s	a	y	a	r
İndeks	0	1	2	3	4	5	6	7	8	9

Substring.java

```java
public class Substring {
    public static void main(String[] args) {
        String s = "Bilgisayar";
        System.out.println(s.substring(3));
        System.out.println(s.substring(0, 3));
        System.out.println(s.substring(3, 7));
        System.out.println(s.substring(3, 7).length());
        System.out.println("[" + s.substring(10) + "]");

        String eposta = "elif.demir@ogr.edu.tr";
        int at = eposta.indexOf('@');
        String kullanici = eposta.substring(0, at);
        String alan = eposta.substring(at + 1);
        System.out.println(kullanici + " | " + alan);
    }
}
```

```
Çıktı
gisayar
Bil
gisa
4
[]
elif.demir | ogr.edu.tr
```

`substring(10)` hata vermedi, boş String döndü: Başlangıç indeksi `length()`'e **eşit** olabilir, ama ondan büyük olamaz. E-posta örneği gerçek hayatta çok kullanılan bir kalıptır: önce `indexOf` ile ayırıcıyı bul, sonra `substring` ile parçala.

- **Sınavda Çıkar: substring kuralları**
  - `s.substring(b, e)` → `b` **dahil**, `e` **hariç**. Sonucun uzunluğu her zaman **`e - b`**'dir: `substring(3, 7)` → 4 karakter.
  - `s.substring(0, 3)` → ilk 3 karakter. `s.substring(s.length() - 3)` → son 3 karakter.
  - `s.substring(i, i + 1)` → `i`. karakter, ama `char` değil **String** olarak.
  - `s.substring(s.length())` → `""` (hata değil). `b > e` veya `e > length()` → hata.

StringIndexOutOfBoundsException

`charAt` ya da `substring` ile String'in dışına çıkmaya çalışırsan program `StringIndexOutOfBoundsException` ile çöker. Bu, Bölüm 7'de dizilerde gördüğün `ArrayIndexOutOfBoundsException`'ın String'deki karşılığıdır:

IndeksHatasi.java

```
public class IndeksHatasi {
    public static void main(String[] args) {
        String s = "Java";
        System.out.println(s.charAt(3));
        System.out.println("[" + s.substring(4) + "]");
        System.out.println(s.charAt(4));
        System.out.println("Bu satıra hiç gelinmez.");
    }
}
```

Çıktı – program hata ile durur

```
a
[]
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: Index 4 out of bounds for
length 4
        at java.base/jdk.internal.util.Preconditions$1.apply(Preconditions.java:55)
        at java.base/jdk.internal.util.Preconditions$1.apply(Preconditions.java:52)
        at java.base/jdk.internal.util.Preconditions$4.apply(Preconditions.java:213)
        at java.base/jdk.internal.util.Preconditions$4.apply(Preconditions.java:210)
        at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java:98)
        at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Preconditions.java:106)
        at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java:302)
        at java.base/java.lang.String.checkIndex(String.java:4904)
        at java.base/java.lang.StringLatin1.charAt(StringLatin1.java:45)
        at java.base/java.lang.String.charAt(String.java:1624)
        at IndeksHatasi.main(IndeksHatasi.java:6)
```

Mesaj her şeyi söylüyor: Uzunluğu 4 olan bir String'de 4. indeks yok (geçerli indeksler 0–3). Bu hatanın en sık sebebi, döngü koşulunda `i < s.length()` yerine **`i <= s.length()`** yazmaktır.

Temizleme ve değiştirme: strip, replace, isBlank

Kullanıcıdan gelen metinlerin başında ve sonunda çoğu zaman istenmeyen boşluklar olur. `trim()` bunları siler; Java 11 ile gelen `strip()` aynı işi yapar ama Unicode boşluk karakterlerini de tanır (yeni kodda `strip()` tercih edilir). İkisi de **aradaki** boşluklara dokunmaz: `" Ali Veli ".strip()` → `"Ali Veli"`. `replace` ise hem `char` hem String ile çalışır ve geçtiği **her** yeri değiştirir; bütün boşlukları silmek için `s.replace(" ", "")` yazılır.

Bir girdinin boş olup olmadığını kontrol ederken dikkat: `isEmpty()` sadece uzunluğa bakar ve `" "` için `false` döner. `isBlank()` ise sadece boşluktan oluşan metni de boş sayar. Kullanıcının gerçekten bir şey yazıp yazmadığını anlamak için `isBlank()` daha güvenlidir.

Bölme ve birleştirme: split, String.join, repeat

`split` bir metni verilen ayırıcıdan böler ve parçaları bir **`String` dizisi** olarak döndürür. Virgülle ya da noktalı virgülle ayrılmış verileri okumanın standart yoludur:

Bolme.java

```java
public class Bolme {
    public static void main(String[] args) {
        String satir = "Ayşe;Yılmaz;70;85";
        String[] p = satir.split(";");
        System.out.println(p.length + " parça: " + p[0] + " " + p[1]);
        int vize = Integer.parseInt(p[2]);
        int fin = Integer.parseInt(p[3]);
        System.out.println("Ortalama: " + (vize + fin) / 2.0);

        String cumle = "  Java   ile    kod  ";
        String[] kelimeler = cumle.strip().split("\\s+");
        System.out.println(kelimeler.length + " kelime");
        System.out.println(String.join("-", kelimeler));
        System.out.println(cumle.split(" ").length + " parça (yanlış yöntem)");
        System.out.println("=".repeat(12));
    }
}
```

Çıktı

```
4 parça: Ayşe Yılmaz
Ortalama: 77.5
3 kelime
Java-ile-kod
10 parça (yanlış yöntem)
============
```

`split`'in parametresi aslında bir **düzenli ifadedir** (regular expression, *regex*): metin desenlerini tarif eden küçük bir dil. Senin bilmen gereken tek desen `"\\s+"`: "bir veya daha fazla boşluk karakteri". Kelimeler arasında birden fazla boşluk varsa `split(" ")` araya **boş String'ler** koyar (yukarıda 10 parça çıktı); `strip().split("\\s+")` ise sadece kelimeleri verir. `String.join` bunun tersini yapar: parçaları aralarına ayırıcı koyarak birleştirir.

Sınavda Çıkar: split ve replaceAll düzenli ifade alır

`replace` metni olduğu gibi arar; ama `replaceAll` ve `split` ilk parametreyi **düzenli ifade** olarak yorumlar. Düzenli ifadede `.` "herhangi bir karakter" demektir:

```java
public class RegexTuzak {
    public static void main(String[] args) {
        String tarih = "07.10.2026";
        System.out.println(tarih.replace(".", "/"));
        System.out.println(tarih.replaceAll(".", "/"));
        System.out.println(tarih.split(".").length);
        System.out.println(tarih.split("\\.").length);
    }
}
```

Çıktı

```
07/10/2026
//////////
0
3
```

`replaceAll(".", "/")` **bütün** karakterleri `/` yaptı; `split(".")` ise her karakteri ayırıcı saydığı için hiç parça bırakmadı. Gerçek noktadan bölmek için `"\\."` yazılır.

8.4 Büyük/küçük harf ve Türkçe I sorunu

`toUpperCase()` ve `toLowerCase()` parametresiz çağrıldığında, harf dönüşümünü **bilgisayarın bölge ayarına** (Bölüm 3'te gördüğün *locale*) göre yapar. Çoğu dilde bu fark etmez; ama Türkçede `i` harfinin **dört** biçimi vardır (`i`, `ı`, `İ`, `I`) ve kurallar İngilizceden farklıdır:

Harf	İngilizce kural	Türkçe kural
`i` büyüyünce	`I`	`İ`
`I` küçülünce	`i`	`ı`

Bu yüzden aynı program Türkçe bir bilgisayarda farklı davranabilir. Çözüm, dönüşümde hangi kuralların kullanılacağını **açıkça** belirtmektir:

Turkcel.java

```
import java.util.Locale;

public class TurkceI {
    public static void main(String[] args) {
        Locale tr = Locale.forLanguageTag("tr-TR");
        String sehir = "izmir";
        System.out.println(sehir.toUpperCase(Locale.ROOT));
        System.out.println(sehir.toUpperCase(tr));

        String komut = "EXIT";
        String kucuk = komut.toLowerCase(tr);
        System.out.println(kucuk + " " + kucuk.equals("exit"));
        System.out.println(komut.toLowerCase(Locale.ROOT).equals("exit"));
    }
}
```

Çıktı

```
IZMIR
İZMİR
exıt false
true
```

`Locale.ROOT` "dilden bağımsız" kurallar demektir (İngilizce ile aynı sonucu verir). Türkçe kurallarla `"EXIT"` küçültülünce `"exıt"` olur ve `"exit"` ile eşleşmez. Türkçe Windows'ta çalışan pek çok programın "anlamsız" hatalarının sebebi budur.

> **İpucu: Hangi Locale'i kullanmalıyım?**
>
> - Kullanıcıya **gösterilecek Türkçe metin** (isim, şehir) → `toUpperCase(tr)` / `toLowerCase(tr)`.
> - **Program içi karşılaştırma** (komut, dosya uzantısı, anahtar kelime) → `Locale.ROOT` ya da hiç çevirmeden `equalsIgnoreCase`.
>
> Bu kitabın çıktıları İngilizce bölge ayarıyla üretildiği için, parametresiz `toUpperCase()` burada `Locale.ROOT` gibi davranır.

8.5 String karşılaştırma: == ve equals

Bu bölümün, belki de bütün dönemin **en önemli sınav konusu**: İki String'i `==` ile karşılaştırmak, içeriklerini değil **adreslerini** karşılaştırır. "İki değişken **aynı nesneyi** mi gösteriyor?" sorusunu sorar. İçeriğin aynı olup olmadığını öğrenmek için **`equals`** kullanılır.

Durumu karıştıran şey **String havuzudur** (string pool). Java, koddaki literal String'leri (`"java"` gibi) bellekte özel bir alanda **tek kopya** olarak tutar. Aynı literal'i iki kez yazarsan iki değişken aynı nesneyi gösterir. `new String(...)` ise havuza bakmadan **her zaman yeni** bir nesne oluşturur:

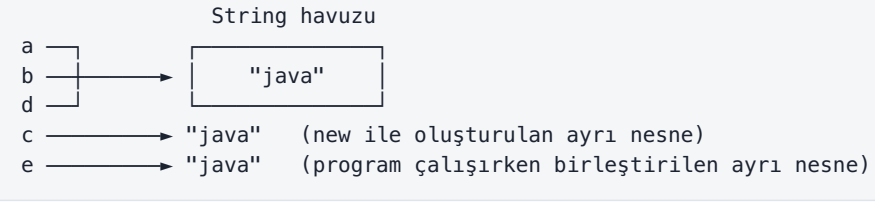
Karsilastirma.java

```
public class Karsilastirma {
    public static void main(String[] args) {
        String a = "java";
        String b = "java";
        String c = new String("java");
        String d = "ja" + "va";
        String parca = "ja";
        String e = parca + "va";

        System.out.println(a == b);
        System.out.println(a == c);
        System.out.println(a.equals(c));
        System.out.println(a == d);
        System.out.println(a == e);
        System.out.println(a.equals(e));
    }
}
```

Çıktı

```
true
false
true
true
false
true
```



- `a == b` → `true`: İkisi de havuzdaki aynı nesneyi gösteriyor.
- `a == c` → `false`: `c` ayrı bir nesne. İçerik aynı olsa da adres farklı.
- `a == d` → `true`: `"ja" + "va"` iki **sabit** literal'in birleşimidir; derleyici bunu derleme sırasında `"java"` yapar, sonuç havuza gider.
- `a == e` → `false`: `parca` bir değişken olduğu için birleştirme **program çalışırken** yapılır ve yeni bir nesne oluşur.
- `equals` ise her durumda içeriğe bakar ve `true` döndürür.

> **Sınavda Çıkar: == mi, equals mi?**
>
> **`==` referansı (adresi), `equals` içeriği karşılaştırır.** Sınavda `==` ile karşılaştırılan String'lerin hangisinin havuzdan geldiğini sorarlar: literal ve sabit birleşimi (`"ja" + "va"`) → havuz; `new String(...)`, değişkenle birleştirme, `Scanner`'dan okunan metin → yeni nesne. Gerçek kodda kural basit: **String'leri her zaman `equals` ile karşılaştır.**

En tehlikeli durum, Bölüm 4'te kısaca gördüğün gibi, kullanıcıdan okunan metni `==` ile karşılaştırmaktır. `Scanner`'ın döndürdüğü String her zaman **yeni bir nesnedir**; havuzdaki literal ile asla `==` olmaz:

EvetHayir.java

```java
import java.util.Scanner;

public class EvetHayir {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Devam edilsin mi? ");
        String cevap = sc.nextLine();
        System.out.println("== ile    : " + (cevap == "evet"));
        System.out.println("equals ile: " + cevap.equals("evet"));
    }
}
```

Çıktı

```
Devam edilsin mi? evet
== ile    : false
equals ile: true
```

Sık Yapılan Hata: Kullanıcı girdisini == ile karşılaştırmak

`if (cevap == "evet")` derlenir ve çalışır, ama kullanıcı `evet` yazsa bile koşul `false` olur ve program **yanlış** dala girer. Kodun doğru göründüğü için bu hatayı bulmak saatler alabilir. Her zaman `cevap.equals("evet")` yaz. Büyük/küçük harf önemsizse `cevap.equalsIgnoreCase("evet")` kullan. Bonus: `"evet".equals(cevap)` şeklinde yazarsan, `cevap` boş bir referans (`null`, Bölüm 9) olsa bile program çökmez.

8.6 Sıralama karşılaştırması: compareTo

`equals` sadece "aynı mı?" sorusunu cevaplar. "Sözlükte hangisi önce gelir?" sorusu için `compareTo` kullanılır. `a.compareTo(b)` bir `int` döndürür:

- **Negatif** → `a`, `b`'den **önce** gelir.
- **0** → ikisi eşit.
- **Pozitif** → `a`, `b`'den **sonra** gelir.

Dönen sayı rastgele değildir. `compareTo` iki String'i baştan itibaren karakter karakter karşılaştırır:

1. İlk **farklı** karakteri bulursa, bu iki karakterin **kod farkını** döndürür: `a.charAt(k) - b.charAt(k)`.
2. Fark bulamadan biri biterse (biri diğerinin başıysa), **uzunluk farkını** döndürür: `a.length() - b.length()`.

CompareTo.java

```java
public class CompareTo {
    public static void main(String[] args) {
        System.out.println("elma".compareTo("armut"));
        System.out.println("armut".compareTo("elma"));
        System.out.println("kale".compareTo("kalem"));
        System.out.println("Java".compareTo("java"));
        System.out.println("Java".compareToIgnoreCase("java"));
        System.out.println("Zeynep".compareTo("ali"));
    }
}
```

```
Çıktı
4
-4
-1
-32
0
-7
```

- `'e' - 'a'` = 101 − 97 = **4**. Ters çevirince **−4**.
- `"kale"` ile `"kalem"`: İlk 4 karakter aynı, `"kale"` bitti → 4 − 5 = **−1**.
- `'J' - 'j'` = 74 − 106 = **−32**. Büyük harflerin kodu küçüklerden küçük olduğu için **büyük harfle başlayan kelimeler önce gelir**: `"Zeynep"`, `"ali"`'den önce sıralanır (90 − 97 = −7).

Bir dizideki alfabetik olarak ilk ismi bulmak için Bölüm 7'deki "en küçüğü bul" algoritmasında `dizi[i] < enKucuk` yerine `dizi[i].compareTo(enKucuk) < 0` yazman yeterlidir. String'lerde `<` ve `>` operatörleri **kullanılamaz**.

> **Sınavda Çıkar: compareTo'nun döndürdüğü değer**
>
> `compareTo` sadece −1, 0, 1 döndürmez; **karakter kodu farkını** veya **uzunluk farkını** döndürür. Sınavda "`"Ankara".compareTo("Antalya")` kaçtır?" diye sorulursa: İlk fark 2. indekste, `'k' - 't'` = 107 − 116 = **−9**. Koşullarda ise sadece işarete bak: `a.compareTo(b) < 0` → "a önce gelir".

> **Not: Türkçe harflerle sıralama**
>
> `compareTo` karakter kodlarına baktığı için Türkçe harfler alfabenin sonuna düşer: `'ç'` (231), `'z'`'den (122) büyüktür ve `"çilek"`, `"zeytin"`'den sonra gelir. Türkçe alfabe sırasına göre sıralama için `java.text.Collator` sınıfı kullanılır; bu ders kapsamında genellikle beklenmez.

8.7 Karakter karakter dolaşma ve Character sınıfı

Pek çok String sorusu aynı kalıpla çözülür: String'i bir `for` döngüsüyle **karakter karakter** dolaş, her karakteri `charAt(i)` ile al ve kontrol et. Karakterleri sınıflandırmak için `Character` sınıfının hazır metotları vardır:

Metot	Ne yapar?	Örnek → Sonuç
`Character.isDigit(c)`	Rakam mı?	`isDigit('7')` → `true`
`Character.isLetter(c)`	Harf mi? (Türkçe harfler dahil)	`isLetter('ş')` → `true`
`Character.isLetterOrDigit(c)`	Harf ya da rakam mı?	`isLetterOrDigit('_')` → `false`
`Character.isUpperCase(c)`	Büyük harf mi?	`isUpperCase('Ç')` → `true`
`Character.isLowerCase(c)`	Küçük harf mi?	`isLowerCase('a')` → `true`
`Character.isWhitespace(c)`	Boşluk, tab, satır sonu mu?	`isWhitespace('\t')` → `true`
`Character.toUpperCase(c)`	Büyük harfe çevir (`char` döner)	`toUpperCase('a')` → `'A'`
`Character.toLowerCase(c)`	Küçük harfe çevir	`toLowerCase('Q')` → `'q'`
`Character.getNumericValue(c)`	Rakamın sayısal değeri	`getNumericValue('7')` → `7`

Örnek: Güçlü bir şifrede büyük harf, küçük harf, rakam ve özel karakter bulunmalıdır. Bir şifredeki her türden karakteri sayalım:

SifreKontrol.java

```java
public class SifreKontrol {
    public static void main(String[] args) {
        String sifre = "Java25!Güçlü";
        int buyuk = 0, kucuk = 0, rakam = 0, diger = 0;

        for (int i = 0; i < sifre.length(); i++) {
            char c = sifre.charAt(i);
            if (Character.isUpperCase(c)) {
                buyuk++;
            } else if (Character.isLowerCase(c)) {
                kucuk++;
            } else if (Character.isDigit(c)) {
                rakam++;
            } else {
                diger++;
            }
        }
        System.out.printf("Büyük: %d, küçük: %d, rakam: %d, diğer: %d%n",
                buyuk, kucuk, rakam, diger);
    }
}
```

Çıktı

```
Büyük: 2, küçük: 7, rakam: 2, diğer: 1
```

`ü` ve `ç` de küçük harf olarak sayıldı: `Character` metotları Unicode'u tanır, Türkçe harflerle sorunsuz çalışır. Sadece karakterlere bakıp indeksle işin yoksa, Bölüm 7'deki gelişmiş `for` döngüsünü `toCharArray()` ile kullanabilirsin: `for (char c : sifre.toCharArray()) { ... }`.

Sık Yapılan Hata: Rakam karakterini sayı sanmak

`'7'` karakterinin değeri 7 değil, karakter kodu olan **55**'tir (Bölüm 2). Bir String'deki rakamları toplarken `toplam += s.charAt(i);` yazarsan kodları toplarsın. Doğrusu `s.charAt(i) - '0'` ya da `Character.getNumericValue(...)`:

```java
public class RakamTuzak {
    public static void main(String[] args) {
        String no = "2026";
        int yanlis = 0, dogru = 0;
        for (int i = 0; i < no.length(); i++) {
            yanlis += no.charAt(i);          // karakter kodu eklenir!
            dogru += no.charAt(i) - '0';     // rakamın değeri eklenir
        }
        System.out.println(yanlis + " " + dogru);
        System.out.println(Character.getNumericValue('a'));
    }
}
```

Çıktı

```
202 10
10
```

Dikkat: `getNumericValue('a')` **10** döndürür (16'lık sistemdeki gibi a=10, b=11...). Karakterin gerçekten rakam olduğundan emin değilsen önce `Character.isDigit` ile kontrol et.

8.8 StringBuilder

String değiştirilemez olduğu için, bir metni parça parça oluştururken her `+` işlemi **yeni bir String** üretir. Bunun yerine **değiştirilebilir** (mutable) bir metin kutusu olan **`StringBuilder`** kullanılır. `StringBuilder`'ın metotları **nesnenin kendisini değiştirir**; sonucu atamana gerek yoktur:

Metot	Ne yapar?
`append(x)`	Sona ekler (her tip olabilir: `int`, `char`, `double`, String...)
`insert(i, x)`	`i`. konuma ekler
`reverse()`	Ters çevirir
`deleteCharAt(i)`	`i`. karakteri siler
`delete(b, e)`	`b` dahil, `e` hariç aralığı siler
`setCharAt(i, c)`	`i`. karakteri `c` yapar
`length()`, `charAt(i)`, `indexOf(x)`	String'deki gibi
`toString()`	İçeriği bir String olarak döndürür

SbOrnek.java

```java
public class SbOrnek {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java");
        sb.append(" 25");
        System.out.println(sb);
        sb.insert(0, "Merhaba ");
        System.out.println(sb);
        sb.setCharAt(0, 'm');
        sb.deleteCharAt(sb.length() - 1);
        System.out.println(sb);
        sb.reverse();
        System.out.println(sb);

        String sonuc = sb.reverse().toString();
        System.out.println(sonuc + " (" + sonuc.length() + ")");
    }
}
```

Çıktı

```
Java 25
Merhaba Java 25
merhaba Java 2
2 avaJ abahrem
merhaba Java 2 (14)
```

`append`, `insert`, `reverse` gibi metotlar değişikliği yaptıktan sonra **aynı nesneyi** döndürür. Bu yüzden `sb.reverse().toString()` ya da `sb.append("a").append(1)` gibi art arda **zincirleme** (chaining) çağrı yapılabilir.

Döngüde String birleştirmenin maliyeti

Bir döngüde `s += ...` yazdığında her adımda şunlar olur: Yeni bir String nesnesi oluşturulur, eski metnin **bütün karakterleri** oraya kopyalanır, sonra yeni parça eklenir. 1'er karakterlik `n` ekleme için toplam kopya sayısı $1 + 2 + \dots + n \approx n^2/2$'dir:

Ekleme sayısı (n)	`+=` ile kopyalanan karakter	`StringBuilder` ile
1 000	yaklaşık 500 bin	yaklaşık birkaç bin
100 000	yaklaşık **5 milyar**	yaklaşık birkaç yüz bin

`StringBuilder` ise içinde genişleyebilen bir **tampon** (buffer) tutar ve ekleri doğrudan oraya yazar; tampon dolunca kapasitesini yaklaşık iki katına çıkarır. Toplam iş `n` ile orantılı kalır. (Bu tür hesapları Bölüm 16'da Big-O ile göreceksin.) Döngüde metin biriktirirken kalıp hep aynıdır: döngüden önce `StringBuilder sb = new StringBuilder();`, döngünün içinde `sb.append(...)`, döngüden sonra `sb.toString()`. Bölümün sonundaki algoritmalarda bu kalıbı sık göreceksin.

> ● **Sınavda Çıkar: String, StringBuilder ve StringBuffer**
>
> "String ile StringBuilder arasındaki fark nedir?" klasik bir sözlü/yazılı sorudur:
>
> - **String** → değiştirilemez (immutable). Her değişiklik yeni nesne üretir.
> - **StringBuilder** → değiştirilebilir (mutable), hızlı. Döngüde metin oluştururken kullanılır.
> - **StringBuffer** → StringBuilder'ın eski, **thread-safe** (aynı anda birden çok iş parçacığından güvenle kullanılabilen) ve biraz daha yavaş hali.
>
> Tuzak: `StringBuilder` için `equals` içeriği **karşılaştırmaz**; iki farklı StringBuilder aynı metni tutsa bile `sb1.equals(sb2)` → `false`. İçerik karşılaştırmak için `sb1.toString().equals(sb2.toString())` yazılır.

8.9 Dönüşümler: String, sayı ve karakter

Kullanıcıdan gelen veriler, dosyadan okunan satırlar hep String'dir; hesap yapmak için sayıya çevirmen gerekir. Tersine, bir sayının basamaklarıyla uğraşmak için onu String'e çevirmek işini kolaylaştırır:

Dönüşüm	Yöntem	Örnek
sayı → String	`String.valueOf(x)` ya da `"" + x`	`String.valueOf(42)` → `"42"`
String → `int`	`Integer.parseInt(s)`	`Integer.parseInt("42")` → `42`
String → `double`	`Double.parseDouble(s)`	`Double.parseDouble("3.5")` → `3.5`
`char` → String	`String.valueOf(c)` ya da `"" + c`	`String.valueOf('a')` → `"a"`
String → `char`	`s.charAt(0)`	`"a".charAt(0)` → `'a'`
rakam `char` → `int`	`c - '0'`	`'7' - '0'` → `7`
String ↔ `char[]`	`s.toCharArray()` / `new String(dizi)`	

Donusumler.java

```java
public class Donusumler {
    public static void main(String[] args) {
        int yil = 2026;
        String metin = String.valueOf(yil);
        System.out.println(metin.length() + " basamaklı");
        System.out.println(metin + 1);
        System.out.println(Integer.parseInt("2026") + 1);
        System.out.println(Double.parseDouble("3.75") * 2);

        int basamakToplami = 0;
        for (char c : metin.toCharArray()) {
            basamakToplami += c - '0';
        }
        System.out.println("Basamak toplamı: " + basamakToplami);
    }
}
```

Çıktı

```
4 basamaklı
20261
2027
7.5
Basamak toplamı: 10
```

Bölüm 5'te bir sayının basamaklarını `% 10` ve `/ 10` ile ayırmıştın; String'e çevirip karakterleri dolaşmak da aynı işi yapar. `metin + 1` ise toplama değil birleştirme yaptı: `"20261"`.

NumberFormatException

`parseInt`'e sayıya çevrilemeyen bir metin verirsen program `NumberFormatException` ile çöker:

SayiHatasi.java

```java
public class SayiHatasi {
    public static void main(String[] args) {
        System.out.println(Integer.parseInt("42"));
        System.out.println(Integer.parseInt(" 42 ".strip()));
        System.out.println(Integer.parseInt("4.2"));
    }
}
```

Çıktı – program hata ile durur

```
42
42
Exception in thread "main" java.lang.NumberFormatException: For input string: "4.2"
        at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
        at java.base/java.lang.Integer.parseInt(Integer.java:565)
        at java.base/java.lang.Integer.parseInt(Integer.java:662)
        at SayiHatasi.main(SayiHatasi.java:5)
```

`"4.2"` bir tam sayı değildir. `"12a"`, `""` ve hatta başında boşluk olan `" 42"` de aynı hatayı verir; bu yüzden ikinci satırda önce `strip()` uyguladık. Bu hatayı `try-catch` ile yakalamayı Bölüm 12'de öğreneceksin; o zamana kadar, gerekirse metnin sadece rakamlardan oluştuğunu `Character.isDigit` ile kontrol edebilirsin (Proje 1'de bunu yapacağız).

8.10 Klasik String algoritmaları

Sınavların "kod yazma" kısmında String soruları çok sevilir. Aşağıdaki algoritmalar, karşına çıkacak soruların büyük kısmının iskeletidir. Hepsi aynı fikre dayanır: **karakter karakter dolaş, karar ver, sonucu biriktir.**

Ters çevirme ve palindrom

Palindrom, tersten okunuşu da aynı olan metindir: `kayak`, `ey edip adana'da pide ye`. Ters çevirmeyi döngüyle yapmak için String'i **sondan başa** dolaşırız. Palindrom kontrolünde ise önce metni temizleriz (küçük harf, sadece harf ve rakamlar), sonra **iki uçtan ortaya doğru** karşılaştırırız:

Palindrom.java

```
import java.util.Locale;

public class Palindrom {
    static String tersCevir(String s) {
        String ters = "";
        for (int i = s.length() - 1; i >= 0; i--) {
            ters += s.charAt(i);
        }
        return ters;
    }

    static boolean palindromMu(String s) {
        String kucuk = s.toLowerCase(Locale.forLanguageTag("tr-TR"));
        StringBuilder temiz = new StringBuilder();   // sadece harf ve rakamlar
        for (char c : kucuk.toCharArray()) {
            if (Character.isLetterOrDigit(c)) {
                temiz.append(c);
            }
        }
        int i = 0, j = temiz.length() - 1;           // iki uçtan ortaya
        while (i < j) {
            if (temiz.charAt(i) != temiz.charAt(j)) {
                return false;
            }
            i++;
            j--;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(tersCevir("Bilgisayar"));
        System.out.println(new StringBuilder("Bilgisayar").reverse());
        System.out.println(palindromMu("kayak") + " " + palindromMu("Java"));
        System.out.println(palindromMu("Ey Edip Adana'da pide ye"));
    }
}
```

Çıktı

```
rayasigliB
rayasigliB
true false
true
```

Sınavda "hazır metot kullanmadan" denirse döngülü `tersCevir`'i yaz; kısıtlama yoksa `StringBuilder.reverse()` tek satırda işi bitirir. İki indeksli palindrom kontrolü ilk farklı karakterde `false` döndürüp durur ve ters çevrilmiş bir kopya oluşturmaz. Kısa yol olarak, temizlenmiş metni kendi tersiyle `equals` ile karşılaştırmak da doğru sonucu verir.

Sesli harf ve kelime sayma

Bir karakterin bir kümede olup olmadığını anlamanın pratik yolu: kümedeki karakterleri bir String'e yaz ve `indexOf` ile ara; `-1` değilse kümededir. Kelime saymak için ise `strip().split("\\s+")` yeterlidir:

MetinAnaliz.java

```
import java.util.Locale;
import java.util.Scanner;

public class MetinAnaliz {
    static int kelimeSayisi(String metin) {
        if (metin.isBlank()) {
            return 0;
        }
        return metin.strip().split("\\s+").length;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String sesliler = "aeıioöuü";

        System.out.print("Bir cümle yaz: ");
        String cumle = sc.nextLine().toLowerCase(Locale.forLanguageTag("tr-TR"));
        int sesli = 0, sessiz = 0;
        for (int i = 0; i < cumle.length(); i++) {
            char c = cumle.charAt(i);
            if (sesliler.indexOf(c) != -1) {
                sesli++;
            } else if (Character.isLetter(c)) {
                sessiz++;
            }
        }
        System.out.println("Kelime: " + kelimeSayisi(cumle));
        System.out.println("Sesli: " + sesli + ", sessiz: " + sessiz);
        System.out.println("Boş metin: " + kelimeSayisi("   ") + " kelime");
    }
}
```

Çıktı

```
Bir cümle yaz:   İzmir'de   Bilgisayar Mühendisliği okuyorum
Kelime: 4
Sesli: 16, sessiz: 21
Boş metin: 0 kelime
```

- Türkçe metin olduğu için küçük harfe çevirirken `tr` kullandık; böylece `İ` → `i`, `I` → `ı` olur ve ikisi de sesli harf sayılır. Boşluk ve kesme işareti harf olmadığı için hiçbir sayaca girmez.
- `isBlank` kontrolü neden var? Çünkü boş bir metni bölünce boş değil, **1 elemanlı** bir dizi (`{""}`) çıkar ve kelime sayısı yanlışlıkla 1 olur. Sınırdaki durumları (boş girdi, tek kelime, fazla boşluk) düşünmek, tam puan ile yarım puan arasındaki farktır.

Sezar şifreleme

Sezar şifresi, her harfi alfabede sabit bir miktar (anahtar) kaydırır: anahtar 3 ise A → D, B → E, ..., X → A, Y → B, Z → C. Alfabenin sonunda başa dönmek için `%` kullanılır. Harfin alfabedeki sırası `c - 'A'` ile bulunur (A=0, B=1, ..., Z=25):

Sezar.java

```
public class Sezar {
    static String sifrele(String metin, int anahtar) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < metin.length(); i++) {
            char c = metin.charAt(i);
            if (c >= 'A' && c <= 'Z') {
                c = (char) ((c - 'A' + anahtar) % 26 + 'A');
            } else if (c >= 'a' && c <= 'z') {
                c = (char) ((c - 'a' + anahtar) % 26 + 'a');
            }
            sb.append(c);
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        String mesaj = "Vize Pazartesi, saat 10:00!";
        String gizli = sifrele(mesaj, 3);
        System.out.println(gizli);
        System.out.println(sifrele(gizli, 26 - 3));
        System.out.println(sifrele("xyz XYZ", 3));
    }
}
```

Çıktı

```
Ylch Sdcduwhvl, vddw 10:00!
Vize Pazartesi, saat 10:00!
abc ABC
```

Şifreyi çözmek için ayrı bir metoda gerek yok: 3 ileri kaydırmanın tersi, 26 − 3 = 23 ileri kaydırmaktır. `(char)` dönüşümü şart, çünkü `char + int` işleminin sonucu `int`'tir (Bölüm 2). Bu basit sürüm sadece İngilizce harfleri kaydırır; rakamlar, noktalama ve Türkçe harfler olduğu gibi kalır.

Harf frekansı: int[26]

"Bir metinde hangi harf kaç kez geçiyor?" sorusunun klasik çözümü 26 elemanlı bir sayaç dizisidir. Püf noktası, harfi doğrudan **indekse** çevirmektir: `'a' - 'a'` = 0, `'b' - 'a'` = 1, ..., `'z' - 'a'` = 25:

HarfFrekansi.java

```
import java.util.Locale;

public class HarfFrekansi {
    public static void main(String[] args) {
        String metin = "Java programlama dili";
        int[] sayac = new int[26];

        // a–z sayılıyor: Türkçe ayarda I → ı olmasın diye ROOT
        for (char c : metin.toLowerCase(Locale.ROOT).toCharArray()) {
            if (c >= 'a' && c <= 'z') {
                sayac[c - 'a']++;
            }
        }
```

```
        int enCok = 0;
        for (int i = 0; i < 26; i++) {
            if (sayac[i] > 0) {
                System.out.print((char) ('a' + i) + "=" + sayac[i] + " ");
            }
            if (sayac[i] > sayac[enCok]) {
                enCok = i;
            }
        }
        System.out.println();
        char harf = (char) ('a' + enCok);
        System.out.println("En sık: " + harf + " (" + sayac[enCok] + ")");
    }
}
```

Çıktı

```
a=5 d=1 g=1 i=2 j=1 l=2 m=2 o=1 p=1 r=2 v=1
En sık: a (5)
```

Aynı fikirle **anagram** kontrolü de yapılır: İki kelimenin sayaç dizileri birebir aynıysa, biri diğerinin harflerinin yer değiştirmesiyle elde edilmiştir (`elma` ↔ `alem`).

Bölüm Özeti – Bilmen Gerekenler

- String bir **sınıftır**; indeksler 0'dan başlar, son indeks `length() - 1`'dir. Dizide `dizi.length`, String'de `s.length()`.
- String **değiştirilemez**: `toUpperCase`, `replace`, `strip` yeni String döndürür. `s.toUpperCase();` tek başına bir şey yapmaz; `s = s.toUpperCase();` yaz.
- `indexOf` bulamazsa **−1** döndürür. `substring(b, e)` → `b` dahil, `e` hariç, uzunluk `e - b`.
- Sınır dışı `charAt`/`substring` → `StringIndexOutOfBoundsException`.
- `split` ve `replaceAll` düzenli ifade alır: boşluklardan bölmek için `strip().split("\\s+")`, noktadan bölmek için `split("\\.")`.
- Türkçe metinde `toUpperCase(tr)`, program içi karşılaştırmada `Locale.ROOT` veya `equalsIgnoreCase`.
- **`==` adres, `equals` içerik karşılaştırır.** Literal'ler ve sabit birleşimler havuzdan gelir; `new String`, değişkenle birleştirme ve `Scanner` girdisi yeni nesnedir. String'leri her zaman `equals` ile karşılaştır.
- `compareTo` → ilk farklı karakterin kod farkı ya da uzunluk farkı. Negatif: önce gelir. Büyük harfler küçüklerden önce gelir.
- `Character.isDigit/isLetter/isUpperCase/toUpperCase`; rakam karakteri → sayı: `c - '0'`.
- Döngüde metin oluştururken **`StringBuilder`** kullan (`append`, `insert`, `reverse`, `deleteCharAt`, `setCharAt`, `toString`). StringBuilder'da `equals` içerik karşılaştırmaz.
- `Integer.parseInt` / `Double.parseDouble` String'i sayıya çevirir; geçersiz metin → `NumberFormatException`.
- **Metin bloğu** (`"""`, Java 15+): çok satırlı String; açılıştan sonra satır sonu gelir, `\n` ve `\"` gerekmez, ortak girinti silinir.

Sorular

A) Çoktan Seçmeli

1 `"Programlama".substring(3, 7)` ifadesinin sonucu nedir?

(A) `"ogra"` (B) `"gra"` (C) `"graml"` (D) `"ogram"` (E) `"gram"`

2 Aşağıdaki kod parçası ne yazdırır?

```
String a = "Java";
String b = new String("Java");
String c = "Ja" + "va";
System.out.println((a == b) + " " + (a == c) + " " + a.equals(b));
```

(A) `true true true`
(B) `false false true`
(C) `false true true`
(D) `true false true`
(E) `false true false`

3 Aşağıdaki kod parçası ne yazdırır?

```
String s = "merhaba";
s.toUpperCase();
s.replace('a', 'e');
System.out.println(s + s.length());
```

(A) `merhaba7` (B) `MERHABA7` (C) `merhebe7` (D) `MERHEBE7` (E) `merhaba8`

4 `int x = "bilgi".compareTo("bilgisayar");` ve `int y = "Elma".compareTo("armut");` satırlarından sonra `x` ve `y`'nin değerleri nedir? (`'E'` = 69, `'a'` = 97)

(A) `x = -5, y = 28` (B) `x = -5, y = -28` (C) `x = -1, y = -1` (D) `x = 5, y = -28` (E) `x = -1, y = 4`

5 `String s = "Java";` tanımından sonra aşağıdaki ifadelerden hangisi `StringIndexOutOfBoundsException` hatasına yol açar?

(A) `s.charAt(3)`
(B) `s.substring(4)`
(C) `s.substring(2, 4)`
(D) `s.charAt(s.length())`
(E) `s.indexOf('z')`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru6 {
    public static void main(String[] args) {
        String s = "Java Programlama";
        System.out.println(s.indexOf('a') + " " + s.lastIndexOf('a'));
        System.out.println(s.indexOf("gram") + " " + s.indexOf('x'));
        System.out.println(s.substring(5).toUpperCase());
        System.out.println(s.charAt(s.length() - 1) + s.charAt(0));
        System.out.println("" + s.charAt(1) + s.charAt(2));
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru7 {
    public static void main(String[] args) {
        String s = "abc";
        String t = s;
        s = s + "d";
        s.toUpperCase();
        System.out.println(s + " " + t);

        StringBuilder sb = new StringBuilder("abc");
        StringBuilder sb2 = sb;
        sb.append("d").reverse();
        System.out.println(sb + " " + sb2);
        sb2.insert(0, 'x').deleteCharAt(2);
        System.out.println(sb);
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru8 {
    public static void main(String[] args) {
        String s = "a1B2c3";
        int toplam = 0;
        String harfler = "";
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (Character.isDigit(c)) {
                toplam += c;
            } else {
                harfler = Character.toUpperCase(c) + harfler;
            }
        }
        System.out.println(harfler);
        System.out.println(toplam);
        System.out.println("elma".compareTo("ekmek") + " " + "ab".compareTo("abc"));
    }
}
```

C) Kod Yazma

9 Kullanıcıdan bir kurum adı okuyan `Kisaltma` programını yaz. Program her kelimenin ilk harfini büyük, kalanını küçük yazmalı ve kelimelerin baş harflerinden bir kısaltma üretmeli. Kelimeler arasında birden fazla boşluk olabilir; Türkçe harfler doğru çevrilmeli (`i` → `İ`).

```
Kurum adı:   istanbul   TEKNİK üniversitesi
Düzgün yazım: İstanbul Teknik Üniversitesi
Kısaltma    : İTÜ
```

10 Bir kelimedeki **art arda tekrar eden** harfleri "harf + tekrar sayısı" biçiminde sıkıştıran `static String sikistir(String s)` metodunu ve bu metodu kullanıcıdan okunan bir kelimeyle deneyen `Sikistir` programını yaz. Örnekler: `aaabccdddd` → `a3b1c2d4`, `xyz` → `x1y1z1`.

Çözümler

Cevap anahtarı

1	2	3	4	5
E	C	A	B	D

1 Cevap: E

İndeksler: P0 r1 o2 g3 r4 a5 m6 l7… `substring(3, 7)` → 3, 4, 5, 6. indeksler: `"gram"`. Uzunluk `7 − 3 = 4` olmalı; B (3 karakter) ve C, D (5 karakter) bu yüzden elenir. A şıkkı 2. indeksten başlamış.

2 Cevap: C

`a` havuzdaki literal'i gösterir. `b`, `new` ile oluşturulan ayrı bir nesnedir → `a == b` **false**. `"Ja" + "va"` iki sabitin birleşimidir; derleyici bunu derleme sırasında `"Java"` literal'ine çevirir ve havuzdaki aynı nesne kullanılır → `a == c` **true**. `equals` içeriğe bakar → **true**.

Çıktı
```
false true true
```

3 Cevap: A

String değiştirilemez. `s.toUpperCase()` ve `s.replace(...)` yeni String'ler üretir ama sonuçları hiçbir değişkene atanmadığı için kaybolur; `s` hâlâ `"merhaba"`'dır. `s + s.length()` → `"merhaba" + 7` → `merhaba7`. Diğer şıklar, metotların `s`'yi değiştirdiğini varsayıyor.

Çıktı
```
merhaba7
```

4 Cevap: B

`"bilgi"`, `"bilgisayar"`'ın başıdır; hiç farklı karakter yoktur, uzunluk farkı döner: 5 − 10 = **−5**. `"Elma"` ile `"armut"`'un ilk karakterleri farklıdır: `'E' − 'a'` = 69 − 97 = **−28**. Büyük harf, küçük harften önce gelir; bu yüzden `y` negatiftir. C ve E şıkları `compareTo`'nun sadece −1/0/1 döndürdüğünü varsayıyor.

Çıktı
```
x = -5, y = -28
```

5 Cevap: D

`"Java"`'nın uzunluğu 4, geçerli indeksler 0–3'tür. `s.charAt(s.length())` → `charAt(4)` sınır dışıdır ve hata verir. A geçerlidir (`'a'`). B, başlangıç = uzunluk olduğu için hata vermez, boş String döndürür. C `"va"` döndürür (bitiş indeksi 4 hariç). E, bulamadığı için `-1` döndürür; hata vermez.

6

`s` = `"Java Programlama"`: J0 a1 v2 a3 ␣4 P5 r6 o7 g8 r9 a10 m11 l12 a13 m14 a15, uzunluk 16.

- İlk `'a'` 1. indekste, son `'a'` 15. indekste.
- `"gram"` 8. indeksten başlar; `'x'` yok → `-1`.

- `substring(5)` → `"Programlama"` → büyük harfle `PROGRAMLAMA`.
- **Tuzak:** `s.charAt(15) + s.charAt(0)` iki `char`'ın toplamıdır, String birleştirme değil! `'a'` (97) + `'J'` (74) = **171**.
- `"" + 'a' + 'v'` → başta String olduğu için birleştirme yapılır: `av`.

Çıktı
```
1 15
8 -1
PROGRAMLAMA
171
av
```

7

- `t = s` → ikisi de `"abc"`'yi gösterir. `s = s + "d"` → `s` yeni nesne `"abcd"`'yi gösterir, `t` hâlâ `"abc"`. `s.toUpperCase()` sonucu atanmadığı için etkisizdir → `abcd abc`.
- `sb2 = sb` → **aynı** StringBuilder nesnesi. `append("d")` → `"abcd"`, `reverse()` → `"dcba"`. İki değişken aynı nesneyi gösterdiği için ikisi de `dcba` yazar.
- `sb2.insert(0, 'x')` → `"xdcba"`, `deleteCharAt(2)` → 2. indeksteki `'c'` silinir → `"xdba"`. `sb` de aynı nesne olduğu için `xdba` yazılır.

Çıktı
```
abcd abc
dcba dcba
xdba
```

8

- Harfler: `Character.toUpperCase(c) + harfler` her yeni harfi **başa** ekler (`char + String` → String). `a` → `"A"`, `B` → `"BA"`, `c` → `"CBA"`.
- Rakamlar: `toplam += c` rakamın değerini değil **karakter kodunu** ekler: `'1'` + `'2'` + `'3'` = 49 + 50 + 51 = **150** (6 değil!).
- `"elma"` ile `"ekmek"`: ilk fark 1. indekste, `'l' - 'k'` = 108 − 107 = **1**. `"ab"`, `"abc"`'nin başı: 2 − 3 = **−1**.

Çıktı
```
CBA
150
1 -1
```

9

Satırı `strip().split("\\s+")` ile kelimelere ayırıyoruz; böylece fazla boşluklar sorun olmuyor. Her kelimenin ilk harfini `substring(0, 1).toUpperCase(tr)`, kalanını `substring(1).toLowerCase(tr)` ile çeviriyoruz. Türkçe kurallar kullanılmazsa `istanbul` → `Istanbul` olur, kısaltma da `ITÜ` çıkar; `TEKNİK` küçültülürken `İ` harfi de hatalı çevrilir. Sonuçları `StringBuilder` ile biriktiriyoruz.

Kisaltma.java
```java
import java.util.Locale;
import java.util.Scanner;

public class Kisaltma {
    public static void main(String[] args) {
        Locale tr = Locale.forLanguageTag("tr-TR");
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Kurum adı: ");
        String[] kelimeler = sc.nextLine().strip().split("\\s+");

        StringBuilder duzgun = new StringBuilder();
        StringBuilder kisa = new StringBuilder();
        for (String k : kelimeler) {
            String ilk = k.substring(0, 1).toUpperCase(tr);
            String kalan = k.substring(1).toLowerCase(tr);
            if (duzgun.length() > 0) {
                duzgun.append(' ');
            }
            duzgun.append(ilk).append(kalan);
            kisa.append(ilk);
        }
        System.out.println("Düzgün yazım: " + duzgun);
        System.out.println("Kısaltma    : " + kisa);
    }
}
```

Çıktı

```
Kurum adı:   istanbul   TEKNİK üniversitesi
Düzgün yazım: İstanbul Teknik Üniversitesi
Kısaltma    : İTÜ
```

10

Dış döngü her yeni harf grubunun başında durur. İç döngü, aynı harf devam ettikçe sayacı ve indeksi ilerletir. İç döngü koşulundaki `i < s.length()` kontrolünün **önce** yazılması şarttır: Kısa devre sayesinde `i` sona ulaştığında `charAt(i)` hiç çağrılmaz, böylece `StringIndexOutOfBoundsException` oluşmaz.

Sikistir.java

```
import java.util.Scanner;

public class Sikistir {
    static String sikistir(String s) {
        StringBuilder sb = new StringBuilder();
        int i = 0;
        while (i < s.length()) {
            char c = s.charAt(i);
            int adet = 0;
            while (i < s.length() && s.charAt(i) == c) {
                adet++;
                i++;
            }
            sb.append(c).append(adet);
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Kelime: ");
        String kelime = sc.next();
        System.out.println("Sıkıştırılmış: " + sikistir(kelime));
        System.out.println(sikistir("xyz") + " " + sikistir("kkkkkkkkkkk"));
    }
}
```

Çıktı

```
Kelime: aaabccdddd
Sıkıştırılmış: a3b1c2d4
x1y1z1 k12
```

PROJE 1

Öğrenci Not Sistemi

İlk sekiz bölümde öğrendiklerini ilk kez **tek bir gerçek programda** birleştiriyoruz: Scanner ile güvenli girdi okuma (Bölüm 3), koşullar ve `switch` (Bölüm 4), döngüler (Bölüm 5), metotlar (Bölüm 6), diziler (Bölüm 7) ve String işlemleri (Bölüm 8). Kodu okumadan önce kendin yazmayı dene; takıldığın yerde "Tasarım" ve "Kod" başlıklarındaki açıklamalara bak.

Amaç

Bir dersin hocasının işini kolaylaştıran bir konsol programı yazacağız. Hoca öğrencilerin adlarını ve notlarını girecek; program her öğrencinin ortalamasını ve harf notunu hesaplayacak, sınıfın durumunu özetleyecek ve bir menü üzerinden istenen bilgiyi gösterecek. Kullanıcı ne yazarsa yazsın program **çökmemeli**.

Gereksinimler

- Program önce öğrenci sayısını sorar (1–50 arası).
- Her öğrenci için **ad soyad**, **vize** ve **final** notu girilir. Notlar 0–100 arası tam sayıdır. Geçersiz girişte (harf, boş satır, aralık dışı sayı) program uyarır ve **aynı soruyu tekrar sorar**.
- Ortalama = vize × 0.4 + final × 0.6.
- Harf notu aşağıdaki tabloya göre verilir (ortalama, harfin altındaki değere eşit veya ondan büyükse o harf alınır):

Harf	AA	BA	BB	CB	CC	DC	DD	FD	FF
En düşük ortalama	90	85	80	75	70	65	60	50	0

- Veriler girildikten sonra bir **menü** gösterilir ve kullanıcı `0` seçene kadar tekrar edilir:
  - `1` → Bütün öğrencileri tablo halinde listele (no, ad, vize, final, ortalama, harf).
  - `2` → İstatistik: sınıf ortalaması, en yüksek ve en düşük ortalamalı öğrenci.
  - `3` → Sınıf ortalamasının üstündeki öğrenciler.
  - `4` → Harf notu dağılımı: her harf için o harfi alan öğrenci sayısı kadar `*`.
  - `5` → İsimle arama: aranan metin ismin herhangi bir yerinde geçebilir; büyük/küçük harf önemsizdir (`ibra` yazınca `İbrahim` bulunmalı).
  - `0` → Çıkış.

Tasarım

Veriler: paralel diziler

Her öğrencinin dört bilgisi var: ad, vize, final, ortalama. Bunları dört ayrı dizide tutacağız. Dizilerin **aynı indeksi aynı öğrenciye** aittir; bu yapıya **paralel diziler** (parallel arrays) denir:

```
indeks         0              1              2              3
adlar       Ayşe Yılmaz    Mehmet Can     Zeynep Kaya    İbrahim Demir
vizeler        70             50             95             60
finaller       85             58             88             40
ortalamalar    79.0           54.8           90.8           48.0
```

Öğrenci sayısı programın başında öğrenildiği için diziler `new String[n]` gibi tam o boyutta oluşturulur. (Bölüm 9'da bu dört bilgiyi tek bir `Ogrenci` sınıfında toplamayı, Bölüm 13'te boyutu sonradan değişebilen `ArrayList`'i öğreneceksin.)

Sabitler: harf tablosu

Harf tablosunu `if–else` zinciri yerine **iki paralel sabit dizi** ile tutuyoruz: `HARFLER` ve `ALT_SINIRLAR`. Bir ortalamanın harfini bulmak için `ALT_SINIRLAR` dizisinde baştan yürürüz; ortalamanın geçtiği ilk sınırın indeksi, harfin indeksidir. Son sınır 0 olduğu için döngü her zaman durur. Bu diziler `main`'in dışında, sınıfın içinde `static final` olarak tanımlanır; böylece sınıftaki **bütün metotlar** onları görebilir (ayrıntısı Bölüm 9'da). Tablo değişirse sadece bu iki satırı düzeltmek yeter.

Metotlar

Her işi tek bir metoda veriyoruz. `main` sadece verileri okur ve menü döngüsünü çalıştırır; asıl işi metotlar yapar:

Metot	Görevi	Dönüş tipi
`sayiOku(sc, mesaj, min, max)`	`min`–`max` arası geçerli bir tam sayı girilene kadar sorar	`int`
`satirOku(sc)`	Bir satır okur; girdi bittiyse (EOF) programı düzgünce kapatır	`String`
`rakamlardanMi(s)`	Metin sadece rakamlardan mı oluşuyor?	`boolean`
`metinOku(sc, mesaj)`	Boş olmayan bir satır okur	`String`
`ortalamaHesapla(vize, fin)`	%40 vize + %60 final	`double`
`harfIndeksi(ort)`	Ortalamanın `HARFLER` dizisindeki indeksi	`int`
`harfNotu(ort)`	Ortalamanın harf notu	`String`
`sinifOrtalamasi(ort)`	Ortalamalar dizisinin ortalaması	`double`
`adlar(ad, ort, hedef)`	Ortalaması `hedef`'e eşit **bütün** öğrencilerin adları	`String`
`menuYazdir()`	Menüyü ekrana yazar	`void`
`listele(ad, vize, fin, ort)`	Öğrenci tablosunu yazar	`void`
`istatistik(ad, ort)`	Sınıf ortalaması, en yüksek, en düşük	`void`
`ortalamaUstu(ad, ort)`	Ortalamanın üstündekileri yazar	`void`
`histogram(ort)`	Harf notu dağılımını yıldızlarla çizer	`void`
`ara(aranan, ad, ort)`	İsimde geçen metne göre arar	`void`

Kod

Tam programı aşağıda tek parça halinde bulacaksın. Önce önemli kısımların neden öyle yazıldığına bakalım.

1. Çökmeyen girdi okuma

Programın en çok emek isteyen kısmı budur. Bütün girdileri `nextLine()` ile **satır olarak** okuyoruz; böylece Bölüm 3'teki `nextInt()` + `nextLine()` tuzağı hiç oluşmuyor. `sayiOku` metodu satırı aldıktan sonra:

1. Baştaki ve sondaki boşlukları `strip()` ile atar.
2. `rakamlardanMi` ile metnin **sadece rakamlardan** oluştuğunu kontrol eder (`Character.isDigit`). `"abc"`, `""`, `"7.5"`, `"-3"` bu kontrolden geçemez. Böylece `Integer.parseInt` hiçbir zaman `NumberFormatException` fırlatmaz. En fazla 9 basamağa izin veriyoruz, çünkü `int` en çok 10 basamaklı olabilir ve 10 basamaklı bir sayı sınırı aşabilir.
3. Sayıyı çevirir ve `min`–`max` aralığında olup olmadığına bakar.

Bir adım başarısız olursa uyarı yazılır ve `while (true)` döngüsü soruyu tekrarlar. Geçerli bir sayı bulunduğunda `return` hem metottan hem döngüden çıkar. `metinOku` da benzer şekilde, kullanıcı boş satır girdiği sürece soruyu tekrarlar (`do-while`).

İkisi de satırı doğrudan `sc.nextLine()` ile değil `satirOku` ile alır. Kullanıcı terminalde Ctrl+D (Windows'ta Ctrl+Z, Enter) ile girdiyi **bitirirse** okunacak satır kalmaz ve `nextLine()` `NoSuchElementException` fırlatıp programı çökertir. `satirOku` önce `sc.hasNextLine()` ile satır olup olmadığına bakar; yoksa bir mesaj yazıp `System.exit(0)` ile programı düzgünce kapatır (`System.exit`, programı o anda bitiren hazır metottur).

2. Ortalama ve harf notu

`ortalamaHesapla` metodu `vize * 0.4 + fin * 0.6` yerine `(vize * 4 + fin * 6) / 10.0` hesaplar. İki formül matematikte aynıdır; ama Bölüm 2'de gördüğün gibi `0.4` ve `0.6` bilgisayarda **tam olarak** saklanamaz ve harf sınırında yanlış not verilebilir:

● Sık Yapılan Hata: Sınırda yanlış harf notu

```
public class OndalikTuzak {
    public static void main(String[] args) {
        int vize = 6, fin = 96;
        System.out.println(vize * 0.4 + fin * 0.6);
        System.out.println((vize * 4 + fin * 6) / 10.0);
    }
}
```

Çıktı

```
59.99999999999999
60.0
```

Gerçek ortalama tam 60, yani **DD**. Ama ilk formülle 60'tan küçük bir sayı çıkıyor ve öğrenci **FD** alıyor! İkinci formülde çarpma ve toplama tam sayılarla yapılır, en sonda tek bir bölme kalır; sonuç her zaman doğru çıkar.

`harfIndeksi`, ortalama `ALT_SINIRLAR[i]`'den küçük olduğu sürece `i`'yi artırır. Örneğin 79.0 için 90, 85 ve 80'den küçüktür ama 75'ten küçük değildir; döngü `i = 3`'te durur ve `HARFLER[3]` = `"CB"` olur.

3. Menü

Menü bir `do-while` döngüsüdür: Menü **en az bir kere** gösterilmeli, kullanıcı `0` seçene kadar tekrarlanmalıdır. Seçim yine `sayiOku` ile 0–5 aralığında okunduğu için `switch`'e geçersiz bir değer hiç gelmez; bu yüzden `default` dalına gerek kalmadı. Her `case`'in sonundaki `break`'i unutma (Bölüm 4).

4. Tablo, istatistik ve histogram

- `listele`, Bölüm 3'teki `printf` genişlikleriyle hizalı bir tablo çizer: `%-16.16s` adı 16 karakterlik alana **sola** yaslar ve 16 karakterden uzun adları keser (yalnız `%-16s` yazsaydık uzun ad tabloyu kaydırırdı), `%6.1f` ortalamayı 6 karakterlik alana **sağa** yaslar. Çizgi için `"-".repeat(46)` kullandık.
- `istatistik`, en yüksek ve en düşük ortalamanın **değerini değil indeksini** tutar (Bölüm 7). Ama aynı en yüksek ortalamayı iki öğrenci paylaşabilir; yalnız `ad[enYuksek]` yazsaydık ilk girilen gösterilir, diğeri unutulurdu. Bu yüzden ad, `adlar` metoduyla bulunur: ortalaması o değere eşit **herkesi** virgülle birleştirir.
- `histogram`, Bölüm 8'deki harf frekansı (`int[26]`) fikrinin aynısıdır: `HARFLER` ile aynı boyutta bir sayaç dizisi oluşturup her öğrenci için `sayac[harfIndeksi(o)]++` yapar. Yıldızları `"*".repeat(sayac[i])` çizer.

5. Türkçe harflerle arama

`ara` metodu hem aranan metni hem her ismi **Türkçe kurallarla** küçük harfe çevirip `contains` ile karşılaştırır. `Locale.ROOT` kullansaydık `İ` harfi küçültülünce **iki karaktere** (`i` + ayrı bir nokta işareti) dönüşürdü ve `ibra` araması `İbrahim`'i bulamazdı. Bölüm 8'deki kural: Kullanıcıya ait Türkçe metinlerde `tr`.

Tam program

NotSistemi.java

```
import java.util.Locale;
import java.util.Scanner;

public class NotSistemi {
    // Harf notları ve alt sınırları: aynı indeksler birbirine karşılık gelir
    static final String[] HARFLER =
            {"AA", "BA", "BB", "CB", "CC", "DC", "DD", "FD", "FF"};
    static final int[] ALT_SINIRLAR = {90, 85, 80, 75, 70, 65, 60, 50, 0};
    static final Locale TR = Locale.forLanguageTag("tr-TR");

    public static void main(String[] args) {
        Locale.setDefault(Locale.US);          // ondalık ayırıcı her yerde nokta
        Scanner sc = new Scanner(System.in);

        System.out.println("===== ÖĞRENCİ NOT SİSTEMİ =====");
        int n = sayiOku(sc, "Öğrenci sayısı: ", 1, 50);

        String[] adlar = new String[n];
        int[] vizeler = new int[n];
        int[] finaller = new int[n];
        double[] ortalamalar = new double[n];

        for (int i = 0; i < n; i++) {
            System.out.println("--- " + (i + 1) + ". öğrenci ---");
            adlar[i] = metinOku(sc, "Ad soyad: ");
            vizeler[i] = sayiOku(sc, "Vize: ", 0, 100);
            finaller[i] = sayiOku(sc, "Final: ", 0, 100);
            ortalamalar[i] = ortalamaHesapla(vizeler[i], finaller[i]);
        }

        int secim;
        do {
            menuYazdir();
            secim = sayiOku(sc, "Seçimin: ", 0, 5);
            switch (secim) {
                case 1:
                    listele(adlar, vizeler, finaller, ortalamalar);
                    break;
                case 2:
                    istatistik(adlar, ortalamalar);
```

```
                break;
            case 3:
                ortalamaUstu(adlar, ortalamalar);
                break;
            case 4:
                histogram(ortalamalar);
                break;
            case 5:
                ara(metinOku(sc, "Aranacak isim: "), adlar, ortalamalar);
                break;
            case 0:
                System.out.println("Program kapatıldı. İyi çalışmalar!");
                break;
        }
    } while (secim != 0);
}

// ---------- Girdi okuma ----------

// min–max aralığında geçerli bir tam sayı girilene kadar tekrar sorar
static int sayiOku(Scanner sc, String mesaj, int min, int max) {
    while (true) {
        System.out.print(mesaj);
        String satir = satirOku(sc);
        if (rakamlardanMi(satir)) {
            int deger = Integer.parseInt(satir);
            if (deger >= min && deger <= max) {
                return deger;
            }
        }
        System.out.println("  Geçersiz! " + min + "–" + max
                + " arası bir tam sayı gir.");
    }
}

// Boş değil, en fazla 9 karakter ve sadece rakam mı? (parseInt güvenli olsun)
static boolean rakamlardanMi(String s) {
    if (s.isEmpty() || s.length() > 9) {
        return false;
    }
    for (int i = 0; i < s.length(); i++) {
        if (!Character.isDigit(s.charAt(i))) {
            return false;
        }
    }
    return true;
}

// Boş bırakılamayan bir satır okur
static String metinOku(Scanner sc, String mesaj) {
    String satir;
    do {
        System.out.print(mesaj);
        satir = satirOku(sc);
    } while (satir.isEmpty());
    return satir;
}

// Girdi bittiyse (Ctrl+D / Ctrl+Z) çökmeden kapanır
static String satirOku(Scanner sc) {
    if (!sc.hasNextLine()) {
        System.out.println();
        System.out.println("Girdi bitti, program kapanıyor.");
        System.exit(0);
    }
```

```
        return sc.nextLine().strip();
    }

    // ---------- Hesaplama ----------

    // vize * 0.4 + final * 0.6; ondalık hatası olmasın diye tam sayıyla hesaplanır
    static double ortalamaHesapla(int vize, int fin) {
        return (vize * 4 + fin * 6) / 10.0;
    }

    // Ortalamanın HARFLER dizisindeki indeksi
    static int harfIndeksi(double ort) {
        int i = 0;
        while (ort < ALT_SINIRLAR[i]) {
            i++;
        }
        return i;
    }

    static String harfNotu(double ort) {
        return HARFLER[harfIndeksi(ort)];
    }

    static double sinifOrtalamasi(double[] ort) {
        double toplam = 0;
        for (double o : ort) {
            toplam += o;
        }
        return toplam / ort.length;
    }

    // ---------- Menü işlemleri ----------

    static void menuYazdir() {
        System.out.println();
        System.out.println("1) Liste  2) İstatistik  3) Ortalama üstü");
        System.out.println("4) Harf dağılımı  5) İsimle ara  0) Çıkış");
    }

    static void listele(String[] ad, int[] vize, int[] fin, double[] ort) {
        System.out.printf("%-3s %-16s %5s %6s %6s %5s%n",
                "No", "Ad Soyad", "Vize", "Final", "Ort.", "Harf");
        System.out.println("-".repeat(46));
        for (int i = 0; i < ad.length; i++) {
            System.out.printf("%-3d %-16.16s %5d %6d %6.1f %5s%n",
                    i + 1, ad[i], vize[i], fin[i], ort[i], harfNotu(ort[i]));
        }
    }

    static void istatistik(String[] ad, double[] ort) {
        int enYuksek = 0, enDusuk = 0;
        for (int i = 1; i < ort.length; i++) {
            if (ort[i] > ort[enYuksek]) {
                enYuksek = i;
            }
            if (ort[i] < ort[enDusuk]) {
                enDusuk = i;
            }
        }
        System.out.printf("Sınıf ortalaması: %.2f%n", sinifOrtalamasi(ort));
        System.out.printf("En yüksek       : %s (%.1f)%n",
                adlar(ad, ort, ort[enYuksek]), ort[enYuksek]);
        System.out.printf("En düşük        : %s (%.1f)%n",
                adlar(ad, ort, ort[enDusuk]), ort[enDusuk]);
    }
```

```
    // Ortalaması hedefe eşit olan herkes (beraberlik olabilir)
    static String adlar(String[] ad, double[] ort, double hedef) {
        String sonuc = "";
        for (int i = 0; i < ad.length; i++) {
            if (ort[i] == hedef) {
                sonuc += sonuc.isEmpty() ? ad[i] : ", " + ad[i];
            }
        }
        return sonuc;
    }

    static void ortalamaUstu(String[] ad, double[] ort) {
        double sinifOrt = sinifOrtalamasi(ort);
        System.out.printf("Sınıf ortalamasının (%.2f) üstündekiler:%n", sinifOrt);
        int sayac = 0;
        for (int i = 0; i < ad.length; i++) {
            if (ort[i] > sinifOrt) {
                System.out.printf("  %-16.16s %5.1f%n", ad[i], ort[i]);
                sayac++;
            }
        }
        System.out.println("Toplam " + sayac + " öğrenci.");
    }

    static void histogram(double[] ort) {
        int[] sayac = new int[HARFLER.length];
        for (double o : ort) {
            sayac[harfIndeksi(o)]++;
        }
        System.out.println("Harf notu dağılımı:");
        for (int i = 0; i < HARFLER.length; i++) {
            System.out.printf("  %s | %s%n", HARFLER[i], "*".repeat(sayac[i]));
        }
    }

    // Büyük/küçük harf duyarsız, isim parçasıyla arama
    static void ara(String aranan, String[] ad, double[] ort) {
        String kucukAranan = aranan.toLowerCase(TR);
        boolean bulundu = false;
        for (int i = 0; i < ad.length; i++) {
            if (ad[i].toLowerCase(TR).contains(kucukAranan)) {
                System.out.printf("  %s -> ortalama %.1f (%s)%n",
                        ad[i], ort[i], harfNotu(ort[i]));
                bulundu = true;
            }
        }
        if (!bulundu) {
            System.out.println("  \"" + aranan + "\" bulunamadı.");
        }
    }
}
```

Örnek çalışma

Aşağıdaki çalışmada 4 öğrenci giriliyor. Üçüncü öğrencinin vizesine önce yanlışlıkla `105` yazılıyor. Ardından menünün bütün seçenekleri sırayla deneniyor: iki farklı arama (`AY` ve `ibra`), geçersiz bir menü seçimi (`7`) ve çıkış.

Çıktı

```
===== ÖĞRENCİ NOT SİSTEMİ =====
Öğrenci sayısı: 4
--- 1. öğrenci ---
Ad soyad: Ayşe Yılmaz
Vize: 70
Final: 85
--- 2. öğrenci ---
Ad soyad: Mehmet Can
Vize: 50
Final: 58
--- 3. öğrenci ---
Ad soyad: Zeynep Kaya
Vize: 105
  Geçersiz! 0-100 arası bir tam sayı gir.
Vize: 95
Final: 88
--- 4. öğrenci ---
Ad soyad: İbrahim Demir
Vize: 60
Final: 40

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 1
No  Ad Soyad          Vize  Final   Ort.  Harf
-----------------------------------------------
1   Ayşe Yılmaz         70     85   79.0    CB
2   Mehmet Can          50     58   54.8    FD
3   Zeynep Kaya         95     88   90.8    AA
4   İbrahim Demir       60     40   48.0    FF

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 2
Sınıf ortalaması: 68.15
En yüksek       : Zeynep Kaya (90.8)
En düşük        : İbrahim Demir (48.0)

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 3
Sınıf ortalamasının (68.15) üstündekiler:
  Ayşe Yılmaz       79.0
  Zeynep Kaya       90.8
Toplam 2 öğrenci.

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 4
Harf notu dağılımı:
  AA | *
  BA |
  BB |
  CB | *
  CC |
  DC |
  DD |
  FD | *
  FF | *

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 5
Aranacak isim: AY
  Ayşe Yılmaz -> ortalama 79.0 (CB)
```

```
  Zeynep Kaya -> ortalama 90.8 (AA)

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 5
Aranacak isim: ibra
  İbrahim Demir -> ortalama 48.0 (FF)

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 7
  Geçersiz! 0-5 arası bir tam sayı gir.
Seçimin: 0
Program kapatıldı. İyi çalışmalar!
```

`AY` araması büyük harfle yazılmasına rağmen hem **Ay**şe Yılmaz'ı hem de K**ay**a soyadını buldu; `ibra` araması ise Türkçe kurallar sayesinde **İbra**him'i buldu.

> **Kendin Geliştir**
>
> - **Final barajı:** Finalden 50'nin altında alan öğrenci, ortalaması ne olursa olsun `FF` alsın.
> - **Geçti/kaldı:** İstatistik ekranına geçen (DD ve üstü) ve kalan öğrenci sayısını ekle.
> - **Sıralı liste:** Öğrencileri ortalamaya göre büyükten küçüğe listeleyen yeni bir menü seçeneği ekle. İpucu: Sıralarken dört dizinin elemanlarını **birlikte** yer değiştirmelisin (sıralama algoritmaları Bölüm 16'da).
> - **Not güncelleme:** İsmi verilen öğrencinin vize/final notunu değiştirip ortalamasını yeniden hesaplayan bir seçenek ekle.
> - **Standart sapma:** Sınıf ortalamasının yanında notların standart sapmasını da göster: `Math.sqrt(Σ(xᵢ − ort)² / n)`.
> - **Bütünleme:** `FD` veya `FF` alan öğrenciler için bütünleme notu iste; bütünleme notu final yerine geçsin.

PROJE 2

XOX (Tic-Tac-Toe) Oyunu

İkinci projemiz, herkesin bildiği bir oyun: **XOX** (İngilizcesiyle *Tic-Tac-Toe*). İki oyuncu aynı klavyeden sırayla oynar; 3×3'lük tahtada kendi işaretini **yan yana üç kareye** (satır, sütun ya da çapraz) ilk yerleştiren kazanır. Bu proje özellikle **iki boyutlu diziler** (Bölüm 7), metotlar (Bölüm 6), döngüler (Bölüm 5) ve kullanıcı girdisini String olarak işleme (Bölüm 8) için harika bir alıştırmadır.

Amaç

Oyunun kurallarını eksiksiz uygulayan, kullanıcının yanlış girişlerine karşı **sağlam** bir konsol oyunu yazmak. Tahta her hamleden sonra ekrana çizilecek, geçersiz hamleler reddedilecek, oyun bitince kazanan (ya da beraberlik) ilan edilecek ve oyunculara tekrar oynamak isteyip istemedikleri sorulacak.

Gereksinimler

- Tahta 3×3'lük bir `char[][]` dizisidir. Boş kareler boşluk karakteri (`' '`) ile gösterilir.
- Tahta, satır ve sütun numaralarıyla birlikte çizilir.
- Oyunu her zaman **X** başlatır; her geçerli hamleden sonra sıra diğer oyuncuya geçer.
- Hamle, satır ve sütun numarasının (1–3) arada boşlukla yazılmasıyla yapılır: `2 3` → 2. satır, 3. sütun.
- Geçersiz hamle reddedilir ve aynı oyuncuya tekrar sorulur. Üç durum ayrı ayrı uyarılır:
  - **Biçim hatası:** İki sayı yok ya da sayı olmayan bir şey yazılmış (`22`, `a b`, boş satır).
  - **Tahtanın dışı:** Sayılardan biri 1–3 aralığında değil (`3 4`).
  - **Dolu kare:** Seçilen karede zaten X ya da O var.
- Her hamleden sonra o hamleyi yapan oyuncunun kazanıp kazanmadığı kontrol edilir: 3 satır, 3 sütun ve 2 çapraz.
- Kimse kazanamadan 9 kare dolarsa oyun **berabere** biter.
- Oyun bitince "Tekrar oynamak ister misin? (e/h)" sorulur; `e` (veya `E`) dışındaki her cevap programı bitirir.

Tasarım

Tahta: char[][]

Tahtayı 3 satır, 3 sütunluk bir karakter dizisinde tutuyoruz. Kullanıcı 1–3 arası numaralar yazar, dizinin indeksleri ise 0–2'dir; bu yüzden tahtaya erişirken her zaman **1 çıkarırız**: kullanıcının `2 3` hamlesi `tahta[1][2]` karesine yazılır.

```
              sütun indeksi
               0   1   2
satır     0   [0][0] [0][1] [0][2]
indeksi   1   [1][0] [1][1] [1][2]
          2   [2][0] [2][1] [2][2]
```

`new char[3][3]` ile oluşturulan dizinin elemanları otomatik olarak `'\u0000'` (görünmeyen "boş karakter") değerini alır. Bu karakter ekranda düzgün görünmediği için tahtayı oluşturur oluşturmaz her kareye `BOS` (boşluk) koyuyoruz. `BOS`, `static final` bir sabittir (Not Sistemi projesindeki `HARFLER` gibi; ayrıntısı Bölüm 9'da); kodda `' '` yerine `BOS` yazmak, "bu kare boş mu?" sorusunu okunur kılar.

Kazanma kuralı

Bir oyuncunun kazanması için aşağıdaki **8 üçlüden** birinin tamamen onun işaretiyle dolu olması gerekir:

Tür	Kareler
Satırlar (3 tane)	`[i][0]`, `[i][1]`, `[i][2]` → `i` = 0, 1, 2
Sütunlar (3 tane)	`[0][i]`, `[1][i]`, `[2][i]` → `i` = 0, 1, 2
Ana çapraz	`[0][0]`, `[1][1]`, `[2][2]`
Ters çapraz	`[0][2]`, `[1][1]`, `[2][0]`

Satır ve sütunlar aynı döngüde kontrol edilebilir: `i` hem satır hem sütun numarası olarak kullanılır. Sadece **son hamleyi yapan** oyuncuyu kontrol etmek yeterlidir; çünkü diğer oyuncu bir önceki hamlesinde kazanmış olsaydı oyun zaten bitmiş olurdu.

Metotlar

Metot	Görevi	Dönüş tipi
`oyunOyna(sc)`	Bir oyunu başından sonuna oynatır	`void`
`yeniTahta()`	Bütün kareleri boş bir 3×3 tahta oluşturur	`char[][]`
`tahtayiCiz(tahta)`	Tahtayı numaralarla ekrana çizer	`void`
`hamleAl(sc, tahta, oyuncu)`	Geçerli bir hamle girilene kadar sorar, hamleyi tahtaya yazar	`void`
`satirOku(sc)`	Bir satır okur; girdi bittiyse oyunu düzgünce kapatır	`String`
`sayiMi(s)`	Metin sadece rakamlardan mı oluşuyor?	`boolean`
`kazandiMi(tahta, oyuncu)`	Oyuncunun 8 üçlüden birini doldurup doldurmadığına bakar	`boolean`

`main` sadece "tekrar oyna" döngüsünü yönetir; her tur için `oyunOyna`'yı çağırır.

Kod

Önce kodun önemli kısımlarını tek tek inceleyelim, sonra tam programı görelim.

1. Oyun döngüsü ve sıra değişimi

`oyunOyna` metodunun kalbi bir `while (true)` döngüsüdür. Her turda sırasıyla: tahta çizilir, hamle alınır, hamle sayısı artırılır, **önce kazanma sonra beraberlik** kontrol edilir. Sıralama önemlidir: 9. hamle aynı zamanda kazandıran hamle olabilir; beraberliğe önce baksaydık kazanan oyuncuya "berabere" demiş olurduk. Oyun bitince `return` hem döngüden hem metottan çıkar.

Sıra değişimi tek satırdır: `oyuncu = (oyuncu == 'X') ? 'O' : 'X';`. Bu, Bölüm 4'teki **üçlü operatördür** (`?:`): Koşul doğruysa `?`'den sonraki, yanlışsa `:`'dan sonraki değer seçilir. Aynı işi bir `if-else` ile de yapabilirdin.

Beraberliği anlamak için tahtayı taramaya gerek yok: Her geçerli hamle bir kare doldurduğu için **hamle sayısı 9 olduğunda** tahtada boş kare kalmamıştır.

2. Hamle alma ve doğrulama

`hamleAl` satırı `satirOku` ile okur (Not Sistemi projesindeki gibi: girdi biterse çökmeden kapanır) ve Bölüm 8'deki `split("\\s+")` kalıbıyla parçalara ayırır. Ardından üç kontrol **sırayla** yapılır; biri başarısız olursa uyarı yazılır ve `continue` ile döngünün başına dönülüp soru tekrarlanır:

1. **Biçim:** Tam olarak iki parça var mı ve ikisi de `sayiMi` kontrolünden geçiyor mu? Bu kontrol sayesinde `Integer.parseInt` asla `NumberFormatException` fırlatmaz.
2. **Sınır:** Sayılar 1–3 aralığında mı? Bu kontrol yapılmazsa `tahta[satir - 1][sutun - 1]` satırı `ArrayIndexOutOfBoundsException` ile programı çökertir.
3. **Doluluk:** Kare hâlâ `BOS` mu?

Kontrollerin sırası da önemlidir: Diziye erişen doluluk kontrolü, sınır kontrolünden **sonra** gelmek zorundadır. Üç kontrol de geçilirse hamle tahtaya yazılır ve `return` ile metottan çıkılır.

3. Tahtayı çizme

`tahtayiCiz` iç içe iki döngüyle her satırı yazar. Her karenin iki yanına birer boşluk koyup kareleri `|` ile ayırır; satırların arasına `---+---+---` çizgisi çeker. `if (j < 2)` ve `if (i < 2)` kontrolleri, son sütundan sonra `|`, son satırdan sonra çizgi konmasını engeller.

4. Tekrar oynama

`main`'deki döngü, `oyunOyna` her bittiğinde cevabı okur ve `equalsIgnoreCase("e")` ile kontrol eder; böylece `e` ve `E` aynı kabul edilir. Cevabı `==` ile karşılaştırmadığımıza dikkat et (Bölüm 8)! Her yeni oyunda `oyunOyna` kendi **yeni** tahtasını oluşturduğu için önceki oyunun izleri kalmaz.

Tam program

Xox.java

```java
import java.util.Scanner;

public class Xox {
    static final char BOS = ' ';

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("===== XOX =====");
        System.out.println("Hamle: satır ve sütunu boşlukla ayır (ör. 2 3).");

        boolean devam = true;
        while (devam) {
            oyunOyna(sc);
            System.out.print("Tekrar oynamak ister misin? (e/h): ");
            String cevap = satirOku(sc);
            devam = cevap.equalsIgnoreCase("e");
        }
        System.out.println("Görüşmek üzere!");
    }

    // Bir oyunu baştan sona oynatır
    static void oyunOyna(Scanner sc) {
        char[][] tahta = yeniTahta();
        char oyuncu = 'X';
        int hamleSayisi = 0;

        while (true) {
            tahtayiCiz(tahta);
            hamleAl(sc, tahta, oyuncu);
```

```
            hamleSayisi++;

            if (kazandiMi(tahta, oyuncu)) {
                tahtayiCiz(tahta);
                System.out.println("Tebrikler! " + oyuncu + " kazandı.");
                return;
            }
            if (hamleSayisi == 9) {
                tahtayiCiz(tahta);
                System.out.println("Berabere! Tahtada boş kare kalmadı.");
                return;
            }
            oyuncu = (oyuncu == 'X') ? 'O' : 'X';      // sıra değişir
        }
    }

    // Bütün kareleri boş (BOS) olan 3x3 tahta
    static char[][] yeniTahta() {
        char[][] tahta = new char[3][3];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                tahta[i][j] = BOS;
            }
        }
        return tahta;
    }

    static void tahtayiCiz(char[][] tahta) {
        System.out.println();
        System.out.println("     1   2   3");
        for (int i = 0; i < 3; i++) {
            System.out.print("  " + (i + 1) + " ");
            for (int j = 0; j < 3; j++) {
                System.out.print(" " + tahta[i][j] + " ");
                if (j < 2) {
                    System.out.print("|");
                }
            }
            System.out.println();
            if (i < 2) {
                System.out.println("    ---+---+---");
            }
        }
        System.out.println();
    }

    // Geçerli bir hamle girilene kadar sorar ve hamleyi tahtaya yazar
    static void hamleAl(Scanner sc, char[][] tahta, char oyuncu) {
        while (true) {
            System.out.print("Oyuncu " + oyuncu + ", hamlen (satır sütun): ");
            String[] parca = satirOku(sc).split("\\s+");

            if (parca.length != 2 || !sayiMi(parca[0]) || !sayiMi(parca[1])) {
                System.out.println("  Geçersiz giriş! Örnek: 2 3");
                continue;
            }
            int satir = Integer.parseInt(parca[0]);
            int sutun = Integer.parseInt(parca[1]);
            if (satir < 1 || satir > 3 || sutun < 1 || sutun > 3) {
                System.out.println("  Tahtanın dışı! 1–3 arası gir.");
                continue;
            }
            if (tahta[satir - 1][sutun - 1] != BOS) {
                System.out.println("  O kare dolu! Başka bir kare seç.");
                continue;
```

```
            }
            tahta[satir - 1][sutun - 1] = oyuncu;
            return;
        }
    }

    // Girdi bittiyse (Ctrl+D / Ctrl+Z) çökmeden kapanır (Not Sistemi'ndeki gibi)
    static String satirOku(Scanner sc) {
        if (!sc.hasNextLine()) {
            System.out.println();
            System.out.println("Girdi bitti, oyun kapanıyor.");
            System.exit(0);
        }
        return sc.nextLine().strip();
    }

    // 1–3 basamaklı, sadece rakamlardan oluşan metin mi?
    static boolean sayiMi(String s) {
        if (s.isEmpty() || s.length() > 3) {
            return false;
        }
        for (int i = 0; i < s.length(); i++) {
            if (!Character.isDigit(s.charAt(i))) {
                return false;
            }
        }
        return true;
    }

    // Oyuncu 3 satırdan, 3 sütundan veya 2 çaprazdan birini doldurdu mu?
    static boolean kazandiMi(char[][] t, char o) {
        for (int i = 0; i < 3; i++) {
            if (t[i][0] == o && t[i][1] == o && t[i][2] == o) {
                return true;                            // i. satır
            }
            if (t[0][i] == o && t[1][i] == o && t[2][i] == o) {
                return true;                            // i. sütun
            }
        }
        if (t[0][0] == o && t[1][1] == o && t[2][2] == o) {
            return true;                                // ana çapraz
        }
        return t[0][2] == o && t[1][1] == o && t[2][0] == o;   // ters çapraz
    }
}
```

Örnek çalışma

Bu çalışmada X oyuncusu önce biçimi yanlış bir giriş (`22`) yapıyor. Oyun ilerlerken X, O'nun aldığı kareyi (`1 1`) seçmeye çalışıyor; O ise tahtanın dışında bir kare (`3 4`) giriyor. Sonunda X ters çaprazı (`1 3`, `2 2`, `3 1`) tamamlayarak kazanıyor ve oyuncular tekrar oynamak istemiyor.

Çıktı

```
===== XOX =====
Hamle: satır ve sütunu boşlukla ayır (ör. 2 3).

     1   2   3
  1    |   |
    ---+---+---
  2    |   |
    ---+---+---
  3    |   |
```

```
Oyuncu X, hamlen (satır sütun): 22
  Geçersiz giriş! Örnek: 2 3
Oyuncu X, hamlen (satır sütun): 2 2

     1   2   3
  1    |   |
    ---+---+---
  2    | X |
    ---+---+---
  3    |   |

Oyuncu O, hamlen (satır sütun): 1 1

     1   2   3
  1  O |   |
    ---+---+---
  2    | X |
    ---+---+---
  3    |   |

Oyuncu X, hamlen (satır sütun): 1 1
  O kare dolu! Başka bir kare seç.
Oyuncu X, hamlen (satır sütun): 1 3

     1   2   3
  1  O |   | X
    ---+---+---
  2    | X |
    ---+---+---
  3    |   |

Oyuncu O, hamlen (satır sütun): 3 4
  Tahtanın dışı! 1–3 arası gir.
Oyuncu O, hamlen (satır sütun): 2 1

     1   2   3
  1  O |   | X
    ---+---+---
  2  O | X |
    ---+---+---
  3    |   |

Oyuncu X, hamlen (satır sütun): 3 1

     1   2   3
  1  O |   | X
    ---+---+---
  2  O | X |
    ---+---+---
  3  X |   |

Tebrikler! X kazandı.
Tekrar oynamak ister misin? (e/h): h
Görüşmek üzere!
```

Geçersiz girişlerden sonra tahta yeniden çizilmedi, sadece soru tekrarlandı: `hamleAl` metodu geçerli hamle gelene kadar kendi döngüsünden çıkmaz. Ayrıca X'in kazandığı hamle 5. hamleydi; kazanma kontrolü beraberlik kontrolünden önce yapıldığı için oyun doğru şekilde sona erdi.

Kendin Geliştir

- **Bilgisayara karşı oyun:** O oyuncusunu bilgisayar oynasın. İlk sürümde `new Random(42)` ile rastgele **boş** bir kare seçsin. Sonra akıllandır: Kazanabileceği bir hamle varsa onu yapsın; yoksa rakibin kazanacağı kareyi kapatsın; o da yoksa ortayı, sonra köşeleri tercih etsin.
- **Skor tablosu:** Oyunlar arasında X'in, O'nun galibiyet sayılarını ve beraberlikleri tut; her oyunun sonunda göster. İpucu: `oyunOyna` kazananı (`'X'`, `'O'` ya da beraberlik için `'-'`) `char` olarak döndürsün.
- **Sırayı değiştir:** Her yeni oyunda başlayan oyuncu değişsin (ilk oyunu X, ikinciyi O başlatsın).
- **Oyuncu isimleri:** Oyun başında iki oyuncunun adını sor; mesajlarda "Oyuncu X" yerine isimler yazılsın.
- **N×N tahta:** Tahta boyutunu kullanıcıdan al (3–9). `kazandiMi` metodunu sabit indeksler yerine döngülerle yeniden yaz; bir satırın tamamının aynı işaret olup olmadığını iç döngüyle kontrol et.
- **Satranç tarzı giriş:** Hamleyi `b3` gibi (sütun harfi + satır numarası) alan bir seçenek ekle. Harfi `charAt(0) - 'a'` ile sütun indeksine çevirebilirsin.

DENEME SINAVI

Deneme Vize Sınavı

Bu deneme sınavı kitabın ilk sekiz bölümünü kapsar: Java'ya giriş, değişkenler ve operatörler, Scanner ve `printf`, koşullar, döngüler, metotlar, diziler ve String. Soru türleri ve zorluk, bilgisayar mühendisliği 1. sınıf vize sınavlarına göre ayarlandı; tuzaklar **bilerek** kondu. Kolay görünen sorulara özellikle dikkat et.

Bölüm	Soru sayısı	Soru başına puan	Toplam
A) Çoktan seçmeli	10	3	30
B) Çıktı tahmini	4	10	40
C) Kod yazma	3	10	30
Toplam	**17**		**100**

Süre: 90 dakika. Bilgisayar, kitap ve not kullanmak yok; sadece kâğıt ve kalem.

> **İpucu: Önce süre tutarak çöz**
>
> Sınavı gerçek sınav gibi çöz: 90 dakikalık bir sayaç kur ve süre bitene kadar çözümlere bakma. Kabaca **20 dakikayı** testlere, **35 dakikayı** çıktı sorularına, **35 dakikayı** kod yazmaya ayır. Bir soruda takılırsan işaretle ve geç; sona dönersin. Süre bitince kodlarını VS Code'da çalıştırmadan önce çözümlerle karşılaştırıp kendini puanla.

> **Not: Kendini nasıl puanlarsın?**
>
> - **Çıktı sorularında** çıktının her satırı eşit puan değerindedir. Eksik boşluk, fazladan satır veya `5` yerine `5.0` yazmak o satırı yanlış yapar; gerçek sınavda da böyledir.
> - **Kod sorularında** çözümdeki program tek doğru cevap değildir. Senin kodun aynı işi yapıyorsa tam puan al. Derleme hatası olan her satır için 1 puan, yanlış sonuç veren her mantık hatası için 2–3 puan kır.
> - Her cevabın sonunda konunun anlatıldığı bölüm yazıyor. Yanlış yaptığın soruların bölümlerindeki **Sınavda Çıkar** ve **Sık Yapılan Hata** kutularını tekrar oku.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki satırlardan hangisi **derlenmez**?

(A) `double d = 7 / 2;`

(B) `byte b = 10; b += 300;`

(C) `char c = 'a' + 1;`

(D) `long n = 'x';`

(E) `int y = 7 / 2.0;`

2 Aşağıdaki kod parçası çalıştıktan sonra `x` ve `y`'nin değerleri nedir?

```
int x = 4;
int y = x++ + x-- * 2 - --x;
```

(A) `x = 3, y = 11` (B) `x = 4, y = 11` (C) `x = 3, y = 12` (D) `x = 4, y = 13` (E) `x = 5, y = 9`

3 `System.out.printf("[%6.2f][%-4d][%s]%n", 9.996, 42, 'A' + 1);` komutunun çıktısı nedir? (Boşluklar `␣` ile gösterilmiştir.)

(A) `[␣␣9.99][42␣␣][B]`

(B) `[␣10.00][42␣␣][66]`

(C) `[␣10.00][␣␣42][66]`

(D) `[␣␣9.99][42␣␣][66]`

(E) `[10.00␣][42␣␣][B]`

4 Aşağıdaki kod parçası çalıştıktan sonra `r`, `a` ve `b`'nin değerleri nedir?

```
int a = 3, b = 5;
boolean r = a > b && ++a > 0 || b++ > 4;
```

(A) `r = true, a = 4, b = 6`

(B) `r = false, a = 3, b = 5`

(C) `r = true, a = 3, b = 6`

(D) `r = true, a = 4, b = 5`

(E) `r = false, a = 4, b = 6`

5 Aşağıdaki kod parçası çalıştıktan sonra `t`'nin değeri nedir?

```
int k = 2, t = 0;
switch (k) {
    case 1:
        t += 1;
    case 2:
        t += 2;
    case 3:
        t += 3;
    default:
        t += 10;
    case 4:
        t += 100;
        break;
}
```

(A) `2` (B) `5` (C) `15` (D) `115` (E) `116`

6 Aşağıdaki kod parçası ne yazdırır?

```
int i = 0, s = 0;
while (i++ < 5) {
    s += i;
}
System.out.println(s + " " + i);
```

(A) `15 5` (B) `10 5` (C) `21 6` (D) `10 6` (E) `15 6`

7 Aşağıdaki program ne yazdırır?

```
public class Test7 {
    static void f(int x)    { System.out.print("int "); }
    static void f(long x)   { System.out.print("long "); }
    static void f(double x) { System.out.print("double "); }

    public static void main(String[] args) {
        f('A');
        f(5L);
        f(2.5f);
    }
}
```

(A) `double long double`

(B) `int long double`

(C) Derleme hatası: `f('A')` için uygun metot yoktur.

(D) `int int double`

(E) `long long double`

8 Aşağıdaki kod parçası ne yazdırır?

```
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 10;
a = new int[] {4, 5, 6};
System.out.println(a[0] + b[0] + "" + b.length);
```

(A) `53` (B) `83` (C) `4103` (D) `143` (E) `113`

9 Aşağıdaki kod parçası ne yazdırır?

```
String s1 = "Java";
String s2 = "Ja" + "va";
String s3 = new String("Java");
String s4 = s3;
System.out.println((s1 == s2) + " " + (s1 == s3) + " "
        + s1.equals(s3) + " " + (s3 == s4));
```

A) `true false true true`
B) `false false true true`
C) `true true true true`
D) `false false false true`
E) `true false false true`

10 Aşağıdaki kod parçası ne yazdırır?

```
String s = "Bilgisayar";
s.toUpperCase();
s.replace('a', 'e');
System.out.println(s.substring(3, 6) + s.indexOf('a') + s.lastIndexOf("a"));
```

A) `GIS68`
B) `gisa68`
C) `gis68`
D) `gis14`
E) `gis57`

B) Çıktı Tahmini

11 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru11 {
    public static void main(String[] args) {
        int toplam = 0;
        for (int i = 1; i <= 10; i++) {
            if (i % 3 == 0) {
                continue;
            }
            if (i * i > 50) {
                break;
            }
            toplam += i;
            System.out.print(i + " ");
        }
        System.out.println("| toplam=" + toplam);

        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print((i + j) % 2 == 0 ? "*" : "-");
            }
            System.out.println();
        }

        int n = 5381, ters = 0;
        while (n > 0) {
            ters = ters * 10 + n % 10;
            n /= 10;
        }
        System.out.println(ters + 1);
    }
}
```

12 Aşağıdaki program ekrana ne yazdırır?

```java
import java.util.Arrays;

public class Soru12 {
    static int degistir(int x, int[] d) {
        x = x * 2;
        d[0] = d[0] + x;
        d = new int[] {100, 200};
        d[1] = 0;
        return x + d[0];
    }

    static void kaydir(int[] d) {
        int ilk = d[0];
        for (int i = 0; i < d.length - 1; i++) {
            d[i] = d[i + 1];
        }
        d[d.length - 1] = ilk;
    }

    public static void main(String[] args) {
        int a = 5;
        int[] dizi = {1, 2, 3};
        int r = degistir(a, dizi);
        System.out.println(a + " " + r);
        System.out.println(Arrays.toString(dizi));
        kaydir(dizi);
        kaydir(dizi);
        System.out.println(Arrays.toString(dizi));
    }
}
```

13 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru13 {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        int[] satir = m[1];
        satir[2] = 0;
        m[2] = m[0];
        m[2][0] = 99;

        for (int[] s : m) {
            int t = 0;
            for (int v : s) {
                t += v;
            }
            System.out.print(t + " ");
        }
        System.out.println();

        int kosegen = 0;
        for (int i = 0; i < m.length; i++) {
            kosegen += m[i][i];
        }
        System.out.println(kosegen);
        System.out.println(m[0][0] + m[1][2] + "" + m[2].length);
    }
}
```

14 Aşağıdaki program ekrana ne yazdırır?

```java
public class Soru14 {
    public static void main(String[] args) {
        String s = "Programlama";
        s.toUpperCase();
        String t = s.replace("am", "_");
        System.out.println(s.length() + " " + t + " " + t.length());

        String u = s.substring(3, 7) + s.charAt(s.length() - 1);
        System.out.println(u + s.indexOf("ram") + s.lastIndexOf('a'));

        StringBuilder sb = new StringBuilder("abc");
        sb.append(12).insert(0, 'x').reverse();
        System.out.println(sb + " " + sb.length());

        String a = "ja";
        String c = a + "va";
        System.out.println((c == "java") + " " + c.equals("java") + " "
                + "Ali".compareTo("Ayse"));
    }
}
```

C) Kod Yazma

15 Kullanıcıdan **0 girilene kadar** tam sayı okuyan `SayiAnaliz` programını yaz. 0 sayılmaz. Program sonunda şunları yazdırsın:

- girilen sayı adedi,
- **pozitif** sayıların ortalaması (virgülden sonra 2 basamak; pozitif sayı yoksa `Pozitif sayı yok.`),
- en büyük sayı (sayıların hepsi negatif olabilir!),
- çift sayıların adedi.

Hiç sayı girilmeden 0 girilirse sadece `Hiç sayı girilmedi.` yazsın. Örnek çalışma:

```
Sayı (bitirmek için 0): 12
Sayı (bitirmek için 0): -5
Sayı (bitirmek için 0): 7
Sayı (bitirmek için 0): 30
Sayı (bitirmek için 0): -8
Sayı (bitirmek için 0): 0
Adet: 5
Pozitif ortalaması: 16.33
En büyük: 30
Çift sayı adedi: 3
```

16 `DiziIslemleri` sınıfına aşağıdaki iki metodu yaz ve `main`'de `{7, 3, 9, 1, 9, 4}` dizisiyle test et:

- `static void tersCevir(int[] d)`: Diziyi **yeni bir dizi oluşturmadan** (yerinde) ters çevirir.
- `static int ikinciEnBuyuk(int[] d)`: Dizideki ikinci en büyük **farklı** değeri döndürür. Diziyi sıralama; tek döngü kullan. Dizide en az iki farklı değer olduğunu varsayabilirsin.

Beklenen çıktı:

```
Önce : [7, 3, 9, 1, 9, 4]
Sonra: [4, 9, 1, 9, 3, 7]
İkinci en büyük: 7
```

17 Kullanıcıdan bir cümle okuyan `CumleAnaliz` programını yaz. Program cümlenin **kelime sayısını**, **sesli harf sayısını** (Türkçe sesliler: a e ı i o ö u ü, büyük ve küçük) ve boşluklar ile büyük/küçük harf farkı yok sayıldığında cümlenin **palindrom** (tersten okunuşu aynı) olup olmadığını yazdırsın. Kelimeler arasında birden fazla boşluk olabilir. Örnek çalışma:

```
Cümle: Ey Edip Adanada pide ye
Kelime sayısı: 5
Sesli harf sayısı: 10
Palindrom: evet
```

Çözümler

Cevap anahtarı

1 E	2 A	3 B	4 C	5 D	6 E	7 B	8 D
9 A	10 C						

1 Cevap: E

`7 / 2.0` işleminin sonucu `double`'dır (`3.5`); `double` → `int` daralan dönüşüm cast olmadan yapılmaz: *possible lossy conversion from double to int*. Diğerleri derlenir:

- A) `7 / 2` tam sayı bölmesidir, sonuç `3` olur ve `double`'a otomatik genişler: `3.0`.
- B) Bileşik atama gizli cast içerir: `b = (byte) (10 + 300)`. Derlenir ama taşar; `b` 54 olur.
- C) `'a' + 1` **sabit** bir ifadedir ve sonucu (98) `char` aralığına sığdığı için derlenir: `'b'`.
- D) `char` → `long` genişleyen dönüşümdür: `120`.

A–D şıklarındaki satırlar bir programda çalıştırılıp değişkenler yazdırıldığında:

Çıktı
```
3.0 54 b 120
```

E şıkkındaki satırın derleyici çıktısı:

Derleyici çıktısı – program derlenmez
```
Test1E.java:3: error: incompatible types: possible lossy conversion from double to int
        int y = 7 / 2.0;
                    ^
1 error
```

Konu: Bölüm 2.

2 Cevap: A

Değişken tablosuyla soldan sağa git:

1. `x++` → değer **4**, sonra `x` = 5.
2. `x--` → değer **5**, sonra `x` = 4. Çarpma önce: `5 * 2 = 10`.
3. `--x` → önce `x` = 3, değer **3**.
4. `y = 4 + 10 - 3 = 11`, `x` = 3.

Çıktı
```
x = 3, y = 11
```

Konu: Bölüm 2.

3 Cevap: B

- `%6.2f`: `9.996` iki basamağa **yuvarlanınca** `10.00` olur (5 karakter); 6'ya tamamlamak için başa 1 boşluk eklenir. `9.99` yazan şıklar yuvarlamayı kesme sanıyor.
- `%-4d`: `42` sola yaslanır, sağına 2 boşluk gelir. C şıkkı sağa yaslamış.

- `'A' + 1`: `char + int` → `int` (66). `%s` bunu `66` diye yazar; `B` görmek için `(char) ('A' + 1)` gerekirdi.

```
Çıktı
[ 10.00][42  ][66]
```

Konu: Bölüm 2 ve 3.

4 Cevap: C

`&&`, `||`'dan önceliklidir; ifade `(a > b && ++a > 0) || (b++ > 4)` olarak okunur.

1. `a > b` → `3 > 5` → `false`. `&&`'nin sol tarafı `false` olduğu için **kısa devre** olur, `++a` **hiç çalışmaz**: `a` 3 kalır.
2. `||`'nin sol tarafı `false`, sağ tarafa bakılır: `b++ > 4` → `5 > 4` → `true`, sonra `b` = 6.

```
Çıktı
r = true, a = 3, b = 6
```

Konu: Bölüm 4.

5 Cevap: D

`k = 2` olduğu için `case 2`'den girilir. `break` olmadığı için alttaki **bütün** etiketlere düşülür (*fall-through*); `default`'un ortada olması bir şey değiştirmez, o da bir etikettir: `t = 2 + 3 + 10 + 100 = 115`. İlk `break` `case 4`'ün sonundadır.

```
Çıktı
115
```

Konu: Bölüm 4.

6 Cevap: E

`i++ < 5` karşılaştırmada `i`'nin **eski** değerini kullanır, ama artırma her karşılaştırmada yapılır:

Kontrol	Karşılaştırılan	`i` (sonra)	`s`
1.	`0 < 5`, doğru	1	1
2.	`1 < 5`, doğru	2	3
3.	`2 < 5`, doğru	3	6
4.	`3 < 5`, doğru	4	10
5.	`4 < 5`, doğru	5	15
6.	`5 < 5`, yanlış	**6**	15

Koşul yanlış çıksa bile `i++` çalışmıştır; bu yüzden `i` 5 değil **6**'dır.

Konu: Bölüm 5.

7 Cevap: B

Aşırı yüklenmiş metotlardan, argümana **en yakın** genişleyen dönüşümle uyan seçilir:

- `f('A')`: `f(char)` yok. `char` → `int` en yakın genişlemedir: `int`.

- `f(5L)`: Tam uyan `f(long)` var: `long`.
- `f(2.5f)`: `f(float)` yok; `float` → `double` genişler: `double`. `float`, `int`'e veya `long`'a otomatik dönüşmez.

C ve E yanlıştır: Tam uyan metot olmaması derleme hatası değildir, genişleyen dönüşüm denenir.

Konu: Bölüm 6.

---

8 Cevap: D

`int[] b = a;` dizinin kopyasını değil, **aynı diziyi** gösteren ikinci bir referans oluşturur (*aliasing*). `b[0] = 10` ortak diziyi değiştirir. Sonra `a` **yeni** bir diziye yönlendirilir; `b` hâlâ eski diziyi (`{10, 2, 3}`) gösterir.

`a[0] + b[0]` iki `int` toplamıdır: `4 + 10 = 14`. Sonra `+ ""` ile String olur ve `b.length` (3) eklenir: `"143"`.

Konu: Bölüm 7.

---

9 Cevap: A

- `s1 == s2`: `"Ja" + "va"` iki **sabitin** birleşimidir; derleyici bunu derleme sırasında `"Java"` yapar ve havuzdaki (*string pool*) aynı nesneyi kullanır: `true`.
- `s1 == s3`: `new String(...)` her zaman **yeni** bir nesne oluşturur: `false`.
- `s1.equals(s3)`: İçerikler aynı: `true`.
- `s3 == s4`: İkisi aynı nesneyi gösterir: `true`.

Ders: String içeriğini karşılaştırmak için **her zaman** `equals` kullan; `==` sadece "aynı nesne mi?" sorusunu cevaplar.

Konu: Bölüm 8.

---

10 Cevap: C

String **değiştirilemez** (*immutable*): `toUpperCase()` ve `replace()` yeni String döndürür, ama sonuç bir değişkene atanmadığı için kaybolur; `s` hâlâ `"Bilgisayar"`'dır. A ve B bu tuzağa düşüyor.

`B0 i1 l2 g3 i4 s5 a6 y7 a8 r9` → `substring(3, 6)` 3, 4, 5. karakterleri alır: `"gis"`. `indexOf('a')` = 6, `lastIndexOf("a")` = 8. String ile başladığı için `+` birleştirme yapar: `"gis68"` (D şıkkındaki gibi `6 + 8` toplanmaz).

Çıktı
```
gis68
```

Konu: Bölüm 8.

11

İlk döngü: 3'ün katları (`3`, `6`, `9`) `continue` ile atlanır. `i = 8`'de `64 > 50` olduğu için `break` ile döngüden çıkılır.

i	Durum	`toplam`
1, 2	yazılır	3
3	`continue`	3
4, 5	yazılır	12
6	`continue`	12
7	`49 > 50` değil, yazılır	19
8	`64 > 50` → `break`	19

Desen: `i`. satırda `i` karakter vardır; `i + j` çiftse `*`, tekse `-`.

Son kısım: `n % 10` son basamağı alır, `n /= 10` siler; `ters` sayıyı ters çevirir: 1835. Ekrana `1835 + 1 = 1836` yazılır.

```
Çıktı
1 2 4 5 7 | toplam=19
*
-*
*-*
-*-*
1836
```

12

- `degistir(a, dizi)`: `x`, `a`'nın **kopyasıdır**; `x = 10` olur ama `a` 5 kalır. `d` ise `dizi` ile **aynı diziyi** gösterir: `d[0] = 1 + 10 = 11` ana dizi üzerinde yapılır.
- `d = new int[] {100, 200};` sadece metodun içindeki `d` referansını yeni diziye yönlendirir; `dizi` etkilenmez. `d[1] = 0` yeni diziyi değiştirir.
- Dönüş: `x + d[0]` = `10 + 100` = 110.
- `kaydir` diziyi bir adım **sola** döndürür: `[11, 2, 3]` → `[2, 3, 11]` → `[3, 11, 2]`.

```
Çıktı
5 110
[11, 2, 3]
[3, 11, 2]
```

13

Önce referansları izle:

- `satir` ile `m[1]` aynı diziyi gösterir: `satir[2] = 0` → `m[1]` = `{4, 5, 0}`.
- `m[2] = m[0]`: Artık `m[2]` ve `m[0]` **aynı** satırdır. Eski `{7, 8, 9}` satırına kimse erişemez.
- `m[2][0] = 99` → `m[0][0]` de 99 olur: `m[0]` = `m[2]` = `{99, 2, 3}`.

Satır toplamları: `104`, `9`, `104`. Köşegen: `m[0][0] + m[1][1] + m[2][2]` = `99 + 5 + 3` = 107. Son satır: `99 + 0 = 99` (tam sayı toplamı), sonra `""` ile String ve `m[2].length` = 3 → `"993"`.

```
Çıktı
104 9 104
107
993
```

14

`"Programlama"` → `P0 r1 o2 g3 r4 a5 m6 l7 a8 m9 a10`, uzunluk 11. `s.toUpperCase()` sonucu atanmadığı için `s` değişmez.

- `replace("am", "_")`: İki `am` de değişir: `"Progr_l_a"` (9 karakter).
- `substring(3, 7)` = `"gram"`, `charAt(10)` = `'a'` → `u = "grama"`. Sonra `indexOf("ram")` = 4 ve `lastIndexOf('a')` = 10 **metin olarak** eklenir: `"grama410"`.
- `StringBuilder` kendisi değişir: `"abc"` → `"abc12"` → `"xabc12"` → ters: `"21cbax"`, uzunluk 6.
- `a + "va"` çalışma zamanında yeni bir nesne üretir, `==` → `false`; `equals` → `true`. `compareTo` ilk farklı karakterlerin farkını verir: `'l' - 'y'` = 108 − 121 = −13.

Çıktı

```
11 Progr_l_a 9
grama410
21cbax 6
false true -13
```

15

Klasik **nöbetçi değerli** (*sentinel*) döngü: Önce bir sayı oku, sayı 0 olmadığı sürece işle ve bir sonrakini oku. İki tuzak var:

- En büyüğün başlangıç değeri `0` olursa, sayıların hepsi negatifken yanlış sonuç çıkar. `Integer.MIN_VALUE` ile başla (ya da ilk sayıyla).
- Çiftlik kontrolü `% 2 == 0` ile yapılmalı. `% 2 == 1` ile tek sayı kontrolü negatiflerde çalışmaz (`-5 % 2` $= -1$).

SayiAnaliz.java

```java
import java.util.Scanner;

public class SayiAnaliz {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int adet = 0, ciftAdet = 0, pozitifAdet = 0, pozitifToplam = 0;
        int enBuyuk = Integer.MIN_VALUE;

        System.out.print("Sayı (bitirmek için 0): ");
        int sayi = sc.nextInt();
        while (sayi != 0) {
            adet++;
            if (sayi > 0) {
                pozitifToplam += sayi;
                pozitifAdet++;
            }
            if (sayi % 2 == 0) {
                ciftAdet++;
            }
            if (sayi > enBuyuk) {
                enBuyuk = sayi;
            }
            System.out.print("Sayı (bitirmek için 0): ");
            sayi = sc.nextInt();
        }

        if (adet == 0) {
            System.out.println("Hiç sayı girilmedi.");
        } else {
            System.out.println("Adet: " + adet);
            if (pozitifAdet > 0) {
                double ort = (double) pozitifToplam / pozitifAdet;
                System.out.printf("Pozitif ortalaması: %.2f%n", ort);
            } else {
                System.out.println("Pozitif sayı yok.");
            }
            System.out.println("En büyük: " + enBuyuk);
            System.out.println("Çift sayı adedi: " + ciftAdet);
        }
    }
}
```

```
Çıktı
Sayı (bitirmek için 0): 12
Sayı (bitirmek için 0): -5
Sayı (bitirmek için 0): 7
Sayı (bitirmek için 0): 30
Sayı (bitirmek için 0): -8
Sayı (bitirmek için 0): 0
Adet: 5
Pozitif ortalaması: 16.33
En büyük: 30
Çift sayı adedi: 3
```

Ortalamada `(double)` cast'ini unutursan `49 / 3` tam sayı bölmesi olur ve `16.00` yazar.

16

Ters çevirme: Baştaki ve sondaki elemanı yer değiştir (*swap*), ortaya doğru ilerle. Döngü `d.length / 2` kere döner; sonuna kadar gidersen dizi iki kere ters çevrilip eski haline döner.

İkinci en büyük: İki değişken tut. Yeni bir en büyük gelirse eski en büyük ikinciye kayar. Eşit değerleri (`9`, `9`) ikinciye almamak için `x < birinci` şartı gerekir.

DiziIslemleri.java

```java
import java.util.Arrays;

public class DiziIslemleri {
    static void tersCevir(int[] d) {
        for (int i = 0; i < d.length / 2; i++) {
            int gecici = d[i];
            d[i] = d[d.length - 1 - i];
            d[d.length - 1 - i] = gecici;
        }
    }

    static int ikinciEnBuyuk(int[] d) {
        int birinci = Integer.MIN_VALUE, ikinci = Integer.MIN_VALUE;
        for (int x : d) {
            if (x > birinci) {
                ikinci = birinci;
                birinci = x;
            } else if (x < birinci && x > ikinci) {
                ikinci = x;
            }
        }
        return ikinci;
    }

    public static void main(String[] args) {
        int[] dizi = {7, 3, 9, 1, 9, 4};
        System.out.println("Önce : " + Arrays.toString(dizi));
        tersCevir(dizi);
        System.out.println("Sonra: " + Arrays.toString(dizi));
        System.out.println("İkinci en büyük: " + ikinciEnBuyuk(dizi));
    }
}
```

Çıktı
```
Önce : [7, 3, 9, 1, 9, 4]
Sonra: [4, 9, 1, 9, 3, 7]
İkinci en büyük: 7
```

`tersCevir`'in `void` olduğuna dikkat et: Dizi referansla geldiği için metodun yaptığı değişiklik `main`'deki diziye yansır (Bölüm 7).

17

- **Kelime sayısı:** `trim()` ile baştaki/sondaki boşlukları at, `split("\\s+")` ile bir veya daha fazla boşluktan böl. Cümle boşsa bölme yapma: boş metni bölmek 1 elemanlı dizi verir (Bölüm 8.10).
- **Sesli harf:** Bütün sesli harfleri (büyük ve küçük) bir String'e koy; her karakter için `indexOf` ile bu String'de var mı diye bak. `-1` değilse sesli harftir.
- **Palindrom:** Boşlukları sil, Türkçe kurallarla küçük harfe çevir (`I`/`İ` için `Locale` şart, Bölüm 8), `StringBuilder.reverse()` ile tersini al ve `equals` ile karşılaştır.

CumleAnaliz.java
```java
import java.util.Locale;
import java.util.Scanner;

public class CumleAnaliz {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Cümle: ");
        String cumle = sc.nextLine().trim();

        int kelime = 0;
        if (!cumle.isEmpty()) {
            kelime = cumle.split("\\s+").length;
        }
        System.out.println("Kelime sayısı: " + kelime);

        String sesliler = "aeıioöuüAEIİOÖUÜ";
        int sesli = 0;
        for (int i = 0; i < cumle.length(); i++) {
            if (sesliler.indexOf(cumle.charAt(i)) != -1) {
                sesli++;
            }
        }
        System.out.println("Sesli harf sayısı: " + sesli);

        Locale tr = Locale.forLanguageTag("tr-TR");
        String sade = cumle.replace(" ", "").toLowerCase(tr);
        String ters = new StringBuilder(sade).reverse().toString();
        if (sade.equals(ters)) {
            System.out.println("Palindrom: evet");
        } else {
            System.out.println("Palindrom: hayır");
        }
    }
}
```

Çıktı
```
Cümle: Ey Edip Adanada pide ye
Kelime sayısı: 5
Sesli harf sayısı: 10
Palindrom: evet
```

`sade == ters` yazsaydın, içerikler aynı olsa bile `false` alırdın: `reverse().toString()` yeni bir nesne üretir.

BÖLÜM 9

Sınıflar ve Nesneler

Bu Bölümde Öğreneceklerin

- Nesneye yönelik programlamanın (OOP) temel fikrini
- Sınıf (şablon) ile nesne (örnek) arasındaki farkı; `new` ve nokta operatörünü
- Referans değişkenlerin bellekte nasıl çalıştığını (stack/heap) ve iki referansın aynı nesneyi göstermesini
- Yapıcıları (constructor), yapıcı aşırı yüklemeyi ve `this(...)` ile zincirlemeyi
- `this` anahtar kelimesini ve **gölgeleme** (shadowing) tuzağını
- Kapsüllemeyi: `private` alanlar, getter/setter ve erişim belirteçleri
- `static` alan ve metotları, `toString`, `null` ve `NullPointerException`
- Nesne dizilerini, nesneleri `==` ile karşılaştırmayı ve UML sınıf diyagramını

9.1 Nesneye yönelik programlama fikri

Şimdiye kadar yazdığın programlarda veriler (değişkenler, diziler) ve bu veriler üzerinde çalışan metotlar birbirinden ayrıydı. Bir sınıftaki öğrencilerin adını, numarasını ve ortalamasını tutmak için büyük ihtimalle üç ayrı dizi kullanırdın:

```
String[] adlar       = {"Ali", "Ayşe", "Can"};
int[]    numaralar   = {101,   102,    103};
double[] ortalamalar = {72.5,  88.0,   64.0};
```

Bu yaklaşım çalışır ama kırılgandır: Bir diziyi sıralayıp diğerlerini unutursan Ali'nin notu Ayşe'ye geçer. Yeni bir bilgi (telefon, bölüm...) eklemek her yere yeni bir dizi eklemek demektir.

Nesneye yönelik programlama (Object-Oriented Programming, **OOP**), birbirine ait verileri ve o veriler üzerinde çalışan işlemleri **tek bir paket** içinde toplar. "Öğrenci" diye yeni bir tip tanımlarsın; her öğrenci kendi adını, numarasını ve ortalamasını içinde taşır, "bilgilerini yazdır" gibi işleri de kendisi yapar.

Gerçek dünyadaki her nesnenin iki yönü vardır:

Yön	Anlamı	Öğrenci örneği	Araba örneği
Durum (state)	Nesnenin sahip olduğu bilgiler	ad, numara, ortalama	marka, hız, yakıt
Davranış (behavior)	Nesnenin yapabildiği işler	ders seç, bilgilerini yazdır	hızlan, fren yap

Java'da durum **alanlarla** (field), davranış **metotlarla** (method) ifade edilir.

Sınavda Çıkar: OOP'nin dört temel ilkesi

"OOP'nin temel ilkelerini yazınız" klasik bir sorudur: **Kapsülleme** (encapsulation, bu bölüm), **Kalıtım** (inheritance, Bölüm 10), **Çok biçimlilik** (polymorphism, Bölüm 11) ve **Soyutlama** (abstraction, Bölüm 11). Cevabında İngilizce karşılıklarını da yaz.

9.2 Sınıf ve nesne

Sınıf (class), nesnelerin nasıl olacağını tarif eden bir **şablondur**: Hangi bilgileri tutacaklarını ve neler yapabileceklerini söyler. **Nesne** (object) ise bu şablondan üretilmiş somut bir **örnektir** (instance).

Sınıf (şablon)	Nesneler (örnekler)
Kurabiye kalıbı	Kalıptan çıkan her bir kurabiye
Ev planı (mimari çizim)	Plana göre yapılmış her bir ev
`Ogrenci` sınıfı	Ali, Ayşe, Can nesneleri

Kalıp tek, kurabiye çok. Kurabiyelerin şekli aynıdır ama biri çikolatalı, biri sade olabilir; yani her nesnenin **yapısı aynı, değerleri farklıdır**. İlk sınıfımızı yazalım:

SinifGiris.java

```
class Ogrenci {
    // alanlar (field): her öğrenci nesnesinin kendi kopyası olur
    String ad;
    int no;
    double ortalama;

    // metot: nesnenin davranışı
    void bilgiYazdir() {
        System.out.println(no + " - " + ad + " - " + ortalama);
    }
}

public class SinifGiris {
    public static void main(String[] args) {
        Ogrenci a = new Ogrenci();      // 1. nesne
        a.ad = "Ali";
        a.no = 101;
        a.ortalama = 72.5;

        Ogrenci b = new Ogrenci();      // 2. nesne
        b.ad = "Ayşe";
        b.no = 102;
        b.ortalama = 88.0;

        a.bilgiYazdir();
        b.bilgiYazdir();
    }
}
```

Çıktı

```
101 - Ali - 72.5
102 - Ayşe - 88.0
```

- `class Ogrenci { ... }` → Yeni bir **tip** tanımladık. Artık `int` veya `String` gibi `Ogrenci` tipinde de değişken tanımlayabiliriz.
- `new Ogrenci()` → Bellekte yeni bir `Ogrenci` **nesnesi oluşturur** (instantiation). Her `new`, yepyeni ve bağımsız bir nesne demektir.
- `a.ad = "Ali";` → **Nokta operatörü** (`.`) ile nesnenin alanına ulaşılır; `a.bilgiYazdir()` ile de nesnenin metodu çağrılır.

- `bilgiYazdir` metodu `ad`, `no` ve `ortalama` alanlarını doğrudan kullanır. `a.bilgiYazdir()` çağrıldığında Ali'nin, `b.bilgiYazdir()` çağrıldığında Ayşe'nin değerleri yazılır: Metot, **hangi nesne üzerinden çağrıldıysa** onun alanlarını kullanır.

Bölüm 6'daki metotların başında `static` vardı, `bilgiYazdir`'da yok. `static` olmayan bir metot mutlaka bir **nesne üzerinden** çağrılır (`a.bilgiYazdir()`). Aradaki farkı bu bölümün sonlarındaki `static` başlığında netleştireceğiz.

> **Not: Bu kitapta her örnek tek dosyadır**
>
> Kitaptaki her program tek bir `.java` dosyasıdır: `main` metodu, dosya adıyla aynı olan `public class` içindedir (burada `SinifGiris`); diğer sınıflar (`Ogrenci`) aynı dosyada başına `public` yazılmadan tanımlanır. Bir dosyada **en fazla bir** `public` sınıf olabilir. Derleyince her sınıf için ayrı bir `.class` dosyası oluşur: `SinifGiris.class` ve `Ogrenci.class`.
>
> Gerçek projelerde ise her sınıf **kendi dosyasında** yazılır ve genellikle `public` olur. VS Code'da:
>
> 1. Aynı klasörde `Ogrenci.java` dosyasını oluştur, içine `public class Ogrenci { ... }` yaz.
> 2. Yine aynı klasörde `Main.java` dosyasını oluştur ve `main` metodunu buraya yaz.
> 3. `Main.java`'daki **Run**'a tıkla. Java eklentisi klasördeki bütün sınıfları birlikte derler. `import` gerekmez; aynı klasördeki (aynı paketteki) sınıflar birbirini görür.
>
> Terminalden çalışıyorsan önce `javac *.java` ile hepsini derle, sonra `java Main` ile çalıştır.

9.3 Alanlar ve metotlar

Sınıfın içinde ama metotların dışında tanımlanan değişkenlere **alan** (field, attribute, *özellik*) denir. Bir nesnenin alanları onun **durumunu** oluşturur. Sınıfın metotları bu alanları okuyabilir ve **değiştirebilir**:

AlanMetot.java

```
class Araba {
    String marka;
    int hiz;                    // km/sa

    void hizlan(int miktar) {
        hiz = Math.min(hiz + miktar, 180);    // en fazla 180
    }

    boolean hizliMi() {
        return hiz > 90;
    }

    void durumYaz() {
        System.out.println(marka + ": " + hiz + " km/sa, hızlı mı? " + hizliMi());
    }
}

public class AlanMetot {
    public static void main(String[] args) {
        Araba a1 = new Araba();
        a1.marka = "Togg";
        Araba a2 = new Araba();
        a2.marka = "Fiat";

        a1.hizlan(70);
        a1.hizlan(150);
        a2.hizlan(40);
```

```
        a1.durumYaz();
        a2.durumYaz();
    }
}
```

Çıktı

```
Togg: 180 km/sa, hızlı mı? true
Fiat: 40 km/sa, hızlı mı? false
```

`a1` ile `a2`'nin hızları birbirinden bağımsızdır: `a1.hizlan(...)` sadece `a1`'in `hiz` alanını değiştirir (70 + 150 = 220 olurdu, ama `Math.min` 180'de sınırladı). Ayrıca `durumYaz` metodu aynı sınıftaki `hizliMi()` metodunu da doğrudan, nesne adı yazmadan çağırabiliyor.

	Alan (field)	Yerel değişken (local variable)
Nerede tanımlanır?	Sınıfın içinde, metotların dışında	Bir metodun (veya bloğun) içinde
Ömrü	Nesne yaşadığı sürece	Metot bitince yok olur
Kapsamı	Sınıfın bütün metotlarından erişilir	Sadece tanımlandığı blokta
Varsayılan değer	**Var** (0, false, null...)	**Yok**; değer verilmeden okunamaz

Alanların varsayılan değerleri

Bölüm 2'de, değer verilmemiş bir yerel değişkenin okunamadığını görmüştün. Alanlar farklıdır: `new` ile nesne oluşturulduğunda Java bütün alanlara **otomatik olarak varsayılan değer** verir (Bölüm 7'de dizi elemanlarında gördüğün kuralın aynısı):

VarsayilanDegerler.java

```
class Ornek {
    int tamSayi;
    double ondalik;
    boolean mantiksal;
    char karakter;
    String metin;
    int[] dizi;
}

public class VarsayilanDegerler {
    public static void main(String[] args) {
        Ornek o = new Ornek();
        System.out.println("int     : " + o.tamSayi);
        System.out.println("double  : " + o.ondalik);
        System.out.println("boolean : " + o.mantiksal);
        System.out.println("char    : kodu " + (int) o.karakter);
        System.out.println("String  : " + o.metin);
        System.out.println("int[]   : " + o.dizi);
    }
}
```

Çıktı

```
int     : 0
double  : 0.0
boolean : false
char    : kodu 0
String  : null
int[]   : null
```

Kural: Tam sayı tipleri `0`, `float`/`double` `0.0`, `boolean` `false`, `char` kodu 0 olan görünmez karakter (`'\u0000'`), **referans tipleri** (`String`, diziler, sınıflar) ise `null` alır. `null`, "bu değişken henüz hiçbir nesneyi göstermiyor" demektir. Ayrıntısını birazdan göreceğiz.

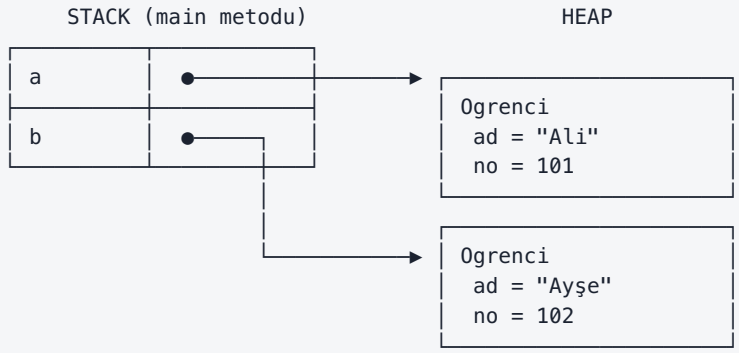
9.4 Referans değişkenler: stack ve heap

`int x = 5;` yazdığında `x` değişkeninin içinde doğrudan **5 değeri** durur. `Ogrenci a = new Ogrenci();` yazdığında ise `a`'nın içinde nesnenin kendisi değil, nesnenin bellekte **nerede olduğu bilgisi** (adresi) durur. Bu yüzden sınıf tipindeki değişkenlere **referans değişken** (reference variable) denir.

Java belleği iki bölge olarak kullanır:

- **Stack** (yığın): Metot çağrıları ve onların yerel değişkenleri burada tutulur. Metot bitince silinir.
- **Heap** (öbek): `new` ile oluşturulan **bütün nesneler** burada yaşar.

`SinifGiris` programının sonunda bellek kabaca şöyledir:



İki referans, tek nesne

Bir referans değişkeni diğerine atadığında **nesne kopyalanmaz**, sadece adres kopyalanır. Artık iki değişken **aynı nesneyi** gösterir (*aliasing*). İlkel tiplerle farkı görelim:

Referans.java

```
class Kutu {
    int deger;
}

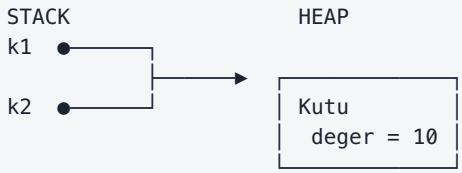
public class Referans {
    public static void main(String[] args) {
        int x = 5;
        int y = x;          // değerin kopyası
        y = 10;
        System.out.println("x = " + x + ", y = " + y);

        Kutu k1 = new Kutu();
        k1.deger = 5;
        Kutu k2 = k1;       // adresin kopyası: AYNI nesne!
        k2.deger = 10;
        System.out.println("k1.deger = " + k1.deger + ", k2.deger = " + k2.deger);

        k2 = new Kutu();    // k2 artık başka bir nesneyi gösteriyor
        k2.deger = 99;
        System.out.println("k1.deger = " + k1.deger + ", k2.deger = " + k2.deger);
    }
}
```

```
Çıktı
x = 5, y = 10
k1.deger = 10, k2.deger = 10
k1.deger = 10, k2.deger = 99
```

`Kutu k2 = k1;` satırından sonra bellekte tek bir `Kutu` nesnesi vardır ve iki referans da onu gösterir:



`k2.deger = 10;` nesnenin içini değiştirdiği için `k1.deger` da 10 görünür. Ama `k2 = new Kutu();` satırı nesneyi değil, **`k2`'nin gösterdiği yeri** değiştirir; `k1` eski nesneyi göstermeye devam eder.

Hiçbir referansın göstermediği bir nesneye artık ulaşılamaz. Java'nın **çöp toplayıcısı** (garbage collector) bu nesneleri bir süre sonra bellekten kendiliğinden siler. Java'da nesneyi elle silen bir komut yoktur.

Nesneyi metoda göndermek

Java'da parametreler **her zaman değerle** (pass by value) aktarılır: Metoda değişkenin bir **kopyası** gider. Nesnelerde kopyalanan şey **referanstır**; yani metot, çağıranla **aynı nesneyi** görür:

MetodaNesne.java

```
class Kutu {
    int deger;
}

public class MetodaNesne {
    static void artir(int sayi) {
        sayi++;                 // sadece kopya değişir
    }

    static void artir(Kutu k) {
        k.deger++;              // ortak nesnenin İÇİ değişir
    }

    static void yenile(Kutu k) {
        k = new Kutu();         // sadece yerel k başka nesneye döner
        k.deger = 100;
    }

    public static void main(String[] args) {
        int s = 1;
        Kutu kutu = new Kutu();
        kutu.deger = 1;

        artir(s);
        artir(kutu);
        System.out.println(s + " " + kutu.deger);

        yenile(kutu);
        System.out.println(kutu.deger);
    }
}
```

Çıktı
```
1 2
2
```

Sınavda Çıkar: "Java'da nesneler referansla mı aktarılır?"

Hayır; Java'da **her şey değerle** aktarılır. Nesnelerde kopyalanan değer referanstır. Sonuç: Metot nesnenin **içini değiştirebilir** (`k.deger++` kalıcıdır), ama parametreye yeni bir nesne atayarak çağıranın değişkenini **başka nesneye yönlendiremez** (`yenile` hiçbir şey değiştirmedi). Bölüm 7'de dizi parametrelerinde gördüğün davranışın aynısıdır.

9.5 Yapıcılar (constructor)

Nesneyi oluşturduktan sonra alanları tek tek doldurmak hem uzun hem de tehlikelidir: Birini unutursan alan `null` veya `0` kalır. **Yapıcı** (constructor, *kurucu*), `new` ile nesne oluşturulurken **otomatik çalışan** ve nesneye ilk değerlerini veren özel bir metottur. Kuralları:

1. Adı, sınıfın adıyla **birebir aynıdır**.
2. **Dönüş tipi yoktur**; `void` bile yazılmaz.
3. Normal metot gibi çağrılmaz; `new` ile nesne oluşturulurken kendiliğinden çalışır.

Parametreli yapıcı

YapiciOrnek.java
```
class Ogrenci {
    String ad;
    int no;
    double ortalama;

    Ogrenci(String a, int n) {
        ad = a;
        no = n;
        ortalama = 0.0;
        System.out.println(ad + " oluşturuldu.");
    }

    void bilgiYazdir() {
        System.out.println(no + " - " + ad + " - " + ortalama);
    }
}

public class YapiciOrnek {
    public static void main(String[] args) {
        Ogrenci a = new Ogrenci("Ali", 101);
        Ogrenci b = new Ogrenci("Ayşe", 102);
        b.ortalama = 88.0;
        a.bilgiYazdir();
        b.bilgiYazdir();
    }
}
```

```
Çıktı
Ali oluşturuldu.
Ayşe oluşturuldu.
101 - Ali - 0.0
102 - Ayşe - 88.0
```

`new Ogrenci("Ali", 101)` ifadesi önce bellekte nesneyi ayırır, sonra yapıcıyı `a = "Ali"`, `n = 101` ile çalıştırır. Parametre adlarını `a` ve `n` seçtik, çünkü `ad` ve `no` seçseydik bir tuzağa düşecektik (`this` başlığında göreceksin).

Varsayılan yapıcı ve kaybolması

İlk örneklerde hiç yapıcı yazmadığımız halde `new Ogrenci()` çalışıyordu. Çünkü sınıfta **hiç yapıcı yoksa** derleyici kendisi parametresiz, boş bir yapıcı ekler: `Ogrenci() { }`. Buna **varsayılan yapıcı** (default constructor) denir.

Dikkat: Derleyici bunu **sadece hiç yapıcı yazmadıysan** yapar. Parametreli bir yapıcı yazdığın anda varsayılan yapıcı ortadan kalkar:

VarsayilanKayip.java

```java
class Ogrenci {
    String ad;

    Ogrenci(String a) {
        ad = a;
    }
}

public class VarsayilanKayip {
    public static void main(String[] args) {
        Ogrenci x = new Ogrenci("Ali");     // tamam
        Ogrenci y = new Ogrenci();          // HATA
    }
}
```

```
Derleyici çıktısı – program derlenmez
VarsayilanKayip.java:12: error: constructor Ogrenci in class Ogrenci cannot be applied to given
types;
        Ogrenci y = new Ogrenci();          // HATA
                    ^
  required: String
  found:    no arguments
  reason: actual and formal argument lists differ in length
1 error
```

Hem `new Ogrenci()` hem `new Ogrenci("Ali")` kullanmak istiyorsan parametresiz yapıcıyı da **kendin yazmalısın**.

Sık Yapılan Hata: Yapıcıya void yazmak

`void Kitap() { ... }` yazarsan bu artık yapıcı değil, adı sınıfla aynı olan **sıradan bir metottur**. Derleyici hata vermez, ama `new` sırasında bu kod hiç çalışmaz:

```
class Kitap {
    String ad = "bilinmiyor";   // alana ilk değer verilebilir

    void Kitap() {              // void var: bu bir yapıcı DEĞİL!
        ad = "Sefiller";
    }
}

public class VoidYapici {
    public static void main(String[] args) {
        Kitap k = new Kitap();  // derleyicinin eklediği boş yapıcı çalışır
        System.out.println(k.ad);
    }
}
```

Çıktı

```
bilinmiyor
```

Yapıcıları aşırı yükleme ve this(...) ile zincirleme

Bölüm 6'da metotları **aşırı yüklemeyi** (overloading) görmüştün: aynı ad, farklı parametre listesi. Yapıcılar da aşırı yüklenebilir; böylece nesne farklı bilgilerle oluşturulabilir. Derleyici, `new` ile verilen argümanların sayısına ve tiplerine bakarak hangi yapıcının çalışacağına karar verir.

Birden fazla yapıcı genellikle aynı işi tekrar eder (alanlara değer vermek). Kod tekrarını önlemek için bir yapıcı, `this(...)` yazarak **aynı sınıftaki başka bir yapıcıyı** çağırabilir. Aşağıdaki `Saat` sınıfında üç aşırı yüklenmiş yapıcı var ve asıl işi sadece biri yapıyor:

Zincirleme.java

```
class Saat {
    int saat, dakika, saniye;

    Saat(int s, int d, int sn) {
        saat = s;
        dakika = d;
        saniye = sn;
        System.out.println("3 parametreli yapıcı");
    }

    Saat(int s, int d) {
        this(s, d, 0);              // 3 parametreli yapıcıyı çağır
        System.out.println("2 parametreli yapıcı");
    }

    Saat() {
        this(0, 0);                 // 2 parametreli yapıcıyı çağır
        System.out.println("parametresiz yapıcı");
    }

    void yaz() {
        System.out.printf("%02d:%02d:%02d%n", saat, dakika, saniye);
    }
}
```

```java
public class Zincirleme {
    public static void main(String[] args) {
        Saat s1 = new Saat();
        s1.yaz();
        Saat s2 = new Saat(14, 30);
        s2.yaz();
    }
}
```

Çıktı

```
3 parametreli yapıcı
2 parametreli yapıcı
parametresiz yapıcı
00:00:00
3 parametreli yapıcı
2 parametreli yapıcı
14:30:00
```

`new Saat()` parametresiz yapıcıyı başlatır, ama onun ilk işi `this(0, 0)` olduğu için kontrol 2 parametreliye, oradan da 3 parametreliye geçer. **En içteki yapıcı önce biter**, sonra kontrol geri dönerek kalan satırlar çalışır. Bu yüzden mesajlar "3 → 2 → parametresiz" sırasıyla yazıldı.

Klasik kural: `this(...)` çağrısı yapıcının **ilk komutu** olmalıdır ve bir yapıcıda en fazla bir kez kullanılabilir. (Java 25 bu kuralı biraz esnetti; Bölüm 10'daki Java 25 notunda göreceksin. Derslerde ve sınavlarda genellikle klasik kural sorulur.)

9.6 this anahtar kelimesi

`this`, "**şu an üzerinde çalışılan nesne**" demektir. `a.bilgiYazdir()` çağrısında metodun içindeki `this`, `a`'nın gösterdiği nesnedir; `b.bilgiYazdir()` çağrısında ise `b`'nin.

En sık kullanıldığı yer, parametre adının alan adıyla aynı olduğu durumlardır. Bir metodun içinde aynı adlı bir yerel değişken veya parametre varsa, o ad **alanı gizler**; buna **gölgeleme** (shadowing) denir:

Golgeleme.java

```java
class HataliOgrenci {
    String ad;
    int no;

    HataliOgrenci(String ad, int no) {
        ad = ad;            // parametre kendine atanıyor!
        no = no;            // alan hiç değişmiyor
    }
}

class Ogrenci {
    String ad;
    int no;

    Ogrenci(String ad, int no) {
        this.ad = ad;       // alan = parametre
        this.no = no;
    }
}

public class Golgeleme {
    public static void main(String[] args) {
        HataliOgrenci h = new HataliOgrenci("Ali", 101);
        Ogrenci o = new Ogrenci("Ali", 101);
```

```
        System.out.println(h.ad + " " + h.no);
        System.out.println(o.ad + " " + o.no);
    }
}
```

Çıktı

```
null 0
Ali 101
```

`HataliOgrenci` yapıcısının içinde `ad` denildiğinde Java en yakındaki tanıma, yani **parametreye** bakar. `ad = ad;` parametreyi kendisine atar; alan varsayılan değerinde (`null`, `0`) kalır. `Ogrenci` sınıfında ise `this.ad` yazarak alanı kastettiğimizi açıkça söyledik.

> **Sık Yapılan Hata: Gölgeleme tuzağı**
>
> `ad = ad;` satırı **derleme hatası vermez**, program çalışır ama alan boş kalır. Bu yüzden fark etmesi zordur. Parametreyle alan aynı adı taşıyorsa her zaman `this.ad = ad;` yaz. Çıktı sorularında `null 0` gibi bir sonuç görürsen ilk bakacağın yer burasıdır.

Kullanım	Anlamı	Örnek
`this.alan`	Nesnenin alanı (gölgelemeyi çözer)	`this.ad = ad;`
`this.metot()`	Nesnenin kendi metodu (`this.` genelde yazılmaz)	`this.hizliMi()`
`this(...)`	Aynı sınıfın başka bir yapıcısını çağırır	`this(0, 0);`
`this`	Nesnenin kendisi (parametre olarak verilebilir, döndürülebilir)	`return this;`

9.7 Kapsülleme (encapsulation)

Şu ana kadar alanlara dışarıdan doğrudan eriştik: `a.ortalama = 72.5;`. Peki biri `a.ortalama = 250;` veya `hesap.bakiye = -5000;` yazarsa? Nesne **geçersiz** bir duruma düşer ve sınıf bunu engelleyemez.

Kapsülleme (encapsulation), alanları dış dünyadan gizleyip (`private`) onlara sadece sınıfın izin verdiği **metotlar** üzerinden erişilmesini sağlamaktır. Bankamatik gibi düşün: Kasayı açıp parayı kendin alamazsın, ama düğmeler (metotlar) aracılığıyla kurallara uygun işlem yapabilirsin.

- Alanın değerini okuyan metoda **getter** (alıcı), değiştiren metoda **setter** (ayarlayıcı) denir.
- Setter'ın içinde **doğrulama** (validation) yapılır; geçersiz değerler alana hiç yazılmaz.

Kapsulleme.java

```
class Ogrenci {
    private String ad;
    private double ortalama;

    public Ogrenci(String ad) {
        this.ad = ad;
    }

    public String getAd() {
        return ad;
    }

    public double getOrtalama() {
        return ortalama;
    }
```

```
    public void setOrtalama(double ortalama) {
        if (ortalama < 0 || ortalama > 100) {
            System.out.println("Geçersiz ortalama: " + ortalama);
            return;                 // alanı değiştirmeden çık
        }
        this.ortalama = ortalama;
    }
}

public class Kapsulleme {
    public static void main(String[] args) {
        Ogrenci o = new Ogrenci("Elif");
        o.setOrtalama(78.5);
        o.setOrtalama(250);
        o.setOrtalama(-3);
        System.out.println(o.getAd() + ": " + o.getOrtalama());
    }
}
```

Çıktı

```
Geçersiz ortalama: 250.0
Geçersiz ortalama: -3.0
Elif: 78.5
```

`ad` alanının sadece getter'ı var, setter'ı yok: Ad dışarıdan **okunabilir ama değiştirilemez** (salt okunur). `private` bir alana sınıfın dışından doğrudan erişmeye çalışırsan program derlenmez:

PrivateErisim.java

```
class Hesap {
    private double bakiye;
}

public class PrivateErisim {
    public static void main(String[] args) {
        Hesap h = new Hesap();
        h.bakiye = -5000;
    }
}
```

Derleyici çıktısı – program derlenmez

```
PrivateErisim.java:8: error: bakiye has private access in Hesap
        h.bakiye = -5000;
         ^
1 error
```

`Hesap` ve `PrivateErisim` aynı dosyada olmasına rağmen erişim engellendi: `private`, dosyaya değil **sınıfa** özeldir.

İsimlendirme geleneği: `ad` alanı için getter `getAd()`, setter `setAd(String ad)`; `boolean` alanlarda getter `is` ile başlar: `isAktif()`.

> **İpucu: Getter/setter'ları VS Code'a yazdır**
>
> Alanları yazdıktan sonra sınıfın içinde sağ tıkla → **Source Action...** → **Generate Getters and Setters**. Hangi alanlar için üretileceğini seçersin, metotlar hazır gelir. Doğrulama kodunu setter'a sonra kendin eklersin.

> **Sınavda Çıkar: "Kapsülleme nedir, faydaları nelerdir?"**
>
> Tanım: Alanları `private` yapıp erişimi `public` getter/setter metotlarıyla **kontrol altına almak** (veri gizleme, *data hiding*). Faydaları: (1) Geçersiz değerler setter'daki doğrulamayla engellenir. (2) Sadece getter yazarak salt okunur alan yapılabilir. (3) Sınıfın iç yapısı değişse bile onu kullanan kodlar etkilenmez.

9.8 Erişim belirteçleri

Bir sınıf üyesine (alan, metot, yapıcı) kimlerin erişebileceğini **erişim belirteçleri** (access modifiers) belirler:

Belirteç	Aynı sınıf	Aynı paket	Alt sınıf (başka paket)	Her yer
`public`	Evet	Evet	Evet	Evet
`protected`	Evet	Evet	Evet	Hayır
(hiçbiri yazılmaz) paket erişimi	Evet	Evet	Hayır	Hayır
`private`	Evet	Hayır	Hayır	Hayır

- **Paket** (package), birbiriyle ilgili sınıfları gruplayan bir klasördür; dosyanın en başına `package okul;` gibi bir satır yazılarak belirtilir. Bu kitaptaki örneklerde `package` satırı yok; hepsi isimsiz **varsayılan pakettedir**.
- Belirteç yazılmazsa üye **paket erişimlidir** (default, *package-private*): Aynı paketteki her sınıf erişebilir.
- `protected`'ın asıl anlamı kalıtımla ortaya çıkar (Bölüm 10).

Genel kural: **Alanlar `private`**, dışarıya sunulan metotlar `public`, sadece sınıfın kendi içinde kullanılan yardımcı metotlar `private`.

> **Sınavda Çıkar: Erişim sırası**
>
> En açıktan en kapalıya: **`public` > `protected` > paket (default) > `private`**. Klasik tuzak: "`protected` sadece alt sınıflardan erişilebilir" cümlesi **yanlıştır**; `protected` üyeye **aynı paketteki** bütün sınıflar da erişebilir. Yani `protected` = paket erişimi + başka paketteki alt sınıflar.

9.9 static alanlar ve metotlar

Normal bir alanın **her nesnede ayrı bir kopyası** vardır; bunlara **örnek alanı** (instance field) denir. `static` anahtar kelimesiyle tanımlanan alan ise **sınıfa aittir**: Kaç nesne oluşturulursa oluşturulsun tek bir kopyası vardır ve bütün nesneler onu **paylaşır**. Klasik örnek, oluşturulan nesneleri sayan bir sayaçtır:

StaticSayac.java

```
class Ogrenci {
    static int sayac = 0;       // sınıfa ait: tek kopya
    private String ad;          // nesneye ait: her nesnede ayrı
    private int no;

    Ogrenci(String ad) {
        this.ad = ad;
        sayac++;
        no = 1000 + sayac;      // her öğrenciye sıradaki numara
    }

    void yaz() {
        System.out.println(no + " " + ad);
    }
```

```
    static int getSayac() {
        return sayac;
    }
}

public class StaticSayac {
    public static void main(String[] args) {
        System.out.println("Başlangıç: " + Ogrenci.getSayac());
        Ogrenci a = new Ogrenci("Ali");
        Ogrenci b = new Ogrenci("Berk");
        Ogrenci c = new Ogrenci("Cem");
        a.yaz();
        b.yaz();
        c.yaz();
        System.out.println("Toplam öğrenci: " + Ogrenci.sayac);
    }
}
```

Çıktı

```
Başlangıç: 0
1001 Ali
1002 Berk
1003 Cem
Toplam öğrenci: 3
```

```
Ogrenci sınıfı :  sayac = 3              ← tek kopya, herkes paylaşır
a nesnesi      :  ad = "Ali",  no = 1001
b nesnesi      :  ad = "Berk", no = 1002
c nesnesi      :  ad = "Cem",  no = 1003
```

`static` üyelere **sınıf adıyla** erişilir: `Ogrenci.sayac`, `Ogrenci.getSayac()`. Hiç nesne oluşturmadan da kullanılabilirler; programın ilk satırında henüz öğrenci yokken `getSayac()` çağrılabildi. Tanıdık örnekler: `Math.sqrt(2)`, `Math.PI`, `Integer.parseInt("5")` ve tabii ki `main`.

static metottan örnek üyelere erişilemez

`static` bir metot belirli bir nesne üzerinden çağrılmaz; bu yüzden içinde `this` yoktur ve **hangi nesnenin** alanını kullanacağını bilemez:

StaticHata.java

```
public class StaticHata {
    int sayi = 5;

    void selamVer() {
        System.out.println("Merhaba");
    }

    public static void main(String[] args) {
        System.out.println(sayi);
        selamVer();
    }
}
```

Derleyici çıktısı – program derlenmez

```
StaticHata.java:9: error: non-static variable sayi cannot be referenced from a static context
        System.out.println(sayi);
                           ^
StaticHata.java:10: error: non-static method selamVer() cannot be referenced from a static context
        selamVer();
        ^
2 errors
```

Çözüm iki yoldan biridir: Ya önce bir nesne oluşturup üyeye onun üzerinden erişirsin (`StaticHata s = new StaticHata(); s.selamVer();`), ya da üyeyi `static` yaparsın. Bölüm 6'da `main`'den çağırdığımız bütün metotlara `static` yazmamızın sebebi buydu.

	Örnek (instance) üye	Sınıf (`static`) üye
Kime ait?	Her nesneye ayrı	Sınıfa; tek kopya
Nasıl erişilir?	`nesne.uye`	`SinifAdi.uye`
Ne zaman oluşur?	`new` ile nesne oluşunca	Sınıf ilk kullanıldığında
`this` kullanılabilir mi?	Evet	Hayır
Neye doğrudan erişebilir?	Hem örnek hem `static` üyelere	Sadece `static` üyelere
Örnek	`o.getAd()`, `metin.length()`	`Math.sqrt(2)`, `Ogrenci.sayac`

Bütün nesneler için aynı olan sabitler genellikle `static final` tanımlanır: `static final double KDV_ORANI = 0.20;`. `Math.PI` de böyle bir sabittir.

> **Sınavda Çıkar: static sorularının anahtarı**
>
> Çıktı sorularında `static` bir alanı `s1.sayac` gibi **nesne üzerinden** yazarlar ki kafan karışsın. Bu yazım geçerlidir ama hangi nesneyle yazılırsa yazılsın **aynı tek değişken** kastedilir. Bir nesne üzerinden yapılan değişiklik hepsinde görünür. Değişken tablosunda `static` alanı nesnelerin dışında, tek bir sütunda takip et.

9.10 toString metodu

Bir nesneyi doğrudan `println` ile yazdırırsan anlamsız bir çıktı alırsın. Sınıfa `toString` metodu yazarsan Java nesneyi metne çevirmesi gerektiğinde **otomatik olarak** onu kullanır:

ToStringOrnek.java

```
class NoktaYalin {
    int x, y;

    NoktaYalin(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

class Nokta {
    int x, y;

    Nokta(int x, int y) {
        this.x = x;
        this.y = y;
    }
```

```
    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}

public class ToStringOrnek {
    public static void main(String[] args) {
        NoktaYalin p = new NoktaYalin(3, 4);
        Nokta q = new Nokta(3, 4);
        System.out.println(p);
        System.out.println(q);
        System.out.println("Konum: " + q);
        String s = q.toString();
        System.out.println(s.length());
    }
}
```

Çıktı

```
NoktaYalin@18b4aac2
(3, 4)
Konum: (3, 4)
6
```

- `toString` yazılmamışsa çıktı `SinifAdi@sayı` biçimindedir. `@`'ten sonraki onaltılık (hexadecimal) sayı nesneye özgüdür; **senin bilgisayarında farklı çıkabilir**.
- `println(q)` ve `"Konum: " + q` ifadeleri, arka planda `q.toString()`'i çağırır.
- Metodun imzası tam olarak `public String toString()` olmalıdır. Üstündeki `@Override` satırı derleyiciye "bu metodu bilerek yeniden tanımlıyorum" der; neden gerektiğini Bölüm 10'da göreceksin.

9.11 null ve NullPointerException

`null`, bir referans değişkenin **hiçbir nesneyi göstermediği** anlamına gelir. `null` bir referans üzerinden alan okumaya veya metot çağırmaya çalışırsan program `NullPointerException` (kısaca NPE) ile çöker:

NullHata.java

```
class Ogrenci {
    String ad;

    Ogrenci(String ad) {
        this.ad = ad;
    }
}

public class NullHata {
    public static void main(String[] args) {
        Ogrenci o = new Ogrenci("Ali");
        System.out.println(o.ad.length());
        o = null;               // artık hiçbir nesneyi göstermiyor
        System.out.println(o.ad.length());
    }
}
```

Çıktı – program hata ile durur

```
3
Exception in thread "main" java.lang.NullPointerException: Cannot read field "ad" because "o" is
null
        at NullHata.main(NullHata.java:14)
```

Java 15'ten beri NPE mesajları **yardımcıdır** (helpful NullPointerException): Hangi işlemin yapılamadığını (`Cannot read field "ad"`) ve **hangi referansın** null olduğunu (`because "o" is null`) açıkça söyler. Mesajı okumak hatayı bulmanın yarısıdır.

`null` olabilecek bir referansı kullanmadan önce kontrol et: `if (o != null) { ... }`. `o == null` ve `o != null` karşılaştırmaları her zaman güvenlidir; tehlikeli olan `null` üzerinden **nokta** ile bir şeye erişmektir.

9.12 Nesne dizileri

Bölüm 7'de `int` dizilerini gördün. Aynı şekilde nesne dizisi de oluşturabilirsin. Ama çok önemli bir fark var: `new Ogrenci[4]` **dört öğrenci oluşturmaz**; sadece dört tane **boş referans** (`null`) içeren bir dizi oluşturur. Her elemana nesneyi ayrıca `new` ile koyman gerekir:

NesneDizisi.java

```
class Ogrenci {
    private String ad;
    private double ortalama;

    Ogrenci(String ad, double ortalama) {
        this.ad = ad;
        this.ortalama = ortalama;
    }

    String getAd() { return ad; }
    double getOrtalama() { return ortalama; }
}

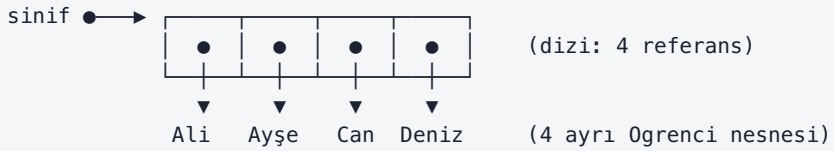
public class NesneDizisi {
    public static void main(String[] args) {
        Ogrenci[] sinif = new Ogrenci[4];       // 4 tane null
        sinif[0] = new Ogrenci("Ali", 72.5);
        sinif[1] = new Ogrenci("Ayşe", 88.0);
        sinif[2] = new Ogrenci("Can", 64.0);
        sinif[3] = new Ogrenci("Deniz", 91.5);

        double toplam = 0;
        Ogrenci enIyi = sinif[0];
        for (Ogrenci o : sinif) {
            toplam += o.getOrtalama();
            if (o.getOrtalama() > enIyi.getOrtalama()) {
                enIyi = o;
            }
        }
        System.out.printf("Sınıf ortalaması: %.2f%n", toplam / sinif.length);
        System.out.println("En başarılı: " + enIyi.getAd());
    }
}
```

Çıktı

```
Sınıf ortalaması: 79.00
En başarılı: Deniz
```

Kısa yoldan, dizi oluşturulurken de doldurabilirsin: `Ogrenci[] sinif = { new Ogrenci("Ali", 72.5), new Ogrenci("Can", 64.0) };`



Sık Yapılan Hata: Dizinin elemanlarını oluşturmayı unutmak

Diziyi oluşturup nesneleri oluşturmayı unutursan elemanlar `null` kalır ve ilk erişimde NPE alırsın:

```
class Ogrenci {
    String ad;
}

public class DiziNull {
    public static void main(String[] args) {
        Ogrenci[] sinif = new Ogrenci[3];
        System.out.println(sinif[0]);
        sinif[0].ad = "Ali";
    }
}
```

Çıktı – program hata ile durur

```
null
Exception in thread "main" java.lang.NullPointerException: Cannot assign field "ad" because
"sinif[0]" is null
        at DiziNull.main(DiziNull.java:9)
```

Mesajdaki `"sinif[0]" is null` ifadesi sorunu doğrudan gösteriyor.

9.13 Nesneleri == ile karşılaştırmak

Referans değişkenlerde `==` operatörü nesnelerin **içeriğini değil, adreslerini** karşılaştırır: "İkisi **aynı nesneyi** mi gösteriyor?" Bölüm 8'de String'lerde gördüğün durumun aynısı:

NesneKarsilastirma.java

```
class Nokta {
    int x, y;

    Nokta(int x, int y) {
        this.x = x;
        this.y = y;
    }

    boolean ayniMi(Nokta diger) {
        return x == diger.x && y == diger.y;
    }
}

public class NesneKarsilastirma {
    public static void main(String[] args) {
        Nokta a = new Nokta(3, 4);
        Nokta b = new Nokta(3, 4);
        Nokta c = a;
        System.out.println(a == b);
```

```
        System.out.println(a == c);
        System.out.println(a.equals(b));
        System.out.println(a.ayniMi(b));
    }
}
```

Çıktı

```
false
true
false
true
```

- `a == b` → `false`: İçerikleri aynı ama iki **ayrı** nesne.
- `a == c` → `true`: İkisi de **aynı** nesneyi gösteriyor.
- `a.equals(b)` → `false`! Kendi sınıfımızda `equals` yazmadığımız için Java'nın hazır `equals`'ı kullanıldı ve o da `==` gibi davranır. İçerik karşılaştıran doğru `equals` yazmayı Bölüm 10'da öğreneceksin; şimdilik `ayniMi` gibi kendi metodumuzla karşılaştırıyoruz.

9.14 UML sınıf diyagramı

UML (Unified Modeling Language), yazılım tasarımını çizmek için kullanılan standart bir gösterimdir. Hocalar sınıf tasarımını genellikle UML ile verir ve "bu diyagrama göre sınıfı yazın" diye sorar. Bir sınıf, üç bölmeli bir kutuyla gösterilir: **ad**, **alanlar**, **metotlar**.

```
┌──────────────────────────────────────┐
│              Ogrenci                 │   ← sınıf adı
├──────────────────────────────────────┤
│ - ad : String                        │
│ - no : int                           │   ← alanlar
│ - ortalama : double                  │
│ - sayac : int            {static}    │
├──────────────────────────────────────┤
│ + Ogrenci(ad : String, no : int)     │
│ + getAd() : String                   │
│ + getOrtalama() : double             │   ← metotlar
│ + setOrtalama(o : double) : void     │
│ + getSayac() : int       {static}    │
│ + toString() : String                │
└──────────────────────────────────────┘
```

UML işareti	Java karşılığı
`+`	`public`
`-`	`private`
`#`	`protected`
`~`	paket erişimi (belirteç yok)
Altı çizili veya `{static}`	`static`

UML'de tip, adın **sonuna** iki noktayla yazılır; Java'da ise **önüne**. Yapıcının dönüş tipi yoktur.

Sınavda Çıkar: UML'den kod yazma

Satır satır çevir: `- no : int` → `private int no;` · `+ getAd() : String` → `public String getAd()` · `+ setOrtalama(o : double) : void` → `public void setOrtalama(double o)`. Önce alanları, sonra yapıcıyı, en son metotları yaz. Diyagramda setter yoksa setter ekleme; hocalar fazlalığı da hata sayabilir.

Bölüm Özeti – Bilmen Gerekenler

- **Sınıf** şablondur, **nesne** o şablondan `new` ile üretilen örnektir. Nesnenin durumu **alanlarda**, davranışı **metotlarda** tutulur. Üyelere **nokta** operatörüyle erişilir.
- Alanlar otomatik varsayılan değer alır (`0`, `0.0`, `false`, `null`); yerel değişkenler almaz.
- Sınıf tipindeki değişken nesnenin kendisini değil **referansını** tutar. Nesneler **heap**'te, yerel değişkenler **stack**'te yaşar. `b = a;` nesneyi kopyalamaz; iki referans **aynı nesneyi** gösterir.
- Java'da parametreler **değerle** aktarılır; nesnede kopyalanan referanstır. Metot nesnenin içini değiştirebilir ama çağıranın değişkenini başka nesneye yönlendiremez.
- **Yapıcı**: sınıfla aynı ad, dönüş tipi yok (`void` yazılırsa sıradan metot olur). Hiç yapıcı yoksa derleyici parametresiz bir **varsayılan yapıcı** ekler; parametreli yapıcı yazınca bu kaybolur.
- Yapıcılar aşırı yüklenebilir; `this(...)` aynı sınıfın başka yapıcısını çağırır (klasik kural: ilk komut).
- **Gölgeleme**: `ad = ad;` alanı değiştirmez. Doğrusu `this.ad = ad;`.
- **Kapsülleme**: alanlar `private`, erişim `public` getter/setter ile; setter'da doğrulama yapılır.
- Erişim: `public` > `protected` > paket (belirteç yok) > `private`. `private` sınıfa özeldir.
- `static` üye sınıfa aittir, tek kopyadır, `SinifAdi.uye` ile erişilir. `static` metot örnek üyelere doğrudan erişemez (*non-static ... cannot be referenced from a static context*).
- `public String toString()` yazılırsa `println(nesne)` ve `"..." + nesne` onu kullanır.
- `null` referans üzerinden erişim → `NullPointerException`. `new Ogrenci[3]` **sıfır** öğrenci nesnesi oluşturur.
- `==` nesnelerde adres karşılaştırır; içerik için `equals` gerekir (Bölüm 10).
- UML: `+` public, `-` private, `#` protected, `~` paket; tip adın sonunda: `- no : int`.

Sorular

A) Çoktan Seçmeli

1 Yapıcılar (constructor) ile ilgili aşağıdakilerden hangisi **yanlıştır**?

(A) Yapıcının adı sınıfın adıyla aynıdır.

(B) Bir sınıfta birden fazla yapıcı olabilir.

(C) Sınıfta hiç yapıcı yazılmazsa derleyici parametresiz bir yapıcı ekler.

(D) Yapıcının dönüş tipi `void` olarak yazılmalıdır.

(E) Yapıcı, `new` ile nesne oluşturulurken otomatik olarak çalışır.

2 Aşağıdaki programda numaralandırılmış satırlardan hangisi **derleme hatasına** yol açar?

```
public class Test2 {
    int a = 1;
    static int b = 2;

    void f() { }

    static void g() { }

    public static void main(String[] args) {
        System.out.println(b);      // satır 1
        g();                        // satır 2
        f();                        // satır 3
        Test2 t = new Test2();
        t.f();                      // satır 4
        System.out.println(t.a);    // satır 5
    }
}
```

A) satır 1 B) satır 2 C) satır 3 D) satır 4 E) satır 5

3 Bir alana **aynı paketteki** bütün sınıfların erişebilmesi, ama **başka paketteki alt sınıfların** erişememesi isteniyor. Hangi erişim belirteci kullanılmalıdır?

A) `public`
B) `private`
C) `protected`
D) `static`
E) Hiçbir belirteç yazılmaz (paket erişimi)

4 `Kutu` sınıfında sadece `int deger;` alanı vardır. Aşağıdaki kod parçası ne yazdırır?

```
Kutu k1 = new Kutu();
k1.deger = 5;
Kutu k2 = k1;
k2.deger += 3;
k1 = new Kutu();
k1.deger = 1;
System.out.println(k1.deger + " " + k2.deger);
```

A) `1 8` B) `8 8` C) `1 5` D) `5 8` E) `1 1`

5 Aşağıdaki kod çalıştıktan sonra bellekte kaç tane `Ogrenci` **nesnesi** vardır?

```
Ogrenci[] dizi = new Ogrenci[5];
dizi[0] = new Ogrenci();
dizi[1] = dizi[0];
Ogrenci x = new Ogrenci();
Ogrenci y = x;
```

A) 1 B) 2 C) 3 D) 5 E) 7

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
class Sayac {
    static int toplam = 0;
    int adet = 0;

    void artir() {
        adet++;
        toplam++;
    }
}

public class Soru6 {
    public static void main(String[] args) {
        Sayac s1 = new Sayac();
        Sayac s2 = new Sayac();
        s1.artir();
        s1.artir();
        s2.artir();
        Sayac s3 = s2;
        s3.artir();
        System.out.println(s1.adet + " " + s2.adet + " " + s3.adet);
        System.out.println(s1.toplam + " " + Sayac.toplam);
        s1 = new Sayac();
        s1.artir();
        System.out.println(s1.adet + " " + Sayac.toplam);
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
class Urun {
    String ad = "?";
    int fiyat;

    Urun() {
        this("Kalem");
        System.out.println("A");
    }

    Urun(String ad) {
        this(ad, 10);
        System.out.println("B " + ad);
    }

    Urun(String ad, int fiyat) {
        ad = ad;
        this.fiyat = fiyat * 2;
        System.out.println("C " + this.ad);
    }
}

public class Soru7 {
    public static void main(String[] args) {
        Urun u = new Urun();
        System.out.println(u.ad + " " + u.fiyat);
        Urun v = new Urun("Defter", 5);
        System.out.println(v.ad + " " + v.fiyat);
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```java
class Nokta {
    int x;

    Nokta(int x) {
        this.x = x;
    }

    @Override
    public String toString() {
        return "N" + x;
    }
}

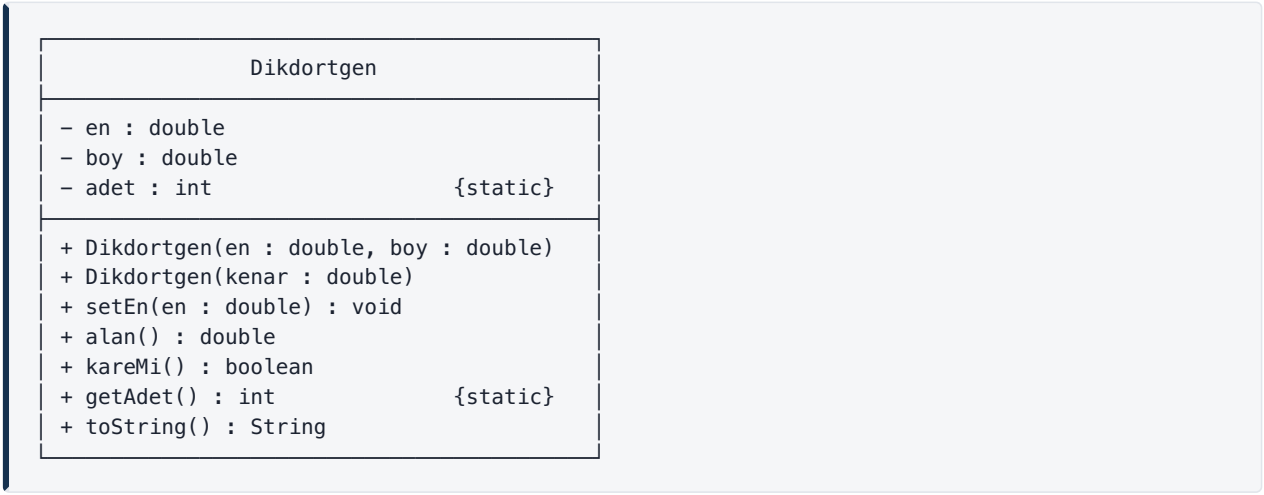
public class Soru8 {
    static void degistir(Nokta p, int sayi) {
        p.x = p.x + 10;
        sayi = sayi + 10;
        p = new Nokta(99);
        p.x++;
    }

    public static void main(String[] args) {
        Nokta a = new Nokta(1);
        Nokta b = new Nokta(1);
        int s = 1;
        degistir(a, s);
        System.out.println(a + " " + s);
        System.out.println(a == b);
        b = a;
        b.x = 7;
        System.out.println(a + " " + (a == b));
    }
}
```

C) Kod Yazma

9 Aşağıdaki UML diyagramına göre `Dikdortgen` sınıfını yaz. Kurallar:

- Tek parametreli yapıcı bir **kare** oluşturur ve işini `this(...)` ile iki parametreli yapıcıya yaptırır.
- `adet`, oluşturulan dikdörtgen sayısını tutar.
- `setEn` sadece pozitif değerleri kabul eder; aksi halde `Geçersiz en: ...` yazar ve alanı değiştirmez.
- `toString` → `Dikdortgen[3.0 x 4.0]` biçiminde metin döndürür.



`main` içinde şu işlemleri yapan `DikdortgenTest` programı aşağıdaki çıktıyı vermelidir: `(3, 4)` ve `(5)` ile iki nesne oluştur, birincinin alanını, ikincinin kare olup olmadığını yazdır; birincinin enini önce `-2`, sonra `4` yap ve tekrar kare mi diye sor; son olarak oluşturulan dikdörtgen sayısını yazdır.

```
Dikdortgen[3.0 x 4.0] alan=12.0
Dikdortgen[5.0 x 5.0] kare mi? true
Geçersiz en: -2.0
Dikdortgen[4.0 x 4.0] kare mi? true
Oluşturulan: 2
```

10 Kullanıcıdan önce öğrenci sayısını, sonra her öğrencinin **adını** (tek kelime) ve **notunu** (tam sayı) okuyan `SinifListesi` programını yaz. Öğrenciler, alanları `private` olan bir `Ogrenci` sınıfının nesneleri olarak bir **dizide** saklansın (`toString` → `Ayşe (95)`). Program sınıf ortalamasını, en yüksek notu alan öğrenciyi ve ortalamanın üstündeki öğrencileri yazdırsın:

```
Öğrenci sayısı: 4
1. öğrenci (ad not): Ali 70
2. öğrenci (ad not): Ayşe 95
3. öğrenci (ad not): Can 55
4. öğrenci (ad not): Deniz 84
Ortalama: 76.00
En yüksek: Ayşe (95)
Ortalamanın üstündekiler:
Ayşe (95)
Deniz (84)
```

Çözümler

Cevap anahtarı

1 D	2 C	3 E	4 A	5 B

1 Cevap: D

Yapıcının **dönüş tipi yoktur**; `void` bile yazılmaz. `void` yazılırsa o metot artık yapıcı değil, adı sınıfla aynı olan sıradan bir metot olur ve `new` sırasında çalışmaz. A, B (aşırı yükleme), C (varsayılan yapıcı) ve E doğrudur.

2 Cevap: C

`main` bir `static` metottur; `f()` ise örnek (instance) metodudur. `static` bağlamda nesne olmadığı için `f()` doğrudan çağrılamaz. `b` ve `g()` static olduğu için sorunsuzdur; satır 4 ve 5'te ise üyelere bir nesne (`t`) üzerinden erişildiği için hata yoktur. Derleyici çıktısı:

Derleyici çıktısı – program derlenmez

```
Test2.java:12: error: non-static method f() cannot be referenced from a static context
        f();                        // satır 3
        ^
1 error
```

3 Cevap: E

Belirteç yazılmayan üyeler **paket erişimlidir**: Aynı paketteki sınıflar erişebilir, başka paketteki sınıflar (alt sınıf olsalar bile) erişemez. `protected` başka paketteki alt sınıflara da izin verir; `public` herkese açıktır; `private` aynı paketteki sınıflara bile kapalıdır. `static` bir erişim belirteci değildir.

4 Cevap: A

`k2 = k1` ile iki referans aynı nesneyi gösterir; `k2.deger += 3` bu ortak nesneyi 8 yapar. Sonra `k1` **yeni** bir nesneye yönlendirilir ve onun değeri 1 olur. `k2` hâlâ eski nesneyi (8) göstermektedir.

Çıktı

```
1 8
```

5 Cevap: B

`new Ogrenci[5]` sadece 5 tane `null` referans içeren bir **dizi** oluşturur, hiç `Ogrenci` nesnesi oluşturmaz. Nesne oluşturan sadece iki `new Ogrenci()` ifadesidir. `dizi[1] = dizi[0]` ve `y = x` atamaları yeni nesne değil, aynı nesneye ikinci birer referans üretir. Toplam: **2** nesne. Yapıcıya sayaç koyarak doğrulayalım:

Çıktı

```
Oluşan Ogrenci nesnesi: 2
```

6

`toplam` static olduğu için tek kopyadır; `adet` her nesnede ayrıdır.

İşlem	s1.adet	s2.adet (= s3.adet)	toplam
`s1.artir()` × 2	2	0	2
`s2.artir()`	2	1	3
`s3 = s2; s3.artir()`	2	2	4
`s1 = new Sayac(); s1.artir()`	1 (yeni nesne)	2	5

`s3`, `s2` ile aynı nesneyi gösterdiği için `s3.artir()` o nesnenin `adet`'ini artırır. `s1.toplam` yazımı da `Sayac.toplam` ile aynı değişkendir.

Çıktı
```
2 2 2
4 4
1 5
```

7

- `new Urun()` → `this("Kalem")` → `this("Kalem", 10)`. En içteki yapıcı önce biter.
- İki parametreli yapıcıdaki `ad = ad;` **gölgeleme** tuzağıdır: Parametre kendine atanır, alan `"?"` olarak kalır. `this.fiyat = 10 * 2 = 20`. Yazdırılan `this.ad` → `C ?`.
- Kontrol geri döner: `B Kalem` (burada yazdırılan `ad` parametredir), sonra `A`.
- `u.ad` hâlâ `"?"`, `u.fiyat` 20.
- `new Urun("Defter", 5)` doğrudan iki parametreli yapıcıyı çalıştırır: `C ?`, fiyat `5 * 2 = 10`, ad yine `"?"`.

Çıktı
```
C ?
B Kalem
A
? 20
C ?
? 10
```

8

- `degistir(a, s)`: `p` ile `a` aynı nesneyi gösterir; `p.x = 1 + 10 = 11` kalıcıdır. `sayi` bir kopyadır, `s` 1 kalır. `p = new Nokta(99)` sadece yerel `p`'yi başka nesneye yönlendirir; `p.x++` o yeni nesneyi (100) değiştirir, `a`'yı etkilemez. → `N11 1` (`toString` sayesinde `N11`).
- `a == b`: `a` (x = 11) ve `b` (x = 1) iki **ayrı** nesnedir → `false`. (`==` içeriğe bakmaz; x'ler eşit olsaydı da sonuç `false` olurdu.)
- `b = a;` artık ikisi aynı nesne; `b.x = 7` → `a` da `N7`, `a == b` → `true`.

Çıktı
```
N11 1
false
N7 true
```

9

UML'yi satır satır çeviriyoruz: `-` alanlar `private`, `+` metotlar `public`, `{static}` olanlar `static`. Sayacı yalnızca iki parametreli yapıcıda artırıyoruz; tek parametreli yapıcı oraya `this(...)` ile yönlendiği için her nesne **bir kez** sayılır.

DikdortgenTest.java

```java
class Dikdortgen {
    private double en;
    private double boy;
    private static int adet = 0;

    public Dikdortgen(double en, double boy) {
        this.en = en;
        this.boy = boy;
        adet++;
    }

    public Dikdortgen(double kenar) {
        this(kenar, kenar);
    }

    public void setEn(double en) {
        if (en <= 0) {
            System.out.println("Geçersiz en: " + en);
            return;
        }
        this.en = en;
    }

    public double alan() {
        return en * boy;
    }

    public boolean kareMi() {
        return en == boy;
    }

    public static int getAdet() {
        return adet;
    }

    @Override
    public String toString() {
        return "Dikdortgen[" + en + " x " + boy + "]";
    }
}

public class DikdortgenTest {
    public static void main(String[] args) {
        Dikdortgen d1 = new Dikdortgen(3, 4);
        Dikdortgen d2 = new Dikdortgen(5);
        System.out.println(d1 + " alan=" + d1.alan());
        System.out.println(d2 + " kare mi? " + d2.kareMi());
        d1.setEn(-2);
        d1.setEn(4);
        System.out.println(d1 + " kare mi? " + d1.kareMi());
        System.out.println("Oluşturulan: " + Dikdortgen.getAdet());
    }
}
```

Çıktı

```
Dikdortgen[3.0 x 4.0] alan=12.0
Dikdortgen[5.0 x 5.0] kare mi? true
Geçersiz en: -2.0
Dikdortgen[4.0 x 4.0] kare mi? true
Oluşturulan: 2
```

`adet++` satırını tek parametreli yapıcıya da yazsaydık kare oluşturulurken sayaç iki kez artardı. `this(...)` zinciri, işi tek bir yerde toplamanın da yoludur.

10

Önce `n` okunur ve `new Ogrenci[n]` ile **boş** dizi oluşturulur; döngüde her eleman için ayrıca `new Ogrenci(...)` yapılır. Ad tek kelime olduğu için `next()` yeterli; `nextLine` kullanmadığımız için Bölüm 3'teki tuzak yoktur. Ortalama ve en yüksek not ilk döngüde, ortalamanın üstündekiler ikinci döngüde bulunur.

SinifListesi.java

```
import java.util.Scanner;

class Ogrenci {
    private String ad;
    private int not;

    Ogrenci(String ad, int not) {
        this.ad = ad;
        this.not = not;
    }

    int getNot() { return not; }

    @Override
    public String toString() {
        return ad + " (" + not + ")";
    }
}

public class SinifListesi {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Öğrenci sayısı: ");
        int n = sc.nextInt();
        Ogrenci[] sinif = new Ogrenci[n];

        for (int i = 0; i < n; i++) {
            System.out.print((i + 1) + ". öğrenci (ad not): ");
            String ad = sc.next();
            int not = sc.nextInt();
            sinif[i] = new Ogrenci(ad, not);
        }

        double toplam = 0;
        Ogrenci enYuksek = sinif[0];
        for (Ogrenci o : sinif) {
            toplam += o.getNot();
            if (o.getNot() > enYuksek.getNot()) {
                enYuksek = o;
            }
        }
        double ortalama = toplam / n;
        System.out.printf("Ortalama: %.2f%n", ortalama);
        System.out.println("En yüksek: " + enYuksek);

        System.out.println("Ortalamanın üstündekiler:");
        for (Ogrenci o : sinif) {
            if (o.getNot() > ortalama) {
                System.out.println(o);
```

```
            }
        }
    }
}
```

Çıktı

```
Öğrenci sayısı: 4
1. öğrenci (ad not): Ali 70
2. öğrenci (ad not): Ayşe 95
3. öğrenci (ad not): Can 55
4. öğrenci (ad not): Deniz 84
Ortalama: 76.00
En yüksek: Ayşe (95)
Ortalamanın üstündekiler:
Ayşe (95)
Deniz (84)
```

`toplam`'ı `double` tanımladığımız için `toplam / n` tam sayı bölmesi yapmaz. `println(enYuksek)` ve `println(o)` satırları `toString`'i otomatik çağırır.

BÖLÜM 10

Kalıtım

Bu Bölümde Öğreneceklerin

- Kod tekrarını kalıtımla önlemeyi; `extends`, üst sınıf/alt sınıf ve "is-a" ilişkisini
- Alt sınıfa neyin geçip neyin geçmediğini (`private` alanlar, yapıcılar)
- `super(...)` ile üst sınıf yapıcısını çağırmayı ve yapıcıların **çalışma sırasını**
- Metot ezmeyi (overriding), `@Override`'ı ve `super.metot()` çağrısını
- Overloading ile overriding arasındaki farkı (sınav favorisi)
- `Object` sınıfını: `toString`, `equals` ve `hashCode`'u doğru yazmayı
- `protected` ve `final` anahtar kelimelerini
- Java'da sınıflar için neden çoklu kalıtım olmadığını ve "is-a" ile "has-a" farkını

10.1 Kod tekrarı problemi

Bir okul uygulaması yazdığını düşün. Öğrenci ve öğretmen için iki sınıf gerekiyor:

```
class Ogrenci {
    String ad;
    int yas;
    int no;

    void tanit() {
        System.out.println("Ben " + ad + ", " + yas + " yaşındayım.");
    }
}

class Ogretmen {
    String ad;
    int yas;
    String brans;

    void tanit() {
        System.out.println("Ben " + ad + ", " + yas + " yaşındayım.");
    }
}
```

`ad`, `yas` ve `tanit()` iki sınıfta **birebir aynı**. Yarın "e-posta" alanı eklemek istersen iki yeri değiştirmen, `tanit()`'teki bir hatayı düzeltirken ikisini birden düzeltmeyi hatırlaman gerekir. Memur, veli, rektör sınıfları da gelince iş içinden çıkılmaz hale gelir.

Çözüm: Ortak kısımları **tek bir sınıfta** (`Kisi`) topla, diğer sınıflar bu sınıftan **kalıtım** (inheritance) yoluyla bu kısımları devralsın ve sadece kendilerine özgü olanları eklesin.

10.2 extends ile kalıtım

Bir sınıfın başka bir sınıftan türediğini `extends` (genişletir) anahtar kelimesiyle belirtiriz:

KalitimGiris.java

```java
class Kisi {
    String ad;
    int yas;

    void tanit() {
        System.out.println("Ben " + ad + ", " + yas + " yaşındayım.");
    }
}

class Ogrenci extends Kisi {
    int no;                     // sadece öğrencide olan alan

    void dersCalis() {
        System.out.println(ad + " ders çalışıyor.");
    }
}

class Ogretmen extends Kisi {
    String brans;

    void dersAnlat() {
        System.out.println(ad + " " + brans + " anlatıyor.");
    }
}

public class KalitimGiris {
    public static void main(String[] args) {
        Ogrenci o = new Ogrenci();
        o.ad = "Ali";           // Kisi'den gelen alan
        o.yas = 19;
        o.no = 101;             // Ogrenci'nin kendi alanı
        o.tanit();              // Kisi'den gelen metot
        o.dersCalis();

        Ogretmen t = new Ogretmen();
        t.ad = "Selin Hoca";
        t.yas = 41;
        t.brans = "Java";
        t.tanit();
        t.dersAnlat();
    }
}
```

Çıktı

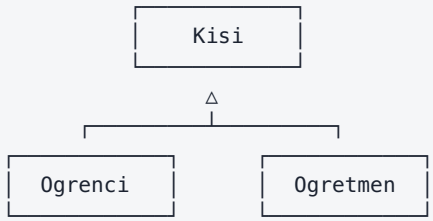
```
Ben Ali, 19 yaşındayım.
Ali ders çalışıyor.
Ben Selin Hoca, 41 yaşındayım.
Selin Hoca Java anlatıyor.
```

`Ogrenci` sınıfında `ad`, `yas` ve `tanit()` yazmadığımız halde `o.ad` ve `o.tanit()` çalıştı: Bunlar `Kisi`'den **kalıtıldı**. `Ogrenci`'nin metotları da kalıtılan `ad` alanını kendi alanı gibi kullanabiliyor. (Bölüm 9'daki gibi bütün sınıflar tek dosyada; gerçek projede her biri ayrı dosyada olurdu.)

Kalıtım veren	Kalıtım alan
Üst sınıf (superclass)	**Alt sınıf** (subclass)
Ebeveyn sınıf (parent class)	Çocuk sınıf (child class)
Temel sınıf (base class)	Türetilmiş sınıf (derived class)

Kalıtım bir **"is-a"** (*bir … dır*) ilişkisidir: "Öğrenci **bir** kişidir" cümlesi doğrudur. Tersi doğru değildir: Her kişi öğrenci değildir. Alt sınıf, üst sınıfın **özelleşmiş** bir halidir; üst sınıfın her şeyine sahiptir, üstüne kendi alan ve metotlarını ekler.

UML'de kalıtım, alt sınıftan üst sınıfa doğru giden ve ucu **içi boş üçgen** olan bir okla gösterilir:



> **İpucu: "is-a" testi**
>
> Kalıtım kurmadan önce cümleyi yüksek sesle söyle: "**Kedi bir Hayvandır**" (doğru → `Kedi extends Hayvan`), "**Kare bir Şekildir**" (doğru). "Araba bir Motordur" cümlesi saçmaysa kalıtım da yanlıştır (bölümün sonunda "has-a" ilişkisini göreceksin).

10.3 Neler kalıtılır?

Alt sınıf, üst sınıfın `public` ve `protected` üyelerini (aynı paketteyse paket erişimli üyelerini de) kalıtır. `private` üyeler için durum biraz incedir: Alt sınıf nesnesinin içinde üst sınıfın `private` alanları **vardır** ve bellekte yer kaplar; ama alt sınıfın kodu onlara **adıyla erişemez**. Bunlara üst sınıfın `public` (veya `protected`) metotları üzerinden ulaşılır:

PrivateKalitim.java

```java
class Kisi {
    private String ad = "Ali";

    public String getAd() {
        return ad;
    }
}

class Ogrenci extends Kisi {
    void yaz() {
        System.out.println(getAd());    // tamam: public metot
        System.out.println(ad);         // HATA: private alan
    }
}

public class PrivateKalitim {
    public static void main(String[] args) {
        new Ogrenci().yaz();
    }
}
```

Derleyici çıktısı – program derlenmez

```
PrivateKalitim.java:12: error: ad has private access in Kisi
        System.out.println(ad);         // HATA: private alan
                           ^
1 error
```

Üst sınıftaki üye	Alt sınıf nesnesinde var mı?	Alt sınıfın kodundan adıyla erişilir mi?
`public` / `protected` alan ve metot	Evet	Evet
Paket erişimli (belirteç yok)	Evet	Sadece aynı paketteyse
`private` alan	**Evet** (bellekte yer kaplar)	**Hayır**; getter/setter ile
`private` metot	Hayır, alt sınıf onu göremez	Hayır
Yapıcılar	**Kalıtılmaz**	Sadece `super(...)` ile çağrılır

Sınavda "`private` üyeler kalıtılır mı?" diye sorulursa: Java dil belirtimine göre **kalıtılmaz**, çünkü alt sınıf onları kullanamaz; ama alan nesnenin içinde **vardır**, üst sınıfın metotları onu kullanmaya devam eder ve alt sınıf ona `public` getter/setter ile ulaşır. İki cümleyi birlikte yaz.

10.4 super(...) ile üst sınıf yapıcısını çağırma

Yapıcılar kalıtılmaz. Ama bir `Ogrenci` nesnesinin içinde bir de `Kisi` kısmı vardır ve o kısmın ilk değerlerini **`Kisi`'nin yapıcısı** vermelidir (hele alanlar `private` ise, alt sınıf onlara zaten erişemez). Alt sınıfın yapıcısı, üst sınıfın yapıcısını `super(...)` ile çağırır:

SuperYapici.java

```
class Kisi {
    private String ad;
    private int yas;

    Kisi(String ad, int yas) {
        this.ad = ad;
        this.yas = yas;
        System.out.println("Kisi yapıcısı çalıştı");
    }

    String getAd() { return ad; }
    int getYas() { return yas; }
}

class Ogrenci extends Kisi {
    private int no;

    Ogrenci(String ad, int yas, int no) {
        super(ad, yas);         // önce üst sınıfın kısmı kurulur
        this.no = no;
        System.out.println("Ogrenci yapıcısı çalıştı");
    }

    void yaz() {
        System.out.println(no + " " + getAd() + " (" + getYas() + ")");
    }
}

public class SuperYapici {
    public static void main(String[] args) {
        Ogrenci o = new Ogrenci("Ali", 19, 101);
        o.yaz();
    }
}
```

Çıktı
```
Kisi yapıcısı çalıştı
Ogrenci yapıcısı çalıştı
101 Ali (19)
```

Kurallar:

1. `super(...)` alt sınıf yapıcısının **ilk komutu** olmalıdır (klasik kural; Java 25 esnekliği için aşağıdaki nota bak).
2. `super(...)` yazmazsan derleyici yapıcının başına **örtük** (implicit) olarak `super();` ekler; yani üst sınıfın **parametresiz** yapıcısını çağırır.
3. Bir yapıcıda `super(...)` ve `this(...)` birlikte kullanılamaz; ikisi de "ilk komut" olmak ister.

İkinci kuralın sonucu sınavların en sevilen derleme hatasıdır: Üst sınıfta parametresiz yapıcı **yoksa** ve alt sınıf `super(...)` yazmazsa, derleyicinin eklediği `super();` çağrısının karşılığı bulunamaz:

SuperEksik.java
```java
class Kisi {
    String ad;

    Kisi(String ad) {           // tek yapıcı: parametreli
        this.ad = ad;
    }
}

class Ogrenci extends Kisi {
    int no;

    Ogrenci(int no) {           // derleyici buraya super(); ekler
        this.no = no;
    }
}

public class SuperEksik {
    public static void main(String[] args) {
        new Ogrenci(101);
    }
}
```

Derleyici çıktısı – program derlenmez
```
SuperEksik.java:12: error: constructor Kisi in class Kisi cannot be applied to given types;
    Ogrenci(int no) {           // derleyici buraya super(); ekler
                    ^
  required: String
  found:    no arguments
  reason: actual and formal argument lists differ in length
1 error
```

Hata mesajı `Ogrenci` yapıcısının satırını gösteriyor ve "`Kisi` yapıcısı `String` istiyor, hiç argüman bulunamadı" diyor. Çözüm: Ya `Ogrenci` yapıcısının ilk satırına `super("...")` yaz, ya da `Kisi`'ye parametresiz bir yapıcı ekle. Dikkat: `Ogrenci`'de **hiç yapıcı yazmasan** da aynı hatayı alırsın, çünkü derleyicinin eklediği varsayılan yapıcı `Ogrenci() { super(); }` şeklindedir.

> **Java 25 Notu: Esnek yapıcı gövdeleri**
>
> Java 25 ile (JEP 513), `super(...)` ve `this(...)` çağrısından **önce** komut yazılabilir; yeter ki bu komutlar henüz kurulmamış nesneye dokunmasın. Parametre kontrolü gibi işler için kullanışlıdır:

```java
class Kisi {
    String ad;

    Kisi(String ad) {
        this.ad = ad;
    }
}

class Ogrenci extends Kisi {
    Ogrenci(String ad) {
        if (ad == null || ad.isEmpty()) {
            ad = "İsimsiz";         // parametreyi düzeltmek serbest
        }
        super(ad);                  // Java 25'te geçerli
    }
}

public class EsnekYapici {
    public static void main(String[] args) {
        System.out.println(new Ogrenci("").ad);
        System.out.println(new Ogrenci("Ece").ad);
    }
}
```

Çıktı

```
İsimsiz
Ece
```

`super(...)`'dan önce nesnenin metotlarını çağırmak veya alanlarını okumak ise hâlâ yasaktır (*cannot reference ... before supertype constructor has been called*). Java 24 ve öncesinde bu kod hiç derlenmez; derslerde ve sınavlarda "`super(...)` ilk satırda olmalı" kuralı sorulur.

10.5 Yapıcıların çalışma sırası

Kalıtım zincirinde her yapıcının ilk işi (açıkça veya örtük olarak) **üst sınıfın yapıcısını çağırmaktır**. Bu yüzden `new Kopek()` dediğinde yapıcı gövdeleri **en üstten aşağıya doğru** çalışır:

YapiciSirasi.java

```java
class Canli {
    Canli() {
        System.out.println("1. Canli yapıcısı");
    }
}

class Hayvan extends Canli {
    Hayvan() {
        System.out.println("2. Hayvan yapıcısı");
    }
}

class Kopek extends Hayvan {
    Kopek() {
        System.out.println("3. Kopek yapıcısı");
    }
}

public class YapiciSirasi {
    public static void main(String[] args) {
        new Kopek();
```

```
        System.out.println("---");
        new Hayvan();
    }
}
```

Çıktı

```
1. Canli yapıcısı
2. Hayvan yapıcısı
3. Kopek yapıcısı
---
1. Canli yapıcısı
2. Hayvan yapıcısı
```

Hiçbir yapıcıda `super()` yazmadık; derleyici hepsine örtük `super();` ekledi. Çağrı ve çalışma sırası:

```
new Kopek()
 └─ Kopek()          ilk iş: super()
     └─ Hayvan()        ilk iş: super()
         └─ Canli()        ilk iş: super()
             └─ Object()      (her sınıfın en üst atası)
             Canli gövdesi çalışır      → 1. satır
         Hayvan gövdesi çalışır         → 2. satır
     Kopek gövdesi çalışır              → 3. satır
```

Sınavda Çıkar: Yapıcı sırası sorusu

"Aşağıdaki program ne yazdırır?" sorularında kural: Yapıcılar **alttan yukarı çağrılır**, gövdeleri **yukarıdan aşağı biter**. Önce temel atılır, sonra katlar çıkılır. Soruda `this(...)` da varsa, önce aynı sınıftaki hedef yapıcıya git, zincirin `super(...)`'ya ulaştığı yerden yukarı çık. Adım adım bir **çağrı ağacı** çiz; ezbere gitme.

10.6 Metot ezme (overriding)

Alt sınıf, üst sınıftan kalıttığı bir metodu **aynı imzayla yeniden yazarak** kendi davranışını tanımlayabilir. Buna **metot ezme** (overriding, *geçersiz kılma*) denir:

Ezme.java

```
class Hayvan {
    String ad;

    Hayvan(String ad) {
        this.ad = ad;
    }

    void sesCikar() {
        System.out.println("...");
    }

    void tanit() {
        System.out.print("Ben " + ad + ". Sesim: ");
        sesCikar();             // hangi sesCikar çalışacak?
    }
}

class Kedi extends Hayvan {
    Kedi(String ad) {
        super(ad);
    }
```

```
    @Override
    void sesCikar() {
        System.out.println("Miyav!");
    }
}

class Kopek extends Hayvan {
    Kopek(String ad) {
        super(ad);
    }

    @Override
    void sesCikar() {
        System.out.println("Hav hav!");
    }
}

class Balik extends Hayvan {
    Balik(String ad) {
        super(ad);
    }
    // sesCikar ezilmedi: Hayvan'ınki kullanılır
}

public class Ezme {
    public static void main(String[] args) {
        Kedi k = new Kedi("Tekir");
        Kopek d = new Kopek("Karabaş");
        Balik b = new Balik("Nemo");
        k.sesCikar();
        d.sesCikar();
        b.sesCikar();
        System.out.println("---");
        k.tanit();
        d.tanit();
        b.tanit();
    }
}
```

Çıktı

```
Miyav!
Hav hav!
...
---
Ben Tekir. Sesim: Miyav!
Ben Karabaş. Sesim: Hav hav!
Ben Nemo. Sesim: ...
```

İkinci kısma dikkat: `tanit()` metodu **`Hayvan`'ın kodudur** ve içinde `sesCikar()` çağrısı vardır. Yine de `k.tanit()` "Miyav!" yazdı. Java, ezilmiş bir metot çağrıldığında **nesnenin gerçek sınıfına** bakar: Nesne `Kedi` ise `Kedi`'nin `sesCikar`'ı çalışır, çağrı üst sınıfın içinden yapılmış olsa bile. Buna **dinamik bağlama** (dynamic binding) denir; Bölüm 11'de çok biçimliliğin temeli olarak tekrar göreceksin.

@Override etiketi

Metodun üstündeki `@Override` bir **açıklamadır** (annotation): Derleyiciye "bu metot üst sınıftaki bir metodu eziyor" der. Program çalışırken hiçbir etkisi yoktur, ama derleyici bu iddiayı **kontrol eder**. Metot adını yanlış yazar veya bir ezme kuralını çiğnersen derleme hatası alırsın:

EzmeHatalari.java

```java
class Hayvan {
    public void sesCikar() {
        System.out.println("...");
    }
}

class Kedi extends Hayvan {
    @Override
    public void sesCikart() {       // yazım hatası: sesCikar değil!
        System.out.println("Miyav!");
    }
}

class Kopek extends Hayvan {
    @Override
    void sesCikar() {               // public → paket erişimi: daraltma!
        System.out.println("Hav hav!");
    }
}

public class EzmeHatalari {
    public static void main(String[] args) {
        new Kedi().sesCikar();
    }
}
```

Derleyici çıktısı – program derlenmez

```
EzmeHatalari.java:8: error: method does not override or implement a method from a supertype
    @Override
    ^
EzmeHatalari.java:16: error: sesCikar() in Kopek cannot override sesCikar() in Hayvan
    void sesCikar() {               // public → paket erişimi: daraltma!
         ^
  attempting to assign weaker access privileges; was public
2 errors
```

Sık Yapılan Hata: @Override yazmamak

`Kedi`'deki yazım hatalı metodun üstünde `@Override` olmasaydı program **sessizce derlenirdi**: `sesCikart` yeni bir metot olurdu ve `new Kedi().sesCikar()` "Miyav!" yerine "..." yazardı. Aynı tuzak `toString` yerine `tostring` yazınca da olur. Ezdiğin **her** metodun üstüne `@Override` yaz; hatayı derleyici yakalasın.

Ezme kuralları

Kural	Açıklama
İmza aynı olmalı	Metot adı ve parametre listesi (sayı, tip, sıra) birebir aynı. Parametre listesi farklıysa bu ezme değil **aşırı yüklemedir**. Parametre **adlarının** önemi yoktur.
Dönüş tipi	Aynı olmalı. (Nesne döndüren metotlarda alt tipi de olabilir; ilkel tiplerde birebir aynı.)
Erişim **daraltılamaz**	`public` metot alt sınıfta `protected`, paket erişimli veya `private` yapılamaz. Genişletmek serbesttir.
`private` metotlar ezilemez	Alt sınıf görmediği metodu ezemez; aynı adla yazarsa bu yeni bir metottur.
`static` metotlar ezilemez	Alt sınıfta aynı imzayla yazılan `static` metot üsttekini sadece **gizler** (hiding).
`final` metotlar ezilemez	Derleme hatası (birazdan göreceksin).

Erişim belirteçlerinin açıklık sırası (Bölüm 9): `private` < paket < `protected` < `public`. Ezerken bu sırada sadece **sağa** gidebilirsin.

10.7 super.metot() ile üst sınıfın metodunu çağırma

Ezme, üst sınıfın metodunu tamamen değiştirir. Çoğu zaman ise üst sınıfın yaptığını **tekrar yazmadan genişletmek** isteriz. Alt sınıfın içinde `super.metotAdi(...)` yazarak üst sınıfın versiyonunu çağırabilirsin:

SuperMetot.java

```
class Calisan {
    private String ad;
    private double maas;

    Calisan(String ad, double maas) {
        this.ad = ad;
        this.maas = maas;
    }

    double maasHesapla() {
        return maas;
    }

    void bilgiYazdir() {
        System.out.println("Ad: " + ad);
        System.out.println("Maaş: " + maasHesapla());
    }
}

class Mudur extends Calisan {
    private double prim;

    Mudur(String ad, double maas, double prim) {
        super(ad, maas);
        this.prim = prim;
    }

    @Override
    double maasHesapla() {
        return super.maasHesapla() + prim;  // üst sınıfın hesabı + prim
    }
```

```
    @Override
    void bilgiYazdir() {
        super.bilgiYazdir();                // önce ortak bilgiler
        System.out.println("Prim: " + prim);
    }
}

public class SuperMetot {
    public static void main(String[] args) {
        Calisan c = new Calisan("Ayşe", 40000);
        Mudur m = new Mudur("Burak", 60000, 15000);
        c.bilgiYazdir();
        System.out.println("---");
        m.bilgiYazdir();
    }
}
```

Çıktı

```
Ad: Ayşe
Maaş: 40000.0
---
Ad: Burak
Maaş: 75000.0
Prim: 15000.0
```

`m.bilgiYazdir()` önce `super.bilgiYazdir()` ile `Calisan`'ın kodunu çalıştırır. O kodun içindeki `maasHesapla()` çağrısı, nesne bir `Mudur` olduğu için **`Mudur`'un** versiyonuna gider: 60000 + 15000 = 75000.

> **● Sık Yapılan Hata: super. yazmayı unutmak**
>
> `Mudur`'da `return super.maasHesapla() + prim;` yerine `return maasHesapla() + prim;` yazarsan metot **kendini** çağırır, o da yine kendini... Sonu gelmeyen bu çağrı zinciri programı `StackOverflowError` ile çökertir (özyinelemeyi Bölüm 15'te göreceksin). Üst sınıfın versiyonunu istiyorsan `super.` şarttır. Ayrıca `super.super.metot()` diye bir yazım **yoktur**; sadece bir üst seviyeye ulaşabilirsin.

10.8 Overloading ve overriding farkı

İki terim birbirine çok benzer ama bambaşka şeylerdir. Hocaların en sevdiği karşılaştırma sorusudur:

	Aşırı yükleme (overloading)	Ezme (overriding)
Nerede?	Aynı sınıfta (veya alt sınıfta)	Alt sınıfta; kalıtım şart
Metot adı	Aynı	Aynı
Parametre listesi	**Farklı** olmalı	**Aynı** olmalı
Dönüş tipi	Serbest (ama tek başına fark yetmez)	Aynı (veya alt tipi)
Erişim belirteci	Serbest	Daraltılamaz
`static`, `private`, `final` metotlar	Aşırı yüklenebilir	Ezilemez
Hangi metot çalışacak, ne zaman belli olur?	**Derleme** zamanında, argüman tiplerine bakılarak	**Çalışma** zamanında, nesnenin gerçek sınıfına bakılarak
Diğer adı	Statik bağlama, derleme zamanı çok biçimliliği	Dinamik bağlama, çalışma zamanı çok biçimliliği
`@Override`	Kullanılmaz	Kullanılmalı

Bölüm 6'da yazdığın aynı adlı, farklı parametreli metotlar (ör. `int` ve `double` alan iki `topla`) aşırı yüklemeydi; bu bölümdeki `Kedi.sesCikar()` ise ezmedir. İkisi aynı sınıfta bir arada da olabilir: Alt sınıf, üst sınıfın bir metodunu ezerken aynı adla farklı parametreli yeni bir metot da ekleyebilir (Soru 7'de göreceksin).

> **Sınavda Çıkar: Ezberleme yolu**
>
> Over**loading** → aynı ad, farklı **yük** (parametre). Over**riding** → aynı imza, üst sınıfın metodunun **üzerine yaz**. Bir sınıfta yalnızca dönüş tipi farklı iki metot (`int f()` ve `double f()`) yazılamaz: Bu aşırı yükleme değildir, derleme hatasıdır (Bölüm 6). Alt sınıfta parametre adı farklı ama tipleri aynı bir metot ise **ezmedir**.

10.9 Object sınıfı: her sınıfın atası

`extends` yazmadığın her sınıf, örtük olarak Java'nın **`Object`** sınıfından türer: `class Kisi { }` ile `class Kisi extends Object { }` aynıdır. Yapıcı zincirinin en tepesinde `Object()` olmasının sebebi budur. Böylece her nesnede `Object`'ten gelen bazı metotlar hazır bulunur:

Metot	Varsayılan davranış	Genelde ezilir mi?
`toString()`	`SinifAdi@onaltılıkSayı` döndürür	Evet
`equals(Object o)`	`==` ile aynı: **aynı nesne mi?**	Evet; içerik karşılaştırması için
`hashCode()`	Nesneye özgü bir tam sayı	`equals` ezildiyse **mutlaka**
`getClass()`	Nesnenin gerçek sınıfı	Hayır (`final`)

toString

Bölüm 9'da yazdığın `toString` aslında `Object`'in metodunu **ezmekti**. `public` olmasının zorunlu olması da buradan gelir: `Object`'teki `toString` `public`'tir ve ezerken erişim daraltılamaz. Alt sınıf, üst sınıfın metnini `super.toString()` ile kullanıp genişletebilir:

ToStringKalitim.java

```
class Kisi {
    private String ad;

    Kisi(String ad) {
```

```
        this.ad = ad;
    }

    @Override
    public String toString() {
        return "Kisi[" + ad + "]";
    }
}

class Ogrenci extends Kisi {
    private int no;

    Ogrenci(String ad, int no) {
        super(ad);
        this.no = no;
    }

    @Override
    public String toString() {
        return super.toString() + " no=" + no;
    }
}

class Bos { }

public class ToStringKalitim {
    public static void main(String[] args) {
        System.out.println(new Kisi("Ali"));
        System.out.println(new Ogrenci("Ayşe", 102));
        System.out.println(new Bos());
    }
}
```

Çıktı

```
Kisi[Ali]
Kisi[Ayşe] no=102
Bos@4e0e2f2a
```

`Bos` sınıfı `toString` ezmediği için `Object`'in versiyonu çalıştı (`@`'ten sonraki sayı senin bilgisayarında farklı olabilir).

equals

`Object`'in `equals`'ı sadece `==` yapar. İki öğrencinin numarası ve adı aynıysa **eşit** sayılmasını istiyorsak `equals`'ı ezmeliyiz. Doğru yazım dört adımlık bir kalıptır:

EqualsOrnek.java

```
import java.util.Objects;

class Ogrenci {
    private int no;
    private String ad;

    Ogrenci(int no, String ad) {
        this.no = no;
        this.ad = ad;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {                        // 1. aynı nesne mi?
```

```
            return true;
        }
        if (o == null || getClass() != o.getClass()) {
            return false;                       // 2. null mı, sınıf farklı mı?
        }
        Ogrenci diger = (Ogrenci) o;            // 3. Object → Ogrenci
        return no == diger.no && Objects.equals(ad, diger.ad);  // 4. alanlar
    }

    @Override
    public int hashCode() {
        return Objects.hash(no, ad);
    }
}

public class EqualsOrnek {
    public static void main(String[] args) {
        Ogrenci a = new Ogrenci(101, "Ali");
        Ogrenci b = new Ogrenci(101, "Ali");
        Ogrenci c = new Ogrenci(102, "Ali");
        System.out.println(a == b);
        System.out.println(a.equals(b));
        System.out.println(a.equals(c));
        System.out.println(a.equals(null));
        System.out.println(a.equals("Ali"));
        System.out.println(a.hashCode() == b.hashCode());
    }
}
```

Çıktı

```
false
true
false
false
false
true
```

1. **Aynı nesne mi?** Öyleyse kesinlikle eşittir; hızlı çıkış.
2. **`null` mı veya başka sınıftan mı?** `a.equals(null)` ve `a.equals("Ali")` burada `false` döner. `getClass()` nesnenin gerçek sınıfını verir.
3. **Dönüştür:** Parametrenin tipi `Object` olduğu için `no` alanına erişemeyiz. `(Ogrenci) o` ile "bu aslında bir `Ogrenci`" diyoruz (aşağı dönüşüm, *downcasting*; Bölüm 11). 2. adım sayesinde bu dönüşüm güvenlidir.
4. **Alanları karşılaştır:** İlkel alanlar `==` ile, nesne alanları (`String`) `Objects.equals(ad, diger.ad)` ile. `ad.equals(diger.ad)` de çalışır, ama `ad` alanı `null` ise `NullPointerException` fırlatır; `Objects.equals` iki tarafın `null` olmasını da güvenle karşılar (`hashCode`'daki `Objects.hash` gibi).

`getClass()` yerine `instanceof` da kullanılabilir: `if (!(o instanceof Ogrenci)) return false;`. Fark şudur: `getClass()` **tam olarak aynı sınıfı** ister; `instanceof` ise alt sınıf nesnelerini de kabul eder. `null instanceof Ogrenci` her zaman `false` olduğu için bu yazımda ayrıca `null` kontrolüne gerek kalmaz. Derslerde ikisi de görülür; alt sınıflar işin içindeyse `getClass()` daha güvenlidir.

equals'ın yanlış imzası

En sık yapılan hata, parametreyi `Object` yerine kendi sınıfın olarak yazmaktır. Bu bir **ezme değil, aşırı yüklemedir**:

EqualsTuzak.java

```java
class Nokta {
    int x, y;

    Nokta(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public boolean equals(Nokta p) {        // DİKKAT: Object değil!
        return x == p.x && y == p.y;
    }
}

public class EqualsTuzak {
    public static void main(String[] args) {
        Nokta a = new Nokta(1, 2);
        Nokta b = new Nokta(1, 2);
        Object c = b;                       // aynı nesne, Object tipinde
        System.out.println(a.equals(b));
        System.out.println(a.equals(c));
    }
}
```

Çıktı

```
true
false
```

Her sınıf `Object`'in alt sınıfı olduğu için `Object` tipindeki bir değişken her nesneyi gösterebilir (Bölüm 11'de ayrıntılı). `a.equals(c)` çağrısında argümanın tipi `Object` olduğundan derleyici `equals(Nokta)`'yı değil, `Object`'ten gelen `equals(Object)`'i seçer; o da `==` yapar. Java'nın kendi kütüphaneleri (listeler, kümeler; Bölüm 13) hep `equals(Object)`'i çağırdığı için bu sınıf orada **yanlış çalışır**. Metodun üstüne `@Override` yazsaydık derleyici hatayı hemen yakalardı.

hashCode

Kural: **İki nesne `equals`'a göre eşitse `hashCode` değerleri de aynı olmalıdır.** (Tersi gerekmez: Eşit olmayan nesnelerin `hashCode`'u tesadüfen aynı çıkabilir.) `HashSet` ve `HashMap` gibi yapılar (Bölüm 13) nesneleri önce `hashCode`'a göre gruplar; `equals`'ı ezip `hashCode`'u ezmezsen "eşit" iki nesneyi farklı sanırlar. En kolay doğru yol, `equals`'ta kullandığın alanları `Objects.hash(...)`'e vermektir (`java.util.Objects`).

Sınavda Çıkar: equals sorularının özeti

- `==` → aynı nesne mi? `equals` → (ezildiyse) içerik aynı mı?
- Doğru imza: **`public boolean equals(Object o)`**. `equals(Ogrenci o)` aşırı yüklemedir.
- `equals` ezilirse `hashCode` da ezilir; eşit nesnelerin `hashCode`'u eşit olmalı.
- `equals` sözleşmesi: `a.equals(a)` her zaman `true`; `a.equals(b)` ise `b.equals(a)`; `a.equals(null)` her zaman `false`.

10.10 protected erişim

Bölüm 9'daki tabloyu hatırla: `protected` üyelere **alt sınıflar** (başka pakette olsalar bile) ve **aynı paketteki** sınıflar erişebilir. Üst sınıf, alt sınıflarının doğrudan kullanmasını istediği alanları `protected` yapar:

ProtectedOrnek.java

```java
class Hesap {
    protected double bakiye;        // alt sınıflar doğrudan kullanabilir
    private String hesapNo;         // alt sınıf dahil kimse göremez

    Hesap(String hesapNo) {
        this.hesapNo = hesapNo;
    }

    void yatir(double miktar) {
        bakiye += miktar;
    }

    String getHesapNo() { return hesapNo; }
}

class VadeliHesap extends Hesap {
    private double faizOrani;

    VadeliHesap(String hesapNo, double faizOrani) {
        super(hesapNo);
        this.faizOrani = faizOrani;
    }

    void faizEkle() {
        bakiye += bakiye * faizOrani;   // protected alana doğrudan erişim
        // hesapNo = "X";               // private: derlenmezdi
    }
}

public class ProtectedOrnek {
    public static void main(String[] args) {
        VadeliHesap h = new VadeliHesap("TR-001", 0.05);
        h.yatir(1000);
        h.faizEkle();
        System.out.println(h.getHesapNo() + ": " + h.bakiye);
    }
}
```

Çıktı

```
TR-001: 1050.0
```

`main` metodu da `h.bakiye`'ye erişebildi, çünkü bütün sınıflar aynı (varsayılan) paketteler. Yani tek dosyalık örneklerimizde `protected` ile paket erişimi arasında fark görünmez; fark, sınıflar farklı paketlere ayrıldığında ortaya çıkar. `protected` alan, alt sınıflara kolaylık sağlar ama kapsüllemeyi zayıflatır; birçok programcı alanları yine `private` tutup alt sınıflara `protected` getter/setter sunmayı tercih eder.

10.11 final: ezilemez, genişletilemez

`final`'ı Bölüm 2'de sabitler için görmüştün. Kalıtımda iki yeni anlamı vardır:

Nerede?	Anlamı	Örnek
Değişken / alan	Değeri **bir kez** atanır	`final int MAX = 100;`
Metot	Alt sınıfta **ezilemez**	`final void kritikIslem()`
Sınıf	**Kalıtılamaz**; alt sınıfı olamaz	`final class String`

FinalHatalari.java

```java
final class Sabit {
}

class Deneme extends Sabit {        // HATA 1: final sınıftan türetme
}

class Ust {
    final void kritik() {
        System.out.println("Değiştirilemez");
    }
}

class Alt extends Ust {
    @Override
    void kritik() {                 // HATA 2: final metot ezilemez
        System.out.println("Değiştirdim!");
    }
}

public class FinalHatalari {
    public static void main(String[] args) {
    }
}
```

Derleyici çıktısı – program derlenmez

```
FinalHatalari.java:4: error: cannot inherit from final Sabit
class Deneme extends Sabit {        // HATA 1: final sınıftan türetme
                       ^
FinalHatalari.java:15: error: kritik() in Alt cannot override kritik() in Ust
    void kritik() {                 // HATA 2: final metot ezilemez
         ^
  overridden method is final
2 errors
```

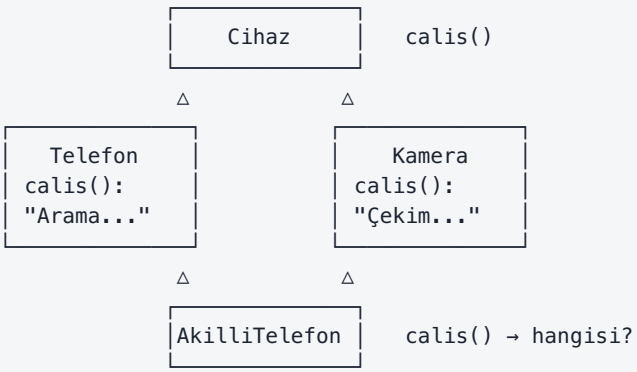
Java'nın `String` sınıfı da `final`'dır; `class BenimString extends String` yazamazsın. Bir metodun davranışının alt sınıflarca bozulmasını istemediğinde (güvenlik, doğruluk) metodu veya sınıfı `final` yaparsın. `final` bir **alan** ise ya tanımlandığı yerde ya da yapıcıda **tam bir kez** değer almalıdır; öğrenci numarası gibi hiç değişmeyecek bilgiler için idealdir: `private final int no;`.

> **● Sık Yapılan Hata: final referans ≠ değişmez nesne**
>
> `final Kutu k = new Kutu();` dediğinde **`k` değişkeni** başka nesneye yönlendirilemez (`k = new Kutu();` derlenmez); ama gösterdiği nesnenin **içi** değiştirilebilir: `k.deger = 5;` ve `k.deger++;` serbesttir. `final`, referansı sabitler, nesneyi değil. Aynı şey `final int[] dizi` için de geçerlidir: `dizi[0] = 9;` serbest, `dizi = new int[3];` yasak.

10.12 Java'da tekli kalıtım

Java'da bir sınıf `extends` ile **yalnızca bir** sınıftan türeyebilir. `class C extends A, B` yazarsan derleyici virgülü görünce sınıf gövdesini bekler ve `'{' expected` hatası verir.

Neden? **Elmas problemi** (diamond problem) yüzünden. Şu hiyerarşiyi düşün:



`AkilliTelefon` hem `Telefon`'dan hem `Kamera`'dan türeyebilseydi, `calis()` çağrısında iki farklı ezilmiş versiyondan hangisinin çalışacağı belirsiz olurdu. Java bu belirsizliği kökten engellemek için sınıflarda çoklu kalıtıma izin vermez. Bir sınıfın birden fazla "tür" olması gerektiğinde **arayüzler** (interface) kullanılır; bir sınıf istediği kadar arayüzü uygulayabilir (Bölüm 11).

Kalıtım türü	Yapı	Java'da sınıflarla mümkün mü?
Tekli (single)	`B extends A`	Evet
Çok seviyeli (multilevel)	`B extends A`, `C extends B`	Evet
Hiyerarşik (hierarchical)	`B extends A`, `C extends A`	Evet
Çoklu (multiple)	`C extends A, B`	**Hayır** (arayüzlerle, Bölüm 11)

10.13 is-a ve has-a: kalıtım mı, kompozisyon mu?

İki sınıf arasındaki her ilişki kalıtım değildir:

- **is-a** (*bir … dır*): "Araba **bir** taşıttır." → kalıtım: `class Araba extends Tasit`
- **has-a** (*… sahiptir*): "Arabanın **bir** motoru **vardır**." → **kompozisyon** (composition): `Araba` sınıfında `Motor` tipinde bir **alan** bulunur.

`class Araba extends Motor` yazmak teknik olarak derlenir ama tasarım olarak yanlıştır: Araba bir motor değildir ve motorun bütün metotlarını (ör. `yagDegistir`) dışarıya açmak istemezsin.

Kompozisyon.java

```
class Motor {
    private int beygirGucu;
    private boolean calisiyor;

    Motor(int beygirGucu) {
        this.beygirGucu = beygirGucu;
    }

    void calistir() {
        calisiyor = true;
        System.out.println(beygirGucu + " beygirlik motor çalıştı.");
    }

    boolean calisiyorMu() { return calisiyor; }
}

class Tasit {
    protected String marka;
```

```
    Tasit(String marka) {
        this.marka = marka;
    }
}

class Araba extends Tasit {          // is-a: Araba bir Taşıttır
    private Motor motor;             // has-a: Arabanın bir Motoru vardır

    Araba(String marka, int beygir) {
        super(marka);
        motor = new Motor(beygir);
    }

    void yolaCik() {
        motor.calistir();            // işi motora devreder
        if (motor.calisiyorMu()) {
            System.out.println(marka + " yola çıktı.");
        }
    }
}

public class Kompozisyon {
    public static void main(String[] args) {
        Araba a = new Araba("Togg", 218);
        a.yolaCik();
    }
}
```

Çıktı

```
218 beygirlik motor çalıştı.
Togg yola çıktı.
```

	is-a (kalıtım)	has-a (kompozisyon)
Java'da	`extends`	Sınıf tipinde alan
Cümle testi	"X bir Y'dir"	"X'in bir Y'si vardır"
Örnek	Kedi – Hayvan, Kare – Şekil	Araba – Motor, Okul – Öğrenci
Bağ	Sıkı: Üst sınıftaki değişiklik alt sınıfları etkiler	Gevşek: Parça kolayca değiştirilebilir

> **Sınavda Çıkar: Kalıtım mı, kompozisyon mu?**
>
> "Aşağıdaki ilişkilerden hangisi kalıtımla modellenmelidir?" sorusunda cümle testini uygula. Kalıtım her zaman cazip görünür ama "has-a" ilişkisini `extends` ile kurmak klasik bir tasarım hatasıdır. Profesyonellerin kuralı: **"Kalıtım yerine kompozisyonu tercih et"** (*favor composition over inheritance*); kalıtımı sadece gerçek bir "is-a" ilişkisi varsa kullan.

10.14 Çok seviyeli kalıtım örneği

Bölümdeki her şeyi bir arada görelim: `Canli → Hayvan → Kopek` zinciri; her seviye kendi alanını ekliyor, `super(...)` ile yukarıya değer gönderiyor, metotları eziyor veya genişletiyor:

CokSeviyeli.java

```
class Canli {
    private int yas;

    Canli(int yas) {
```

```
        this.yas = yas;
    }

    int getYas() { return yas; }

    void nefesAl() {
        System.out.println("Nefes alıyor.");
    }

    @Override
    public String toString() {
        return "yaş=" + yas;
    }
}

class Hayvan extends Canli {
    private int ayakSayisi;

    Hayvan(int yas, int ayakSayisi) {
        super(yas);
        this.ayakSayisi = ayakSayisi;
    }

    void hareketEt() {
        System.out.println(ayakSayisi + " ayakla yürüyor.");
    }

    void sesCikar() {
        System.out.println("...");
    }

    @Override
    public String toString() {
        return super.toString() + ", ayak=" + ayakSayisi;
    }
}

class Kopek extends Hayvan {
    private String cins;

    Kopek(int yas, String cins) {
        super(yas, 4);              // köpeğin her zaman 4 ayağı var
        this.cins = cins;
    }

    @Override
    void sesCikar() {
        System.out.println("Hav hav!");
    }

    @Override
    void hareketEt() {
        super.hareketEt();
        System.out.println("Kuyruğunu sallıyor.");
    }

    @Override
    public String toString() {
        return "Kopek[" + super.toString() + ", cins=" + cins + "]";
    }
}

public class CokSeviyeli {
    public static void main(String[] args) {
        Kopek k = new Kopek(3, "Kangal");
```

```
        k.nefesAl();
        k.hareketEt();
        k.sesCikar();
        System.out.println(k.getYas());
        System.out.println(k);
    }
}
```

Çıktı

```
Nefes alıyor.
4 ayakla yürüyor.
Kuyruğunu sallıyor.
Hav hav!
3
Kopek[yaş=3, ayak=4, cins=Kangal]
```

Çağrı	Hangi sınıfın kodu çalışır?
`k.nefesAl()`	`Canli` (iki seviye yukarıdan kalıtıldı)
`k.hareketEt()`	`Kopek`, içinden `super.hareketEt()` ile `Hayvan`
`k.sesCikar()`	`Kopek` (ezdi)
`k.getYas()`	`Canli` (`yas` private ama getter public erişimli)
`println(k)`	`Kopek.toString` → `Hayvan.toString` → `Canli.toString` zinciri

`toString` zincirinde her seviye `super.toString()` ile bir üsttekinin metnini alıp kendi parçasını ekliyor; sonuçta metin en üstten en alta doğru büyüyor.

Bölüm Özeti – Bilmen Gerekenler

- Kalıtım: `class Alt extends Ust`. Alt sınıf üst sınıfın üyelerini devralır ve kendi üyelerini ekler. İlişki **is-a**'dır: "Öğrenci bir Kişidir".
- `private` alanlar alt sınıf nesnesinde vardır ama alt sınıfın kodu onlara **adıyla erişemez**; getter/setter kullanılır. **Yapıcılar kalıtılmaz.**
- Alt sınıf yapıcısı üst sınıfınkini **`super(...)`** ile çağırır (klasik kural: ilk komut). Yazılmazsa örtük `super();` eklenir; üst sınıfta parametresiz yapıcı yoksa **derleme hatası**.
- Yapıcılar alttan yukarı çağrılır, gövdeleri **yukarıdan aşağı** çalışır: `Object` → `Canli` → `Hayvan` → `Kopek`.
- **Ezme** (overriding): alt sınıfta aynı imzalı metot. Dönüş tipi aynı, erişim **daraltılamaz**; `private`, `static`, `final` metotlar ezilemez. Ezdiğin her metoda **`@Override`** yaz.
- Ezilmiş metot, üst sınıfın kodundan çağrılsa bile **nesnenin gerçek sınıfının** versiyonu çalışır (dinamik bağlama).
- `super.metot()` üst sınıfın versiyonunu çağırır; `super.` unutulursa metot kendini çağırır.
- Overloading: farklı parametre, derleme zamanı. Overriding: aynı imza, çalışma zamanı.
- Her sınıf `Object`'ten türer. `toString` → `Sinif@sayı`, `equals` → `==`. Doğru imza: `public boolean equals(Object o)`; `equals` ezilirse `hashCode` da ezilir (`Objects.hash`).
- `protected`: alt sınıflar + aynı paket. `final` metot ezilemez, `final` sınıf kalıtılamaz (`String`).
- Java'da sınıflar için **tekli kalıtım** vardır (elmas problemi); çoklu tür için arayüzler (Bölüm 11).
- **has-a** ilişkisi kalıtımla değil **kompozisyonla** (alan olarak) kurulur: Araba–Motor.

Sorular

A) Çoktan Seçmeli

1 Java'da kalıtım ile ilgili aşağıdakilerden hangisi **doğrudur**?

(A) Bir sınıf `extends` ile birden fazla sınıftan türeyebilir.

(B) Alt sınıf, üst sınıfın `private` alanlarına adıyla doğrudan erişebilir.

(C) Üst sınıfın yapıcıları alt sınıfa kalıtılır.

(D) `final` olarak tanımlanmış bir sınıftan başka bir sınıf türetilemez.

(E) Alt sınıfta ezilen bir metodun erişimi daraltılabilir (`public` → `private`).

2 Aşağıdaki `B` sınıfında işaretli üç metot için hangisi doğrudur?

```
class A {
    void yaz(int x) { System.out.println("A: " + x); }
}

class B extends A {
    void yaz(int y) { System.out.println("B: " + y); }       // (I)
    void yaz(String s) { System.out.println("B: " + s); }    // (II)
    int yaz(int x, int y) { return x + y; }                  // (III)
}
```

(A) (I) ezmedir (overriding); (II) ve (III) aşırı yüklemedir (overloading).

(B) Üçü de ezmedir.

(C) (I) aşırı yüklemedir; (II) ve (III) ezmedir.

(D) (III) derleme hatası verir, çünkü dönüş tipi `void` değildir.

(E) (I) derleme hatası verir, çünkü parametre adı üst sınıftakinden farklıdır.

3 Aşağıdaki program için hangisi doğrudur?

```
class Arac {
    String plaka;

    Arac(String plaka) {
        this.plaka = plaka;
    }
}

class Otobus extends Arac {
    int koltuk;

    Otobus(int koltuk) {
        this.koltuk = koltuk;
    }
}

public class Test3 {
    public static void main(String[] args) {
        Otobus o = new Otobus(45);
        System.out.println(o.koltuk);
    }
}
```

(A) Derlenir ve `45` yazdırır; `plaka` alanı `null` kalır.

(B) Derlenmez; `Otobus` yapıcısına örtük olarak eklenen `super()` çağrısının karşılığı olan parametresiz yapıcı `Arac`'ta yoktur.

(C) Derlenir, ama çalışırken `NullPointerException` oluşur.

(D) Derlenmez; çünkü `Otobus` sınıfında `plaka` alanı tanımlanmamıştır.

(E) Derlenmez; çünkü alt sınıfın yapıcısında `this` kullanılamaz.

4 Aşağıdaki sınıf çiftlerinden hangisi kalıtımla (`extends`) değil, **kompozisyonla** (bir sınıfın diğerini alan olarak tutmasıyla) modellenmelidir?

(A) `Kedi` – `Hayvan` (B) `Kare` – `Sekil` (C) `Araba` – `Motor` (D) `Ogrenci` – `Kisi` (E) `Kamyon` – `Tasit`

5 `Ogrenci` sınıfında, `Object` sınıfının `equals` metodunu **doğru şekilde ezen** (override) metot imzası hangisidir?

(A) `public boolean equals(Ogrenci o)`

(B) `private boolean equals(Object o)`

(C) `public int equals(Object o)`

(D) `public static boolean equals(Object o)`

(E) `public boolean equals(Object o)`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
class Arac {
    Arac() {
        System.out.println("Arac()");
    }

    Arac(String tur) {
        this();
        System.out.println("Arac(" + tur + ")");
    }
}

class Otomobil extends Arac {
    Otomobil() {
        super("oto");
        System.out.println("Otomobil()");
    }

    Otomobil(int kapi) {
        System.out.println("Otomobil(" + kapi + ")");
    }
}

class Taksi extends Otomobil {
    Taksi() {
        this(4);
        System.out.println("Taksi()");
    }

    Taksi(int kapi) {
        super(kapi);
        System.out.println("Taksi(" + kapi + ")");
    }
}

public class Soru6 {
    public static void main(String[] args) {
        new Taksi();
        System.out.println("---");
        new Otomobil();
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
class Tasit {
    String tur() { return "Taşıt"; }
    int tekerlek() { return 4; }
    String bilgi() { return tur() + "/" + tekerlek(); }
}

class Bisiklet extends Tasit {
    @Override
    String tur() { return "Bisiklet"; }

    @Override
    int tekerlek() { return super.tekerlek() - 2; }

    int tekerlek(int ekstra) { return tekerlek() + ekstra; }
}

public class Soru7 {
    public static void main(String[] args) {
        Tasit t = new Tasit();
        Bisiklet b = new Bisiklet();
        System.out.println(t.bilgi());
        System.out.println(b.bilgi());
        System.out.println(b.tekerlek(1) + t.tekerlek());
        System.out.println(b.tur().length() + t.tur().length());
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
class Para {
    int miktar;
    String birim;

    Para(int miktar, String birim) {
        this.miktar = miktar;
        this.birim = birim;
    }

    public boolean equals(Para p) {
        return miktar == p.miktar && birim.equals(p.birim);
    }

    @Override
    public String toString() {
        return miktar + " " + birim;
    }
}

public class Soru8 {
    public static void main(String[] args) {
        Para a = new Para(50, "TL");
        Para b = new Para(50, "TL");
        Object c = b;
        System.out.println(a == b);
        System.out.println(a.equals(b));
        System.out.println(a.equals(c));
        System.out.println(c.equals(b));
        System.out.println(a + " / " + c);
    }
}
```

C) Kod Yazma

9 Bir mağaza için ürün sınıflarını yaz:

- `Urun`: `private` alanlar `ad` (String) ve `fiyat` (double); yapıcı; `getFiyat()`; KDV'li fiyatı döndüren `kdvliFiyat()` (KDV %20); `toString` → `Kalem - 24.00 TL` (KDV'li fiyat, 2 basamak).
- `Kitap extends Urun`: ek alan `yazar`. Kitaplarda KDV %10 olduğu için `kdvliFiyat()`'i ezer. `toString`, üst sınıfın metnine `(yazar: ...)` ekler.
- `Elektronik extends Urun`: ek alan `garantiYil` (int). `toString`, üst sınıfın metnine `(garanti: ... yıl)` ekler.

`UrunTest` programı 20 TL'lik kalem, 150 TL'lik "Sefiller" (Victor Hugo) ve 1200 TL'lik, 2 yıl garantili kulaklık oluşturup hepsini ve toplam KDV'li tutarı yazdırsın:

```
Kalem - 24.00 TL
Sefiller - 165.00 TL (yazar: Victor Hugo)
Kulaklık - 1440.00 TL (garanti: 2 yıl)
Toplam: 1629.00 TL
```

10 Bir `Ders` sınıfı yaz: `private` alanlar `kod`, `ad` (String) ve `akts` (int). İki ders, **kodları aynıysa** eşit sayılsın (adları farklı olsa bile). `equals` ve `hashCode`'u doğru imzayla ez, `toString` → `[BLM101] Programlama I (6 AKTS)`. `DersTest` programında aşağıdaki karşılaştırmaları yap; ardından `{d1, d2, d3, d1}` dizisindeki **farklı** ders sayısını `equals` kullanarak bul (`d2`'nin kodu `d1` ile aynı, adı farklı; `d3` başka bir ders):

```
[BLM101] Programlama I (6 AKTS)
d1 == d2       : false
d1.equals(d2)  : true
d1.equals(d3)  : false
d1.equals(null): false
hashCode eşit  : true
Farklı ders sayısı: 2
```

Çözümler

Cevap anahtarı

1 D	2 A	3 B	4 C	5 E

1 Cevap: D

`final` bir sınıf kalıtılamaz (`String` buna örnektir). A yanlış: Java'da sınıflar için tekli kalıtım vardır. B yanlış: `private` alanlara alt sınıftan adıyla erişilemez. C yanlış: Yapıcılar kalıtılmaz, `super(...)` ile çağrılır. E yanlış: Ezerken erişim daraltılamaz, sadece genişletilebilir.

2 Cevap: A

(I), üst sınıftaki `yaz(int)` ile **aynı imzaya** sahiptir; parametre adının `x` yerine `y` olması önemsizdir → ezme. (II) ve (III)'ün parametre listeleri farklıdır → aşırı yükleme; aşırı yüklemede dönüş tipi serbesttir. Program derlenir ve çalışır:

Çıktı

```
B: 1
B: x
5
```

3 Cevap: B

`Otobus` yapıcısında `super(...)` yazılmadığı için derleyici ilk satıra `super();` ekler. `Arac`'ın tek yapıcısı `String` parametre istediğinden bu çağrının karşılığı yoktur:

Derleyici çıktısı – program derlenmez

```
Test3.java:12: error: constructor Arac in class Arac cannot be applied to given types;
    Otobus(int koltuk) {
                       ^
  required: String
  found:    no arguments
  reason: actual and formal argument lists differ in length
1 error
```

Düzeltmek için `Otobus(int koltuk) { super("34 ABC 01"); ... }` gibi bir `super(...)` çağrısı yazılmalı ya da `Arac`'a parametresiz yapıcı eklenmelidir.

4 Cevap: C

Cümle testi: "Kedi bir hayvandır", "Kare bir şekildir", "Öğrenci bir kişidir", "Kamyon bir taşıttır" doğrudur → kalıtım (is-a). "Araba bir motordur" yanlıştır; doğrusu "Arabanın bir motoru **vardır**" → has-a, yani kompozisyon: `Araba` sınıfında `private Motor motor;` alanı.

5 Cevap: E

Ezmek için imza ve dönüş tipi `Object`'tekiyle aynı olmalı, erişim daraltılmamalı: `public boolean equals(Object o)`. A derlenir ama **aşırı yüklemedir**; `equals(Object)` ezilmemiş olur (`EqualsTuzak` örneği). B, C ve D derlenmez; üçünü bir dosyada denediğimizde derleyici sırasıyla erişim daraltma, uyumsuz dönüş tipi ve `static` hatalarını verir:

Derleyici çıktısı – program derlenmez

```
Test5.java:5: error: equals(Object) in B cannot override equals(Object) in Object
class B { private boolean equals(Object o) { return true; } }
                          ^
  attempting to assign weaker access privileges; was public
Test5.java:7: error: equals(Object) in C cannot override equals(Object) in Object
class C { public int equals(Object o) { return 1; } }
                     ^
  return type int is not compatible with boolean
Test5.java:9: error: equals(Object) in D cannot override equals(Object) in Object
class D { public static boolean equals(Object o) { return true; } }
                                ^
  overriding method is static
3 errors
```

6

Çağrı ağacını çizelim:

```
new Taksi()
 └ Taksi()          ilk iş: this(4)
     └ Taksi(4)        ilk iş: super(4)
         └ Otomobil(4)    super() yazılmamış → örtük super()
             └ Arac()        → "Arac()"
             → "Otomobil(4)"
         → "Taksi(4)"
     → "Taksi()"

new Otomobil()
 └ Otomobil()       ilk iş: super("oto")
     └ Arac("oto")     ilk iş: this()
         └ Arac()         → "Arac()"
         → "Arac(oto)"
     → "Otomobil()"
```

Tuzak: `Otomobil(int)` yapıcısı `super(...)` yazmadığı için `Arac(String)` değil **parametresiz** `Arac()` çağrılır.

Çıktı

```
Arac()
Otomobil(4)
Taksi(4)
Taksi()
---
Arac()
Arac(oto)
Otomobil()
```

7

- `t.bilgi()` → `Taşıt/4`.
- `b.bilgi()`: `bilgi` metodu `Tasit`'te yazılı, ama nesne `Bisiklet` olduğu için içindeki `tur()` ve `tekerlek()` çağrıları **ezilmiş** versiyonlara gider: `"Bisiklet"` ve `4 - 2 = 2` → `Bisiklet/2`.
- `b.tekerlek(1)` aşırı yüklenmiş versiyondur: `tekerlek() + 1 = 2 + 1 = 3`; `t.tekerlek()` = 4 → `3 + 4 = 7`.
- `"Bisiklet".length()` = 8, `"Taşıt".length()` = 5 → `13`.

Çıktı
```
Taşıt/4
Bisiklet/2
7
13
```

8

- `a == b` → iki ayrı nesne: `false`.
- `a.equals(b)`: Argümanın tipi `Para` olduğu için aşırı yüklenmiş `equals(Para)` seçilir, alanlar karşılaştırılır: `true`.
- `a.equals(c)`: `c`'nin tipi `Object`; derleyici `Object`'ten gelen `equals(Object)`'i seçer, o da `==` yapar. `a` ile `c` farklı nesneler: `false`.
- `c.equals(b)`: `c` değişkeninin tipi `Object` olduğu için yine `equals(Object)` çağrılır, yani `c == b`. `c` ve `b` **aynı** nesneyi gösteriyor: `true`.
- `a + " / " + c`: `toString` doğru ezildiği için (`@Override`) `c` için de `Para`'nın versiyonu çalışır.

Çıktı
```
false
true
false
true
50 TL / 50 TL
```

Ders: `equals`'ı `Object` parametresiyle yazmazsan sonuç, değişkenin tipine göre değişen tutarsız bir davranış olur.

9

`Kitap`, `fiyat` alanına (`private`) doğrudan erişemediği için `getFiyat()` kullanır. `toString`'i `Urun`'de bir kez yazıp alt sınıflarda `super.toString()` ile genişletiyoruz. `Urun.toString` içindeki `kdvliFiyat()` çağrısı, nesne `Kitap` ise **`Kitap`'ın** versiyonuna gider; bu yüzden kitabın fiyatı %10 ile hesaplanır.

UrunTest.java
```
class Urun {
    private String ad;
    private double fiyat;

    Urun(String ad, double fiyat) {
        this.ad = ad;
        this.fiyat = fiyat;
    }

    double getFiyat() { return fiyat; }

    double kdvliFiyat() {
        return fiyat * 1.20;
    }

    @Override
    public String toString() {
        return String.format("%s – %.2f TL", ad, kdvliFiyat());
    }
}

class Kitap extends Urun {
    private String yazar;
```

```
    Kitap(String ad, double fiyat, String yazar) {
        super(ad, fiyat);
        this.yazar = yazar;
    }

    @Override
    double kdvliFiyat() {
        return getFiyat() * 1.10;
    }

    @Override
    public String toString() {
        return super.toString() + " (yazar: " + yazar + ")";
    }
}

class Elektronik extends Urun {
    private int garantiYil;

    Elektronik(String ad, double fiyat, int garantiYil) {
        super(ad, fiyat);
        this.garantiYil = garantiYil;
    }

    @Override
    public String toString() {
        return super.toString() + " (garanti: " + garantiYil + " yıl)";
    }
}

public class UrunTest {
    public static void main(String[] args) {
        Urun u = new Urun("Kalem", 20);
        Kitap k = new Kitap("Sefiller", 150, "Victor Hugo");
        Elektronik e = new Elektronik("Kulaklık", 1200, 2);
        System.out.println(u);
        System.out.println(k);
        System.out.println(e);
        double toplam = u.kdvliFiyat() + k.kdvliFiyat() + e.kdvliFiyat();
        System.out.printf("Toplam: %.2f TL%n", toplam);
    }
}
```

Çıktı

```
Kalem - 24.00 TL
Sefiller - 165.00 TL (yazar: Victor Hugo)
Kulaklık - 1440.00 TL (garanti: 2 yıl)
Toplam: 1629.00 TL
```

10

`equals` için bölümdeki dört adımlık kalıbı kullanıyoruz, ama sadece `kod` alanını karşılaştırıyoruz. `hashCode` da **aynı alanı** kullanmalı: `Objects.hash(kod)`. Farklı ders sayısı için her elemana, kendinden önceki elemanlar arasında `equals`'a göre eşiti olup olmadığını soruyoruz; yoksa sayaç artar.

DersTest.java

```
import java.util.Objects;

class Ders {
    private String kod;
    private String ad;
```

```
    private int akts;

    Ders(String kod, String ad, int akts) {
        this.kod = kod;
        this.ad = ad;
        this.akts = akts;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (o == null || getClass() != o.getClass()) {
            return false;
        }
        Ders diger = (Ders) o;
        return Objects.equals(kod, diger.kod);
    }

    @Override
    public int hashCode() {
        return Objects.hash(kod);
    }

    @Override
    public String toString() {
        return "[" + kod + "] " + ad + " (" + akts + " AKTS)";
    }
}

public class DersTest {
    public static void main(String[] args) {
        Ders d1 = new Ders("BLM101", "Programlama I", 6);
        Ders d2 = new Ders("BLM101", "Programlamaya Giriş", 6);
        Ders d3 = new Ders("MAT101", "Matematik I", 5);
        System.out.println(d1);
        System.out.println("d1 == d2       : " + (d1 == d2));
        System.out.println("d1.equals(d2)  : " + d1.equals(d2));
        System.out.println("d1.equals(d3)  : " + d1.equals(d3));
        System.out.println("d1.equals(null): " + d1.equals(null));
        System.out.println("hashCode eşit  : " + (d1.hashCode() == d2.hashCode()));

        Ders[] liste = {d1, d2, d3, d1};
        int farkli = 0;
        for (int i = 0; i < liste.length; i++) {
            boolean dahaOnceVar = false;
            for (int j = 0; j < i; j++) {
                if (liste[j].equals(liste[i])) {
                    dahaOnceVar = true;
                    break;
                }
            }
            if (!dahaOnceVar) {
                farkli++;
            }
        }
        System.out.println("Farklı ders sayısı: " + farkli);
    }
}
```

Çıktı

```
[BLM101] Programlama I (6 AKTS)
d1 == d2       : false
d1.equals(d2)  : true
d1.equals(d3)  : false
d1.equals(null): false
hashCode eşit  : true
Farklı ders sayısı: 2
```

BÖLÜM 11

Polimorfizm, Soyut Sınıflar ve Arayüzler

Bu Bölümde Öğreneceklerin

- Polimorfizmin (çok biçimlilik) ne olduğunu ve neden nesneye yönelik programlamanın kalbi sayıldığını
- Üst sınıf referansıyla alt sınıf nesnesi tutmayı: yukarı ve aşağı dönüşüm (upcasting, downcasting)
- **Dinamik bağlama** kuralını: derleyici referans tipine, JVM nesne tipine bakar
- `instanceof`, pattern matching ve `ClassCastException`'ı
- Soyut sınıf (abstract class) ve arayüz (interface) yazmayı, ikisinin farkını
- `enum` ile sabit değer kümeleri, `record` ile kısa veri sınıfları tanımlamayı

11.1 Polimorfizm nedir?

Bölüm 10'da `Kedi extends Hayvan` yazarak "her kedi bir hayvandır" (**is-a**) ilişkisini kurduk ve alt sınıfın üst sınıftaki metodu **ezmesini** (override) gördük. Bu bölüm o ilişkinin en güçlü sonucunu anlatıyor: Bir `Hayvan` değişkeni bir `Kedi` nesnesini tutabilir ve ona `sesCikar()` dediğinde hayvan **kedi gibi** ses çıkarır.

Polimorfizm (polymorphism, çok biçimlilik): Aynı metot çağrısının, nesnenin **gerçek tipine** göre farklı davranmasıdır. Tek bir `Hayvan` değişkeni bazen kedi, bazen köpek "biçimine" girer.

Bu bölümdeki sınıflar çok sayıda kısa metot içerdiği için, sınav kâğıtlarında olduğu gibi tek satırlık metotları **tek satırda** yazacağız: `String ad() { return "Kare"; }`.

Polimorfizm.java

```
class Hayvan {
    protected String ad;

    Hayvan(String ad) { this.ad = ad; }

    void sesCikar() { System.out.println(ad + ": ..."); }
}

class Kedi extends Hayvan {
    Kedi(String ad) { super(ad); }

    @Override
    void sesCikar() { System.out.println(ad + ": Miyav"); }
}

class Kopek extends Hayvan {
    Kopek(String ad) { super(ad); }

    @Override
    void sesCikar() { System.out.println(ad + ": Hav hav"); }
}

public class Polimorfizm {
```

```
    static void konustur(Hayvan h) {    // tek metot, her türlü hayvanla çalışır
        h.sesCikar();
    }

    public static void main(String[] args) {
        Hayvan h1 = new Kedi("Tekir");
        Hayvan h2 = new Kopek("Karabaş");
        Hayvan h3 = new Hayvan("Bilinmeyen");

        h1.sesCikar();
        h2.sesCikar();
        h3.sesCikar();
        konustur(new Kedi("Pamuk"));
    }
}
```

Çıktı

```
Tekir: Miyav
Karabaş: Hav hav
Bilinmeyen: ...
Pamuk: Miyav
```

Üç değişkenin tipi de `Hayvan`, üç satırda da aynı `sesCikar()` çağrısı var; ama üç farklı sonuç çıktı. `konustur` metodu ise parametre olarak `Hayvan` alıyor ve yarın yazacağın `Kus`, `At` gibi yeni alt sınıflarla da **hiç değiştirilmeden** çalışacak. Polimorfizmin asıl kazancı budur: Kodu üst sınıfa göre yazarsın, her alt sınıf kendi davranışını getirir.

11.2 Yukarı dönüşüm (upcasting)

`Hayvan h = new Kedi("Tekir");` satırında iki ayrı tip vardır. Bu ayrımı oturtmadan bu bölümdeki hiçbir çıktı sorusunu çözemezsin:

Kavram	Nerede yazar?	Örnekte	Neyi belirler?
Referans tipi (değişkenin tipi)	`=` işaretinin solunda	`Hayvan`	Hangi metotları **çağırabileceğini** (derleme zamanı)
Nesne tipi (gerçek tip)	`new`'den sonra	`Kedi`	Çağrılan metodun **hangi sürümünün** çalışacağını (çalışma zamanı)

Alt sınıf nesnesini üst sınıf referansına atamaya **yukarı dönüşüm** (upcasting) denir. Her kedi bir hayvan olduğu için güvenlidir ve **otomatik** yapılır; cast yazmana gerek yoktur. Bölüm 10'da gördüğün gibi her sınıf `Object`'ten türediği için `Object o = new Kedi("Tekir");` de geçerlidir. Tersi ise geçerli değildir, çünkü her hayvan bir kedi değildir: `Kedi k = new Hayvan("x");` satırı *incompatible types: Hayvan cannot be converted to Kedi* hatasıyla derlenmez.

11.3 Dinamik bağlama: hangi metot çalışır?

`h.sesCikar()` gibi bir çağrıda Java iki ayrı soru sorar:

1. **Derleme zamanında** (compile time): Derleyici sadece **referans tipine** bakar: "`Hayvan` sınıfında `sesCikar()` diye bir metot var mı?" Yoksa program derlenmez.
2. **Çalışma zamanında** (runtime): JVM nesnenin **gerçek tipine** bakar ve o sınıftaki sürümü çalıştırır. O sınıf metodu ezmemişse bir üst sınıfa çıkar, bulana kadar devam eder.

Çağrılacak metodun program çalışırken, nesneye bakılarak seçilmesine **dinamik bağlama** (dynamic binding, geç bağlama) denir. Kural, metot **üst sınıfın içinden** çağrıldığında bile geçerlidir. Sınavların en sevdiği tuzak budur:

DinamikBaglama.java

```java
class Calisan {
    protected String ad;
    protected double taban = 30000;

    Calisan(String ad) { this.ad = ad; }

    double maas() { return taban; }
    String unvan() { return "Çalışan"; }

    void bordro() {   // sadece burada tanımlı; alt sınıflar ezmiyor
        System.out.printf("%-5s %-8s %9.2f%n", ad, unvan(), maas());
    }
}

class Mudur extends Calisan {
    Mudur(String ad) { super(ad); }

    @Override double maas() { return super.maas() * 2; }
    @Override String unvan() { return "Müdür"; }
}

class Stajyer extends Calisan {
    Stajyer(String ad) { super(ad); }

    @Override double maas() { return taban / 3; }
}

public class DinamikBaglama {
    public static void main(String[] args) {
        Calisan c1 = new Calisan("Ali");
        Calisan c2 = new Mudur("Ayşe");
        Calisan c3 = new Stajyer("Can");
        c1.bordro();
        c2.bordro();
        c3.bordro();
    }
}
```

Çıktı

```
Ali   Çalışan   30000.00
Ayşe  Müdür     60000.00
Can   Çalışan   10000.00
```

`bordro()` metodu sadece `Calisan` sınıfında var. Ama `c2.bordro()` çalışırken içindeki `unvan()` ve `maas()` çağrıları aslında `this.unvan()` ve `this.maas()` demektir; `this` ise bir `Mudur` nesnesidir. Bu yüzden müdürün sürümleri çalışır. `Stajyer` sadece `maas()`'ı ezdiği için unvanı `Calisan`'dan gelir. `super.maas()` ise dinamik bağlamayı atlar ve **doğrudan üst sınıfın** sürümünü çağırır.

> **Sınavda Çıkar: İki soru, iki tip**
>
> Polimorfizm sorularında her satır için iki soru sor:
>
> - **"Derlenir mi?" → Referans tipine** bak. Metot o sınıfta (veya üstlerinde) yoksa derleme hatası.
> - **"Ne yazdırır?" → Nesne tipine** bak. `new` ile hangi sınıf oluşturulduysa, metot aramaya o sınıftan başlanır.
>
> Üst sınıftaki bir metodun içinden çağrılan metotlar da nesne tipine göre seçilir. Bunu unutan öğrenci `Çalışan` yazar, doğrusu `Müdür`'dür.

Referans tipinde olmayan metot çağrılamaz

Nesne gerçekten bir kedi olsa bile, değişkenin tipi `Hayvan` ise sadece `Hayvan`'da bulunan metotları çağırabilirsin:

EksikMetot.java

```
class Hayvan {
    void sesCikar() { System.out.println("..."); }
}

class Kedi extends Hayvan {
    void tirmala() { System.out.println("Ağaca tırmandı"); }
}

public class EksikMetot {
    public static void main(String[] args) {
        Hayvan h = new Kedi();
        h.sesCikar();
        h.tirmala();
    }
}
```

Derleyici çıktısı – program derlenmez

```
EksikMetot.java:13: error: cannot find symbol
        h.tirmala();
         ^
  symbol:   method tirmala()
  location: variable h of type Hayvan
1 error
```

Derleyici `h`'nin içinde gerçekten ne olduğuna bakmaz (çoğu zaman bilemez de; nesne bir metoda parametre olarak gelmiş olabilir). Sadece `h`'nin tipi olan `Hayvan` sınıfına bakar ve orada `tirmala()` bulamaz. Çözüm, birazdan göreceğin aşağı dönüşümdür.

Overload derleme zamanında seçilir

Override (ezme) ile overload (aşırı yükleme: aynı isimli, farklı parametreli metotlar) sık karıştırılır. Aralarındaki fark burada ortaya çıkar: **Hangi overload'un çağrılacağına derleyici, argümanın referans tipine bakarak karar verir.** Nesne tipine bakılmaz.

OverloadTuzak.java

```
class Hayvan { }

class Kedi extends Hayvan { }

public class OverloadTuzak {
    static void tanit(Hayvan h) { System.out.println("Bir hayvan"); }
    static void tanit(Kedi k) { System.out.println("Bir kedi"); }

    public static void main(String[] args) {
        Hayvan h = new Kedi();
        Kedi k = new Kedi();
        tanit(h);
        tanit(k);
        tanit((Kedi) h);
    }
}
```

Çıktı

```
Bir hayvan
Bir kedi
Bir kedi
```

`h` bir kedi nesnesini tuttuğu hâlde ilk satır `Bir hayvan` yazdırdı, çünkü `h`'nin referans tipi `Hayvan`'dır. Özetle: **override → çalışma zamanı, nesne tipi; overload → derleme zamanı, referans tipi.**

11.4 Aşağı dönüşüm (downcasting)

Üst sınıf referansındaki nesneyi tekrar alt sınıf referansına almak için **aşağı dönüşüm** (downcasting) yapılır. Bu işlem otomatik değildir, çünkü her hayvan kedi değildir; parantez içinde hedef tipi yazarak derleyiciye "Ben eminim, bu bir kedi" dersin. Peki yanılıyorsan? Derleyici cast'e güvenip programı derler; ama çalışırken JVM nesnenin gerçek tipini kontrol eder ve uymuyorsa `ClassCastException` fırlatır:

AsagiDonusum.java

```java
class Hayvan { }

class Kedi extends Hayvan {
    void tirmala() { System.out.println("Ağaca tırmandı"); }
}

class Kopek extends Hayvan { }

public class AsagiDonusum {
    public static void main(String[] args) {
        Hayvan h1 = new Kedi();     // yukarı dönüşüm: otomatik
        Kedi k1 = (Kedi) h1;        // aşağı dönüşüm: cast şart
        k1.tirmala();
        ((Kedi) h1).tirmala();      // tek satırda da olur

        Hayvan h2 = new Kopek();
        Kedi k2 = (Kedi) h2;        // derlenir ama nesne kedi değil!
        System.out.println("Bu satıra gelinmez.");
    }
}
```

Çıktı – program hata ile durur

```
Ağaca tırmandı
Ağaca tırmandı
Exception in thread "main" java.lang.ClassCastException: class Kopek cannot be cast to class Kedi
(Kopek and Kedi are in unnamed module of loader 'app')
        at AsagiDonusum.main(AsagiDonusum.java:17)
```

Sınavda Çıkar: Derleme hatası mı, ClassCastException mı?

`Kedi` ve `Kopek`, `Hayvan`'ın iki alt sınıfı olsun:

Kod	Sonuç
`Hayvan h = new Kedi();`	Derlenir, çalışır (yukarı dönüşüm)
`Kedi k = new Hayvan();`	**Derleme hatası** (cast yok)
`Kedi k = (Kedi) new Hayvan();`	Derlenir, çalışırken **ClassCastException**
`Hayvan h = new Kopek(); Kedi k = (Kedi) h;`	Derlenir, çalışırken **ClassCastException**
`Kopek kp = new Kopek(); Kedi k = (Kedi) kp;`	**Derleme hatası** (kardeş sınıflar arasında cast olmaz)

Kural: Derleyici sadece "bu cast **olabilir mi**?" diye bakar. `Hayvan` referansı bir kediyi tutuyor **olabilir**, o yüzden derler. `Kopek` referansı ise asla bir kedi tutamaz; derleyici bunu bilir ve derlemez.

11.5 instanceof ve pattern matching

Cast'ten önce nesnenin gerçekten o tipte olup olmadığını **`instanceof`** operatörüyle sorabilirsin. `nesne instanceof Tip` ifadesi, nesne o sınıftansa **veya o sınıfın bir alt sınıfındansa** `true` döner. Java 16'dan beri kontrol ve cast tek adımda yapılabilir: `if (h instanceof Kedi k)` kontrol başarılıysa `k` adında, zaten cast edilmiş bir değişken oluşturur. Buna **pattern matching** (desen eşleme) denir.

Aşağıdaki örnekte `Hayvan`, `Kedi` ve `Kopek` sınıfları bölümün ilk örneğindeki gibidir; tek fark, kedinin `tirmala()`, köpeğin `kemikGetir()` metoduna sahip olması. Sadece `main`'in içini gösteriyoruz:

```
Hayvan[] hayvanlar = {
    new Kedi("Tekir"), new Kopek("Karabaş"), new Kedi("Pamuk")
};
for (Hayvan h : hayvanlar) {
    if (h instanceof Kedi k) {          // Java 16+: kontrol + cast
        k.tirmala();
    } else if (h instanceof Kopek) {    // klasik yol
        Kopek kp = (Kopek) h;
        kp.kemikGetir();
    }
}

Hayvan ilk = hayvanlar[0];
System.out.println(ilk instanceof Hayvan);
System.out.println(ilk instanceof Object);
System.out.println(ilk instanceof Kopek);
Hayvan bos = null;
System.out.println(bos instanceof Hayvan);
```

Çıktı

```
Tekir ağaca tırmandı
Karabaş kemiği getirdi
Pamuk ağaca tırmandı
true
true
false
false
```

Son satıra dikkat: `null instanceof Herhangi` her zaman **`false`** döner, hata vermez. Bu yüzden `instanceof` kontrolü aynı zamanda null kontrolü yerine de geçer.

> **İpucu: instanceof zinciri bir uyarı işaretidir**
>
> Kodunda `if (x instanceof A) ... else if (x instanceof B) ... else if (x instanceof C)` gibi uzun zincirler varsa, polimorfizmi kullanmıyorsun demektir. Ortak davranışı üst sınıfa bir metot olarak koy, her alt sınıf kendi sürümünü yazsın; zincir tek bir `x.metot()` çağrısına dönüşür. `instanceof`'u sadece **alt sınıfa özgü** bir şey yapman gerektiğinde kullan.

11.6 Polimorfik diziler

Polimorfizmin en çok işe yaradığı yer dizilerdir. Bir `Sekil[]` dizisinin her elemanı daire, dikdörtgen veya kare olabilir; döngü hangisinin ne olduğunu bilmeden hepsinin alanını hesaplar:

PolimorfikDizi.java

```
class Sekil {
    double alan() { return 0; }
    String ad() { return "Şekil"; }
}

class Daire extends Sekil {
    private double r;

    Daire(double r) { this.r = r; }

    @Override double alan() { return Math.PI * r * r; }
    @Override String ad() { return "Daire"; }
}

class Dikdortgen extends Sekil {
    private double en, boy;

    Dikdortgen(double en, double boy) { this.en = en; this.boy = boy; }

    @Override double alan() { return en * boy; }
    @Override String ad() { return "Dikdörtgen"; }
}

class Kare extends Dikdortgen {
    Kare(double kenar) { super(kenar, kenar); }

    @Override String ad() { return "Kare"; }
}

public class PolimorfikDizi {
    public static void main(String[] args) {
        Sekil[] sekiller = {
            new Daire(1), new Dikdortgen(3, 4), new Kare(2), new Daire(2.5)
        };
        double toplam = 0;
        Sekil enBuyuk = sekiller[0];
        for (Sekil s : sekiller) {
            System.out.printf("%-11s %7.2f%n", s.ad(), s.alan());
            toplam += s.alan();
            if (s.alan() > enBuyuk.alan()) {
                enBuyuk = s;
            }
        }
        System.out.printf("Toplam alan: %.2f%n", toplam);
        System.out.println("En büyük: " + enBuyuk.ad());
    }
}
```

```
Çıktı
Daire          3.14
Dikdörtgen    12.00
Kare           4.00
Daire         19.63
Toplam alan: 38.78
En büyük: Daire
```

`Kare` alanı hesaplamayı `Dikdortgen`'den miras aldı, sadece adını değiştirdi. Programa yarın `Ucgen` sınıfı eklersen `main`'deki döngüde **tek bir harf** değiştirmen gerekmez.

Not: Her kare dikdörtgendir, ama `Kare` bir `Dikdortgen` midir?

Matematikte her kare bir dikdörtgendir, o yüzden `Kare extends Dikdortgen` yazdık. Ancak kalıtım bir **davranış sözüdür**: "`Dikdortgen`'in kullanıldığı her yerde `Kare` de sorunsuz kullanılabilir" (*Liskov yerine koyma ilkesi*). "Matematiksel is-a" ile "davranışsal is-a" her zaman aynı şey değildir. Dikdörtgenin alanları **değiştirilebilir** olsaydı bu söz bozulurdu:

KareSorunu.java

```java
class Dikdortgen {
    protected double en, boy;           // değiştirilebilir alanlar

    Dikdortgen(double en, double boy) { this.en = en; this.boy = boy; }

    void setEn(double en) { this.en = en; }
    void setBoy(double boy) { this.boy = boy; }
    double alan() { return en * boy; }
}

class Kare extends Dikdortgen {
    Kare(double kenar) { super(kenar, kenar); }

    @Override void setEn(double en) { this.en = en; this.boy = en; }
    @Override void setBoy(double boy) { this.en = boy; this.boy = boy; }
}

public class KareSorunu {
    public static void main(String[] args) {
        Dikdortgen d = new Dikdortgen(3, 3);
        d.setEn(5);
        d.setBoy(2);
        System.out.println(d.alan());

        Dikdortgen k = new Kare(3);       // "her kare bir dikdörtgendir"
        k.setEn(5);
        k.setBoy(2);
        System.out.println(k.alan());
    }
}
```

```
Çıktı
10.0
4.0
```

> Aynı `setEn(5)` ve `setBoy(2)` çağrıları `Dikdortgen` için 10 verirken `Kare` için 4 verir: `Kare` kare kalabilmek için `setEn`'in boyu da değiştirmesi gerekir. Bu bölümdeki sınıflarda alanlar `private` ve **değiştirilemez** (set metodu yok), bu yüzden sorun çıkmaz. Alanlar değiştirilebilir olacaksa `Kare`'yi `Dikdortgen`'den türetme; ikisini ortak bir üst tipin (`Sekil`) kardeşi yap.

Yalnız `Sekil` tasarımında bir tuhaflık var: `Sekil` sınıfının `alan()` metodu `0` döndürüyor. "Şekil" soyut bir kavramdır; alanı sıfır değil, **tanımsızdır**. Ayrıca kimse `new Sekil()` yazıp anlamsız bir nesne oluşturmamalı. İşte bu ihtiyaç, soyut sınıfları doğurur.

11.7 Soyut sınıflar (abstract class)

Soyut sınıf (abstract class), kendisinden nesne üretilmeyen, sadece **kalıtılmak için** var olan sınıftır. İçinde **soyut metotlar** (abstract method) bulunabilir: Sadece imzası yazılan, gövdesi olmayan metotlar. Soyut metot, "her alt sınıf bu metodu **yazmak zorunda**" demektir. `Sekil`'i soyut yapıp bir de `cevre()` metodu ekleyelim (`Dikdortgen` sınıfı `Daire` ile aynı kalıpta yazıldığı için gösterilmedi):

```
abstract class Sekil {
    private String ad;

    Sekil(String ad) { this.ad = ad; }    // soyut sınıfın da yapıcısı olur

    abstract double alan();               // gövde yok, sonunda ; var
    abstract double cevre();

    void bilgi() {                        // somut (gövdeli) metot
        System.out.printf("%-10s alan=%6.2f  çevre=%6.2f%n", ad, alan(), cevre());
    }
}

class Daire extends Sekil {
    private double r;

    Daire(double r) { super("Daire"); this.r = r; }

    @Override double alan() { return Math.PI * r * r; }
    @Override double cevre() { return 2 * Math.PI * r; }
}

// ...

Sekil[] sekiller = {new Daire(1.5), new Dikdortgen(2, 5)};
for (Sekil s : sekiller) {
    s.bilgi();
}
```

Çıktı

```
Daire      alan=  7.07  çevre=  9.42
Dikdörtgen alan= 10.00  çevre= 14.00
```

Soyut sınıfın kuralları:

- `abstract` metodun gövdesi olmaz: `abstract double alan();` (süslü parantez yok, noktalı virgül var).
- İçinde **en az bir** soyut metot olan sınıf `abstract` olmak **zorundadır**. (Tersi gerekmez: hiç soyut metodu olmayan bir sınıf da `abstract` yapılabilir.)
- Soyut sınıfın **alanları, yapıcısı ve gövdeli metotları** olabilir. Yapıcı, alt sınıfların `super(...)` çağrısıyla çalışır.

- Soyut olmayan bir alt sınıf, üst sınıftaki **bütün** soyut metotları ezmek zorundadır. Ezmeyecekse kendisi de `abstract` olmalıdır.
- Soyut sınıf türünde **referans** tanımlanabilir (`Sekil s`), ama soyut sınıftan **nesne** üretilemez:

SoyutNesne.java

```
abstract class Sekil {
    abstract double alan();
}

public class SoyutNesne {
    public static void main(String[] args) {
        Sekil s = new Sekil();
    }
}
```

Derleyici çıktısı – program derlenmez

```
SoyutNesne.java:7: error: Sekil is abstract; cannot be instantiated
        Sekil s = new Sekil();
                  ^
1 error
```

Sık Yapılan Hata: Soyut metodu ezmeyi unutmak

Alt sınıf, üst sınıftaki soyut metotlardan birini bile yazmayı unutursa program derlenmez. `Sekil`'in `alan()` ve `cevre()` metotları varken `Kare` sadece `alan()`'ı yazarsa derleyici şunu söyler: *Kare is not abstract and does not override abstract method cevre() in Sekil*. Yani "`Kare` soyut değil ve `cevre()`'yi ezmiyor." Ya `cevre()`'yi yaz ya da `Kare`'yi `abstract` yap.

Sınavda Çıkar: Soyut metotla birlikte kullanılamayanlar

Soyut metot alt sınıfta **ezilmek** için vardır. Bu yüzden ezilmeyi engelleyen belirteçlerle birlikte yazılamaz: `abstract` metot `private`, `static` veya `final` **olamaz** (*illegal combination of modifiers*). Aynı mantıkla `abstract final class` da yazılamaz: final sınıf kalıtılamaz, soyut sınıf ise kalıtılmak zorundadır.

11.8 Arayüzler (interface)

Soyut sınıf "**ne olduğunu**" anlatır: Daire bir şekildir. Bazen ise ortak olan şey kimlik değil, **yetenektir**: Ördek uçabilir, uçak da uçabilir; ama uçak bir hayvan değildir. Böyle bir "yapabilir" sözleşmesi için **arayüz** (interface) kullanılır.

Arayüz, bir sınıfın **hangi metotları sağlayacağını** söyleyen bir sözleşmedir. Bir sınıf arayüzü `implements` ile uygular ve içindeki bütün soyut metotları yazmak zorundadır. Bir sınıf sadece **bir** sınıftan `extends` yapabilir ama **istediği kadar** arayüzü `implements` edebilir:

ArayuzOrnek.java

```
interface Ucabilir {
    void uc();                    // otomatik olarak public abstract
}

interface Yuzebilir {
    void yuz();
}

class Hayvan {
    protected String ad;
```

```
    Hayvan(String ad) { this.ad = ad; }
}

class Ordek extends Hayvan implements Ucabilir, Yuzebilir {
    Ordek(String ad) { super(ad); }

    @Override
    public void uc() { System.out.println(ad + " kanat çırparak uçuyor"); }

    @Override
    public void yuz() { System.out.println(ad + " gölde yüzüyor"); }
}

class Ucak implements Ucabilir {  // hayvan değil, ama uçabilir
    private String kod;

    Ucak(String kod) { this.kod = kod; }

    @Override
    public void uc() { System.out.println(kod + " 10.000 metrede uçuyor"); }
}

public class ArayuzOrnek {
    public static void main(String[] args) {
        Ucabilir[] ucanlar = {new Ordek("Donald"), new Ucak("TC-JAN")};
        for (Ucabilir u : ucanlar) {
            u.uc();
        }
        Yuzebilir y = new Ordek("Vakvak");   // ördek aynı zamanda Yuzebilir
        y.yuz();
    }
}
```

Çıktı

```
Donald kanat çırparak uçuyor
TC-JAN 10.000 metrede uçuyor
Vakvak gölde yüzüyor
```

Arayüz de bir **tiptir**: `Ucabilir` türünde bir değişken veya dizi tanımlayabilir, içine onu uygulayan her sınıfın nesnesini koyabilirsin. Ama arayüzden de nesne üretilemez: `new Ucabilir()` derlenmez. Polimorfizm burada da aynen geçerlidir: `u.uc()` çağrısında çalışan sürümü nesnenin gerçek tipi belirler.

● Sık Yapılan Hata: Arayüz metodunu public yazmamak

Arayüzdeki metotlar otomatik olarak `public`'tir. Sınıfta bu metodu yazarken `public`'i unutursan, erişimi **daraltmış** olursun ve Java buna izin vermez (Bölüm 10: ezilen metodun erişimi daraltılamaz):

```java
interface Ucabilir {
    void uc();
}

class Kus implements Ucabilir {
    void uc() { System.out.println("Kuş uçuyor"); }
}

public class ZayifErisim {
    public static void main(String[] args) {
        new Kus().uc();
    }
}
```

Derleyici çıktısı – program derlenmez

```
ZayifErisim.java:6: error: uc() in Kus cannot implement uc() in Ucabilir
    void uc() { System.out.println("Kuş uçuyor"); }
         ^
  attempting to assign weaker access privileges; was public
1 error
```

Sabitler, default ve static metotlar

Arayüzde soyut metotlar dışında başka üyeler de bulunabilir:

Üye	Sen yazarsın	Derleyici şöyle anlar
Sabit	`int MAX = 100;`	`public static final int MAX = 100;`
Soyut metot	`void uc();`	`public abstract void uc();`
`default` metot (Java 8+)	`default void f() { ... }`	Gövdeli, `public`; uygulayan sınıf isterse ezer
`static` metot (Java 8+)	`static int g() { ... }`	Sadece `ArayuzAdi.g()` şeklinde çağrılır
`private` metot (Java 9+)	`private void h() { ... }` ya da `private static int k() { ... }`	Arayüzün kendi metotlarına yardımcı

`default` metot, arayüze gövdeli bir metot eklemeyi sağlar. Uygulayan sınıflar bu metodu hazır olarak kullanır; isteyen ezer. Böylece bir arayüze yeni bir metot eklendiğinde, onu uygulayan yüzlerce sınıfın hepsini değiştirmek gerekmez.

ArayuzUyeleri.java

```java
interface Sarjli {
    int TAM_DOLU = 100;                    // public static final

    int seviye();                          // soyut

    default boolean sarjGerekli() {        // gövdeli, miras alınır
        return seviye() < 20;
    }

    default String durum() {
        return "%" + seviye() + (sarjGerekli() ? " (şarj et!)" : "");
    }
```

```
    static int yuzde(int mah, int kapasite) {  // Sarjli.yuzde(...) ile çağrılır
        return mah * TAM_DOLU / kapasite;
    }
}

class Telefon implements Sarjli {
    private int mah;

    Telefon(int mah) { this.mah = mah; }

    @Override
    public int seviye() { return Sarjli.yuzde(mah, 5000); }
}

public class ArayuzUyeleri {
    public static void main(String[] args) {
        Sarjli t1 = new Telefon(4200);
        Sarjli t2 = new Telefon(600);
        System.out.println(t1.durum());
        System.out.println(t2.durum());
        System.out.println(t2.sarjGerekli());
        System.out.println(Sarjli.TAM_DOLU + " " + Sarjli.yuzde(1, 4));
    }
}
```

Çıktı

```
%84
%12 (şarj et!)
true
100 25
```

`Telefon` sınıfı sadece soyut `seviye()` metodunu yazdı; `durum()` ve `sarjGerekli()` metotlarını arayüzden hazır aldı. İsteseydi bu default metotları ezebilirdi (Soru 8'de bir örneğini göreceksin). Arayüzdeki sabit `final` olduğu için `Sarjli.TAM_DOLU = 50;` gibi bir atama derlenmez (*cannot assign a value to static final variable*).

11.9 Soyut sınıf mı, arayüz mü?

Bu karşılaştırma, vize ve finallerin klasik "açıklayınız" sorusudur:

Özellik	Soyut sınıf	Arayüz
Tanım / kullanım	`abstract class A` / `extends`	`interface A` / `implements`
Kaç tane?	Bir sınıf sadece **bir** sınıfı `extends` edebilir	Bir sınıf **birden çok** arayüzü `implements` edebilir
Nesne üretilebilir mi?	Hayır	Hayır
Yapıcı (constructor)	Olabilir	**Olamaz**
Alanlar	Her türlü (nesne alanı, `private`, `protected`...)	Sadece `public static final` sabitler
Metotlar	Soyut + gövdeli, her erişim belirteci	Soyut + `default` + `static` + `private`
Metotların erişimi	Sen belirlersin	Erişim belirteci yazılmayan metotlar `public`'tir
İlişki	"**is-a**" (bir ...dır): Daire bir şekildir	"**can-do**" (...yapabilir): Ördek uçabilir
Ne zaman?	Ortak **durum** (alan) ve kod paylaşan yakın akraba sınıflar	İlgisiz sınıflara ortak bir **yetenek** kazandırmak

> **Sınavda Çıkar: Neden sınıflarda çoklu kalıtım yok?**
>
> Java'da `class C extends A, B` yazılamaz. Sebep "**elmas problemi**" (diamond problem): `A` ve `B`'de aynı imzalı iki farklı metot ve aynı isimli alanlar olsaydı, `C` hangisini miras alacaktı? Arayüzlerde **durum** (nesne alanı) olmadığı için çoklu uygulama güvenlidir. İki arayüzde aynı imzalı iki `default` metot varsa da Java karar vermez; sınıf o metodu **ezmek zorundadır**, aksi halde derleme hatası olur.

11.10 Comparable'a kısa bir bakış

Java'nın kendi kütüphanesi de arayüzlerle doludur. En meşhurlarından biri **`Comparable`**'dır: "Bu sınıfın nesneleri birbiriyle **karşılaştırılabilir**" sözleşmesi. Bölüm 7'de `Arrays.sort` ile sayı dizilerini sıraladın; kendi sınıfının nesnelerini sıralamak istiyorsan, sınıf `Comparable` arayüzünü uygulamalı ve `compareTo` metodunu yazmalıdır:

ComparableOrnek.java

```
import java.util.Arrays;

class Ogrenci implements Comparable<Ogrenci> {
    private String ad;
    private int puan;

    Ogrenci(String ad, int puan) { this.ad = ad; this.puan = puan; }

    @Override
    public int compareTo(Ogrenci diger) {
        return Integer.compare(this.puan, diger.puan);
    }

    @Override
    public String toString() { return ad + "(" + puan + ")"; }
}

public class ComparableOrnek {
    public static void main(String[] args) {
        Ogrenci[] sinif = {
            new Ogrenci("Elif", 85), new Ogrenci("Can", 62),
            new Ogrenci("Deniz", 91)
        };
        Arrays.sort(sinif);     // compareTo'yu kullanarak sıralar
        System.out.println(Arrays.toString(sinif));
        System.out.println(sinif[0].compareTo(sinif[2]));
    }
}
```

Çıktı

```
[Can(62), Elif(85), Deniz(91)]
-1
```

`a.compareTo(b)` sözleşmesi: `a` önce gelmeliyse **negatif**, eşitse **0**, sonra gelmeliyse **pozitif** bir sayı döndür. `Integer.compare(x, y)` bunu bizim için yapar: Can'ın 62'si Deniz'in 91'inden küçük olduğu için `-1` döndü. `Comparable<Ogrenci>` yazımındaki köşeli parantezler **generics**'tir; ayrıntısını, `Comparator` ile farklı sıralamaları ve koleksiyonları Bölüm 13'te göreceksin. Bölüm 8'deki `String`'in `compareTo` metodu da aslında bu arayüzden gelir.

11.11 enum: sabit değer kümeleri

Haftanın günlerini `int gun = 3;` diye tutarsan iki sorun çıkar: `3`'ün hangi gün olduğu belli değildir ve kimse seni `gun = 42;` yazmaktan alıkoyamaz. **`enum`** (enumeration, sayım tipi), alabileceği değerler **önceden sayılmış** bir tip tanımlar:

EnumOrnek.java

```
enum Gun {
    PAZARTESI, SALI, CARSAMBA, PERSEMBE, CUMA, CUMARTESI, PAZAR
}

public class EnumOrnek {
    static String tur(Gun g) {
        return switch (g) {
            case CUMARTESI, PAZAR -> "hafta sonu";   // Gun. yazılmaz
            default -> "hafta içi";
        };
    }

    public static void main(String[] args) {
        Gun bugun = Gun.CUMA;
        System.out.println(bugun);
        System.out.println(bugun.ordinal());
        System.out.println(bugun == Gun.CUMA);
        System.out.println(Gun.valueOf("SALI").ordinal());
        System.out.println(bugun.compareTo(Gun.SALI));
        System.out.println(tur(bugun) + ", " + tur(Gun.PAZAR));

        for (Gun g : Gun.values()) {
            System.out.print(g.name().charAt(0));
        }
        System.out.println(" -> " + Gun.values().length + " gün");
    }
}
```

Çıktı

```
CUMA
4
true
1
3
hafta içi, hafta sonu
PSCPCCP -> 7 gün
```

Metot / işlem	Ne yapar?	Örnek → Sonuç
`values()`	Bütün sabitleri tanım sırasıyla dizi olarak verir	`Gun.values().length` → `7`
`ordinal()`	Sabitin sıra numarası (**0'dan** başlar)	`Gun.CUMA.ordinal()` → `4`
`name()`	Sabitin koddaki adı (`final`, ezilemez)	`Gun.CUMA.name()` → `"CUMA"`
`toString()`	Varsayılan olarak `name()` ile aynı; ezilebilir	`Gun.CUMA.toString()` → `"CUMA"`
`valueOf(String)`	Addan sabiti bulur	`Gun.valueOf("SALI")` → `SALI`
`compareTo(...)`	Sıra numaralarının farkı	`CUMA.compareTo(SALI)` → `3`
`==`	Karşılaştırma (her sabitten tek nesne vardır)	`bugun == Gun.CUMA` → `true`

`switch` içinde `case` etiketlerine sadece sabitin adı yazılır (`case CUMARTESI`), `Gun.CUMARTESI` yazılmaz. `valueOf` büyük/küçük harfe duyarlıdır: `Gun.valueOf("Cuma")` çalışırken *No enum constant Gun.Cuma* mesajıyla `IllegalArgumentException` fırlatır.

Alanlı ve yapıcılı enum

`enum` aslında özel bir sınıftır: Alanları, yapıcısı ve metotları olabilir. Her sabit, yapıcıya parantez içinde değer gönderir:

HarfNotuOrnek.java

```
enum HarfNotu {
    AA(4.0, 90), BA(3.5, 85), BB(3.0, 80), CB(2.5, 75),
    CC(2.0, 70), DC(1.5, 65), DD(1.0, 60), FF(0.0, 0);   // sabitlerden sonra ;

    private final double katsayi;
    private final int altSinir;

    HarfNotu(double katsayi, int altSinir) {   // yapıcı otomatik olarak private
        this.katsayi = katsayi;
        this.altSinir = altSinir;
    }

    double getKatsayi() { return katsayi; }

    static HarfNotu puandan(int puan) {
        for (HarfNotu h : values()) {          // AA'dan FF'ye doğru dener
            if (puan >= h.altSinir) {
                return h;
            }
        }
        return FF;
    }
}

public class HarfNotuOrnek {
    public static void main(String[] args) {
        int[] puanlar = {93, 78, 66, 41};
        for (int p : puanlar) {
            HarfNotu h = HarfNotu.puandan(p);
            System.out.printf("%d -> %s (%.1f)%n", p, h, h.getKatsayi());
        }
    }
}
```

Çıktı

```
93 -> AA (4.0)
78 -> CB (2.5)
66 -> DC (1.5)
41 -> FF (0.0)
```

Kurallar: Sabitler **en başta** yazılır ve listeyi `;` bitirir. enum'un yapıcısı `public` olamaz ve `new HarfNotu(...)` ile dışarıdan nesne üretilemez; nesneler sadece listede yazdığın sabitlerdir.

> **Sık Yapılan Hata: Sabit yerine String veya int kullanmak**
>
> `String durum = "ONAYLANDI";` yazıp sonra `if (durum == "onaylandi")` gibi karşılaştırmalar yapmak hem yazım hatalarına hem de `==` / `equals` tuzağına (Bölüm 8) açıktır. Değerleri belli, sınırlı bir küme (günler, aylar, sipariş durumu, harf notu, yön) için **enum** kullan: Sabitin adını yanlış yazarsan program **derlenmez**, ve enum'lar `==` ile güvenle karşılaştırılır.

11.12 record: veri taşıyan kısa sınıflar

Bazı sınıfların tek işi birkaç değeri bir arada taşımaktır: bir nokta (x, y), bir öğrencinin numarası ve notu... Bunlar için alanları, yapıcıyı, get metotlarını, `toString`'i ve `equals`'ı tek tek yazmak yorucudur. Java 16 ile gelen **`record`** bunların hepsini tek satırda üretir:

RecordOrnek.java

```
record Nokta(int x, int y) {
    double uzaklik() { return Math.sqrt(x * x + y * y); }   // metot eklenebilir
}

public class RecordOrnek {
    public static void main(String[] args) {
        Nokta p = new Nokta(3, 4);        // yapıcı hazır
        Nokta q = new Nokta(3, 4);

        System.out.println(p);            // toString hazır
        System.out.println(p.x() + " " + p.y());
        System.out.println(p.uzaklik());
        System.out.println(p == q);
        System.out.println(p.equals(q));  // equals hazır: değerleri karşılaştırır
    }
}
```

Çıktı

```
Nokta[x=3, y=4]
3 4
5.0
false
true
```

`record Nokta(int x, int y)` yazınca derleyici şunları kendiliğinden oluşturur: `private final int x, y` alanları, `Nokta(int x, int y)` yapıcısı, `x()` ve `y()` erişim metotları (dikkat: `getX()` değil), değerleri yazdıran `toString`, değerleri karşılaştıran `equals` ve `hashCode`. `p == q` ise yine iki ayrı nesne olduğu için `false`'tur.

> **● Not: record değiştirilemez (immutable)**
>
> record'un alanları `final`'dır ve set metodu yoktur; nesne oluşturulduktan sonra alanlarına yeni değer atanamaz, `p.x = 5;` derlenmez. Bu değişmezlik **yüzeyseldir** (*shallowly immutable*): bileşenlerden biri dizi ya da `ArrayList` ise, referans değişmez ama içindeki elemanlar değiştirilebilir. Ayrıca record başka bir sınıftan `extends` yapamaz (ama arayüz uygulayabilir). Basit veri taşıyıcılar için çok kullanışlıdır; ama hocan "getter/setter'lı sınıf yaz" diyorsa Bölüm 9'daki klasik sınıfı yaz.

11.13 sealed sınıflar ve switch ile desen eşleme

Kalıtım **açıktır**: Bir arayüzü ya da sınıfı herkes, istediği dosyada genişletebilir. Bazen tersini istersin: "Şekil yalnızca daire, dikdörtgen veya üçgendir; başka tür yok." Java 17'deki **`sealed`** (mühürlü) sınıf ve arayüzler bunu derleyiciye söyler: Alt tipleri **`permits`** ile sayarsın, listede olmayan hiçbir sınıf kalıtım yapamaz. Kazancı büyüktür: Derleyici alt tiplerin tam listesini bildiği için `switch` ifadesinde **`default` yazmadan** da bütün durumların işlendiğini kontrol eder.

`switch` tarafında ise Java 21 ile `case` etiketi bir sabit yerine **desen** (pattern) olabilir: `case Daire d ->` "nesne bir `Daire` ise ona `d` adıyla eriş" demektir; bu bölümün başında gördüğün `instanceof Kedi k` ile aynı fikirdir ama kontrol ve cast `switch` içinde yapılır. Desenin ardına `when` ile ek koşul (**koruma**, guard) eklenebilir:

SealedOrnek.java

```java
sealed interface Sekil permits Daire, Dikdortgen, Ucgen { }

record Daire(double r) implements Sekil { }
record Dikdortgen(double en, double boy) implements Sekil { }
record Ucgen(double taban, double yukseklik) implements Sekil { }

public class SealedOrnek {
    static double alan(Sekil s) {
        return switch (s) {
            case Daire d -> Math.PI * d.r() * d.r();
            case Dikdortgen dk -> dk.en() * dk.boy();
            case Ucgen u -> u.taban() * u.yukseklik() / 2;
        };
    }

    static String betimle(Sekil s) {
        return switch (s) {
            case Daire d when d.r() > 10 -> "büyük daire";
            case Daire d -> "daire";
            case Dikdortgen dk -> "dikdörtgen";
            case Ucgen u -> "üçgen";
        };
    }

    public static void main(String[] args) {
        Sekil[] sekiller = {new Daire(1), new Dikdortgen(3, 4),
                            new Ucgen(6, 5), new Daire(12)};
        for (Sekil s : sekiller) {
            System.out.printf("%-12s %8.2f%n", betimle(s), alan(s));
        }
    }
}
```

Çıktı

```
daire            3.14
dikdörtgen      12.00
üçgen           15.00
büyük daire    452.39
```

Her iki `switch`'te de `default` yoktur ve program derlenir, çünkü `Sekil`'in alt tipleri yalnız bu üçüdür. `record`'lar zaten `final` olduğu için ayrıca bir belirteç yazmadık. Bir sınıf `sealed` bir üst tipe bağlanırken kendi kalıtım durumunu da bildirmek zorundadır:

Alt tipin belirteci	Anlamı
`final`	Kalıtım burada biter (`record`'lar zaten `final`'dır)
`sealed`	Bu alt tip de kendi `permits` listesiyle mühürlüdür
`non-sealed`	Mühür burada **açılır**; herkes bu sınıftan kalıtım yapabilir

SealedSinif.java

```
sealed abstract class Hayvan permits Kedi, Kopek { }
final class Kedi extends Hayvan { }
non-sealed class Kopek extends Hayvan { }
class Tazi extends Kopek { }              // Kopek non-sealed: serbest

public class SealedSinif {
    public static void main(String[] args) {
        Hayvan h = new Tazi();
        System.out.println(h instanceof Kopek);
        System.out.println(h.getClass().getSimpleName());
    }
}
```

Çıktı

```
true
Tazi
```

`Tazi`, `Hayvan`'ın doğrudan izinli alt tipi değildir; `Kopek` `non-sealed` olduğu için ondan kalıtım serbesttir. `Kedi` ise `final`'dır, ondan kalıtım yapılamaz. Tüm alt tipler **aynı dosyadaysa** `permits` yazmasan da olur (derleyici listeyi kendisi çıkarır); kitapta açık yazıyoruz.

Sık Yapılan Hata: Eksik kol: switch tüm alt tipleri kapsamalı

`sealed` bir tipin üzerinde `default`'suz `switch` yazıp bir alt tipi unutursan program derlenmez. Yeni bir alt tip eklediğinde derleyicinin seni uyarmasını istiyorsan `default` yazma:

```
sealed interface Sekil permits Daire, Dikdortgen, Ucgen { }

record Daire(double r) implements Sekil { }
record Dikdortgen(double en, double boy) implements Sekil { }
record Ucgen(double taban, double yukseklik) implements Sekil { }

public class SealedHata {
    static double alan(Sekil s) {
        return switch (s) {
            case Daire d -> Math.PI * d.r() * d.r();
            case Dikdortgen dk -> dk.en() * dk.boy();
        };
    }

    public static void main(String[] args) {
        System.out.println(alan(new Daire(1)));
    }
}
```

Derleyici çıktısı – program derlenmez

```
SealedHata.java:9: error: the switch expression does not cover all possible input values
        return switch (s) {
               ^
1 error
```

11.14 Polimorfik olmayanlar: static, final ve alanlar

Dinamik bağlama sadece **ezilebilen nesne metotları** için geçerlidir. Şunlar polimorfik **değildir**:

- `final` **metot** alt sınıfta ezilemez (*overridden method is final*). `final` **sınıf** ise hiç kalıtılamaz; `String` böyle bir sınıftır. `private` **metot** alt sınıfa görünmediği için ezilemez.
- `static` **metot** ezilmez, **gizlenir** (hiding). Alt sınıfta aynı imzalı static metot yazarsan, hangisinin çağrılacağına **referans tipi** karar verir.
- **Alanlar** (fields) da polimorfik değildir. Alt sınıfta üst sınıftakiyle aynı isimli bir alan tanımlarsan, `ref.alan` ifadesi yine **referans tipine** göre seçilir. Buna **alan gizleme** (field hiding) denir.

AlanGizleme.java

```
class Ust {
    String ad = "Üst alanı";

    static String tip() { return "Ust.tip()"; }
    String kim() { return "Ust.kim()"; }
}

class Alt extends Ust {
    String ad = "Alt alanı";                    // Ust'teki ad'ı gizler

    static String tip() { return "Alt.tip()"; }   // gizler, ezmez
    @Override String kim() { return "Alt.kim()"; } // ezer
}

public class AlanGizleme {
    public static void main(String[] args) {
        Ust u = new Alt();
        System.out.println(u.ad);
        System.out.println(u.tip());
        System.out.println(u.kim());
        System.out.println(((Alt) u).ad);
    }
}
```

Çıktı

```
Üst alanı
Ust.tip()
Alt.kim()
Alt alanı
```

Sadece `kim()` nesne tipine (`Alt`) göre çalıştı; alan ve static metot referans tipine (`Ust`) göre seçildi. Gerçek kodda alt sınıfta aynı isimli alan tanımlamak kafa karıştırır, **yapma**. Static metotları da her zaman sınıf adıyla çağır (`Alt.tip()`).

Bölüm Özeti – Bilmen Gerekenler

- **Polimorfizm:** Aynı çağrı, nesnenin gerçek tipine göre farklı davranır. `Hayvan h = new Kedi();` → referans tipi `Hayvan`, nesne tipi `Kedi`.
- **Yukarı dönüşüm** otomatiktir. **Aşağı dönüşüm** cast ister: `(Kedi) h`. Nesne gerçekten o tipte değilse çalışırken `ClassCastException`; kardeş sınıflar arası cast **derleme hatası**.
- **Derleyici referans tipine bakar** (metot var mı?), **JVM nesne tipine bakar** (hangi sürüm çalışır?). Üst sınıf metodunun içinden yapılan çağrılar da nesne tipine göre seçilir.
- **Override** çalışma zamanında nesne tipine göre, **overload** derleme zamanında referans tipine göre seçilir.

- `x instanceof T`: `x` bir `T` (veya alt sınıfı) ise `true`; `null` için hep `false`. `if (x instanceof T t)` kontrol + cast yapar.
- **Soyut sınıf:** `abstract`, nesnesi üretilemez, yapıcısı ve alanları olabilir. Soyut metodun gövdesi yoktur; somut alt sınıf hepsini ezmek zorundadır.
- **Arayüz:** `implements`, bir sınıf birden çok arayüz uygulayabilir. Metotlar varsayılan `public abstract`, alanlar `public static final`. `default`, `static` ve `private` metotlar gövdelidir. Uygularken `public` yazmayı unutma.
- Soyut sınıf "is-a" ve ortak durum, arayüz "can-do" ve ortak yetenek içindir.
- **enum:** Sınırlı sabit kümesi. `values()`, `ordinal()` (0'dan), `name()`, `valueOf()`, `==` ile karşılaştırma; alan, yapıcı ve metot içerebilir.
- **record:** `record Nokta(int x, int y)` → yapıcı, `x()`, `toString`, `equals` hazır; değiştirilemez.
- **`sealed`** + `permits`: alt tipler kapalı kümedir; izinli alt tip `final`, `sealed` ya da `non-sealed` olur. Java 21 `switch` desenleri (`case Daire d ->`, `when` koruması) `sealed` ile `default`'suz ve tam kapsamlı yazılır.
- Kare–Dikdörtgen: matematiksel is-a her zaman davranışsal is-a değildir (Liskov); değiştirilebilir alanlarla `Kare extends Dikdortgen` bozulur.
- `static` metotlar, `private`/`final` metotlar ve **alanlar** polimorfik değildir: referans tipine göre seçilir.

Sorular

A) Çoktan Seçmeli

1 `class Hayvan {}` ve `class Kedi extends Hayvan {}` tanımlı olsun. Aşağıdaki satırlardan hangisi **derlenmez**?

(A) `Hayvan h = new Kedi();`
(B) `Object o = new Kedi();`
(C) `Kedi k = new Hayvan();`
(D) `Hayvan h = (Hayvan) new Kedi();`
(E) `Kedi k = (Kedi) new Hayvan();`

2 Soyut sınıflar (abstract class) hakkında aşağıdakilerden hangisi **yanlıştır**?

(A) Soyut sınıfın yapıcısı (constructor) olabilir.
(B) Soyut sınıfta gövdeli (somut) metotlar bulunabilir.
(C) İçinde soyut metot bulunan sınıf `abstract` olarak tanımlanmak zorundadır.
(D) Soyut sınıftan `new` ile doğrudan nesne oluşturulabilir.
(E) Soyut sınıf türünde bir referans değişkeni tanımlanabilir.

3 Bir arayüzün içine `int LIMIT = 10;` yazılırsa derleyici bu satırı nasıl kabul eder?

(A) `public static final int LIMIT = 10;`
(B) `private int LIMIT = 10;`
(C) `protected static int LIMIT = 10;`
(D) `public int LIMIT = 10;` (her nesnenin kendi alanı)
(E) Derleme hatası verir; arayüzde alan tanımlanamaz.

4 Aşağıdaki kod parçası ne yazdırır?

```
class A {
    String f() { return "A"; }
    String g() { return f() + "g"; }
}
class B extends A {
    String f() { return "B"; }
}
// main içinde:
A x = new B();
System.out.println(x.g() + x.f());
```

A) `AgA`

B) `AgB`

C) `BgA`

D) Derleme hatası verir, çünkü `g()` metodu `B` sınıfında yoktur.

E) `BgB`

5 `enum Renk { KIRMIZI, YESIL, MAVI }` tanımlı olsun. Aşağıdaki satırlar ne yazdırır?

```
Renk r = Renk.valueOf("MAVI");
System.out.println(r.ordinal() + Renk.values().length);
```

A) `MAVI3`

B) `5`

C) `23`

D) `6`

E) Çalışırken `IllegalArgumentException` fırlatır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
class Arac {
    String ad() { return "Araç"; }
    int teker() { return 4; }
    void yaz() { System.out.println(ad() + " " + teker()); }
}

class Motor extends Arac {
    @Override String ad() { return "Motor"; }
    @Override int teker() { return 2; }
}

class Kamyon extends Arac {
    @Override int teker() { return super.teker() + 2; }

    @Override
    void yaz() {
        System.out.print("[Yük] ");
        super.yaz();
    }
}

public class Soru6 {
    public static void main(String[] args) {
        Arac[] dizi = {new Arac(), new Motor(), new Kamyon()};
        for (Arac a : dizi) {
            a.yaz();
        }
        Arac x = dizi[2];
        System.out.println(x instanceof Motor);
        System.out.println(x.ad().length() + x.teker());
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır? (Override ile overload'a dikkat!)

```
class Hayvan {
    String ses() { return "..."; }
}

class Kedi extends Hayvan {
    @Override String ses() { return "Miyav"; }
}

public class Soru7 {
    static String sor(Hayvan h) { return "H:" + h.ses(); }
    static String sor(Kedi k) { return "K:" + k.ses(); }

    public static void main(String[] args) {
        Hayvan a = new Kedi();
        Kedi b = new Kedi();
        Hayvan c = new Hayvan();
        System.out.println(sor(a));
        System.out.println(sor(b));
        System.out.println(sor(c));
        System.out.println(sor((Kedi) a));
        System.out.println(sor((Hayvan) b));
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
interface Selam {
    String ad();

    default String selam() { return "Merhaba " + ad(); }
}

abstract class Kisi implements Selam {
    static int sayac = 0;
    protected String isim;

    Kisi(String isim) { this.isim = isim; sayac++; }

    public String ad() { return isim; }
}

class Ogrenci extends Kisi {
    Ogrenci(String isim) { super(isim); }

    @Override
    public String selam() { return "Selam, ben " + ad(); }
}

class Hoca extends Kisi {
    Hoca(String isim) { super(isim); }

    @Override
    public String ad() { return "Dr. " + isim; }
}

public class Soru8 {
    public static void main(String[] args) {
        Selam[] liste = {new Ogrenci("Ece"), new Hoca("Kaya"), new Ogrenci("Mert")};
        for (Selam s : liste) {
            System.out.println(s.selam());
        }
        System.out.println(Kisi.sayac);
    }
}
```

C) Kod Yazma

9 Bir şirketin bordrosunu hesaplayan `Bordro` programını yaz:

- Soyut `Calisan` sınıfı: `ad` alanı, yapıcı, soyut `double maas()` ve soyut `String tur()` metotları.
- `TamZamanli`: Aylık sabit maaş alır.
- `SaatUcretli`: Maaş = çalışılan saat × saatlik ücret. Ancak **160 saatin üzerindeki** her saat için ücretin **1.5 katı** ödenir.
- `main`: Şu dört çalışanı bir `Calisan[]` dizisine koysun: Ayşe (tam zamanlı, 45000), Burak (150 saat, saati 200 TL), Ceren (180 saat, saati 200 TL), Deniz (tam zamanlı, 38500). Her çalışanı ve toplam bordroyu yazdırsın:

```
Ayşe   Tam zamanlı     45000.00
Burak  Saat ücretli    30000.00
Ceren  Saat ücretli    38000.00
Deniz  Tam zamanlı     38500.00
Toplam bordro: 151500.00 TL
```

10 Bir trafik ışığını modelleyen `Isik` enum'unu ve `TrafikIsigi` programını yaz:

- Sabitler ve süreleri (saniye): `KIRMIZI(30)`, `YESIL(25)`, `SARI(5)`.
- `int getSure()` metodu ve sıradaki ışığı döndüren `Isik sonraki()` metodu (`SARI`'dan sonra yine `KIRMIZI` gelir). `sonraki()`'yi `values()` ve `ordinal()` ile yaz; `if`/`switch` kullanma.
- `main`: `KIRMIZI`'dan başlayıp 5 adım boyunca ışığı ve süresini yazdırsın, sonunda toplam süreyi versin:

```
KIRMIZI 30 sn
YESIL 25 sn
SARI 5 sn
KIRMIZI 30 sn
YESIL 25 sn
Toplam: 115 sn
```

Çözümler

Cevap anahtarı

1	2	3	4	5
C	D	A	E	B

1 Cevap: C

Her hayvan bir kedi değildir; üst sınıf nesnesi cast olmadan alt sınıf referansına atanamaz (*incompatible types*). A ve B yukarı dönüşümdür, otomatiktir. D gereksiz ama geçerli bir yukarı cast'tir. E **derlenir**, çünkü cast yazılmıştır; ama çalışırken nesne bir kedi olmadığı için `ClassCastException` fırlatır. Soru derlenmeyeni sorduğu için cevap C.

2 Cevap: D

Soyut sınıftan nesne üretilemez; `new Sekil()` derleme hatası verir (*is abstract; cannot be instantiated*). Diğerleri doğrudur: Soyut sınıfın yapıcısı alt sınıfların `super(...)` çağrısıyla çalışır, gövdeli metotları olabilir, soyut metodu olan sınıf `abstract` olmak zorundadır ve `Sekil s = new Daire(1);` gibi referanslar tanımlanabilir.

3 Cevap: A

Arayüzdeki her alan otomatik olarak `public static final`'dır, yani bir **sabittir**. Arayüzün nesnesi olmadığı için nesne alanı (D) olamaz; `private` (B) ve `protected` (C) alan da yazılamaz. Alan tanımlamak derleme hatası değildir (E).

4 Cevap: E

`x.g()` çağrısı derlenir, çünkü `g()` referans tipi `A`'da vardır ve `B` onu miras alır (D yanlış). `g()`'nin içindeki `f()` çağrısı dinamik bağlanır: Nesne `B` olduğu için `B`'nin `f()`'i çalışır → `"Bg"`. `x.f()` de `"B"` döndürür. Sonuç: `BgB`. A ve B şıkları, `g()` `A`'da tanımlı olduğu için `A`'nın `f()`'inin çağrılacağını sanan öğrencinin cevabıdır.

5 Cevap: B

`valueOf("MAVI")` adı tam olarak eşleştiği için hata vermez (E yanlış) ve `MAVI` sabitini döndürür. `ordinal()` 0'dan başlar: `KIRMIZI` =0, `YESIL` =1, `MAVI` =2. `values().length` = 3. İkisi de `int` olduğu için toplanır: `2 + 3 = 5`. D, sıranın 1'den başladığını sanan; A ve C, String birleştirme yapıldığını sanan öğrencinin cevabıdır.

6

- `new Arac()`: Kendi metotları → `Araç 4`.
- `new Motor()`: `yaz()` `Arac`'tan gelir ama içindeki `ad()` ve `teker()` `Motor`'unkilerdir → `Motor 2`.
- `new Kamyon()`: Kendi `yaz()`'ı önce `[Yük]` yazar, sonra `super.yaz()` ile `Arac`'ın `yaz()`'ı çalışır. Orada `ad()` ezilmediği için `Araç`; `teker()` ise `Kamyon`'unki: `super.teker() + 2 = 6` → `[Yük] Araç 6`.
- `x` bir `Kamyon`'dur, `Motor` değildir → `false`.
- `"Araç".length()` = 4, `x.teker()` = 6 → iki `int` toplanır: `10`.

Çıktı

```
Araç 4
Motor 2
[Yük] Araç 6
false
10
```

7

Her satırda iki karar var: **Hangi `sor`?** (overload, derleme zamanı, argümanın **referans tipi**) ve **hangi `ses`?** (override, çalışma zamanı, **nesne tipi**).

Çağrı	Referans tipi → `sor`	Nesne tipi → `ses`	Sonuç
`sor(a)`	`Hayvan` → `H:`	`Kedi` → `Miyav`	`H:Miyav`
`sor(b)`	`Kedi` → `K:`	`Kedi` → `Miyav`	`K:Miyav`
`sor(c)`	`Hayvan` → `H:`	`Hayvan` → `...`	`H:...`
`sor((Kedi) a)`	`Kedi` → `K:`	`Kedi` → `Miyav`	`K:Miyav`
`sor((Hayvan) b)`	`Hayvan` → `H:`	`Kedi` → `Miyav`	`H:Miyav`

Son satır öğreticidir: `(Hayvan) b` cast'i nesneyi değiştirmez, sadece referans tipini değiştirir. Bu yüzden `H:` seçilir ama ses hâlâ `Miyav`'dır.

Çıktı

```
H:Miyav
K:Miyav
H:...
K:Miyav
H:Miyav
```

8

- `Ogrenci("Ece")`: `selam()`'ı ezmiş → `Selam, ben Ece`.
- `Hoca("Kaya")`: `selam()`'ı ezmemiş, arayüzdeki `default` sürüm çalışır. O sürümdeki `ad()` çağrısı dinamik bağlanır ve `Hoca`'nın `ad()`'ı çalışır → `Merhaba Dr. Kaya`.
- `Ogrenci("Mert")` → `Selam, ben Mert`.
- Her nesne oluşturulurken `Kisi` yapıcısı çalıştı ve `static` sayaç 3 kez arttı. `static` alan bütün nesnelerce **ortak** olduğu için sonuç `3`.

Çıktı

```
Selam, ben Ece
Merhaba Dr. Kaya
Selam, ben Mert
3
```

9

`Calisan` sınıfı `maas()` ve `tur()`'ü soyut bırakır; ortak yazdırma metodu `satir()` bu iki metodu çağırır ve dinamik bağlama doğru sürümü seçer. Fazla mesai için 160 saati aşan kısım ayrıca hesaplanır: Ceren için `160 × 200 + 20 × 200 × 1.5 = 32000 + 6000 = 38000`.

Bordro.java

```
abstract class Calisan {
    protected String ad;

    Calisan(String ad) { this.ad = ad; }

    abstract double maas();
    abstract String tur();
```

```
    String satir() {
        return String.format("%-6s %-14s %9.2f", ad, tur(), maas());
    }
}

class TamZamanli extends Calisan {
    private double aylik;

    TamZamanli(String ad, double aylik) { super(ad); this.aylik = aylik; }

    @Override double maas() { return aylik; }
    @Override String tur() { return "Tam zamanlı"; }
}

class SaatUcretli extends Calisan {
    private int saat;
    private double ucret;

    SaatUcretli(String ad, int saat, double ucret) {
        super(ad);
        this.saat = saat;
        this.ucret = ucret;
    }

    @Override
    double maas() {
        if (saat <= 160) {
            return saat * ucret;
        }
        return 160 * ucret + (saat - 160) * ucret * 1.5;
    }

    @Override String tur() { return "Saat ücretli"; }
}

public class Bordro {
    public static void main(String[] args) {
        Calisan[] ekip = {
            new TamZamanli("Ayşe", 45000),
            new SaatUcretli("Burak", 150, 200),
            new SaatUcretli("Ceren", 180, 200),
            new TamZamanli("Deniz", 38500)
        };
        double toplam = 0;
        for (Calisan c : ekip) {
            System.out.println(c.satir());
            toplam += c.maas();
        }
        System.out.printf("Toplam bordro: %.2f TL%n", toplam);
    }
}
```

Çıktı

```
Ayşe   Tam zamanlı     45000.00
Burak  Saat ücretli    30000.00
Ceren  Saat ücretli    38000.00
Deniz  Tam zamanlı     38500.00
Toplam bordro: 151500.00 TL
```

10

`values()` sabitleri tanım sırasıyla verir. Sıradaki sabitin indeksi `(ordinal() + 1) % values().length` olur; `%` sayesinde son sabitten (`SARI`, indeks 2) sonra indeks 0'a (`KIRMIZI`) dönülür.

TrafikIsigi.java

```
enum Isik {
    KIRMIZI(30), YESIL(25), SARI(5);

    private final int sure;

    Isik(int sure) { this.sure = sure; }

    int getSure() { return sure; }

    Isik sonraki() {
        Isik[] hepsi = values();
        return hepsi[(ordinal() + 1) % hepsi.length];
    }
}

public class TrafikIsigi {
    public static void main(String[] args) {
        Isik isik = Isik.KIRMIZI;
        int toplam = 0;
        for (int i = 0; i < 5; i++) {
            System.out.println(isik + " " + isik.getSure() + " sn");
            toplam += isik.getSure();
            isik = isik.sonraki();
        }
        System.out.println("Toplam: " + toplam + " sn");
    }
}
```

Çıktı

```
KIRMIZI 30 sn
YESIL 25 sn
SARI 5 sn
KIRMIZI 30 sn
YESIL 25 sn
Toplam: 115 sn
```

BÖLÜM 12

İstisna (Exception) Yönetimi

Bu Bölümde Öğreneceklerin

- Exception'ın ne olduğunu, programın **çökmesi** ile hatanın **ele alınması** arasındaki farkı
- İstisna hiyerarşisini (`Throwable`, `Error`, `Exception`, `RuntimeException`)
- **Checked** ve **unchecked** exception farkını (sınavın klasik tablosu)
- `try-catch-finally` yazmayı, birden çok `catch` ve multi-catch kullanmayı
- `throw` ile exception fırlatmayı, `throws` ile bildirmeyi, kendi exception sınıfını yazmayı
- Stack trace okumayı ve try-with-resources ile kaynakları otomatik kapatmayı
- Kullanıcı girdisini exception'larla doğrulamayı

12.1 Exception nedir?

Bölüm 1'de bir tam sayıyı sıfıra bölünce programın `ArithmeticException` ile **çöktüğünü** gördük. **Exception** (istisna), program çalışırken oluşan ve normal akışı bozan bir olaydır: olmayan bir dizi elemanına erişmek, sayı beklenirken harf okumak, `null` bir referansın metodunu çağırmak... Java'da her exception bir **nesnedir**; içinde hatanın türü, mesajı ve nerede oluştuğu bilgisi bulunur.

Exception ele alınmazsa JVM programı durdurur ve ekrana hata raporunu basar:

Cokme.java

```
public class Cokme {
    public static void main(String[] args) {
        String[] notlar = {"85", "70", "doksan", "60"};
        int toplam = 0;
        for (String s : notlar) {
            toplam += Integer.parseInt(s);
            System.out.println("Toplam: " + toplam);
        }
        System.out.println("Ortalama: " + toplam / notlar.length);
    }
}
```

Çıktı – program hata ile durur

```
Toplam: 85
Toplam: 155
Exception in thread "main" java.lang.NumberFormatException: For input string: "doksan"
        at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
        at java.base/java.lang.Integer.parseInt(Integer.java:565)
        at java.base/java.lang.Integer.parseInt(Integer.java:662)
        at Cokme.main(Cokme.java:6)
```

Tek bir hatalı veri yüzünden `60` hiç toplanmadı ve ortalama hiç yazılmadı. Oysa hatayı **yakalayıp** (catch) programın devam etmesini sağlayabiliriz:

Yakalama.java

```java
public class Yakalama {
    public static void main(String[] args) {
        String[] notlar = {"85", "70", "doksan", "60"};
        int toplam = 0, adet = 0;
        for (String s : notlar) {
            try {
                toplam += Integer.parseInt(s);
                adet++;
            } catch (NumberFormatException e) {
                System.out.println("Geçersiz not atlandı: " + s);
            }
        }
        if (adet > 0) {                       // hepsi geçersizse 0'a bölme olmasın
            System.out.printf("Ortalama: %.2f%n", (double) toplam / adet);
        }
    }
}
```

Çıktı

```
Geçersiz not atlandı: doksan
Ortalama: 71.67
```

`(85 + 70 + 60) / 3 = 71.67`. `(double)` dönüşümü olmasaydı tam sayı bölmesi `71` verirdi (Bölüm 2); `adet > 0` kontrolü de bütün notlar geçersiz olsaydı `/ by zero` hatasını önler. Program hatalı veriyi atladı ve işini bitirdi. Exception yönetiminin amacı budur: **Beklenmedik durumda programı çökertmek yerine kontrollü bir şekilde tepki vermek.**

12.2 İstisna hiyerarşisi

Exception'lar da sınıftır ve Bölüm 10'daki kalıtım ağacına göre düzenlenmiştir. Hepsinin atası `Throwable`'dır (fırlatılabilen):

```
Throwable
├── Error                              ciddi JVM sorunları; yakalanmaz
│   ├── OutOfMemoryError
│   └── StackOverflowError
└── Exception                          CHECKED (RuntimeException kolu hariç)
    ├── IOException
    │   └── FileNotFoundException
    ├── InterruptedException
    └── RuntimeException               UNCHECKED
        ├── ArithmeticException
        ├── NullPointerException
        ├── ClassCastException
        ├── IllegalStateException
        ├── IllegalArgumentException
        │   └── NumberFormatException
        ├── IndexOutOfBoundsException
        │   ├── ArrayIndexOutOfBoundsException
        │   └── StringIndexOutOfBoundsException
        └── NoSuchElementException
            └── InputMismatchException
```

(Şemada sadece sık karşılaşacağın sınıflar var; bazı ara sınıflar gösterilmedi.)

- `Error`: JVM'in kendisiyle ilgili, programın genelde düzeltemeyeceği ciddi sorunlar. Bellek bitmesi (`OutOfMemoryError`) veya sonsuz özyineleme (`StackOverflowError`, Bölüm 15) gibi. Bunları **yakalamaya çalışmazsın**.
- `Exception`: Programın ele alabileceği durumlar. Bu bölüm bunlarla ilgili.
- `RuntimeException`: Genellikle **programcı hatasından** kaynaklanan exception'lar.

En sık karşılaşacakların:

Exception	Ne zaman oluşur?	Örnek
`ArithmeticException`	Tam sayıyı sıfıra bölme	`10 / 0`
`ArrayIndexOutOfBoundsException`	Dizinin sınırları dışına erişme	3 elemanlı dizide `a[3]`
`StringIndexOutOfBoundsException`	String'in sınırları dışına erişme	`"abc".charAt(3)`
`NullPointerException`	`null` referans üzerinden metot/alan kullanma	`String s = null; s.length()`
`NumberFormatException`	Metni sayıya çevirememe	`Integer.parseInt("12a")`
`InputMismatchException`	Scanner beklediği tipte girdi bulamadı	`nextInt()` için `"yirmi"`
`ClassCastException`	Yanlış aşağı dönüşüm (Bölüm 11)	`(Kedi) kopekNesnesi`
`IllegalArgumentException`	Metoda geçersiz argüman verildi	Negatif yaş (genelde **sen** fırlatırsın)
`IllegalStateException`	Nesne o işlem için uygun durumda değil	Kapalı bir hesaptan para çekme

12.3 Checked ve unchecked exception'lar

Java, exception'ları derleyicinin kontrol edip etmemesine göre ikiye ayırır. Bu ayrım vize ve finallerin **kesin sorusudur**:

	Checked (denetlenen)	Unchecked (denetlenmeyen)
Hangi sınıflar?	`Exception` ve alt sınıfları (`RuntimeException` **kolu hariç**)	`RuntimeException` ve alt sınıfları + `Error`
Derleyici kontrol eder mi?	**Evet.** Ya yakalamalı ya da `throws` ile bildirmelisin; yoksa **derlenmez**	Hayır. Yakalamak serbest
Genelde sebebi	Programın **dışındaki** durumlar: dosya yok, ağ koptu	**Programcı hatası**: null, yanlış indeks, sıfıra bölme
Örnekler	`IOException`, `FileNotFoundException`, `InterruptedException`	`ArithmeticException`, `NullPointerException`, `NumberFormatException`

Mantığı şudur: Dosyanın diskte olup olmaması senin kontrolünde değildir; derleyici "bu ihtimali düşündün mü?" diye seni **zorlar**. Ama `dizi[5]` hatası senin kodundaki bir hatadır; doğru çözüm onu yakalamak değil, **kodu düzeltmektir**.

> **Sınavda Çıkar: Checked mı, unchecked mı?**
>
> Bir exception'ın türünü bulmak için hiyerarşide yukarı çık: Yolda `RuntimeException` varsa **unchecked**, yoksa (ve `Error` değilse) **checked**. Tuzak: `NumberFormatException` ve `InputMismatchException` kullanıcı girdisinden kaynaklansa da **unchecked**'tır. `IOException` ve `FileNotFoundException` ise **checked**'tır.

12.4 try-catch ile yakalama

Exception çıkabilecek kod `try` bloğuna, hata olursa çalışacak kod `catch` bloğuna yazılır:

```
try {
    // exception fırlatabilecek kod
} catch (ExceptionTipi e) {
    // o tipte bir exception olursa çalışacak kod
}
```

Akışı adım adım izleyelim:

TryAkis.java

```
public class TryAkis {
    public static void main(String[] args) {
        int[] dizi = {10, 20, 30};
        try {
            System.out.println("1. try başladı");
            System.out.println("2. " + dizi[1]);
            System.out.println("3. " + dizi[5]);     // exception!
            System.out.println("4. bu satır atlanır");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("5. yakalandı: " + e.getMessage());
        }
        System.out.println("6. program devam ediyor");
    }
}
```

Çıktı

```
1. try başladı
2. 20
5. yakalandı: Index 5 out of bounds for length 3
6. program devam ediyor
```

Kurallar:

1. `try` içinde exception oluştuğu anda, `try`'ın **geri kalanı atlanır** (4. satır hiç çalışmadı).
2. Java, exception'ın tipine **uyan** ilk `catch` bloğunu bulur ve çalıştırır. `catch (X e)` bloğu, `X` tipindeki ve `X`'in **alt sınıflarındaki** exception'ları yakalar.
3. `catch` bittikten sonra program `try-catch`'in **altından** devam eder. (Exception'ın oluştuğu yere geri dönülmez.)
4. `try` hatasız biterse `catch` blokları hiç çalışmaz.
5. Uyan bir `catch` yoksa exception metodu terk eder ve **çağıran metoda** geçer. Orada da yakalanmazsa bir üst çağırana... `main`'den de çıkarsa program çöker.

`catch` parantezindeki `e`, yakalanan exception **nesnesidir**; `e.getMessage()` hatanın açıklamasını verir.

> **Sık Yapılan Hata: try içinde tanımlanan değişken**
>
> `try { int x = Integer.parseInt(s); } catch (...) { }` yazdıktan sonra `x`'i `try`'ın dışında kullanamazsın; her blok gibi `try` da kendi **kapsamını** (scope) oluşturur. Değişkeni dışarıda kullanacaksan `try`'dan **önce** tanımla: `int x = 0; try { x = ... }`.

12.5 Birden çok catch

Bir `try` bloğundan farklı türde exception'lar çıkabiliyorsa, her biri için ayrı bir `catch` yazabilirsin. Java yukarıdan aşağıya bakar ve **uyan ilk** `catch`'i çalıştırır; diğerlerini atlar:

CokluCatch.java

```
public class CokluCatch {
    static int[] bolenler = {4, 0, 25};

    static void dene(String girdi) {
        try {
            int i = Integer.parseInt(girdi);
            int sonuc = 100 / bolenler[i];
            System.out.println(girdi + " -> " + sonuc);
        } catch (NumberFormatException e) {
            System.out.println(girdi + " -> sayı değil");
        } catch (ArithmeticException e) {
            System.out.println(girdi + " -> sıfıra bölme");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(girdi + " -> geçersiz indeks");
        }
    }

    public static void main(String[] args) {
        String[] girdiler = {"0", "abc", "1", "7", "2"};
        for (String g : girdiler) {
            dene(g);
        }
    }
}
```

Çıktı

```
0 -> 25
abc -> sayı değil
1 -> sıfıra bölme
7 -> geçersiz indeks
2 -> 4
```

Üst sınıf tipindeki bir `catch` bütün alt sınıflarını da yakaladığı için, `catch` bloklarının **sırası** önemlidir: **Önce özel (alt sınıf), sonra genel (üst sınıf).** Genel olanı önce yazarsan, alttaki özel `catch`'e asla sıra gelmez ve Java bu "ölü kodu" derlemez:

CatchSirasi.java

```
public class CatchSirasi {
    public static void main(String[] args) {
        try {
            int x = 10 / 0;
        } catch (Exception e) {
            System.out.println("Genel hata");
        } catch (ArithmeticException e) {
            System.out.println("Sıfıra bölme");
        }
    }
}
```

Derleyici çıktısı – program derlenmez

```
CatchSirasi.java:7: error: exception ArithmeticException has already been caught
        } catch (ArithmeticException e) {
          ^
1 error
```

> **Sınavda Çıkar: catch sırası kuralı**
>
> "Hangi sıralama derleme hatası verir?" sorusunda her `catch` için şuna bak: **Kendinden önce, kendi üst sınıfını yakalayan bir `catch` var mı?** Varsa derleme hatası (*has already been caught*). `catch (Exception e)` her zaman **en sona** yazılır. Kardeş sınıflar (`ArithmeticException` ile `NullPointerException` gibi) her sırada yazılabilir.

Multi-catch

Birkaç farklı exception için **aynı** işi yapacaksan, Java 7'den beri hepsini tek `catch`'te `|` ile birleştirebilirsin:

MultiCatch.java

```java
public class MultiCatch {
    public static void main(String[] args) {
        String[] veriler = {"12", "x", "3"};
        for (int i = 0; i <= veriler.length; i++) {     // <= bilerek yazıldı
            try {
                int n = Integer.parseInt(veriler[i]);
                System.out.println("Okundu: " + n);
            } catch (NumberFormatException | ArrayIndexOutOfBoundsException e) {
                System.out.println("Atlandı: " + e.getMessage());
            }
        }
    }
}
```

Çıktı

```
Okundu: 12
Atlandı: For input string: "x"
Okundu: 3
Atlandı: Index 3 out of bounds for length 3
```

Multi-catch'te birleştirilen tipler arasında kalıtım ilişkisi **olamaz**: `catch (NumberFormatException | IllegalArgumentException e)` derlenmez, çünkü `NumberFormatException` zaten `IllegalArgumentException`'ın alt sınıfıdır; sadece üst sınıfı yazmak yeterlidir (*Alternatives in a multi-catch statement cannot be related by subclassing*).

12.6 finally: her durumda çalışan blok

`try-catch`'ten sonra isteğe bağlı bir `finally` bloğu yazılabilir. `finally` bloğu **her durumda** çalışır: exception oluşsa da oluşmasa da, yakalansa da yakalanmasa da, hatta `try` içinde `return` olsa bile. Açılan dosyayı kapatmak, bağlantıyı kesmek gibi "ne olursa olsun yapılmalı" işleri için kullanılır.

FinallyOrnek.java

```java
public class FinallyOrnek {
    static int bol(int a, int b) {
        try {
            System.out.println("  try: " + a + "/" + b);
            return a / b;
        } catch (ArithmeticException e) {
            System.out.println("  catch: " + e.getMessage());
            return -1;
        } finally {
            System.out.println("  finally çalıştı");
        }
    }

    public static void main(String[] args) {
        System.out.println("Sonuç: " + bol(10, 2));
        System.out.println("Sonuç: " + bol(10, 0));
    }
}
```

Çıktı

```
  try: 10/2
  finally çalıştı
Sonuç: 5
  try: 10/0
  catch: / by zero
  finally çalıştı
Sonuç: -1
```

İlk çağrıda `try` içindeki `return a / b;` çalıştı, ama metot hemen dönmedi: Dönüş değeri (5) bir kenara alındı, **önce `finally` çalıştı**, sonra metot 5'i döndürdü. `Sonuç:` yazısının `finally`'den **sonra** çıkmasının sebebi de bu: `println`, `bol`'un dönmesini bekler.

> **Sınavda Çıkar: finally soruları**
>
> - `finally` her durumda çalışır. Tek istisna: `try` veya `catch` içinde `System.exit(...)` çağrılırsa JVM kapanır ve `finally` çalışmaz.
> - `try` içinde `return x;` varsa, `x`'in değeri **`finally`'den önce** alınır. `finally` içinde `x`'i değiştirmek dönen değeri değiştirmez.
> - `finally` içinde **kendi `return`'ü** varsa, `try`'daki `return`'ü ve hatta fırlatılan exception'ı **ezer**. Bu yüzden `finally` içinde `return` yazmak kötü bir uygulamadır; ama sınavda karşına çıkabilir (Soru 7).
> - `catch` olmadan `try-finally` yazılabilir; ama tek başına `try` yazılamaz.

12.7 throw: exception fırlatmak

Şimdiye kadar exception'ları Java fırlattı. `throw` anahtar kelimesiyle sen de fırlatabilirsin. Genellikle metoda geçersiz bir değer geldiğinde, yanlış veriyle devam etmek yerine işi hemen durdurmak için kullanılır:

ThrowOrnek.java

```java
class Ogrenci {
    private String ad;
    private int not;

    Ogrenci(String ad) { this.ad = ad; }

    void setNot(int not) {
```

```
        if (not < 0 || not > 100) {
            throw new IllegalArgumentException("Not 0-100 arası olmalı: " + not);
        }
        this.not = not;
        System.out.println(ad + " notu " + not + " olarak kaydedildi.");
    }
}

public class ThrowOrnek {
    public static void main(String[] args) {
        Ogrenci o = new Ogrenci("Elif");
        int[] denemeler = {85, 120, -5, 70};
        for (int n : denemeler) {
            try {
                o.setNot(n);
            } catch (IllegalArgumentException e) {
                System.out.println("Reddedildi -> " + e.getMessage());
            }
        }
    }
}
```

Çıktı

```
Elif notu 85 olarak kaydedildi.
Reddedildi -> Not 0-100 arası olmalı: 120
Reddedildi -> Not 0-100 arası olmalı: -5
Elif notu 70 olarak kaydedildi.
```

`throw new IllegalArgumentException("...")` satırında önce `new` ile bir exception **nesnesi** oluşturulur, sonra `throw` ile fırlatılır. `throw`'dan sonraki satırlar çalışmaz; tıpkı `return` gibi metodu o anda terk eder. Bu sayede `not` alanına hiçbir zaman geçersiz değer yazılmaz. Bu teknik, Bölüm 9'daki **kapsülleme** fikrinin doğal devamıdır: set metodu sadece doğrulama yapmakla kalmaz, hatayı çağırana da **bildirir**.

12.8 throws: checked exception'ı bildirmek

Checked bir exception fırlatabilen her metot için derleyici şu kuralı uygular: **"Yakala ya da bildir"** (handle or declare). Metot ya exception'ı kendi içinde `try-catch` ile yakalar ya da imzasına `throws` ekleyerek "ben bu exception'ı fırlatabilirim, beni çağıran ilgilensin" der:

ThrowsOrnek.java

```
import java.io.IOException;

public class ThrowsOrnek {
    static String dosyaAc(String ad) throws IOException {
        if (!ad.endsWith(".txt")) {
            throw new IOException("Desteklenmeyen dosya türü: " + ad);
        }
        return ad + " açıldı";     // gerçek dosya işlemleri Bölüm 14'te
    }

    public static void main(String[] args) {
        String[] dosyalar = {"notlar.txt", "resim.png"};
        for (String d : dosyalar) {
            try {
                System.out.println(dosyaAc(d));
            } catch (IOException e) {
                System.out.println("Hata: " + e.getMessage());
```

```
            }
        }
    }
}
```

Çıktı

```
notlar.txt açıldı
Hata: Desteklenmeyen dosya türü: resim.png
```

`dosyaAc` metodu exception'ı yakalamadı, `throws IOException` ile **bildirdi**. Bu yüzden onu çağıran `main` artık sorumludur. `main` de ne yakalar ne bildirirse program **derlenmez** (`dosyaAc` metodu yukarıdakiyle aynı; sadece `main`'i gösteriyoruz):

```
public static void main(String[] args) {
    System.out.println(dosyaAc("resim.png"));
}
```

Derleyici çıktısı – program derlenmez

```
Bildirilmemis.java:13: error: unreported exception IOException; must be caught or declared to be
thrown
        System.out.println(dosyaAc("resim.png"));
                                   ^
1 error
```

"unreported exception IOException; must be caught or declared to be thrown": "Bildirilmemiş exception; yakalanmalı veya fırlatılacağı bildirilmeli." Bu mesajı checked exception'larla çalışırken çok göreceksin. İki çözüm var: çağrıyı `try-catch` içine almak ya da `main`'in imzasına `throws IOException` eklemek. İkincisinde exception gerçekten oluşursa program çöker; küçük denemelerde kabul edilir ama iyi bir uygulama değildir.

Sınavda Çıkar: throw ile throws farkı

	`throw`	`throws`
Nerede?	Metodun **gövdesinde**, bir komut olarak	Metodun **imzasında**, parametrelerden sonra
Ne yazılır?	Bir exception **nesnesi**: `throw new X("...")`	Bir veya daha çok exception **tipi**: `throws X, Y`
Ne yapar?	Exception'ı **o anda fırlatır**	"Bu metot şunları fırlatabilir" diye **bildirir**

Unchecked exception'lar için `throws` yazmak serbesttir ama zorunlu değildir; checked olanlar için ise yakalamıyorsan **zorunludur**.

12.9 Kendi exception sınıfını yazmak

Java'nın hazır exception'ları her durumu anlatmaya yetmez. "Yaş sınırı aşıldı", "stok yetersiz" gibi programına özgü hatalar için **kendi exception sınıfını** yazabilirsin. Kural basit: Checked olacaksa `Exception`'dan, unchecked olacaksa `RuntimeException`'dan türet; mesajı `super(mesaj)` ile üst sınıfa ilet. Sınıf adı geleneksel olarak `Exception` ile biter.

OzelException.java

```
class YasSiniriException extends Exception {      // checked
    private final int yas;

    YasSiniriException(int yas) {
```

```
        super("Yaş sınırı 18, girilen yaş: " + yas);
        this.yas = yas;
    }

    int getYas() { return yas; }
}

public class OzelException {
    static void biletSat(String ad, int yas) throws YasSiniriException {
        if (yas < 18) {
            throw new YasSiniriException(yas);
        }
        System.out.println(ad + " için bilet satıldı.");
    }

    public static void main(String[] args) {
        String[] adlar = {"Ali", "Ece", "Can"};
        int[] yaslar = {25, 15, 18};
        for (int i = 0; i < adlar.length; i++) {
            try {
                biletSat(adlar[i], yaslar[i]);
            } catch (YasSiniriException e) {
                System.out.println(adlar[i] + ": " + e.getMessage());
                int kalan = 18 - e.getYas();
                System.out.println("  " + kalan + " yıl sonra tekrar gel.");
            }
        }
    }
}
```

Çıktı

```
Ali için bilet satıldı.
Ece: Yaş sınırı 18, girilen yaş: 15
  3 yıl sonra tekrar gel.
Can için bilet satıldı.
```

Kendi exception'ın sıradan bir sınıftır: Alanları (`yas`) ve metotları (`getYas()`) olabilir. Böylece `catch` bloğu sadece mesajı değil, hatayla ilgili **veriyi** de kullanabilir.

> **İpucu: Checked mı yapmalı, unchecked mı?**
>
> Kendine şunu sor: **"Çağıran bu durumu makul bir şekilde düzeltebilir mi?"** Evetse (yetersiz bakiye → kullanıcıya daha az tutar girdir) **checked** (`extends Exception`) yap; derleyici çağıranı bu durumu düşünmeye zorlasın. Durum bir programlama hatasıysa (metoda negatif miktar geçmek) **unchecked** (`extends RuntimeException`) yap veya hazır `IllegalArgumentException`'ı kullan. Proje 3'te iki türü de bir arada kullanacağız.

12.10 Exception bilgilerini okumak

getMessage ve toString

Yakalanan exception nesnesinden bilgi almanın iki temel yolu vardır:

- `e.getMessage()` → Sadece açıklama metni: `/ by zero`
- `e.toString()` (veya doğrudan `println(e)`) → Sınıfın tam adı + açıklama: `java.lang.ArithmeticException: / by zero`

Mesajlar.java

```
public class Mesajlar {
    public static void main(String[] args) {
        String s = null;
        int[] dizi = new int[3];
        for (int i = 1; i <= 4; i++) {
            try {
                switch (i) {
                    case 1 -> System.out.println(10 / 0);
                    case 2 -> System.out.println(dizi[3]);
                    case 3 -> System.out.println(s.length());
                    default -> System.out.println(Integer.parseInt("12a"));
                }
            } catch (RuntimeException e) {
                System.out.println(e);
            }
        }
    }
}
```

Çıktı

```
java.lang.ArithmeticException: / by zero
java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
java.lang.NullPointerException: Cannot invoke "String.length()" because "s" is null
java.lang.NumberFormatException: For input string: "12a"
```

Bu mesajları ezberlemene gerek yok ama **tanıman** gerekir: Program çöktüğünde ilk iş, mesajın hangi exception'dan geldiğini anlamaktır. Üçüncü satırdaki `NullPointerException` mesajı hangi değişkenin `null` olduğunu bile söylüyor: `"s" is null`. (Burada bütün hataları tek `catch (RuntimeException e)` ile yakaladık; bu sadece bir gösteri. Gerçek kodda neden kaçınman gerektiğini bölüm sonunda göreceksin.)

Stack trace okumak

Yakalanmayan bir exception programı çökerttiğinde JVM bir **stack trace** (yığın izi) basar. Bu rapor, exception'ın oluştuğu anda hangi metodun hangi metodu çağırdığını gösteren listedir:

YiginIzi.java

```
public class YiginIzi {
    static int ortalama(int[] notlar) {
        return toplam(notlar) / notlar.length;
    }

    static int toplam(int[] notlar) {
        int t = 0;
        for (int i = 0; i <= notlar.length; i++) {   // hata burada: < olmalıydı
            t += notlar[i];
        }
        return t;
    }

    public static void main(String[] args) {
        int[] notlar = {70, 80, 90};
        System.out.println("Ortalama hesaplanıyor...");
        System.out.println(ortalama(notlar));
    }
}
```

Çıktı – program hata ile durur

```
Ortalama hesaplanıyor...
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for
length 3
        at YiginIzi.toplam(YiginIzi.java:9)
        at YiginIzi.ortalama(YiginIzi.java:3)
        at YiginIzi.main(YiginIzi.java:17)
```

Stack trace'i **yukarıdan aşağıya** oku:

1. **İlk satır:** Exception'ın **tipi** ve **mesajı**: 3 uzunluklu dizide 3 numaralı indekse erişilmiş.
2. **İlk `at` satırı:** Exception'ın **fırlatıldığı yer**: `toplam` metodu, `YiginIzi.java` dosyasının 9. satırı. Hatayı ararken bakacağın ilk yer burasıdır.
3. **Alttaki `at` satırları:** O metodu **kimin çağırdığı**: `toplam`'ı `ortalama` (3. satır), `ortalama`'yı da `main` (17. satır) çağırmış.

Uzun stack trace'lerde `java.base/...` ile başlayan satırlar Java'nın kendi kütüphanesine aittir. Sen yukarıdan başlayıp **kendi dosyanın adının geçtiği ilk satırı** ara.

> **Not: printStackTrace**
>
> Yakaladığın bir exception'ın stack trace'ini programı durdurmadan görmek için `catch` içinde `e.printStackTrace();` yazabilirsin. Hata ayıklarken (debug) çok işe yarar. Bu metot raporu standart hata akışına (`System.err`) yazdığı için VS Code'da genellikle kırmızı görünür.

12.11 try-with-resources

Dosya, ağ bağlantısı, veritabanı bağlantısı gibi **kaynaklar** (resource) işin sonunda mutlaka kapatılmalıdır. Bunu `finally` ile yapmak mümkündür ama uzun ve hataya açıktır. Java 7 ile gelen **try-with-resources** yapısında kaynak, `try`'dan hemen sonraki parantezde açılır ve blok bitince **otomatik olarak** kapatılır; exception oluşsa bile.

Bunun için kaynağın sınıfı `AutoCloseable` arayüzünü (Bölüm 11) uygulamalıdır; bu arayüzün tek metodu `close()`'dur. Ne zaman kapandığını görmek için kendi kaynak sınıfımızı yazalım:

KaynakOrnek.java

```
class Kaynak implements AutoCloseable {
    private String ad;

    Kaynak(String ad) {
        this.ad = ad;
        System.out.println(ad + " açıldı");
    }

    void kullan() { System.out.println(ad + " kullanılıyor"); }

    @Override
    public void close() { System.out.println(ad + " kapandı"); }
}

public class KaynakOrnek {
    public static void main(String[] args) {
        try (Kaynak a = new Kaynak("Dosya"); Kaynak b = new Kaynak("Bağlantı")) {
            a.kullan();
            b.kullan();
            throw new IllegalStateException("işlem yarıda kaldı");
        } catch (IllegalStateException e) {
            System.out.println("catch: " + e.getMessage());
        } finally {
```

```
            System.out.println("finally");
        }
    }
}
```

Çıktı

```
Dosya açıldı
Bağlantı açıldı
Dosya kullanılıyor
Bağlantı kullanılıyor
Bağlantı kapandı
Dosya kapandı
catch: işlem yarıda kaldı
finally
```

Bu çıktıdan iki kural çıkar:

1. Kaynaklar açıldıkları sıranın **tersine** kapanır: Önce `Bağlantı` (son açılan), sonra `Dosya`.
2. Kaynaklar `catch` ve `finally`'den **önce** kapanır. `catch` çalıştığında kaynaklar çoktan kapatılmıştır.

`Scanner` da `AutoCloseable`'dır. Bölüm 14'te dosya okurken `try (Scanner dosya = new Scanner(new File("notlar.txt")))` kalıbını sürekli kullanacaksın.

> **Sık Yapılan Hata: Klavye Scanner'ını kapatmak**
>
> `Scanner sc = new Scanner(System.in)` ile oluşturduğun klavye okuyucusunu `sc.close()` ile veya try-with-resources içinde kapatırsan, **`System.in` de kapanır**. Programın sonraki bir yerinde yeniden `new Scanner(System.in)` oluşturup okumaya çalışırsan `NoSuchElementException` alırsın. Klavye Scanner'ını programın başında **bir kere** oluştur ve kapatma; dosya Scanner'larını ise mutlaka kapat.

12.12 Kullanıcı girdisini doğrulamak

Bölüm 3'te kullanıcı sayı yerine harf yazınca `nextInt()`'in `InputMismatchException` fırlatıp programı çökerttiğini görmüştük. Artık bu durumu yakalayıp kullanıcıdan **tekrar** girdi isteyebiliriz:

GuvenliOkuma.java

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class GuvenliOkuma {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int yas;
        while (true) {
            System.out.print("Yaşın: ");
            try {
                yas = sc.nextInt();
                sc.nextLine();                      // satırın kalanını at
                if (yas >= 0 && yas <= 120) {
                    break;                          // geçerli girdi: döngüden çık
                }
                System.out.println("Yaş 0 ile 120 arasında olmalı.");
            } catch (InputMismatchException e) {
                String hatali = sc.nextLine();      // hatalı girdiyi tampondan at
                System.out.println("\"" + hatali + "\" sayı değil!");
            }
```

```
        }
        System.out.println("Kaydedildi: " + yas);
    }
}
```

Çıktı

```
Yaşın: yirmi
"yirmi" sayı değil!
Yaşın: -5
Yaş 0 ile 120 arasında olmalı.
Yaşın: 19
Kaydedildi: 19
```

En kritik satır `catch` içindeki `sc.nextLine()`'dır. `nextInt()` başarısız olduğunda hatalı metni (`yirmi`) **tamponda bırakır**; onu okuyup atmazsak döngünün bir sonraki turunda `nextInt()` yine aynı metni görür, yine exception fırlatır ve program **sonsuz döngüye** girer. `nextLine()`, satırın geri kalanını okuyarak tamponu temizler. Başarılı okumadan sonraki `sc.nextLine()` da aynı işi yapar: Kullanıcı `-5 yirmi` ya da `19 abc` gibi tek satıra fazla şey yazarsa, sayıdan sonra kalan kısım tamponda kirli veri olarak kalmaz.

> **Sık Yapılan Hata: Tamponu temizlemeyi unutmak**
>
> `catch (InputMismatchException e) { System.out.println("Hatalı!"); }` yazıp `sc.nextLine()`'ı unutursan ekran durmadan `Yaşın: Hatalı!` yazar. Bu, Bölüm 3'teki `nextInt` + `nextLine` tuzağının kardeşidir: Scanner okuyamadığı şeyi **tüketmez**. Aynı işi exception'sız yapmanın yolu, Bölüm 5'te gördüğün `if (sc.hasNextInt())` kontrolüdür.

12.13 İyi uygulamalar

Exception'lar güçlü bir araçtır ama yanlış kullanılınca hataları **gizler**. Hocaların ödevlerde en çok puan kırdığı alışkanlıklar:

1. **Exception yutma (boş catch).** `catch (Exception e) { }` yazmak, hatayı halının altına süpürmektir: Program çökmez ama yanlış sonuç üretir ve neden yanlış olduğunu kimse bilmez. En azından bir mesaj yazdır; yapacak bir şeyin yoksa hiç yakalama.
2. **Genel `Exception` yakalamak.** `catch (Exception e)` her şeyi yakalar; beklemediğin bir `NullPointerException`'ı da sessizce "geçersiz girdi" diye yorumlarsın. Beklediğin **özel** tipi yakala (`NumberFormatException` gibi).
3. **Exception'ı `if` yerine kullanmak.** Önceden kontrol edebileceğin durumları kontrol et: `if (b != 0)` yazmak, `ArithmeticException` yakalamaktan hem daha okunur hem daha hızlıdır. `ArrayIndexOutOfBoundsException` yakalıyorsan muhtemelen döngünde hata vardır, düzelt.
4. **Anlamlı mesaj ver.** `throw new IllegalArgumentException("Hata")` yerine `"Tutar pozitif olmalı: " + tutar` yaz; mesaj hatanın sebebini ve değeri söylesin.
5. **Kaynakları try-with-resources ile kapat.** Elle `close()` çağırmayı unutmak kolaydır.

Yutma.java

```java
public class Yutma {
    public static void main(String[] args) {
        String[] veriler = {"10", "20", "30"};       // ortadaki "20": O harfi!
        int toplam = 0;
        for (String v : veriler) {
            try {
                toplam += Integer.parseInt(v);
            } catch (Exception e) {
                // boş catch: hata sessizce yutuldu
            }
        }
        System.out.println("Toplam: " + toplam);
    }
}
```

Çıktı

```
Toplam: 40
```

Program `60` beklenirken `40` yazdı ve hiçbir uyarı vermedi. Bu tür hatalar haftalarca fark edilmeyebilir. `catch` içine `System.out.println("Geçersiz veri: " + v);` yazsaydık sorun ilk çalıştırmada görünürdü.

Bölüm Özeti – Bilmen Gerekenler

- **Exception**, çalışma zamanında normal akışı bozan olaydır ve bir **nesnedir**. Yakalanmazsa program stack trace basıp çöker.
- Hiyerarşi: `Throwable` → `Error` (yakalanmaz) ve `Exception` → `RuntimeException`.
- **Checked:** `Exception`'ın `RuntimeException` dışındaki alt sınıfları (`IOException`...). **Yakala ya da bildir** zorunlu; yoksa *unreported exception* derleme hatası. **Unchecked:** `RuntimeException` ve alt sınıfları + `Error`.
- `try`'da exception oluşunca try'ın geri kalanı atlanır, **uyan ilk** `catch` çalışır, program try-catch'in altından devam eder.
- `catch` sırası: **özelden genele**. Üst sınıf önce yazılırsa *has already been caught* derleme hatası. Multi-catch: `catch (A | B e)`, A ile B akraba olamaz.
- **`finally`** her durumda çalışır (`return` olsa bile); sadece `System.exit` engeller. `return x` değeri finally'den önce alınır.
- **`throw new X("mesaj")`** exception fırlatır; **`throws X`** metot imzasında bildirir.
- Kendi exception'ın: `extends Exception` (checked) veya `extends RuntimeException` (unchecked), mesaj için `super(mesaj)`.
- `getMessage()` sadece mesaj, `toString()` sınıf adı + mesaj. Stack trace'te ilk `at` satırı hatanın yeri, alttakiler çağıranlar.
- **try-with-resources:** `try (Kaynak k = ...)`, sınıf `AutoCloseable` olmalı; kaynaklar **ters sırayla** ve catch'ten **önce** kapanır.
- Scanner'da `InputMismatchException` yakalandıktan sonra hatalı girdi **`sc.nextLine()`** ile temizlenmeli; yoksa sonsuz döngü.
- Boş catch yazma, genel `Exception` yakalamaktan kaçın, kontrol edilebilen durumlar için `if` kullan.

Sorular

A) Çoktan Seçmeli

1 Aşağıdakilerden hangisi bir **checked** exception'dır?

(A) `NullPointerException`
(B) `ArithmeticException`
(C) `IOException`
(D) `NumberFormatException`
(E) `ArrayIndexOutOfBoundsException`

2 Aşağıdakilerden hangisi `RuntimeException`'ın bir alt sınıfı **değildir**?

(A) `NumberFormatException`
(B) `InputMismatchException`
(C) `ClassCastException`
(D) `ArithmeticException`
(E) `FileNotFoundException`

3 Bir `try` bloğundan sonra yazılan aşağıdaki `catch` sıralamalarından hangisi **derleme hatası** verir?

(A) `catch (Exception e)` → `catch (ArithmeticException e)`
(B) `catch (ArithmeticException e)` → `catch (Exception e)`
(C) `catch (NumberFormatException e)` → `catch (IllegalArgumentException e)`
(D) `catch (ArithmeticException e)` → `catch (NullPointerException e)`
(E) `catch (RuntimeException e)` → `catch (Exception e)`

4 Aşağıdaki durumların hangisinde `finally` bloğu **çalışmaz**?

(A) `try` bloğu hiç exception oluşmadan bittiğinde
(B) `try` içinde oluşan exception bir `catch` tarafından yakalandığında
(C) `try` bloğu içinde bir `return` komutu çalıştığında
(D) `try` bloğu içinde `System.exit(0)` çağrıldığında
(E) `catch` bloğu içinde yeni bir exception fırlatıldığında

5 Aşağıdaki kod parçası ne yazdırır?

```
try {
    int[] a = new int[2];
    a[2] = 5;
    System.out.print("X");
} catch (ArithmeticException e) {
    System.out.print("A");
} catch (RuntimeException e) {
    System.out.print("R");
} finally {
    System.out.print("F");
}
System.out.print("S");
```

(A) `XFS`

(B) `RFS`

(C) `ARFS`

(D) `RF`

(E) Program çöker, sadece `F` yazdırılır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru6 {
    static void c(int x) {
        System.out.println("c başladı");
        if (x > 0) {
            throw new IllegalArgumentException("x pozitif");
        }
        System.out.println("c bitti");
    }

    static void b(int x) {
        try {
            c(x);
            System.out.println("b: c'den döndü");
        } finally {
            System.out.println("b: finally");
        }
    }

    public static void main(String[] args) {
        try {
            b(0);
            b(5);
            System.out.println("main: b'den döndü");
        } catch (IllegalArgumentException e) {
            System.out.println("main yakaladı: " + e.getMessage());
        }
        System.out.println("son");
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır? (`finally` ve `return` ilişkisine dikkat!)

```
public class Soru7 {
    static int f() {
        int x = 1;
        try {
            return x;
        } finally {
            x = 2;
            System.out.println("f: finally, x=" + x);
        }
    }

    static int g() {
        try {
            throw new RuntimeException("g hatası");
        } finally {
            return 3;
        }
    }

    static int h(String s) {
        try {
            return Integer.parseInt(s);
        } catch (NumberFormatException e) {
            return -1;
        } finally {
            System.out.println("h: finally");
        }
    }

    public static void main(String[] args) {
        System.out.println(f());
        System.out.println(g());
        System.out.println(h("7") + h("x"));
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
class Kapi implements AutoCloseable {
    private final String ad;

    Kapi(String ad) {
        this.ad = ad;
        System.out.println(ad + " açık");
    }

    @Override
    public void close() { System.out.println(ad + " kapalı"); }
}

class KilitException extends Exception {
    KilitException(String mesaj) { super(mesaj); }
}

public class Soru8 {
    static void gir(String oda) throws KilitException {
        try (Kapi dis = new Kapi("Dış"); Kapi ic = new Kapi("İç")) {
            if (oda.equals("kasa")) {
                throw new KilitException("Kasa kilitli");
```

```
            }
            System.out.println(oda + " odasına girildi");
        }
    }

    public static void main(String[] args) {
        String[] odalar = {"salon", "kasa"};
        for (String o : odalar) {
            try {
                gir(o);
            } catch (KilitException e) {
                System.out.println("Hata: " + e.getMessage());
            } finally {
                System.out.println("---");
            }
        }
    }
}
```

C) Kod Yazma

9 Bir mağazanın stok sistemini yazan `StokTakip` programını yaz:

- Checked bir `StokYetersizException` sınıfı yaz. Yapıcısı eksik kalan miktarı alsın, mesajı `Stok yetersiz, X adet eksik` olsun ve `getEksik()` metodu bu miktarı döndürsün.
- `Urun` sınıfı: `ad` ve `stok` alanları. `void sat(int adet) throws StokYetersizException` metodu: `adet <= 0` ise `IllegalArgumentException` (mesaj: `Adet pozitif olmalı: X`), `adet > stok` ise `StokYetersizException` fırlatsın; ikisi de değilse stoktan düşüp satışı yazdırsın.
- `main`: Stoğu 10 olan bir "Kalem" için sırasıyla 4, 0, 9, 6 adetlik satış denesin; her denemeyi ayrı `try-catch` ile yakalasın. Beklenen çıktı:

```
4 adet Kalem satıldı, kalan stok: 6
Geçersiz istek: Adet pozitif olmalı: 0
Satılamadı: Stok yetersiz, 3 adet eksik
6 adet Kalem satıldı, kalan stok: 0
```

10 Kullanıcıdan `sayı işlem sayı` biçiminde satırlar okuyan (örn. `12 + 30`) ve sonucu yazdıran `HesapMakinesi` programını yaz. Kullanıcı `q` yazınca program bitsin. Sayılar tam sayıdır; işlemler `+ - * / %`. Program **hiçbir girdide çökmemeli**:

- Sayı olmayan değer → `Hata: geçersiz sayı`
- Sıfıra bölme → `Hata: sıfıra bölünemez`
- Bilinmeyen işlem (örn. `x`) → `IllegalArgumentException` fırlat: `Hata: bilinmeyen işlem: x`
- Üç parçadan oluşmayan satır → `IllegalArgumentException` fırlat: `Hata: biçim 'sayı işlem sayı' olmalı`

```
Hesap> 12 + 30
= 42
Hesap> 7 / 0
Hata: sıfıra bölünemez
Hesap> abc * 2
Hata: geçersiz sayı
Hesap> 5 % 3
= 2
Hesap> 9 x 3
Hata: bilinmeyen işlem: x
Hesap> 8 *
Hata: biçim 'sayı işlem sayı' olmalı
Hesap> q
Güle güle!
```

Çözümler

Cevap anahtarı

1	2	3	4	5
C	E	A	D	B

1 Cevap: C

`IOException`, `Exception`'ın doğrudan alt sınıfıdır ve yolunda `RuntimeException` yoktur; bu yüzden **checked**'tır. Onu fırlatabilecek bir metodu çağıran kod, exception'ı yakalamak ya da `throws` ile bildirmek zorundadır. Diğer dört şık `RuntimeException`'ın alt sınıfıdır, yani unchecked'tır.

2 Cevap: E

`FileNotFoundException`, `IOException`'ın alt sınıfıdır: `Exception` → `IOException` → `FileNotFoundException`. Yolda `RuntimeException` olmadığı için checked'tır. `NumberFormatException` (`IllegalArgumentException` üzerinden), `InputMismatchException` (`NoSuchElementException` üzerinden), `ClassCastException` ve `ArithmeticException` ise `RuntimeException`'dan türer.

3 Cevap: A

`Exception`, `ArithmeticException`'ın üst sınıfıdır ve önce yazıldığı için bütün aritmetik hataları o yakalar; ikinci `catch`'e asla sıra gelmez. Derleyici bu durumda *exception ArithmeticException has already been caught* hatası verir. B, C ve E'de özel olan önce, genel olan sonra yazılmıştır (doğru sıra). D'deki iki sınıf kardeştir; sıraları önemli değildir.

4 Cevap: D

`System.exit(0)` JVM'i hemen kapatır; `finally` dahil hiçbir kod çalışmaz. Diğer bütün durumlarda `finally` çalışır: Hatasız bitişte (A), yakalanan exception'da (B), `return`'den hemen önce (C) ve `catch` yeni bir exception fırlattığında, exception yukarı iletilmeden önce (E).

5 Cevap: B

`a[2]`, 2 elemanlı dizinin sınırı dışındadır → `ArrayIndexOutOfBoundsException`. `X` yazdırılmadan try terk edilir. İlk `catch` (`ArithmeticException`) uymaz; ikinci `catch` (`RuntimeException`) uyar çünkü `ArrayIndexOutOfBoundsException` onun alt sınıfıdır → `R`. Ardından `finally` → `F`. Exception yakalandığı için program devam eder → `S`. Sonuç: `RFS`.

6

- `b(0)`: `c(0)` normal çalışır (`c başladı`, `c bitti`), `b` devam eder (`b: c'den döndü`), sonra `b`'nin `finally`'si.
- `b(5)`: `c(5)` `c başladı` yazar ve exception fırlatır. `c` onu yakalamaz → `b`'ye geçer. `b`'de `catch` yok ama `finally` var: `b: finally` yazılır ve exception `main`'e iletilir. `b: c'den döndü` yazılmaz.
- `main`'de `b(5)`'ten sonraki `main: b'den döndü` atlanır; `catch` exception'ı yakalar. Ardından program devam eder: `son`.

```
Çıktı
c başladı
c bitti
b: c'den döndü
b: finally
c başladı
b: finally
main yakaladı: x pozitif
son
```

7

- `f()`: `return x;` çalıştığında dönecek değer (**1**) kaydedilir. `finally` `x`'i 2 yapar ve yazdırır, ama kaydedilmiş değer değişmez → önce `f: finally, x=2`, sonra `1`.
- `g()`: `try` bir exception fırlatır, ama `finally`'deki `return 3;` hem exception'ı **yutar** hem de 3 döndürür. Program çökmez → `3`. (Bu yüzden `finally` içinde `return` yazılmaz!)
- `h("7")`: `try` 7 döndürecek, önce `finally` → `h: finally`. `h("x")`: `catch` -1 döndürecek, önce `finally` → `h: finally`. Toplam `7 + (-1) = 6`. İki `h: finally` satırı, `println` çalışmadan **önce** yazılır.

```
Çıktı
f: finally, x=2
1
3
h: finally
h: finally
6
```

8

- `"salon"`: Kapılar açılır (`Dış açık`, `İç açık`), oda girilir, blok bitince kapılar **ters sırayla** kapanır (`İç kapalı`, `Dış kapalı`). `main`'deki `finally` → `---`.
- `"kasa"`: Kapılar açılır, exception fırlatılır. Kaynaklar **exception yukarı iletilmeden önce** kapanır (`İç kapalı`, `Dış kapalı`). `gir` exception'ı `throws` ile bildirdiği için `main`'deki `catch` yakalar → `Hata: Kasa kilitli`. Sonra `finally` → `---`.

```
Çıktı
Dış açık
İç açık
salon odasına girildi
İç kapalı
Dış kapalı
---
Dış açık
İç açık
İç kapalı
Dış kapalı
Hata: Kasa kilitli
---
```

9

Yetersiz stok, çağıranın düzeltebileceği (daha az adet isteyebileceği) bir durum olduğu için checked yaptık. Geçersiz adet ise bir programlama hatası sayılır; hazır `IllegalArgumentException` yeterli. `main`'de iki farklı `catch` ile iki durumu ayrı mesajlarla ele alıyoruz.

StokTakip.java

```java
class StokYetersizException extends Exception {
    private final int eksik;

    StokYetersizException(int eksik) {
        super("Stok yetersiz, " + eksik + " adet eksik");
        this.eksik = eksik;
    }

    int getEksik() { return eksik; }
}

class Urun {
    private String ad;
    private int stok;

    Urun(String ad, int stok) { this.ad = ad; this.stok = stok; }

    void sat(int adet) throws StokYetersizException {
        if (adet <= 0) {
            throw new IllegalArgumentException("Adet pozitif olmalı: " + adet);
        }
        if (adet > stok) {
            throw new StokYetersizException(adet - stok);
        }
        stok -= adet;
        System.out.println(adet + " adet " + ad + " satıldı, kalan stok: " + stok);
    }
}

public class StokTakip {
    public static void main(String[] args) {
        Urun kalem = new Urun("Kalem", 10);
        int[] istekler = {4, 0, 9, 6};
        for (int adet : istekler) {
            try {
                kalem.sat(adet);
            } catch (StokYetersizException e) {
                System.out.println("Satılamadı: " + e.getMessage());
            } catch (IllegalArgumentException e) {
                System.out.println("Geçersiz istek: " + e.getMessage());
            }
        }
    }
}
```

Çıktı

```
4 adet Kalem satıldı, kalan stok: 6
Geçersiz istek: Adet pozitif olmalı: 0
Satılamadı: Stok yetersiz, 3 adet eksik
6 adet Kalem satıldı, kalan stok: 0
```

10

Satırı `nextLine()` ile okuyup `split("\\s+")` ile parçalara ayırıyoruz (Bölüm 8); böylece `12 + 30` gibi fazladan boşluklu girdiler de kabul edilir. Hatalı durumların bir kısmını Java kendisi fırlatır (`parseInt` → `NumberFormatException`, `/ 0` → `ArithmeticException`), bir kısmını biz `throw` ile fırlatırız. **Dikkat:** `NumberFormatException`, `IllegalArgumentException`'ın alt sınıfıdır; bu yüzden onun `catch`'i **önce** yazılmalıdır. Ters yazılırsa program derlenmez.

HesapMakinesi.java

```java
import java.util.Scanner;

public class HesapMakinesi {
    static int hesapla(String satir) {
        String[] p = satir.trim().split("\\s+");
        if (p.length != 3) {
            throw new IllegalArgumentException("biçim 'sayı işlem sayı' olmalı");
        }
        int a = Integer.parseInt(p[0]);
        int b = Integer.parseInt(p[2]);
        return switch (p[1]) {
            case "+" -> a + b;
            case "-" -> a - b;
            case "*" -> a * b;
            case "/" -> a / b;
            case "%" -> a % b;
            default -> throw new IllegalArgumentException(
                    "bilinmeyen işlem: " + p[1]);
        };
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.print("Hesap> ");
            String satir = sc.nextLine();
            if (satir.equals("q")) {
                break;
            }
            try {
                System.out.println("= " + hesapla(satir));
            } catch (NumberFormatException e) {
                System.out.println("Hata: geçersiz sayı");
            } catch (ArithmeticException e) {
                System.out.println("Hata: sıfıra bölünemez");
            } catch (IllegalArgumentException e) {
                System.out.println("Hata: " + e.getMessage());
            }
        }
        System.out.println("Güle güle!");
    }
}
```

Çıktı

```
Hesap> 12 + 30
= 42
Hesap> 7 / 0
Hata: sıfıra bölünemez
Hesap> abc * 2
Hata: geçersiz sayı
Hesap> 5 % 3
= 2
Hesap> 9 x 3
Hata: bilinmeyen işlem: x
Hesap> 8 *
Hata: biçim 'sayı işlem sayı' olmalı
Hesap> q
Güle güle!
```

PROJE 3

Banka Hesabı Simülasyonu

Amaç

Bu projede küçük bir bankanın hesap sistemini yazacaksın. Kullanıcı menüden hesap açacak, para yatırıp çekecek, hesaplar arasında transfer yapacak, ay sonunda faizleri işletecek ve hesap dökümlerini görecek. Proje, nesneye yönelik programlama kısmının dört bölümünü tek programda birleştiriyor:

Bölüm	Projede nerede?
9 · Sınıflar, kapsülleme, `static`	`private` alanlar, get metotları, `static` sabitler, hesap numarası sayacı
10 · Kalıtım	`VadesizHesap` ve `VadeliHesap`, `Hesap`'tan türer; `super(...)` ile yapıcı zinciri
11 · Soyut sınıf, polimorfizm	Soyut `Hesap` sınıfı; `aySonu()` ve `cekilebilir()` her hesap türünde farklı çalışır
12 · Exception'lar	Kendi checked exception'ımız, `IllegalArgumentException`, menüde `try-catch`

Programın en önemli özelliği **hiçbir girdide çökmemesi**: Yetersiz bakiye, olmayan hesap numarası, negatif tutar, sayı yerine harf... Her hatalı durum kullanıcıya anlaşılır bir mesajla bildirilir ve menü çalışmaya devam eder.

Gereksinimler

İşlevler (menü seçenekleri):

1. **Hesap aç:** Kullanıcı hesap türünü (vadesiz/vadeli) ve sahibinin adını girer; vadeli hesap için yıllık faiz oranı da istenir. Banka hesaba sırayla `1001`, `1002`, ... numaralarını verir.
2. **Para yatır:** Hesap numarası ve tutar alınır.
3. **Para çek:** Çekilebilecek tutarı aşan istek reddedilir. - **Vadesiz hesap** 1000 TL'ye kadar **ek hesap** (eksi bakiye) kullanabilir. - **Vadeli hesap** eksiye düşemez.
4. **Transfer:** Bir hesaptan diğerine para gönderilir. Gönderen hesapta yeterli para yoksa **hiçbir hesap değişmez**.
5. **Hesap dökümü:** Hesabın işlem geçmişi, her işlemden sonraki bakiyeyle listelenir.
6. **Ay sonu:** Vadeli hesaplara aylık faiz (yıllık oran / 12) eklenir; eksi bakiyedeki vadesiz hesaplardan, borcun %5'i kadar ek hesap faizi kesilir.
7. **Listele:** Bütün hesaplar ve bankadaki toplam para gösterilir.
8. **Çıkış.**

Teknik kurallar:

- `Hesap` **soyut** bir sınıf olmalı; hesap türleri ondan türemeli.
- Yetersiz bakiye için **checked** bir `YetersizBakiyeException` yazılmalı. Sıfır/negatif tutar, olmayan hesap gibi programlama hataları için `IllegalArgumentException` kullanılmalı.
- Hesaplar sabit boyutlu bir **dizide** tutulmalı (`ArrayList`'i Bölüm 13'te göreceğiz). İşlem geçmişi de hesabın içinde bir `String` dizisinde saklanmalı.
- Girdiler `nextLine()` ile okunup `Integer.parseInt` / `Double.parseDouble` ile çevrilmeli (Bölüm 3'teki `nextInt` tuzağına düşmemek için). Menü `switch` ile yazılmalı, her işlem `try-catch` içinde olmalı.

Tasarım

Sınıflar ve sorumlulukları

Sınıf	Tür	Önemli üyeler	Sorumluluğu
`YetersizBakiyeException`	Checked exception	`getEksik()`	Çekilmek istenen tutarın neden reddedildiğini taşır
`Hesap`	**Soyut** sınıf	`no`, `sahip`, `bakiye`, `gecmis[]`; `yatir`, `cek`, `dokum`; soyut `tur()`, `cekilebilir()`, `aySonu()`	Bütün hesapların ortak davranışı: tutar kontrolü, bakiye değişimi, işlem kaydı
`VadesizHesap`	`Hesap`'ın alt sınıfı	`EK_HESAP_LIMITI`, `EK_HESAP_FAIZI`	Ek hesap kuralı: bakiye + 1000 TL'ye kadar çekim, eksi bakiyeye faiz
`VadeliHesap`	`Hesap`'ın alt sınıfı	`yillikOran`	Eksiye düşmeme kuralı ve ay sonu faizi
`Banka`	Sınıf	`hesaplar[]`, `adet`, `sonNo`; `hesapAc`, `bul`, `transfer`, `aySonu`, `listele`	Hesapları saklar, numara verir, hesaplar arası işlemleri yönetir
`BankaUygulamasi`	`main` sınıfı	`menuYaz`, `hesapAc`, `sayiOku`, `tutarOku`	Kullanıcıyla konuşur: menü, girdi okuma, hata mesajları

Kalıtım ağacı ve ezilen metotlar:

```
                    Hesap  (abstract)
          yatir() · cek() · dokum() · toString()
       abstract: tur() · cekilebilir() · aySonu()
                /                          \
        VadesizHesap                    VadeliHesap
  cekilebilir() =                  cekilebilir() = bakiye
    max(0, bakiye + 1000)
  aySonu(): eksiyse %5 faiz kes    aySonu(): yıllık oran / 12 faiz ekle
```

Hata durumları

Durum	Ne fırlatılır?	Kim fırlatır?
Çekilecek tutar, çekilebilir tutardan fazla	`YetersizBakiyeException` (checked)	`Hesap.cek`
Tutar sıfır, negatif, `NaN` veya `Infinity`	`IllegalArgumentException`	`Hesap.tutarKontrol`
Olmayan hesap numarası	`IllegalArgumentException`	`Banka.bul`
Aynı hesaba transfer, geçersiz hesap türü veya faiz oranı	`IllegalArgumentException`	`Banka`, `VadeliHesap`
Banka dolu	`IllegalStateException`	`Banka.hesapAc`
Ad soyad boş	`IllegalArgumentException`	`Banka.hesapAc`
Sayı yerine harf girildi	`NumberFormatException` (Java fırlatır)	`Integer.parseInt` / `Double.parseDouble`

Bütün bu exception'lar `main`'deki menü döngüsünün `try-catch`'inde yakalanır. Böylece iş kuralları (`Hesap`, `Banka`) ekrana bir şey yazmakla uğraşmaz; sadece hatayı **fırlatır**. Mesajı kullanıcıya göstermek arayüzün (`main`) işidir.

> **İpucu: Önce sınıfları, sonra menüyü yaz**
>
> Bu büyüklükte bir programı tek seferde yazıp çalıştırmaya çalışma. Önce `Hesap` ve `VadesizHesap`'ı yaz, `main`'de elle birkaç nesne oluşturup `yatir`/`cek` çağırarak dene. Çalışınca `VadeliHesap`'ı, sonra `Banka`'yı ekle. Menü (`Scanner` + `switch`) en son gelsin. Her adımda programın derlendiğinden emin ol.

Kod

BankaUygulamasi.java

```
import java.util.Locale;
import java.util.Scanner;

// ------------------------------------------------ Özel exception (checked)
class YetersizBakiyeException extends Exception {
    private final double eksik;

    YetersizBakiyeException(int hesapNo, double istenen, double cekilebilir) {
        super(String.format(
                "Hesap %d: %.2f TL istendi, en fazla %.2f TL çekilebilir",
                hesapNo, istenen, cekilebilir));
        this.eksik = istenen - cekilebilir;
    }

    double getEksik() { return eksik; }
}

// ------------------------------------------------ Soyut hesap sınıfı
abstract class Hesap {
    private final int no;
    private final String sahip;
    protected double bakiye;
    private final String[] gecmis = new String[20];    // son 20 işlem
    private int islemSayisi = 0;

    Hesap(int no, String sahip) {
        this.no = no;
        this.sahip = sahip;
        kaydet("Hesap açıldı");
    }

    int getNo() { return no; }
    double getBakiye() { return bakiye; }

    abstract String tur();
    abstract double cekilebilir();      // şu an çekilebilecek en yüksek tutar
    abstract void aySonu();             // her hesap türünün kendi kuralı

    void yatir(double tutar, String aciklama) {
        tutarKontrol(tutar);
        bakiye += tutar;
        kaydet(String.format("%-14s +%10.2f", aciklama, tutar));
    }

    void cek(double tutar, String aciklama) throws YetersizBakiyeException {
        tutarKontrol(tutar);
        if (tutar > cekilebilir()) {
            throw new YetersizBakiyeException(no, tutar, cekilebilir());
        }
        bakiye -= tutar;
        kaydet(String.format("%-14s -%10.2f", aciklama, tutar));
    }
```

```
    protected void kaydet(String islem) {
        if (islemSayisi == gecmis.length) {        // dizi doldu: en eskiyi at
            for (int i = 1; i < gecmis.length; i++) {
                gecmis[i - 1] = gecmis[i];
            }
            islemSayisi--;
        }
        String satir = String.format("%-26s | bakiye %10.2f", islem, bakiye);
        gecmis[islemSayisi++] = satir;
    }

    void dokum() {
        System.out.println("Döküm: " + this);
        for (int i = 0; i < islemSayisi; i++) {
            System.out.println("  " + gecmis[i]);
        }
    }

    private static void tutarKontrol(double tutar) {
        if (!Double.isFinite(tutar) || tutar <= 0) {  // NaN ve Infinity de
            throw new IllegalArgumentException(
                    String.format("Tutar pozitif olmalı: %.2f", tutar));
        }
    }

    @Override
    public String toString() {
        return String.format("%d %-8s %-12s %10.2f TL", no, tur(), sahip, bakiye);
    }
}

// ------------------------------------------------ Hesap türleri
class VadesizHesap extends Hesap {
    static final double EK_HESAP_LIMITI = 1000;
    static final double EK_HESAP_FAIZI = 0.05;     // aylık, borç üzerinden

    VadesizHesap(int no, String sahip) { super(no, sahip); }

    @Override String tur() { return "Vadesiz"; }

    @Override
    double cekilebilir() {                      // eksi olamaz
        return Math.max(0, bakiye + EK_HESAP_LIMITI);
    }

    @Override
    void aySonu() {
        if (bakiye < 0) {
            double faiz = -bakiye * EK_HESAP_FAIZI;
            bakiye -= faiz;
            kaydet(String.format("%-14s -%10.2f", "Ek hesap faizi", faiz));
        }
    }
}

class VadeliHesap extends Hesap {
    private final double yillikOran;              // 0.36 = yıllık %36

    VadeliHesap(int no, String sahip, double yuzde) {
        super(no, sahip);
        if (!Double.isFinite(yuzde) || yuzde <= 0 || yuzde > 100) {
            throw new IllegalArgumentException("Faiz oranı 0-100 arası olmalı");
        }
        this.yillikOran = yuzde / 100;
```

```
    }

    @Override String tur() { return "Vadeli"; }

    @Override double cekilebilir() { return bakiye; }   // eksiye düşemez

    @Override
    void aySonu() {
        double faiz = bakiye * yillikOran / 12;
        if (faiz > 0) {
            bakiye += faiz;
            kaydet(String.format("%-14s +%10.2f", "Aylık faiz", faiz));
        }
    }
}

// ------------------------------------------------ Banka
class Banka {
    private final Hesap[] hesaplar;
    private int adet = 0;
    private int sonNo = 1000;

    Banka(int kapasite) { hesaplar = new Hesap[kapasite]; }

    boolean doluMu() { return adet == hesaplar.length; }

    Hesap hesapAc(int tur, String sahip, double yuzde) {
        if (doluMu()) {
            throw new IllegalStateException("Banka dolu, yeni hesap açılamaz");
        }
        if (sahip.isBlank()) {
            throw new IllegalArgumentException("Ad soyad boş olamaz");
        }
        Hesap h = switch (tur) {
            case 1 -> new VadesizHesap(sonNo + 1, sahip);
            case 2 -> new VadeliHesap(sonNo + 1, sahip, yuzde);
            default -> throw new IllegalArgumentException(
                    "Geçersiz hesap türü: " + tur);
        };
        sonNo++;                        // numara, hesap başarıyla oluşunca artar
        hesaplar[adet++] = h;
        return h;
    }

    Hesap bul(int no) {
        for (int i = 0; i < adet; i++) {
            if (hesaplar[i].getNo() == no) {
                return hesaplar[i];
            }
        }
        throw new IllegalArgumentException("Hesap bulunamadı: " + no);
    }

    void transfer(int kaynakNo, int hedefNo, double tutar)
            throws YetersizBakiyeException {
        Hesap kaynak = bul(kaynakNo);
        Hesap hedef = bul(hedefNo);
        if (kaynak == hedef) {
            throw new IllegalArgumentException("Aynı hesaba transfer yapılamaz");
        }
        kaynak.cek(tutar, "Giden transfer");     // para yetmezse burada durur
        hedef.yatir(tutar, "Gelen transfer");
    }

    void aySonu() {
```

```
        for (int i = 0; i < adet; i++) {
            hesaplar[i].aySonu();                // polimorfizm
        }
    }

    void listele() {
        double toplam = 0;
        for (int i = 0; i < adet; i++) {
            System.out.println(hesaplar[i]);
            toplam += hesaplar[i].getBakiye();
        }
        System.out.printf("Toplam: %.2f TL, %d hesap%n", toplam, adet);
    }
}

// ---------------------------------------------- Kullanıcı arayüzü
public class BankaUygulamasi {
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        Locale.setDefault(Locale.US);            // ondalık ayırıcı: nokta
        Banka banka = new Banka(10);
        menuYaz();
        boolean devam = true;
        while (devam) {
            System.out.print("\nSeçim: ");
            String secim = sc.nextLine().trim();
            try {
                switch (secim) {
                    case "1" -> hesapAc(banka);
                    case "2" -> {
                        Hesap h = banka.bul(sayiOku("Hesap no: "));
                        h.yatir(tutarOku(), "Para yatırma");
                        System.out.printf("Yeni bakiye: %.2f TL%n", h.getBakiye());
                    }
                    case "3" -> {
                        Hesap h = banka.bul(sayiOku("Hesap no: "));
                        h.cek(tutarOku(), "Para çekme");
                        System.out.printf("Yeni bakiye: %.2f TL%n", h.getBakiye());
                    }
                    case "4" -> {
                        int kaynak = sayiOku("Gönderen hesap: ");
                        int hedef = sayiOku("Alıcı hesap: ");
                        banka.transfer(kaynak, hedef, tutarOku());
                        System.out.println("Transfer tamamlandı.");
                    }
                    case "5" -> banka.bul(sayiOku("Hesap no: ")).dokum();
                    case "6" -> {
                        banka.aySonu();
                        System.out.println("Ay sonu işlemleri yapıldı.");
                    }
                    case "7" -> banka.listele();
                    case "0" -> devam = false;
                    default -> System.out.println("Geçersiz seçim: " + secim);
                }
            } catch (YetersizBakiyeException e) {
                System.out.println("HATA: " + e.getMessage());
                System.out.printf("Eksik tutar: %.2f TL%n", e.getEksik());
            } catch (NumberFormatException e) {
                System.out.println("HATA: Sayı bekleniyordu.");
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println("HATA: " + e.getMessage());
            }
        }
        System.out.println("İyi günler!");
```

```
    }

    static void menuYaz() {
        System.out.println("============ JAVA BANK ============");
        System.out.println("1) Hesap aç        2) Para yatır");
        System.out.println("3) Para çek        4) Transfer");
        System.out.println("5) Hesap dökümü    6) Ay sonu işlemleri");
        System.out.println("7) Listele         0) Çıkış");
    }

    static void hesapAc(Banka banka) {
        if (banka.doluMu()) {                  // bilgileri sormadan önce
            throw new IllegalStateException("Banka dolu, yeni hesap açılamaz");
        }
        int tur = sayiOku("Tür (1=Vadesiz, 2=Vadeli): ");
        if (tur != 1 && tur != 2) {            // adı sormadan önce
            throw new IllegalArgumentException("Geçersiz hesap türü: " + tur);
        }
        System.out.print("Ad soyad: ");
        String ad = sc.nextLine().trim();
        double yuzde = 0;
        if (tur == 2) {
            yuzde = ondalikOku("Yıllık faiz (%): ");
        }
        System.out.println("Açıldı: " + banka.hesapAc(tur, ad, yuzde));
    }

    static int sayiOku(String soru) {
        System.out.print(soru);
        return Integer.parseInt(sc.nextLine().trim());
    }

    static double ondalikOku(String soru) {
        System.out.print(soru);
        return Double.parseDouble(sc.nextLine().trim());
    }

    static double tutarOku() { return ondalikOku("Tutar (TL): "); }
}
```

Kodun önemli noktaları

- **Soyut sınıf + polimorfizm:** `Banka.aySonu()` sadece `hesaplar[i].aySonu()` çağırır; vadeli hesap faiz ekler, vadesiz hesap ek hesap faizi keser. Aynı şekilde `Hesap.cek` içindeki `cekilebilir()` çağrısı nesnenin türüne göre `max(0, bakiye + 1000)` veya `bakiye` döndürür (dinamik bağlama, Bölüm 11). Yeni bir hesap türü (örneğin "öğrenci hesabı") eklemek için `Banka`'da tek satır değiştirmek gerekmez; sadece `hesapAc`'taki `switch`'e bir `case` eklenir.
- **`NaN` tuzağı:** `Double.parseDouble("NaN")` ve `Double.parseDouble("Infinity")` hata vermez, geçerli bir `double` döndürür. `NaN` ile yapılan her karşılaştırma `false` olduğu için yalnız `tutar <= 0` kontrolü onu yakalayamaz ve `bakiye += NaN` bütün bakiyeyi bozar. Bu yüzden `tutarKontrol` önce `Double.isFinite(tutar)` sorar. `cekilebilir()` ise `Math.max(0, ...)` ile hiçbir zaman eksi dönmez: ek hesap faizi bakiyeyi `-1050`'ye indirse bile "en fazla -50 TL çekilebilir" gibi anlamsız bir mesaj çıkmaz.
- **Checked exception:** `cek` metodu `throws YetersizBakiyeException` diye bildirdiği için onu çağıran `transfer` de aynı bildirimi yapmak zorunda kaldı (*yakala ya da bildir*). Sonunda `main` yakaladı. Exception sadece mesaj değil, eksik tutarı da (`getEksik()`) taşıyor.
- **Transferin güvenliği:** `transfer` önce `cek`, sonra `yatir` çağırır. Para yetmezse `cek` exception fırlatır ve `yatir` satırına **hiç gelinmez**; iki hesap da değişmeden kalır. Sırayı ters yazsaydık alıcıya para yatıp sonra gönderenden çekemeyebilirdik: Banka yoktan para yaratmış olurdu.

- **Numara sayacı:** `sonNo` ancak hesap nesnesi **başarıyla** oluşturulduktan sonra artar. Geçersiz faiz oranı yüzünden `VadeliHesap` yapıcısı exception fırlatırsa numara boşa harcanmaz.
- **Önce kontrol, sonra soru:** Arayüzdeki `hesapAc`, banka doluysa ya da tür geçersizse kullanıcıdan adı **sormadan** hata verir; kullanıcı boşuna bilgi girmez. Boş ad ise iş kuralı olduğu için `Banka.hesapAc` içinde reddedilir.
- **catch sırası:** `NumberFormatException`, `IllegalArgumentException`'ın alt sınıfıdır; bu yüzden önce yazıldı (Bölüm 12). Ters sırada program derlenmezdi.
- **Girdi okuma:** Her şey `nextLine()` ile okunup çevrildiği için `nextInt` + `nextLine` tuzağı yok. Kullanıcı `abc` yazarsa `parseInt`'in fırlattığı `NumberFormatException` menü döngüsünde yakalanır.
- **Sabit boyutlu diziler:** `Banka` en fazla 10 hesap, her hesap son 20 işlemi tutar. İşlem geçmişi dolunca `kaydet` en eski kaydı silip diğerlerini bir sola kaydırır.

Örnek çalışma

Aşağıdaki oturumda iki hesap açılıyor, para yatırılıyor ve sırayla şu hatalar deneniyor: ek hesap limitini aşan çekim, sayı yerine harf, negatif tutar ve olmayan hesap numarası. Ardından transfer, ek hesaba düşüren bir çekim, ay sonu işlemleri, döküm ve listeleme geliyor:

Çıktı

```
============ JAVA BANK ============
1) Hesap aç        2) Para yatır
3) Para çek        4) Transfer
5) Hesap dökümü    6) Ay sonu işlemleri
7) Listele         0) Çıkış

Seçim: 1
Tür (1=Vadesiz, 2=Vadeli): 1
Ad soyad: Ayşe Demir
Açıldı: 1001 Vadesiz  Ayşe Demir         0.00 TL

Seçim: 1
Tür (1=Vadesiz, 2=Vadeli): 2
Ad soyad: Mehmet Kaya
Yıllık faiz (%): 36
Açıldı: 1002 Vadeli   Mehmet Kaya        0.00 TL

Seçim: 2
Hesap no: 1001
Tutar (TL): 2000
Yeni bakiye: 2000.00 TL

Seçim: 2
Hesap no: 1002
Tutar (TL): 10000
Yeni bakiye: 10000.00 TL

Seçim: 3
Hesap no: 1001
Tutar (TL): 3500
HATA: Hesap 1001: 3500.00 TL istendi, en fazla 3000.00 TL çekilebilir
Eksik tutar: 500.00 TL

Seçim: 3
Hesap no: 1001
Tutar (TL): abc
HATA: Sayı bekleniyordu.

Seçim: 2
Hesap no: 1001
```

```
Tutar (TL): -50
HATA: Tutar pozitif olmalı: -50.00

Seçim: 2
Hesap no: 1005
HATA: Hesap bulunamadı: 1005

Seçim: 4
Gönderen hesap: 1002
Alıcı hesap: 1001
Tutar (TL): 1500
Transfer tamamlandı.

Seçim: 3
Hesap no: 1001
Tutar (TL): 3800
Yeni bakiye: -300.00 TL

Seçim: 6
Ay sonu işlemleri yapıldı.

Seçim: 5
Hesap no: 1001
Döküm: 1001 Vadesiz  Ayşe Demir      -315.00 TL
  Hesap açıldı               | bakiye       0.00
  Para yatırma   +   2000.00 | bakiye    2000.00
  Gelen transfer +   1500.00 | bakiye    3500.00
  Para çekme     -   3800.00 | bakiye    -300.00
  Ek hesap faizi -     15.00 | bakiye    -315.00

Seçim: 7
1001 Vadesiz  Ayşe Demir      -315.00 TL
1002 Vadeli   Mehmet Kaya     8755.00 TL
Toplam: 8440.00 TL, 2 hesap

Seçim: 0
İyi günler!
```

Sonuçları elle kontrol edelim:

- Ayşe'nin vadesiz hesabında 2000 TL varken çekilebilir tutar `2000 + 1000 = 3000` TL'dir; 3500 TL isteği 500 TL eksikle reddedildi.
- Transferden sonra Mehmet'te `10000 − 1500 = 8500`, Ayşe'de `2000 + 1500 = 3500` TL kaldı. Ayşe 3800 TL çekince ek hesaba düştü: `3500 − 3800 = −300`.
- Ay sonunda Ayşe'den `300 × 0.05 = 15` TL ek hesap faizi kesildi (`−315`), Mehmet'e `8500 × 0.36 / 12 = 255` TL faiz eklendi (`8755`). Toplam: `8755 − 315 = 8440` TL.
- Hatalı işlemlerin hiçbiri bakiyeleri değiştirmedi ve döküme girmedi.

Kendin Geliştir

- **ArrayList'e geçir (Bölüm 13):** `Hesap[]` ve `String[] gecmis` yerine `ArrayList` kullan; "banka dolu" ve "geçmiş doldu" sınırları ortadan kalksın. Hesapları numaralarına göre `HashMap<Integer, Hesap>` içinde tutarak `bul` metodunu döngüsüz yaz.
- **Dosyaya kaydet (Bölüm 14):** Çıkışta bütün hesapları bir metin dosyasına yaz, program açılırken geri oku.
- **Yeni hesap türü:** Günlük en fazla 2000 TL çekim izni olan ve faiz almayan bir `OgrenciHesabi` ekle. Sadece yeni bir alt sınıf ve `hesapAc`'ta bir `case` yazman gerektiğini gör.
- **Hesap kapatma:** Bakiyesi sıfır olmayan hesabın kapatılmasını `IllegalStateException` ile engelle; kapanan hesabı diziden çıkar.
- **PIN kontrolü:** Her hesaba 4 haneli bir PIN ekle; para çekme ve transferden önce sor. Üç yanlış denemede hesabı kilitleyen bir `HesapKilitliException` yaz.
- **Tarihli kayıt:** İşlem geçmişine `java.time.LocalDate` ile tarih ekle ve dökümü tarihe göre filtrele.

BÖLÜM 13

Koleksiyonlar ve Generics

Bu Bölümde Öğreneceklerin

- Dizilerin neden yetmediğini ve **koleksiyon** (collection) kavramını
- Sarmalayıcı sınıfları (`Integer`, `Double`...), autoboxing/unboxing ve **`Integer` ile `==` tuzağını**
- `ArrayList` metotlarını ve sınavların sevdiği `remove(int)` / `remove(Object)` farkını
- Dolaşırken eleman silmeyi: `ConcurrentModificationException`, `Iterator` ve `removeIf`
- `HashMap` ile anahtar–değer saklamayı, kelime saymayı; `HashSet` ve `TreeSet` ile tekrarsız veri tutmayı
- Generics: tip güvenliğini, `Kutu<T>` gibi generic sınıf ve generic metot yazmayı
- `Collections` sınıfını, `Comparable` ve `Comparator` ile sıralamayı, lambda ifadelerini

13.1 Dizilerin sınırı: koleksiyonlar

Bölüm 7'de gördüğün dizilerin büyük bir kısıtı var: **Boyutu oluşturulurken belirlenir ve bir daha değişmez.** Sınıfa yeni bir öğrenci geldiğinde diziye "bir kutu daha" ekleyemezsin; daha büyük yeni bir dizi oluşturup eskisini kopyalaman gerekir. Java'nın hazır **koleksiyon** sınıfları bu işi senin yerine yapar:

DiziVsListe.java

```
import java.util.ArrayList;
import java.util.Arrays;

public class DiziVsListe {
    public static void main(String[] args) {
        // Dizi: boyut sabit, büyütmek için yeni dizi + kopyalama gerekir
        String[] dizi = {"Ali", "Ece", "Can"};
        dizi = Arrays.copyOf(dizi, dizi.length + 1);
        dizi[3] = "Naz";
        System.out.println(Arrays.toString(dizi));

        // ArrayList: büyüme işini kendisi yapar
        ArrayList<String> liste = new ArrayList<>();
        liste.add("Ali");
        liste.add("Ece");
        liste.add("Can");
        liste.add("Naz");
        System.out.println(liste);
        System.out.println("Eleman sayısı: " + liste.size());
    }
}
```

Çıktı

```
[Ali, Ece, Can, Naz]
[Ali, Ece, Can, Naz]
Eleman sayısı: 4
```

Koleksiyonlar `java.util` paketindedir ve üç ana aileye ayrılır. Hepsi birer **arayüzdür** (Bölüm 11); altlarında farklı şekilde çalışan sınıflar vardır:

Arayüz	Ne saklar?	Sık kullanılan sınıflar	Örnek kullanım
`List`	Sıralı elemanlar, tekrar olabilir, indeksle erişilir	`ArrayList`, `LinkedList`	Sınıf listesi
`Set`	**Tekrarsız** elemanlar	`HashSet`, `TreeSet`	Farklı kelimeler
`Map`	**Anahtar → değer** çiftleri	`HashMap`, `TreeMap`	Öğrenci no → ad

13.2 Sarmalayıcı sınıflar ve autoboxing

Koleksiyonlar sadece **nesne** saklar; `int`, `double` gibi ilkel tipleri doğrudan saklayamaz. Bu yüzden her ilkel tipin onu bir nesne içine "saran" bir **sarmalayıcı sınıfı** (wrapper class) vardır:

İlkel tip	`byte`	`short`	`int`	`long`	`float`	`double`	`char`	`boolean`
Sarmalayıcı	`Byte`	`Short`	**`Integer`**	`Long`	`Float`	`Double`	**`Character`**	`Boolean`

İkisi hariç hepsi tipin büyük harfle başlayan hali; istisnalar `Integer` ve `Character`. Java ilkel tip ile sarmalayıcı arasındaki dönüşümü kendisi yapar: ilkelden nesneye dönüşüme **autoboxing**, nesneden ilkele dönüşüme **unboxing** denir.

Sarmalayici.java

```
public class Sarmalayici {
    public static void main(String[] args) {
        Integer a = 42;              // autoboxing: int -> Integer
        int b = a + 8;               // unboxing: Integer -> int
        Double d = 3.5;
        System.out.println(a + " " + b + " " + d);

        // Sarmalayıcıların faydalı statik metotları ve sabitleri
        int sayi = Integer.parseInt("2026");
        double oran = Double.parseDouble("0.75");
        boolean dogru = Boolean.parseBoolean("TRUE");
        System.out.println((sayi + 1) + " " + oran * 2 + " " + dogru);
        System.out.println(Integer.MAX_VALUE);
        System.out.println(Character.isDigit('7') + " " + Character.isLetter('7'));
        System.out.println(Character.toUpperCase('q'));
    }
}
```

Çıktı

```
42 50 3.5
2027 1.5 true
2147483647
true false
Q
```

Integer'ları == ile karşılaştırma

`Integer` bir **nesnedir**. Bölüm 8'de String için gördüğün kural burada da geçerli: `==` nesnelerde **içeriği değil referansı** (aynı nesne mi?) karşılaştırır. İşin kötüsü, bu hata bazen ortaya çıkmaz:

IntegerEsitlik.java

```java
public class IntegerEsitlik {
    public static void main(String[] args) {
        Integer a = 127, b = 127;
        Integer c = 128, d = 128;
        System.out.println(a == b);
        System.out.println(c == d);
        System.out.println(c.equals(d));
        System.out.println(c.intValue() == d.intValue());
        int e = 128;
        System.out.println(c == e);   // bir taraf ilkel: değer karşılaştırılır
    }
}
```

Çıktı

```
true
false
true
true
true
```

Neden 127'de `true`, 128'de `false`? Autoboxing arka planda `Integer.valueOf(...)` çağırır. Java, **−128 ile 127 arasındaki** `Integer` nesnelerini önceden oluşturup bir **önbellekte** (cache) tutar ve bu aralıktaki her kutulamada **aynı nesneyi** verir. 128 için ise her seferinde yeni bir nesne oluşturulur; iki farklı nesnenin referansları farklıdır. Bir taraf ilkel `int` ise öbür taraf açılır (unboxing) ve **değerler** karşılaştırılır.

> **Sınavda Çıkar: Integer önbelleği: 127 ile 128 arası**
>
> `Integer x = 127, y = 127;` → `x == y` **true**. `Integer x = 128, y = 128;` → `x == y` **false**. Önbellek aralığı **−128 … 127**'dir. Sarmalayıcıları her zaman `equals` ile karşılaştır. `<`, `>`, `<=`, `>=` ve aritmetik operatörler ise önce unboxing yapar, yani değerle çalışır.

null ve unboxing

İlkel bir `int` asla `null` olamaz; ama `Integer` bir referans olduğu için `null` olabilir. `null` bir sarmalayıcıyı ilkel tipe açmaya çalışırsan program çöker:

NullUnboxing.java

```java
public class NullUnboxing {
    public static void main(String[] args) {
        Integer stok = null;          // nesne referansı: null olabilir
        System.out.println("Stok: " + stok);
        int adet = stok;              // unboxing: stok.intValue() çağrılır
        System.out.println(adet);
    }
}
```

Çıktı – program hata ile durur

```
Stok: null
Exception in thread "main" java.lang.NullPointerException: Cannot invoke
"java.lang.Integer.intValue()" because "stok" is null
        at NullUnboxing.main(NullUnboxing.java:5)
```

> **Sık Yapılan Hata: Görünmez unboxing ve NullPointerException**
>
> `int adet = stok;` satırında hiçbir metot çağrısı görünmüyor; ama derleyici onu `stok.intValue()`'ya çevirir ve `null` üzerinde metot çağrılınca `NullPointerException` oluşur. Bu hatayla en sık, `HashMap`'te olmayan bir anahtarı okurken karşılaşacaksın (13.6).

13.3 ArrayList

`ArrayList`, boyutu otomatik büyüyen bir dizi gibi düşünülebilir. Tanımlarken `<` ve `>` işaretleri arasına **eleman tipini** yazarsın:

```
ArrayList<ElemanTipi> ad = new ArrayList<>();
```

Sağdaki boş `<>` işaretine **elmas** (diamond) denir; tipi soldan çıkardığı için tekrar yazmana gerek kalmaz. `<...>` yazımının adı **generics**'tir (13.8).

ListeTemel.java

```
import java.util.ArrayList;

public class ListeTemel {
    public static void main(String[] args) {
        ArrayList<String> dersler = new ArrayList<>();
        System.out.println(dersler + " boş mu? " + dersler.isEmpty());

        dersler.add("Fizik");                // sona ekle
        dersler.add("Kimya");
        dersler.add("Java");
        dersler.add(0, "Matematik");         // 0. indekse ekle, diğerleri kayar
        System.out.println(dersler + " boyut: " + dersler.size());

        System.out.println("1. indeks: " + dersler.get(1));
        String eski = dersler.set(2, "Biyoloji");    // değiştir, eskiyi döndür
        System.out.println(eski + " gitti: " + dersler);

        System.out.println(dersler.contains("Java") + " "
                + dersler.contains("Kimya"));
        System.out.println(dersler.indexOf("Java") + " "
                + dersler.indexOf("Tarih"));

        dersler.remove("Fizik");             // değere göre sil
        dersler.remove(0);                   // indekse göre sil
        System.out.println(dersler);

        dersler.clear();
        System.out.println(dersler + " boyut: " + dersler.size());
    }
}
```

Çıktı

```
[] boş mu? true
[Matematik, Fizik, Kimya, Java] boyut: 4
1. indeks: Fizik
Kimya gitti: [Matematik, Fizik, Biyoloji, Java]
true false
3 -1
[Biyoloji, Java]
[] boyut: 0
```

Metot	Ne yapar?	Dönüş
`add(e)`	Sona ekler	`boolean` (`true`)
`add(i, e)`	`i`. konuma ekler; sonraki elemanlar **sağa kayar**	—
`get(i)`	`i`. elemanı verir	eleman
`set(i, e)`	`i`. elemanı değiştirir	**eski** eleman
`remove(i)`	`i`. elemanı siler; sonrakiler **sola kayar**	silinen eleman
`remove(o)`	`o`'ya eşit **ilk** elemanı siler	`boolean` (bulundu mu?)
`size()`	Eleman sayısı	`int`
`contains(o)` / `indexOf(o)`	Var mı? / İlk konumu, yoksa **−1**	`boolean` / `int`
`isEmpty()` / `clear()`	Boş mu? / Hepsini sil	`boolean` / —

`contains`, `indexOf` ve `remove(o)` karşılaştırmayı `==` ile değil **`equals`** ile yapar. Listeyi `println` ile doğrudan yazdırabilirsin: `[a, b, c]`.

> **● Sık Yapılan Hata: length, length() ve size()**
>
> Uzunluk için üç farklı yazım var ve karıştırmak derleme hatasıdır: dizi → `dizi.length` (parantez yok), String → `s.length()`, koleksiyon → `liste.size()`. Ayrıca `liste[0]` yazamazsın; `liste.get(0)` kullanılır. Olmayan bir indekse erişim (`liste.get(liste.size())`) `IndexOutOfBoundsException` verir.

Listeyi dolaşmak için Bölüm 7'deki iki döngü aynen kullanılır: İndeks gerekiyorsa `for (int i = 0; i < liste.size(); i++)` ve `liste.get(i)`; sadece elemanlar gerekiyorsa for-each: `for (int not : notlar)`. for-each'te her `Integer` otomatik olarak `int`'e açılır.

remove(int) mi, remove(Object) mi?

`ArrayList<Integer>` ile çalışırken çok sinsi bir tuzak var. `remove` metodunun iki **aşırı yüklenmiş** (overload, Bölüm 6) hali bulunur: `remove(int indeks)` ve `remove(Object eleman)`.

RemoveTuzak.java

```java
import java.util.ArrayList;

public class RemoveTuzak {
    public static void main(String[] args) {
        ArrayList<Integer> liste = new ArrayList<>();
        liste.add(5);
        liste.add(1);
        liste.add(3);
        liste.add(1);
        System.out.println(liste);

        liste.remove(1);                    // 1. İNDEKSTEKİ elemanı siler
        System.out.println(liste);

        liste.remove(Integer.valueOf(1));   // 1 DEĞERİNİ (ilk bulunanı) siler
        System.out.println(liste);
    }
}
```

Çıktı
```
[5, 1, 3, 1]
[5, 3, 1]
[5, 3]
```

> **Sınavda Çıkar: `list.remove(1)` neyi siler?**
>
> Derleyici aşırı yüklenmiş metotlar arasında seçim yaparken önce **kutulama gerektirmeyen** metodu arar. `1` bir `int` olduğu için `remove(int indeks)` seçilir: **1. indeksteki** eleman silinir. **Değeri** silmek için argümanı nesne yapmalısın: `liste.remove(Integer.valueOf(1))` veya `liste.remove((Integer) 1)`. `ArrayList<String>`'de bu karışıklık olmaz, çünkü `"Ali"` zaten bir nesnedir.

13.4 Dolaşırken eleman silmek

"Listedeki çift sayıları sil" gibi bir görevde ilk akla gelen, for-each ile dolaşıp silmektir. Deneyelim. (`new ArrayList<>(List.of(...))` verilen elemanlarla dolu bir liste oluşturmanın kısa yoludur; 13.5'te açıklanıyor.)

ForEachSilme.java
```java
import java.util.ArrayList;
import java.util.List;

public class ForEachSilme {
    public static void main(String[] args) {
        ArrayList<Integer> sayilar = new ArrayList<>(List.of(3, 8, 5, 12, 7));
        for (int s : sayilar) {
            if (s % 2 == 0) {
                sayilar.remove(Integer.valueOf(s));   // dolaşırken değiştir!
            }
        }
        System.out.println(sayilar);
    }
}
```

Çıktı – program hata ile durur
```
Exception in thread "main" java.util.ConcurrentModificationException
        at java.base/java.util.ArrayList$Itr.checkForComodification(ArrayList.java:1096)
        at java.base/java.util.ArrayList$Itr.next(ArrayList.java:1050)
        at ForEachSilme.main(ForEachSilme.java:7)
```

for-each döngüsü arka planda bir **Iterator** (gezici) nesnesi kullanır. Liste, iterator'ın haberi olmadan değiştirildiğinde iterator bunu fark eder ve `ConcurrentModificationException` fırlatır. Doğru yollar şunlardır:

DogruSilme.java
```java
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class DogruSilme {
    public static void main(String[] args) {
        // Yol 1: Iterator ile dolaş, silmeyi iterator'ın remove'u yapsın
        ArrayList<Integer> a = new ArrayList<>(List.of(3, 8, 5, 12, 7));
        Iterator<Integer> it = a.iterator();
        while (it.hasNext()) {
            int s = it.next();
            if (s % 2 == 0) {
                it.remove();          // son next() ile gelen elemanı siler
            }
```

```
        }
        System.out.println(a);

        // Yol 2: removeIf — en kısa yol
        ArrayList<Integer> b = new ArrayList<>(List.of(3, 8, 5, 12, 7));
        b.removeIf(s -> s % 2 == 0);
        System.out.println(b);
    }
}
```

Çıktı

```
[3, 5, 7]
[3, 5, 7]
```

`Iterator`'ın üç metodu vardır: `hasNext()` (sırada eleman var mı?), `next()` (sıradakini ver), `remove()` (az önce verileni sil). `removeIf`'in içindeki `s -> s % 2 == 0` bir **lambda ifadesidir**; şimdilik "her `s` için: `s` çift mi?" diye oku, ayrıntısı 13.10'da.

> **Sık Yapılan Hata: İndeksle baştan sona dolaşırken silmek**
>
> `for (int i = 0; i < liste.size(); i++)` döngüsünde `liste.remove(i)` yaparsan exception oluşmaz ama **eleman atlanır**: Silinen elemanın yerine sağdaki kayar, ardından `i++` o elemanın üzerinden geçer. Yan yana iki çift sayıdan ikincisi silinmeden kalır. Ya `removeIf` / `Iterator` kullan ya da `for (int i = liste.size() - 1; i >= 0; i--)` ile **sondan başa** dolaş: Silme, henüz bakılmamış elemanları kaydırmaz.

13.5 List arayüzü, List.of ve LinkedList

Dizi ile `ArrayList` arasındaki farklar sınavlarda tablo olarak sorulur:

	Dizi	`ArrayList`
Boyut	Sabit, oluştururken belirlenir	Dinamik, kendiliğinden büyür
Eleman tipi	İlkel veya nesne: `int[]`, `String[]`	**Sadece nesne**: `ArrayList<Integer>`
Eleman sayısı	`dizi.length`	`liste.size()`
Okuma / yazma	`dizi[i]`, `dizi[i] = x`	`liste.get(i)`, `liste.set(i, x)`
Ekleme / silme	Yok (yeni dizi gerekir)	`add`, `remove`
Yazdırma	`Arrays.toString(dizi)`	`System.out.println(liste)`
Ne zaman?	Boyut belli ve sabit, ilkel veri	Boyut değişken, sık ekleme/silme

Arayüz tipiyle tanımlama

Profesyonel kodda liste değişkenleri genellikle **arayüz tipiyle** tanımlanır: `List<String> l = new ArrayList<>();`. Bu, Bölüm 11'deki "arayüze göre programla" ilkesidir: Değişkenin tipi `List`, nesnenin kendisi `ArrayList`. İleride `LinkedList`'e geçmek istersen sadece sağ tarafı değiştirirsin. Aynı şekilde `static int toplam(List<Integer> sayilar)` gibi bir metot, parametresi `List` olduğu için hem `ArrayList` hem `LinkedList` kabul eder. Bundan sonraki örneklerde bu yazımı kullanacağız.

List.of: değiştirilemez liste

`List.of(...)` verilen elemanlardan hızlıca bir liste oluşturur; ama bu liste **değiştirilemez** (immutable). Eleman ekleme, silme veya değiştirme denemesi exception fırlatır. Değiştirilebilir bir kopya için `new ArrayList<>(List.of(...))` yazılır:

ListOf.java

```java
import java.util.ArrayList;
import java.util.List;

public class ListOf {
    public static void main(String[] args) {
        List<String> gunler = List.of("Pzt", "Sal", "Çar");
        System.out.println(gunler + " " + gunler.get(1));

        List<String> kopya = new ArrayList<>(gunler);   // değiştirilebilir kopya
        kopya.add("Per");
        System.out.println(kopya);

        gunler.add("Per");                               // değiştirilemez!
    }
}
```

Çıktı – program hata ile durur

```
[Pzt, Sal, Çar] Sal
[Pzt, Sal, Çar, Per]
Exception in thread "main" java.lang.UnsupportedOperationException
        at java.base/java.util.ImmutableCollections.uoe(ImmutableCollections.java:159)
        at java.base/
java.util.ImmutableCollections$AbstractImmutableCollection.add(ImmutableCollections.java:164)
        at ListOf.main(ListOf.java:13)
```

LinkedList

`LinkedList` de `List` arayüzünü uygular, yani `add`, `get`, `remove` gibi bütün metotlar aynıdır. Fark iç yapıdadır: `ArrayList` elemanları arka planda bir **dizide** tutar; `LinkedList` ise her elemanı bir öncekine ve sonrakine bağlı **düğümlerde** (node) tutar.

BagliListe.java

```java
import java.util.LinkedList;

public class BagliListe {
    public static void main(String[] args) {
        LinkedList<String> kuyruk = new LinkedList<>();
        kuyruk.add("Ayşe");
        kuyruk.add("Burak");
        kuyruk.addFirst("Cem");          // başa ekle
        kuyruk.addLast("Deniz");         // sona ekle
        System.out.println(kuyruk);
        System.out.println("Sıradaki: " + kuyruk.removeFirst());
        System.out.println(kuyruk + " ilk: " + kuyruk.getFirst());
    }
}
```

```
Çıktı
[Cem, Ayşe, Burak, Deniz]
Sıradaki: Cem
[Ayşe, Burak, Deniz] ilk: Ayşe
```

İşlem	`ArrayList`	`LinkedList`
`get(i)` (indeksle erişim)	**Hızlı**: doğrudan dizi elemanı	Yavaş: baştan düğüm düğüm yürür
Başa ekleme / baştan silme	Yavaş: bütün elemanlar kayar	**Hızlı**: sadece bağlantılar değişir
Sona ekleme	Hızlı	Hızlı

Java 21'den beri `addFirst`, `getFirst`, `removeFirst` metotları `ArrayList`'te de var; ama `ArrayList`'te başa ekleme bütün elemanları kaydırdığı için yavaştır. Pratikte varsayılan seçim **`ArrayList`**'tir. Bu farkların "ne kadar" hızlı/yavaş olduğunu Bölüm 16'da Big-O ile ölçeceğiz.

Kuyruk, yığın ve öncelik kuyruğu

`LinkedList`'in başa ve sona ekleyebilmesi iki klasik veri yapısını doğurur. **Kuyrukta** (queue) ilk giren ilk çıkar (**FIFO**, first in first out): bankadaki sıra gibi. **Yığında** (stack) son giren ilk çıkar (**LIFO**, last in first out): üst üste dizilmiş tabaklar gibi. İkisi için de önerilen sınıf `ArrayDeque`'tir; `Queue` (kuyruk) ve `Deque` (iki uçlu kuyruk, "dek" diye okunur) arayüzlerini uygular. Kuyruk metotları iki aile halinde gelir:

İşlem	Sorun olursa exception fırlatan	Sorun olursa özel değer dönen
Sona ekle	`add(e)`	`offer(e)`
Baştakini çıkar	`remove()`	`poll()`
Baştakine bak (çıkarma)	`element()`	`peek()`

Kuyruk boşken `remove()` ve `element()` `NoSuchElementException` fırlatır; `poll()` ve `peek()` ise `null` döndürür. Yığın için `push` başa ekler, `pop` baştan çıkarır, `peek` baştakine bakar. **`PriorityQueue`** (öncelik kuyruğu) ise ekleme sırasına değil **önceliğe** bakar: `poll()` her zaman doğal sıraya göre en küçük elemanı verir. En büyüğü önce istersen `Comparator.reverseOrder()` verirsin (`Comparator`'ı bu bölümün sonunda göreceksin).

KuyrukYigin.java

```java
import java.util.ArrayDeque;
import java.util.Comparator;
import java.util.Deque;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.PriorityQueue;
import java.util.Queue;

public class KuyrukYigin {
    public static void main(String[] args) {
        Queue<String> kuyruk = new ArrayDeque<>();
        kuyruk.offer("Ayşe");
        kuyruk.offer("Burak");
        kuyruk.offer("Cem");
        System.out.println(kuyruk.poll() + " çıktı, sırada: " + kuyruk.peek());
        System.out.println(kuyruk);

        Queue<String> bos = new ArrayDeque<>();
        System.out.println(bos.poll() + " " + bos.peek());
        try {
            bos.remove();
```

```
        } catch (NoSuchElementException e) {
            System.out.println("remove(): NoSuchElementException");
        }

        Deque<Integer> yigin = new ArrayDeque<>();
        yigin.push(10);
        yigin.push(20);
        yigin.push(30);
        System.out.println(yigin + " tepe: " + yigin.peek());
        System.out.println(yigin.pop() + " " + yigin.pop() + " " + yigin);

        PriorityQueue<Integer> oncelik = new PriorityQueue<>();
        oncelik.addAll(List.of(40, 10, 30, 50, 20));
        System.out.println(oncelik + " en küçük: " + oncelik.peek());
        while (!oncelik.isEmpty()) {
            System.out.print(oncelik.poll() + " ");
        }
        System.out.println();

        Comparator<Integer> tersSira = Comparator.reverseOrder();
        PriorityQueue<Integer> tersten = new PriorityQueue<>(tersSira);
        tersten.addAll(List.of(40, 10, 30, 50, 20));
        System.out.println(tersten.poll() + " " + tersten.poll());
    }
}
```

Çıktı

```
Ayşe çıktı, sırada: Burak
[Burak, Cem]
null null
remove(): NoSuchElementException
[30, 20, 10] tepe: 30
30 20 [10]
[10, 20, 30, 50, 40] en küçük: 10
10 20 30 40 50
50 40
```

`push(10)`, `push(20)`, `push(30)` sonrası yığın `[30, 20, 10]` görünür: Yığının "tepesi" listenin başıdır, son eklenen ilk çıkar. Dikkat: `PriorityQueue`'yu yazdırınca elemanlar **sıralı çıkmaz** (`[10, 20, 30, 50, 40]`); iç yapısı tam sıralama yapmaz, yalnızca en küçüğün hep başta durmasını sağlar. Sıralı sonuç için `poll()` ile tek tek çekmen gerekir.

13.6 HashMap: anahtar–değer çiftleri

Bir telefon rehberi düşün: İsme (anahtar) bakarsın, numarayı (değer) bulursun. `Map` tam olarak budur: Her **anahtar** (key) bir **değere** (value) eşlenir. Anahtarlar **benzersizdir**; değerler tekrar edebilir. Tanımlarken iki tip yazılır: `HashMap<AnahtarTipi, DegerTipi>`.

HaritaTemel.java

```
import java.util.HashMap;

public class HaritaTemel {
    public static void main(String[] args) {
        HashMap<String, Integer> stok = new HashMap<>();
        stok.put("kalem", 40);
        stok.put("defter", 25);
        stok.put("silgi", 60);
        System.out.println("Defter: " + stok.get("defter"));
```

```
        Integer eski = stok.put("defter", 30);   // anahtar varsa değeri DEĞİŞİR
        System.out.println("Eski: " + eski + ", yeni: " + stok.get("defter"));
        System.out.println("Boyut: " + stok.size());

        System.out.println(stok.get("cetvel"));                // yok -> null
        System.out.println(stok.getOrDefault("cetvel", 0));    // yok -> 0
        System.out.println(stok.containsKey("silgi") + " "
                + stok.containsValue(60));

        stok.remove("silgi");
        System.out.println(stok.containsKey("silgi") + " " + stok.size());
    }
}
```

Çıktı

```
Defter: 25
Eski: 25, yeni: 30
Boyut: 3
null
0
true true
false 2
```

Metot	Ne yapar?
`put(k, v)`	`k` yoksa ekler; **varsa değerini değiştirir** ve eski değeri döndürür
`get(k)`	`k`'nin değeri; anahtar yoksa **`null`**
`getOrDefault(k, varsayilan)`	`k` yoksa `null` yerine verilen varsayılanı döndürür
`containsKey(k)` / `containsValue(v)`	Anahtar / değer var mı?
`remove(k)`	Anahtarı (ve değerini) siler
`size()`, `isEmpty()`	Çift sayısı, boş mu?
`keySet()`, `values()`, `entrySet()`	Anahtarlar, değerler, çiftler (dolaşmak için)

Sıra garantisi yok: TreeMap ve LinkedHashMap

`HashMap` elemanları, anahtarların **hash kodlarına** göre yerleştirir. Bu yüzden yazdırdığında ya da dolaştığında elemanların sırası **ne ekleme sırasıdır ne de alfabetiktir**; Java bu sıra için hiçbir garanti vermez. Sıra önemliyse diğer iki `Map` sınıfını kullan:

HaritaSirasi.java

```
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.TreeMap;

public class HaritaSirasi {
    public static void main(String[] args) {
        String[] sehirler = {"İzmir", "Ankara", "Van", "Bursa", "Edirne"};
        int[] plakalar = {35, 6, 65, 16, 22};

        Map<String, Integer> hash = new HashMap<>();
        Map<String, Integer> bagli = new LinkedHashMap<>();
        Map<String, Integer> agac = new TreeMap<>();
        for (int i = 0; i < sehirler.length; i++) {
            hash.put(sehirler[i], plakalar[i]);
            bagli.put(sehirler[i], plakalar[i]);
```

```
            agac.put(sehirler[i], plakalar[i]);
        }
        System.out.println("HashMap      : " + hash);
        System.out.println("LinkedHashMap: " + bagli);
        System.out.println("TreeMap      : " + agac);
    }
}
```

Çıktı

```
HashMap      : {Van=65, Bursa=16, İzmir=35, Ankara=6, Edirne=22}
LinkedHashMap: {İzmir=35, Ankara=6, Van=65, Bursa=16, Edirne=22}
TreeMap      : {Ankara=6, Bursa=16, Edirne=22, Van=65, İzmir=35}
```

İlk satırdaki sıra bu kitabın derlendiği JDK'da böyle çıktı; başka bir sürümde veya eleman sayısı değişince farklı olabilir. Ona **asla güvenme**. Bu bölümün geri kalanında, çıktısı sıraya bağlı örneklerde `TreeMap` kullanacağız.

Sınıf	Sıra	Not
`HashMap`	Belirsiz	En hızlısı; sıra önemli değilse varsayılan seçim
`LinkedHashMap`	**Ekleme sırası**	`HashMap` + sırayı hatırlar
`TreeMap`	Anahtarlara göre **sıralı**	Anahtarlar `Comparable` olmalı (13.9)

`TreeMap` satırında `İzmir`'in en sona düştüğüne dikkat et: String'ler karakterlerin **Unicode numaralarına** göre karşılaştırılır ve `İ`'nin numarası (304), `Z`'ninkinden (90) büyüktür. Türkçe alfabetik sıralama için `java.text.Collator` sınıfı gerekir (Proje 4'te kullanacağız).

Map'i dolaşmak: kelime frekansı

`Map`'in en klasik kullanımı **saymaktır**: Her kelime bir anahtar, kaç kez geçtiği değer. Sayım bitince `keySet()` (anahtarlar), `values()` (değerler) ve `entrySet()` (çiftler) ile sonuçları dolaşabilirsin:

KelimeSayaci.java

```
import java.util.Map;
import java.util.TreeMap;

public class KelimeSayaci {
    public static void main(String[] args) {
        String metin = "bir elma bir armut iki elma bir de kiraz";
        Map<String, Integer> sayac = new TreeMap<>();

        for (String kelime : metin.split(" ")) {
            int eski = sayac.getOrDefault(kelime, 0);   // ilk kez görülüyorsa 0
            sayac.put(kelime, eski + 1);
        }
        System.out.println(sayac);
        System.out.println("Kelimeler: " + sayac.keySet());
        System.out.println("Sayılar  : " + sayac.values());

        // entrySet ile dolaşma: anahtar ve değer birlikte gelir
        for (Map.Entry<String, Integer> e : sayac.entrySet()) {
            if (e.getValue() > 1) {
                System.out.printf("%-5s %d kez%n", e.getKey(), e.getValue());
            }
        }
    }
}
```

Çıktı

```
{armut=1, bir=3, de=1, elma=2, iki=1, kiraz=1}
Kelimeler: [armut, bir, de, elma, iki, kiraz]
Sayılar  : [1, 3, 1, 2, 1, 1]
bir   3 kez
elma  2 kez
```

`Map.Entry<K, V>` tek bir anahtar–değer çiftidir; `getKey()` ve `getValue()` metotları vardır. `for (String k : sayac.keySet())` ile anahtarlar üzerinden dönüp her birinde `get(k)` çağırmak da olur; ama hem anahtar hem değer gerektiğinde `entrySet` daha verimlidir, çünkü her adımda ayrıca arama yapılmaz.

> **Sık Yapılan Hata: Olmayan anahtarı int'e açmak**
>
> `int adet = sayac.get("muz");` → `"muz"` anahtarı yoksa `get` **`null`** döndürür, `null` da `int`'e açılırken `NullPointerException` fırlatır (13.2). Sayma işlemlerinde her zaman `getOrDefault(k, 0)` kullan ya da önce `containsKey` ile kontrol et.

> **Sınavda Çıkar: Map soruları**
>
> - Aynı anahtarla ikinci `put` **yeni çift eklemez**, değeri değiştirir; `size()` artmaz.
> - `get` olmayan anahtar için **exception değil `null`** döndürür.
> - Anahtarlar tekrarsızdır (`keySet()` bir `Set` döndürür); değerler tekrar edebilir (`values()` bir koleksiyon döndürür).

13.7 Set: HashSet ve TreeSet

`Set` (küme) **tekrarsız** elemanlar tutar. Zaten olan bir elemanı eklemeye çalışırsan `add` hata vermez, sadece `false` döndürür ve hiçbir şey yapmaz. İndeks yoktur; `get(i)` diye bir metot bulunmaz.

KumeOrnek.java

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;
import java.util.TreeSet;

public class KumeOrnek {
    public static void main(String[] args) {
        Set<String> dersler = new HashSet<>();
        System.out.println(dersler.add("Java") + " " + dersler.add("Fizik"));
        System.out.println(dersler.add("Java"));    // zaten var: eklenmez
        System.out.println(dersler.size() + " " + dersler.contains("Fizik"));

        TreeSet<Integer> notlar = new TreeSet<>(List.of(70, 85, 70, 100, 55, 85));
        System.out.println(notlar + " en düşük: " + notlar.first()
                + ", en yüksek: " + notlar.last());

        // Listedeki tekrarları silmenin kısa yolu: kümeye kopyala
        List<String> yoklama = List.of("Can", "Ece", "Ali", "Can", "Ece", "Can");
        List<String> tekil = new ArrayList<>(new LinkedHashSet<>(yoklama));
        System.out.println(tekil + " " + yoklama.size() + " kayıt, "
                + tekil.size() + " kişi");
    }
}
```

Çıktı

```
true true
false
2 true
[55, 70, 85, 100] en düşük: 55, en yüksek: 100
[Can, Ece, Ali] 6 kayıt, 3 kişi
```

`Set`'in üç sınıfı, `Map`'tekilerle aynı mantıkta çalışır: `HashSet` sırasızdır (en hızlısı), `LinkedHashSet` ekleme sırasını korur (yukarıda tekrarlar silinirken ilk görülme sırası korundu), `TreeSet` elemanları sıralı tutar ve `first()`/`last()` gibi ek metotlar sunar.

> **Not: Kendi sınıfların ve equals/hashCode**
>
> `HashSet` ve `HashMap`, iki elemanın "aynı" olup olmadığına `equals` ve `hashCode` metotlarıyla karar verir. Kendi yazdığın bir sınıfın nesnelerini kümeye koyacak veya anahtar yapacaksan bu iki metodu **birlikte** override etmelisin; yoksa içeriği aynı iki nesne "farklı" sayılır. `record`'lar (Bölüm 11) bu metotları otomatik üretir.

13.8 Generics

Neden generics?

Generics'ten önce (Java 5 öncesi) koleksiyonların eleman tipi yoktu; her şey `Object` olarak saklanırdı. Bu **ham tip** (raw type) yazımı hâlâ derlenir, ama ne kadar tehlikeli olduğunu gör:

HamListe.java

```java
import java.util.ArrayList;

public class HamListe {
    public static void main(String[] args) {
        ArrayList liste = new ArrayList();      // eleman tipi yok (ham tip)
        liste.add("Ali");
        liste.add(42);                          // derleyici itiraz etmez
        String ilk = (String) liste.get(0);     // her okumada cast gerekir
        System.out.println(ilk);
        String ikinci = (String) liste.get(1);  // çalışırken patlar
        System.out.println(ikinci);
    }
}
```

Çıktı – program hata ile durur

```
Ali
Exception in thread "main" java.lang.ClassCastException: class java.lang.Integer cannot be cast to
class java.lang.String (java.lang.Integer and java.lang.String are in module java.base of loader
'bootstrap')
        at HamListe.main(HamListe.java:10)
```

Aynı kodu eleman tipi belirterek yazınca hata **derleme anına** taşınır; program hiç çalışmadan sorunu görürsün:

TipliListe.java

```java
import java.util.ArrayList;

public class TipliListe {
    public static void main(String[] args) {
        ArrayList<String> liste = new ArrayList<>();
        liste.add("Ali");
        liste.add(42);
        String ilk = liste.get(0);              // cast gerekmez
    }
}
```

Derleyici çıktısı – program derlenmez

```
TipliListe.java:7: error: incompatible types: int cannot be converted to String
        liste.add(42);
                  ^
Note: Some messages have been simplified; recompile with -Xdiags:verbose to get full output
1 error
```

Generics'in (generic programming, **tip parametreli programlama**) iki kazancı: **tip güvenliği** (yanlış tipte eleman derlenmez) ve **cast'ten kurtulmak** (`get` zaten `String` döndürür).

Generic sınıf yazmak

Kendi sınıfına da bir **tip parametresi** verebilirsin. Sınıf adından sonra `<T>` yazılır; `T`, sınıf kullanılırken verilecek gerçek tipin yerini tutar:

KutuOrnek.java

```java
class Kutu<T> {
    private T icerik;

    public void koy(T icerik) {
        this.icerik = icerik;
    }

    public T al() {
        return icerik;
    }
}

public class KutuOrnek {
    public static void main(String[] args) {
        Kutu<String> mektup = new Kutu<>();
        mektup.koy("Merhaba");
        String s = mektup.al();             // cast yok
        System.out.println(s.toUpperCase());

        Kutu<Integer> sayi = new Kutu<>();
        sayi.koy(7);
        int k = sayi.al() * 6;              // unboxing
        System.out.println(k);
    }
}
```

Çıktı

```
MERHABA
42
```

`Kutu<String>` için derleyici, sınıftaki her `T`'yi `String` gibi düşünür: `koy` sadece `String` kabul eder, `al` `String` döndürür. Tip parametresi adları geleneksel olarak tek büyük harftir: `T` (type, herhangi bir tip), `E` (element, `List<E>`), `K` ve `V` (key/value, `Map<K, V>`). Birden fazla parametre virgülle yazılır: `class Ikili<A, B>`.

Generic metot

Sadece tek bir metodu generic yapmak da mümkündür. Tip parametresi, **dönüş tipinden önce** yazılır:

GenericMetot.java

```java
public class GenericMetot {
    static <T> void yazdir(T[] dizi) {          // her tipte dizi için çalışır
        for (T eleman : dizi) {
            System.out.print(eleman + " ");
        }
        System.out.println();
    }

    // T'nin compareTo metodu olmalı: sınırlı tip parametresi
    static <T extends Comparable<T>> T buyugu(T a, T b) {
        return a.compareTo(b) >= 0 ? a : b;
    }

    public static void main(String[] args) {
        Integer[] sayilar = {3, 1, 4};
        String[] harfler = {"x", "y"};
        yazdir(sayilar);
        yazdir(harfler);
        System.out.println(buyugu(12, 7) + " " + buyugu("Zeynep", "Ali"));
    }
}
```

Çıktı

```
3 1 4
x y
12 Zeynep
```

`<T extends Comparable<T>>` ifadesi "T herhangi bir tip olabilir, **yeter ki** karşılaştırılabilir olsun" demektir; böylece metodun içinde `compareTo` çağrılabilir. `Comparable`'ı 13.9'da göreceksin.

İlkel tipler kullanılamaz

Tip parametresi yerine **sadece referans tipleri** (sınıflar, arayüzler) yazılabilir:

IlkelGeneric.java

```java
import java.util.ArrayList;
import java.util.List;

public class IlkelGeneric {
    public static void main(String[] args) {
        List<int> sayilar = new ArrayList<>();
    }
}
```

Derleyici çıktısı – program derlenmez

```
IlkelGeneric.java:6: error: unexpected type
        List<int> sayilar = new ArrayList<>();
                ^
  required: reference
  found:    int
1 error
```

Sınavda Çıkar: Generics'te en sık sorulanlar

- `List<int>`, `Map<String, double>` **derlenmez** → `List<Integer>`, `Map<String, Double>` yazılır.
- `List<Integer> l = new ArrayList<>();` → sağdaki `<>` elmas, tipi soldan çıkarır.
- `new List<>()` derlenmez: `List` bir arayüzdür, nesnesi oluşturulamaz.
- Ham tip (`ArrayList liste`) derlenir ama tip kontrolü yoktur; hata çalışma anında `ClassCastException` olarak çıkar.

Joker karakterler: `? extends` ve `? super`

`Integer`, sayı sarmalayıcılarının ortak üst sınıfı `Number`'ın alt sınıfıdır. Yine de `List<Integer>`, `List<Number>`'ın alt tipi **değildir**: Generics'te tip parametresi kalıtımı taşımaz. `static double topla(List<Number> l)` yazarsan bu metoda `List<Integer>` veremezsin. Çözüm **joker karakterdir** (wildcard) `?`:

Yazım	Kabul ettiği liste	Ne yapılır?
`List<? extends Number>`	`Number` ya da alt tipi (`Integer`, `Double`...) elemanlı herhangi bir liste	**Okumak** için: elemanlar `Number` olarak alınır, eleman eklenemez
`List<? super Integer>`	`Integer` ya da üst tipi (`Number`, `Object`) elemanlı liste	**Yazmak** için: `Integer` eklenebilir

Hatırlama kuralı: listeden **veri alıyorsan** `extends`, listeye **veri koyuyorsan** `super`.

Joker.java

```java
import java.util.ArrayList;
import java.util.List;

public class Joker {
    static double topla(List<? extends Number> liste) {
        double toplam = 0;
        for (Number n : liste) {
            toplam += n.doubleValue();
        }
        return toplam;
    }

    static void birIkiUcEkle(List<? super Integer> liste) {
        liste.add(1);
        liste.add(2);
        liste.add(3);
    }

    public static void main(String[] args) {
        List<Integer> tamlar = List.of(1, 2, 3);
        List<Double> ondaliklar = List.of(1.5, 2.5);
        System.out.println(topla(tamlar) + " " + topla(ondaliklar));

        List<Number> sayilar = new ArrayList<>();
        List<Object> nesneler = new ArrayList<>();
        birIkiUcEkle(sayilar);
```

```
        birIkiUcEkle(nesneler);
        System.out.println(sayilar + " " + nesneler);
        System.out.println(sayilar.getClass() == nesneler.getClass());
    }
}
```

Çıktı

```
6.0 4.0
[1, 2, 3] [1, 2, 3]
true
```

`topla` hem `List<Integer>` hem `List<Double>` kabul etti; `birIkiUcEkle` ise `List<Number>` ve `List<Object>` ile çalıştı. Jokerin sınırlarını derleyici korur: `List<Integer> tamlar` için `List<Number> sayilar = tamlar;` satırı derlenmez (`List<Integer>`, `List<Number>`'a dönüştürülemez); `List<? extends Number> okunur = tamlar;` yazıp `okunur.add(4);` dersen de `int cannot be converted to CAP#1` hatası alırsın, çünkü gerçek eleman tipi bilinmez.

Tip silme (type erasure)

Generics bir **derleme zamanı** özelliğidir. Derleyici tip kontrolünü yapar, sonra `<Integer>`, `<String>` bilgisini **siler**; program çalışırken hepsi aynı sınıftır. `Joker` programının son satırı bunu gösterdi: `List<Number>` ile `List<Object>` aynı `ArrayList` sınıfıdır (`true`). Çalışma zamanında `T` bilinmediği için şunlar yasaktır:

```
import java.util.List;

class Kutu<T> {
    T[] dizi = new T[5];                 // T çalışırken bilinmez
    static void yaz(List<String> l) { }
    static void yaz(List<Integer> l) { }  // silinince ikisi de yaz(List)
}

public class TipSilmeHata {
    public static void main(String[] args) { }
}
```

Derleyici çıktısı – program derlenmez

```
TipSilmeHata.java:6: error: name clash: yaz(List<Integer>) and yaz(List<String>) have the same
erasure
    static void yaz(List<Integer> l) { }  // silinince ikisi de yaz(List)
                ^
TipSilmeHata.java:4: error: generic array creation
    T[] dizi = new T[5];                 // T çalışırken bilinmez
               ^
2 errors
```

13.9 Collections sınıfı ve Comparable

`java.util.Collections` sınıfı, koleksiyonlar üzerinde çalışan hazır **statik** metotlar sunar (diziler için `Arrays` sınıfı neyse, listeler için `Collections` odur):

CollectionsOrnek.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Random;
```

```java
public class CollectionsOrnek {
    public static void main(String[] args) {
        List<Integer> sayilar = new ArrayList<>(List.of(42, 7, 19, 88, 3, 19));

        Collections.sort(sayilar);                    // küçükten büyüğe
        System.out.println("sort     : " + sayilar);
        Collections.reverse(sayilar);                 // ters çevir
        System.out.println("reverse  : " + sayilar);
        System.out.println("max, min : " + Collections.max(sayilar) + ", "
                + Collections.min(sayilar));
        System.out.println("frequency: " + Collections.frequency(sayilar, 19));
        Collections.swap(sayilar, 0, 5);              // 0. ve 5. yer değiştirir
        System.out.println("swap     : " + sayilar);
        Collections.shuffle(sayilar, new Random(42));  // karıştır (sabit tohum)
        System.out.println("shuffle  : " + sayilar);

        List<String> adlar = new ArrayList<>(List.of("ece", "Can", "ali", "Bora"));
        Collections.sort(adlar);
        System.out.println(adlar);                    // büyük harfler önce!
    }
}
```

Çıktı

```
sort     : [3, 7, 19, 19, 42, 88]
reverse  : [88, 42, 19, 19, 7, 3]
max, min : 88, 3
frequency: 2
swap     : [3, 42, 19, 19, 7, 88]
shuffle  : [42, 3, 7, 88, 19, 19]
[Bora, Can, ali, ece]
```

Son satırda `Bora` ve `Can`, `ali`'den önce geldi: String sıralaması Unicode numarasına göredir ve bütün büyük harflerin numarası küçük harflerinkinden küçüktür (`'B'` = 66, `'a'` = 97). Büyük/küçük harf duyarsız sıralama için `adlar.sort(String.CASE_INSENSITIVE_ORDER)` kullanılır. `shuffle` her çalıştırmada farklı karıştırır; burada aynı sonucu görmen için `new Random(42)` ile sabit tohum verdik.

> **İpucu: Collection mı, Collections mı?**
>
> `Collection` (tekil) `List` ve `Set`'in üst **arayüzüdür**. `Collections` (çoğul, sonu **s**) ise `sort`, `max`, `reverse` gibi statik yardımcı metotların bulunduğu **sınıftır**. Dizilerde aynı işleri `Arrays.sort(dizi)` ve `Arrays.toString(dizi)` yapar.

Kendi sınıflarını sıralamak: Comparable

`Collections.sort` sayıları ve String'leri nasıl sıralayacağını bilir; peki `Ogrenci` nesnelerini? Bölüm 11'de gördüğün gibi, sınıfa **`Comparable<T>`** arayüzünü uygulatıp `compareTo` metodunu yazarsan sınıfın bir **doğal sırası** (natural ordering) olur. `Arrays.sort` gibi `Collections.sort`, `Collections.max`, `TreeSet` ve `TreeMap` de bu sırayı kullanır:

ComparableOrnek.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

class Ogrenci implements Comparable<Ogrenci> {
    private String ad;
    private int no;
```

```
    public Ogrenci(String ad, int no) { this.ad = ad; this.no = no; }

    @Override
    public int compareTo(Ogrenci diger) {      // doğal sıra: numaraya göre
        return Integer.compare(this.no, diger.no);
    }

    @Override
    public String toString() { return ad + "(" + no + ")"; }
}

public class ComparableOrnek {
    public static void main(String[] args) {
        List<Ogrenci> sinif = new ArrayList<>();
        sinif.add(new Ogrenci("Zeynep", 1203));
        sinif.add(new Ogrenci("Burak", 1187));
        sinif.add(new Ogrenci("Ayşe", 1250));
        sinif.add(new Ogrenci("Mert", 1199));

        Collections.sort(sinif);
        System.out.println(sinif);
        System.out.println("En büyük no: " + Collections.max(sinif));
    }
}
```

Çıktı

```
[Burak(1187), Mert(1199), Zeynep(1203), Ayşe(1250)]
En büyük no: Ayşe(1250)
```

Hatırlatma: `a.compareTo(b)` `a` önce gelmeliyse **negatif**, eşitse **0**, sonra gelmeliyse **pozitif** döndürür. Hazır metotları kullan: `Integer.compare`, `Double.compare`, `String`'in `compareTo`'su. `return this.no - diger.no;` kısa görünür ama çok büyük pozitif ve negatif sayılarda çıkarma **taşar** (Bölüm 2) ve işaret ters döner. Sınıf `Comparable` uygulamıyorsa `Collections.sort(liste)` satırı **derlenmez**.

13.10 Lambda ifadeleri ve Comparator

Lambda nedir?

Bir öğrenci listesini bazen ada, bazen ortalamaya göre sıralamak isteyebilirsin. Doğal sıra tek olduğu için bu iş `Comparator<T>` arayüzüyle yapılır: Sıralama kuralı sınıfın dışında, ayrı bir nesne olarak verilir. `Comparator`'ın tek bir soyut metodu vardır: `int compare(T a, T b)`. Lambda olmasaydı her sıralama için `class AdaGore implements Comparator<Ogrenci> { ... }` gibi ayrı bir sınıf yazman gerekirdi. Lambda ise bu metodu tek satırda yazmanı sağlar: `(a, b) -> a.getAd().compareTo(b.getAd())`.

Lambda ifadesi (lambda expression), **adı olmayan kısa bir fonksiyondur**: `(parametreler) -> gövde`. Sadece **tek soyut metodu olan** bir arayüzün (buna **fonksiyonel arayüz**, functional interface, denir) beklendiği yerde kullanılabilir. Parametrelerin tipini yazmazsın; derleyici bağlamdan çıkarır (bir `Comparator<Ogrenci>` beklendiğini görünce `a` ve `b`'nin `Ogrenci` olduğunu anlar).

Lambda	Anlamı
`x -> x * 2`	Bir parametre: parantez gerekmez; ifadenin değeri döndürülür
`(a, b) -> a + b`	İki parametre: parantez şart
`() -> System.out.println("Selam")`	Parametresiz
`s -> s.isEmpty()`	`boolean` döndürür: bir **koşul**
`x -> { int y = x * x; return y + 1; }`	Çok satırlı: süslü parantez ve `return` gerekir

Koleksiyonlarla en sık kullanacağın üç yer: `forEach` (her elemanla bir şey yap), `removeIf` (koşulu sağlayanı sil) ve `sort` (karşılaştır).

LambdaGiris.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

public class LambdaGiris {
    public static void main(String[] args) {
        List<String> sehirler = new ArrayList<>(
                List.of("Rize", "Ankara", "Muş", "Kars", "Van", "Bolu"));

        sehirler.forEach(s -> System.out.print(s.length() + " "));
        System.out.println();

        sehirler.removeIf(s -> s.length() <= 3);    // 3 harf ve altını sil
        sehirler.forEach(System.out::println);      // metot referansı

        Map<String, Integer> plaka = new TreeMap<>();
        plaka.put("Rize", 53);
        plaka.put("Kars", 36);
        plaka.forEach((ad, no) -> System.out.println(ad + " = " + no));
    }
}
```

Çıktı

```
4 6 3 4 3 4
Rize
Ankara
Kars
Bolu
Kars = 36
Rize = 53
```

`System.out::println` bir **metot referansıdır** (method reference): `s -> System.out.println(s)` lambda'sının kısaltmasıdır. Aynı şekilde `Ogrenci::getAd`, `o -> o.getAd()` demektir. `Map`'in `forEach`'i iki parametreli bir lambda alır: anahtar ve değer.

Sık Yapılan Hata: Lambda içinden yerel değişkeni değiştirmek

Lambda, dışındaki yerel değişkenleri **okuyabilir** ama **değiştiremez**; kullandığı yerel değişkenler `final` veya "fiilen final" (hiç değişmeyen) olmalıdır:

```
import java.util.List;

public class LambdaSayac {
    public static void main(String[] args) {
        List<Integer> notlar = List.of(70, 85, 40);
        int toplam = 0;
        notlar.forEach(n -> toplam += n);
        System.out.println(toplam);
    }
}
```

Derleyici çıktısı – program derlenmez

```
LambdaSayac.java:7: error: local variables referenced from a lambda expression must be final or
effectively final
        notlar.forEach(n -> toplam += n);
                               ^
1 error
```

Toplam, sayaç gibi değişen değerler için normal for-each döngüsü kullan.

Comparator ile farklı sıralamalar

`Comparator` arayüzünün hazır **fabrika metotları** sayesinde çoğu zaman `compare` yazmana bile gerek kalmaz:

- `Comparator.comparing(Ogrenci::getAd)` → ada göre (artan)
- `.reversed()` → o ana kadar kurulan sıralamanın tersi
- `.thenComparing(...)` → ilk ölçüt eşitse ikinci ölçüte bak

ComparatorOrnek.java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Ogrenci {
    private String ad;
    private double ortalama;

    public Ogrenci(String ad, double ortalama) {
        this.ad = ad;
        this.ortalama = ortalama;
    }

    public String getAd() { return ad; }
    public double getOrtalama() { return ortalama; }

    @Override
    public String toString() { return ad + "/" + ortalama; }
}

public class ComparatorOrnek {
    // Orijinal listeyi bozmamak için kopyasını sıralayıp yazdırır
    static void sirala(String baslik, List<Ogrenci> liste, Comparator<Ogrenci> c) {
        List<Ogrenci> kopya = new ArrayList<>(liste);
        kopya.sort(c);
```

```
        System.out.printf("%-17s: %s%n", baslik, kopya);
    }

    public static void main(String[] args) {
        List<Ogrenci> sinif = new ArrayList<>();
        sinif.add(new Ogrenci("Zeynep", 88.5));
        sinif.add(new Ogrenci("Burak", 72.0));
        sinif.add(new Ogrenci("Ayşe", 88.5));
        sinif.add(new Ogrenci("Mert", 64.25));

        sirala("Ad (lambda)", sinif, (a, b) -> a.getAd().compareTo(b.getAd()));
        sirala("Ad (comparing)", sinif, Comparator.comparing(Ogrenci::getAd));
        sirala("Ortalama artan", sinif, Comparator.comparing(Ogrenci::getOrtalama));
        sirala("Ortalama azalan", sinif,
                Comparator.comparing(Ogrenci::getOrtalama).reversed());
        sirala("Ort. azalan + ad", sinif,
                Comparator.comparing(Ogrenci::getOrtalama).reversed()
                        .thenComparing(Ogrenci::getAd));
    }
}
```

Çıktı

```
Ad (lambda)      : [Ayşe/88.5, Burak/72.0, Mert/64.25, Zeynep/88.5]
Ad (comparing)   : [Ayşe/88.5, Burak/72.0, Mert/64.25, Zeynep/88.5]
Ortalama artan   : [Mert/64.25, Burak/72.0, Zeynep/88.5, Ayşe/88.5]
Ortalama azalan  : [Zeynep/88.5, Ayşe/88.5, Burak/72.0, Mert/64.25]
Ort. azalan + ad : [Ayşe/88.5, Zeynep/88.5, Burak/72.0, Mert/64.25]
```

Zeynep ile Ayşe'nin ortalaması eşit (88.5). Sadece ortalamaya göre sıralarken eşit elemanlar **listedeki eski sıralarını korur** (Java'nın `sort`'u kararlıdır, *stable*); bu yüzden Zeynep önde kaldı. Son satırda `thenComparing` devreye girdi ve eşitlik ada göre çözüldü: Ayşe öne geçti.

	`Comparable<T>`	`Comparator<T>`
Paket	`java.lang`	`java.util`
Metot	`int compareTo(T diger)`	`int compare(T a, T b)`
Nerede yazılır?	Sınıfın **içinde** (`implements`)	Sınıfın **dışında**: lambda veya ayrı nesne
Kaç farklı sıralama?	Tek (doğal sıra)	İstediğin kadar
Kullanım	`Collections.sort(liste)`	`liste.sort(c)` veya `Collections.sort(liste, c)`

`Collections.max(liste, c)` ve `Collections.min(liste, c)` de bir `Comparator` alabilir. Doğal sıranın tersi için `Comparator.reverseOrder()` kullanılır: `sayilar.sort(Comparator.reverseOrder());`.

> ● **Sınavda Çıkar: `reversed()` nereye yazılırsa oraya kadar olanı ters çevirir**
>
> `comparing(getOrtalama).reversed().thenComparing(getAd)` → ortalama **azalan**, eşitse ad **artan**. Ama `comparing(getOrtalama).thenComparing(getAd).reversed()` → **ikisi birden** ters döner: ortalama azalan, eşitse ad **azalan**. Hangi ölçütün ters olacağını `reversed()`'ın konumu belirler.

13.11 Stream API'ye giriş

Lambda'ların en çok kullanıldığı yer **stream**'dir (akış). Koleksiyondan `stream()` ile başlayan zincirde elemanlar bir boru hattından geçer: önce **ara işlemler** (`filter`, `map`, `sorted`...), sonda bir **son işlem** (`count`, `sum`, `collect`, `forEach`...). Stream veriyi saklamaz ve kaynak listeyi **değiştirmez**; sonucu yeni bir nesne olarak verir.

İşlem	Ne yapar?	Tür
`filter(koşul)`	Koşulu sağlayan elemanları geçirir	ara
`map(dönüşüm)`	Her elemanı dönüştürür (`String::toUpperCase`)	ara
`sorted()`	Doğal sıraya (veya verilen `Comparator`'a) göre sıralar	ara
`mapToInt(...)`	`int` akışı üretir; `sum()`, `average()`, `max()` bu akışta bulunur	ara
`count()`	Eleman sayısını `long` olarak verir	son
`collect(Collectors.toList())`	Sonucu listeye toplar; Java 16+'da kısası `toList()`	son
`forEach(...)`	Her eleman için bir iş yapar	son

StreamGiris.java

```
import java.util.List;
import java.util.stream.Collectors;

public class StreamGiris {
    public static void main(String[] args) {
        List<Integer> notlar = List.of(72, 45, 88, 91, 60, 55, 100);

        long gecen = notlar.stream().filter(n -> n >= 60).count();
        int toplam = notlar.stream().mapToInt(n -> n).sum();
        double ort = notlar.stream().mapToInt(n -> n).average().orElse(0);
        System.out.println(gecen + " " + toplam + " " + ort);

        List<Integer> yuksek = notlar.stream()
                .filter(n -> n >= 70)
                .sorted()
                .collect(Collectors.toList());
        System.out.println(yuksek);

        List<String> isimler = List.of("zeynep", "ali", "can", "ece");
        List<String> kisalar = isimler.stream()
                .filter(s -> s.length() <= 3)
                .map(String::toUpperCase)
                .sorted()
                .toList();
        System.out.println(kisalar);
        System.out.println(notlar);
    }
}
```

Çıktı

```
5 511 73.0
[72, 88, 91, 100]
[ALI, CAN, ECE]
[72, 45, 88, 91, 60, 55, 100]
```

İlk satırda 7 notun 5'i 60 ve üstüdür; toplam 511, ortalama 511 / 7 = 73.0'dır. `average()` boş bir akışta sonuç veremeyeceği için `OptionalDouble` döndürür; `.orElse(0)` "sonuç yoksa 0 al" demektir. Son satır, `notlar` listesinin değişmediğini gösterir. Lambda içinde `toplam += n` yazamadığın yerde (önceki `hata` kutusu), stream'in `sum()` metodu işini görür.

Sınavda Çıkar: Stream'in iki kuralı

1. **Son işlem yoksa hiçbir şey çalışmaz.** `liste.stream().filter(...).map(...)` ifadesi tek başına eleman işlemez; sonuç almak için `count`, `collect`, `forEach` gibi bir son işlem gerekir.
2. **Stream tek kullanımlıktır.** Bir son işlemden sonra aynı stream nesnesini yeniden kullanırsan `IllegalStateException` (*stream has already been operated upon or closed*) alırsın.

Bölüm Özeti – Bilmen Gerekenler

- Dizi boyutu sabittir; koleksiyonlar dinamiktir. `List` (sıralı, tekrar olabilir), `Set` (tekrarsız), `Map` (anahtar → değer). Hepsi `java.util`'de.
- Koleksiyonlar sadece nesne tutar: `int` → `Integer`, `double` → `Double`, `char` → `Character`. Dönüşüm otomatiktir (**autoboxing/unboxing**).
- `Integer`'ları `equals` ile karşılaştır: `==` sadece −128…127 arasında (önbellek) tesadüfen doğru çalışır. `null` bir `Integer`'ı `int`'e açmak → `NullPointerException`.
- `ArrayList`: `add`, `add(i, e)`, `get`, `set`, `remove`, `size`, `contains`, `indexOf` (yoksa −1), `isEmpty`, `clear`. `List<Integer>`'da `remove(1)` **indeks** siler; değer için `remove(Integer.valueOf(1))`.
- for-each içinde listeden silmek → `ConcurrentModificationException`. Doğrusu: `Iterator.remove()`, `removeIf(...)` veya sondan başa indeksli döngü.
- Değişkeni arayüz tipiyle tanımla: `List<String> l = new ArrayList<>();`. `List.of(...)` değiştirilemez. `LinkedList` başa ekleme/silmede, `ArrayList` indeksle erişimde hızlıdır.
- `HashMap`: `put` (varsa değiştirir), `get` (yoksa `null`), `getOrDefault`, `containsKey`, `remove`, `keySet`, `values`, `entrySet`. Sırası belirsizdir: sıralı için `TreeMap`, ekleme sırası için `LinkedHashMap`.
- `HashSet` tekrarsız (sırasız), `TreeSet` tekrarsız ve sıralı; tekrar eklenen eleman için `add` → `false`.
- Generics: tip güvenliği + cast'siz okuma. `class Kutu<T>`, `static <T> T metot(...)`. `List<int>` derlenmez.
- `Collections.sort/reverse/max/min/shuffle/frequency`. Kendi sınıfın için `Comparable` (`compareTo`, tek doğal sıra) veya `Comparator` (`comparing`, `thenComparing`, `reversed`, sınırsız sıralama).
- Lambda: `(parametreler) -> gövde`; fonksiyonel arayüz yerine geçer. `forEach`, `removeIf`, `sort` ile kullanılır; dışarıdaki yerel değişkeni değiştiremez.
- **Kuyruk** FIFO (`offer`/`poll`/`peek`), **yığın** LIFO (`push`/`pop`/`peek`); ikisi için `ArrayDeque`. `PriorityQueue.poll()` hep en küçüğü verir; `println` sıralı yazmaz. `remove()`/`element()` boşken exception, `poll()`/`peek()` `null` verir.
- Joker: `List<? extends T>` okumak, `List<? super T>` yazmak içindir; `List<Integer>`, `List<Number>` değildir. **Tip silme:** generics derleme zamanındadır; `new T()` ve `new T[n]` yazılamaz.
- **Stream:** `kaynak.stream()` → `filter`/`map`/`sorted` (ara) → `count`/`sum`/`collect(Collectors.toList())`/`toList()` (son). Orijinal listeyi değiştirmez; son işlem yoksa çalışmaz; tek kullanımlıktır. `mapToInt(...).average()` → `OptionalDouble`.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki tanımlardan hangisi **derlenir**?

(A) `List<int> a = new ArrayList<>();`

(B) `List<Integer> b = new List<>();`

(C) `ArrayList<double> c = new ArrayList<double>();`

(D) `Map<String> d = new HashMap<>();`

(E) `List<Character> e = new ArrayList<>();`

2 Aşağıdaki kod parçası ne yazdırır?

```
Integer a = 100, b = 100;
Integer c = 200, d = 200;
System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));
```

(A) `true true true`

(B) `true false true`

(C) `false false true`

(D) `true false false`

(E) `false false false`

3 Aşağıdaki kod parçası ne yazdırır?

```
List<Integer> l = new ArrayList<>(List.of(4, 2, 0, 2));
l.remove(2);
l.remove(Integer.valueOf(2));
System.out.println(l);
```

(A) `[4, 0]`

(B) `[4, 2, 0]`

(C) `[4, 0, 2]`

(D) `[4, 2]`

(E) `IndexOutOfBoundsException` fırlatır.

4 Bir sınavdaki öğrenci numaralarını **tekrarsız** ve **küçükten büyüğe sıralı** tutmak için en uygun koleksiyon hangisidir?

(A) `ArrayList<Integer>`

(B) `HashSet<Integer>`

(C) `TreeSet<Integer>`

(D) `HashMap<Integer, Integer>`

(E) `LinkedList<Integer>`

5 Aşağıdaki kod parçası ne yazdırır?

```
Map<String, Integer> m = new HashMap<>();
m.put("a", 1);
m.put("b", 2);
m.put("a", 3);
System.out.println(m.size() + " " + m.get("a") + " " + m.get("c"));
```

(A) `2 3 null` (B) `3 3 null` (C) `2 1 null` (D) `3 1 0` (E) `2 3 0`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
import java.util.ArrayList;
import java.util.List;

public class Soru6 {
    public static void main(String[] args) {
        List<Integer> l = new ArrayList<>(List.of(2, 4, 4, 5, 6, 8));
        for (int i = 0; i < l.size(); i++) {
            if (l.get(i) % 2 == 0) {
                l.remove(i);
            }
        }
        System.out.println(l + " " + l.size());
        l.add(1, 9);
        System.out.println(l.indexOf(8) + " " + l.contains(4) + " " + l);
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
import java.util.Map;
import java.util.TreeMap;

public class Soru7 {
    public static void main(String[] args) {
        String[] kazananlar = {"Ece", "Can", "Ece", "Bora", "Ece", "Can"};
        Map<String, Integer> puan = new TreeMap<>();
        for (String k : kazananlar) {
            puan.put(k, puan.getOrDefault(k, 0) + 10);
        }
        System.out.println(puan);
        puan.put("Can", puan.get("Can") - 5);
        puan.remove("Bora");
        System.out.println(puan + " " + puan.containsKey("Bora"));
        int toplam = 0;
        for (int p : puan.values()) {
            toplam += p;
        }
        System.out.println(toplam + " " + puan.size());
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Set;
import java.util.TreeSet;

public class Soru8 {
    public static void main(String[] args) {
        List<Integer> a = new ArrayList<>(List.of(3, 1, 2));
        List<Integer> b = a;
        List<Integer> c = new ArrayList<>(a);
        b.add(9);
        Collections.sort(c);
        System.out.println(a + " " + b + " " + c);

        Set<Integer> s = new TreeSet<>(a);
        s.add(1);
        s.add(5);
        System.out.println(s + " " + s.size());

        Integer x = 1000, y = 1000;
        System.out.println((x == y) + " " + x.equals(y) + " " + (x <= y));
    }
}
```

C) Kod Yazma

9 Kullanıcıdan **bir satır** metin okuyan (kelimeler tek boşlukla ayrılmış) `Frekans` programını yaz. Program kelimeleri **küçük harfe** çevirip saysın, kelimeleri **alfabetik sırayla** sayılarıyla birlikte yazdırsın ve en sık geçen kelimeyi göstersin. Örnek:

```
Metin: Java kolay java eğlenceli Java zor değil
değil: 1
eğlenceli: 1
java: 3
kolay: 1
zor: 1
En sık geçen: java (3)
```

10 `ad` (String), `fiyat` (double) ve `stok` (int) alanları olan bir `Urun` sınıfı yaz. `UrunListesi` programında 6 ürünlük bir `ArrayList` oluştur ve sırasıyla:

1. Stoğu 0 olan ürünleri `removeIf` ile sil,
2. Kalanları **fiyata göre artan**, fiyatı eşit olanları **ada göre** sırala ve tablo halinde yazdır,
3. Stoğu en fazla olan ürünü `Collections.max` ve bir `Comparator` ile bul.

Çözümler

Cevap anahtarı

1	2	3	4	5
E	B	D	C	A

1 Cevap: E

Tip parametresi olarak sadece referans tipler yazılabilir; `Character` bir sarmalayıcı sınıf olduğu için E derlenir. A ve C ilkel tip (`int`, `double`) kullandığı için, B `List` bir arayüz olduğu ve `new` ile nesnesi oluşturulamadığı için, D ise `Map` iki tip parametresi (`<K, V>`) istediği için derlenmez.

2 Cevap: B

Autoboxing `Integer.valueOf` ile yapılır ve −128…127 arasındaki değerler için **önbellekteki aynı nesne** kullanılır: `a == b` → `true`. 200 bu aralığın dışında olduğu için `c` ve `d` iki farklı nesnedir: `c == d` → `false`. `equals` değeri karşılaştırır: `true`.

Çıktı

```
true false true
```

3 Cevap: D

`l.remove(2)`: Argüman `int` olduğu için `remove(int indeks)` çağrılır ve **2. indeksteki** `0` silinir → `[4, 2, 2]`. `l.remove(Integer.valueOf(2))`: Değeri 2 olan **ilk** eleman silinir → `[4, 2]`. A şıkkı, ilk `remove(2)`'nin değer sildiğini sananların cevabıdır.

Çıktı

```
[4, 2]
```

4 Cevap: C

`TreeSet` hem tekrarları engeller (küme) hem de elemanları sıralı tutar. `HashSet` tekrarsızdır ama sırası belirsizdir. `ArrayList` ve `LinkedList` tekrara izin verir ve kendiliğinden sıralamaz. `HashMap` anahtar–değer içindir ve sırası belirsizdir.

5 Cevap: A

İkinci `m.put("a", 3)` yeni bir çift eklemez, `"a"`'nın değerini 3 yapar; bu yüzden boyut 2 kalır. Olmayan `"c"` anahtarı için `get` exception da `0` da değil, **`null`** döndürür.

Çıktı

```
2 3 null
```

6

Silme işlemi baştan sona indeksle yapıldığı için elemanlar atlanır:

i	Liste (kontrol öncesi)	l.get(i)	İşlem
0	`[2, 4, 4, 5, 6, 8]`	2	çift → sil
1	`[4, 4, 5, 6, 8]`	4	çift → sil (0. indeksteki 4 atlandı!)
2	`[4, 5, 6, 8]`	6	çift → sil
3	`[4, 5, 8]`	—	`3 < 3` yanlış, döngü biter

Sonuç `[4, 5, 8]`, boyut 3. `add(1, 9)` → `[4, 9, 5, 8]`; `8`'in indeksi 3, `4` hâlâ listede.

Çıktı
```
[4, 5, 8] 3
3 true [4, 9, 5, 8]
```

7

- Döngü: Ece 3 kez (30), Can 2 kez (20), Bora 1 kez (10). `TreeMap` anahtarları alfabetik yazar.
- `put("Can", 20 - 5)` → Can = 15 (değer değişir, yeni çift eklenmez). `remove("Bora")` → Bora silinir.
- Toplam: 15 + 30 = 45, boyut 2.

Çıktı
```
{Bora=10, Can=20, Ece=30}
{Can=15, Ece=30} false
45 2
```

8

- `b = a` kopya değil, **aynı listeye ikinci referanstır** (aliasing). `b.add(9)` → `a` da `[3, 1, 2, 9]` olur. `c` ise `new ArrayList<>(a)` ile 9 eklenmeden **önce** alınmış bağımsız bir kopyadır; sıralanınca `[1, 2, 3]`.
- `TreeSet` `a`'nın elemanlarını sıralı alır: `[1, 2, 3, 9]`. 1 zaten var, eklenmez; 5 eklenir → 5 eleman.
- 1000 önbellek aralığında değil: `x == y` → `false`; `equals` → `true`; `<=` unboxing yapıp değerleri karşılaştırır → `true`.

Çıktı
```
[3, 1, 2, 9] [3, 1, 2, 9] [1, 2, 3]
[1, 2, 3, 5, 9] 5
false true true
```

9

Kelimeleri bir `TreeMap<String, Integer>` içinde sayarsak anahtarlar kendiliğinden alfabetik sıralanır. Sayma için `getOrDefault`, en sık kelime için `entrySet` üzerinde en büyüğü bulma döngüsü kullanılır. Küçük harfe çevirirken Türkçe `Locale` veriyoruz (`I` → `ı`, Bölüm 8); boş satırı bölmüyor ve `\\s+` ile böldüğümüz için fazladan boşluklar boş kelime olarak sayılmıyor.

Frekans.java
```
import java.util.Locale;
import java.util.Map;
import java.util.Scanner;
import java.util.TreeMap;

public class Frekans {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Metin: ");
        Locale tr = Locale.forLanguageTag("tr-TR");
        String metin = sc.nextLine().trim().toLowerCase(tr);

        Map<String, Integer> sayac = new TreeMap<>();
        if (!metin.isEmpty()) {            // boş metni bölmek {""} verir
            for (String k : metin.split("\\s+")) {
                sayac.put(k, sayac.getOrDefault(k, 0) + 1);
            }
        }

        String enSik = "";
        int enCok = 0;
        for (Map.Entry<String, Integer> e : sayac.entrySet()) {
            System.out.println(e.getKey() + ": " + e.getValue());
            if (e.getValue() > enCok) {
                enCok = e.getValue();
                enSik = e.getKey();
            }
        }
        System.out.println("En sık geçen: " + enSik + " (" + enCok + ")");
    }
}
```

Çıktı

```
Metin: Java kolay java eğlenceli Java zor değil
değil: 1
eğlenceli: 1
java: 3
kolay: 1
zor: 1
En sık geçen: java (3)
```

10

Silme için `removeIf` ve bir koşul lambda'sı; iki ölçütlü sıralama için `Comparator.comparing(...).thenComparing(...)`; en büyük stok için `Collections.max(liste, Comparator.comparing(Urun::getStok))` yeterlidir.

UrunListesi.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

class Urun {
    private String ad;
    private double fiyat;
    private int stok;

    public Urun(String ad, double fiyat, int stok) {
        this.ad = ad;
        this.fiyat = fiyat;
        this.stok = stok;
    }

    public String getAd() { return ad; }
    public double getFiyat() { return fiyat; }
    public int getStok() { return stok; }
}
```

```java
public class UrunListesi {
    public static void main(String[] args) {
        List<Urun> urunler = new ArrayList<>();
        urunler.add(new Urun("Defter", 45.0, 12));
        urunler.add(new Urun("Kalem", 12.5, 0));
        urunler.add(new Urun("Silgi", 7.25, 125));
        urunler.add(new Urun("Cetvel", 45.0, 8));
        urunler.add(new Urun("Boya", 89.9, 0));
        urunler.add(new Urun("Makas", 30.0, 15));

        urunler.removeIf(u -> u.getStok() == 0);
        urunler.sort(Comparator.comparing(Urun::getFiyat)
                .thenComparing(Urun::getAd));

        System.out.printf("%-8s%8s%6s%n", "Ürün", "Fiyat", "Stok");
        for (Urun u : urunler) {
            System.out.printf("%-8s%8.2f%6d%n",
                    u.getAd(), u.getFiyat(), u.getStok());
        }

        Urun enCok = Collections.max(urunler, Comparator.comparing(Urun::getStok));
        System.out.println("En çok stok: " + enCok.getAd());
    }
}
```

Çıktı

```
Ürün       Fiyat  Stok
Silgi       7.25   125
Makas      30.00    15
Cetvel     45.00     8
Defter     45.00    12
En çok stok: Silgi
```

`Cetvel` ve `Defter`'in fiyatı eşit (45.00) olduğu için `thenComparing` devreye girdi ve ada göre sıralandılar.

BÖLÜM 14

Dosya İşlemleri

Bu Bölümde Öğreneceklerin

- Verinin program bittikten sonra da kalması için dosya kullanmayı
- `Path` ile dosya yollarını, göreli/mutlak yol farkını ve **çalışma klasörünü**
- `Files` sınıfıyla tek satırda okuma/yazma: `readString`, `readAllLines`, `writeString`, `write`, dosya sonuna ekleme
- `IOException`'ın neden zorunlu olarak ele alındığını (checked exception)
- `BufferedReader`/`BufferedWriter`, `Scanner` ve `PrintWriter` ile satır satır çalışmayı, try-with-resources ile dosyayı güvenle kapatmayı
- UTF-8 karakter kodlamasını ve Türkçe karakter sorunlarını
- CSV dosyası ayrıştırmayı, bozuk satırları atlamayı ve sonucu rapor dosyasına yazmayı

14.1 Neden dosya?

Şimdiye kadar yazdığın programlarda bütün veriler (değişkenler, diziler, `ArrayList`'ler) **RAM**'de duruyordu. Program bittiği anda hepsi silinir; bir sonraki çalıştırmada her şey sıfırdan başlar. Verinin **kalıcı** (persistent) olması için onu diske, bir **dosyaya** yazmak gerekir: not listesi, oyun skorları, kütüphane kayıtları...

Bu bölümde **metin dosyaları** (text file: `.txt`, `.csv`) ile çalışacağız; içleri Not Defteri veya VS Code ile açılıp okunabilir. Java'da dosya işlemleri için iki paket kullanılır:

Paket	Önemli sınıflar	Özelliği
`java.nio.file`	`Path`, `Files`, `StandardOpenOption`	Modern yol (Java 7+): tek satırda okuma/yazma
`java.io`	`BufferedReader`, `BufferedWriter`, `FileReader`, `FileWriter`, `PrintWriter`, `IOException`	Klasik yol: satır satır, akış (stream) tabanlı

İlk programımız bir dosyaya iki satır yazıp geri okuyor:

IlkDosya.java

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class IlkDosya {
    public static void main(String[] args) throws IOException {
        Path dosya = Path.of("selam.txt");
        Files.writeString(dosya, "Merhaba dosya!\nBu satır diskte duruyor.\n");
        System.out.println(dosya + " yazıldı.");

        String icerik = Files.readString(dosya);
        System.out.print(icerik);
    }
}
```

```
Çıktı
selam.txt yazıldı.
Merhaba dosya!
Bu satır diskte duruyor.
```

Program bittikten sonra klasörüne bakarsan `selam.txt` dosyasını görürsün. `main`'in sonundaki `throws IOException` ifadesinin neden gerektiğini 14.4'te açıklayacağız; şimdilik kalıp olarak yaz.

14.2 Dosya yolları ve çalışma klasörü

Bir dosyanın diskteki adresine **yol** (path) denir ve Java'da `Path` nesnesiyle temsil edilir. `Path.of(...)` ile oluşturulur; parçaları ayrı ayrı verirsen işletim sistemine uygun ayırıcıyla (`/` veya `\`) birleştirir:

YolOrnek.java

```java
import java.nio.file.Path;

public class YolOrnek {
    public static void main(String[] args) {
        Path p = Path.of("veriler", "2026", "notlar.csv");
        System.out.println("Yol       : " + p);
        System.out.println("Dosya adı : " + p.getFileName());
        System.out.println("Üst klasör: " + p.getParent());
        System.out.println("Mutlak mı?: " + p.isAbsolute());
    }
}
```

```
Çıktı
Yol       : veriler/2026/notlar.csv
Dosya adı : notlar.csv
Üst klasör: veriler/2026
Mutlak mı?: false
```

(Bu çıktı macOS'ta üretildi; Windows'ta ayırıcı `\` olur: `veriler\2026\notlar.csv`.)

İki tür yol vardır:

Tür	Örnek	Nereden başlar?
Mutlak (absolute)	`C:\Users\elif\Ders\notlar.csv`, `/home/elif/Ders/notlar.csv`	Diskin kökünden; her yerden aynı dosyayı gösterir
Göreli (relative)	`notlar.csv`, `veriler/notlar.csv`	Programın **çalışma klasöründen** (working directory)

Göreli bir yol, programın o anki **çalışma klasörüne** göre çözülür. VS Code'da **Run** düğmesine bastığında çalışma klasörü, **File → Open Folder** ile açtığın klasördür. Terminalden çalıştırırken ise hangi klasördeysen odur. Göreli yolun tam olarak nereyi gösterdiğini merak ediyorsan `toAbsolutePath()` ile yazdır:

CalismaKlasoru.java

```java
import java.nio.file.Path;

public class CalismaKlasoru {
    public static void main(String[] args) {
        System.out.println(Path.of("notlar.csv").toAbsolutePath());
    }
}
```

`C:\Users\elif\JavaDersleri` klasörünü VS Code'da açtıysan bu program `C:\Users\elif\JavaDersleri\notlar.csv` yazdırır. (Çıktı her bilgisayarda farklı olduğu için buraya koymadık.)

> **İpucu: "Dosya bulunamadı" diyorsa önce çalışma klasörüne bak**
>
> Veri dosyasını `.java` dosyasının yanına koydun ama program bulamıyorsa, büyük ihtimalle çalışma klasörü farklıdır (örneğin VS Code'da üst klasörü açmışsındır). Yukarıdaki `toAbsolutePath()` satırını çalıştır, Java'nın dosyayı nerede aradığını gör ve dosyayı oraya taşı.

> **Sık Yapılan Hata: Windows yolunda ters bölü**
>
> String içinde `\` bir kaçış karakteridir (Bölüm 1). `"C:\ders\notlar.txt"` yazarsan Java `\d`'yi tanımaz ve program derlenmez:

```
import java.nio.file.Path;

public class TersBolu {
    public static void main(String[] args) {
        Path p = Path.of("C:\ders\notlar.txt");
    }
}
```

Derleyici çıktısı – program derlenmez

```
TersBolu.java:5: error: illegal escape character
        Path p = Path.of("C:\ders\notlar.txt");
                           ^
1 error
```

> Ya ters bölüleri ikile (`"C:\\ders\\notlar.txt"`) ya da düz bölü kullan (`"C:/ders/notlar.txt"`); Java, Windows'ta da `/` kabul eder.

14.3 Files ile okuma ve yazma

`java.nio.file.Files` sınıfının statik metotları, küçük ve orta boy dosyalar için en kolay yoldur:

Metot	Ne yapar?
`Files.writeString(p, metin)`	Metni dosyaya yazar. Dosya yoksa **oluşturur**, varsa içeriğini **silip üzerine yazar**
`Files.write(p, liste)`	Listedeki her elemanı **ayrı bir satır** olarak yazar
`Files.readString(p)`	Bütün dosyayı **tek bir String** olarak okur
`Files.readAllLines(p)`	Dosyayı `List<String>` olarak okur; her satır bir eleman (satır sonu karakteri olmadan)
`Files.exists(p)`	Dosya var mı?
`Files.size(p)`	Dosyanın boyutu (**bayt** cinsinden)
`Files.delete(p)`	Dosyayı siler; dosya yoksa exception fırlatır
`Files.deleteIfExists(p)`	Varsa siler, sonucu `boolean` döndürür

Yazma metotlarına `StandardOpenOption` seçenekleri verilerek davranışları değiştirilebilir. En önemlisi, dosyanın **sonuna ekleyen** `APPEND`'dir:

FilesOrnek.java

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.List;

public class FilesOrnek {
    public static void main(String[] args) throws IOException {
        Path p = Path.of("alisveris.txt");
        System.out.println("Var mı? " + Files.exists(p));

        Files.write(p, List.of("ekmek", "süt", "yumurta"));   // her biri bir satır
        System.out.println("Var mı? " + Files.exists(p) + ", boyut: "
                + Files.size(p) + " bayt");

        // Sona ekleme: APPEND (dosya yoksa oluşturması için CREATE de verilir)
        Files.writeString(p, "peynir\n", StandardOpenOption.CREATE,
                StandardOpenOption.APPEND);

        List<String> satirlar = Files.readAllLines(p);
        System.out.println(satirlar.size() + " satır: " + satirlar);
        for (int i = 0; i < satirlar.size(); i++) {
            System.out.println((i + 1) + ". " + satirlar.get(i));
        }

        Files.writeString(p, "sadece bu kaldı\n");             // ÜZERİNE yazar
        System.out.print(Files.readString(p));

        Files.delete(p);
        System.out.println("Silindi, var mı? " + Files.exists(p));
    }
}
```

Çıktı

```
Var mı? false
Var mı? true, boyut: 19 bayt
4 satır: [ekmek, süt, yumurta, peynir]
1. ekmek
2. süt
3. yumurta
4. peynir
sadece bu kaldı
Silindi, var mı? false
```

`ekmek`, `süt` ve `yumurta` 15 harf, 3 satır sonuyla birlikte toplam 18 karakter; ama dosya 19 bayt. Aradaki fark `ü` harfinden geliyor: Türkçe karakterler dosyada **2 bayt** yer kaplar (14.8).

> ● **Sık Yapılan Hata: Seçeneksiz yazma, eski içeriği siler**
>
> `Files.writeString(p, metin)` ve `Files.write(p, liste)` seçenek verilmezse dosyanın **eski içeriğini sorgusuz sualsiz siler**. Bir günlüğe (log) kayıt eklerken bunu unutursan her çalıştırmada yalnızca son kayıt kalır. Eklemek için `StandardOpenOption.APPEND` kullan.

> **Sınavda Çıkar: APPEND tek başına yetmez**
>
> `APPEND` sadece **var olan** bir dosyanın sonuna ekler; dosya yoksa `NoSuchFileException` fırlatır. "Dosya yoksa oluştur, varsa sonuna ekle" için iki seçenek birlikte verilir: `StandardOpenOption.CREATE, StandardOpenOption.APPEND`. Seçenek verilmediğinde varsayılan davranış ise `CREATE` + `TRUNCATE_EXISTING` (içeriği sil) + `WRITE`'tır.

14.4 IOException: kontrol edilen istisna

Dosya işlemleri, programın kontrolü dışındaki nedenlerle başarısız olabilir: Dosya yoktur, disk doludur, yazma izni yoktur, USB bellek çıkarılmıştır. Bu yüzden dosya metotlarının çoğu `IOException` fırlatabileceğini bildirir. Bölüm 12'den hatırla: `IOException` **kontrol edilen** (checked) bir istisnadır; derleyici onu ele almanı **zorunlu** tutar. Ele almazsan program derlenmez:

KontrolsuzOkuma.java

```
import java.nio.file.Files;
import java.nio.file.Path;

public class KontrolsuzOkuma {
    public static void main(String[] args) {
        String s = Files.readString(Path.of("notlar.csv"));
        System.out.println(s);
    }
}
```

Derleyici çıktısı – program derlenmez

```
KontrolsuzOkuma.java:6: error: unreported exception IOException; must be caught or declared to be
thrown
        String s = Files.readString(Path.of("notlar.csv"));
                                            ^
1 error
```

İki seçeneğin var:

Yol	Yazım	Hata olursa ne olur?
Bildirmek	`public static void main(String[] args) throws IOException`	Exception yukarı iletilir; `main`'den çıkarsa program çöker
Yakalamak	`try { ... } catch (IOException e) { ... }`	Sen karar verirsin: mesaj yaz, varsayılan değer kullan, devam et

Kısa örneklerde `throws IOException` pratiktir; ama kullanıcıya dönük programlarda hatayı **yakalayıp anlaşılır bir mesaj** vermelisin. Olmayan bir dosyayı okumaya çalışalım:

GuvenliOkuma.java

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.NoSuchFileException;
import java.nio.file.Path;

public class GuvenliOkuma {
    static void oku(String ad) {
        try {
            String icerik = Files.readString(Path.of(ad));
            System.out.println(ad + " okundu: " + icerik.length() + " karakter");
        } catch (NoSuchFileException e) {
```

```
            System.out.println("Dosya bulunamadı: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("Okuma hatası: " + e.getMessage());
        }
    }

    public static void main(String[] args) throws IOException {
        Files.writeString(Path.of("var.txt"), "Java");
        oku("var.txt");
        oku("yok.txt");
        System.out.println("Program çökmedi, devam ediyor.");
    }
}
```

Çıktı

```
var.txt okundu: 4 karakter
Dosya bulunamadı: yok.txt
Program çökmedi, devam ediyor.
```

`try-catch` olmasaydı, `yok.txt` satırında program `NoSuchFileException` ile çökecek ve ekrana uzun bir hata yığını (stack trace) basacaktı.

> **Sınavda Çıkar: İstisna sınıflarının sırası**
>
> `NoSuchFileException`, `IOException`'ın **alt sınıfıdır**. Bölüm 12'deki kural burada da geçerli: Birden fazla `catch` varsa **önce alt sınıf**, sonra üst sınıf yazılır. Ters sırada yazarsan alt sınıfın `catch`'ine hiç ulaşılamaz ve program **derlenmez**. Klasik `java.io` sınıfları (`FileReader`, `PrintWriter`) olmayan dosya için `NoSuchFileException` yerine `FileNotFoundException` fırlatır; o da `IOException`'ın alt sınıfıdır.

14.5 BufferedReader ve BufferedWriter

`readAllLines` bütün dosyayı tek seferde belleğe alır. Dosya çok büyükse (milyonlarca satırlık bir kayıt dosyası) ya da her satırı okur okumaz işlemek istiyorsan **satır satır** okumak gerekir. Bunun için `BufferedReader`, yazmak için `BufferedWriter` kullanılır. İkisi de verileri bir **tampon** (buffer) bellekte biriktirip diske toplu halde ulaştırdığı için hızlıdır.

Bu sınıflar bir **kaynak** (resource) açar ve işin sonunda kapatılmaları (`close()`) gerekir. Bölüm 12'de gördüğün **try-with-resources** yapısı kapatmayı otomatik yapar; hata olsa bile:

TamponluYazOku.java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class TamponluYazOku {
    public static void main(String[] args) {
        Path p = Path.of("program.txt");

        try (BufferedWriter yazici = Files.newBufferedWriter(p)) {
            yazici.write("Pazartesi: Java dersi");
            yazici.newLine();                          // satır sonu ekle
            yazici.write("Salı: Fizik laboratuvarı");
            yazici.newLine();
            yazici.write("Çarşamba: Matematik quiz");
            yazici.newLine();
        } catch (IOException e) {
```

```
            System.out.println("Yazma hatası: " + e.getMessage());
        }

        try (BufferedReader okuyucu = Files.newBufferedReader(p)) {
            String satir;
            int no = 0;
            while ((satir = okuyucu.readLine()) != null) {   // null: dosya bitti
                no++;
                System.out.println(no + ") " + satir);
            }
        } catch (IOException e) {
            System.out.println("Okuma hatası: " + e.getMessage());
        }
    }
}
```

Çıktı

```
1) Pazartesi: Java dersi
2) Salı: Fizik laboratuvarı
3) Çarşamba: Matematik quiz
```

`while ((satir = okuyucu.readLine()) != null)` satırı dosya okumanın **klasik kalıbıdır**, ezberle. İçten dışa okunur: `readLine()` bir satır okur, sonuç `satir`'a atanır, atamanın değeri de `null` ile karşılaştırılır. İçteki parantezler şarttır; yoksa önce `readLine() != null` hesaplanır ve `boolean` bir değer `String`'e atanmaya çalışılır.

> **Sınavda Çıkar: readLine ve write**
>
> - `readLine()` dosya sonunda `null` döndürür; boş bir satırda ise **boş String** (`""`) döndürür. Döngü koşulu bu yüzden `!= null`'dır, `isEmpty()` değil.
> - `readLine()` satır sonu karakterini (`\n`) **döndürmez**.
> - `write("...")` satır sonu **eklemez**; `newLine()` çağırmalı veya metnin sonuna `\n` koymalısın.

FileReader ve FileWriter

`Files.newBufferedReader(p)` yerine, eski kodlarda ve ders kitaplarında sık göreceğin klasik yazım şudur: Bir `FileReader`'ı (`FileWriter`'ı) bir `BufferedReader`'ın (`BufferedWriter`'ın) içine sarmak. `FileWriter`'ın ikinci parametresi `true` verilirse dosyanın **sonuna ekler**:

KlasikYol.java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class KlasikYol {
    public static void main(String[] args) throws IOException {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("kayit.txt"))) {
            bw.write("ilk kayıt");
            bw.newLine();
        }
        // FileWriter(dosyaAdi, true): ekleme (append) modu
        try (BufferedWriter bw =
                 new BufferedWriter(new FileWriter("kayit.txt", true))) {
            bw.write("eklenen kayıt");
            bw.newLine();
        }
        try (BufferedReader br = new BufferedReader(new FileReader("kayit.txt"))) {
```

```
            String satir;
            while ((satir = br.readLine()) != null) {
                System.out.println(satir);
            }
        }
    }
}
```

Çıktı

```
ilk kayıt
eklenen kayıt
```

`FileReader` dosyadan karakter okur; `BufferedReader` onu sarar ve üstüne tampon ile `readLine()` yeteneği ekler. Böyle iç içe sarmalama, Java'nın giriş/çıkış sınıflarında çok yaygındır. İki yazım da doğrudur; `Files.newBufferedReader/Writer` daha kısa ve karakter kodlaması konusunda daha güvenlidir (14.8).

14.6 Scanner ile dosya okuma

Bölüm 3'ten tanıdığın `Scanner`, klavye yerine bir dosyadan da okuyabilir: `System.in` yerine bir `Path` verirsin. Metotlar aynıdır: `next()`, `nextInt()`, `nextDouble()`, `nextLine()`, `hasNext()`, `hasNextLine()`. Dosyada **sayı** ve **kelime** karışık olduğunda Scanner, `BufferedReader`'dan daha pratiktir. Okuyacağımız dosya:

sicakliklar.txt

```
Pazartesi 18.5
Salı 21.0
Çarşamba 19.5
Perşembe 23.0
Cuma 22.5
```

ScannerDosya.java

```
import java.io.IOException;
import java.nio.file.Path;
import java.util.Locale;
import java.util.NoSuchElementException;
import java.util.Scanner;

public class ScannerDosya {
    public static void main(String[] args) {
        try (Scanner sc = new Scanner(Path.of("sicakliklar.txt"))) {
            sc.useLocale(Locale.US);                  // ondalık ayırıcı: nokta
            double toplam = 0;
            int gun = 0;
            while (sc.hasNext()) {                    // okunacak kelime kaldı mı?
                String ad = sc.next();
                double derece = sc.nextDouble();
                System.out.printf("%-10s %5.1f%n", ad, derece);
                toplam += derece;
                gun++;
            }
            if (gun > 0) {
                System.out.printf("Ortalama: %.2f%n", toplam / gun);
            } else {
                System.out.println("Dosyada veri yok.");    // 0.0 / 0 = NaN olurdu
            }
        } catch (IOException e) {
            System.out.println("Dosya açılamadı: " + e.getMessage());
        } catch (NoSuchElementException e) {  // InputMismatch... da dahil
```

```
            System.out.println("Dosyada bozuk satır var: ad ve sayı bekleniyordu.");
        }
    }
}
```

Çıktı

```
Pazartesi   18.5
Salı        21.0
Çarşamba    19.5
Perşembe    23.0
Cuma        22.5
Ortalama: 20.90
```

`Scanner(Path)` yapıcısı `IOException` fırlatabildiği için try-with-resources içine alındı; Scanner da kapatılması gereken bir kaynaktır. Dosya bozuksa (sayı yerine metin varsa `nextDouble()` → `InputMismatchException`; bir satırda sayı eksikse → `NoSuchElementException`) program çökmesin diye ikinci `catch` ikisini birden yakalar, çünkü `InputMismatchException`, `NoSuchElementException`'ın alt sınıfıdır. Dosya boşsa `gun` 0 kalır ve `0.0 / 0` `NaN` verirdi; `if (gun > 0)` bunu önler. `useLocale(Locale.US)` satırı Bölüm 3'teki ondalık virgül sorununu önler: Türkçe ayarlı bir bilgisayarda `nextDouble()` dosyadaki `18.5`'i okuyamazdı.

● Sık Yapılan Hata: Scanner'a dosya adını String olarak vermek

`new Scanner("sicakliklar.txt")` dosyayı **açmaz**! `Scanner`'ın String alan yapıcısı, verilen **metnin kendisini** tarar:

```
import java.util.Scanner;

public class ScannerTuzak {
    public static void main(String[] args) {
        Scanner sc = new Scanner("sicakliklar.txt");     // dosya değil, METİN!
        System.out.println(sc.nextLine());
    }
}
```

Çıktı

```
sicakliklar.txt
```

Doğrusu `new Scanner(Path.of("sicakliklar.txt"))` veya `new Scanner(new File("sicakliklar.txt"))`'tir.

14.7 PrintWriter ile biçimli yazma

Dosyaya `printf` ile tablo yazmak istiyorsan `PrintWriter` kullan. `System.out` ile aynı metotlara sahiptir: `print`, `println`, `printf`. Fark, çıktının ekrana değil dosyaya gitmesidir:

FaturaYaz.java

```
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;

public class FaturaYaz {
    public static void main(String[] args) throws IOException {
        String[] urunler = {"Kalem", "Defter", "Silgi"};
        int[] adetler = {3, 12, 125};
```

```
        double[] fiyatlar = {12.5, 45.0, 7.25};
        Path p = Path.of("fatura.txt");

        try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(p))) {
            pw.printf("%-8s%6s%10s%n", "Ürün", "Adet", "Tutar");
            double genel = 0;
            for (int i = 0; i < urunler.length; i++) {
                double tutar = adetler[i] * fiyatlar[i];
                pw.printf("%-8s%6d%10.2f%n", urunler[i], adetler[i], tutar);
                genel += tutar;
            }
            pw.println("------------------------");
            pw.printf("%-14s%10.2f%n", "TOPLAM", genel);
        }

        // Doğrulama: dosyayı geri oku ve ekrana yaz
        System.out.print(Files.readString(p));
    }
}
```

Çıktı

```
Ürün      Adet     Tutar
Kalem        3     37.50
Defter      12    540.00
Silgi      125    906.25
------------------------
TOPLAM            1483.75
```

Yazdığın dosyayı programın sonunda **geri okuyup ekrana basmak**, yazmanın gerçekten başarılı olduğunu doğrulamanın en kolay yoludur; ödevlerde de bunu yapmaya alış. `PrintWriter`'ın `printf`'i de bölge ayarını kullanır (Bölüm 3); her bilgisayarda nokta görmek için `pw.printf(Locale.US, ...)` yazabilirsin. Dosya adıyla doğrudan `new PrintWriter("fatura.txt")` da yazılabilir; ama bu yapıcı dosyayı her seferinde baştan yazar, ekleme modu için `new PrintWriter(new FileWriter("fatura.txt", true))` gerekir.

Sık Yapılan Hata: close() unutulursa dosya boş kalabilir

`PrintWriter` ve `BufferedWriter` yazdıklarını önce tamponda biriktirir; tampon **ancak** `flush()` veya `close()` çağrılınca diske boşaltılır. Kapatmayı unutursan veri diske hiç ulaşmayabilir:

```
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;

public class KapatmaUnutuldu {
    public static void main(String[] args) throws IOException {
        PrintWriter pw = new PrintWriter("sonuc.txt");
        pw.println("Önemli sonuç: 42");
        System.out.println("close öncesi : " + Files.size(Path.of("sonuc.txt")));
        pw.close();
        System.out.println("close sonrası: " + Files.size(Path.of("sonuc.txt")));
    }
}
```

Çıktı

```
close öncesi : 0
close sonrası: 19
```

`close()` çağrılmadan önce dosya 0 bayttı. try-with-resources kullanırsan bu hatayı yapman imkânsızdır.

14.8 Karakter kodlaması: UTF-8 ve Türkçe karakterler

Dosyada aslında karakterler değil **baytlar** saklanır. Hangi karakterin hangi baytlarla yazılacağını belirleyen kurala **karakter kodlaması** (character encoding) denir. Bugün standart olan **UTF-8**'de İngilizce harfler 1 bayt, `ç ğ ı ö ş ü İ` gibi Türkçe harfler **2 bayt** tutar. Dosya bir kodlamayla yazılıp başka bir kodlamayla okunursa Türkçe karakterler bozulur:

Kodlama.java

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class Kodlama {
    public static void main(String[] args) throws IOException {
        Path p = Path.of("turkce.txt");
        String metin = "Ilık süt ve çörek";
        Files.writeString(p, metin, StandardCharsets.UTF_8);

        System.out.println("Karakter sayısı: " + metin.length());
        System.out.println("Dosya boyutu   : " + Files.size(p) + " bayt");

        String dogru = Files.readString(p, StandardCharsets.UTF_8);
        String yanlis = Files.readString(p, StandardCharsets.ISO_8859_1);
        System.out.println("UTF-8 ile oku  : " + dogru);
        System.out.println("Latin-1 ile oku: " + yanlis);
    }
}
```

Çıktı

```
Karakter sayısı: 17
Dosya boyutu   : 21 bayt
UTF-8 ile oku  : Ilık süt ve çörek
Latin-1 ile oku: IlÄ±k sÃ¼t ve Ã§Ã¶rek
```

Metinde 4 Türkçe harf (`ı`, `ü`, `ç`, `ö`) var; her biri bir bayt fazla yer kapladığı için 17 karakterlik metin 21 bayt tuttu. Yanlış kodlamayla okununca her Türkçe harf iki anlamsız karaktere dönüştü. Ekranda `Ã§` gibi garip işaretler görürsen sebep budur.

Java 18'den beri **varsayılan kodlama her platformda UTF-8**'dir; `Files` sınıfının metotları ise zaten her zaman UTF-8 kullanır. Sorun genellikle başka bir programla (ör. eski bir Windows Not Defteri veya Excel) **Windows-1254** kodlamasıyla kaydedilmiş bir dosyayı okurken çıkar. Böyle bir dosya için kodlamayı açıkça verirsin: `Files.readAllLines(p, Charset.forName("windows-1254"))`.

İpucu: VS Code'da kodlamayı görmek

VS Code, açık dosyanın kodlamasını sağ alttaki durum çubuğunda gösterir (`UTF-8`). Türkçe karakterleri bozuk bir dosyada oraya tıklayıp **Reopen with Encoding** ile doğru kodlamayı seçebilir, **Save with Encoding → UTF-8** ile dosyayı düzeltebilirsin.

> **Not: Satır sonu karakteri**
>
> Satır sonu Linux ve macOS'ta tek karakterdir (`\n`), Windows'ta iki karakterdir (`\r\n`). `newLine()`, `println` ve `Files.write` işletim sisteminin satır sonunu kullanır; bu yüzden aynı dosya Windows'ta birkaç bayt büyük çıkabilir. `readLine()` ve `readAllLines` ikisini de tanır.

14.9 CSV dosyası işleme

CSV (Comma-Separated Values, virgülle ayrılmış değerler), tablo halindeki verinin en yaygın metin biçimidir. Her satır bir kayıt, virgüller arasındaki her parça bir alandır; ilk satır genellikle **başlık** satırıdır. Excel ve Google Sheets CSV açıp kaydedebilir. Gerçek dünyadan gelen veri her zaman temiz değildir; aşağıdaki dosyada boşluklu bir alan, sayı yerine yazı, eksik alan ve boş bir satır var:

notlar.csv

```
ogrenci_no,ad_soyad,vize,final
2026101,Ayşe Kaya,72,85
2026102,Burak Demir,45,38
2026103,Cem Yıldız,  90 , 95
2026104,Deniz Ak,sekiz,70
2026105,Elif Şahin,60

2026106,Fatih Çelik,55,62
2026107,Gizem Öztürk,100,88
```

Bir satırı parçalara ayırmak

Satırı alanlara bölmek için Bölüm 8'deki `split(",")`, alanların başındaki/sonundaki boşlukları atmak için `trim()`, sayıya çevirmek için `Integer.parseInt` kullanılır:

SatirAyristir.java

```
public class SatirAyristir {
    public static void main(String[] args) {
        String satir = "2026103,Cem Yıldız,  90 , 95";
        String[] p = satir.split(",");
        System.out.println(p.length + " parça");
        for (String parca : p) {
            System.out.println("[" + parca + "]");
        }

        int vize = Integer.parseInt(p[2].trim());     // "  90 " -> "90" -> 90
        int fin = Integer.parseInt(p[3].trim());
        System.out.printf("%s: %.2f%n", p[1], vize * 0.4 + fin * 0.6);

        try {
            Integer.parseInt(p[2]);                   // trim unutuldu!
        } catch (NumberFormatException e) {
            System.out.println("trim'siz: " + e.getMessage());
        }
    }
}
```

```
Çıktı
4 parça
[2026103]
[Cem Yıldız]
[  90 ]
[ 95]
Cem Yıldız: 93.00
trim'siz: For input string: "  90 "
```

- **Sınavda Çıkar: split ve parseInt tuzakları**
  - `Integer.parseInt(" 90")` boşluğu kabul etmez: **`NumberFormatException`**. Önce `trim()`.
  - `split(",")` **sondaki boş alanları atar**: `"a,b,,".split(",")` → 2 parça (`"a"`, `"b"`). Ortadaki boş alanlar kalır: `"a,,b".split(",")` → 3 parça. Sondaki boş alanları da korumak için `split(",", -1)` yazılır.
  - Eksik alanlı bir satırda `p[3]`'e erişmek `ArrayIndexOutOfBoundsException` verir; erişmeden önce `p.length`'i kontrol et.

Bozuk satırları atlayıp rapor yazmak

Şimdi bütün dosyayı işleyelim. Program her öğrencinin ortalamasını (vize %40, final %60) hesaplayıp `rapor.txt`'ye yazacak; boş satırları sessizce, bozuk satırları ise bir uyarı vererek **atlayacak**. Tek bir bozuk satır yüzünden bütün programın çökmesi kabul edilemez. En sonda raporu geri okuyup ekrana basarak doğrulayacak:

NotRaporu.java

```
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class NotRaporu {
    public static void main(String[] args) {
        List<String> satirlar;
        try {
            satirlar = Files.readAllLines(Path.of("notlar.csv"));
        } catch (IOException e) {
            System.out.println("notlar.csv okunamadı: " + e.getMessage());
            return;
        }

        int gecen = 0, kalan = 0, atlanan = 0;
        double toplam = 0;
        Path rapor = Path.of("rapor.txt");
        try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(rapor))) {
            pw.printf("%-8s %-14s %6s  %s%n", "No", "Ad Soyad", "Ort.", "Durum");
            for (int i = 1; i < satirlar.size(); i++) {     // 0. satır başlık
                String satir = satirlar.get(i).trim();
                if (satir.isEmpty()) {
                    continue;                                // boş satırı atla
                }
                String[] p = satir.split(",");
                if (p.length != 4) {
                    System.out.println("Satır " + (i + 1) + " atlandı: eksik alan");
                    atlanan++;
                    continue;
                }
                try {
                    int vize = Integer.parseInt(p[2].trim());
```

```
                    int fin = Integer.parseInt(p[3].trim());
                    double ort = vize * 0.4 + fin * 0.6;
                    if (ort >= 50) {
                        gecen++;
                    } else {
                        kalan++;
                    }
                    toplam += ort;
                    pw.printf("%-8s %-14s %6.2f  %s%n", p[0].trim(), p[1].trim(),
                            ort, ort >= 50 ? "GEÇTİ" : "KALDI");
                } catch (NumberFormatException e) {
                    System.out.println("Satır " + (i + 1) + " atlandı: "
                            + e.getMessage());
                    atlanan++;
                }
            }
            pw.println("-".repeat(38));
            if (gecen + kalan == 0) {               // 0'a bölüp NaN yazmasın
                pw.println("Geçerli kayıt yok.");
            } else {
                pw.printf("Geçen: %d, kalan: %d, ortalama: %.2f%n",
                        gecen, kalan, toplam / (gecen + kalan));
            }
        } catch (IOException e) {
            System.out.println("rapor.txt yazılamadı: " + e.getMessage());
            return;
        }

        System.out.println(atlanan + " satır atlandı. rapor.txt içeriği:");
        try {
            System.out.print(Files.readString(rapor));      // doğrulama
        } catch (IOException e) {
            System.out.println("rapor.txt okunamadı: " + e.getMessage());
        }
    }
}
```

Çıktı

```
Satır 5 atlandı: For input string: "sekiz"
Satır 6 atlandı: eksik alan
2 satır atlandı. rapor.txt içeriği:
No       Ad Soyad         Ort.  Durum
2026101  Ayşe Kaya       79.80  GEÇTİ
2026102  Burak Demir      40.80  KALDI
2026103  Cem Yıldız       93.00  GEÇTİ
2026106  Fatih Çelik      59.20  GEÇTİ
2026107  Gizem Öztürk     92.80  GEÇTİ
--------------------------------------
Geçen: 4, kalan: 1, ortalama: 73.12
```

Programın yapısına dikkat et:

1. **Okuma** ayrı bir `try` içinde: Dosya okunamazsa yapacak bir şey yok, mesaj verip `return` ile çıkıyoruz.
2. **Satır döngüsü** 1'den başlıyor (başlığı atlamak için). Satır numarası kullanıcıya `i + 1` olarak gösteriliyor, çünkü dosyadaki satırlar 1'den sayılır ve başlık 1. satırdır.
3. **Doğrulama sırası**: önce boş satır, sonra alan sayısı (`p.length`), en son sayıya çevirme. Böylece `ArrayIndexOutOfBoundsException` hiç oluşmuyor, `NumberFormatException` ise yakalanıp satır atlanıyor.
4. Rapor **try-with-resources** ile yazıldığı için, `printf`'ler bitince dosya kapanıyor; en sondaki `readString` dosyanın tam halini okuyor.

Bölüm Özeti – Bilmen Gerekenler

- RAM'deki veri program bitince kaybolur; kalıcılık için dosyaya yaz. `java.nio.file` (`Path`, `Files`) modern, `java.io` (`BufferedReader`, `FileWriter`, `PrintWriter`) klasik yoldur.
- `Path.of("a.txt")` göreli yoldur ve **çalışma klasörüne** göre çözülür (VS Code'da açtığın klasör). Windows yolunda `\\` veya `/` kullan.
- `Files.writeString/write` dosyayı oluşturur ama **eski içeriği siler**; eklemek için `StandardOpenOption.CREATE, StandardOpenOption.APPEND`. `readString` tek String, `readAllLines` satır listesi verir. `exists`, `size` (bayt), `delete`.
- `IOException` **checked**'tir: `throws IOException` ile bildir veya `try-catch` ile yakala; yoksa derlenmez. Olmayan dosya → `NoSuchFileException` (`java.io`'da `FileNotFoundException`), ikisi de `IOException`'ın alt sınıfı.
- Satır satır okuma kalıbı: `while ((satir = br.readLine()) != null)`. `readLine` dosya sonunda `null` döner. `write` satır sonu eklemez, `newLine()` ekler.
- Dosya kaynaklarını **try-with-resources** ile aç; kapatılmayan yazıcının tamponu diske boşalmayabilir.
- `new Scanner(Path.of("x.txt"))` dosyayı okur; `new Scanner("x.txt")` sadece metni tarar.
- `PrintWriter` ile dosyaya `printf`. Yazdıktan sonra geri okuyup doğrula.
- UTF-8'de Türkçe harfler 2 bayt; yanlış kodlamayla okuma `Ã§` gibi bozulmalar üretir. Java 18+ ve `Files` varsayılan olarak UTF-8 kullanır.
- CSV: `split(",")` → `trim()` → `Integer.parseInt`. Önce boş satır, sonra `length`, en son `NumberFormatException` kontrolü. `split` sondaki boş alanları atar.

Sorular

A) Çoktan Seçmeli

1 `main` metodunda `throws` ifadesi ve `try-catch` olmadan `String s = Files.readString(Path.of("a.txt"));` satırı yazılırsa ne olur?

(A) Dosya yoksa `s` değişkenine `null` atanır.

(B) Dosya yoksa `s` değişkenine boş String atanır.

(C) Program derlenir; dosya yoksa çalışırken `NoSuchFileException` ile çöker.

(D) Derleme hatası: `IOException` yakalanmalı veya `throws` ile bildirilmelidir.

(E) Derleme hatası: `Files` sınıfı `main` içinde kullanılamaz.

2 `Path p = Path.of("log.txt");` olsun. Dosya **yoksa oluşturulsun, varsa sonuna** `giriş` satırı eklensin isteniyor. Hangisi doğrudur?

(A) `Files.writeString(p, "giriş\n");`

(B) `Files.writeString(p, "giriş\n", StandardOpenOption.APPEND);`

(C) `Files.readString(p, "giriş\n");`

(D) `new PrintWriter("log.txt").println("giriş");`

(E) `Files.writeString(p, "giriş\n", StandardOpenOption.CREATE, StandardOpenOption.APPEND);`

3 `BufferedReader` nesnesinin `readLine()` metodu **dosyanın sonuna gelindiğinde** ne döndürür?

A) Boş String (`""`)
B) `null`
C) `-1`
D) `EOFException` fırlatır.
E) `"\n"`

4 `System.out.println("x,,y,".split(",").length);` komutu ne yazdırır?

A) `3`
B) `4`
C) `2`
D) `5`
E) `ArrayIndexOutOfBoundsException` fırlatır.

5 try-with-resources ile ilgili aşağıdakilerden hangisi **doğrudur**?

A) Kaynak yalnızca `try` bloğu hatasız biterse kapatılır.
B) try-with-resources kullanıldığında `catch` bloğu yazmak zorunludur.
C) `try` bloğunda exception oluşsa bile kaynak otomatik kapatılır ve bu, `catch` bloğu çalışmadan önce olur.
D) Birden fazla kaynak açıldığında, kaynaklar açıldıkları sırayla kapatılır.
E) Parantez içinde her sınıftan nesne oluşturulabilir; `AutoCloseable` uygulamak gerekmez.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (Çalışma klasöründe `s6.txt` adında dosya yok.)

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.List;

public class Soru6 {
    public static void main(String[] args) throws IOException {
        Path p = Path.of("s6.txt");
        Files.writeString(p, "bir\niki\n");
        Files.writeString(p, "üç\n", StandardOpenOption.APPEND);
        Files.write(p, List.of("dört", "beş"), StandardOpenOption.APPEND);
        List<String> l = Files.readAllLines(p);
        System.out.println(l.size() + " " + l);
        Files.writeString(p, "altı");
        System.out.println(Files.readAllLines(p));
        System.out.println(Files.readString(p).length());
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class Soru7 {
    public static void main(String[] args) throws IOException {
        Path p = Path.of("s7.csv");
        Files.write(p, List.of("a,10", "b, 20", "c,x", "d", "e,5,"));
        int toplam = 0, hata = 0;
        for (String satir : Files.readAllLines(p)) {
            String[] parca = satir.split(",");
            try {
                toplam += Integer.parseInt(parca[1].trim());
            } catch (NumberFormatException e) {
                hata++;
            } catch (ArrayIndexOutOfBoundsException e) {
                hata += 10;
            }
        }
        System.out.println(toplam + " " + hata);
        System.out.println("e,5,".split(",").length + " "
                + "b, 20".split(",")[1].length());
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
class Kaynak implements AutoCloseable {
    private String ad;

    Kaynak(String ad) {
        this.ad = ad;
        System.out.println(ad + " açıldı");
    }

    @Override
    public void close() {
        System.out.println(ad + " kapandı");
    }
}

public class Soru8 {
    public static void main(String[] args) {
        try (Kaynak a = new Kaynak("dosya"); Kaynak b = new Kaynak("tampon")) {
            System.out.println("okunuyor");
            throw new IllegalStateException("bozuk satır");
        } catch (IllegalStateException e) {
            System.out.println("yakalandı: " + e.getMessage());
        } finally {
            System.out.println("finally");
        }
    }
}
```

C) Kod Yazma

9 Çalışma klasöründeki `sayilar.txt` dosyasında her satırda bir tam sayı olması gerekiyor; ama bazı satırlar bozuk veya boş. `BufferedReader` ile dosyayı satır satır okuyan `SayiDosyasi` programını yaz. Program boş satırları sessizce atlasın, geçersiz satırları **satır numarasıyla** bildirsin; sonunda geçerli sayıların adedini, toplamını, en büyüğünü ve ortalamasını (2 basamak) yazdırsın.

sayilar.txt

```
15
8
abc
42

-3
7.5
20
```

10 `ogrenciler.csv` dosyasında öğrencilerin ad soyadı, vize ve final notları var. `SonucDosyasi` programını yaz: Her öğrencinin ortalamasını (vize %40 + final %60) hesaplasın, `sonuc.csv` dosyasına `ad,ortalama,durum` başlığıyla yazsın (ortalama 2 basamak; 50 ve üstü `GEÇTİ`, altı `KALDI`), sonra `sonuc.csv`'yi geri okuyup ekrana basarak doğrulasın.

ogrenciler.csv

```
ad,vize,final
Ali Yılmaz,60,70
Buse Er,30,40
Can Tan,80,45
Derya Ak,55,50
```

Çözümler

Cevap anahtarı

1 D	2 E	3 B	4 A	5 C

1 Cevap: D

`Files.readString` `IOException` fırlatabileceğini bildirir ve `IOException` **checked** bir istisnadır: Ya `try-catch` ile yakalanmalı ya da `throws IOException` ile bildirilmelidir. İkisi de yoksa program **derlenmez**. C şıkkı, `throws IOException` yazılmış olsaydı doğru olurdu. `readString` hiçbir durumda `null` veya boş String ile hatayı gizlemez.

2 Cevap: E

`APPEND` sonuna ekler, `CREATE` dosya yoksa oluşturur; ikisi birlikte istenen davranışı verir. A dosyanın içeriğini siler. B dosya yoksa `NoSuchFileException` fırlatır. C geçerli bir metot çağrısı değildir (`readString`'in ikinci parametresi bir karakter kodlamasıdır). D hem dosyayı baştan yazar hem de `PrintWriter` kapatılmadığı için satır diske hiç ulaşmayabilir.

3 Cevap: B

`readLine()` dosya sonunda `null` döndürür; okuma döngüsünün `!= null` koşulu bu yüzdendir. Boş String ise dosyadaki **boş bir satırı** temsil eder. `-1`, karakter karakter okuyan `read()` metodunun dosya sonu değeridir.

4 Cevap: A

`split(",")` parçaları `"x"`, `""`, `"y"`, `""` olarak böler; ama **sondaki boş String'leri atar**. Ortadaki boş alan kalır: `["x", "", "y"]` → 3 parça. Sondaki boş alanı da korumak için `split(",", -1)` yazılmalıydı; o zaman sonuç 4 olurdu.

Çıktı

```
3
```

5 Cevap: C

try-with-resources'ın amacı, `try` bloğundan nasıl çıkılırsa çıkılsın (normal bitiş veya exception) kaynağı kapatmaktır; kapatma `catch` ve `finally`'den **önce** yapılır. `catch` zorunlu değildir (A, B yanlış). Kaynaklar açılış sırasının **tersiyle** kapatılır (D yanlış). Parantez içindeki sınıf `AutoCloseable` (veya `Closeable`) uygulamalıdır (E yanlış). Bu davranışı Soru 8'de adım adım görebilirsin.

6

- `writeString(p, "bir\niki\n")` → dosya oluşur: `bir`, `iki`.
- `APPEND` ile `üç` eklenir; `Files.write` + `APPEND` ile `dört` ve `beş` ayrı satırlar olarak eklenir → 5 satır.
- Seçeneksiz `writeString(p, "altı")` eski içeriği **siler**: dosyada sadece `altı` kalır (sonunda satır sonu yok).
- `readString(p).length()` → `"altı"` 4 karakter (bayt olarak 5 olsa da `length()` karakter sayar).

```
Çıktı
5 [bir, iki, üç, dört, beş]
[altı]
4
```

7

Satır satır:

Satır	`split(",")`	Sonuç
`a,10`	`["a", "10"]`	toplam = 10
`b, 20`	`["b", " 20"]`	`trim` → 20, toplam = 30
`c,x`	`["c", "x"]`	`parseInt("x")` → `NumberFormatException`, hata = 1
`d`	`["d"]`	`parca[1]` yok → `ArrayIndexOutOfBoundsException`, hata = 11
`e,5,`	`["e", "5"]` (sondaki boş alan atıldı)	toplam = 35

Son satır: `"e,5,"` 2 parçaya bölünür; `"b, 20"`'nin ikinci parçası `" 20"` boşlukla birlikte 3 karakterdir.

```
Çıktı
35 11
2 3
```

8

1. Kaynaklar yazıldıkları sırayla **açılır**: önce `dosya`, sonra `tampon`.
2. Gövde çalışır, `okunuyor` yazılır ve exception fırlatılır.
3. `catch`'e geçmeden **önce** kaynaklar açılış sırasının **tersiyle** kapatılır: önce `tampon`, sonra `dosya`.
4. `catch` çalışır, en son `finally`.

```
Çıktı
dosya açıldı
tampon açıldı
okunuyor
tampon kapandı
dosya kapandı
yakalandı: bozuk satır
finally
```

9

Kalıp `while ((satir = br.readLine()) != null)` döngüsü. Satır numarasını ayrı bir sayaçla tutuyoruz; `trim()` sonrası boş satırlar `continue` ile atlanıyor, `parseInt` hataları `NumberFormatException` ile yakalanıyor. En büyük değer için başlangıç olarak `Integer.MIN_VALUE` kullanılır.

SayiDosyasi.java

```java
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class SayiDosyasi {
    public static void main(String[] args) {
```

```
        int adet = 0, toplam = 0, enBuyuk = Integer.MIN_VALUE, no = 0;
        try (BufferedReader br = Files.newBufferedReader(Path.of("sayilar.txt"))) {
            String satir;
            while ((satir = br.readLine()) != null) {
                no++;
                satir = satir.trim();
                if (satir.isEmpty()) {
                    continue;
                }
                try {
                    int sayi = Integer.parseInt(satir);
                    adet++;
                    toplam += sayi;
                    enBuyuk = Math.max(enBuyuk, sayi);
                } catch (NumberFormatException e) {
                    System.out.println("Satır " + no + " geçersiz: " + satir);
                }
            }
        } catch (IOException e) {
            System.out.println("Dosya okunamadı: " + e.getMessage());
            return;
        }

        if (adet == 0) {
            System.out.println("Geçerli sayı yok.");
        } else {
            System.out.println("Geçerli sayı: " + adet + ", toplam: " + toplam
                    + ", en büyük: " + enBuyuk);
            System.out.printf("Ortalama: %.2f%n", (double) toplam / adet);
        }
    }
}
```

Çıktı

```
Satır 3 geçersiz: abc
Satır 7 geçersiz: 7.5
Geçerli sayı: 5, toplam: 82, en büyük: 42
Ortalama: 16.40
```

`adet == 0` kontrolü, dosyada hiç geçerli sayı olmadığında sıfıra bölmeyi önler.

10

Okuma için `readAllLines`, yazma için `PrintWriter` + try-with-resources kullanıyoruz. Ondalık ayırıcının her bilgisayarda nokta olması için `printf`'e `Locale.US` veriyoruz; CSV'de virgül hem alan ayırıcı hem ondalık ayırıcı olursa dosya bozulur.

SonucDosyasi.java

```
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Locale;

public class SonucDosyasi {
    public static void main(String[] args) {
        Path girdi = Path.of("ogrenciler.csv");
        Path cikti = Path.of("sonuc.csv");
        try {
            List<String> satirlar = Files.readAllLines(girdi);
```

```
            try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(cikti))) {
                pw.println("ad,ortalama,durum");
                for (int i = 1; i < satirlar.size(); i++) {
                    String[] p = satirlar.get(i).split(",");
                    double ort = Integer.parseInt(p[1].trim()) * 0.4
                               + Integer.parseInt(p[2].trim()) * 0.6;
                    String durum = ort >= 50 ? "GEÇTİ" : "KALDI";
                    pw.printf(Locale.US, "%s,%.2f,%s%n", p[0].trim(), ort, durum);
                }
            }
            System.out.print(Files.readString(cikti));    // doğrulama
        } catch (IOException e) {
            System.out.println("Dosya hatası: " + e.getMessage());
        }
    }
}
```

Çıktı

```
ad,ortalama,durum
Ali Yılmaz,66.00,GEÇTİ
Buse Er,36.00,KALDI
Can Tan,59.00,GEÇTİ
Derya Ak,52.00,GEÇTİ
```

Bu çözüm verinin temiz olduğunu varsayıyor. Bozuk satırlara karşı sağlam olması için `NotRaporu`'ndaki gibi boş satır, alan sayısı ve `NumberFormatException` kontrollerini ekleyebilirsin.

PROJE 4

Kütüphane Yönetim Sistemi

Amaç

Bu projede küçük bir okul kütüphanesinin kayıtlarını tutan, menülü bir konsol programı yazacağız. Program açılışta kitap listesini bir CSV dosyasından okuyacak, kullanıcı kitap arayıp ödünç verip iade alabilecek, yeni kitap ekleyebilecek ve sonunda her şeyi aynı dosyaya geri kaydedecek. Böylece program kapatılıp açıldığında hiçbir bilgi kaybolmayacak.

Proje, son bölümlerin neredeyse tamamını bir araya getiriyor:

Konu	Projede nerede?
Sınıf, kapsülleme (Bölüm 9)	`Kitap` ve `Kutuphane` sınıfları, `private` alanlar
Exception, özel exception (Bölüm 12)	`KitapBulunamadiException`, `IllegalStateException`, çoklu `catch`
`ArrayList`, `HashMap`, `TreeMap` (Bölüm 13)	Kitap listesi, ISBN indeksi, yazar istatistiği
`Comparator`, lambda (Bölüm 13)	Yazara, sonra yıla göre sıralama; `forEach`
Dosya okuma/yazma, CSV (Bölüm 14)	`Files.readAllLines`, `PrintWriter`, bozuk satır atlama
`Scanner`, `switch` (Bölüm 3, 4)	Menü döngüsü

Gereksinimler

1. Program açılırken çalışma klasöründeki `kitaplar.csv` dosyasını okusun. Dosya yoksa boş bir kütüphaneyle başlasın; **bozuk satırları** uyarı vererek atlasın, çökmesin.
2. Her kitabın ISBN'i, başlığı, yazarı, basım yılı, ödünçte olup olmadığı ve ödünçteyse kimde olduğu tutulsun. ISBN **benzersiz** olsun.
3. Menüde şu işlemler olsun: - **Listele:** Bütün kitaplar, **yazara göre** (Türkçe alfabetik), aynı yazarınkiler **yıla göre** sıralı. - **Ara:** Başlıkta veya yazar adında geçen bir kelimeyle, **büyük/küçük harf duyarsız** arama. - **Ödünç ver / İade al:** ISBN ile. Zaten ödünçteki bir kitap tekrar verilemesin, raftaki kitap iade alınamasın; olmayan ISBN anlaşılır bir hata mesajı versin. - **Kitap ekle:** Aynı ISBN ikinci kez eklenemesin; yıl sayı değilse kabul edilmesin. - **İstatistik:** Toplam kitap ve yazar sayısı, **yazar başına kitap sayısı** (yazara göre sıralı). - **Kaydet:** Bütün kitapları aynı CSV dosyasına yazsın; doğrulama için dosyayı diskten geri okuyup göstersin.
4. Hiçbir kullanıcı hatası (yanlış menü seçimi, olmayan ISBN, sayı olmayan yıl) programı çökertmesin.

Tasarım

Programı dört sınıfa bölüyoruz. Her sınıfın **tek bir sorumluluğu** var: `Kitap` sadece kendi bilgisini ve durumunu bilir, `Kutuphane` kitap koleksiyonunu yönetir ama kullanıcıyla konuşmaz, kullanıcıyla konuşmak (menü, `Scanner`, mesajlar) `KutuphaneProgrami`'nın işidir.

Sınıf	Sorumluluk	Önemli üyeler
`Kitap`	Tek bir kitabın bilgileri ve ödünç durumu	`oduncVer`, `iadeAl`, `csvSatiri`, `toString`
`Kutuphane`	Kitap koleksiyonu: ekleme, bulma, arama, sıralama, istatistik, dosya	`List<Kitap> kitaplar`, `Map<String, Kitap> isbnIndeksi`, `yukle`, `kaydet`
`KitapBulunamadiException`	Olmayan ISBN için özel, kontrol edilen (checked) istisna	yapıcı
`KutuphaneProgrami`	Kullanıcı arayüzü: menü döngüsü, girdi okuma, hata mesajları	`main` ve her menü seçeneği için bir metot

Neden hem liste hem harita?

`Kutuphane` her kitabı **iki** yapıda birden tutuyor:

- `ArrayList<Kitap> kitaplar`: Kitapların **ekleme sırasını** korur. Listeleme, arama ve kaydetme bu liste üzerinden yapılır; kaydedilen dosyadaki sıra, okunan dosyadaki sırayla aynı kalır.
- `HashMap<String, Kitap> isbnIndeksi`: ISBN'den kitaba **doğrudan** ulaşmayı sağlar. Ödünç verirken 10 bin kitaplık bir listeyi baştan sona dolaşmak yerine `isbnIndeksi.get(isbn)` ile tek adımda buluruz. Aynı ISBN'in ikinci kez eklenip eklenmediğini de `containsKey` ile anında anlarız.

İki yapı da **aynı `Kitap` nesnelerine** referans tutar (Bölüm 9'daki aliasing); bir kitap ödünç verildiğinde değişiklik iki yapıdan da görünür. Tek kural: Kitap eklerken ikisine birden eklemek. Bu yüzden ekleme işi sadece `ekle` metodundan yapılıyor.

Dosya biçimi

`kitaplar.csv` bir başlık satırı ve her kitap için bir satırdan oluşur. Son alan, kitabı ödünç alan kişinin adıdır; kitap raftaysa **boştur**:

```
isbn,baslik,yazar,yil,odunc_alan
975-001,Tutunamayanlar,Oğuz Atay,1972,
975-002,Tehlikeli Oyunlar,Oğuz Atay,1973,Ayşe
```

Kod

KutuphaneProgrami.java

```java
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.NoSuchFileException;
import java.nio.file.Path;
import java.text.Collator;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Scanner;
import java.util.TreeMap;

class KitapBulunamadiException extends Exception {
    public KitapBulunamadiException(String isbn) {
        super("ISBN bulunamadı: " + isbn);
```

```
    }
}

class Kitap {
    private final String isbn;
    private final String baslik;
    private final String yazar;
    private final int yil;
    private boolean oduncte;
    private String oduncAlan;                // rafta ise null

    public Kitap(String isbn, String baslik, String yazar, int yil) {
        this.isbn = isbn;
        this.baslik = baslik;
        this.yazar = yazar;
        this.yil = yil;
    }

    public String getIsbn() { return isbn; }
    public String getBaslik() { return baslik; }
    public String getYazar() { return yazar; }
    public int getYil() { return yil; }
    public String getOduncAlan() { return oduncAlan; }

    // Ödünçteyse fırlatır; menü, ad sormadan önce de bunu çağırır
    public void raftaOlmali() {
        if (oduncte) {
            throw new IllegalStateException("\"" + baslik + "\" zaten ödünçte ("
                    + oduncAlan + ")");
        }
    }

    public void oduncVer(String kisi) {
        raftaOlmali();
        if (kisi.isBlank()) {     // boş adla kaydedilirse açılışta rafta görünürdü
            throw new IllegalArgumentException("Ödünç alanın adı boş olamaz");
        }
        oduncte = true;
        oduncAlan = kisi;
    }

    public void iadeAl() {
        if (!oduncte) {
            throw new IllegalStateException("\"" + baslik + "\" zaten rafta");
        }
        oduncte = false;
        oduncAlan = null;
    }

    public String csvSatiri() {              // rafta ise son alan boş kalır
        return isbn + "," + baslik + "," + yazar + "," + yil + ","
                + (oduncte ? oduncAlan : "");
    }

    @Override
    public String toString() {
        String durum = oduncte ? "ödünçte: " + oduncAlan : "rafta";
        return String.format("%-8s%-28s%-21s%5d  %s",
                isbn, baslik, yazar, yil, durum);
    }
}

class Kutuphane {
    private static final Locale TR = Locale.forLanguageTag("tr-TR");
    private static final Collator TR_SIRA = Collator.getInstance(TR);
```

```
    private final List<Kitap> kitaplar = new ArrayList<>();
    private final Map<String, Kitap> isbnIndeksi = new HashMap<>();
    private final List<String> bozukSatirlar = new ArrayList<>();  // silinmesin

    public int kitapSayisi() { return kitaplar.size(); }

    public boolean ekle(Kitap k) {
        if (isbnIndeksi.containsKey(k.getIsbn())) {
            return false;                    // aynı ISBN ikinci kez eklenmez
        }
        kitaplar.add(k);
        isbnIndeksi.put(k.getIsbn(), k);
        return true;
    }

    public Kitap bul(String isbn) throws KitapBulunamadiException {
        Kitap k = isbnIndeksi.get(isbn);      // listeyi dolaşmadan, tek adımda
        if (k == null) {
            throw new KitapBulunamadiException(isbn);
        }
        return k;
    }

    // Başlıkta veya yazarda geçen kitaplar (büyük/küçük harf duyarsız)
    public List<Kitap> ara(String sorgu) {
        String s = sorgu.toLowerCase(TR);
        List<Kitap> sonuc = new ArrayList<>();
        for (Kitap k : kitaplar) {
            if (k.getBaslik().toLowerCase(TR).contains(s)
                    || k.getYazar().toLowerCase(TR).contains(s)) {
                sonuc.add(k);
            }
        }
        return sonuc;
    }

    public List<Kitap> yazaraGoreSirali() {
        List<Kitap> kopya = new ArrayList<>(kitaplar);   // asıl sıra bozulmasın
        kopya.sort(Comparator.comparing(Kitap::getYazar, TR_SIRA)
                .thenComparingInt(Kitap::getYil));
        return kopya;
    }

    public Map<String, Integer> yazarIstatistigi() {
        Map<String, Integer> sayac = new TreeMap<>(TR_SIRA);
        for (Kitap k : kitaplar) {
            sayac.put(k.getYazar(), sayac.getOrDefault(k.getYazar(), 0) + 1);
        }
        return sayac;
    }

    public void yukle(Path dosya) throws IOException {
        List<String> satirlar = Files.readAllLines(dosya);
        for (int i = 1; i < satirlar.size(); i++) {       // 0. satır başlık
            if (satirlar.get(i).trim().isEmpty()) {
                continue;
            }
            // -1: sondaki boş alan (rafta olan kitap) atılmasın
            String[] p = satirlar.get(i).split(",", -1);
            try {
                if (p.length != 5) {
                    throw new IllegalArgumentException(p.length + " alan var");
                }
                Kitap k = new Kitap(p[0].trim(), p[1].trim(), p[2].trim(),
```

```
                        Integer.parseInt(p[3].trim()));
                if (!p[4].trim().isEmpty()) {
                    k.oduncVer(p[4].trim());
                }
                if (!ekle(k)) {
                    throw new IllegalArgumentException("tekrarlanan ISBN");
                }
            } catch (IllegalArgumentException e) {    // parseInt hatası da
                System.out.println("Uyarı: satır " + (i + 1) + " atlandı ("
                        + e.getMessage() + "), kaydederken korunacak");
                bozukSatirlar.add(satirlar.get(i));
            }
        }
    }

    public void kaydet(Path dosya) throws IOException {
        try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(dosya))) {
            pw.println("isbn,baslik,yazar,yil,odunc_alan");
            for (Kitap k : kitaplar) {
                pw.println(k.csvSatiri());
            }
            for (String satir : bozukSatirlar) {  // düzeltilene kadar kalsın
                pw.println(satir);
            }
        }
    }
}

public class KutuphaneProgrami {
    static final Path DOSYA = Path.of("kitaplar.csv");

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Kutuphane kutuphane = new Kutuphane();
        try {
            kutuphane.yukle(DOSYA);
            System.out.println(kutuphane.kitapSayisi() + " kitap yüklendi.");
        } catch (NoSuchFileException e) {
            System.out.println(DOSYA + " yok, boş kütüphaneyle başlanıyor.");
        } catch (IOException e) {
            System.out.println("Dosya okunamadı: " + e.getMessage());
        }

        while (true) {
            System.out.println();
            System.out.println("1) Listele  2) Ara  3) Ödünç ver  4) İade al");
            System.out.println("5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış");
            String secim = sor(sc, "Seçim: ");
            try {
                switch (secim) {
                    case "1" -> listele(kutuphane);
                    case "2" -> ara(kutuphane, sc);
                    case "3" -> oduncVer(kutuphane, sc);
                    case "4" -> iadeAl(kutuphane, sc);
                    case "5" -> kitapEkle(kutuphane, sc);
                    case "6" -> istatistik(kutuphane);
                    case "7" -> kaydet(kutuphane);
                    case "0" -> {
                        System.out.println("Güle güle!");
                        return;
                    }
                    default -> System.out.println("Geçersiz seçim.");
                }
            } catch (KitapBulunamadiException | IllegalStateException
                     | IllegalArgumentException e) {
```

```
                System.out.println("HATA: " + e.getMessage());
            }
        }
    }

    static String sor(Scanner sc, String soru) {    // soruyu yaz, satırı oku
        System.out.print(soru);
        return sc.nextLine().trim();
    }

    static void listele(Kutuphane kutuphane) {
        System.out.printf("%-8s%-28s%-21s%5s  %s%n",
                "ISBN", "Başlık", "Yazar", "Yıl", "Durum");
        for (Kitap k : kutuphane.yazaraGoreSirali()) {
            System.out.println(k);
        }
    }

    static void ara(Kutuphane kutuphane, Scanner sc) {
        List<Kitap> sonuc = kutuphane.ara(sor(sc, "Aranacak kelime: "));
        System.out.println(sonuc.size() + " sonuç");
        sonuc.forEach(System.out::println);
    }

    static void oduncVer(Kutuphane kutuphane, Scanner sc)
            throws KitapBulunamadiException {
        Kitap k = kutuphane.bul(sor(sc, "ISBN: "));
        k.raftaOlmali();                  // zaten ödünçteyse adı boşuna sorma
        String kisi = sor(sc, "Ödünç alan: ");
        k.oduncVer(kisi);
        System.out.println("Verildi: " + k.getBaslik() + " -> " + kisi);
    }

    static void iadeAl(Kutuphane kutuphane, Scanner sc)
            throws KitapBulunamadiException {
        Kitap k = kutuphane.bul(sor(sc, "ISBN: "));
        String kisi = k.getOduncAlan();
        k.iadeAl();
        System.out.println("İade alındı: " + k.getBaslik() + " (" + kisi + ")");
    }

    static void kitapEkle(Kutuphane kutuphane, Scanner sc) {
        String isbn = sor(sc, "ISBN: ");
        String baslik = sor(sc, "Başlık: ");
        String yazar = sor(sc, "Yazar: ");
        String yil = sor(sc, "Yıl: ");
        try {
            Kitap yeni = new Kitap(isbn, baslik, yazar, Integer.parseInt(yil));
            if (kutuphane.ekle(yeni)) {
                System.out.println("\"" + baslik + "\" eklendi.");
            } else {
                System.out.println("Bu ISBN zaten kayıtlı: " + isbn);
            }
        } catch (NumberFormatException e) {
            System.out.println("Yıl bir tam sayı olmalı.");
        }
    }

    static void istatistik(Kutuphane kutuphane) {
        Map<String, Integer> sayac = kutuphane.yazarIstatistigi();
        System.out.println("Toplam " + kutuphane.kitapSayisi() + " kitap, "
                + sayac.size() + " yazar:");
        sayac.forEach((yazar, adet) ->
                System.out.printf("  %-22s %d kitap%n", yazar, adet));
    }
```

```
    static void kaydet(Kutuphane kutuphane) {
        try {
            kutuphane.kaydet(DOSYA);
            System.out.println(DOSYA + " kaydedildi. Diskteki içerik:");
            System.out.print(Files.readString(DOSYA));     // doğrulama
        } catch (IOException e) {
            System.out.println("Kaydetme hatası: " + e.getMessage());
        }
    }
}
```

Kodun önemli noktaları

İki tür istisna. Olmayan bir ISBN, kütüphanenin normal işleyişinde her an karşılaşılabilecek, çağıranın **mutlaka ele alması gereken** bir durumdur; bu yüzden `KitapBulunamadiException`'ı `Exception`'dan türettik (checked). `bul` metodunu çağıran her metot ya onu yakalamak ya da `throws` ile bildirmek zorunda. Zaten ödünçteki bir kitabı tekrar vermek ise bir **kural ihlalidir**; bunun için Java'nın hazır `IllegalStateException`'ını (unchecked) kullandık. Boş ödünç alan adı gibi geçersiz bir girdi için de `IllegalArgumentException` fırlatılıyor. Menü döngüsündeki tek bir çoklu `catch` (Bölüm 12) bunların hepsini yakalayıp aynı biçimde yazdırıyor; böylece hiçbir menü hatası programı çökertmiyor.

`split(",", -1)`. Rafta olan bir kitabın satırı virgülle biter: `975-001,Tutunamayanlar,Oğuz Atay,1972,`. Bölüm 14'te gördüğün gibi, `split(",")` sondaki boş alanı **atar** ve 5 yerine 4 parça döndürür; bütün raftaki kitaplar "eksik alan" diye reddedilirdi. `-1` parametresi boş alanları korur.

`IllegalArgumentException` yakalamak. `yukle` metodunda yanlış alan sayısı ve tekrarlanan ISBN için kendimiz `IllegalArgumentException` fırlatıyoruz. Sayı olmayan bir yıl için `Integer.parseInt`'in fırlattığı `NumberFormatException` da `IllegalArgumentException`'ın **alt sınıfıdır**; bu yüzden tek bir `catch` üç bozukluğu birden yakalıyor.

Türkçe sıralama ve arama. `String`'in `compareTo`'su karakterlerin Unicode numaralarına bakar (Bölüm 13): `ğ`'nin numarası `r`'den büyük olduğu için "Orhan" "Oğuz"dan önce gelirdi; `İ` ile başlayan adlar da Bölüm 13'teki *İzmir* gibi en sona düşerdi. `Collator.getInstance(TR)` Türk alfabesinin sırasını bilen bir `Comparator`'dır; hem `Comparator.comparing(Kitap::getYazar, TR_SIRA)` ile listede hem `new TreeMap<>(TR_SIRA)` ile istatistikte kullandık. Aramada da `toLowerCase(TR)` kullanıyoruz: Türkçe kurallara göre `"İnce"` → `"ince"`, `"I"` → `"ı"` olur. Bölge belirtilmezse sonuç bilgisayarın dil ayarına bağlı olur: İngilizce ayarlı bir bilgisayarda `toLowerCase()`, `İ` harfini `i` + üst nokta işaretine çevirir ve `"ince"` araması *İnce Memed*'i bulamaz.

Her şey `nextLine` ile okunuyor. Menü seçimi de yıl da `sor` yardımcı metoduyla satır olarak okunuyor, sayıya gerekirse `Integer.parseInt` ile çevriliyor. Böylece Bölüm 3'teki `nextInt()` + `nextLine()` tuzağı hiç oluşmuyor; kullanıcı menüde harf yazsa bile program çökmüyor, sadece "Geçersiz seçim." diyor.

Kopyalayarak sıralama. `yazaraGoreSirali` asıl listeyi değil **kopyasını** sıralıyor. Böylece `kitaplar` listesi ekleme sırasını koruyor ve dosya her kaydedildiğinde satırların yeri değişmiyor.

Örnek çalışma

Başlangıçta çalışma klasöründe şu dosya var. Son satırın yıl alanı bozuk (sayı yerine yazı):

kitaplar.csv

```
isbn,baslik,yazar,yil,odunc_alan
975-001,Tutunamayanlar,Oğuz Atay,1972,
975-002,Tehlikeli Oyunlar,Oğuz Atay,1973,Ayşe
975-003,Kürk Mantolu Madonna,Sabahattin Ali,1943,
975-004,İçimizdeki Şeytan,Sabahattin Ali,1940,
975-005,Saatleri Ayarlama Enstitüsü,Ahmet Hamdi Tanpınar,1961,
975-006,Huzur,Ahmet Hamdi Tanpınar,1949,Mert
975-007,Benim Adım Kırmızı,Orhan Pamuk,1998,
975-008,İnce Memed,Yaşar Kemal,1955,
975-009,Sinekli Bakkal,Halide Edib Adıvar,bilinmiyor,
```

Kullanıcı sırasıyla şunları yapıyor: kitapları listeliyor; `ATAY` ve `ince` kelimelerini arıyor; *Kürk Mantolu Madonna*'yı Elif'e veriyor; aynı kitabı Can'a da vermeye çalışıyor; olmayan bir ISBN deniyor; Ayşe'deki *Tehlikeli Oyunlar*'ı iade alıyor; yeni bir kitap ekliyor; istatistiğe bakıyor; kaydediyor ve çıkıyor.

Çıktı

```
Uyarı: satır 10 atlandı (For input string: "bilinmiyor"), kaydederken korunacak
8 kitap yüklendi.

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 1
ISBN    Başlık                     Yazar                 Yıl  Durum
975-006 Huzur                      Ahmet Hamdi Tanpınar  1949  ödünçte: Mert
975-005 Saatleri Ayarlama Enstitüsü Ahmet Hamdi Tanpınar  1961  rafta
975-001 Tutunamayanlar             Oğuz Atay             1972  rafta
975-002 Tehlikeli Oyunlar          Oğuz Atay             1973  ödünçte: Ayşe
975-007 Benim Adım Kırmızı         Orhan Pamuk           1998  rafta
975-004 İçimizdeki Şeytan          Sabahattin Ali        1940  rafta
975-003 Kürk Mantolu Madonna       Sabahattin Ali        1943  rafta
975-008 İnce Memed                 Yaşar Kemal           1955  rafta

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 2
Aranacak kelime: ATAY
2 sonuç
975-001 Tutunamayanlar             Oğuz Atay             1972  rafta
975-002 Tehlikeli Oyunlar          Oğuz Atay             1973  ödünçte: Ayşe

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 2
Aranacak kelime: ince
1 sonuç
975-008 İnce Memed                 Yaşar Kemal           1955  rafta

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 3
ISBN: 975-003
Ödünç alan: Elif
Verildi: Kürk Mantolu Madonna -> Elif

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 3
ISBN: 975-003
HATA: "Kürk Mantolu Madonna" zaten ödünçte (Elif)

1) Listele  2) Ara  3) Ödünç ver  4) İade al
```

```
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 3
ISBN: 975-004
Ödünç alan:
HATA: Ödünç alanın adı boş olamaz

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 3
ISBN: 975-999
HATA: ISBN bulunamadı: 975-999

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 4
ISBN: 975-002
İade alındı: Tehlikeli Oyunlar (Ayşe)

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 5
ISBN: 975-010
Başlık: Çalıkuşu
Yazar: Reşat Nuri Güntekin
Yıl: 1922
"Çalıkuşu" eklendi.

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 6
Toplam 9 kitap, 6 yazar:
  Ahmet Hamdi Tanpınar   2 kitap
  Oğuz Atay              2 kitap
  Orhan Pamuk            1 kitap
  Reşat Nuri Güntekin    1 kitap
  Sabahattin Ali         2 kitap
  Yaşar Kemal            1 kitap

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 7
kitaplar.csv kaydedildi. Diskteki içerik:
isbn,baslik,yazar,yil,odunc_alan
975-001,Tutunamayanlar,Oğuz Atay,1972,
975-002,Tehlikeli Oyunlar,Oğuz Atay,1973,
975-003,Kürk Mantolu Madonna,Sabahattin Ali,1943,Elif
975-004,İçimizdeki Şeytan,Sabahattin Ali,1940,
975-005,Saatleri Ayarlama Enstitüsü,Ahmet Hamdi Tanpınar,1961,
975-006,Huzur,Ahmet Hamdi Tanpınar,1949,Mert
975-007,Benim Adım Kırmızı,Orhan Pamuk,1998,
975-008,İnce Memed,Yaşar Kemal,1955,
975-010,Çalıkuşu,Reşat Nuri Güntekin,1922,
975-009,Sinekli Bakkal,Halide Edib Adıvar,bilinmiyor,

1) Listele  2) Ara  3) Ödünç ver  4) İade al
5) Ekle  6) İstatistik  7) Kaydet  0) Çıkış
Seçim: 0
Güle güle!
```

Çıktıyı adım adım kontrol edelim:

- **Yükleme:** 9 kitap satırından biri (`bilinmiyor`) atlandı, 8 kitap yüklendi. Program çökmedi.
- **Listeleme:** Yazarlar Türk alfabesine göre sıralandı: *Oğuz Atay*, *Orhan Pamuk*'tan önce geliyor. Aynı yazarın kitapları yıla göre sıralı (*Huzur* 1949, *Saatleri Ayarlama Enstitüsü* 1961).

- **Arama:** Büyük harfle yazılan `ATAY` iki Oğuz Atay kitabını, `ince` de büyük `İ` ile başlayan *İnce Memed*'i buldu.
- **Ödünç verme:** İkinci deneme `IllegalStateException` ile reddedildi; kitap zaten ödünçte olduğu için `raftaOlmali` ödünç alanın adını **sormadan** hata verdi. Rafta olan `975-004` için ad boş bırakılınca da istek `IllegalArgumentException` ile reddedildi: boş adla kaydedilen bir kitap, `yukle`'deki `!p[4].trim().isEmpty()` kontrolü yüzünden bir sonraki açılışta sessizce rafa dönerdi; olmayan ISBN `KitapBulunamadiException` verdi. Program iki durumda da menüye döndü.
- **İstatistik:** Yeni eklenen *Reşat Nuri Güntekin* de yazar listesinde alfabetik yerinde.
- **Kaydetme ve doğrulama:** `7` seçeneği dosyayı yazdıktan sonra diskten **yeniden okuyup** gösterdi. Elif'in aldığı kitap, Ayşe'nin iade ettiği kitap ve yeni eklenen *Çalıkuşu* dosyaya doğru yazılmış. Bozuk *Sinekli Bakkal* satırı yüklenmedi ama **silinmedi** de: `bozukSatirlar` listesinde saklandı ve kaydederken dosyanın sonuna olduğu gibi geri yazıldı. Böylece kullanıcı dosyayı açıp düzeltebilir; programın her kaydedişte bozuk veriyi sessizce yok etmesi veri kaybı olurdu.

Kendin Geliştir

- **Kitap silme:** ISBN ile kitap silen bir menü seçeneği ekle. Kitabı hem `kitaplar` listesinden hem `isbnIndeksi`'nden silmeyi unutma; ödünçteki bir kitap silinememeli.
- **Okur sınırı:** Bir okur aynı anda en fazla 3 kitap alabilsin. İpucu: `Map<String, Integer>` ile okur başına ödünç sayısını tut ya da listeyi dolaşarak say.
- **Kaydetmeden çıkma uyarısı:** Son kayıttan sonra değişiklik yapıldıysa `0` seçildiğinde "Kaydedilmemiş değişiklikler var, yine de çıkılsın mı? (e/h)" diye sor. İpucu: `Kutuphane`'ye bir `boolean degisti` alanı ekle.
- **Ödünç tarihi ve gecikme:** `java.time.LocalDate` ile ödünç verme tarihini sakla, CSV'ye altıncı alan olarak yaz; 15 günü geçen kitapları listeleyen bir "Gecikenler" seçeneği ekle.
- **Daha iyi sorumluluk ayrımı:** `yukle` metodu uyarıları şu an doğrudan ekrana yazıyor. Uyarıları bir `List<String>` olarak döndürmesini sağla ve ekrana yazmayı `KutuphaneProgrami`'na bırak.
- **Virgüllü başlıklar:** Şu an başlığa virgül yazılırsa (`Savaş, Barış`) kayıt satırı 6 alana bölünür ve bir sonraki açılışta "bozuk satır" diye atlanır. Önce `kitapEkle`'de virgüllü girişi reddet; sonra daha iyisini dene: Gerçek CSV'lerde virgül içeren alanlar tırnak içine alınır (`"Savaş, Barış"`). Tırnaklı alanları doğru ayrıştıran bir `csvBol(String satir)` metodu yaz.

BÖLÜM 15

Özyineleme (Recursion)

Bu Bölümde Öğreneceklerin

- Özyinelemenin ne olduğunu ve bir metodun kendini çağırmasının ne anlama geldiğini
- Her özyinelemeli metodun iki parçasını: **temel durum** (base case) ve **özyinelemeli durum**
- Çağrı yığınını (call stack) adım adım izlemeyi: iniş ve çıkış
- Faktöriyel, üs alma, basamak toplamı, EBOB, ikili sayıya çevirme gibi sayı örneklerini
- Fibonacci'nin saf halinin neden yavaş olduğunu ve dizi ile hafızalamayı (memoization)
- String ters çevirme, palindrom, dizi toplamı/en büyüğü gibi indeksli özyinelemeyi
- Hanoi kuleleri problemini ve özyinelemeli ikili aramayı
- Temel durum unutulunca oluşan `StackOverflowError`'ı
- "Çağrıdan önce mi, sonra mı yazdırılıyor?" çıktı sorularını çözmeyi

15.1 Özyineleme nedir?

Özyineleme (recursion), bir metodun **kendi kendini çağırmasıdır**. Kulağa garip gelir: "Metot daha bitmeden kendini nasıl çağırır?" Cevap: Her çağrı problemin **daha küçük** bir halini çözer ve en sonunda cevabı doğrudan bilinen çok küçük bir duruma ulaşılır.

Günlük hayattan bir örnek: Sinemada uzun bir kuyrukta beklerken kaçıncı sırada olduğunu merak ediyorsun. Herkesi tek tek saymak yerine önündekine soruyorsun: "Sen kaçıncısın?" O da kendi önündekine soruyor, o da önündekine... En öndeki kişinin önünde kimse yok; o hemen **"1"** diyor. Sonra cevaplar geri dönüyor: Arkasındaki "2", onun arkasındaki "3"... Sana cevap gelince sen de 1 ekliyorsun.

Bu küçük hikâyede özyinelemenin bütün parçaları var:

- **Aynı soru, daha küçük problem:** Herkes aynı soruyu, kendinden bir kişi daha kısa bir kuyruk için soruyor.
- **Durma noktası:** En öndeki kişi kimseye sormadan cevap veriyor.
- **İniş ve çıkış:** Sorular öne doğru gidiyor (iniş), cevaplar arkaya doğru dönüyor (çıkış).

İlk özyinelemeli metodumuz geriye sayıyor:

GeriSayim.java

```java
public class GeriSayim {
    static void say(int n) {
        if (n == 0) {                    // durma noktası
            System.out.println("Ateş!");
            return;
        }
        System.out.println(n);
        say(n - 1);                      // kendini daha küçük n ile çağırır
    }

    public static void main(String[] args) {
        say(3);
    }
}
```

Çıktı

```
3
2
1
Ateş!
```

`say(3)` önce `3` yazar, sonra `say(2)`'yi çağırır; o `2` yazıp `say(1)`'i çağırır... `say(0)` ise kendini çağırmaz, `Ateş!` yazıp döner. Döngü kullanmadan tekrar elde ettik.

15.2 Temel durum ve özyinelemeli durum

Doğru yazılmış her özyinelemeli metot iki parçadan oluşur:

1. **Temel durum** (base case): Cevabı **doğrudan** bilinen, metodun kendini **çağırmadığı** en küçük durum. Özyinelemeyi durduran fren budur.
2. **Özyinelemeli durum** (recursive case): Problemi **bir adım küçültüp** metodu tekrar çağıran, gelen cevabı kullanarak kendi cevabını kuran kısım. Her çağrı temel duruma **yaklaşmalıdır**.

Klasik örnek **faktöriyel**: `n! = n × (n−1) × ... × 1` ve `0! = 1`. Dikkat edersen `5! = 5 × 4!`, `4! = 4 × 3!`... Yani faktöriyel kendisi cinsinden tanımlanabilir:

Matematik	Java
`n ≤ 1` ise `n! = 1`	`if (n <= 1) return 1;` → temel durum
`n > 1` ise `n! = n × (n−1)!`	`return n * faktoriyel(n - 1);` → özyinelemeli durum

Faktoriyel.java

```java
public class Faktoriyel {
    static long faktoriyel(int n) {
        if (n <= 1) {
            return 1;                    // temel durum
        }
        return n * faktoriyel(n - 1);    // özyinelemeli durum
    }

    public static void main(String[] args) {
        for (int i = 0; i <= 5; i++) {
            System.out.println(i + "! = " + faktoriyel(i));
        }
        System.out.println("20! = " + faktoriyel(20));
    }
}
```

Çıktı

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
20! = 2432902008176640000
```

Dönüş tipini `long` seçtik, çünkü faktöriyel çok hızlı büyür: `13!` zaten `int`'e sığmaz. `20!` `long`'a sığan en büyük faktöriyeldir; `21!` `long`'u da taşırır (Bölüm 2'deki taşma).

> **İpucu: Özyinelemeli metot yazarken üç soru**
>
> 1. **En küçük durum ne ve cevabı ne?** (0, 1, boş String, dizinin sonu...) → temel durum.
> 2. **Problemi bir adım nasıl küçültürüm?** (`n - 1`, `n / 10`, `s.substring(1)`, `i + 1`...)
> 3. **Küçük problemin cevabı elimdeyse, büyüğün cevabını nasıl kurarım?** (`n * ...`, `... + n % 10`...)
>
> Üçüncü adımda küçük çağrının **doğru çalıştığına güven**; içine girip her adımı kafanda çalıştırmaya çalışma. Programcılar buna "inanç sıçraması" (leap of faith) der.

15.3 Çağrı yığını (call stack)

Bir metot çağrıldığında Java, o çağrı için bellekte bir **çerçeve** (stack frame) ayırır. Çerçevede metodun parametreleri, yerel değişkenleri ve "iş bitince nereye dönüleceği" bilgisi tutulur. Bu çerçeveler üst üste bir **yığın** (stack) halinde dizilir: Yeni çağrı en üste konur, biten çağrı en üstten kaldırılır. Bu düzene **LIFO** (Last In, First Out: son giren ilk çıkar) denir; üst üste konmuş tabaklar gibi.

Özyinelemede aynı metodun **birçok çerçevesi** aynı anda yığında bulunur ve her birinin kendi `n`'i vardır. `faktoriyel(4)` çağrısını adım adım izleyelim (kısaca `f` yazıyoruz):

Adım	Olay	Yığındaki çerçeveler (alttan üste)	Bekleyen işlem
1	İniş: `f(4)` çağrıldı	main, f(4)	`4 * f(3)`
2	İniş: `f(3)` çağrıldı	main, f(4), f(3)	`3 * f(2)`
3	İniş: `f(2)` çağrıldı	main, f(4), f(3), f(2)	`2 * f(1)`
4	Dip: `f(1)` temel durum	main, f(4), f(3), f(2), f(1)	`1` döner
5	Çıkış: `f(2)` devam eder	main, f(4), f(3), f(2)	`2 * 1 = 2` döner
6	Çıkış: `f(3)` devam eder	main, f(4), f(3)	`3 * 2 = 6` döner
7	Çıkış: `f(4)` devam eder	main, f(4)	`4 * 6 = 24` döner

İnişte hiçbir çarpma yapılmaz; her çerçeve bir alttakinin cevabını **bekler**. Hesaplama çıkışta, temel durumdan geriye doğru yapılır.

Aynı şeyi programın kendisine anlattıralım. Her çağrı, derinliği kadar girintiyle ne yaptığını yazsın:

FaktoriyelIz.java

```java
public class FaktoriyelIz {
    static long faktoriyel(int n, int derinlik) {
        String girinti = "  ".repeat(derinlik);
        System.out.println(girinti + "faktoriyel(" + n + ") çağrıldı");
        if (n <= 1) {
            System.out.println(girinti + "temel durum -> 1 döner");
            return 1;
        }
        long sonuc = n * faktoriyel(n - 1, derinlik + 1);
        System.out.println(girinti + "faktoriyel(" + n + ") = " + sonuc + " döner");
        return sonuc;
    }

    public static void main(String[] args) {
        long f = faktoriyel(4, 0);
        System.out.println("Sonuç: " + f);
    }
}
```

Çıktı

```
faktoriyel(4) çağrıldı
  faktoriyel(3) çağrıldı
    faktoriyel(2) çağrıldı
      faktoriyel(1) çağrıldı
      temel durum -> 1 döner
    faktoriyel(2) = 2 döner
  faktoriyel(3) = 6 döner
faktoriyel(4) = 24 döner
Sonuç: 24
```

Çıktının şekline dikkat et: Girinti önce artıyor (iniş), sonra azalıyor (çıkış). Çağrıdan **önceki** satır (`çağrıldı`) büyükten küçüğe, çağrıdan **sonraki** satır (`döner`) küçükten büyüğe yazıldı. Bu gözlem, bölümün sonundaki çıktı sorularının anahtarı.

> **Sık Yapılan Hata: Problem küçülmüyorsa özyineleme bitmez**
>
> `return n * faktoriyel(n);` yazarsan (yani `n - 1` yerine `n`), her çağrı **aynı** problemi tekrar sorar ve temel duruma hiç ulaşılmaz. Bir diğer sık hata, özyinelemeli çağrının sonucunu kullanmayı unutmaktır: `faktoriyel(n - 1);` satırı tek başına yazılırsa hesaplanan değer çöpe gider. Değer döndüren özyinelemede çağrı mutlaka bir `return` ifadesinin içinde kullanılmalıdır.

15.4 Sayılarla özyineleme

Üs alma

$a^n = a \times a^{n-1}$ ve $a^0 = 1$. Tanımı doğrudan Java'ya çevirelim:

UsAlma.java

```
public class UsAlma {
    static long us(long a, int n) {
        if (n < 0) {                    // yoksa n 0'a inmez: StackOverflowError
            throw new IllegalArgumentException("Üs negatif olamaz: " + n);
        }
        if (n == 0) {
            return 1;                   // a^0 = 1
        }
        return a * us(a, n - 1);        // a^n = a * a^(n-1)
    }

    public static void main(String[] args) {
        System.out.println("2^10 = " + us(2, 10));
        System.out.println("3^4  = " + us(3, 4));
        System.out.println("5^0  = " + us(5, 0));
    }
}
```

Çıktı

```
2^10 = 1024
3^4  = 81
5^0  = 1
```

`us(2, 10)` için metot `n` = 10, 9, …, 0 değerleriyle toplam 11 kez çağrılır. Baştaki `n < 0` kontrolü önemli: negatif bir üste `n` her çağrıda biraz daha küçülür ve temel durum (`n == 0`) **hiç gelmez**; program `StackOverflowError` ile çöker. Temel duruma ulaşılamayan girdileri baştan reddetmek, özyinelemede iyi bir alışkanlıktır. Daha akıllı bir yol da var: $2^{10} = (2^5)^2$ olduğu için üssü her adımda **yarıya** indirmek mümkündür (tek üslerde bir taban fazladan çarpılır). O zaman `n` yerine yaklaşık $\log_2 n$ çağrı yeter. Bu "yarıya bölme" fikrini Bölüm 16'da ikili aramada ve Big-O'da göreceksin.

Basamak toplamı, EBOB ve ikili sayıya çevirme

Bölüm 2'de gördüğün `n % 10` (son basamak) ve `n / 10` (son basamağı at) ikilisi, sayılarla özyinelemenin temelidir:

- **Basamak toplamı:** `4728`'in basamak toplamı = `8` + `472`'nin basamak toplamı. Tek basamaklı sayıda cevap sayının kendisidir; negatif sayıda işareti atıp aynı işi yaparız.
- **EBOB (Öklid algoritması):** `ebob(a, b) = ebob(b, a % b)`, `b` sıfır olunca cevap `a`'dır.
- **İkili sayıya çevirme:** `n`'in ikilik gösterimi = `n / 2`'nin ikilik gösterimi + son bit (`n % 2`).

SayiOzyineleme.java

```java
public class SayiOzyineleme {
    static int basamakTopla(int n) {
        if (n < 0) {
            return basamakTopla(-n);     // -47 → 47'nin basamakları
        }
        if (n < 10) {
            return n;
        }
        return n % 10 + basamakTopla(n / 10);
    }

    static int ebob(int a, int b) {
        if (b == 0) {
            return a;
        }
        return ebob(b, a % b);
    }

    static String ikilik(int n) {
        if (n < 2) {
            return "" + n;               // 0 veya 1: tek bit
        }
        return ikilik(n / 2) + n % 2;    // önce baştaki bitler, sonra son bit
    }

    public static void main(String[] args) {
        System.out.println("basamakTopla(4728) = " + basamakTopla(4728));
        System.out.println("ebob(48, 18) = " + ebob(48, 18));
        System.out.println("ekok(48, 18) = " + 48 / ebob(48, 18) * 18);
        System.out.println("ikilik(13) = " + ikilik(13));
        System.out.println("ikilik(64) = " + ikilik(64));
    }
}
```

Çıktı

```
basamakTopla(4728) = 21
ebob(48, 18) = 6
ekok(48, 18) = 144
ikilik(13) = 1101
ikilik(64) = 1000000
```

`ebob(48, 18)` çağrısının izi:

Çağrı	a	b	a % b
`ebob(48, 18)`	48	18	12
`ebob(18, 12)`	18	12	6
`ebob(12, 6)`	12	6	0
`ebob(6, 0)`	6	0	temel durum → **6** döner

EKOK'u da EBOB'dan bulduk: `ekok(a, b) = a × b / ebob(a, b)`. Kodda önce bölüp sonra çarptık (`a / ebob(a, b) * b`): sonuç aynıdır, ama büyük sayılarda `a * b` çarpımı `int` sınırını aşıp taşabilir; önce bölmek bunu önler.

`ikilik(13)` ise şöyle açılır: `ikilik(6) + 1` → `(ikilik(3) + 0) + 1` → `((ikilik(1) + 1) + 0) + 1` → `"1" + "1" + "0" + "1"` = `"1101"`. Özyinelemeli çağrı **önce**, son bit **sonra** eklendiği için bitler doğru sırada çıkar. Döngüyle yapsaydın bitler ters sırada elde edilirdi.

15.5 Fibonacci ve tekrar eden hesaplar

Fibonacci dizisinde her terim, kendinden önceki iki terimin toplamıdır: `0, 1, 1, 2, 3, 5, 8, 13, 21, ...` Tanımı zaten özyinelemelidir:

- `fib(0) = 0`, `fib(1) = 1` (iki temel durum)
- `fib(n) = fib(n−1) + fib(n−2)`

Java'ya çevirmek çok kolay. Ama metodun kaç kez çağrıldığını da sayalım:

FibonacciSayac.java

```java
public class FibonacciSayac {
    static long cagriSayisi = 0;

    static long fib(int n) {
        cagriSayisi++;
        if (n <= 1) {
            return n;
        }
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        int[] degerler = {5, 10, 20, 30};
        for (int n : degerler) {
            cagriSayisi = 0;
            long sonuc = fib(n);
            System.out.printf("fib(%d) = %d, çağrı sayısı: %,d%n",
                    n, sonuc, cagriSayisi);
        }
    }
}
```

Çıktı

```
fib(5) = 5, çağrı sayısı: 15
fib(10) = 55, çağrı sayısı: 177
fib(20) = 6765, çağrı sayısı: 21,891
fib(30) = 832040, çağrı sayısı: 2,692,537
```

`fib(30)`'u bulmak için metot iki buçuk milyondan fazla kez çağrıldı! Neden? `fib(5)`'in çağrı ağacına bakalım:

```
fib(5)
├── fib(4)
│   ├── fib(3)
│   │   ├── fib(2)
│   │   │   ├── fib(1)
│   │   │   └── fib(0)
│   │   └── fib(1)
│   └── fib(2)
│       ├── fib(1)
│       └── fib(0)
└── fib(3)
    ├── fib(2)
    │   ├── fib(1)
    │   └── fib(0)
    └── fib(1)
```

Her çağrı **iki** yeni çağrı yapıyor ve aynı değerler tekrar tekrar hesaplanıyor: `fib(3)` 2 kez, `fib(2)` 3 kez hesaplandı. Toplam 15 çağrı. `n` bir artınca çağrı sayısı yaklaşık 1,6 katına çıkar; `fib(40)` için 300 milyonu, `fib(50)` için 40 milyarı aşar. Bu tür büyümeye **üstel** (exponential) büyüme denir; Bölüm 16'da $O(2^n)$ olarak göreceksin.

Hafızalama (memoization)

Çözüm basit: Bir değeri **ilk kez** hesapladığında bir diziye yaz; tekrar istendiğinde hesaplamak yerine diziden oku. Buna **hafızalama** (memoization) denir:

FibonacciMemo.java

```
public class FibonacciMemo {
    static long[] hafiza = new long[91];    // fib(90)'a kadar
    static int cagriSayisi = 0;

    static long fib(int n) {
        cagriSayisi++;
        if (n <= 1) {
            return n;
        }
        if (hafiza[n] != 0) {               // daha önce hesaplandı mı?
            return hafiza[n];
        }
        hafiza[n] = fib(n - 1) + fib(n - 2);
        return hafiza[n];
    }

    public static void main(String[] args) {
        long sonuc = fib(30);
        System.out.println("fib(30) = " + sonuc + ", çağrı sayısı: " + cagriSayisi);
        System.out.println("fib(90) = " + fib(90));
    }
}
```

Çıktı

```
fib(30) = 832040, çağrı sayısı: 59
fib(90) = 2880067194370816120
```

Çağrı sayısı 2,7 milyondan **59**'a düştü; `fib(90)` bile anında hesaplandı. `long` dizisinin elemanları başlangıçta `0` olduğu için "0 ise henüz hesaplanmamış" diyebiliyoruz.

> **Sınavda Çıkar**
>
> "Saf özyinelemeli Fibonacci neden yavaştır?" sorusunun cevabı: **Aynı alt problemler defalarca hesaplanır** (tekrar eden hesaplar); çağrı sayısı üstel artar. Çözüm: hafızalama (sonuçları dizide saklamak) veya döngü ile hesaplamak. "`fib(5)` kaç çağrı yapar?" (15) ve "`fib(5)` hesaplanırken `fib(2)` kaç kez çağrılır?" (3) da sık sorulur; ağacı çizerek çöz.

15.6 String ve dizilerle özyineleme

String ters çevirme ve palindrom

String'i küçültmenin yolu `substring`'dir (Bölüm 8):

- **Ters çevirme:** `"JAVA"`'nın tersi = `"AVA"`'nın tersi + `'J'`. Boş veya tek harfli String'in tersi kendisidir.

- **Palindrom:** Baştan ve sondan okunuşu aynı olan kelime (`kayak`). İlk ve son harf aynıysa, **ortadaki kısım** palindrom mu diye bak. Farklıysa hemen `false`.

StringOzyineleme.java

```java
public class StringOzyineleme {
    static String ters(String s) {
        if (s.length() <= 1) {
            return s;
        }
        return ters(s.substring(1)) + s.charAt(0);
    }

    static boolean palindromMu(String s) {
        if (s.length() <= 1) {
            return true;                          // boş veya tek harf
        }
        if (s.charAt(0) != s.charAt(s.length() - 1)) {
            return false;                         // uçlar farklı
        }
        return palindromMu(s.substring(1, s.length() - 1));   // ortası
    }

    public static void main(String[] args) {
        System.out.println(ters("JAVA"));
        System.out.println(ters("özyineleme"));
        String[] kelimeler = {"kayak", "ey edip adanada pide ye", "java", "abba"};
        for (String k : kelimeler) {
            System.out.println(k + " -> " + palindromMu(k));
        }
    }
}
```

Çıktı

```
AVAJ
emeleniyzö
kayak -> true
ey edip adanada pide ye -> true
java -> false
abba -> true
```

`ters("JAVA")` çağrısı: `ters("AVA") + 'J'` → `(ters("VA") + 'A') + 'J'` → `((ters("A") + 'V') + 'A') + 'J'` → `"A" + 'V' + 'A' + 'J'` = `"AVAJ"`. İlk harf her seferinde **sona** eklendiği için String ters döner.

Dizide toplam ve en büyük eleman

Dizinin kendisini küçültmek yerine **indeks** ile ilerleriz: `toplam(d, i)` = "`i` indeksinden sona kadar olan elemanların toplamı". Temel durum dizinin sonu (`i == d.length`):

DiziOzyineleme.java

```java
public class DiziOzyineleme {
    // i. indeksten sona kadar olan elemanların toplamı
    static int toplam(int[] d, int i) {
        if (i == d.length) {
            return 0;                         // toplanacak eleman kalmadı
        }
        return d[i] + toplam(d, i + 1);
    }

    // i. indeksten sona kadar olan elemanların en büyüğü
    static int enBuyuk(int[] d, int i) {
```

```
        if (i == d.length - 1) {
            return d[i];                    // tek eleman kaldı
        }
        int kalaninEnBuyugu = enBuyuk(d, i + 1);
        return Math.max(d[i], kalaninEnBuyugu);
    }

    // kullanıcı indeksle uğraşmasın diye yardımcı metot (overload)
    static int toplam(int[] d) {
        return toplam(d, 0);
    }

    public static void main(String[] args) {
        int[] sayilar = {4, 9, 2, 7, 5};
        System.out.println("Toplam: " + toplam(sayilar));
        System.out.println("En büyük: " + enBuyuk(sayilar, 0));
        System.out.println("3. indeksten itibaren toplam: " + toplam(sayilar, 3));
    }
}
```

Çıktı

```
Toplam: 27
En büyük: 9
3. indeksten itibaren toplam: 12
```

Bu kalıp çok yaygındır: Asıl iş, fazladan bir **indeks parametresi** alan metotta yapılır; dışarıya ise sadece diziyi alan sade bir metot sunulur. İki metodun adı aynı, parametreleri farklı: Bölüm 6'daki aşırı yükleme (overloading).

15.7 Hanoi kuleleri

Özyinelemenin gücünü en iyi gösteren problem budur. Üç çubuk var: A, B, C. A çubuğunda büyükten küçüğe dizilmiş `n` disk duruyor. Amaç bütün diskleri C'ye taşımak. Kurallar:

1. Bir hamlede **sadece bir** disk taşınır (en üstteki).
2. Büyük disk asla küçük diskin **üstüne** konamaz.

Döngüyle çözmeye çalışırsan kafan karışır. Özyinelemeyle düşün: En alttaki (en büyük) diski C'ye taşıyabilmek için üstündeki `n - 1` diskin B'de olması gerekir. Yani:

1. Üstteki `n - 1` diski A'dan **B**'ye taşı (C'yi yardımcı kullanarak) → aynı problem, bir disk eksik!
2. En büyük diski A'dan **C**'ye taşı.
3. B'deki `n - 1` diski **C**'ye taşı (A'yı yardımcı kullanarak) → yine aynı problem.

Temel durum: Taşınacak disk yoksa (`n == 0`) hiçbir şey yapma.

Hanoi.java

```java
public class Hanoi {
    static int hamle = 0;

    static void hanoi(int n, char kaynak, char hedef, char yardimci) {
        if (n == 0) {
            return;
        }
        hanoi(n - 1, kaynak, yardimci, hedef);   // 1) n-1 diski yardımcıya
        hamle++;                                  // 2) en büyük diski hedefe
        System.out.println(hamle + ". hamle: disk " + n + "  "
                + kaynak + " -> " + hedef);
        hanoi(n - 1, yardimci, hedef, kaynak);   // 3) n-1 diski hedefe
    }

    public static void main(String[] args) {
        hanoi(3, 'A', 'C', 'B');
        System.out.println("Toplam hamle: " + hamle);
    }
}
```

Çıktı

```
1. hamle: disk 1  A -> C
2. hamle: disk 2  A -> B
3. hamle: disk 1  C -> B
4. hamle: disk 3  A -> C
5. hamle: disk 1  B -> A
6. hamle: disk 2  B -> C
7. hamle: disk 1  A -> C
Toplam hamle: 7
```

Hamle sayısını `H(n)` ile gösterelim: `n - 1` diski iki kez taşıyoruz, arada 1 hamle var: `H(n) = 2·H(n−1) + 1`, `H(1) = 1`. Buradan `H(n) = 2ⁿ − 1` çıkar:

Disk sayısı (n)	1	2	3	4	10	20	64
Hamle ($2^n - 1$)	1	3	7	15	1 023	1 048 575	$\approx 1{,}8 \times 10^{19}$

Efsaneye göre bir tapınakta rahipler 64 diskli kuleyi taşıyor ve iş bitince dünyanın sonu gelecek. Saniyede bir hamle yapsalar bile yaklaşık 585 milyar yıl sürer; endişelenmene gerek yok.

Sınavda Çıkar: Hanoi'de bilmen gerekenler

- `n` disk için en az hamle sayısı **$2^n - 1$**'dir: 3 disk → 7, 4 disk → 15, 5 disk → 31.
- Özyineleme bağıntısı: `H(n) = 2·H(n−1) + 1`.
- Metotta **iki** özyinelemeli çağrı vardır ve `println` ikisinin **arasındadır**. "3 disk için 4. hamle nedir?" gibi sorularda yukarıdaki çıktıyı kendin üretebilmelisin: Ortadaki hamle (4.) her zaman en büyük diskin A'dan C'ye taşınmasıdır.

15.8 Özyinelemeli ikili arama

Bölüm 16'da ayrıntısıyla göreceğin **ikili arama** (binary search), sıralı bir dizide aranan değeri bulmak için diziyi her adımda ikiye böler: Ortadaki elemana bakar; aranan daha küçükse sol yarıda, daha büyükse sağ yarıda aramaya devam eder. "Yarıda aramaya devam et" cümlesi zaten özyinelemelidir:

IkiliAramaOzyinelemeli.java

```java
public class IkiliAramaOzyinelemeli {
    static int ara(int[] d, int aranan, int low, int high) {
        if (low > high) {
            return -1;                      // temel durum 1: aralık boş
        }
        int mid = (low + high) / 2;
        System.out.println("  low=" + low + " high=" + high + " mid=" + mid
                + " d[mid]=" + d[mid]);
        if (d[mid] == aranan) {
            return mid;                     // temel durum 2: bulundu
        }
        if (aranan < d[mid]) {
            return ara(d, aranan, low, mid - 1);    // sol yarı
        }
        return ara(d, aranan, mid + 1, high);       // sağ yarı
    }

    public static void main(String[] args) {
        int[] d = {3, 8, 12, 17, 23, 31, 38, 45, 52, 60, 71};
        System.out.println("52 aranıyor:");
        System.out.println("Sonuç: " + ara(d, 52, 0, d.length - 1));
        System.out.println("5 aranıyor:");
        System.out.println("Sonuç: " + ara(d, 5, 0, d.length - 1));
    }
}
```

Çıktı

```
52 aranıyor:
  low=0 high=10 mid=5 d[mid]=31
  low=6 high=10 mid=8 d[mid]=52
Sonuç: 8
5 aranıyor:
  low=0 high=10 mid=5 d[mid]=31
  low=0 high=4 mid=2 d[mid]=12
  low=0 high=1 mid=0 d[mid]=3
  low=1 high=1 mid=1 d[mid]=8
Sonuç: -1
```

Bu metodun **iki** temel durumu var: aranan bulunduysa ya da aranacak aralık boşaldıysa (`low > high`). 11 elemanlı dizide en fazla 4 adımda sonuca ulaşıldı. Aynı algoritmanın döngülü halini, izleme tablosunu ve neden bu kadar hızlı olduğunu Bölüm 16'da göreceksin.

15.9 Temel durum unutulursa: StackOverflowError

Temel durumu olmayan (ya da temel duruma hiç ulaşamayan) bir metot kendini durmadan çağırır. Her çağrı yığına yeni bir çerçeve ekler. Yığın belleği sınırlı olduğu için birkaç bin (ayarlara göre on binlerce) çağrıdan sonra yer biter ve JVM `StackOverflowError` fırlatır:

SonsuzOzyineleme.java

```java
public class SonsuzOzyineleme {
    static int say(int n) {
        return say(n + 1);          // temel durum YOK!
    }

    public static void main(String[] args) {
        System.out.println("Başlıyor...");
        try {
            say(1);
        } catch (StackOverflowError e) {
            System.out.println("Yakalandı: " + e.getClass().getName());
        }
        System.out.println("Program devam ediyor.");
    }
}
```

Çıktı

```
Başlıyor...
Yakalandı: java.lang.StackOverflowError
Program devam ediyor.
```

Burada hatayı sadece göstermek için `try-catch` ile yakaladık. `try-catch` olmasa program `Exception in thread "main" java.lang.StackOverflowError` mesajıyla çöker ve ekrana aynı satırı yüzlerce kez tekrarlayan upuzun bir yığın izi (stack trace) basar. Yığın izinde **aynı metodun aynı satırının** tekrar tekrar görünmesi, sonsuz özyinelemenin kesin işaretidir.

`StackOverflowError`, adının sonundaki `Error`'dan anlaşılacağı gibi bir `Exception` değil, ciddi bir **hata** (Error) sınıfıdır (Bölüm 12). Gerçek programlarda yakalanmaz; çözüm, kodu düzeltmektir.

> **Sık Yapılan Hata: Temel durum var ama ulaşılamıyor**
>
> `if (n == 0) return;` temel durumuyla `geriSay(n - 2)` çağrısı yapan bir metot düşün. `geriSay(6)` sorunsuz çalışır: 6, 4, 2, 0. Ama `geriSay(5)` çağrısında `n` sırasıyla 5, 3, 1, −1, −3... olur ve **hiçbir zaman 0'a eşit olmaz**: `StackOverflowError`. Aynı tuzak `faktoriyel(-3)` gibi beklenmeyen girdilerde de vardır. Güvenli yazım: Eşitlik yerine aralık kontrolü yap: `if (n <= 0)`.

15.10 Özyineleme mi, döngü mü?

Her özyinelemeli metot döngüyle de yazılabilir (ve tersi). Faktöriyel için `for` döngüsünde `sonuc *= i` yeterlidir. Fibonacci için de son iki terimi iki değişkende tutan bir döngü, hafızalamaya bile gerek kalmadan `fib(90)`'ı anında hesaplar.

Hangisini seçmeli? Kısaca: **Problem doğası gereği özyinelemeliyse** (ağaçlar, Hanoi, böl ve yönet algoritmaları, iç içe klasörleri gezmek) özyineleme; **basit tekrar** (sayma, toplama, diziyi gezme) varsa döngü.

Özellik	Özyineleme	Döngü (iteration)
Durma koşulu	Temel durum	Döngü koşulu
Kod	Kısa, matematiksel tanıma yakın	Bazen daha uzun
Bellek	Her çağrı yığında bir çerçeve kaplar	Birkaç değişken, sabit bellek
Hız	Her çağrının bir maliyeti var	Genellikle daha hızlı
Sınırı aşarsa	`StackOverflowError`	Sonsuz döngü (program donar)
Tipik kullanım	Ağaç/klasör gezme, Hanoi, merge sort, quicksort	Sayma, toplama, dizi gezme

> **Not: Kuyruk özyinelemesi**
>
> Özyinelemeli çağrı metodun **en son** işiyse (`return ebob(b, a % b);` gibi, çağrıdan sonra yapılacak başka iş yok) buna **kuyruk özyinelemesi** (tail recursion) denir. Bazı diller bunu otomatik olarak döngüye çevirip yığın kullanmaz. Java bu iyileştirmeyi **yapmaz**; derin özyinelemede Java'da yine `StackOverflowError` riski vardır.

15.11 Çağrıdan önce mi, sonra mı?

Özyineleme çıktı sorularının neredeyse hepsi tek bir fikre dayanır: **Özyinelemeli çağrıdan önceki** satırlar inişte çalışır, **sonraki** satırlar çıkışta, yani **ters sırada** çalışır.

OnceSonra.java

```
public class OnceSonra {
    static void once(int n) {
        if (n == 0) {
            return;
        }
        System.out.print(n + " ");      // çağrıdan ÖNCE
        once(n - 1);
    }

    static void sonra(int n) {
        if (n == 0) {
            return;
        }
        sonra(n - 1);
        System.out.print(n + " ");      // çağrıdan SONRA
    }

    static void ikiCagri(int n) {
        if (n == 0) {
            return;
        }
        ikiCagri(n - 1);
        System.out.print(n + " ");      // iki çağrının ARASINDA
        ikiCagri(n - 1);
    }

    public static void main(String[] args) {
        once(3);
        System.out.println("<- once(3)");
        sonra(3);
        System.out.println("<- sonra(3)");
        ikiCagri(3);
        System.out.println("<- ikiCagri(3)");
    }
}
```

Çıktı

```
3 2 1 <- once(3)
1 2 3 <- sonra(3)
1 2 1 3 1 2 1 <- ikiCagri(3)
```

- `once(3)`: Yazdırma inişte yapılır → `3 2 1`.
- `sonra(3)`: `sonra(2)`, `sonra(1)`'i bekler... İlk yazdırma en dipteki `sonra(1)`'de olur → `1 2 3`.
- `ikiCagri(3)`: Hanoi ile aynı yapı. `ikiCagri(2)` → `1 2 1`, sonra `3`, sonra yine `1 2 1`. Toplam $2^3 - 1 = 7$ sayı.

Sınavda Çıkar: Çıktı sorusu çözme yöntemi

1. Temel durumu bul ve en dipteki çağrının ne yaptığını yaz.
2. Çağrıdan **önceki** `print`'leri inerken, **sonraki** `print`'leri çıkarken yaz.
3. Birden fazla çağrı varsa küçük `n` için sonucu bul (`f(1)`, `f(2)`) ve büyük `n`'i onlar cinsinden kur: `ikiCagri(3)` = `ikiCagri(2)` + `"3 "` + `ikiCagri(2)`.
4. Değer döndüren metotlarda (`return n + f(n-1)`) çıkışta toplanan değerleri bir tabloda tut.

Bölüm Özeti – Bilmen Gerekenler

- **Özyineleme:** Metodun kendini daha küçük bir problemle çağırması.
- İki parça şart: **temel durum** (kendini çağırmadan cevap verir) ve **özyinelemeli durum** (problemi küçültüp kendini çağırır). Her çağrı temel duruma yaklaşmalı.
- Her çağrının **çağrı yığınında** kendi çerçevesi (kendi parametreleri) vardır. İnişte çağrılar birikir, çıkışta sonuçlar ters sırada hesaplanır (LIFO).
- Faktöriyel: `n * f(n-1)`; üs: `a * us(a, n-1)` veya yarıya bölerek; basamak toplamı: `n % 10 + f(n / 10)`; EBOB: `ebob(b, a % b)`; ikilik: `ikilik(n / 2) + n % 2`.
- Saf Fibonacci aynı değerleri tekrar tekrar hesapladığı için **üstel** sayıda çağrı yapar; hafızalama (dizi) veya döngü bunu çözer.
- String'de `substring`, dizide **indeks parametresi** ile problem küçültülür.
- Hanoi: `n` disk için $\mathbf{2^n - 1}$ hamle; iki özyinelemeli çağrı, arada bir hamle.
- Temel durum yoksa veya ulaşılamıyorsa → `StackOverflowError`.
- Çağrıdan **önceki** satırlar inişte (büyükten küçüğe), **sonraki** satırlar çıkışta (küçükten büyüğe) çalışır.
- Özyineleme kısa ve zarif ama bellek ve hız maliyeti var; basit tekrarlarda döngü tercih edilir.

Sorular

A) Çoktan Seçmeli

1 Özyinelemeli bir metodun **temel durumu** (base case) için aşağıdakilerden hangisi doğrudur?

(A) Metodun kendini en az iki kez çağırdığı kısımdır.
(B) Sadece `void` metotlarda bulunur.
(C) Metodun kendini çağırmadan doğrudan sonuç verdiği durumdur; özyinelemeyi durdurur.
(D) İsteğe bağlıdır; derleyici sonsuz çağrıyı zaten engeller.
(E) Her zaman metodun son satırında yazılmalıdır.

2 Aşağıdaki metot için `f(5)` çağrısı hangi değeri döndürür?

```
static int f(int n) {
    if (n <= 0) {
        return 0;
    }
    return n + f(n - 2);
}
```

(A) `15`
(B) `8`
(C) `StackOverflowError` oluşur
(D) `6`
(E) `9`

3 5 diskli Hanoi kuleleri problemi en az kaç hamlede çözülür?

(A) `31` (B) `32` (C) `25` (D) `10` (E) `63`

4 Temel durumu olmayan bir özyinelemeli metot çağrıldığında genellikle ne olur?

(A) Program derlenmez.
(B) Program hiç durmadan sonsuza kadar çalışır.
(C) `NullPointerException` fırlatılır.
(D) Yığın dolar ve `StackOverflowError` oluşur.
(E) Metot otomatik olarak `0` döndürür.

5 Aşağıdaki metot neyi hesaplar ve `gizem(4, 3)` çağrısının sonucu nedir?

```
static int gizem(int a, int b) {
    if (b == 0) {
        return 0;
    }
    return a + gizem(a, b - 1);
}
```

(A) `a` üzeri `b`; sonuç `64`

(B) `a × b`; sonuç `12`

(C) `a + b`; sonuç `7`

(D) `b` üzeri `a`; sonuç `81`

(E) `a × b`; sonuç `7`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru6 {
    static void yaz(int n) {
        if (n > 0) {
            System.out.print(n + " ");
            yaz(n - 2);
            System.out.print(n * 10 + " ");
        }
    }

    public static void main(String[] args) {
        yaz(5);
        System.out.println();
        yaz(4);
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru7 {
    static String m(String s) {
        if (s.length() <= 1) {
            return s;
        }
        return m(s.substring(2)) + s.charAt(0);
    }

    public static void main(String[] args) {
        System.out.println(m("JAVACI"));
        System.out.println(m("ABCDE"));
        System.out.println(m("XY").length());
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru8 {
    static int f(int n) {
        if (n <= 0) {
            return 1;
        }
        System.out.print(n + " ");
        return f(n - 1) + f(n - 2);
    }

    public static void main(String[] args) {
        int sonuc = f(3);
        System.out.println("| sonuç = " + sonuc);
    }
}
```

C) Kod Yazma

9 Bir String'de belirli bir karakterin kaç kez geçtiğini **döngü kullanmadan**, özyinelemeyle bulan `static int say(String s, char c)` metodunu yaz ve `KarakterSay` programında dene. Örnek:

```
muhendislik içinde 'i': 2
banana içinde 'a': 3
java içinde 'x': 0
```

10 **Hiç döngü kullanmadan** aşağıdaki iki deseni çizen `YildizUcgen` programını yaz. İpucu: Aynı satıra `k` tane yıldız basan özyinelemeli bir `yildizlar(int k)` metodu yaz; üçgenleri çizen metotlar da özyinelemeli olsun. Önce `ucgen(4)`, sonra `tersUcgen(3)` çağrılmıştır:

```
*
**
***
****
***
**
*
```

Çözümler

Cevap anahtarı

1 C	2 E	3 A	4 D	5 B

1 Cevap: C

Temel durum, cevabı doğrudan bilinen ve metodun kendini **çağırmadığı** durumdur; özyinelemeyi durduran budur. A özyinelemeli durumu tarif eder (üstelik iki çağrı şart değildir). Değer döndüren metotlarda da temel durum vardır (B yanlış). Derleyici sonsuz özyinelemeyi engellemez; program çalışırken `StackOverflowError` oluşur (D yanlış). Temel durumun yeri serbesttir; genellikle metodun **başına** yazılır (E yanlış).

2 Cevap: E

`f(5) = 5 + f(3)`, `f(3) = 3 + f(1)`, `f(1) = 1 + f(-1)`, `f(-1) = 0` (çünkü `-1 <= 0`). Çıkışta toplanır: `1 + 0 = 1`, `3 + 1 = 4`, `5 + 4 = 9`.

Çıktı
```
9
```

C tuzaktır: Temel durum `n == 0` olsaydı tek sayılarda 0'a hiç ulaşılamaz ve `StackOverflowError` oluşurdu. Ama burada `n <= 0` kullanıldığı için `-1` de temel durumdur. A, `n - 1` ile azaltılsaydı çıkacak sonuçtur (`5 + 4 + 3 + 2 + 1`).

3 Cevap: A

`n` disk için en az hamle sayısı $2^n - 1$'dir: $2^5 - 1 = 32 - 1 = 31$. B, `-1`'i unutanların; E ise 6 diskin cevabıdır ($2^6 - 1 = 63$).

4 Cevap: D

Her çağrı yığına yeni bir çerçeve ekler; yığın belleği sınırlı olduğu için bir noktada dolar ve JVM `StackOverflowError` fırlatır. Sonsuz döngüden farkı budur: Sonsuz döngü bellek tüketmeden sonsuza kadar dönebilir (B), ama sonsuz özyineleme kısa sürede çöker. Program sorunsuz derlenir (A yanlış).

5 Cevap: B

Metot `b` kez `a` toplar: `gizem(4, 3) = 4 + gizem(4, 2) = 4 + 4 + gizem(4, 1) = 4 + 4 + 4 + gizem(4, 0) = 4 + 4 + 4 + 0 = 12`. Yani tekrarlı toplama ile **çarpma** yapar: `a × b`.

Çıktı
```
12
```

`a` üzeri `b` için `a * gizem(a, b - 1)` ve temel durumda `return 1` gerekirdi (A). E'de işlem doğru ama sonuç yanlış.

6

Çağrıdan önceki `print` inişte, sonraki `print` çıkışta çalışır.

- `yaz(5)`: İnişte `5`, `3`, `1` yazılır; `yaz(-1)` hiçbir şey yapmaz. Çıkışta ters sırayla `1*10`, `3*10`, `5*10` → `10 30 50`.
- `yaz(4)`: İnişte `4 2`; `yaz(0)` hiçbir şey yapmaz (`0 > 0` yanlış). Çıkışta `20 40`.

```
Çıktı
5 3 1 10 30 50
4 2 20 40
```

7

Metot her adımda **ilk iki** karakteri atıp (`substring(2)`) kalanı işliyor ve ilk karakteri **sona** ekliyor. Sonuçta 0., 2., 4. ... indeksteki karakterler **ters sırada** yazılır.

- `m("JAVACI")` = `m("VACI") + 'J'` = `(m("CI") + 'V') + 'J'`. `m("CI")` = `m("") + 'C'` = `"C"`. Sonuç: `"CVJ"`.
- `m("ABCDE")` = `m("CDE") + 'A'` = `(m("E") + 'C') + 'A'`. `m("E")` tek harf: `"E"`. Sonuç: `"ECA"`.
- `m("XY")` = `m("") + 'X'` = `"X"`, uzunluğu 1.

```
Çıktı
CVJ
ECA
1
```

8

`print` her çağrının başında (çağrılardan önce) çalışır; `n <= 0` olan çağrılar bir şey yazmadan `1` döndürür.

Çağrı	Yazdırılan	Dönen değer
`f(3)`	`3`	`f(2) + f(1)`
`f(2)`	`2`	`f(1) + f(0)`
`f(1)`	`1`	`f(0) + f(-1) = 1 + 1 = 2`
`f(0)`	—	`1` → `f(2) = 2 + 1 = 3`
`f(1)` (yine)	`1`	`2` → `f(3) = 3 + 2 = 5`

Yazdırma sırası: `3 2 1 1`. Sonra `main` sonucu yazar. `f(1)`'in **iki kez** çağrıldığına dikkat: Fibonacci'deki tekrar eden hesaplar.

```
Çıktı
3 2 1 1 | sonuç = 5
```

9

Temel durum: Boş String'de karakter yoktur → `0`. Özyinelemeli durum: İlk karakter `c` ise 1, değilse 0; buna geri kalan String'deki (`substring(1)`) sayıyı ekle.

KarakterSay.java

```java
public class KarakterSay {
    static int say(String s, char c) {
        if (s.isEmpty()) {
            return 0;
        }
        int ilk = (s.charAt(0) == c) ? 1 : 0;
        return ilk + say(s.substring(1), c);
    }

    public static void main(String[] args) {
        System.out.println("muhendislik içinde 'i': " + say("muhendislik", 'i'));
        System.out.println("banana içinde 'a': " + say("banana", 'a'));
        System.out.println("java içinde 'x': " + say("java", 'x'));
    }
}
```

Çıktı

```
muhendislik içinde 'i': 2
banana içinde 'a': 3
java içinde 'x': 0
```

10

Üç metot gerekiyor:

- `yildizlar(k)`: Bir yıldız bas, sonra `yildizlar(k - 1)`. `k == 0` ise dur. (Döngünün yerini tutar.)
- `ucgen(n)`: Önce küçük üçgeni çiz (`ucgen(n - 1)`), **sonra** `n` yıldızlık satırı bas. Çağrıdan sonra yazdırdığımız için satırlar kısadan uzuna çıkar.
- `tersUcgen(n)`: Önce `n` yıldızlık satırı bas, **sonra** `tersUcgen(n - 1)`. Satırlar uzundan kısaya iner.

YildizUcgen.java

```java
public class YildizUcgen {
    static void yildizlar(int k) {
        if (k == 0) {
            return;
        }
        System.out.print("*");
        yildizlar(k - 1);
    }

    static void ucgen(int n) {
        if (n == 0) {
            return;
        }
        ucgen(n - 1);            // önce küçük üçgen
        yildizlar(n);            // sonra en uzun satır
        System.out.println();
    }

    static void tersUcgen(int n) {
        if (n == 0) {
            return;
        }
        yildizlar(n);            // önce en uzun satır
        System.out.println();
        tersUcgen(n - 1);        // sonra küçük üçgen
    }
```

```
    public static void main(String[] args) {
        ucgen(4);
        tersUcgen(3);
    }
}
```

Çıktı

```
*
**
***
****
***
**
*
```

BÖLÜM 16

Arama, Sıralama ve Big-O

Bu Bölümde Öğreneceklerin

- Doğrusal arama ve ikili arama algoritmalarını, ikili aramanın **önkoşulunu**
- İkili aramayı `low` / `mid` / `high` izleme tablosuyla adım adım çözmeyi
- Kabarcık, seçmeli ve eklemeli sıralamayı; **"k. geçişten sonra dizi nedir?"** sorularını
- Sıralama algoritmalarını en iyi/ortalama/en kötü durum, yer değiştirme sayısı ve kararlılık açısından karşılaştırmayı
- `Arrays.sort` ve `Collections.sort`'un arka planda ne kullandığını
- Big-O gösterimini ve döngülerden karmaşıklık çıkarmayı

16.1 Algoritma ve verimlilik

Algoritma (algorithm), bir problemi çözmek için izlenen, sonlu sayıda ve açıkça tanımlanmış adımlar dizisidir. Aynı problemin birçok algoritması olabilir ve hepsi doğru sonucu verse de **harcadıkları iş** çok farklı olabilir. Bölüm 15'te gördüğün iki Fibonacci bunun iyi bir örneğiydi: Biri 2,7 milyon çağrı yaptı, diğeri 59.

Bir algoritmanın hızını **saniye** ile ölçmek yanıltıcıdır; aynı program yeni bir bilgisayarda daha hızlı, eski bir telefonda daha yavaş çalışır. Bu yüzden algoritmaları **temel işlem sayısı** ile karşılaştırırız: kaç karşılaştırma, kaç yer değiştirme yapıldı? Daha da önemlisi: **Veri büyüdükçe bu sayı nasıl büyüyor?** Bu bölümün sonunda bu sorunun cevabını veren **Big-O** gösterimini göreceksin.

16.2 Doğrusal arama (linear search)

En basit arama yöntemi: Dizinin başından başla, her elemana tek tek bak; aradığını bulursan indeksini döndür, sona kadar bulamazsan `-1` döndür. **Doğrusal arama** (linear search, sıralı arama) dizinin sıralı olmasını gerektirmez.

DogrusalArama.java

```
public class DogrusalArama {
    static int karsilastirma = 0;

    static int ara(int[] dizi, int aranan) {
        for (int i = 0; i < dizi.length; i++) {
            karsilastirma++;
            if (dizi[i] == aranan) {
                return i;              // bulundu: indeksini döndür
            }
        }
        return -1;                     // bulunamadı
    }

    public static void main(String[] args) {
        int[] sayilar = {42, 7, 19, 73, 5, 61, 28};
        int[] arananlar = {42, 73, 28, 50};
        for (int x : arananlar) {
            karsilastirma = 0;
            int sonuc = ara(sayilar, x);
```

```
            System.out.println(x + " -> indeks " + sonuc
                    + " (" + karsilastirma + " karşılaştırma)");
        }
    }
}
```

Çıktı

```
42 -> indeks 0 (1 karşılaştırma)
73 -> indeks 3 (4 karşılaştırma)
28 -> indeks 6 (7 karşılaştırma)
50 -> indeks -1 (7 karşılaştırma)
```

Karşılaştırma sayısı aranan elemanın **nerede** olduğuna bağlı. Algoritmaları incelerken üç durumdan söz ederiz:

Durum	Ne zaman?	Karşılaştırma (n elemanlı dizi)
En iyi durum (best case)	Aranan ilk elemanda	1
Ortalama durum (average case)	Aranan rastgele bir yerde	yaklaşık n / 2
En kötü durum (worst case)	Aranan son elemanda ya da hiç yok	n

Sınavlarda ve Big-O analizinde genellikle **en kötü durum** konuşulur, çünkü algoritmanın verebileceği garantiyi o gösterir.

> ● **Sık Yapılan Hata: String ararken == kullanmak**
>
> `String[]` bir dizide arama yaparken `dizi[i] == aranan` yazarsan, içerikleri aynı olan iki farklı String nesnesi "eşit değil" sayılabilir. Bölüm 8'de gördüğün gibi String'ler (ve bütün nesneler) `equals` ile karşılaştırılır: `if (dizi[i].equals(aranan))`.

16.3 İkili arama (binary search)

Sözlükte "kelebek" kelimesini ararken ilk sayfadan başlayıp tek tek bakmazsın. Ortalarda bir yerden açarsın; "m" harfine denk geldiysen kelimenin **solda** kaldığını bilirsin ve sağ yarıyı tamamen unutursun. Sonra kalan kısmın ortasını açarsın... Her bakışta arama alanı **yarıya** iner.

İkili arama (binary search) bu fikrin algoritmasıdır. Tek ama çok önemli bir **önkoşulu** vardır: **Dizi sıralı olmalıdır.**

Üç indeks tutulur: aramanın yapıldığı aralığın başı `low`, sonu `high` ve ortası `mid`. Her adımda:

1. `mid` hesaplanır, `dizi[mid]` aranan değerle karşılaştırılır.
2. Eşitse → bulundu, `mid` döndürülür.
3. `dizi[mid]` küçükse → aranan sağdadır: `low = mid + 1`.
4. `dizi[mid]` büyükse → aranan soldadır: `high = mid - 1`.
5. `low > high` olursa aralık boşalmıştır → aranan dizide yoktur, `-1` döndürülür.

IkiliArama.java

```
import java.util.Arrays;

public class IkiliArama {
    static int ara(int[] dizi, int aranan) {
        int low = 0;
        int high = dizi.length - 1;
        int adim = 0;
        while (low <= high) {
            int mid = low + (high - low) / 2;
```

```
            adim++;
            System.out.printf("  %d. adım: low=%d high=%d mid=%d dizi[mid]=%d%n",
                    adim, low, high, mid, dizi[mid]);
            if (dizi[mid] == aranan) {
                return mid;
            } else if (dizi[mid] < aranan) {
                low = mid + 1;              // sağ yarıda ara
            } else {
                high = mid - 1;             // sol yarıda ara
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] d = {3, 8, 12, 17, 23, 31, 38, 45, 52, 60, 71};
        System.out.println("45 aranıyor:");
        System.out.println("Sonuç: " + ara(d, 45));
        System.out.println("10 aranıyor:");
        System.out.println("Sonuç: " + ara(d, 10));

        System.out.println("binarySearch(45) = " + Arrays.binarySearch(d, 45));
        System.out.println("binarySearch(10) = " + Arrays.binarySearch(d, 10));
    }
}
```

Çıktı

```
45 aranıyor:
  1. adım: low=0 high=10 mid=5 dizi[mid]=31
  2. adım: low=6 high=10 mid=8 dizi[mid]=52
  3. adım: low=6 high=7 mid=6 dizi[mid]=38
  4. adım: low=7 high=7 mid=7 dizi[mid]=45
Sonuç: 7
10 aranıyor:
  1. adım: low=0 high=10 mid=5 dizi[mid]=31
  2. adım: low=0 high=4 mid=2 dizi[mid]=12
  3. adım: low=0 high=1 mid=0 dizi[mid]=3
  4. adım: low=1 high=1 mid=1 dizi[mid]=8
Sonuç: -1
binarySearch(45) = 7
binarySearch(10) = -3
```

Sınavda bu çıktıyı bir **izleme tablosu** (trace table) olarak yazman istenir. 45 araması için:

Adım	low	high	mid	dizi[mid]	Karar
1	0	10	5	31	31 < 45 → sağa: `low = 6`
2	6	10	8	52	52 > 45 → sola: `high = 7`
3	6	7	6	38	38 < 45 → sağa: `low = 7`
4	7	7	7	45	eşit → **7** döndürülür

10 aramasında aralık her adımda daralır ve sonunda `low = 2`, `high = 1` olur. `low > high` olduğu için döngü biter ve `-1` döner.

Java'nın hazır `Arrays.binarySearch` metodu da aynı algoritmayı kullanır. Bulursa indeksi döndürür; bulamazsa **negatif** bir sayı döndürür: `-(ekleme noktası) - 1`. 10, dizide 2. indekse eklenmesi gereken bir değer olduğu için sonuç `-2 - 1 = -3` oldu. Senin için önemli olan kural: **Sonuç negatifse eleman yoktur.**

Neden bu kadar hızlı?

Her adımda aralık yarıya indiği için `n` elemanlı dizide en fazla yaklaşık **log₂ n** adım gerekir. "Kaç kere ikiye bölersem 1'e inerim?" sorusunun cevabı log₂ n'dir:

Dizi boyutu (n)	Doğrusal arama (en kötü)	İkili arama (en kötü)
10	10	4
1 000	1 000	10
1 000 000	1 000 000	20
1 000 000 000	1 000 000 000	30

Bir milyar elemanlı sıralı bir dizide aradığını en fazla 30 bakışta bulursun. Eleman sayısı **iki katına** çıktığında ikili aramaya sadece **bir adım** eklenir.

Sık Yapılan Hata: İkili aramayı sıralı olmayan dizide kullanmak

İkili arama, "ortadaki eleman büyükse aranan soldadır" varsayımına dayanır. Dizi sıralı değilse bu varsayım çöker ve algoritma **hata vermeden yanlış cevap** döndürür:

```
import java.util.Arrays;

public class SirasizArama {
    public static void main(String[] args) {
        int[] karisik = {31, 8, 52, 3, 45};
        // 8 aslında 1. indekste!
        System.out.println(Arrays.binarySearch(karisik, 8));
        Arrays.sort(karisik);
        System.out.println(Arrays.toString(karisik));
        System.out.println(Arrays.binarySearch(karisik, 8));
    }
}
```

Çıktı

```
-1
[3, 8, 31, 45, 52]
1
```

8 dizide olduğu halde ilk arama "yok" (negatif) dedi. Önce `Arrays.sort` ile sırala, sonra ara.

(low + high) / 2 taşma tuzağı

Ortayı bulmanın ilk akla gelen yolu `(low + high) / 2`'dir. Bölüm 15'teki özyinelemeli aramada biz de böyle yazdık ve küçük dizilerde sorun yoktur. Ama `low` ve `high` çok büyükse **toplamları** `int` sınırını (~2,1 milyar) aşar ve Bölüm 2'deki **taşma** oluşur:

OrtaNokta.java

```java
public class OrtaNokta {
    public static void main(String[] args) {
        int low = 1_500_000_000;
        int high = 2_000_000_000;
        int yanlis = (low + high) / 2;
        int dogru = low + (high - low) / 2;
        System.out.println("(low + high) / 2       = " + yanlis);
        System.out.println("low + (high - low) / 2 = " + dogru);
    }
}
```

Çıktı

```
(low + high) / 2       = -397483648
low + (high - low) / 2 = 1750000000
```

`low + (high - low) / 2` matematiksel olarak aynı sonucu verir ama hiçbir ara adımda `high`'dan büyük bir sayı oluşmaz. Bu hata Java'nın kendi kütüphanesindeki `binarySearch`'te bile yıllarca fark edilmeden kaldı ve 2006'da düzeltildi. Profesyonel kodda her zaman güvenli yazımı kullan.

> **Sınavda Çıkar: İkili arama soruları**
>
> - **Önkoşul:** dizi sıralı olmalı. "Hangi durumda ikili arama kullanılamaz?" → sıralı olmayan dizi.
> - **İzleme tablosu:** Her adımda `low`, `high`, `mid` değerlerini yaz. `mid` tam sayı bölmesiyle bulunur: `(0 + 9) / 2 = 4`.
> - **Adım sayısı:** `n` elemanda en fazla yaklaşık log₂ n: 1 000 → 10, 1 000 000 → 20.
> - **Bitiş:** `while (low <= high)`. `<` yazarsan tek elemanlı aralık hiç kontrol edilmez; `low = mid` (`+1` olmadan) yazarsan döngü sonsuza girebilir.

16.4 Kabarcık sıralaması (bubble sort)

Sıralama (sorting), elemanları belirli bir düzene (genellikle küçükten büyüğe) koymaktır. İkili aramanın önkoşulu olduğu için de çok önemlidir.

Kabarcık sıralaması (bubble sort) en basit sıralama yöntemidir: Dizinin başından sonuna kadar **komşu** elemanları karşılaştır; soldaki büyükse ikisinin **yerini değiştir** (swap). Bir **geçiş** (pass) bitince en büyük eleman, sudaki kabarcık gibi yükselip **en sona** yerleşmiş olur. Sonraki geçişte son elemana bakmaya gerek yoktur.

Bir de akıllı iyileştirme: Bir geçişte **hiç yer değiştirme olmadıysa** dizi zaten sıralıdır, kalan geçişlere gerek yoktur. Bunu bir `boolean` **bayrakla** (flag) anlarız:

KabarcikSiralama.java

```java
import java.util.Arrays;

public class KabarcikSiralama {
    static void sirala(int[] d) {
        int n = d.length;
        for (int gecis = 1; gecis < n; gecis++) {
            boolean degisti = false;
            for (int j = 0; j < n - gecis; j++) {   // sondakiler zaten yerinde
                if (d[j] > d[j + 1]) {              // komşular ters sıradaysa
                    int gecici = d[j];
                    d[j] = d[j + 1];
                    d[j + 1] = gecici;
                    degisti = true;
                }
```

```
            }
            System.out.println(gecis + ". geçiş: " + Arrays.toString(d));
            if (!degisti) {
                System.out.println("   hiç yer değişmedi -> dizi sıralı, dur");
                break;
            }
        }
    }

    public static void main(String[] args) {
        int[] a = {29, 10, 14, 37, 13, 5};
        System.out.println("Başlangıç: " + Arrays.toString(a));
        sirala(a);
        int[] b = {1, 2, 4, 3, 5, 6};
        System.out.println("Başlangıç: " + Arrays.toString(b));
        sirala(b);
    }
}
```

Çıktı

```
Başlangıç: [29, 10, 14, 37, 13, 5]
1. geçiş: [10, 14, 29, 13, 5, 37]
2. geçiş: [10, 14, 13, 5, 29, 37]
3. geçiş: [10, 13, 5, 14, 29, 37]
4. geçiş: [10, 5, 13, 14, 29, 37]
5. geçiş: [5, 10, 13, 14, 29, 37]
Başlangıç: [1, 2, 4, 3, 5, 6]
1. geçiş: [1, 2, 3, 4, 5, 6]
2. geçiş: [1, 2, 3, 4, 5, 6]
   hiç yer değişmedi -> dizi sıralı, dur
```

İlk dizide her geçişin sonunda **sağdaki** sıralı bölge bir eleman büyüdü: 1. geçişten sonra `37`, 2. geçişten sonra `29` yerine oturdu. En küçük eleman `5` ise her geçişte sadece **bir adım** sola gidebildiği için 5 geçiş gerekti. İkinci dizi neredeyse sıralıydı: 1. geçişte tek bir yer değiştirme yapıldı, 2. geçişte hiç yapılmadı ve bayrak sayesinde algoritma durdu.

`n` elemanlı dizide en fazla `n - 1` geçiş yapılır ve toplam karşılaştırma sayısı en fazla `(n-1) + (n-2) + ... + 1 = n(n-1)/2` olur.

> **Sık Yapılan Hata: Yer değiştirmede geçici değişkeni unutmak**
>
> `d[j] = d[j + 1]; d[j + 1] = d[j];` yazarsan iki değer yer değiştirmez; ilk satırda `d[j]`'nin eski değeri kaybolur ve iki eleman da aynı olur. Değiş tokuş için **üçüncü bir değişken** şart: `int gecici = d[j]; d[j] = d[j + 1]; d[j + 1] = gecici;`. Bir diğer klasik hata, iç döngüyü `j < n` diye yazmaktır: `j = n - 1` olduğunda `d[j + 1]` dizinin dışına çıkar ve `ArrayIndexOutOfBoundsException` oluşur.

16.5 Seçmeli sıralama (selection sort)

Seçmeli sıralama (selection sort) şöyle düşünür: "Dizideki **en küçük** elemanı bul, **en başa** koy. Sonra geri kalanların en küçüğünü bul, **ikinci** sıraya koy..." Dizi solda **sıralı**, sağda **sıralanmamış** iki bölgeye ayrılır; her geçişte sıralanmamış bölgenin en küçüğü seçilip sıralı bölgenin sonuna eklenir.

SecmeliSiralama.java

```
import java.util.Arrays;

public class SecmeliSiralama {
    static void sirala(int[] d) {
        int n = d.length;
```

```
        for (int i = 0; i < n - 1; i++) {
            int enKucuk = i;                        // en küçüğün indeksi
            for (int j = i + 1; j < n; j++) {       // sıralanmamış kısmı tara
                if (d[j] < d[enKucuk]) {
                    enKucuk = j;
                }
            }
            int gecici = d[i];                      // en küçüğü i. yere koy
            d[i] = d[enKucuk];
            d[enKucuk] = gecici;
            System.out.println((i + 1) + ". geçiş: " + Arrays.toString(d));
        }
    }

    public static void main(String[] args) {
        int[] a = {29, 10, 14, 37, 13, 5};
        System.out.println("Başlangıç: " + Arrays.toString(a));
        sirala(a);
    }
}
```

Çıktı

```
Başlangıç: [29, 10, 14, 37, 13, 5]
1. geçiş: [5, 10, 14, 37, 13, 29]
2. geçiş: [5, 10, 14, 37, 13, 29]
3. geçiş: [5, 10, 13, 37, 14, 29]
4. geçiş: [5, 10, 13, 14, 37, 29]
5. geçiş: [5, 10, 13, 14, 29, 37]
```

- 1. geçiş: En küçük `5`, baştaki `29` ile yer değiştirdi.
- 1. geçiş: Kalanların en küçüğü `10` zaten 1. indeksteydi; dizi değişmedi (eleman kendisiyle yer değiştirdi).
- Her geçişten sonra **soldaki** bölge kesin yerine oturmuş en küçük elemanlardan oluşur.

Seçmeli sıralama dizi baştan sıralı olsa bile **her zaman** aynı sayıda karşılaştırma yapar: `n(n−1)/2`. Erken çıkma imkânı yoktur. Buna karşılık **yer değiştirme** sayısı en fazla `n − 1`'dir; yer değiştirmenin pahalı olduğu durumlarda bu bir avantajdır.

16.6 Eklemeli sıralama (insertion sort)

Elindeki iskambil kartlarını nasıl sıralarsın? Kartları soldan sağa tek tek alır, her yeni kartı soldaki **zaten sıralı** kartların arasında doğru yere **yerleştirirsin**. **Eklemeli sıralama** (insertion sort) tam olarak budur:

1. `i`. elemanı bir kenara al (**anahtar**, key).
2. Soldaki sıralı bölgede anahtardan **büyük** olanları birer adım **sağa kaydır**.
3. Açılan boşluğa anahtarı yerleştir.

EklemeliSiralama.java

```
import java.util.Arrays;

public class EklemeliSiralama {
    static void sirala(int[] d) {
        for (int i = 1; i < d.length; i++) {
            int anahtar = d[i];                     // yerleştirilecek eleman
            int j = i - 1;
            while (j >= 0 && d[j] > anahtar) {      // büyükleri sağa kaydır
                d[j + 1] = d[j];
                j--;
```

```
            }
            d[j + 1] = anahtar;                    // boşluğa yerleştir
            System.out.println(i + ". geçiş: " + Arrays.toString(d));
        }
    }

    public static void main(String[] args) {
        int[] a = {29, 10, 14, 37, 13, 5};
        System.out.println("Başlangıç: " + Arrays.toString(a));
        sirala(a);
    }
}
```

Çıktı

```
Başlangıç: [29, 10, 14, 37, 13, 5]
1. geçiş: [10, 29, 14, 37, 13, 5]
2. geçiş: [10, 14, 29, 37, 13, 5]
3. geçiş: [10, 14, 29, 37, 13, 5]
4. geçiş: [10, 13, 14, 29, 37, 5]
5. geçiş: [5, 10, 13, 14, 29, 37]
```

- 1. geçiş: `10` alındı, `29` sağa kaydı, `10` başa yerleşti.
- 1. geçiş: `37` soldakilerin hepsinden büyük; hiçbir şey kaymadı.
- 1. geçiş: `5` en küçük; soldaki beş elemanın hepsi kaydı.

Her geçişten sonra **soldaki** `i + 1` eleman **kendi aralarında** sıralıdır; ama seçmeli sıralamadan farklı olarak **son yerlerinde olmayabilirler** (sonradan gelen `5`, hepsini kaydırdı). Dizi zaten sıralıysa `while` döngüsü hiç dönmez ve algoritma sadece `n − 1` karşılaştırma yapar. Bu yüzden eklemeli sıralama **neredeyse sıralı** dizilerde çok hızlıdır.

> **Sınavda Çıkar: "k. geçişten sonra dizi nedir?" sorularında algoritmayı tanı**
>
> Aynı dizi `{29, 10, 14, 37, 13, 5}` için 2. geçişten sonraki durumlar:
>
> | Algoritma | 2. geçişten sonra | İpucu |
> |---|---|---|
> | Kabarcık | `[10, 14, 13, 5, 29, 37]` | **Sağda** en büyük k eleman, son yerlerinde |
> | Seçmeli | `[5, 10, 14, 37, 13, 29]` | **Solda** en küçük k eleman, son yerlerinde |
> | Eklemeli | `[10, 14, 29, 37, 13, 5]` | **Soldaki** k+1 eleman kendi arasında sıralı, sağ taraf **hiç dokunulmamış** |
>
> Soruyu çözerken diziyi kâğıda yaz ve her geçişi tek tek uygula. Kabarcıkta geçiş içinde birden çok yer değiştirme olabileceğini unutma.

16.7 Algoritmaları karşılaştırma

Algoritma	En iyi	Ortalama	En kötü	Yer değiştirme (en kötü)	Kararlı mı?	Ek bellek
Kabarcık (bayraklı)	O(n)	O(n²)	O(n²)	O(n²)	Evet	O(1)
Seçmeli	O(n²)	O(n²)	O(n²)	**n − 1**	**Hayır**	O(1)
Eklemeli	O(n)	O(n²)	O(n²)	O(n²) kaydırma	Evet	O(1)
Merge sort*	O(n log n)	O(n log n)	O(n log n)	—	Evet	O(n)
Quicksort*	O(n log n)	O(n log n)	O(n²)	—	Hayır	O(log n)

* Bu kitapta kodlanmıyor; ikinci sınıftaki *Veri Yapıları* dersinde göreceksin. Big-O gösterimini birazdan açıklayacağız; şimdilik O(n²)'nin "n büyüdükçe karesiyle büyüyen iş" anlamına geldiğini bil.

Kararlılık (stability)

Bir sıralama algoritması, **eşit** anahtarlı elemanların başlangıçtaki **göreli sırasını koruyorsa kararlıdır** (stable). Önemi şuradan gelir: Öğrenci listesi zaten isme göre sıralıysa ve sen onu nota göre sıralarsan, aynı notu alanların isim sırası bozulmasın istersin.

Kararlilik.java

```
import java.util.Arrays;

record Ogrenci(String ad, int not) {
    @Override
    public String toString() {
        return ad + "(" + not + ")";
    }
}

public class Kararlilik {
    static void secmeli(Ogrenci[] d) {
        for (int i = 0; i < d.length - 1; i++) {
            int min = i;
            for (int j = i + 1; j < d.length; j++) {
                if (d[j].not() < d[min].not()) {
                    min = j;
                }
            }
            Ogrenci gecici = d[i];
            d[i] = d[min];
            d[min] = gecici;
        }
    }

    static void eklemeli(Ogrenci[] d) {
        for (int i = 1; i < d.length; i++) {
            Ogrenci anahtar = d[i];
            int j = i - 1;
            while (j >= 0 && d[j].not() > anahtar.not()) {
                d[j + 1] = d[j];
                j--;
            }
            d[j + 1] = anahtar;
        }
    }

    public static void main(String[] args) {
        Ogrenci[] liste = {new Ogrenci("Ali", 85), new Ogrenci("Ece", 85),
                new Ogrenci("Can", 70)};
        System.out.println("Başta   : " + Arrays.toString(liste));
        Ogrenci[] a = liste.clone();
        secmeli(a);
        System.out.println("Seçmeli : " + Arrays.toString(a));
        Ogrenci[] b = liste.clone();
        eklemeli(b);
        System.out.println("Eklemeli: " + Arrays.toString(b));
    }
}
```

```
Çıktı
Başta   : [Ali(85), Ece(85), Can(70)]
Seçmeli : [Can(70), Ece(85), Ali(85)]
Eklemeli: [Can(70), Ali(85), Ece(85)]
```

Başta Ali, Ece'den önceydi. Seçmeli sıralama ilk geçişte `Can`'ı başa almak için `Ali` ile **uzak bir yer değiştirme** yaptı ve Ali'yi Ece'nin arkasına attı: Sıra bozuldu, **kararsız**. Eklemeli sıralama sadece kesin büyük (`>`) olanları kaydırdığı için eşitlerin sırasına dokunmadı: **kararlı**.

16.8 Java'nın hazır sıralama metotları

Gerçek programlarda sıralama algoritmasını kendin yazmazsın; Java'nın hazır metotlarını kullanırsın:

Metot	Ne sıralar?	Arka planda	Kararlı mı?
`Arrays.sort(int[])` (ve diğer ilkel tipler)	İlkel tip dizisi	Dual-Pivot Quicksort	Önemsiz (ilkel değerler ayırt edilemez)
`Arrays.sort(Object[])`	Nesne dizisi (`String[]` vb.)	TimSort	Evet
`Collections.sort(list)`, `list.sort(...)`	`List` (Bölüm 13)	TimSort	Evet

TimSort, merge sort ile eklemeli sıralamanın birleşimidir: Küçük parçaları eklemeli sıralama ile sıralar (küçük ve neredeyse sıralı veride en hızlısı odur), sonra parçaları birleştirir. Hepsi O(n log n)'dir.

HazirSiralama.java

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public class HazirSiralama {
    public static void main(String[] args) {
        int[] sayilar = {29, 10, 14, 37, 13, 5};
        Arrays.sort(sayilar);
        System.out.println(Arrays.toString(sayilar));

        String[] isimler = {"Zeynep", "ali", "Can", "Ece"};
        Arrays.sort(isimler);                    // büyük harfler önce gelir!
        System.out.println(Arrays.toString(isimler));

        List<String> liste = new ArrayList<>();
        Collections.addAll(liste, "Zeynep", "Ali", "Can", "Ece");
        Collections.sort(liste);
        System.out.println(liste);
        liste.sort(Collections.reverseOrder());  // büyükten küçüğe
        System.out.println(liste);
        // uzunluğa göre (çıkarma değil Integer.compare, Bölüm 13)
        liste.sort((x, y) -> Integer.compare(x.length(), y.length()));
        System.out.println(liste);
    }
}
```

Çıktı
```
[5, 10, 13, 14, 29, 37]
[Can, Ece, Zeynep, ali]
[Ali, Can, Ece, Zeynep]
[Zeynep, Ece, Can, Ali]
[Ece, Can, Ali, Zeynep]
```

İki ayrıntıya dikkat et:

- `"ali"` küçük harfle başladığı için en sona gitti: String'ler `compareTo` ile, yani Unicode değerlerine göre karşılaştırılır ve bütün büyük harfler küçük harflerden önce gelir (Bölüm 8). Büyük/küçük harf farkını önemsememek için `String.CASE_INSENSITIVE_ORDER` kullanılabilir.
- Son satırda `Ece`, `Can` ve `Ali` aynı uzunlukta (3 harf). Liste bir önceki satırdaki **ters alfabetik** sırasını korudu: TimSort kararlı olduğu için eşitlerin sırası bozulmadı.

16.9 Big-O gösterimi

Bir algoritmanın verimliliğini anlatmanın standart dili **Big-O gösterimidir** (Big-O notation). Big-O, "girdi boyutu `n` büyüdükçe yapılan iş **nasıl büyür**?" sorusunu cevaplar. Kesin işlem sayısıyla değil, **büyüme hızıyla** ilgilenir. Bu yüzden iki kural uygulanır:

1. **Sabit çarpanlar atılır:** `3n` ile `n` aynı sınıftadır → O(n). `n / 2` → O(n). `500` → O(1).
2. **Küçük terimler atılır:** `n² + 5n + 2` ifadesinde `n` büyüdükçe `n²` her şeyi ezer → O(n²).

Örneğin kabarcık sıralamasının karşılaştırma sayısı `n(n−1)/2 = n²/2 − n/2`'dir. Sabiti (`1/2`) ve küçük terimi (`n/2`) atınca **O(n²)** kalır.

Big-O	Adı	Örnek
O(1)	Sabit (constant)	Dizinin i. elemanına erişmek, `dizi.length`
O(log n)	Logaritmik	İkili arama
O(n)	Doğrusal (linear)	Doğrusal arama, dizinin toplamı, en büyüğünü bulmak
O(n log n)	Doğrusal-logaritmik	Merge sort, `Arrays.sort`
O(n²)	Karesel (quadratic)	Kabarcık, seçmeli, eklemeli sıralama; iç içe iki döngü
O(2ⁿ)	Üstel (exponential)	Saf özyinelemeli Fibonacci, Hanoi kuleleri

Farkın ne kadar büyük olduğunu görmek için her sınıfın yaklaşık işlem sayısına bakalım:

n	O(1)	O(log n)	O(n)	O(n log n)	O(n²)	O(2ⁿ)
10	1	≈ 3	10	≈ 33	100	1 024
100	1	≈ 7	100	≈ 664	10 000	$\approx 1{,}3 \times 10^{30}$
1 000	1	≈ 10	1 000	≈ 9 966	10^6	$\approx 1{,}1 \times 10^{301}$
10^6	1	≈ 20	10^6	$\approx 2 \times 10^7$	10^{12}	$\approx 10^{301030}$

Bilgisayarın saniyede yaklaşık 10^9 basit işlem yaptığını varsayalım. Bir milyon elemanı O(n log n) bir algoritmayla sıralamak yaklaşık **0,02 saniye** sürer; O(n²) bir algoritmayla (kabarcık) 10^{12} işlem, yani yaklaşık **17 dakika**. O(2ⁿ) ise `n = 100` için bile evrenin yaşından uzun sürer. Algoritma seçimi, daha hızlı bilgisayar almaktan her zaman daha etkilidir.

16.10 Döngülerden karmaşıklık çıkarma

Sınavda “Bu kodun karmaşıklığı nedir?” sorusu kodun **döngü yapısına** bakarak çözülür. Aşağıdaki program en sık görülen kalıpları içeriyor; her kalıbın gövdesi bir sayaç artırıyor ve farklı `n` değerleri için gövdenin kaç kez çalıştığını yazdırıyoruz:

KarmasiklikSayac.java

```
public class KarmasiklikSayac {
    public static void main(String[] args) {
        System.out.printf("%6s%8s%7s%9s%9s%9s%10s%n",
                "n", "yarıya", "tek", "ardışık", "nLogN", "üçgen", "içİçe");
        int[] boyutlar = {10, 100, 1000};
        for (int n : boyutlar) {
            long yariya = 0, tek = 0, ardisik = 0;
            long nLogN = 0, ucgen = 0, icIce = 0;

            // O(log n): sayaç her adımda ikiye katlanıyor
            for (int i = 1; i < n; i *= 2) {
                yariya++;
            }
            // O(n): tek döngü
            for (int i = 0; i < n; i++) {
                tek++;
            }
            // O(n): art arda iki bağımsız döngü -> n + n = 2n
            for (int i = 0; i < n; i++) {
                ardisik++;
            }
            for (int j = 0; j < n; j++) {
                ardisik++;
            }
            // O(n log n): dışta n, içte log n
            for (int i = 0; i < n; i++) {
                for (int j = 1; j < n; j *= 2) {
                    nLogN++;
                }
            }
            // O(n^2): iç döngü i+1'den başlıyor -> n(n-1)/2
            for (int i = 0; i < n; i++) {
                for (int j = i + 1; j < n; j++) {
                    ucgen++;
                }
            }
            // O(n^2): iç içe iki tam döngü -> n * n
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    icIce++;
                }
            }
            System.out.printf("%6d%8d%7d%9d%9d%9d%10d%n",
                    n, yariya, tek, ardisik, nLogN, ucgen, icIce);
        }
    }
}
```

Çıktı

```
     n  yarıya    tek  ardışık    nLogN    üçgen     içİçe
    10       4     10       20       40       45       100
   100       7    100      200      700     4950     10000
  1000      10   1000     2000    10000   499500   1000000
```

Tabloyu sütun sütun oku: `n` 10 kat artınca `tek` ve `ardisik` 10 kat, `ucgen` ve `icIce` yaklaşık **100 kat** arttı; `yariya` ise sadece birkaç adım arttı. `ucgen` sütunu (`45`, `4 950`, `499 500`) seçmeli sıralamanın karşılaştırma sayısıyla aynı: O kodda da iç döngü `j = i + 1`'den başlıyordu.

Kalıp	Örnek	Karmaşıklık
Sabit sayıda tekrar	`for (i = 0; i < 100; i++)`	O(1)
Tek döngü, `i++`	`for (i = 0; i < n; i++)`	O(n)
Adım büyük ama sabit	`for (i = 0; i < n; i += 4)`	O(n) (n/4 kez)
İkiye katlama / yarıya bölme	`for (i = 1; i < n; i *= 2)`, `for (i = n; i > 0; i /= 2)`	O(log n)
Art arda iki döngü	`for (n) {...}` sonra `for (n) {...}`	O(n + n) = O(n)
Art arda farklı döngüler	`for (n) {...}` sonra `for (n) for (n) {...}`	O(n + n²) = O(n²)
İç içe döngü	`for (n) for (n)`	O(n²)
İç içe, iç döngü sabit	`for (n) for (j < 5)`	O(5n) = O(n)
İç içe, iç döngü yarıya	`for (n) for (j *= 2)`	O(n log n)

● **Sınavda Çıkar: Karmaşıklık sorusu çözme kuralları**

- **İç içe döngüler → çarp:** dış döngü kaç kez × iç döngü kaç kez.
- **Art arda döngüler → topla, en büyüğü al:** O(n) + O(n²) = O(n²).
- **Sayaç ikiye katlanıyor ya da yarıya iniyorsa → log n.**
- **Döngü sınırı `n`'e bağlı değilse → O(1).** Tuzak: `for (j = 0; j < 5; j++)` iç döngüsü kodu O(n²) yapmaz.
- İç döngü dış döngünün değişkenine bağlıysa (`j = i`, `j < i`) toplam `n(n±1)/2` olur → yine **O(n²)**.

● **İpucu**

Big-O'yu ezberlemek yerine küçük bir `n` (ör. 8 veya 16) seçip döngünün kaç kez döndüğünü **say**. Sonra `n`'i iki katına çıkarıp tekrar düşün: İş de iki katına çıkıyorsa O(n), dört katına çıkıyorsa O(n²), sadece bir artıyorsa O(log n).

16.11 Deneysel karşılaştırma

Teoriyi deneyle doğrulayalım. Üç sıralama algoritmasını 1 000 elemanlı üç farklı dizi üzerinde çalıştırıp **karşılaştırma** ve **yer değiştirme** (eklemelide kaydırma) sayılarını sayıyoruz. Dizilerden biri zaten sıralı, biri ters sıralı, biri sabit tohumlu (`new Random(42)`) karışık:

DeneyselKarsilastirma.java

```
import java.util.Random;

public class DeneyselKarsilastirma {
    static long karsilastirma, tasima;

    static void kabarcik(int[] d) {
        for (int gecis = 1; gecis < d.length; gecis++) {
            boolean degisti = false;
            for (int j = 0; j < d.length - gecis; j++) {
                karsilastirma++;
                if (d[j] > d[j + 1]) {
                    int t = d[j];
```

```
                    d[j] = d[j + 1];
                    d[j + 1] = t;
                    tasima++;
                    degisti = true;
                }
            }
            if (!degisti) {
                break;
            }
        }
    }

    static void secmeli(int[] d) {
        for (int i = 0; i < d.length - 1; i++) {
            int min = i;
            for (int j = i + 1; j < d.length; j++) {
                karsilastirma++;
                if (d[j] < d[min]) {
                    min = j;
                }
            }
            if (min != i) {
                int t = d[i];
                d[i] = d[min];
                d[min] = t;
                tasima++;
            }
        }
    }

    static void eklemeli(int[] d) {
        for (int i = 1; i < d.length; i++) {
            int anahtar = d[i];
            int j = i - 1;
            while (j >= 0) {
                karsilastirma++;
                if (d[j] <= anahtar) {
                    break;
                }
                d[j + 1] = d[j];
                tasima++;
                j--;
            }
            d[j + 1] = anahtar;
        }
    }

    static void dene(String ad, int[] kaynak) {
        String[] algoritmalar = {"Kabarcık", "Seçmeli", "Eklemeli"};
        for (String alg : algoritmalar) {
            int[] d = kaynak.clone();      // her algoritma aynı diziyle başlasın
            karsilastirma = 0;
            tasima = 0;
            switch (alg) {
                case "Kabarcık" -> kabarcik(d);
                case "Seçmeli" -> secmeli(d);
                default -> eklemeli(d);
            }
            System.out.printf("%-9s %-9s %,14d %,14d%n",
                    ad, alg, karsilastirma, tasima);
        }
    }

    public static void main(String[] args) {
        int n = 1000;
```

```
        int[] sirali = new int[n], ters = new int[n], karisik = new int[n];
        Random r = new Random(42);
        for (int i = 0; i < n; i++) {
            sirali[i] = i;
            ters[i] = n - i;
            karisik[i] = r.nextInt(10_000);
        }
        System.out.printf("%-9s %-9s %14s %14s%n", "Dizi", "Algoritma",
                "Karşılaştırma", "Yer değişt.");
        dene("Sıralı", sirali);
        dene("Ters", ters);
        dene("Karışık", karisik);
    }
}
```

Çıktı

```
Dizi      Algoritma  Karşılaştırma    Yer değişt.
Sıralı    Kabarcık             999              0
Sıralı    Seçmeli          499,500              0
Sıralı    Eklemeli             999              0
Ters      Kabarcık         499,500        499,500
Ters      Seçmeli          499,500            500
Ters      Eklemeli         499,500        499,500
Karışık   Kabarcık         499,122        249,040
Karışık   Seçmeli          499,500            993
Karışık   Eklemeli         250,032        249,040
```

Sonuçlar karşılaştırma tablosunu birebir doğruluyor:

- **Sıralı dizide** bayraklı kabarcık ve eklemeli sıralama sadece `n - 1 = 999` karşılaştırma yaptı (en iyi durum, O(n)). Seçmeli sıralama ise dizi sıralı olduğu halde yine `n(n-1)/2 = 499 500` karşılaştırma yaptı.
- **Ters dizide** kabarcık ve eklemeli en kötü durumlarına düştü: 499 500 karşılaştırma ve 499 500 yer değiştirme. Seçmeli sıralama ise sadece **500** yer değiştirme yaptı (her yer değiştirme iki elemanı birden yerine oturttu).
- **Karışık dizide** seçmelinin karşılaştırma sayısı yine aynı; eklemeli, kabarcığın yaklaşık yarısı kadar karşılaştırma yaptı.
- Kabarcığın yer değiştirme sayısı ile eklemelinin kaydırma sayısı her dizide **aynı**. Tesadüf değil: İkisi de dizideki **ters sıralı çiftlerin** (inversion: solda olup sağındakinden büyük eleman çiftleri) sayısına eşittir.

> **Not: Neden süre ölçmedik?**
>
> `System.nanoTime()` ile süre ölçmek mümkün, ama sonuçlar bilgisayara, o an çalışan diğer programlara ve JVM'in ısınmasına göre her çalıştırmada değişir. Karşılaştırma sayısı ise her bilgisayarda **aynıdır** ve algoritmanın kendisini ölçer. Kendi bilgisayarında süre ölçmeyi deneyebilirsin; büyüme oranının (n iki katına çıkınca süre ~4 katına çıkar) yine $O(n^2)$'ye uyduğunu göreceksin.

Bölüm Özeti – Bilmen Gerekenler

- **Doğrusal arama:** Sırayla bak; sıralı olmayan dizide de çalışır. En iyi 1, en kötü n karşılaştırma → O(n).
- **İkili arama:** Dizi **sıralı olmalı**. Ortaya bak, yarıyı at. `low`, `high`, `mid`; döngü `while (low <= high)`; en fazla ~$\log_2 n$ adım → O(log n). Ortayı `low + (high - low) / 2` ile bul (taşma olmaz).
- **Kabarcık:** Komşuları karşılaştır, ters ise yer değiştir; her geçişte en büyük **sona** gider. Bayrakla sıralı dizide O(n), genelde $O(n^2)$. Kararlı.
- **Seçmeli:** En küçüğü bul, **başa** koy. Her durumda $n(n-1)/2$ karşılaştırma, en fazla $\mathbf{n - 1}$ yer değiştirme. Kararsız.
- **Eklemeli:** Elemanı sol taraftaki sıralı bölgede doğru yere yerleştir. Sıralı/neredeyse sıralı dizide çok hızlı (O(n)), genelde $O(n^2)$. Kararlı.
- **Kararlı sıralama**, eşit anahtarlı elemanların göreli sırasını korur.
- `Arrays.sort(int[])` → Dual-Pivot Quicksort; nesneler ve `Collections.sort` → TimSort (kararlı). Hepsi O(n log n).
- **Big-O:** Büyüme hızını gösterir; sabitler ve küçük terimler atılır. Sıra: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.
- İç içe döngüler **çarpılır**, art arda döngüler **toplanır** (büyük olan kalır), yarıya bölme/ikiye katlama **log n** verir.

Sorular

A) Çoktan Seçmeli

1 İkili aramanın (binary search) doğru çalışması için dizinin hangi özelliği taşıması gerekir?

(A) Elemanları birbirinden farklı (tekrarsız) olmalıdır.
(B) Sıralı olmalıdır.
(C) Eleman sayısı 2'nin kuvveti olmalıdır.
(D) Eleman sayısı tek olmalıdır.
(E) Sadece pozitif sayılar içermelidir.

2 `{29, 10, 14, 37, 13}` dizisi küçükten büyüğe **seçmeli sıralama** (selection sort) ile sıralanıyor. **2. geçişten sonra** dizi nasıl görünür?

(A) `[10, 14, 13, 29, 37]`
(B) `[10, 14, 29, 37, 13]`
(C) `[10, 13, 14, 29, 37]`
(D) `[10, 13, 14, 37, 29]`
(E) `[10, 29, 14, 37, 13]`

3 Aşağıdaki kod parçasının zaman karmaşıklığı nedir?

```
for (int i = 1; i < n; i *= 2) {
    for (int j = 0; j < n; j++) {
        toplam++;
    }
}
```

(A) $O(n)$ (B) $O(n^2)$ (C) $O(\log n)$ (D) $O(2^n)$ (E) $O(n \log n)$

4 1 000 000 elemanlı **sıralı** bir dizide ikili arama, aranan değeri en fazla yaklaşık kaç adımda (ortadaki elemana kaç kez bakarak) bulur ya da bulunamadığına karar verir?

(A) 20 (B) 1 000 000 (C) 500 000 (D) 1 000 (E) 2

5 Aşağıdakilerden hangisi **seçmeli sıralama** (selection sort) için doğrudur?

(A) Dizi zaten sıralıysa O(n) sürede biter.

(B) Kararlı (stable) bir algoritmadır.

(C) `n` elemanlı dizide en fazla `n - 1` yer değiştirme yapar.

(D) Her geçişte en büyük eleman komşularıyla yer değiştirerek sona taşınır.

(E) Sıralama için `n` elemanlık ek bir yardımcı dizi kullanır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru6 {
    static int ara(int[] d, int x) {
        int low = 0, high = d.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            System.out.print(mid + " ");
            if (d[mid] == x) {
                return mid;
            } else if (d[mid] < x) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] d = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
        System.out.println("-> " + ara(d, 23));
        System.out.println("-> " + ara(d, 50));
        System.out.println("-> " + ara(d, 2));
    }
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
import java.util.Arrays;

public class Soru7 {
    public static void main(String[] args) {
        int[] d = {4, 3, 5, 1, 2};
        int takas = 0, gecis = 0;
        boolean degisti = true;
        while (degisti) {
            degisti = false;
            gecis++;
            for (int j = 0; j < d.length - 1; j++) {
                if (d[j] > d[j + 1]) {
                    int t = d[j];
                    d[j] = d[j + 1];
                    d[j + 1] = t;
                    takas++;
                    degisti = true;
                }
            }
            System.out.println(gecis + ": " + Arrays.toString(d));
        }
        System.out.println("geçiş=" + gecis + " takas=" + takas);
    }
}
```

8 Aşağıdaki program ekrana ne yazdırır? Her döngü grubunun Big-O karmaşıklığını da belirt.

```
public class Soru8 {
    public static void main(String[] args) {
        int n = 16;
        int a = 0, b = 0, c = 0, d = 0;
        for (int i = n; i > 0; i /= 2) {
            a++;
        }
        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                b++;
            }
        }
        for (int i = 0; i < n; i += 4) {
            c++;
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < 3; j++) {
                d++;
            }
        }
        System.out.println(a + " " + b + " " + c + " " + d);
    }
}
```

C) Kod Yazma

9 String dizisini **alfabetik** sıraya koyan **eklemeli sıralama** metodunu yaz (`compareTo` kullan) ve `KelimeSirala` programında her geçişten sonra diziyi yazdır. `{"elma", "armut", "kiraz", "erik", "muz"}` için çıktı:

```
1. geçiş: [armut, elma, kiraz, erik, muz]
2. geçiş: [armut, elma, kiraz, erik, muz]
3. geçiş: [armut, elma, erik, kiraz, muz]
4. geçiş: [armut, elma, erik, kiraz, muz]
```

10 Sıralı bir dizide aynı değer birden çok kez geçebilir. Aranan değerin **ilk** geçtiği indeksi O(log n) sürede bulan `static int ilkIndeks(int[] d, int x)` metodunu ikili arama ile yaz (`IlkIndeks` programı). Değer yoksa `-1` döndürsün. `{1, 3, 3, 3, 5, 8, 8, 9}` için:

```
ilkIndeks(3) = 1
ilkIndeks(8) = 5
ilkIndeks(1) = 0
ilkIndeks(4) = -1
```

Çözümler

Cevap anahtarı

1 B	2 D	3 E	4 A	5 C

1 Cevap: B

İkili arama, "ortadaki eleman aranandan büyükse aranan soldadır" mantığına dayanır; bu ancak dizi **sıralıysa** doğrudur. Tekrarlı elemanlar olabilir (A); eleman sayısının 2'nin kuvveti ya da tek olması gerekmez (C, D), `mid` tam sayı bölmesiyle her boyutta hesaplanır. Negatif sayılar da sorun değildir (E).

2 Cevap: D

Seçmeli sıralama her geçişte sıralanmamış kısmın en küçüğünü başa getirir:

- 1. geçiş: En küçük `10`, `29` ile yer değiştirir → `[10, 29, 14, 37, 13]`
- 1. geçiş: `[29, 14, 37, 13]` içinde en küçük `13`, `29` ile yer değiştirir → `[10, 13, 14, 37, 29]`

Diğer şıklar tuzaktır: A kabarcık sıralamasının, B eklemeli sıralamanın 2. geçişten sonraki halidir; C tamamen sıralı dizi, E ise 1. geçişin sonucudur.

Çıktı

```
Seçmeli : [10, 13, 14, 37, 29]
Kabarcık: [10, 14, 13, 29, 37]
Eklemeli: [10, 14, 29, 37, 13]
```

3 Cevap: E

Dış döngü `i` = 1, 2, 4, 8, … diye **ikiye katlanır** → log n kez döner. İç döngü her seferinde `n` kez döner. İç içe döngüler çarpılır: **O(n log n)**. `n = 16` için $4 \times 16 = 64$:

Çıktı

```
64
```

İki döngü iç içe diye hemen $O(n^2)$ deme (B); dış döngünün artış biçimine bak.

4 Cevap: A

Her adımda aralık yarıya iner: $10^6 \to 5 \times 10^5 \to \dots \to 1$. $2^{20} = 1\,048\,576 \geq 10^6$ olduğu için en fazla yaklaşık **20** adım yeter ($\log_2 10^6 \approx 20$). B doğrusal aramanın en kötü durumu, C doğrusal aramanın ortalamasıdır.

5 Cevap: C

Seçmeli sıralama her geçişte **en fazla bir** yer değiştirme yapar ve `n - 1` geçiş vardır. A yanlış: Sıralı dizide bile $n(n-1)/2$ karşılaştırma yapar. B yanlış: Uzak yer değiştirmeler eşit elemanların sırasını bozabilir (kararsız). D kabarcık sıralamasını tarif eder. E yanlış: Yerinde (in-place) çalışır, ek bellek $O(1)$'dir.

6

`mid` değerleri metodun **içinde** yazdırılır. `println("-> " + ara(...))` satırında önce `ara` çalışır (ve `mid`'leri basar), `println` ise sonuç hesaplandıktan sonra çalışır. Bu yüzden `->` her satırın **sonunda** görünür.

Arama	low, high → mid	Karar
23	0, 9 → **4** (16)	16 < 23 → low = 5
	5, 9 → **7** (56)	56 > 23 → high = 6
	5, 6 → **5** (23)	bulundu → 5
50	0, 9 → **4**; 5, 9 → **7**; 5, 6 → **5** (23 < 50) → low = 6	
	6, 6 → **6** (38 < 50) → low = 7	low > high → −1
2	0, 9 → **4** (16 > 2) → high = 3; 0, 3 → **1** (5 > 2) → high = 0	
	0, 0 → **0** (2)	bulundu → 0

Çıktı

```
4 7 5 -> 5
4 7 5 6 -> -1
4 1 0 -> 0
```

7

Her geçişte komşular karşılaştırılır; hiç yer değiştirme olmayan ilk geçişten sonra döngü biter.

- 1. geçiş: `4↔3`, `5↔1`, `5↔2` → `[3, 4, 1, 2, 5]` (3 takas)
- 1. geçiş: `4↔1`, `4↔2` → `[3, 1, 2, 4, 5]` (2 takas)
- 1. geçiş: `3↔1`, `3↔2` → `[1, 2, 3, 4, 5]` (2 takas)
- 1. geçiş: Hiç takas yok → `degisti` false kalır, döngü biter.

Toplam 7 takas. Bilgi: Kabarcık sıralamasındaki takas sayısı, dizideki **ters sıralı çiftlerin** sayısına eşittir: (4,3), (4,1), (4,2), (3,1), (3,2), (5,1), (5,2) = 7.

Çıktı

```
1: [3, 4, 1, 2, 5]
2: [3, 1, 2, 4, 5]
3: [1, 2, 3, 4, 5]
4: [1, 2, 3, 4, 5]
geçiş=4 takas=7
```

8

- `a`: `i` = 16, 8, 4, 2, 1 → 5 kez. Yarıya bölme → **O(log n)**.
- `b`: `i = 0` iken 16, `i = 1` iken 15, ..., `i = 15` iken 1 → 16 + 15 + ... + 1 = 16 · 17 / 2 = 136. → **O(n²)**.
- `c`: `i` = 0, 4, 8, 12 → 4 kez (n/4). → **O(n)**.
- `d`: 16 × 3 = 48. İç döngü sabit (3) → **O(n)**.

Çıktı

```
5 136 4 48
```

9

Sayılardaki eklemeli sıralamanın aynısı; sadece `d[j] > anahtar` karşılaştırması String için `d[j].compareTo(anahtar) > 0` olur. `compareTo` sözlük sırasına göre soldaki büyükse pozitif döndürür (Bölüm 8).

KelimeSirala.java

```
import java.util.Arrays;

public class KelimeSirala {
    static void sirala(String[] d) {
        for (int i = 1; i < d.length; i++) {
            String anahtar = d[i];
            int j = i - 1;
            while (j >= 0 && d[j].compareTo(anahtar) > 0) {
                d[j + 1] = d[j];
                j--;
            }
            d[j + 1] = anahtar;
            System.out.println(i + ". geçiş: " + Arrays.toString(d));
        }
    }

    public static void main(String[] args) {
        String[] meyveler = {"elma", "armut", "kiraz", "erik", "muz"};
        sirala(meyveler);
    }
}
```

Çıktı

```
1. geçiş: [armut, elma, kiraz, erik, muz]
2. geçiş: [armut, elma, kiraz, erik, muz]
3. geçiş: [armut, elma, erik, kiraz, muz]
4. geçiş: [armut, elma, erik, kiraz, muz]
```

10

Normal ikili aramada değeri bulunca hemen döneriz. Burada bulduğumuz indeksi **kaydedip** aramaya **solda** devam ederiz (`high = mid - 1`), çünkü solda aynı değerden bir tane daha olabilir. Döngü bittiğinde kaydedilen son indeks, ilk geçiştir. Aralık her adımda yine yarıya indiği için karmaşıklık O(log n) kalır.

IlkIndeks.java

```
public class IlkIndeks {
    static int ilkIndeks(int[] d, int x) {
        int low = 0, high = d.length - 1;
        int sonuc = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (d[mid] == x) {
                sonuc = mid;          // bulduk; ama solda başka var mı?
                high = mid - 1;       // solda aramaya devam
            } else if (d[mid] < x) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return sonuc;
    }
```

```
    public static void main(String[] args) {
        int[] d = {1, 3, 3, 3, 5, 8, 8, 9};
        int[] arananlar = {3, 8, 1, 4};
        for (int x : arananlar) {
            System.out.println("ilkIndeks(" + x + ") = " + ilkIndeks(d, x));
        }
    }
}
```

Çıktı

```
ilkIndeks(3) = 1
ilkIndeks(8) = 5
ilkIndeks(1) = 0
ilkIndeks(4) = -1
```

BÖLÜM 17

Swing ile Grafik Arayüze Giriş

Bu Bölümde Öğreneceklerin

- Konsol programı ile grafik arayüzlü (GUI) program arasındaki farkı ve **olay güdümlü** programlamayı
- `JFrame` ile pencere oluşturmayı
- `JLabel`, `JButton`, `JTextField`, `JTextArea`, `JPanel`, `JCheckBox` bileşenlerini
- `FlowLayout`, `BorderLayout` ve `GridLayout` yerleşim yöneticilerini
- `ActionListener` ile düğme tıklamalarına cevap vermeyi (anonim sınıf ve lambda ile)
- Metin kutusundan sayı okumayı ve hatalı girdide `JOptionPane` ile uyarı göstermeyi
- `SwingUtilities.invokeLater`'ın ve EDT'nin neden gerektiğini
- Tıklama sayacı, sıcaklık dönüştürücü ve hesap makinesi uygulamalarını

17.1 Konsoldan pencereye

Şimdiye kadar yazdığın bütün programlar **konsol** programıydı: Program yukarıdan aşağı sırayla çalışır, bir şey sorması gerektiğinde `Scanner` ile durur ve cevabı bekler. Akışı **program** belirler.

Kullandığın uygulamaların neredeyse hepsi ise **grafik kullanıcı arayüzüne** (GUI, Graphical User Interface) sahiptir: pencereler, düğmeler, metin kutuları... Bu programlarda akışı **kullanıcı** belirler. Program pencereyi açar ve **bekler**; kullanıcı hangi düğmeye ne zaman tıklayacağına kendisi karar verir. Her tıklama, her tuş vuruşu bir **olaydır** (event). Program da olaylara cevap veren küçük kod parçalarından oluşur. Bu yaklaşıma **olay güdümlü programlama** (event-driven programming) denir.

Kapı zilini düşün: Misafir gelecek diye bütün gün kapının önünde beklemezsin. Zil çaldığında (olay) gidip kapıyı açarsın (olaya cevap). GUI programında da her düğmeye "tıklanırsa şunu yap" diyen bir **dinleyici** (listener) bağlanır.

	Konsol programı	GUI programı
Akışı kim belirler?	Program: sırayla, yukarıdan aşağı	Kullanıcı: ne zaman neye tıklarsa
Girdi	`Scanner`, program sorduğunda	Metin kutuları, düğmeler; her an
Çıktı	`System.out.println`	Etiketler, metin alanları, diyalog kutuları
`main` bitince	Program biter	Pencere açık kaldıkça program yaşar
Kodun yapısı	Tek bir akış	Olaylara cevap veren dinleyiciler

Java'nın GUI kütüphaneleri

Kütüphane	Paket	Özellik
AWT (1995)	`java.awt`	İlk kütüphane; işletim sisteminin kendi bileşenlerini kullanır. Bugün çoğunlukla yerleşim yöneticileri ve olay sınıfları için kullanılır.
Swing (1998)	`javax.swing`	AWT'nin üzerine kurulu; bileşenleri Java kendisi çizer. **JDK ile birlikte gelir**, kurulum gerekmez. Adları `J` ile başlar: `JButton`, `JFrame`.
JavaFX	`javafx.*`	Daha modern görünüm, CSS ile stil, animasyon. Java 11'den beri JDK'da **yoktur**, ayrıca indirilir.

Üniversite derslerinde ve sınavlarda genellikle **Swing** kullanılır, çünkü ek kurulum gerektirmez. Biz de Swing ile ilerleyeceğiz.

> **Not: Bu bölümdeki programlar hakkında**
>
> Kitaptaki diğer programlar gibi bu bölümdeki Swing programları da JDK 25 ile **derlendi**; ama kitap hazırlanırken pencere açılamadığı için **çalıştırılmadı**. Pencerelerin görüntüsü yerine **taslak çizimler** göreceksin. Programları kendi bilgisayarında VS Code'dan çalıştırdığında gerçek pencereyi görürsün (yazı tipleri ve renkler işletim sistemine göre biraz farklı olabilir). Olayların mantığını gösteren **konsol** programları ise her zamanki gibi gerçekten çalıştırıldı.

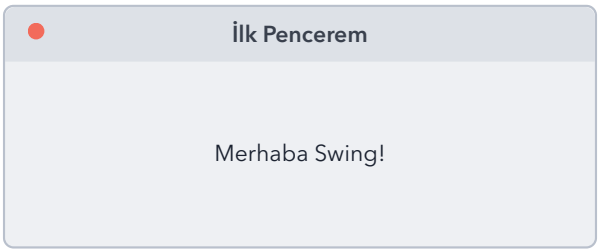
17.2 İlk pencere: JFrame

Swing'de her pencere bir `JFrame` nesnesidir:

IlkPencere.java

```
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class IlkPencere {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame pencere = new JFrame();
            pencere.setTitle("İlk Pencerem");
            pencere.setSize(300, 150);                      // genişlik, yükseklik
            pencere.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            pencere.setLocationRelativeTo(null);            // ekranın ortası
            JLabel etiket = new JLabel("Merhaba Swing!", SwingConstants.CENTER);
            pencere.add(etiket);
            pencere.setVisible(true);                       // en son: göster
        });
    }
}
```



`main`'in içini `SwingUtilities.invokeLater(() -> { ... });` ile sardık. Şimdilik bunu Swing programlarının **kalıbı** olarak kabul et; nedenini bu bölümün ilerisinde, EDT başlığında göreceksin. `() -> { ... }` bir lambda ifadesidir (Bölüm 13).

Metot	Ne yapar?
`setTitle("...")`	Başlık çubuğundaki yazıyı ayarlar (yapıcıya da verilebilir: `new JFrame("Başlık")`)
`setSize(genişlik, yükseklik)`	Pencere boyutunu **piksel** cinsinden ayarlar
`setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`	Pencerenin kapatma düğmesine basılınca **programı sonlandırır**
`setLocationRelativeTo(null)`	Pencereyi ekranın ortasına yerleştirir (`setSize`'dan **sonra** çağır)
`add(bileşen)`	Pencereye bir bileşen ekler
`pack()`	`setSize` yerine: boyutu içindeki bileşenlere göre otomatik ayarlar
`setVisible(true)`	Pencereyi gösterir. Bileşenleri ekledikten sonra, **en son** çağrılır

> **Sık Yapılan Hata: Pencere kapandı ama program çalışmaya devam ediyor**
>
> `setDefaultCloseOperation` çağrılmazsa varsayılan davranış `HIDE_ON_CLOSE`'dur: Kapatma düğmesine basınca pencere sadece **gizlenir**, program arka planda çalışmaya devam eder. VS Code'da terminalin hâlâ meşgul olduğunu görürsün. Her ana pencerede `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` yaz. Bir diğer klasik hata `setVisible(true)`'yu unutmaktır: Program hata vermez, ama ekranda hiçbir şey görünmez.

17.3 Temel bileşenler

Pencereye konan düğme, etiket, metin kutusu gibi her şeye **bileşen** (component) denir. En sık kullanılanlar:

Bileşen	Ne işe yarar?	Sık kullanılan metotlar
`JLabel`	Sabit metin gösterir; kullanıcı değiştiremez	`setText`, `getText`
`JButton`	Tıklanabilir düğme	`addActionListener`, `setText`, `setEnabled`
`JTextField`	**Tek satırlık** metin kutusu	`getText`, `setText`, `setEditable`
`JTextArea`	**Çok satırlı** metin alanı	`getText`, `setText`, `append`
`JPanel`	Bileşenleri gruplayan görünmez kap (container)	`add`, `setLayout`
`JCheckBox`	İşaretlenebilir kutu	`isSelected`, `setSelected`

UyelikFormu.java

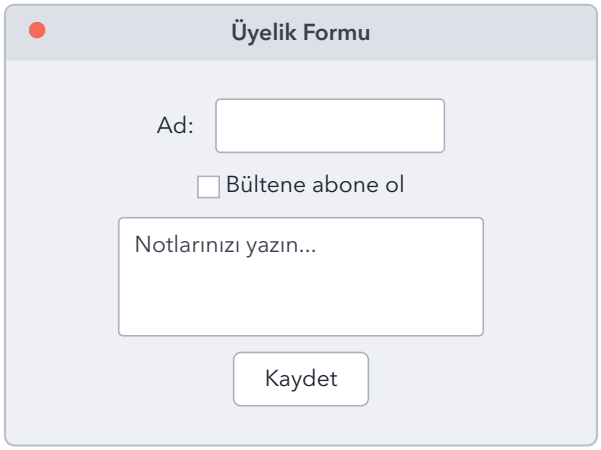
```
import javax.swing.*;

public class UyelikFormu {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame pencere = new JFrame("Üyelik Formu");
            pencere.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

            JPanel panel = new JPanel();             // bileşenleri tutan kap
            panel.add(new JLabel("Ad:"));
```

```
            panel.add(new JTextField(12));          // ~12 karakter genişlik
            panel.add(new JCheckBox("Bültene abone ol"));
            JTextArea notlar = new JTextArea(4, 20); // 4 satır, 20 sütun
            notlar.setText("Notlarınızı yazın...");
            panel.add(notlar);
            panel.add(new JButton("Kaydet"));

            pencere.add(panel);
            pencere.setSize(260, 230);
            pencere.setLocationRelativeTo(null);
            pencere.setVisible(true);
        });
    }
}
```



`import javax.swing.*;` satırındaki `*`, paketteki **bütün** sınıfları içe aktarır; Swing programlarında çok sayıda sınıf kullanıldığı için bu kısaltma yaygındır. Bileşenler panele eklenme sırasıyla, soldan sağa dizildi; satıra sığmayan bileşen alt satıra geçti. Bunu yapan, panelin **yerleşim yöneticisidir**.

17.4 Yerleşim yöneticileri (layout managers)

Swing'de bileşenlerin konumunu ve boyutunu genellikle sen değil, kabın **yerleşim yöneticisi** (layout manager) belirler. Pencere büyüyüp küçüldüğünde bileşenleri yeniden yerleştiren de odur. Üç temel yerleşim yöneticisi `java.awt` paketindedir:

- **`FlowLayout`**: Bileşenleri yazı yazar gibi soldan sağa dizer; satır dolunca alt satıra geçer. Bileşenler kendi **tercih ettikleri boyutta** kalır. `JPanel`'in varsayılanıdır.
- **`BorderLayout`**: Kabı beş bölgeye ayırır: `NORTH` (kuzey), `SOUTH` (güney), `EAST` (doğu), `WEST` (batı), `CENTER` (orta). `CENTER` geriye kalan **bütün alanı** kaplar. `JFrame`'in varsayılanıdır.
- **`GridLayout(satır, sütun)`**: Kabı **eşit boyutlu** hücrelerden oluşan bir ızgaraya böler; bileşenler satır satır, soldan sağa yerleştirilir.

Yerlesimler.java

```
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import javax.swing.*;

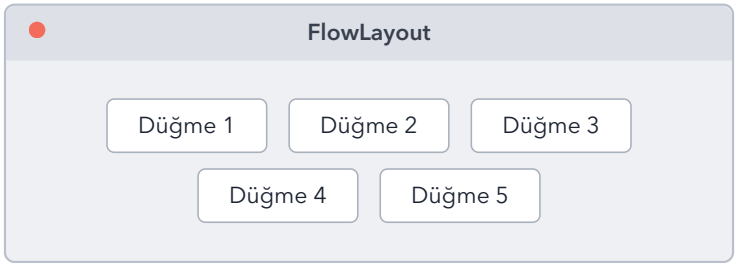
public class Yerlesimler {
    static void goster(String baslik, JPanel icerik, int y) {
        JFrame f = new JFrame(baslik);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.add(icerik);
```

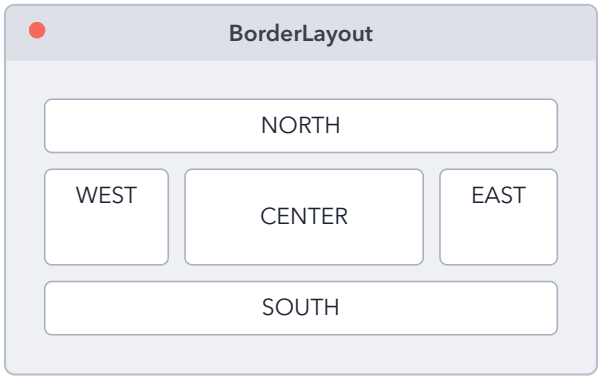
```
        f.setSize(300, 140);
        f.setLocation(100, y);          // üç pencere üst üste binmesin
        f.setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JPanel akis = new JPanel(new FlowLayout());
            for (int i = 1; i <= 5; i++) {
                akis.add(new JButton("Düğme " + i));
            }
            goster("FlowLayout", akis, 50);

            JPanel kenar = new JPanel(new BorderLayout());
            kenar.add(new JButton("NORTH"), BorderLayout.NORTH);
            kenar.add(new JButton("SOUTH"), BorderLayout.SOUTH);
            kenar.add(new JButton("WEST"), BorderLayout.WEST);
            kenar.add(new JButton("EAST"), BorderLayout.EAST);
            kenar.add(new JButton("CENTER"), BorderLayout.CENTER);
            goster("BorderLayout", kenar, 220);

            JPanel izgara = new JPanel(new GridLayout(2, 3));   // 2 satır, 3 sütun
            for (int i = 1; i <= 6; i++) {
                izgara.add(new JButton(String.valueOf(i)));
            }
            goster("GridLayout(2, 3)", izgara, 390);
        });
    }
}
```







	FlowLayout	BorderLayout	GridLayout
Yerleştirme	Soldan sağa, sığmazsa alt satır (ortalı)	5 bölge	Satır satır, eşit hücreler
Bileşen boyutu	Kendi tercih ettiği boyut	Kenarlar ince, `CENTER` kalan her yer	Hücreyi tamamen doldurur, hepsi eşit
Bileşen sayısı	Sınırsız	Her bölgeye **bir** tane (en fazla 5)	satır × sütun
Varsayılan olduğu kap	`JPanel`	`JFrame`	—
Tipik kullanım	Bir sıra düğme, "etiket + kutu"	Ana pencerenin iskeleti	Tuş takımı, form

Sınavda Çıkar: Yerleşim tuzakları

- `JFrame`'in varsayılanı **BorderLayout**, `JPanel`'in varsayılanı **FlowLayout**'tur.
- BorderLayout'a bölge belirtmeden `add(b)` yaparsan bileşen **CENTER**'a gider.
- Her bölgede **tek** bileşen olabilir. Aynı bölgeye ikinci bileşeni eklersen öncekinin **yerini alır**: Pencereye bölgesiz üç düğme eklersen sadece **sonuncusu** görünür.
- Boş kalan bölge yer kaplamaz; `CENTER` o alanı da kullanır.
- GridLayout hücreleri **satır satır** doldurur: `GridLayout(2, 3)`'te 4. bileşen ikinci satırın başına gelir.

İpucu: Karmaşık arayüz = iç içe paneller

Gerçek arayüzler tek bir yerleşim yöneticisiyle kurulmaz. Ana pencereyi `BorderLayout` ile bölgelere ayır, her bölgeye kendi yerleşimi olan bir `JPanel` koy: kuzeye düğmeler için `FlowLayout`'lu bir panel, ortaya tuş takımı için `GridLayout`'lu bir panel... Bu bölümün uygulamaları hep böyle kuruldu.

17.5 Olaylar ve ActionListener

Bir düğmeye tıklandığında üç taraf rol oynar:

1. **Olay kaynağı** (event source): Olayı üreten bileşen, örneğin `JButton`.
2. **Olay nesnesi** (event object): Olayın bilgilerini taşıyan `ActionEvent` nesnesi. `e.getActionCommand()` düğmenin yazısını, `e.getSource()` olayı üreten bileşeni verir.
3. **Dinleyici** (listener): Olay olunca çalışacak kod. Düğme tıklamaları için `ActionListener` arayüzü (Bölüm 11) kullanılır. Bu arayüzün tek bir metodu vardır: `void actionPerformed(ActionEvent e)`.

Dinleyici, kaynağa `addActionListener(...)` ile **kaydedilir** (register). Bundan sonra düğmeye her tıklandığında Swing, kayıtlı dinleyicinin `actionPerformed` metodunu **kendisi** çağırır. Sen bu metodu hiçbir zaman doğrudan çağırmazsın.

Dinleyici yazmanın en yaygın iki yolu **anonim sınıf** ve **lambda**dır:

IkiYol.java

```
import java.awt.FlowLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.*;

public class IkiYol {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
```

```
            JFrame f = new JFrame("İki Yol");
            f.setLayout(new FlowLayout());
            JLabel etiket = new JLabel("Bir düğmeye tıkla");
            JButton b1 = new JButton("Anonim");
            JButton b2 = new JButton("Lambda");

            // 1) Anonim sınıf: adı olmayan, yerinde tanımlanan sınıf
            b1.addActionListener(new ActionListener() {
                @Override
                public void actionPerformed(ActionEvent e) {
                    etiket.setText("Merhaba (anonim sınıf)");
                }
            });

            // 2) Lambda: tek metotlu arayüz için kısa yazım
            b2.addActionListener(e -> etiket.setText("Merhaba (lambda)"));

            f.add(b1);
            f.add(b2);
            f.add(etiket);
            f.setSize(300, 110);
            f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            f.setVisible(true);
        });
    }
}
```



Taslakta “Lambda” düğmesine tıklandıktan sonraki hali görüyorsun.

- **Anonim sınıf** (anonymous class): `new ActionListener() { ... }` ifadesi, adı olmayan bir sınıfı **hem tanımlar hem de ondan bir nesne oluşturur**. Eski kodlarda ve ders kitaplarında çok görürsün.
- **Lambda:** `ActionListener` tek soyut metotlu bir arayüz (fonksiyonel arayüz) olduğu için lambda ile yazılabilir (Bölüm 13). Modern Swing kodunda en çok kullanılan yol budur.
- Üçüncü yol, `ActionListener`'ı uygulayan **adlı, ayrı bir sınıf** yazmaktır (`class SelamDinleyici implements ActionListener { ... }`). En uzun yoldur; aynı dinleyiciyi birçok yerde kullanacaksan mantıklıdır.

Olay mekanizmasını konsolda canlandıralım

Swing'in arka planda ne yaptığını görmek için aynı mekanizmayı kendi küçük sınıflarımızla kuralım. `SimuleDugme` sınıfı dinleyicileri bir listede tutuyor; `tikla()` metodu, kullanıcının fareyle tıklamasının yerini tutuyor:

OlaySimulasyonu.java

```
import java.util.ArrayList;
import java.util.List;

interface TiklamaDinleyici {              // ActionListener'ın karşılığı
    void tiklandi(String komut);
}

class SimuleDugme {                        // JButton'ın karşılığı
    private final String yazi;
    private final List<TiklamaDinleyici> dinleyiciler = new ArrayList<>();
```

```
    SimuleDugme(String yazi) {
        this.yazi = yazi;
    }

    void dinleyiciEkle(TiklamaDinleyici d) {    // addActionListener
        dinleyiciler.add(d);
    }

    void tikla() {                          // kullanıcı tıklayınca Swing bunu yapar
        System.out.println("[" + yazi + "] tıklandı");
        for (TiklamaDinleyici d : dinleyiciler) {
            d.tiklandi(yazi);               // actionPerformed çağrısı
        }
    }
}

public class OlaySimulasyonu {
    static int sayac = 0;

    public static void main(String[] args) {
        SimuleDugme arttir = new SimuleDugme("Arttır");
        SimuleDugme sifirla = new SimuleDugme("Sıfırla");

        arttir.dinleyiciEkle(new TiklamaDinleyici() {     // anonim sınıf
            @Override
            public void tiklandi(String komut) {
                sayac++;
                System.out.println("  sayaç = " + sayac);
            }
        });
        sifirla.dinleyiciEkle(komut -> {                  // lambda
            sayac = 0;
            System.out.println("  sayaç sıfırlandı");
        });

        System.out.println("Pencere açıldı, olay bekleniyor...");
        arttir.tikla();
        arttir.tikla();
        sifirla.tikla();
        arttir.tikla();
    }
}
```

Çıktı

```
Pencere açıldı, olay bekleniyor...
[Arttır] tıklandı
  sayaç = 1
[Arttır] tıklandı
  sayaç = 2
[Sıfırla] tıklandı
  sayaç sıfırlandı
[Arttır] tıklandı
  sayaç = 1
```

`main` içinde dinleyicileri sadece **kaydettik**; hiçbirini doğrudan çağırmadık. Kod, ancak bir “tıklama” olduğunda çalıştı. Gerçek Swing’de `tikla()` çağrılarını sen yazmazsın; onları kullanıcının fareyle yaptığı tıklamalar tetikler.

Sık Yapılan Hata: Lambda içinde yerel değişkeni değiştirmek

Tıklama sayısını tutmak için `main` içinde yerel bir değişken kullanmak ilk akla gelen yoldur, ama **derlenmez**:

```
import javax.swing.JButton;

public class SayacHatasi {
    public static void main(String[] args) {
        int sayac = 0;
        JButton dugme = new JButton("Arttır");
        dugme.addActionListener(e -> sayac++);
    }
}
```

Derleyici çıktısı – program derlenmez

```
SayacHatasi.java:7: error: local variables referenced from a lambda expression must be final or
effectively final
        dugme.addActionListener(e -> sayac++);
                                    ^
1 error
```

Lambda ve anonim sınıflar, dışarıdaki yerel değişkenleri sadece **okuyabilir**; değiştiremez (değişken *effectively final*, yani fiilen sabit olmalıdır). Çözüm: Değişeni bir **alan** (field) yap. Yukarıdaki simülasyonda `sayac`'ı `static` alan yaptık; birazdan yazacağımız sayaç uygulamasında da sınıfın alanı olacak.

17.6 Metin kutusundan sayı okumak

`JTextField`'ın `getText()` metodu kutudaki yazıyı her zaman **String** olarak verir; kullanıcı `19` yazsa bile elindeki `"19"`dur. Sayıya çevirmek için Bölüm 3'te gördüğün `Integer.parseInt` (veya `Double.parseDouble`) kullanılır. Kullanıcı sayı olmayan bir şey yazarsa `NumberFormatException` fırlatılır (Bölüm 12). Konsolda program çökerdi; GUI'de bunu yakalayıp kullanıcıya **hata penceresi** göstermek gerekir:

YasHesabi.java

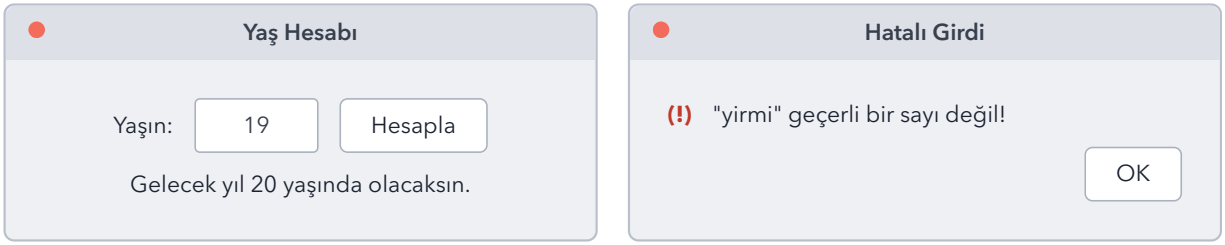
```
import java.awt.FlowLayout;
import javax.swing.*;

public class YasHesabi {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame pencere = new JFrame("Yaş Hesabı");
            pencere.setLayout(new FlowLayout());
            JTextField yasKutusu = new JTextField(5);
            JButton hesapla = new JButton("Hesapla");
            JLabel sonuc = new JLabel("Yaşını yaz ve Hesapla'ya bas.");

            hesapla.addActionListener(e -> {
                String metin = yasKutusu.getText().trim();
                try {
                    int yas = Integer.parseInt(metin);
                    int gelecek = yas + 1;
                    sonuc.setText("Gelecek yıl " + gelecek + " yaşında olacaksın.");
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(pencere,
                            "\"" + metin + "\" geçerli bir sayı değil!",
                            "Hatalı Girdi", JOptionPane.ERROR_MESSAGE);
                    yasKutusu.setText("");          // kutuyu temizle
                    yasKutusu.requestFocus();       // imleci kutuya koy
                }
```

```
            });

            pencere.add(new JLabel("Yaşın:"));
            pencere.add(yasKutusu);
            pencere.add(hesapla);
            pencere.add(sonuc);
            pencere.setSize(300, 120);
            pencere.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            pencere.setLocationRelativeTo(null);
            pencere.setVisible(true);
        });
    }
}
```



Solda geçerli bir yaş girildiğindeki pencere, sağda `yirmi` yazılınca açılan hata penceresi. Hata penceresinin düğme yazısını (OK) Java koyar; bilgisayarının dil ayarına göre İngilizce görünebilir.

> **Not: Lambda parametresi ile catch parametresi**
>
> Dinleyici lambdasının parametresi `e` olduğu için `catch` bloğunda `ex` adını kullandık. `catch (NumberFormatException e)` yazsaydın *variable e is already defined* derleme hatası alırdın: Aynı kapsamda iki değişken aynı adı taşıyamaz.

İş mantığını arayüzden ayır

Pencereyi açmadan test edebilmek için **hesaplama** kodunu ayrı, sade bir metoda koymak iyi bir alışkanlıktır; dinleyici sadece bu metodu çağırır. Böylece mantığı konsolda deneyebilirsin:

YasMantigi.java

```
public class YasMantigi {
    // Dinleyicinin yaptığı işin aynısı, ama pencere olmadan
    static String hesapla(String metin) {
        try {
            int yas = Integer.parseInt(metin.trim());
            return "Gelecek yıl " + (yas + 1) + " yaşında olacaksın.";
        } catch (NumberFormatException e) {
            return "HATA PENCERESİ: \"" + metin.trim() + "\" sayı değil!";
        }
    }

    public static void main(String[] args) {
        String[] girdiler = {"19", "  20 ", "yirmi", "", "19.5"};
        for (String g : girdiler) {
            System.out.println("[" + g + "] -> " + hesapla(g));
        }
    }
}
```

```
Çıktı
[19] -> Gelecek yıl 20 yaşında olacaksın.
[  20 ] -> Gelecek yıl 21 yaşında olacaksın.
[yirmi] -> HATA PENCERESİ: "yirmi" sayı değil!
[] -> HATA PENCERESİ: "" sayı değil!
[19.5] -> HATA PENCERESİ: "19.5" sayı değil!
```

`trim()` sayesinde baştaki/sondaki boşluklar sorun olmadı; ama boş kutu ve `19.5` de `parseInt` için geçersiz. Gerçek programda bu satırların hepsi bir hata penceresi açar.

17.7 EDT ve SwingUtilities.invokeLater

Bilgisayar aynı anda birden fazla iş yapabilir; bir programın içindeki her bağımsız iş akışına **iş parçacığı** (thread) denir. Şimdiye kadarki programlarda tek bir iş parçacığı vardı: `main`'i çalıştıran **main thread**.

Swing ise bütün arayüz işlerini **tek** bir özel iş parçacığında yapar: **EDT** (Event Dispatch Thread, olay dağıtım iş parçacığı). Tıklamalar, tuş vuruşları, pencerenin yeniden çizilmesi bir **kuyruğa** girer ve EDT bunları **sırayla, tek tek** işler. Bütün dinleyici metotların (`actionPerformed`) EDT'de çalışır.

Bunun nedeni, Swing bileşenlerinin **thread-safe** (iş parçacığı güvenli) olmamasıdır: Aynı düğmeyi iki iş parçacığı aynı anda değiştirmeye çalışırsa tutarsız, nadiren ortaya çıkan ve bulunması çok zor hatalar oluşur. Swing'in kuralı bu yüzden basittir:

1. **Arayüzü EDT'de oluştur ve değiştir.** `SwingUtilities.invokeLater(() -> {...})`, verilen kodu hemen çalıştırmaz; EDT'nin kuyruğuna ekler ve EDT sırası gelince çalıştırır. `main` metodu hemen biter, ama EDT çalışmaya devam eder; pencere açık kaldıkça program da yaşar.
2. **Dinleyicide uzun iş yapma.** EDT bir dinleyiciyi çalıştırırken başka hiçbir olayı işleyemez. Dinleyicide 10 saniye süren bir hesaplama yaparsan pencere 10 saniye **donar**: Düğmeler tepki vermez, pencere yeniden çizilmez. Uzun işler için Swing'in `SwingWorker` sınıfı kullanılır (ileri konu).

Bir restorandaki tek garsonu düşün: Bütün siparişleri o alır ve masalara o götürür (EDT). İki garson aynı tabağı aynı anda götürmeye çalışmaz; düzen bozulmaz. Ama garson mutfağa girip 10 dakikalık bir yemeği kendisi pişirmeye kalkarsa o süre boyunca hiçbir masaya bakılmaz.

	main thread	EDT
Ne çalıştırır?	`main` metodu	Arayüz kodu, bütün dinleyiciler
Ne zaman biter?	`main` bitince	Bütün pencereler kapanınca
Swing bileşenine dokunmalı mı?	Hayır, `invokeLater` ile EDT'ye devret	Evet

> **Sınavda Çıkar: EDT soruları**
>
> - "`SwingUtilities.invokeLater` ne işe yarar?" → Arayüzü oluşturan kodu **EDT'de** çalıştırılmak üzere sıraya koyar; çünkü Swing **thread-safe değildir**.
> - "Bir düğmenin dinleyicisinde çok uzun süren bir işlem yapılırsa ne olur?" → EDT meşgul kalır, arayüz **donar**.
> - Dinleyici metotlarının içinde ayrıca `invokeLater` kullanmaya gerek yoktur; onlar zaten EDT'de çalışır.

17.8 JOptionPane ile diyalog kutuları

`JOptionPane` sınıfı, tek satırla hazır **diyalog kutuları** (dialog) açmanı sağlar. Pencere oluşturmadan bile kullanılabilir:

Metot	Ne yapar?	Dönüş değeri
`showMessageDialog(ebeveyn, mesaj)`	Mesaj gösterir, OK bekler	yok (`void`)
`showMessageDialog(ebeveyn, mesaj, başlık, tür)`	Başlık ve simge türüyle mesaj	yok
`showInputDialog(ebeveyn, soru)`	Kullanıcıdan metin ister	`String`; **İptal**'de `null`
`showConfirmDialog(ebeveyn, soru, başlık, seçenek)`	Evet/Hayır sorar	`int`: `YES_OPTION`, `NO_OPTION`, `CANCEL_OPTION`, `CLOSED_OPTION`

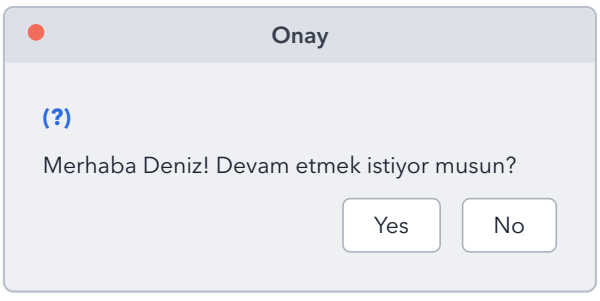
İlk parametre (**ebeveyn**, parent) diyalogun hangi pencerenin üstünde ortalanacağını belirtir; `null` verirsen ekranın ortasında açılır. Mesaj türleri simgeyi belirler: `ERROR_MESSAGE`, `WARNING_MESSAGE`, `INFORMATION_MESSAGE` (varsayılan), `QUESTION_MESSAGE`, `PLAIN_MESSAGE` (simgesiz).

Diyaloglar.java

```
import javax.swing.JOptionPane;
import javax.swing.SwingUtilities;

public class Diyaloglar {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            String ad = JOptionPane.showInputDialog(null, "Adın nedir?");
            if (ad == null || ad.isBlank()) {       // İptal ya da boş
                JOptionPane.showMessageDialog(null, "Ad girilmedi.",
                        "Uyarı", JOptionPane.WARNING_MESSAGE);
                return;
            }
            int cevap = JOptionPane.showConfirmDialog(null,
                    "Merhaba " + ad + "! Devam etmek istiyor musun?",
                    "Onay", JOptionPane.YES_NO_OPTION);
            if (cevap == JOptionPane.YES_OPTION) {
                JOptionPane.showMessageDialog(null, "Harika, devam ediyoruz!");
            } else {
                JOptionPane.showMessageDialog(null, "Görüşmek üzere, " + ad + ".");
            }
        });
    }
}
```





Modal diyaloglar (`show...Dialog`), kullanıcı bir düğmeye basana kadar kodun ilerlemesini **bekletir**; bu yüzden `showInputDialog`'un sonucunu bir sonraki satırda hemen kullanabildik.

> **● Sık Yapılan Hata: showInputDialog'un null dönmesi**
>
> Kullanıcı **Cancel**'a basarsa ya da diyaloğu kapatırsa `showInputDialog` boş String değil **`null`** döndürür. `ad.isBlank()` veya `Integer.parseInt(ad)` satırına `null` ile gelirsen sırasıyla `NullPointerException` ve `NumberFormatException` alırsın. Her zaman önce `null` kontrolü yap: `if (ad == null || ad.isBlank())`. Sıra önemlidir: `||` kısa devre yaptığı için `ad` `null` ise `isBlank()` hiç çağrılmaz.

17.9 Uygulama 1: Tıklama sayacı

İlk tam uygulamamız: ortada büyük bir sayı, altında üç düğme. Bu sefer pencere sınıfımızı **`JFrame`'den kalıtımla** türetiyoruz (Bölüm 10). Bu, Swing'de en yaygın düzendir: Bileşenler sınıfın alanları, kurulum kodu yapıcı metottur; `sayac` da bir alan olduğu için lambdaların içinden değiştirilebilir.

TiklamaSayaci.java

```
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.*;

public class TiklamaSayaci extends JFrame {
    private int sayac = 0;                          // alan: lambdada değişebilir
    private final JLabel gosterge = new JLabel("0", SwingConstants.CENTER);
    private final JButton azalt = new JButton("−1");

    public TiklamaSayaci() {
        super("Tıklama Sayacı");                    // JFrame(String): başlık
        gosterge.setFont(new Font("SansSerif", Font.BOLD, 48));

        JButton sifirla = new JButton("Sıfırla");
        JButton arttir = new JButton("+1");
        azalt.addActionListener(e -> guncelle(sayac - 1));
        sifirla.addActionListener(e -> guncelle(0));
        arttir.addActionListener(e -> guncelle(sayac + 1));

        JPanel dugmeler = new JPanel(new FlowLayout());
        dugmeler.add(azalt);
        dugmeler.add(sifirla);
        dugmeler.add(arttir);

        add(gosterge, BorderLayout.CENTER);
        add(dugmeler, BorderLayout.SOUTH);
        guncelle(0);

        setSize(280, 180);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }

    private void guncelle(int yeniDeger) {
        sayac = yeniDeger;
        gosterge.setText(String.valueOf(sayac));
        azalt.setEnabled(sayac > 0);                // 0'ın altına inmesin
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new TiklamaSayaci().setVisible(true));
    }
}
```



Taslak, `+1`'e üç kez basıldıktan sonraki hali gösteriyor. Dikkat etmen gerekenler:

- Sınıf `JFrame` olduğu için `setSize`, `add` gibi metotları başına `pencere.` yazmadan çağırdık; `EXIT_ON_CLOSE` sabiti de miras alındı.
- Ekranı güncelleyen kodu tek bir `guncelle` metodunda topladık. Üç dinleyici de onu çağırıyor; böylece "sayıyı değiştir, etiketi yenile, `−1`'i aç/kapa" mantığı tek yerde.
- `setEnabled(false)` düğmeyi **pasif** yapar: Gri görünür ve tıklanamaz. Sayaç 0 iken `−1` pasif olduğu için sayaç negatif olamaz.

17.10 Uygulama 2: Sıcaklık dönüştürücü

Kullanıcı bir sıcaklık yazıp iki düğmeden birine basıyor. Arayüzü `GridLayout(3, 2)` ile iki sütunlu bir form olarak kuruyoruz. Dönüştürme formülleri ve metni sayıya çevirme işi ayrı metotlarda:

SicaklikDonusturucu.java

```
import java.awt.GridLayout;
import javax.swing.*;

public class SicaklikDonusturucu extends JFrame {
    private final JTextField giris = new JTextField();
    private final JLabel sonuc = new JLabel("—");

    static double cdenF(double c) {
        return c * 9 / 5 + 32;
    }

    static double fdenC(double f) {
        return (f - 32) * 5 / 9;
    }

    // "36,6" gibi virgüllü girdiyi de kabul et; geçersizse exception fırlatır
    static double sayiyaCevir(String metin) {
        return Double.parseDouble(metin.trim().replace(',', '.'));
    }

    public SicaklikDonusturucu() {
        super("Sıcaklık Dönüştürücü");
        setLayout(new GridLayout(3, 2, 6, 6));    // 3 satır, 2 sütun, 6 px boşluk

        JButton cF = new JButton("°C → °F");
        JButton fC = new JButton("°F → °C");
        cF.addActionListener(e -> donustur(true));
        fC.addActionListener(e -> donustur(false));

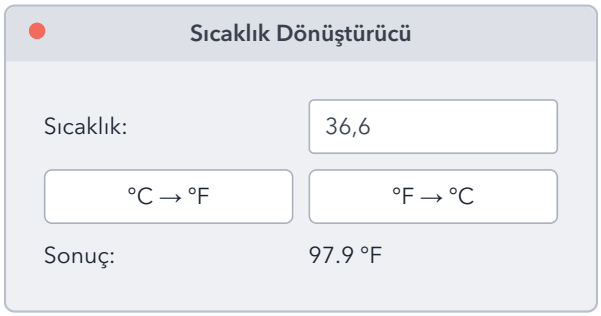
        add(new JLabel("Sıcaklık:"));
        add(giris);
        add(cF);
        add(fC);
```

```
        add(new JLabel("Sonuç:"));
        add(sonuc);

        setSize(300, 140);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }

    private void donustur(boolean cdenFmi) {
        try {
            double deger = sayiyaCevir(giris.getText());
            if (cdenFmi) {
                sonuc.setText(String.format("%.1f °F", cdenF(deger)));
            } else {
                sonuc.setText(String.format("%.1f °C", fdenC(deger)));
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Lütfen bir sayı girin.",
                    "Hatalı Girdi", JOptionPane.ERROR_MESSAGE);
            giris.selectAll();                    // yanlış yazıyı seçili yap
            giris.requestFocus();
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new SicaklikDonusturucu().setVisible(true);
        });
    }
}
```



`JOptionPane.showMessageDialog(this, ...)` satırındaki `this`, pencerenin kendisidir: Hata penceresi bizim pencerenin ortasında açılır. Formülleri ve `sayiyaCevir` metodunu `static` yazdık; arayüze hiç dokunmadıkları için `YasMantigi`'nde yaptığımız gibi konsolda da denenebilirler. `replace(',', '.')` sayesinde Türkçe alışkanlıkla yazılan `36,6` da kabul edilir (Bölüm 3'teki ondalık virgül sorunu).

17.11 Uygulama 3: Mini hesap makinesi

Son uygulamamız `GridLayout(4, 4)` ile bir tuş takımı kuruyor. Bu programda iki önemli fikir var:

- **Tek dinleyici, çok düğme:** 16 düğmenin hepsi aynı lambdayı kullanıyor; hangi tuşa basıldığını `e.getActionCommand()` (düğmenin yazısı) söylüyor.
- **Mantık ayrı sınıfta:** Hesap makinesinin durumu (ekrandaki sayı, bekleyen işlem) `HesapMantigi` sınıfında. Pencere sınıfı sadece tuşları ona iletiyor ve dönen metni ekrana yazıyor.

HesapMakinesi.java

```
import java.awt.BorderLayout;
import java.awt.Font;
import java.awt.GridLayout;
import javax.swing.*;

class HesapMantigi {
    private String ekran = "0";
    private int ilkSayi = 0;            // işlemden önceki sayı
    private char islem = ' ';           // bekleyen işlem: + - * /
    private boolean yeniSayi = true;    // sıradaki rakam ekranı temizlesin mi?

    // Bir tuşa basılınca çağrılır, ekranda görünecek metni döndürür
    String tus(String t) {
        char c = t.charAt(0);
        if (Character.isDigit(c)) {
            if (yeniSayi) {
                ekran = t;
                yeniSayi = false;
            } else if (ekran.length() < 9) {     // int'e sığsın
                ekran = ekran.equals("0") ? t : ekran + t;
            }
        } else if (c == 'C') {
            ekran = "0";
            ilkSayi = 0;
            islem = ' ';
            yeniSayi = true;
        } else if (yeniSayi && islem != ' ' && c != '=') {
            islem = c;                           // ikinci sayı yok: işlemi değiştir
        } else {                                 // + - * / =
            int ikinci = Integer.parseInt(ekran);
            int sonuc = switch (islem) {
                case '+' -> ilkSayi + ikinci;
                case '-' -> ilkSayi - ikinci;
                case '*' -> ilkSayi * ikinci;
                case '/' -> ilkSayi / ikinci;    // 0'a bölme: ArithmeticException
                default -> ikinci;               // bekleyen işlem yok
            };
            ekran = String.valueOf(sonuc);
            ilkSayi = sonuc;
            islem = (c == '=') ? ' ' : c;
            yeniSayi = true;
        }
        return ekran;
    }
}

public class HesapMakinesi extends JFrame {
    private final HesapMantigi mantik = new HesapMantigi();
    private final JTextField ekran = new JTextField("0");

    public HesapMakinesi() {
        super("Hesap Makinesi");
        ekran.setEditable(false);                        // kullanıcı yazamasın
        ekran.setHorizontalAlignment(JTextField.RIGHT);
        ekran.setFont(new Font("Monospaced", Font.BOLD, 28));

        String[] tuslar = {
            "7", "8", "9", "/",
            "4", "5", "6", "*",
            "1", "2", "3", "-",
            "C", "0", "=", "+"
        };
        JPanel tusTakimi = new JPanel(new GridLayout(4, 4, 5, 5));
```

```
        for (String t : tuslar) {
            JButton b = new JButton(t);
            b.setFont(new Font("SansSerif", Font.BOLD, 20));
            b.addActionListener(e -> tusaBasildi(e.getActionCommand()));
            tusTakimi.add(b);
        }

        add(ekran, BorderLayout.NORTH);
        add(tusTakimi, BorderLayout.CENTER);
        setSize(280, 340);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }

    private void tusaBasildi(String tus) {
        try {
            ekran.setText(mantik.tus(tus));
        } catch (ArithmeticException ex) {
            JOptionPane.showMessageDialog(this, "Sıfıra bölünemez!",
                    "Hata", JOptionPane.ERROR_MESSAGE);
            ekran.setText(mantik.tus("C"));            // her şeyi sıfırla
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new HesapMakinesi().setVisible(true));
    }
}
```



`switch` ifadesi (`case '+' -> ...`) Bölüm 4'teki ok sözdizimli `switch`'tir. Hesap makinesi tam sayılarla çalışıyor; bu yüzden `9 / 2` işleminin sonucu `4` olur (tam sayı bölmesi).

Pencereyi açamıyoruz ama tuş dizilerini doğrudan `HesapMantigi`'ne verip ekranda ne görüneceğini konsolda izleyebiliriz. Aşağıdaki program, yukarıdaki `HesapMantigi` sınıfının aynısını kullanıyor (kitapta tekrar gösterilmedi; kendi dosyanda aynı sınıfı alta ekle):

```
public class HesapSimulasyonu {
    // Tuşlara sırayla "basar", son ekranı yazdırır
    static void dene(String tuslar) {
        HesapMantigi m = new HesapMantigi();
        String ekran = "0";
        try {
            for (String t : tuslar.split(" ")) {
                ekran = m.tus(t);
            }
```

```
            System.out.printf("%-20s ekran: %s%n", tuslar, ekran);
        } catch (ArithmeticException ex) {
            System.out.printf("%-20s HATA: %s%n", tuslar, ex.getMessage());
        }
    }

    public static void main(String[] args) {
        HesapMantigi m = new HesapMantigi();
        System.out.println("Tuş tuş: 1 2 + 3 0 =");
        for (String t : "1 2 + 3 0 =".split(" ")) {
            System.out.println("  [" + t + "] -> " + m.tus(t));
        }
        dene("7 * 6 - 2 =");
        dene("9 / 2 =");
        dene("5 + 5 = * 3 =");
        dene("1 2 3 C 4 + 1 =");
        dene("8 / 0 =");
        dene("1 2 + * 3 =");
        dene("0 / + 5 =");
    }
}
```

Çıktı

```
Tuş tuş: 1 2 + 3 0 =
  [1] -> 1
  [2] -> 12
  [+] -> 12
  [3] -> 3
  [0] -> 30
  [=] -> 42
7 * 6 - 2 =          ekran: 40
9 / 2 =              ekran: 4
5 + 5 = * 3 =        ekran: 30
1 2 3 C 4 + 1 =      ekran: 5
8 / 0 =              HATA: / by zero
1 2 + * 3 =          ekran: 36
0 / + 5 =            ekran: 5
```

- Operatör tuşu, **bekleyen** işlemi hemen hesaplar: `7 * 6 -` yazınca `-`'ye basıldığı anda ekranda `42` görünür, sonra `42 - 2 = 40`.
- `=`'den sonra yeni bir operatöre basmak hesabı sonuçtan devam ettirir: `5 + 5 = * 3 =` → `30`.
- İkinci sayıyı yazmadan başka bir operatöre basmak yalnız **işlemi değiştirir** (`yeniSayi && islem != ' '` dalı): `1 2 + * 3 =` → `12 * 3 = 36`. Bu kontrol olmasaydı ekrandaki `12` ikinci sayı sanılır ve `12 + 12` hesaplanırdı; `0 / +` ise daha ikinci sayı girilmeden `0 / 0` hatası verirdi.
- `8 / 0` işleminde `ArithmeticException` fırlatıldı. Pencereli programda bu exception `tusaBasildi` içinde yakalanıp hata penceresi gösteriliyor.

Bölüm Özeti – Bilmen Gerekenler

- GUI programları **olay güdümlüdür**: Program pencereyi açıp bekler, kullanıcının olaylarına (tıklama, tuş) **dinleyicilerle** cevap verir.
- Swing `javax.swing` paketindedir, JDK ile gelir; bileşen adları `J` ile başlar. Yerleşim yöneticileri ve olay sınıfları `java.awt` / `java.awt.event`'tedir.
- `JFrame`: `setTitle`, `setSize`, `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`, `setLocationRelativeTo(null)`, en son `setVisible(true)`.
- Bileşenler: `JLabel` (metin), `JButton` (düğme), `JTextField` (tek satır), `JTextArea` (çok satır), `JPanel` (kap), `JCheckBox` (`isSelected`).

- **FlowLayout** (JPanel varsayılanı): soldan sağa, sığmazsa alt satır. **BorderLayout** (JFrame varsayılanı): 5 bölge, her bölgede tek bileşen, bölgesiz `add` → CENTER. **GridLayout(s, k)**: eşit hücreler, satır satır.
- `ActionListener` → `actionPerformed(ActionEvent e)`; `addActionListener` ile kaydedilir. Ayrı sınıf, **anonim sınıf** veya **lambda** ile yazılır. `e.getActionCommand()` düğmenin yazısını verir.
- Lambda/anonim sınıf içinde **yerel değişken değiştirilemez** → alan (field) kullan.
- `getText()` her zaman String döndürür; `Integer.parseInt` geçersiz metinde `NumberFormatException` fırlatır → yakala ve `JOptionPane.showMessageDialog(..., ERROR_MESSAGE)` ile uyar.
- `JOptionPane`: `showMessageDialog`, `showInputDialog` (İptal'de `null`), `showConfirmDialog` (`YES_OPTION`...).
- Swing **thread-safe değildir**; arayüz kodu **EDT**'de çalışmalıdır → `SwingUtilities.invokeLater`. Dinleyicilerde uzun iş yapılırsa arayüz donar.
- Hesaplama mantığını arayüzden ayır: Hem kod düzenli olur hem de mantık konsolda test edilebilir.

Sorular

A) Çoktan Seçmeli

1 Bir `JFrame`'in varsayılan yerleşim yöneticisi hangisidir?

(A) `FlowLayout`
(B) `GridLayout`
(C) `BorderLayout`
(D) `BoxLayout`
(E) Yerleşim yöneticisi yoktur (`null`)

2 Pencerenin kapatma düğmesine basıldığında programın da sonlanması için ne yapılmalıdır?

(A) `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` çağrılmalıdır.
(B) `setVisible(false)` çağrılmalıdır.
(C) `main` metodunun son satırına `System.exit(0);` yazılmalıdır.
(D) `setLocationRelativeTo(null)` çağrılmalıdır.
(E) `pack()` çağrılmalıdır.

3 `ActionListener` arayüzünü uygulayan bir sınıfın gövdesini yazması (override etmesi) gereken metot hangisidir?

(A) `public void actionListener(ActionEvent e)`
(B) `public void onClick()`
(C) `public void clicked(ActionEvent e)`
(D) `public void handle(Event e)`
(E) `public void actionPerformed(ActionEvent e)`

4 Kullanıcı metin kutusuna `abc` yazıp düğmeye bastığında dinleyicideki `int x = Integer.parseInt(kutu.getText());` satırı ne yapar?

(A) `x` değişkenine `0` atanır.

(B) `InputMismatchException` fırlatılır.

(C) Program derlenmez.

(D) `NumberFormatException` fırlatılır.

(E) `x` değişkenine `null` atanır.

5 Swing programlarında arayüzü oluşturan kod neden `SwingUtilities.invokeLater(...)` içine yazılır?

(A) GUI kodunu birden fazla iş parçacığına dağıtarak programı hızlandırmak için.

(B) Swing thread-safe olmadığı için, kodu olay dağıtım iş parçacığında (EDT) çalıştırılmak üzere sıraya koymak için.

(C) Pencerenin ekranın ortasında açılmasını sağlamak için.

(D) Kodu belirli bir süre bekledikten sonra çalıştırmak için.

(E) Dinleyicileri düğmelere otomatik olarak bağlamak için.

B) Çıktı Tahmini

6 Aşağıdaki program çalıştırıldığında pencerede bileşenler hangi düzende görünür? Kabaca çiz.

```
import java.awt.BorderLayout;
import javax.swing.*;

public class Soru6 {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame f = new JFrame("Soru 6");
            f.add(new JButton("Kuzey"), BorderLayout.NORTH);
            f.add(new JButton("Bir"));
            f.add(new JButton("İki"));
            f.add(new JButton("Güney"), BorderLayout.SOUTH);
            f.add(new JLabel("Doğu"), BorderLayout.EAST);
            f.setSize(300, 200);
            f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            f.setVisible(true);
        });
    }
}
```

7 Aşağıdaki program çalıştırıldığında pencerede bileşenler hangi düzende görünür? Kabaca çiz.

```
import java.awt.BorderLayout;
import java.awt.GridLayout;
import javax.swing.*;

public class Soru7 {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame f = new JFrame("Soru 7");
            JPanel ust = new JPanel();
            ust.add(new JLabel("Ad:"));
            ust.add(new JTextField(8));
            ust.add(new JButton("Ara"));
            JPanel tuslar = new JPanel(new GridLayout(2, 3));
            for (int i = 6; i >= 1; i--) {
                tuslar.add(new JButton("" + i));
            }
            f.add(ust, BorderLayout.NORTH);
            f.add(tuslar);
            f.add(new JButton("Tamam"), BorderLayout.SOUTH);
            f.setSize(260, 200);
            f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            f.setVisible(true);
        });
    }
}
```

8 Aşağıdaki program, bir düğmeyi ve dinleyicilerini konsolda canlandırıyor. Ekrana ne yazdırır?

```
import java.util.ArrayList;
import java.util.List;

interface Dinleyici {
    void tiklandi();
}

class Dugme {
    List<Dinleyici> dinleyiciler = new ArrayList<>();
    boolean aktif = true;

    void tikla() {
        if (!aktif) {
            System.out.println("pasif");
            return;
        }
        for (Dinleyici d : dinleyiciler) {
            d.tiklandi();
        }
    }
}

public class Soru8 {
    static int puan = 0;

    public static void main(String[] args) {
        Dugme b = new Dugme();
        b.dinleyiciler.add(() -> puan += 10);
        b.dinleyiciler.add(new Dinleyici() {
            int sayac = 0;
```

```
            @Override
            public void tiklandi() {
                sayac++;
                System.out.println("#" + sayac + " puan=" + puan);
            }
        });
        b.dinleyiciler.add(() -> {
            if (puan >= 30) {
                b.aktif = false;
            }
        });
        for (int i = 0; i < 4; i++) {
            b.tikla();
        }
        System.out.println("Son puan: " + puan);
    }
}
```

C) Kod Yazma

9 `RenkSecici` adlı bir pencere yaz: Üstte (NORTH) bir panelde **Kırmızı**, **Yeşil** ve **Mavi** düğmeleri, ortada (CENTER) boş bir `JPanel` olsun. Bir düğmeye basılınca ortadaki panelin arka plan rengi o renk olsun (`setBackground(Color.RED)` gibi) ve pencere başlığı `Renk: Mavi` gibi değişsin. Üç düğme için **tek bir** dinleyici nesnesi kullan.

10 Bir **kelime sayacı** yaz: Pencerenin ortasında çok satırlı bir metin alanı (`JTextArea`), altta bir "Say" düğmesi ve `Kelime: 0 | Karakter: 0` yazan bir etiket olsun. "Say"a basılınca metindeki kelime ve karakter sayısı etikete yazılsın. Kelime sayan mantığı `static int kelimeSay(String metin)` metoduna koy ve bu metodu ayrı bir konsol programında (`KelimeSayTest`) şu girdilerle dene: `""`, `"  "`, `"Java öğreniyorum"`, `" bir iki\n üç "`.

Çözümler

Cevap anahtarı

1 C	2 A	3 E	4 D	5 B

1 Cevap: C

`JFrame`'in içerik paneli varsayılan olarak **BorderLayout** kullanır; bu yüzden bölge belirtmeden eklenen bileşen `CENTER`'a gider. `FlowLayout` (A), `JPanel`'in varsayılanıdır. `GridLayout` ve `BoxLayout` hiçbir kabın varsayılanı değildir; `null` yerleşim ancak `setLayout(null)` ile elle seçilir.

2 Cevap: A

`EXIT_ON_CLOSE`, pencere kapatılınca programı sonlandırır. Varsayılan `HIDE_ON_CLOSE` pencereyi sadece gizler. `setVisible(false)` (B) pencereyi gizler ama programı bitirmez. C tuzaktır: `invokeLater` ile pencere kurulduktan hemen sonra `main` sona erer; o satıra gelindiğinde `System.exit(0)` programı **hemen** kapatır ve pencere ya hiç görünmez ya da bir an görünüp kaybolur. D ve E konum ve boyutla ilgilidir.

3 Cevap: E

`ActionListener` arayüzünün tek soyut metodu `void actionPerformed(ActionEvent e)`'dir; Swing tıklamada bu metodu çağırır. Diğer adlar Java'da tanımlı değildir (`handle` JavaFX'te kullanılan bir addır, Swing'de değil). Lambda yazarken (`e -> ...`) yazdığın gövde de aslında bu metodun gövdesidir.

4 Cevap: D

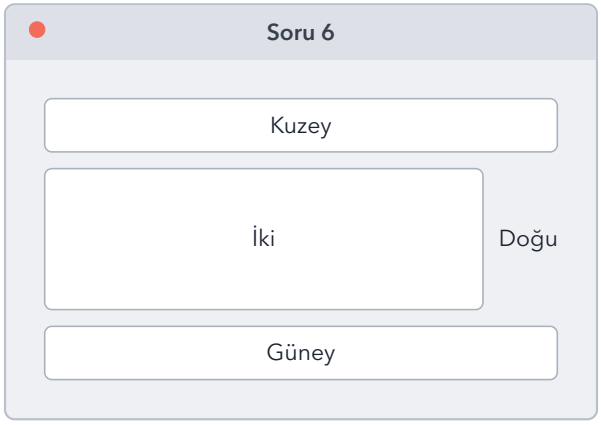
`Integer.parseInt` sayıya çevrilemeyen bir metinle karşılaşınca `NumberFormatException` fırlatır; bu yüzden `try-catch` ile yakalanmalıdır. `InputMismatchException` (B), `Scanner`'ın `nextInt()` metodunun fırlattığı exception'dır. Kod sorunsuz derlenir (C); hata çalışma zamanında oluşur. `int` bir değişkene `null` atanamaz (E).

5 Cevap: B

Swing bileşenleri thread-safe değildir; arayüzü oluşturan ve değiştiren her kod **EDT**'de çalışmalıdır. `invokeLater`, verilen kodu EDT'nin olay kuyruğuna ekler. Programı hızlandırmaz (A); tam tersi, bütün arayüz işi **tek** bir iş parçacığında yapılır. C `setLocationRelativeTo`'nun işidir. Bekleme süresi yoktur (D), "later" kelimesi "EDT sırası gelince" demektir.

6

- `JFrame` **BorderLayout** kullanır. `Kuzey` üst bölgeyi (NORTH), `Güney` alt bölgeyi (SOUTH) tam genişlikte kaplar.
- `Bir` ve `İki` bölge belirtilmeden eklendi, ikisi de **CENTER**'a gider. Her bölgede tek bileşen olabildiği için `İki`, `Bir`'in yerini alır: **`Bir` görünmez**.
- `Doğu` bir etikettir, EAST bölgesine yerleşir; sadece yazısı kadar yer kaplar.
- WEST bölgesine bir şey eklenmediği için yer kaplamaz; `İki` düğmesi sola kadar uzanır.



7

- `ust` paneli `new JPanel()` ile oluşturuldu; varsayılan yerleşimi **FlowLayout**: `Ad:`, metin kutusu ve `Ara` yan yana, ortalı dizilir. Panel NORTH'a konduğu için pencerenin üstündedir.
- `tuslar` paneli `GridLayout(2, 3)`: 2 satır, 3 sütun. Döngü **6'dan 1'e** saydığı için düğmeler eklenme sırasıyla satır satır yerleşir: üst satır `6 5 4`, alt satır `3 2 1`. Panel bölgesiz eklendiği için CENTER'dadır ve ortadaki bütün alanı kaplar; düğmeler eşit boyuttadır.
- `Tamam` SOUTH bölgesinde, tam genişlikte.



8

Bu simülasyonda `tikla()`, dinleyicileri **eklenme sırasıyla** çağırıyor: önce puanı 10 artıran lambda, sonra yazdıran anonim sınıf, en son düğmeyi pasif yapabilen lambda. Anonim sınıfın kendi `sayac` alanı var; her tıklamada o da artıyor.

Tıklama	puan	Yazdırılan	aktif
1	10	`#1 puan=10`	true
2	20	`#2 puan=20`	true
3	30	`#3 puan=30`	`puan >= 30` → **false**
4	30	`pasif` (dinleyiciler çalışmaz)	false

Çıktı

```
#1 puan=10
#2 puan=20
#3 puan=30
pasif
Son puan: 30
```

Not: Gerçek Swing'de bir düğmeye birden fazla dinleyici eklendiğinde çağrılma sırası **garanti edilmez**. Birbirinin sonucuna bağlı dinleyiciler yazma; gerekiyorsa işleri tek bir dinleyicide sırayla yap.

9

Dinleyiciyi bir `ActionListener` değişkeninde bir kez oluşturup üç düğmeye de veriyoruz. Hangi düğmeye basıldığını `e.getActionCommand()` (düğmenin yazısı) söylüyor; `switch` ifadesiyle yazıya karşılık gelen rengi seçiyoruz. `setTitle`, `JFrame`'den miras alınan metottur.

RenkSecici.java

```java
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.event.ActionListener;
import javax.swing.*;

public class RenkSecici extends JFrame {
    private final JPanel tuval = new JPanel();

    public RenkSecici() {
        super("Renk Seçici");
        ActionListener dinleyici = e -> {          // üç düğmenin ortak dinleyicisi
            String secilen = e.getActionCommand(); // basılan düğmenin yazısı
            Color renk = switch (secilen) {
                case "Kırmızı" -> Color.RED;
                case "Yeşil" -> Color.GREEN;
                default -> Color.BLUE;
            };
            tuval.setBackground(renk);
            setTitle("Renk: " + secilen);
        };

        JPanel ust = new JPanel();                 // FlowLayout
        String[] renkler = {"Kırmızı", "Yeşil", "Mavi"};
        for (String r : renkler) {
            JButton b = new JButton(r);
            b.addActionListener(dinleyici);
            ust.add(b);
        }
        add(ust, BorderLayout.NORTH);
        add(tuval, BorderLayout.CENTER);
        setSize(300, 200);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new RenkSecici().setVisible(true));
    }
}
```



Taslak, **Mavi**'ye basıldıktan sonraki hali gösteriyor.

10

Kelime sayımı için baştaki/sondaki boşlukları `trim()` ile at; metin boş kaldıysa 0 kelime vardır. Değilse `split("\s+")` ile **bir veya daha fazla** boşluk karakterinden (boşluk, tab, satır sonu) böl; parça sayısı kelime sayısıdır. Uzun metin alanının kaydırılabilmesi için `JTextArea`'yı bir `JScrollPane` içine koyduk.

KelimeSayaci.java

```
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import javax.swing.*;

public class KelimeSayaci extends JFrame {
    private final JTextArea alan = new JTextArea(6, 30);
    private final JLabel bilgi = new JLabel("Kelime: 0 | Karakter: 0");

    static int kelimeSay(String metin) {
        String temiz = metin.trim();
        if (temiz.isEmpty()) {
            return 0;
        }
        return temiz.split("\\s+").length;
    }

    public KelimeSayaci() {
        super("Kelime Sayacı");
        alan.setLineWrap(true);                    // satır sonunda kaydır
        JButton say = new JButton("Say");
        say.addActionListener(e -> {
            String metin = alan.getText();
            bilgi.setText("Kelime: " + kelimeSay(metin)
                    + " | Karakter: " + metin.length());
        });

        JPanel alt = new JPanel(new FlowLayout());
        alt.add(say);
        alt.add(bilgi);
        add(new JScrollPane(alan), BorderLayout.CENTER);
        add(alt, BorderLayout.SOUTH);

        pack();                                    // boyutu içeriğe göre ayarla
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new KelimeSayaci().setVisible(true));
    }
}
```



`kelimeSay` metodunu konsolda deneyelim:

KelimeSayTest.java

```
public class KelimeSayTest {
    static int kelimeSay(String metin) {
        String temiz = metin.trim();
        if (temiz.isEmpty()) {
            return 0;
        }
        return temiz.split("\\s+").length;
    }

    public static void main(String[] args) {
        String[] girdiler = {"", "   ", "Java öğreniyorum", "  bir  iki\n üç  "};
        for (String g : girdiler) {
            String gorunen = g.replace("\n", "\\n");    // satır sonunu görünür yap
            System.out.println("[" + gorunen + "] -> " + kelimeSay(g) + " kelime");
        }
    }
}
```

Çıktı

```
[] -> 0 kelime
[   ] -> 0 kelime
[Java öğreniyorum] -> 2 kelime
[  bir  iki\n üç  ] -> 3 kelime
```

`trim()` olmasaydı `" bir iki"` gibi boşlukla başlayan metinde `split` ilk eleman olarak **boş bir String** üretir ve kelime sayısı bir fazla çıkardı.

DENEME SINAVI

Deneme Final Sınavı

Bu deneme sınavı kitabın **tamamını** kapsar; ama ağırlık dönemin ikinci yarısındadır: sınıflar ve nesneler, kalıtım, polimorfizm, soyut sınıflar ve arayüzler, `enum` ve `record`, istisnalar, koleksiyonlar, dosya işlemleri, özyineleme, arama/sıralama ve Swing. İlk sekiz bölümün konuları (döngü, dizi, String) soruların içine gömülü olarak karşına çıkacak. Sorular, 1. sınıf final sınavlarının zorluğunda ve tuzaklarıyla hazırlandı.

Bölüm	Soru sayısı	Soru başına puan	Toplam
A) Çoktan seçmeli	12	2	24
B) Çıktı tahmini	5	8	40
C) Kod yazma	4	9	36
Toplam	**21**		**100**

Süre: 120 dakika. Bilgisayar, kitap ve not kullanmak yok; sadece kâğıt ve kalem.

> **İpucu: Önce süre tutarak çöz**
>
> 120 dakikalık bir sayaç kur ve süre bitene kadar çözümlere bakma. Kabaca **20 dakikayı** testlere, **45 dakikayı** çıktı sorularına, **55 dakikayı** kod yazmaya ayır. Çıktı sorularında sınıf hiyerarşisi varsa önce kâğıda küçük bir sınıf şeması çiz: hangi sınıf hangisinden türüyor, hangi metodu kim override ediyor. Bu iki dakika, sonrasında çok zaman kazandırır.

> **Not: Kendini nasıl puanlarsın?**
>
> - **Çıktı sorularında** çıktının her satırı eşit puan değerindedir. Boşluk, satır sonu ve `5 / 5.0` farkı önemlidir.
> - **Kod sorularında** çözümdeki program tek doğru cevap değildir. Senin kodun istenen yapıyı (sınıf, arayüz, özyineleme...) kullanıyor ve aynı sonucu veriyorsa tam puan al. İstenen yapıyı kullanmadıysan (ör. özyineleme yerine döngü) yarım puan al; her derleme hatası için 1 puan kır.
> - Her cevabın sonunda konunun anlatıldığı bölüm yazıyor. Yanlış yaptığın konularda o bölümün **Sınavda Çıkar** ve **Sık Yapılan Hata** kutularını tekrar oku.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod çalıştırıldığında ne yazdırılır?

```
class Sayac {
    static int toplam = 0;
    int adet = 0;

    Sayac() {
        toplam++;
        adet++;
    }
}

// ...

Sayac a = new Sayac();
Sayac b = new Sayac();
Sayac c = b;
c.adet += 5;
System.out.println(Sayac.toplam + " " + a.adet + " " + b.adet);
```

(A) `3 1 6` (B) `2 1 6` (C) `2 6 6` (D) `3 1 1` (E) `2 1 1`

2 Aşağıdaki kodda `new B();` satırı ne yazdırır?

```
class A {
    A() { System.out.print("A"); }
    A(int x) { System.out.print("a" + x); }
}

class B extends A {
    B() { this(5); System.out.print("B"); }
    B(int x) { super(x); System.out.print("b"); }
}
```

(A) `AB` (B) `Ba5b` (C) `Aa5bB` (D) `a5bB` (E) `AbB`

3 Soyut sınıflar ve arayüzler için aşağıdakilerden hangisi **doğrudur**?

(A) `abstract` bir sınıftan `new` ile nesne oluşturulabilir.

(B) Bir sınıf `extends` ile birden fazla sınıftan kalıtım alabilir.

(C) Bir sınıf birden fazla arayüzü `implements` edebilir.

(D) Soyut bir sınıfın yapıcı metodu (constructor) olamaz.

(E) Bir arayüzdeki gövdesiz metotlar varsayılan olarak `private`'tır.

4 Aşağıdaki program ne yazdırır?

```
enum Seviye { DUSUK, ORTA, YUKSEK }

public class Test4 {
    public static void main(String[] args) {
        Seviye s = Seviye.valueOf("ORTA");
        System.out.println(s.ordinal() + " " + Seviye.values().length
                + " " + (s == Seviye.ORTA));
    }
}
```

(A) `1 3 true` (B) `2 3 true` (C) `1 2 false` (D) `ORTA 3 true` (E) `1 3 false`

5 Aşağıdaki kod parçası için hangisi doğrudur?

```
Object o = "123";
Integer i = (Integer) o;
System.out.println(i + 1);
```

(A) `124` yazdırır.

(B) `1231` yazdırır.

(C) Derleme hatası verir.

(D) Çalışırken `NumberFormatException` fırlatır.

(E) Çalışırken `ClassCastException` fırlatır.

6 Aşağıdakilerden hangisi **checked** bir exception'dır (derleyici, yakalanmasını veya `throws` ile bildirilmesini zorunlu tutar)?

(A) `NullPointerException`

(B) `ArithmeticException`

(C) `ArrayIndexOutOfBoundsException`

(D) `IOException`

(E) `NumberFormatException`

7 Aşağıdaki kod parçası için hangisi doğrudur?

```
try {
    int x = 10 / 0;
} catch (Exception e) {
    System.out.println("Genel");
} catch (ArithmeticException e) {
    System.out.println("Aritmetik");
}
```

(A) `Genel` yazdırır.

(B) `Aritmetik` yazdırır.

(C) Önce `Genel`, sonra `Aritmetik` yazdırır.

(D) Program `ArithmeticException` ile çöker.

(E) Derleme hatası verir.

8 Aşağıdaki kod parçası ne yazdırır?

```
ArrayList<Integer> liste = new ArrayList<>(List.of(10, 20, 30, 40));
liste.remove(1);
liste.remove(Integer.valueOf(40));
liste.add(1, 5);
System.out.println(liste + " " + liste.size());
```

A) `[10, 5, 30] 3`

B) `[5, 20, 30] 3`

C) `[10, 5, 20, 30] 4`

D) `[10, 30, 5] 3`

E) Çalışırken `IndexOutOfBoundsException` fırlatır.

9 `notlar.txt` dosyasında daha önceden 5 satır var. Aşağıdaki kod çalıştıktan sonra dosyada ne olur?

```
try (PrintWriter pw = new PrintWriter(new FileWriter("notlar.txt"))) {
    pw.println("Ali 85");
}
```

A) Dosyanın sonuna `Ali 85` satırı eklenir; dosya 6 satır olur.

B) Dosyanın eski içeriği silinir; dosyada sadece `Ali 85` kalır.

C) Dosya zaten var olduğu için `IOException` fırlatılır.

D) `close()` çağrılmadığı için dosyaya hiçbir şey yazılmaz.

E) İlk satırın yerine `Ali 85` yazılır, diğer 4 satır korunur.

10 Aşağıdaki metot `f(5)` ile çağrılırsa ne olur?

```
static int f(int n) {
    if (n == 0) {
        return 0;
    }
    return n + f(n - 2);
}
```

A) `9` döndürür.

B) `15` döndürür.

C) Program `StackOverflowError` ile sonlanır.

D) `0` döndürür.

E) Derleme hatası verir.

11 Aşağıdaki kod parçasının zaman karmaşıklığı nedir?

```
for (int i = 1; i < n; i *= 2) {
    for (int j = 0; j < n; j++) {
        toplam++;
    }
}
```

(A) O(n log n) (B) O(log n) (C) $O(n^2)$ (D) O(n) (E) $O(2^n)$

12 Swing'de bir `JButton`'a tıklandığında çalışacak kodu bağlamak için hangisi kullanılır?

(A) `buton.setOnAction(...)`
(B) `buton.onClick(...)`
(C) `buton.setListener(...)`
(D) `buton.addEventHandler(...)`
(E) `buton.addActionListener(...)`

B) Çıktı Tahmini

13 Aşağıdaki program ekrana ne yazdırır?

```
abstract class Calisan {
    static int sayi = 0;
    protected String ad;

    Calisan(String ad) {
        this.ad = ad;
        sayi++;
    }

    abstract double maas();

    String unvan() {
        return "Çalışan";
    }

    @Override
    public String toString() {
        return unvan() + " " + ad + " " + maas();
    }
}

class Memur extends Calisan {
    Memur(String ad) {
        super(ad);
    }

    @Override
    double maas() {
        return 1000;
    }
}

class Mudur extends Memur {
    Mudur(String ad) {
        super(ad);
```

```
    }

    @Override
    double maas() {
        return super.maas() * 2 + 500;
    }

    @Override
    String unvan() {
        return "Müdür";
    }
}

public class Soru13 {
    static void yaz(Calisan c) {
        System.out.println("C: " + c);
    }

    static void yaz(Memur m) {
        System.out.println("M: " + m);
    }

    public static void main(String[] args) {
        Calisan[] dizi = {new Memur("Ali"), new Mudur("Ece")};
        for (Calisan c : dizi) {
            yaz(c);
        }
        Memur m = new Mudur("Can");
        yaz(m);
        Calisan k = m;
        System.out.println(k.unvan() + " " + (k instanceof Memur) + " "
                + Calisan.sayi);
    }
}
```

14 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru14 {
    static int f(String s) {
        try {
            int n = Integer.parseInt(s);
            return 100 / n;
        } catch (NumberFormatException e) {
            System.out.print("NFE ");
            return -1;
        } finally {
            System.out.print("F ");
        }
    }

    public static void main(String[] args) {
        System.out.println(f("20"));
        System.out.println(f("abc"));
        try {
            System.out.println(f("0"));
        } catch (ArithmeticException e) {
            System.out.println("AE");
        }
        System.out.println("son");
    }
}
```

15 Aşağıdaki program ekrana ne yazdırır?

```
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

record Ogrenci(String ad, int not) {}

public class Soru15 {
    public static void main(String[] args) {
        List<Ogrenci> liste = new ArrayList<>();
        liste.add(new Ogrenci("Ece", 85));
        liste.add(new Ogrenci("Ali", 70));
        liste.add(new Ogrenci("Can", 85));
        liste.add(new Ogrenci("Ayşe", 92));

        liste.sort((a, b) -> Integer.compare(b.not(), a.not()));
        for (Ogrenci o : liste) {
            System.out.print(o.ad() + " ");
        }
        System.out.println();
        System.out.println(liste.get(0));
        System.out.println(liste.contains(new Ogrenci("Ali", 70)) + " "
                + liste.indexOf(new Ogrenci("Can", 85)));

        Map<Character, Integer> harf = new TreeMap<>();
        for (Ogrenci o : liste) {
            char c = o.ad().charAt(0);
            harf.put(c, harf.getOrDefault(c, 0) + 1);
        }
        System.out.println(harf);
    }
}
```

16 Aşağıdaki program ekrana ne yazdırır?

```
public class Soru16 {
    static int sayac = 0;

    static int f(int n) {
        sayac++;
        if (n <= 1) {
            return n;
        }
        return f(n - 1) + f(n - 2);
    }

    static String ters(String s) {
        if (s.length() <= 1) {
            return s;
        }
        return ters(s.substring(1)) + s.charAt(0);
    }

    static void yaz(int n) {
        if (n == 0) {
            return;
        }
        System.out.print(n + " ");
        yaz(n - 1);
        System.out.print(n + " ");
    }

    public static void main(String[] args) {
        System.out.println(f(5) + " " + sayac);
        System.out.println(ters("final"));
        yaz(3);
        System.out.println();
    }
}
```

17 Aşağıdaki program ekrana ne yazdırır?

```
import java.util.Arrays;

public class Soru17 {
    static int ikiliAra(int[] d, int x) {
        int alt = 0, ust = d.length - 1;
        while (alt <= ust) {
            int orta = alt + (ust - alt) / 2;
            System.out.print(d[orta] + " ");
            if (d[orta] == x) {
                return orta;
            }
            if (d[orta] < x) {
                alt = orta + 1;
            } else {
                ust = orta - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] d = {5, 1, 4, 2, 8};
```

```
        for (int tur = 1; tur < d.length; tur++) {
            boolean degisti = false;
            for (int i = 0; i < d.length - tur; i++) {
                if (d[i] > d[i + 1]) {
                    int gecici = d[i];
                    d[i] = d[i + 1];
                    d[i + 1] = gecici;
                    degisti = true;
                }
            }
            System.out.println(tur + ". tur: " + Arrays.toString(d));
            if (!degisti) {
                break;
            }
        }
        int[] s = {3, 8, 15, 21, 34, 47, 52, 60, 71};
        System.out.println("-> " + ikiliAra(s, 52));
        System.out.println("-> " + ikiliAra(s, 10));
    }
}
```

C) Kod Yazma

18 Bir kesri temsil eden `Kesir` sınıfını yaz:

- `private` ve değiştirilemez (`final`) iki alan: `pay`, `payda`.
- Yapıcı: Payda 0 ise `"Payda sıfır olamaz"` mesajıyla `IllegalArgumentException` fırlatsın. Payda negatifse işareti paya taşısın. Kesri **sadeleştirerek** saklasın; EBOB'u **özyinelemeli** bir `private static int ebob(int a, int b)` metoduyla bulsun.
- `Kesir topla(Kesir k)` ve `Kesir carp(Kesir k)`: Sonucu **yeni** bir `Kesir` olarak döndürsün; mevcut nesneler değişmesin.
- `toString()`: `pay/payda` biçiminde.

`KesirTest` sınıfındaki `main` ile test et. Beklenen çıktı:

```
2/4 -> 1/2
1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
3/-6 -> -1/2
0/5 -> 0/1
Hata: Payda sıfır olamaz
```

19 Aşağıdaki sınıf yapısını kur:

- `Olceklenebilir` arayüzü: `void olcekle(double k)` metodu.
- Soyut `Sekil` sınıfı: `ad` alanı; soyut `alan()` ve `cevre()` metotları; adı, alanı ve çevreyi tek satırda yazan `toString()`.
- `Daire` (yarıçap): `Sekil`'den türer ve `Olceklenebilir`'i uygular (yarıçapı `k` katına çıkarır).
- `Dikdortgen` (en, boy): `Sekil`'den türer. `Kare` (kenar): `Dikdortgen`'den türer.

`SekilTest` sınıfının `main` metodunda bir `ArrayList<Sekil>` içine 1.5 yarıçaplı bir daire, 4 × 2.5'lik bir dikdörtgen ve 3 kenarlı bir kare koy. Listedeki **ölçeklenebilir** şekilleri 2 kat büyüt; sonra her şekli, toplam alanı ve alanı en büyük şekli yazdır. Beklenen çıktı:

```
Daire      alan=  28.27 çevre= 18.85
Dikdörtgen alan=  10.00 çevre= 13.00
Kare       alan=   9.00 çevre= 12.00
Toplam alan: 47.27
En büyük: Daire      alan=  28.27 çevre= 18.85
```

20 `metin.txt` dosyasını okuyan `KelimeSayac` programını yaz. Dosyada noktalama işareti yok; kelimeler bir veya daha fazla boşlukla ayrılmış. Program:

1. her kelimenin kaç kez geçtiğini **büyük/küçük harf farkı gözetmeden** saysın ve kelimeleri sıralı yazdırsın (`TreeMap` kullan),
2. toplam kelime sayısını, farklı kelime sayısını ve en sık geçen kelimeyi yazdırsın,
3. kelime–sayı listesini `rapor.txt` dosyasına yazsın.

Dosya yoksa program çökmesin, `Dosya bulunamadı: metin.txt` yazıp bitsin. Kullanılacak dosya:

metin.txt

```
Java nesne yönelimli bir dildir
java ile nesne oluşturmak kolaydır
sınıf nesnenin şablonudur
her nesne bir sınıfın örneğidir
```

21 Kullanıcıdan önce eleman sayısını, sonra o kadar tam sayıyı okuyan `SiralaAra` programını yaz:

- `static void secmeliSirala(int[] d)`: Diziyi **seçmeli sıralama** (selection sort) ile küçükten büyüğe sıralar.
- `static int ikiliAra(int[] d, int x, int alt, int ust)`: Sıralı dizide `x`'i **özyinelemeli ikili arama** ile arar; bulursa indeksini, bulamazsa -1 döndürür.
- Her iki metodun zaman karmaşıklığını (Big-O) metodun üstüne yorum olarak yaz.

Program sıralı diziyi yazdırsın, sonra kullanıcı -1 girene kadar aranacak sayıları okuyup sonucu yazsın. Örnek çalışma:

```
Kaç sayı? 7
Sayılar: 42 17 8 99 23 4 15
Sıralı: [4, 8, 15, 17, 23, 42, 99]
Aranacak (bitirmek için -1): 23
23 bulundu, indeks 4
Aranacak (bitirmek için -1): 50
50 bulunamadı
Aranacak (bitirmek için -1): 4
4 bulundu, indeks 0
Aranacak (bitirmek için -1): -1
```

Çözümler

Cevap anahtarı

1 B	2 D	3 C	4 A	5 E	6 D	7 E	8 A
9 B	10 C	11 A	12 E				

1 Cevap: B

- `toplam` **static**'tir: Bütün nesneler tek bir kopyayı paylaşır. `new` iki kez çağrıldı, yani `toplam` = 2. `Sayac c = b;` yeni nesne oluşturmaz, yapıcı çalışmaz; A ve D bu tuzağa düşüyor.
- `adet` her nesnenin kendi kopyasıdır. `c` ile `b` **aynı nesneyi** gösterdiği için `c.adet += 5` → `b.adet` = 6. `a`'nın kendi `adet`'i 1'de kalır.

Çıktı
```
2 1 6
```

Konu: Bölüm 9.

2 Cevap: D

`new B()` → `B()` çalışır. İlk satırı `this(5)` olduğu için **aynı sınıftaki** `B(int)` çağrılır. `B(int)`'in ilk satırı `super(x)`: `A(int)` çalışır ve `a5` yazar. Sonra `B(int)`'in gövdesi `b`'yi, en son `B()`'nin gövdesi `B`'yi yazar.

`A()` hiç çalışmaz: Bir yapıcı `this(...)` veya `super(...)` ile başlıyorsa, derleyici gizli `super()` eklemez. C şıkkı bunu kaçırıyor. Kural: Önce üst sınıfın yapıcısı **biter**, sonra alt sınıfınki devam eder.

Çıktı
```
a5bB
```

Konu: Bölüm 10.

3 Cevap: C

Java'da sınıflar arasında **tekli kalıtım** vardır (B yanlış), ama bir sınıf istediği kadar arayüz uygulayabilir: `class Ordek extends Hayvan implements Ucabilir, Yuzebilir`.

- A) Soyut sınıftan doğrudan nesne oluşturulamaz; sadece alt sınıflarından oluşturulur.
- D) Soyut sınıfın yapıcısı **olabilir**; alt sınıfın yapıcısı `super(...)` ile onu çağırır.
- E) Arayüzdeki gövdesiz metotlar varsayılan olarak `public abstract`'tır.

Konu: Bölüm 11.

4 Cevap: A

- `valueOf("ORTA")` String'den aynı adlı enum sabitini bulur.
- `ordinal()` sabitin tanımlanma sırasını **0'dan başlayarak** verir: `DUSUK` 0, `ORTA` 1.
- `values()` bütün sabitleri bir dizi olarak döndürür; uzunluğu 3.
- Her enum sabitinden tek bir nesne vardır; bu yüzden enum'lar `==` ile güvenle karşılaştırılır: `true`.

Çıktı
```
1 3 true
```

Konu: Bölüm 11.

5 Cevap: E

`Object` → `Integer` aşağı dönüşümü (*downcasting*) derleyiciye göre mümkündür; çünkü bir `Object` referansı gerçekten bir `Integer`'ı gösteriyor olabilir. Bu yüzden program **derlenir** (C yanlış). Ama çalışırken nesnenin gerçek tipine bakılır: Nesne bir `String`'dir ve `Integer`'a dönüştürülemez.

Cast, metni sayıya **çevirmez**; sadece "bu nesne zaten o tipte" der. Metni sayıya çevirmek için `Integer.parseInt((String) o)` gerekirdi. D şıkkı bu iki işlemi karıştırıyor.

Çıktı – program hata ile durur
```
Exception in thread "main" java.lang.ClassCastException: class java.lang.String cannot be cast to
class java.lang.Integer (java.lang.String and java.lang.Integer are in module java.base of loader
'bootstrap')
        at Test5.main(Test5.java:5)
```

Konu: Bölüm 11.

6 Cevap: D

`IOException`, `Exception`'dan türeyen ama `RuntimeException`'dan **türemeyen** bir sınıftır; yani checked'dır. Dosya açan bir metodu çağırırken ya `try–catch` yazmak ya da `throws IOException` eklemek zorundasın.

Diğer dördü `RuntimeException`'ın alt sınıflarıdır (unchecked): Derleyici onlar için bir şey zorunlu tutmaz, çünkü genellikle programcı hatasından kaynaklanırlar ve kodu düzelterek önlenirler.

Konu: Bölüm 12.

7 Cevap: E

`catch` blokları yukarıdan aşağıya denenir. `Exception`, `ArithmeticException`'ın üst sınıfı olduğu için her aritmetik hatayı zaten ilk blok yakalar; ikinci blok **hiçbir zaman** çalışamaz. Java ulaşılamayan `catch` bloğunu derleme hatası sayar:

Derleyici çıktısı – program derlenmez
```
Test7.java:8: error: exception ArithmeticException has already been caught
        } catch (ArithmeticException e) {
          ^
1 error
```

Kural: Özel (alt sınıf) exception'lar **önce**, genel olanlar **sonra** yazılır. Sıra ters çevrilseydi program `Aritmetik` yazardı.

Konu: Bölüm 12.

8 Cevap: A

`ArrayList<Integer>`'da `remove` metodunun iki hali vardır:

- `remove(1)`: Argüman `int` olduğu için **indeks** sürümü seçilir; 1. indeksteki `20` silinir → `[10, 30, 40]`. (C şıkkı, 1 değerini aradığını sanıyor.)

- `remove(Integer.valueOf(40))`: Argüman bir nesne olduğu için **değer** sürümü seçilir; `40` silinir → `[10, 30]`. (E şıkkı bunu indeks 40 sanıyor.)
- `add(1, 5)`: 1. indekse ekler, sonrakiler kayar → `[10, 5, 30]`.

```
Çıktı
[10, 5, 30] 3
```

Konu: Bölüm 13.

9 Cevap: B

`new FileWriter("notlar.txt")` dosyayı **yazma** modunda açar: Dosya varsa içeriği silinir (*overwrite*), yoksa oluşturulur. Sona eklemek için ikinci parametre `true` verilmelidir: `new FileWriter("notlar.txt", true)`.

D yanlıştır: `try (...)` (try-with-resources) bloğu bitince `close()` **otomatik** çağrılır. Programın sonunda dosya okunup yazdırıldığında:

```
Çıktı
[Ali 85]
```

Konu: Bölüm 14.

10 Cevap: C

`f(5)` → `f(3)` → `f(1)` → `f(-1)` → `f(-3)` → … `n` her adımda 2 azalır ve tek sayılardan geçer; **hiçbir zaman 0 olmaz**. Temel duruma (*base case*) ulaşılamadığı için çağrılar yığını doldurur ve JVM `StackOverflowError` fırlatır. Çift sayılarla (`f(6) = 6 + 4 + 2 + 0 = 12`) metot düzgün çalışır.

```
Çıktı
f(6) = 12
f(5): java.lang.StackOverflowError
```

Temel durumu `n <= 0` yazmak hatayı giderirdi. Derleyici sonsuz özyinelemeyi fark etmez (E yanlış).

Konu: Bölüm 15.

11 Cevap: A

- Dış döngüde `i` her adımda **iki katına** çıkar: 1, 2, 4, 8, … `n`'ye ulaşana kadar yaklaşık **log₂ n** kez döner.
- İç döngü her seferinde **n** kez döner.
- Toplam: n · log n → **O(n log n)**.

`n = 1024` için dış döngü 10 kez ($\log_2 1024 = 10$), iç döngü 1024 kez döner:

```
Çıktı
n = 16 -> 64 adım
n = 1024 -> 10240 adım
```

Dış döngü `i++` ile artsaydı $O(n^2)$ olurdu; C şıkkı bunu düşünüyor.

Konu: Bölüm 16.

12 Cevap: E

Swing'de düğme tıklaması bir `ActionEvent` üretir ve `addActionListener` ile kaydedilen dinleyicinin `actionPerformed` metodu çalışır. Lambda ile kısaca:

```
JButton buton = new JButton("Hesapla");
buton.addActionListener(e -> etiket.setText("Tıklandı!"));
```

A ve D JavaFX kütüphanesindeki adlardır, B ve C ise hiç yoktur.

Konu: Bölüm 17.

13

İki ayrı kuralı birlikte uygula:

- **Overload (aşırı yükleme) derleme zamanında**, referansın **tipine** göre seçilir. Döngüdeki `c`'nin tipi `Calisan` olduğu için iki eleman için de `yaz(Calisan)` çalışır (`C:`). `m`'nin tipi `Memur` olduğu için `yaz(Memur)` seçilir (`M:`), nesne `Mudur` olsa bile.
- **Override (metot ezme) çalışma zamanında**, nesnenin **gerçek** tipine göre seçilir (dinamik bağlama). `toString` içindeki `unvan()` ve `maas()` çağrıları nesnenin kendi sınıfındaki metotlara gider.

Nesne	`unvan()`	`maas()`
`Memur("Ali")`	`Calisan`'dan miras: `Çalışan`	`1000.0`
`Mudur(...)`	`Müdür`	`super.maas() * 2 + 500 = 2500.0`

`maas()` `double` döndürdüğü için `1000` değil `1000.0` yazılır. Son satır: `k`'nin tipi `Calisan` ama nesne `Mudur`: `unvan()` → `Müdür`; `Mudur` bir `Memur` olduğu için `instanceof` → `true`; `sayi` static'tir ve 3 nesne oluşturuldu.

Çıktı

```
C: Çalışan Ali 1000.0
C: Müdür Ece 2500.0
M: Müdür Can 2500.0
Müdür true 3
```

14

`finally` bloğu, `try` veya `catch` içinde `return` olsa bile, metottan çıkmadan **hemen önce** çalışır. `println(f(...))` satırında önce `f` çalışır (ve kendi yazdıklarını basar), sonra `println` dönen değeri yazar.

- `f("20")`: `100 / 20 = 5` döndürülecek; çıkmadan önce `finally` → `F`. Sonra `println` → `5`. Satır: `F 5`.
- `f("abc")`: `parseInt` → `NumberFormatException`, `catch` → `NFE`, dönüş −1 hazırlanır, `finally` → `F`. Satır: `NFE F -1`.
- `f("0")`: `100 / 0` → `ArithmeticException`. `f`'teki `catch` sadece `NumberFormatException` yakalar; exception metottan dışarı fırlar ama önce `finally` → `F`. `main`'deki `catch` yakalar: `AE`. `println(f("0"))` hiç çalışmaz.
- Exception yakalandığı için program devam eder: `son`.

Çıktı

```
F 5
NFE F -1
F AE
son
```

15

- `(a, b) -> Integer.compare(b.not(), a.not())` notları **büyükten küçüğe** sıralar (`b` ile `a` yer değiştirdiği için ters sıra). `List.sort` **kararlıdır** (*stable*): Eşit notlu Ece ve Can, listedeki eski sıralarını korur (Ece önce).
- `record` için `toString` otomatik üretilir: `Ogrenci[ad=Ayşe, not=92]`.
- `record` için `equals` de otomatik üretilir ve **alanları** karşılaştırır. Bu yüzden yeni oluşturulan `new Ogrenci("Ali", 70)` listede bulunur (`true`); `Can` sıralı listede 2. indekstedir.
- `TreeMap` anahtarları sıralı tutar: `A` (Ayşe, Ali) 2, `C` 1, `E` 1.

Çıktı
```
Ayşe Ece Can Ali
Ogrenci[ad=Ayşe, not=92]
true 2
{A=2, C=1, E=1}
```

16

`f` (Fibonacci) ve çağrı sayısı: `f(5) = 5`. Çağrı sayısı $C(n) = 1 + C(n-1) + C(n-2)$, $C(0) = C(1) = 1$: $C(2) = 3$, $C(3) = 5$, $C(4) = 9$, $C(5) = \mathbf{15}$. `f(5) + " " + sayac` ifadesinde önce `f(5)` hesaplanır, `sayac` ondan **sonra** okunur; bu yüzden 0 değil 15 yazılır.

`ters`: `ters("final")` = `ters("inal") + 'f'` = … = `"lanif"`.

`yaz`: Özyinelemeli çağrıdan **önceki** `print` inerken (3 2 1), **sonraki** `print` geri dönerken (1 2 3) çalışır.

Çıktı
```
5 15
lanif
3 2 1 1 2 3
```

17

Kabarcık sıralaması (bubble sort): Her turda komşular karşılaştırılır, büyük olan sağa kayar; tur sonunda en büyük eleman yerine oturur.

- 1. tur: `5↔1`, `5↔4`, `5↔2` yer değiştirir → `[1, 4, 2, 5, 8]`.
- 1. tur: `4↔2` → `[1, 2, 4, 5, 8]`.
- 1. tur: Hiç değişim yok; tur yazdırılır, sonra `break` ile çıkılır.

İkili arama: `ikiliAra` her adımda baktığı elemanı **kendisi** yazdırır. `"-> " + ikiliAra(...)` ifadesinde önce metot çalıştığı için ziyaret edilen elemanlar satırın başına, `->` ve sonuç sona yazılır.

- 52: `orta = 4` → 34 < 52, `alt = 5`; `orta = 6` → 52 bulundu, indeks 6.
- 10: `orta = 4` → 34 > 10, `ust = 3`; `orta = 1` → 8 < 10, `alt = 2`; `orta = 2` → 15 > 10, `ust = 1`; `alt > ust` → −1.

Çıktı
```
1. tur: [1, 4, 2, 5, 8]
2. tur: [1, 2, 4, 5, 8]
3. tur: [1, 2, 4, 5, 8]
34 52 -> 6
34 8 15 -> -1
```

18

Sadeleştirme yapıcıda bir kez yapılır; alanlar `final` olduğu için nesne bir daha değişmez. `topla` ve `carp` yeni `Kesir` oluşturduğundan sonuç da otomatik olarak sadeleşir. EBOB için Öklid algoritması: ebob(a, b) = ebob(b, a % b), ebob(a, 0) = a.

KesirTest.java

```
class Kesir {
    private final int pay;
    private final int payda;

    Kesir(int pay, int payda) {
        if (payda == 0) {
            throw new IllegalArgumentException("Payda sıfır olamaz");
        }
        if (payda < 0) {            // işareti paya taşı
            pay = -pay;
            payda = -payda;
        }
        int e = ebob(Math.abs(pay), payda);
        this.pay = pay / e;
        this.payda = payda / e;
    }

    private static int ebob(int a, int b) {
        if (b == 0) {
            return a;
        }
        return ebob(b, a % b);
    }

    Kesir topla(Kesir k) {
        return new Kesir(pay * k.payda + k.pay * payda, payda * k.payda);
    }

    Kesir carp(Kesir k) {
        return new Kesir(pay * k.pay, payda * k.payda);
    }

    @Override
    public String toString() {
        return pay + "/" + payda;
    }
}

public class KesirTest {
    public static void main(String[] args) {
        Kesir a = new Kesir(2, 4);
        Kesir b = new Kesir(1, 3);
        System.out.println("2/4 -> " + a);
        System.out.println(a + " + " + b + " = " + a.topla(b));
        System.out.println(a + " * " + b + " = " + a.carp(b));
        System.out.println("3/-6 -> " + new Kesir(3, -6));
        System.out.println("0/5 -> " + new Kesir(0, 5));
        try {
            new Kesir(5, 0);
        } catch (IllegalArgumentException e) {
            System.out.println("Hata: " + e.getMessage());
        }
    }
}
```

Çıktı

```
2/4 -> 1/2
1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
3/-6 -> -1/2
0/5 -> 0/1
Hata: Payda sıfır olamaz
```

`0/5` durumuna dikkat: ebob(0, 5) = 5 olduğu için kesir `0/1`'e sadeleşir. `topla` içinde `k.payda` gibi başka bir nesnenin `private` alanına erişmek serbesttir; `private` sınıf düzeyinde bir kısıttır, nesne düzeyinde değil.

19

`Sekil` soyut olduğu için `toString` gibi ortak davranışı bir kez yazar; `alan()` ve `cevre()` çağrıları dinamik bağlama ile her nesnenin kendi metoduna gider. `Kare` kendi alan/çevre formülünü yazmaz, `Dikdortgen`'inkini kullanır; adını üst sınıfa iletmek için `Dikdortgen`'e `protected` bir yapıcı ekledik. Listede sadece `Daire` ölçeklenebilir olduğu için `instanceof` kontrolü ve cast gerekir.

SekilTest.java

```
import java.util.ArrayList;
import java.util.List;

interface Olceklenebilir {
    void olcekle(double k);
}

abstract class Sekil {
    private final String ad;

    Sekil(String ad) {
        this.ad = ad;
    }

    abstract double alan();

    abstract double cevre();

    @Override
    public String toString() {
        return String.format("%-10s alan=%7.2f çevre=%6.2f", ad, alan(), cevre());
    }
}

class Daire extends Sekil implements Olceklenebilir {
    private double r;

    Daire(double r) {
        super("Daire");
        this.r = r;
    }

    @Override
    double alan() {
        return Math.PI * r * r;
    }

    @Override
    double cevre() {
        return 2 * Math.PI * r;
    }
```

```
    @Override
    public void olcekle(double k) {
        r *= k;
    }
}

class Dikdortgen extends Sekil {
    private final double en, boy;

    Dikdortgen(double en, double boy) {
        this("Dikdörtgen", en, boy);
    }

    protected Dikdortgen(String ad, double en, double boy) {
        super(ad);
        this.en = en;
        this.boy = boy;
    }

    @Override
    double alan() {
        return en * boy;
    }

    @Override
    double cevre() {
        return 2 * (en + boy);
    }
}

class Kare extends Dikdortgen {
    Kare(double kenar) {
        super("Kare", kenar, kenar);
    }
}

public class SekilTest {
    public static void main(String[] args) {
        List<Sekil> sekiller = new ArrayList<>();
        sekiller.add(new Daire(1.5));
        sekiller.add(new Dikdortgen(4, 2.5));
        sekiller.add(new Kare(3));

        for (Sekil s : sekiller) {
            if (s instanceof Olceklenebilir) {
                ((Olceklenebilir) s).olcekle(2);
            }
        }

        double toplam = 0;
        Sekil enBuyuk = sekiller.get(0);
        for (Sekil s : sekiller) {
            System.out.println(s);
            toplam += s.alan();
            if (s.alan() > enBuyuk.alan()) {
                enBuyuk = s;
            }
        }
        System.out.printf("Toplam alan: %.2f%n", toplam);
        System.out.println("En büyük: " + enBuyuk);
    }
}
```

Çıktı

```
Daire      alan=  28.27 çevre= 18.85
Dikdörtgen alan=  10.00 çevre= 13.00
Kare       alan=   9.00 çevre= 12.00
Toplam alan: 47.27
En büyük: Daire      alan=  28.27 çevre= 18.85
```

`olcekle` metodu arayüzde örtük olarak `public` olduğu için `Daire`'de de `public` yazılmak **zorundadır**; yazmazsan erişimi daraltmış olursun ve program derlenmez.

20

`Scanner`'ı bir `File` ile oluşturursan klavye yerine dosyadan okur; `next()` boşluklarla ayrılmış kelimeleri tek tek verir, birden fazla boşluk sorun olmaz. Dosya açma `FileNotFoundException` (checked) fırlatabildiği için `try-catch` şarttır; try-with-resources dosyayı otomatik kapatır.

KelimeSayac.java

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintWriter;
import java.util.Locale;
import java.util.Map;
import java.util.Scanner;
import java.util.TreeMap;

public class KelimeSayac {
    public static void main(String[] args) {
        Locale tr = Locale.forLanguageTag("tr-TR");
        Map<String, Integer> sayac = new TreeMap<>();
        int toplam = 0;

        try (Scanner dosya = new Scanner(new File("metin.txt"))) {
            while (dosya.hasNext()) {
                String kelime = dosya.next().toLowerCase(tr);
                sayac.put(kelime, sayac.getOrDefault(kelime, 0) + 1);
                toplam++;
            }
        } catch (FileNotFoundException e) {
            System.out.println("Dosya bulunamadı: metin.txt");
            return;
        }
        if (sayac.isEmpty()) {           // boş dosya: "En sık:  (null)" olmasın
            System.out.println("Dosyada kelime yok.");
            return;
        }

        String enSik = "";
        for (String k : sayac.keySet()) {
            System.out.printf("%-11s %d%n", k, sayac.get(k));
            if (enSik.isEmpty() || sayac.get(k) > sayac.get(enSik)) {
                enSik = k;
            }
        }
        System.out.println("Toplam: " + toplam + ", farklı: " + sayac.size());
        System.out.println("En sık: " + enSik + " (" + sayac.get(enSik) + ")");

        try (PrintWriter pw = new PrintWriter("rapor.txt")) {
            for (String k : sayac.keySet()) {
                pw.println(k + " " + sayac.get(k));
            }
            System.out.println("rapor.txt yazıldı.");
```

```
        } catch (FileNotFoundException e) {
            System.out.println("rapor.txt oluşturulamadı.");
        }
    }
}
```

Çıktı

```
bir         2
dildir      1
her         1
ile         1
java        2
kolaydır    1
nesne       3
nesnenin    1
oluşturmak  1
sınıf       1
sınıfın     1
yönelimli   1
örneğidir   1
şablonudur  1
Toplam: 18, farklı: 14
En sık: nesne (3)
rapor.txt yazıldı.
```

Dosya boşsa `sayac` da boş kalır; kontrol olmasaydı `enSik` boş metin olarak kalır ve `sayac.get("")` `null` döndüğü için ekrana `En sık: (null)` yazılırdı. Kelimeler Türkçe olduğu için küçültmede `Locale` verildi (Bölüm 8).

Sıralamaya dikkat: `TreeMap`, String'leri **Unicode kod numarasına** göre sıralar. `ö` ve `ş` harflerinin kodu `z`'den büyük olduğu için `örneğidir` ve `şablonudur` en sona düştü. Gerçek Türkçe alfabe sırası gerekiyorsa `java.text.Collator` kullanılır; sınavda bu beklenmez.

21

Seçmeli sıralama her turda sıralanmamış kısmın en küçüğünü bulup o kısmın başına koyar: iç içe iki döngü → $O(n^2)$. İkili arama her çağrıda arama aralığını yarıya indirir → $O(\log n)$. Özyinelemeli sürümde döngünün yerini, aralığı daraltarak kendini çağıran metot alır; `alt > ust` temel durumdur.

SiralaAra.java

```
import java.util.Arrays;
import java.util.Scanner;

public class SiralaAra {
    // O(n^2): n-1 tur, her turda kalan kısmın en küçüğü aranır
    static void secmeliSirala(int[] d) {
        for (int i = 0; i < d.length - 1; i++) {
            int enKucuk = i;
            for (int j = i + 1; j < d.length; j++) {
                if (d[j] < d[enKucuk]) {
                    enKucuk = j;
                }
            }
            int gecici = d[i];
            d[i] = d[enKucuk];
            d[enKucuk] = gecici;
        }
    }

    // O(log n): her çağrıda arama aralığı yarıya iner
```

```
    static int ikiliAra(int[] d, int x, int alt, int ust) {
        if (alt > ust) {
            return -1;
        }
        int orta = alt + (ust - alt) / 2;
        if (d[orta] == x) {
            return orta;
        } else if (d[orta] < x) {
            return ikiliAra(d, x, orta + 1, ust);
        } else {
            return ikiliAra(d, x, alt, orta - 1);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Kaç sayı? ");
        int n = sc.nextInt();
        int[] d = new int[n];
        System.out.print("Sayılar: ");
        for (int i = 0; i < n; i++) {
            d[i] = sc.nextInt();
        }
        secmeliSirala(d);
        System.out.println("Sıralı: " + Arrays.toString(d));

        System.out.print("Aranacak (bitirmek için -1): ");
        int x = sc.nextInt();
        while (x != -1) {
            int yer = ikiliAra(d, x, 0, d.length - 1);
            if (yer == -1) {
                System.out.println(x + " bulunamadı");
            } else {
                System.out.println(x + " bulundu, indeks " + yer);
            }
            System.out.print("Aranacak (bitirmek için -1): ");
            x = sc.nextInt();
        }
    }
}
```

Çıktı

```
Kaç sayı? 7
Sayılar: 42 17 8 99 23 4 15
Sıralı: [4, 8, 15, 17, 23, 42, 99]
Aranacak (bitirmek için -1): 23
23 bulundu, indeks 4
Aranacak (bitirmek için -1): 50
50 bulunamadı
Aranacak (bitirmek için -1): 4
4 bulundu, indeks 0
Aranacak (bitirmek için -1): -1
```

İkili arama **sadece sıralı** dizide çalışır; sıralamadan önce çağırsaydın yanlış sonuç alabilirdin. Özyinelemeli çağrıların önündeki `return`'leri unutmak da sık yapılan bir hatadır: Sonuç kaybolur ve metot derlenmez (*missing return statement*).

EK A

Java Cheat Sheet

Sınavdan önceki son tekrar için kitabın sıkıştırılmış hali; anlamadığın satırda ilgili bölüme dön.

```java
import java.util.Scanner;                  // import'lar sınıftan ÖNCE

public class Ana {                         // dosya adı: Ana.java
    public static void main(String[] args) {
        System.out.println("Merhaba");      // her komut ; ile biter
    }
}
```

Temeller

İlkel tipler

```
byte     1 B  −128 … 127
short    2 B  −32 768 … 32 767
int      4 B  ±2,1 milyar   (varsayılan)
long     8 B  8_000_000_000L
float    4 B  3.14f
double   8 B  3.14          (varsayılan)
char     2 B  'A'=65  'a'=97  '0'=48
boolean       true / false
```

- Otomatik: `byte → short → int → long → float → double`; ters yön cast ister: `(int) 3.99` → `3` (keser).
- Alan/dizi varsayılanı `0`, `false`, `null`; **yerel değişkenin varsayılanı yok**.

Operatör önceliği (yüksekten düşüğe)

```
1   ()  []  .   x++  x--
2   ++x  --x  -x  !  (tip)
3   *  /  %
4   +  -
5   <  <=  >  >=  instanceof
6   ==  !=
7   &&
8   ||
9   ?:
10  =  +=  -=  *=  /=  %=
```

Tuzaklar

- `7 / 2` → `3` · `7 / 2.0` → `3.5` · `-7 / 2` → `-3` · `-7 % 3` → `-1` · `(double) a / b` ≠ `(double) (a / b)`
- `x++` önce kullanır sonra artırır; `++x` önce artırır.
- `"a" + 1 + 2` → `"a12"` · `1 + 2 + "a"` → `"3a"`
- `byte b = 1; b += 5;` derlenir; `b = b + 5;` derlenmez.
- `&&`, `||` kısa devre yapar: Sol taraf sonucu belirliyorsa sağ taraf **çalışmaz**.
- `Integer.MAX_VALUE + 1` taşar (hata yok). `0.1 + 0.2 == 0.3` → `false`.

Girdi, çıktı ve kontrol

Scanner

```java
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
double d = sc.nextDouble();
String kelime = sc.next(); // tek kelime
sc.nextLine();             // kalan \n'i at
String satir = sc.nextLine();
char c = sc.next().charAt(0);
```

- `nextInt()`'ten hemen sonraki `nextLine()` boş String döndürür; araya `sc.nextLine();` koy. Yanlış tip → `InputMismatchException`.

printf (42, 1234567, 3.456, "Ali")

```
%d     42            %5d     ␣␣␣42
%-5d   42␣␣␣         %05d    00042
%,d    1,234,567     %+d     +42
%f     3.456000      %.2f    3.46
%8.2f  ␣␣␣␣3.46      %e      3.456000e+00
%s     Ali           %-6s    Ali␣␣␣
%c     A             %b      true
%n     satır sonu    %%      %
```

- `printf` alt satıra kendisi geçmez; `String.format` aynı biçimi String olarak döndürür. `%d`'ye `double` → `IllegalFormatConversionException`.

if, üçlü operatör, switch

```
if (not >= 85) harf = "A";
else if (not >= 70) harf = "B";
else harf = "F";   // {} yoksa tek komut
String s = n % 2 == 0 ? "çift" : "tek";
switch (gun) {              // klasik
    case 1:
    case 7:
        tur = "Tatil";
        break;        // yoksa alta düşer
    default:
        tur = "İş";
}
tur = switch (gun) {        // ok (->)
    case 1, 7 -> "Tatil";
    default -> "İş";
};
```

- `if (x > 0);` → `;` boş komuttur, blok her zaman çalışır. String içeriği `equals` ile karşılaştırılır.
- `switch`: `int`, `char`, `String`, `enum` olur; `double`, `long`, `boolean` olmaz.

Döngüler

```
for (int i = 0; i < n; i++) { ... }
while (kosul) { ... }       // 0+ kez
do { ... } while (kosul);   // en az 1 kez
for (int x : dizi) { ... }  // for-each
break;     // döngüden çık
continue;  // sonraki tura geç
```

Metotlar, diziler, String

Metot

```
static int topla(int a, int b) {
    return a + b;
}
static void yaz(String s) { } // değersiz
// overload: aynı ad, farklı parametre
static double topla(double a, double b)
```

- Java **değer ile** geçirir. Dizi parametresinde `d[0] = 5` çağıranın dizisini değiştirir, `d = new int[3]` değiştirmez.
- Değer döndüren metodun her yolu `return` ile bitmeli.

Diziler ve Arrays

```
int[] d = new int[5];      // {0,0,0,0,0}
int[] e = {3, 1, 2};
e.length                   // 3, () yok!
e[e.length - 1]            // son eleman
int[][] m = new int[3][4];
// m.length → 3, m[0].length → 4
int[] b = e;               // AYNI dizi
Arrays.toString(e)  // "[3, 1, 2]"
Arrays.copyOf(e, 2) // yeni: {3, 1}
Arrays.sort(e)      // e: {1, 2, 3}
Arrays.equals(e, b) // içerik karşılaştır
```

- Geçersiz indeks → `ArrayIndexOutOfBoundsException`. `d == e` içeriği değil referansı karşılaştırır.

String (s = "Merhaba")

```
s.length()              7
s.charAt(1)             'e'
s.substring(1, 4)       "erh"  (4 hariç)
s.substring(3)          "haba"
s.indexOf('a')          4   (yoksa -1)
s.lastIndexOf('a')      6
s.contains("hab")       true
s.toUpperCase()         "MERHABA"
s.replace('a', 'o')     "Merhobo"
"  x ".trim()           "x"
"a,b".split(",")        {"a", "b"}
s.equals("merhaba")     false
s.equalsIgnoreCase(..)  true
"Ali".compareTo("Ayse") -13  ('l'-'y')
Integer.parseInt("42")  42
```

```
var sb = new StringBuilder("ab");
sb.append(3).append('c');  // "ab3c"
sb.insert(0, "x");         // "xab3c"
sb.reverse();              // "c3bax"
String t = sb.toString();
```

- String **değiştirilemez**: `s.toUpperCase();` tek başına `s`'yi değiştirmez; `s = s.toUpperCase();` yaz. Değişen metin için `StringBuilder`.

Nesneye yönelik programlama

Sınıf iskeleti

```
public class Ogrenci {
    private static int sayac = 0; // ortak
    private final String ad;
    private int not;
    // yapıcı: dönüş tipi YOK
    public Ogrenci(String ad, int not) {
        this.ad = ad;    // alan = param.
        setNot(not);
        sayac++;
    }
    public Ogrenci(String ad) {
        this(ad, 0);     // ilk satırda
    }
    public String getAd() { return ad; }
    public void setNot(int not) {
        if (not >= 0 && not <= 100) {
            this.not = not;
        }
    }
    @Override
    public String toString() { return ad; }
}
```

- Hiç yapıcı yazmazsan varsayılan yapıcı eklenir; bir tane yazarsan eklenmez. `static` metot `this`'i ve nesne alanlarını göremez.

	Sınıf	Paket	Alt sınıf	Her yer
`private`	✓			
(yok)	✓	✓		
`protected`	✓	✓	✓	
`public`	✓	✓	✓	✓

Kalıtım ve polimorfizm

```
class Ogr extends Kisi {
    Ogr(String ad) {
        super(ad); // üst yapıcı (klasik
kural: 1. satır)
    }
    @Override
    String tanit() {
        return super.tanit() + " (öğr.)";
    }
}
Kisi h = new Ogr("Ali"); // yukarı
h.tanit();               // Ogr'unki
Ogr o = (Ogr) h;         // aşağı
```

- Tek sınıftan `extends`; yapıcılar miras alınmaz; yazmazsan gizli `super()` çağrılır.
- **Overload** (aynı ad, farklı parametre) derlemede referans tipine göre; **override** (alt sınıfta aynı imza) çalışırken nesne tipine göre seçilir.
- Yanlış aşağı dönüşüm → `ClassCastException` (önce `instanceof`). Alanlar ve `static` metotlar polimorfik değildir.

Soyut sınıf, arayüz, enum, record

```
abstract class Sekil {
    abstract double alan();  // gövdesiz
}
interface Ucabilir {
    void uc();          // public abstract
    default void kon() { } // gövdeli
    private void yardim() { } // Java 9+,
gövdeli
}
class Kus extends Hayvan
        implements Ucabilir, Yuzebilir { }

enum Gun { PZT, SAL, CAR }
Gun.valueOf("SAL").ordinal()  // 1
Gun.values()                  // dizi

record Nokta(int x, int y) { }
new Nokta(1, 2).x()           // 1
// toString: Nokta[x=1, y=2]
```

- Soyut sınıf ve arayüzden `new` ile nesne oluşturulmaz. Arayüzde yapıcı yok, alanlar sabittir.

İstisnalar, koleksiyonlar, dosya

try / catch / finally

```
try {
    int n = Integer.parseInt(s);
    System.out.println(100 / n);
} catch (NumberFormatException e) {
    System.out.println(e.getMessage());
} finally {
    System.out.println("her zaman");
}
throw new IllegalArgumentException("..");
void oku() throws IOException { ... }
```

```
Throwable
├─ Error   (StackOverflowError)
└─ Exception
   ├─ IOException       → checked
   │  (FileNotFoundException)
   └─ RuntimeException  → unchecked
      (Arithmetic, NullPointer,
       IndexOutOfBounds, ClassCast,
       IllegalArgument → NumberFormat)
```

- Özel `catch` önce, genel sonra. `finally`, `return` olsa bile çalışır. Checked exception için `try-catch` veya `throws` zorunlu.

ArrayList ve HashMap

```
ArrayList<Integer> a = new ArrayList<>();
(ArrayList<int> derlenmez)
add(x) / add(i, x)    sona / i. yere ekle
get(i) / set(i, x)    oku / değiştir
remove(i)             i. İNDEKSİ siler
remove(Integer.valueOf(x))  x DEĞERİNİ
size() / isEmpty()    adet / boş mu
contains(x) / indexOf(x)  var mı / yeri
Collections.sort(a)   sıralar
a.sort(Collections.reverseOrder())
                      büyükten küçüğe
```

```
Map<String, Integer> m = new HashMap<>();
m.put("ali", 85);        // ekle/güncelle
m.get("ali");            // 85, yoksa null
m.getOrDefault("x", 0);  // 0
m.containsKey("ali");    // true
m.put(k, m.getOrDefault(k, 0) + 1); // say
for (String k : m.keySet()) { ... }
```

- for-each içinde `remove` → `ConcurrentModificationException`. `HashMap` sırasızdır; sıralı anahtar için `TreeMap`.

Dosya okuma ve yazma

```
// Bölüm 14'ün ana yolu: Path + Files
Path p = Path.of("veri.txt");
Files.writeString(p, "satır\n");
String hepsi = Files.readString(p);
List<String> satirlar =
        Files.readAllLines(p);
// hepsi IOException fırlatabilir

// Klasik yol: File, Scanner, FileWriter
File dosya = new File("veri.txt");
try (Scanner f = new Scanner(dosya)) {
    while (f.hasNextLine()) {
        String satir = f.nextLine();
    }
} catch (FileNotFoundException e) {
    System.out.println("Dosya yok");
}
// true: dosyanın sonuna ekle
try (PrintWriter pw = new PrintWriter(
        new FileWriter("c.txt", true))) {
    pw.println("satır");
} catch (IOException e) {
    System.out.println(e.getMessage());
}
```

- `new FileWriter("x.txt")` eski içeriği **siler**. try-with-resources dosyayı otomatik kapatır.

Algoritma iskeletleri

```
// en büyük ve yeri (max = 0 ile BAŞLAMA)
int max = d[0], yer = 0;
for (int i = 1; i < d.length; i++) {
    if (d[i] > max) {
        max = d[i];
        yer = i;
    }
}
// yer değiştirme (swap)
int gecici = d[i];
d[i] = d[j];
d[j] = gecici;
```

```
// ikili arama: O(log n), dizi SIRALI
int alt = 0, ust = d.length - 1;
while (alt <= ust) {
    int orta = alt + (ust - alt) / 2;
    if (d[orta] == x) return orta;
    if (d[orta] < x) alt = orta + 1;
    else ust = orta - 1;
}
return -1;
```

```
O(1)        d[i], list.get(i), map.get(k)
O(log n)    ikili arama
O(n)        doğrusal arama, tek döngü
O(n log n)  merge sort, Arrays.sort
O(n²)       seçmeli/kabarcık/eklemeli,
            iç içe döngü
O(2ⁿ)       saf özyinelemeli Fibonacci
```

EK B

Terimler Sözlüğü

Kitapta geçen terimlerin Türkçe karşılıkları, İngilizceleri ve kısa açıklamaları. Terimler Türkçe adlarına göre alfabetik sıralandı; her tablonun başlığında o tablodaki harfler yazıyor. İngilizce kaynak okurken, hata mesajlarını anlamaya çalışırken veya hocanın İngilizce terim kullandığı sorularda bu tabloya bak.

A	İngilizce	Açıklama
akış	*stream*	Verinin sırayla okunduğu veya yazıldığı kanal (dosya, klavye).
akış (Stream API)	*stream*	Koleksiyon elemanlarını `filter`, `map` gibi ara işlemlerle ve bir son işlemle işleyen zincir (`liste.stream()`).
alan	*field*	Sınıfta tanımlanan, nesnenin durumunu tutan değişken.
algoritma	*algorithm*	Bir problemi çözen sonlu ve açık adımlar dizisi.
alt sınıf	*subclass*	`extends` ile başka bir sınıftan türeyen sınıf.
ana metot	*main method*	Programın başladığı `public static void main(String[] args)` metodu.
anahtar	*key*	Map'te bir değere ulaşmak için kullanılan benzersiz değer.
anahtar kelime	*keyword*	Dilde özel anlamı olan, ad olarak kullanılamayan kelime (`int`, `class`).
arayüz	*interface*	Bir sınıfın uygulaması gereken metotları tanımlayan sözleşme.
argüman	*argument*	Metot çağrılırken parametreye verilen gerçek değer.
aşağı dönüşüm	*downcasting*	Üst sınıf referansını alt sınıf tipine cast etme.
aşırı yükleme	*overloading*	Aynı adlı, farklı parametreli metotlar tanımlama.
atama	*assignment*	`=` ile bir değişkene değer koyma.
ayarlayıcı metot	*setter*	Özel bir alana denetimli değer atayan metot (`setNot`).
ayrıştırma	*parsing*	Metni sayıya veya başka bir yapıya çevirme (`Integer.parseInt`).

B	İngilizce	Açıklama
bayt kodu	*bytecode*	`javac`'ın ürettiği, JVM'in çalıştırdığı ara kod (`.class`).
bileşen	*component*	Swing'de ekrandaki her öğe: düğme, etiket, metin alanı.
bileşik atama	*compound assignment*	`+=`, `-=` gibi kısa atamalar; gizli cast içerir.
birleştirerek sıralama	*merge sort*	Diziyi ikiye bölüp sıralı parçaları birleştiren O(n log n) sıralama.
birleştirme	*concatenation*	String'leri `+` ile uç uca ekleme.
bit operatörü	*bitwise operator*	Sayıların bitlerini tek tek işleyen `&`, `|`, `^`, `~`, `<<`, `>>`, `>>>`.
blok	*block*	`{ }` arasındaki komut grubu; değişkenlerin kapsamını belirler.
böl ve yönet	*divide and conquer*	Problemi küçük parçalara bölüp sonuçları birleştiren yaklaşım.
büyük O gösterimi	*Big-O notation*	Çalışma süresinin girdi büyüdükçe nasıl arttığını gösteren üst sınır.

C–Ç	İngilizce	Açıklama
çağrı yığını	*call stack*	Devam eden metot çağrılarının tutulduğu bellek alanı.
çalışma zamanı	*runtime*	Programın çalıştığı süre; derleme zamanının karşıtı.
çalışma zamanı hatası	*runtime error*	Program çalışırken oluşan, programı durduran hata (exception).
çerçeve (pencere)	*frame*	Swing'de ana pencere (`JFrame`).
çıktı	*output*	Programın ekrana veya dosyaya yazdığı veri.
çok biçimlilik	*polymorphism*	Aynı çağrının nesnenin gerçek tipine göre farklı davranması.
çöp toplayıcı	*garbage collector*	Artık erişilemeyen nesnelerin belleğini otomatik geri veren JVM parçası.

D	İngilizce	Açıklama
daralan dönüşüm	*narrowing conversion*	Büyük tipten küçük tipe dönüşüm; cast ister, veri kaybedebilir.
değer ile geçirme	*pass-by-value*	Metoda argümanın değerinin kopyasının verilmesi.
değişken	*variable*	Adı ve tipi olan, değer saklayan bellek alanı.
değişken sayıda argüman	*varargs*	Parametre tipinden sonra `...` ile (`int... sayilar`) metoda istenen sayıda argüman verme.
değiştirilemez	*immutable*	Oluşturulduktan sonra durumu değişmeyen (ör. `String`, `record`).
derleme hatası	*compile-time error*	Yazım kuralı ihlali; program derlenmez, `.class` oluşmaz.
derleyici	*compiler*	Kaynak kodu başka bir dile çeviren program (`javac`).
desen eşleme	*pattern matching*	Nesnenin tipini kontrol edip aynı anda değişkene bağlama: `x instanceof Daire d`, `case Daire d ->`.
dinamik bağlama	*dynamic binding*	Çağrılacak metodun çalışırken nesnenin tipine göre seçilmesi.
dinleyici	*listener*	Bir olay olduğunda çalışan nesne (`ActionListener`).
dizi	*array*	Aynı tipte, sabit sayıda elemanı indeksle tutan yapı.
doğrusal arama	*linear search*	Elemanları baştan sona tek tek kontrol eden O(n) arama.
döngü	*loop*	Bir komut grubunu koşul sağlandıkça tekrar çalıştıran yapı.
dönüş tipi	*return type*	Metodun döndürdüğü değerin tipi; `void` değer döndürmez.
düğme	*button*	Tıklanabilen Swing bileşeni (`JButton`).

E	İngilizce	Açıklama
eklemeli sıralama	*insertion sort*	Her elemanı soldaki sıralı kısımda yerine koyan O(n^2) sıralama.
eleman	*element*	Dizi veya koleksiyondaki her bir değer.
erişim belirleyici	*access modifier*	`public`, `protected`, `private`: Üyenin nereden görülebileceği.
erişim metodu	*getter*	Özel bir alanın değerini döndüren metot (`getAd`).
eş referans	*aliasing*	İki referansın aynı nesneyi göstermesi; biri değiştirince öteki de görür.
eşleme	*map*	Anahtar–değer çiftlerini tutan koleksiyon (`HashMap`, `TreeMap`).
etiket	*label*	Metin gösteren Swing bileşeni (`JLabel`).

F-G-H	İngilizce	Açıklama
fırlatma	*throw*	Bir exception'ı `throw` ile oluşturup çağırana gönderme.
genel tip	*generic type*	Tip parametresi alan sınıf veya metot (`ArrayList<String>`).
genişleyen dönüşüm	*widening conversion*	Küçük tipten büyük tipe otomatik, kayıpsız dönüşüm.
girdi	*input*	Programa klavyeden veya dosyadan verilen veri.
gölgeleme	*shadowing*	Yerel değişken veya parametrenin aynı adlı alanı gizlemesi.
gövde	*body*	Metot, döngü veya sınıfın `{ }` içindeki kısmı.
hata ayıklama	*debugging*	Programdaki hataları bulup düzeltme süreci.

I-İ-J	İngilizce	Açıklama
ızgara yerleşimi	*grid layout*	Bileşenleri satır–sütun tablosuna yerleştiren düzen (`GridLayout`).
iç içe döngü	*nested loop*	Başka bir döngünün gövdesindeki döngü.
içe aktarma	*import*	Başka paketteki bir sınıfı adıyla kullanılabilir hale getirme.
iki uçlu kuyruk	*deque*	İki ucundan ekleme ve çıkarma yapılabilen kuyruk; yığın olarak da kullanılır (`ArrayDeque`).
ikili arama	*binary search*	Sıralı dizide aralığı her adımda yarıya indiren O(log n) arama.
ikinin tümleyeni	*two's complement*	Negatif tam sayıların saklanma yöntemi: bitleri ters çevir, 1 ekle; en soldaki bit işaret bitidir.
ilkel tip	*primitive type*	Java'nın 8 temel tipi: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`.
indeks	*index*	Elemanın 0'dan başlayan sıra numarası.
istisna	*exception*	Çalışma zamanında oluşan ve normal akışı bozan olay; `try-catch` ile yakalanır.
işlem önceliği	*operator precedence*	Bir ifadede operatörlerin hangi sırayla uygulanacağı kuralı.
işlenen	*operand*	Operatörün üzerinde çalıştığı değer.
izleme tablosu	*trace table*	Kodu elle çalıştırırken değişken değerlerini adım adım yazdığın tablo.
Java Sanal Makinesi	*Java Virtual Machine (JVM)*	Bytecode'u çalıştıran sanal makine; platform bağımsızlığını sağlar.
JIT derleyici	*just-in-time compiler*	Sık çalışan bytecode'u çalışırken makine koduna çeviren derleyici.
joker karakter	*wildcard*	Genel tiplerde bilinmeyen tip yerine `?` yazımı: `List<? extends Number>`.

K	İngilizce	Açıklama
kabarcık sıralaması	*bubble sort*	Komşu elemanları karşılaştırıp yer değiştiren O(n²) sıralama.
kaçış dizisi	*escape sequence*	`\n`, `\t`, `\"` gibi `\` ile başlayan özel karakter.
kalıtım	*inheritance*	Bir sınıfın başka bir sınıfın alan ve metotlarını devralması (`extends`).
kapsam	*scope*	Bir değişkenin erişilebildiği kod bölgesi.
kapsülleme	*encapsulation*	Alanları `private` yapıp erişimi metotlarla denetleme.
karakter dizisi	*string*	Metin; Java'da `String` sınıfı.
kararlı sıralama	*stable sort*	Eşit elemanların önceki sırasını koruyan sıralama.
karşılaştırıcı	*comparator*	İki nesnenin nasıl sıralanacağını tanımlayan nesne (`Comparator`).
kayan noktalı sayı	*floating-point number*	Ondalıklı sayı (`float`, `double`); her değeri tam tutamaz.
kayıt	*record*	Sadece veri taşıyan, kısa yazılan değiştirilemez sınıf.
kaynak kod	*source code*	İnsanın yazdığı program metni (`.java` dosyası).
kısa devre değerlendirme	*short-circuit evaluation*	`&&` ve `||`'de sonuç belliyse sağ tarafın hiç çalıştırılmaması.
koleksiyon	*collection*	Birden çok nesneyi tutan yapı (`List`, `Set`, `Map`).
kontrol edilen istisna	*checked exception*	Yakalanması veya `throws` ile bildirilmesi zorunlu exception (`IOException`).
kontrol edilmeyen istisna	*unchecked exception*	`RuntimeException` alt sınıfları; derleyici zorunlu tutmaz.
koşul	*condition*	Sonucu `true` veya `false` olan ifade.
kuyruk	*queue*	İlk giren ilk çıkar (FIFO) kuralıyla çalışan koleksiyon (`Queue`, `ArrayDeque`).
küme	*set*	Tekrarsız eleman tutan koleksiyon (`HashSet`, `TreeSet`).

L–M–N	İngilizce	Açıklama
lambda ifadesi	*lambda expression*	Kısa, isimsiz fonksiyon: `(a, b) -> Integer.compare(a, b)`.
liste	*list*	Sıralı, indeksli, büyüyebilen koleksiyon (`ArrayList`).
mantık hatası	*logic error*	Program çalışır ama yanlış sonuç verir; derleyici fark etmez.
mantıksal operatör	*logical operator*	`&&` (ve), `||` (veya), `!` (değil).
metin bloğu	*text block*	`"""` ile yazılan çok satırlı String (Java 15+).
metot	*method*	Bir işi yapan, adı olan ve çağrılabilen kod bloğu.
metot ezme	*overriding*	Alt sınıfta üst sınıf metodunu aynı imzayla yeniden yazma (`@Override`).
metot imzası	*method signature*	Metodun adı ve parametre tipleri; dönüş tipi dahil değildir.
mod (kalan)	*modulo, remainder*	`%` ile bölmeden kalan.
mühürlü sınıf	*sealed class*	`sealed` ve `permits` ile alt tipleri sınırlanmış sınıf veya arayüz.
nesne	*object*	Bir sınıftan `new` ile oluşturulan, durumu ve davranışı olan varlık.
nesneye yönelik programlama	*object-oriented programming (OOP)*	Programı sınıflar ve nesneler üzerinden kurgulama yaklaşımı.
nöbetçi değer	*sentinel value*	Girdinin bittiğini bildiren özel değer (ör. 0 veya −1).
numaralandırma	*enumeration (`enum`)*	Sabit değerlerden oluşan tip: `enum Gun { PZT, SAL }`.

O–Ö	İngilizce	Açıklama
olay	*event*	Tıklama, tuşa basma gibi kullanıcı eylemi.
olay güdümlü programlama	*event-driven programming*	Programın akışını kullanıcı olaylarının belirlediği yaklaşım (Swing).
otomatik kutulama	*autoboxing*	İlkel değerin sarmalayıcı nesneye otomatik çevrilmesi (`int` → `Integer`).
öbek	*heap*	`new` ile oluşturulan nesnelerin tutulduğu bellek alanı.
ön ek artırma	*prefix increment*	`++x`: önce artırır, sonra yeni değeri kullanır.
öncelik kuyruğu	*priority queue*	`poll()` ile her zaman en küçük (en öncelikli) elemanı veren kuyruk (`PriorityQueue`).
örnek	*instance*	Bir sınıftan oluşturulmuş belirli bir nesne.
örnek değişkeni	*instance variable*	Her nesnenin kendi kopyasına sahip olduğu (`static` olmayan) alan.
özyineleme	*recursion*	Bir metodun kendini daha küçük bir problemle çağırması.

P–R	İngilizce	Açıklama
paket	*package*	İlgili sınıfları gruplayan ad alanı (`java.util`).
parametre	*parameter*	Metot tanımında, çağrıda gelen değeri alan değişken.
platform bağımsızlığı	*platform independence*	Aynı bytecode'un JVM olan her sistemde çalışması.
referans	*reference*	Nesnenin bellekteki yerini gösteren değer.
referans tipi	*reference type*	İlkel olmayan tipler: sınıflar, diziler, arayüzler.

S–Ş	İngilizce	Açıklama
sabit	*constant*	`final` ile tanımlanan, değeri değiştirilemeyen değişken.
sabit değer	*literal*	Koda doğrudan yazılan değer: `42`, `3.14`, `'A'`, `"Ali"`.
sarmalayıcı sınıf	*wrapper class*	İlkel tipin nesne hali: `Integer`, `Double`, `Character`.
seçmeli sıralama	*selection sort*	Her turda en küçüğü bulup başa koyan $O(n^2)$ sıralama.
sınıf	*class*	Nesnelerin alanlarını ve metotlarını tanımlayan şablon.
sınıf değişkeni	*class (static) variable*	Tüm nesnelerin paylaştığı tek kopyalı `static` alan.
sıralama	*sorting*	Elemanları belirli bir düzene göre dizme.
son ek artırma	*postfix increment*	`x++`: önce mevcut değeri kullanır, sonra artırır.
sonsuz döngü	*infinite loop*	Koşulu hiç yanlış olmadığı için bitmeyen döngü.
soyut metot	*abstract method*	Gövdesi olmayan, alt sınıfın yazmak zorunda olduğu metot.
soyut sınıf	*abstract class*	Nesnesi oluşturulamayan, soyut metot içerebilen sınıf.
sözdizimi	*syntax*	Dilin yazım kuralları.

T–U–Ü	İngilizce	Açıklama
tampon	*buffer*	Okunan veya yazılan verinin geçici olarak bekletildiği bellek.
tanımlama	*declaration*	Değişken veya metodu tipi ve adıyla bildirme.
tanımlayıcı	*identifier*	Değişken, metot veya sınıfa verilen ad.
taşma	*overflow*	Değerin tipin sınırını aşıp ters uçtan dönmesi; hata vermez.
temel durum	*base case*	Özyinelemenin kendini çağırmadan doğrudan döndüğü durum.
tip çıkarımı	*type inference*	Derleyicinin tipi sağdaki değerden bulması (`var`).
tip dönüşümü	*type casting*	Değeri başka bir tipe çevirme: `(int) 3.7`.
tip silme	*type erasure*	Genel tip bilgisinin derlemeden sonra silinmesi; çalışırken `List<String>` ile `List<Integer>` aynı sınıftır.
tür parametresi	*type parameter*	Genel tiplerde `<T>` ile verilen tip yer tutucusu.
üçlü operatör	*ternary operator*	`koşul ? a : b` biçimindeki tek satırlık if-else.
üst sınıf	*superclass*	Başka bir sınıfın türediği sınıf.
üye	*member*	Sınıfın alanları ve metotları.

V–Y–Z	İngilizce	Açıklama
varsayılan değer	*default value*	Alan ve dizi elemanına otomatik verilen değer: `0`, `false`, `null`.
varsayılan metot	*default method*	Arayüzde `default` ile gövdesi yazılmış metot.
varsayılan yapıcı	*default constructor*	Hiç yapıcı yazılmazsa derleyicinin eklediği parametresiz yapıcı.
veri tipi	*data type*	Bir değişkenin tutabileceği değer türü.
yapıcı	*constructor*	Nesne oluşturulurken çalışan, sınıfla aynı adlı, dönüş tipsiz metot.
yerel değişken	*local variable*	Metot veya blok içinde tanımlanan, sadece orada geçerli değişken.
yerleşim yöneticisi	*layout manager*	Swing'de bileşenlerin pencereye nasıl dizileceğini belirleyen nesne.
yığın taşması	*stack overflow*	Çağrı yığınının dolması (`StackOverflowError`).
yineleme	*iteration*	Döngünün her bir turu.
yineleyici	*iterator*	Koleksiyonu eleman eleman dolaşan nesne (`Iterator`).
yorum	*comment*	Derleyicinin yok saydığı açıklama: `//`, `/* */`, `/** */`.
yorumlayıcı	*interpreter*	Kodu satır satır okuyup o anda çalıştıran program.
yukarı dönüşüm	*upcasting*	Alt sınıf nesnesini üst sınıf tipinde bir referansla gösterme.
zaman karmaşıklığı	*time complexity*	Çalışma süresinin girdi boyutuyla nasıl arttığı (Big-O ile).

EK C

Sık Karşılaşılan Hata Mesajları

Bu ekte, Java öğrenirken en sık göreceğin hata mesajlarını topladık. Her mesaj, onu **gerçekten** üreten küçük bir programla gösteriliyor; altındaki çıktı JDK 25'in verdiği mesajın kendisidir. Mesajlar İngilizcedir; Türkçe karşılıkları ve çözüm yolları açıklamalarda.

Bir derleme hatası mesajı şu kalıptadır:

```
DosyaAdı.java:SATIR: error: MESAJ
        hatalı satırın kendisi
                ^           ← derleyicinin takıldığı yer
1 error
```

Çalışma zamanı hatası ise exception'ın adını, varsa açıklamasını ve **yığın izini** (stack trace) verir. İzde en üstteki satır hatanın oluştuğu yer, alttakiler oraya hangi çağrılarla gelindiğidir:

```
Exception in thread "main" java.lang.ExceptionAdı: açıklama
    at Sinif.metot(Sinif.java:SATIR)     ← hatanın oluştuğu yer
    at Sinif.main(Sinif.java:SATIR)      ← metodu çağıran satır
```

> **İpucu: Hata mesajı okuma alışkanlığı**
>
> - Derleme hatalarında **her zaman ilk hatadan başla**. Tek bir eksik `}` veya `;`, arkasından sahte hatalar zinciri doğurabilir; ilkini düzeltip yeniden derle.
> - Çalışma zamanı hatasında yığın izinde **kendi dosyanın adını taşıyan ilk satırı** bul. `java.base/...` ile başlayan satırlar Java kütüphanesinin içidir; hata çoğunlukla senin o kütüphaneye verdiğin değerdedir.
> - Satır numarası, hatanın **fark edildiği** yeri gösterir; asıl neden daha yukarıda olabilir. Eksik bir `}` dosyanın sonunda, `if (...);` hatası `else` satırında fark edilir. Bulamazsan üst satırlara da bak.

Derleme hataları

`cannot find symbol`

```java
public class Sembol {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int toplam = 10;
        System.out.println(Toplam);
    }
}
```

Derleyici çıktısı – program derlenmez

```
Sembol.java:3: error: cannot find symbol
        Scanner sc = new Scanner(System.in);
        ^
  symbol:   class Scanner
  location: class Sembol
Sembol.java:3: error: cannot find symbol
        Scanner sc = new Scanner(System.in);
                         ^
  symbol:   class Scanner
  location: class Sembol
Sembol.java:5: error: cannot find symbol
        System.out.println(Toplam);
                           ^
  symbol:   variable Toplam
  location: class Sembol
3 errors
```

Ne demek? "Bu adı tanımıyorum." Derleyici `symbol:` satırında neyi bulamadığını (`class Scanner`, `variable Toplam`), `location:` satırında nerede aradığını söyler.

Nasıl düzeltilir? Üç olası neden var: (1) Ad yanlış yazılmış veya büyük/küçük harf farklı (`Toplam` ≠ `toplam`, `system` ≠ `System`); (2) sınıf için `import` unutulmuş (`import java.util.Scanner;`); (3) değişken, kullanıldığı yerde **kapsam dışında** (ör. bir `for` bloğunun içinde tanımlanıp dışında kullanılmış).

`';' expected`

```java
public class NoktaliVirgul {
    public static void main(String[] args) {
        int x = 5
        System.out.println(x);
    }
}
```

Derleyici çıktısı – program derlenmez

```
NoktaliVirgul.java:3: error: ';' expected
        int x = 5
                ^
1 error
```

Ne demek? Komutun sonundaki noktalı virgül eksik.

Nasıl düzeltilir? `^` işaretinin gösterdiği yere `;` koy. Benzer mesajlar: `')' expected` (kapanmayan parantez), `<identifier> expected` (genellikle metot dışına yazılmış bir komut).

`incompatible types`

```java
public class Tipler {
    public static void main(String[] args) {
        int ortalama = 170 / 2.0;
        int yas = "19";
        if (yas = 19) {
            System.out.println("On dokuz");
        }
    }
}
```

Derleyici çıktısı – program derlenmez

```
Tipler.java:3: error: incompatible types: possible lossy conversion from double to int
        int ortalama = 170 / 2.0;
                       ^
Tipler.java:4: error: incompatible types: String cannot be converted to int
        int yas = "19";
                  ^
Tipler.java:5: error: incompatible types: int cannot be converted to boolean
        if (yas = 19) {
                ^
3 errors
```

Ne demek? Sağdaki değerin tipi, soldaki değişkenin (veya koşulun) beklediği tipe uymuyor. Üç sık hali:

- *possible lossy conversion from double to int*: `double` → `int` gibi **daralan** bir dönüşüm, veri kaybına yol açabileceği için otomatik yapılmaz.
- *String cannot be converted to int*: Metin sayıya kendiliğinden dönüşmez.
- *int cannot be converted to boolean*: `if (yas = 19)` bir **atamadır**, karşılaştırma değil; `if` ise `boolean` ister.

Nasıl düzeltilir? Sırasıyla: `(int) (170 / 2.0)` ile bilinçli cast yap veya değişkeni `double` tanımla; `Integer.parseInt("19")` kullan; karşılaştırma için `==` yaz.

`missing return statement`

```java
public class DonusYok {
    static int isaret(int n) {
        if (n > 0) {
            return 1;
        } else if (n < 0) {
            return -1;
        }
    }

    public static void main(String[] args) {
        System.out.println(isaret(5));
    }
}
```

Derleyici çıktısı – program derlenmez

```
DonusYok.java:8: error: missing return statement
    }
    ^
1 error
```

Ne demek? Metot `int` döndüreceğini söylüyor, ama en az bir yolda (`n == 0`) `return` olmadan bitiyor. Derleyici koşulların içini değerlendirmez; "ya hiçbir `if` doğru olmazsa?" diye düşünür.

Nasıl düzeltilir? Metodun sonuna her durumu karşılayan bir `return` ekle (burada `return 0;`) veya son `else if`'i düz `else` yap.

`unreachable statement`

```java
public class Ulasilamaz {
    static int kare(int x) {
        return x * x;
        System.out.println("Hesaplandı");
    }

    public static void main(String[] args) {
        System.out.println(kare(4));
    }
}
```

Derleyici çıktısı – program derlenmez

```
Ulasilamaz.java:4: error: unreachable statement
        System.out.println("Hesaplandı");
        ^
1 error
```

Ne demek? Bu satıra hiçbir zaman gelinemez: Öncesindeki `return` (veya `break`, `continue`, `throw`) akışı her durumda keser.

Nasıl düzeltilir? Satırı `return`'ün önüne taşı ya da gereksizse sil. İçinde `break` olmayan bir `while (true)` döngüsünün arkasına yazılan kod da aynı hatayı verir.

`variable ... might not have been initialized`

```java
public class DegerYok {
    public static void main(String[] args) {
        int not = 75;
        String harf;
        if (not >= 50) {
            harf = "Geçti";
        }
        System.out.println(harf);
    }
}
```

Derleyici çıktısı – program derlenmez

```
DegerYok.java:8: error: variable harf might not have been initialized
        System.out.println(harf);
                           ^
1 error
```

Ne demek? Yerel değişkenin bir değeri olmayabilir: `not >= 50` yanlışsa `harf`'e hiç değer atanmaz. Yerel değişkenlerin (alanlardan farklı olarak) varsayılan değeri yoktur.

Nasıl düzeltilir? Tanımlarken ilk değer ver (`String harf = "";`) veya her yolda atama yap (eksik `else` dalını ekle).

`'else' without 'if'`

```java
public class ElseIfsiz {
    public static void main(String[] args) {
        int x = 7;
        if (x > 0);
            System.out.println("Pozitif");
        else
            System.out.println("Pozitif değil");
    }
}
```

Derleyici çıktısı – program derlenmez

```
ElseIfsiz.java:6: error: 'else' without 'if'
        else
        ^
1 error
```

Ne demek? `else`'in bağlanacağı bir `if` yok. Buradaki neden `if (x > 0);` satırının sonundaki **noktalı virgül**: `if`'in gövdesi boş komut olur, `println` bağımsız bir komuttur, `else` ortada kalır.

Nasıl düzeltilir? `if (...)` satırının sonundaki `;`'yi sil ve gövdeleri süslü parantezle yaz. Aynı hata, `if` ile `else` arasına birden fazla komutu süslü parantezsiz yazdığında da çıkar.

`reached end of file while parsing`

```java
public class DosyaSonu {
    public static void main(String[] args) {
        for (int i = 0; i < 3; i++) {
            System.out.println(i);
        }
    }
```

Derleyici çıktısı – program derlenmez

```
DosyaSonu.java:6: error: reached end of file while parsing
    }
     ^
1 error
```

Ne demek? Dosya bitti ama açılan süslü parantezlerin hepsi kapanmadı.

Nasıl düzeltilir? Her `{` için bir `}` olmalı. VS Code'da bir süslü parantezin yanına tıklayınca eşi vurgulanır; **Shift+Alt+F** (macOS'ta **Shift+Option+F**) ile kodu biçimlendirirsen girintiler eksik parantezi ele verir.

`non-static ... cannot be referenced from a static context`

```java
public class Statik {
    int sayac = 0;

    void artir() {
        sayac++;
    }

    public static void main(String[] args) {
        artir();
        System.out.println(sayac);
    }
}
```

Derleyici çıktısı – program derlenmez

```
Statik.java:9: error: non-static method artir() cannot be referenced from a static context
        artir();
        ^
Statik.java:10: error: non-static variable sayac cannot be referenced from a static context
        System.out.println(sayac);
                           ^
2 errors
```

Ne demek? `main` bir `static` metottur; bir nesneye bağlı değildir. `artir()` ve `sayac` ise **nesneye ait** (static olmayan) üyelerdir; hangi nesnenin `sayac`'ı olduğu belli değildir.

Nasıl düzeltilir? İki yol: Bir nesne oluşturup üyeye onun üzerinden eriş (`Statik s = new Statik(); s.artir();`), ya da üyeler gerçekten nesneye özgü değilse onları da `static` yap. Bölüm 6'daki yardımcı metotların hepsinin `static` olmasının nedeni budur.

`class ... is public, should be declared in a file named ...`

DosyaAdi.java

```java
public class Hesap {
    public static void main(String[] args) {
        System.out.println("Merhaba");
    }
}
```

Derleyici çıktısı – program derlenmez

```
DosyaAdi.java:1: error: class Hesap is public, should be declared in a file named Hesap.java
public class Hesap {
       ^
1 error
```

Ne demek? `public` bir sınıf, kendi adını taşıyan dosyada olmalıdır. Burada `public class Hesap` yazılmış ama dosyanın adı `DosyaAdi.java`.

Nasıl düzeltilir? Dosyanın adını sınıfla **harfi harfine** aynı yap (`Hesap.java`) ya da sınıfın adını dosyaya uydur. Büyük/küçük harf de önemlidir: `hesap.java` olmaz. Bir dosyada en fazla bir `public` sınıf olabilir; yardımcı sınıfları `public` yazmadan aynı dosyaya koyabilirsin.

`method ... cannot be applied to given types`

```java
public class Argumanlar {
    static double ortalama(int a, int b) {
        return (a + b) / 2.0;
    }

    public static void main(String[] args) {
        System.out.println(ortalama(70, 80, 90));
    }
}
```

Derleyici çıktısı – program derlenmez

```
Argumanlar.java:7: error: method ortalama in class Argumanlar cannot be applied to given types;
        System.out.println(ortalama(70, 80, 90));
                           ^
  required: int,int
  found:    int,int,int
  reason: actual and formal argument lists differ in length
1 error
```

Ne demek? Metodu, tanımındaki parametrelere uymayan argümanlarla çağırdın. `required:` beklenen tipleri, `found:` senin verdiklerini, `reason:` farkı söyler: burada argüman **sayısı** farklı.

Nasıl düzeltilir? Çağrıyı tanıma uydur ya da üç parametreli yeni bir `ortalama` metodu yaz (aşırı yükleme). Argümanların **sırası** ve **tipi** de tutmalıdır: `ortalama("70", 80)` da aynı hatayı verir.

`variable ... is already defined`

```java
public class IkiKez {
    public static void main(String[] args) {
        int i = 10;
        for (int i = 0; i < 3; i++) {
            System.out.println(i);
        }
    }
}
```

Derleyici çıktısı – program derlenmez

```
IkiKez.java:4: error: variable i is already defined in method main(String[])
        for (int i = 0; i < 3; i++) {
                 ^
1 error
```

Ne demek? Aynı kapsamda aynı adla ikinci bir değişken tanımlanamaz. `for`'un içindeki `int i`, dışarıdaki `i` hâlâ geçerliyken tanımlandığı için çakışır.

Nasıl düzeltilir? Değişkenlerden birinin adını değiştir ya da mevcut değişkeni tekrar tanımlamadan kullan (`for (i = 0; ...)`). Ardışık iki ayrı `for` döngüsünde `int i` kullanmak ise serbesttir; ilkinin `i`'si döngü bitince ölür.

`unreported exception ...; must be caught or declared to be thrown`

```
import java.io.FileReader;

public class Rapor {
    public static void main(String[] args) {
        FileReader okuyucu = new FileReader("veri.txt");
        System.out.println("Dosya açıldı");
    }
}
```

Derleyici çıktısı – program derlenmez

```
Rapor.java:5: error: unreported exception FileNotFoundException; must be caught or declared to be
thrown
        FileReader okuyucu = new FileReader("veri.txt");
                             ^
1 error
```

Ne demek? `FileReader`'ın yapıcısı, **kontrol edilen** (checked) bir exception olan `FileNotFoundException` fırlatabilir. Java, bu durumu ya yakalamanı ya da bildirmeni zorunlu tutar.

Nasıl düzeltilir? Kodu `try { ... } catch (FileNotFoundException e) { ... }` içine al ya da metodun imzasına `throws FileNotFoundException` (veya daha genel `throws IOException`) ekle. Bölüm 12 ve 14'e bak.

Çalışma zamanı hataları

`ArithmeticException: / by zero`

```
public class SifiraBol {
    public static void main(String[] args) {
        int toplam = 240;
        int adet = 0;
        System.out.println("Ortalama: " + toplam / adet);
    }
}
```

Çıktı – program hata ile durur

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at SifiraBol.main(SifiraBol.java:5)
```

Ne demek? Bir **tam sayı** sıfıra bölündü. (Ondalıklı bölmede hata olmaz: `240.0 / 0` sonucu `Infinity` olur.)

Nasıl düzeltilir? Bölmeden önce böleni kontrol et: `if (adet != 0) { ... } else { ... }`. Ortalama hesaplarken "hiç eleman yok" durumu klasik nedendir.

`ArrayIndexOutOfBoundsException`

```
public class DiziSiniri {
    public static void main(String[] args) {
        int[] notlar = {70, 85, 90, 60, 75};
        int toplam = 0;
        for (int i = 0; i <= notlar.length; i++) {
            toplam += notlar[i];
        }
        System.out.println(toplam);
    }
}
```

Çıktı – program hata ile durur

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for
length 5
        at DiziSiniri.main(DiziSiniri.java:6)
```

Ne demek? Dizide olmayan bir indekse erişildi. 5 elemanlı dizinin geçerli indeksleri **0–4**'tür; mesaj hangi indeksin (5) denendiğini ve dizinin uzunluğunu söyler.

Nasıl düzeltilir? Döngü koşulunu `i < notlar.length` yap (`<=` değil). Son elemana `notlar[notlar.length - 1]` ile eriş. Mümkünse for-each döngüsü kullan; indeks hatası yapamazsın.

`StringIndexOutOfBoundsException`

```
public class MetinSiniri {
    public static void main(String[] args) {
        String s = "Java";
        System.out.println(s.charAt(0));
        System.out.println(s.charAt(s.length()));
    }
}
```

Çıktı – program hata ile durur

```
J
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: Index 4 out of bounds for
length 4
        at java.base/jdk.internal.util.Preconditions$1.apply(Preconditions.java:55)
        at java.base/jdk.internal.util.Preconditions$1.apply(Preconditions.java:52)
        at java.base/jdk.internal.util.Preconditions$4.apply(Preconditions.java:213)
        at java.base/jdk.internal.util.Preconditions$4.apply(Preconditions.java:210)
        at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java:98)
        at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Preconditions.java:106)
        at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java:302)
        at java.base/java.lang.String.checkIndex(String.java:4904)
        at java.base/java.lang.StringLatin1.charAt(StringLatin1.java:45)
        at java.base/java.lang.String.charAt(String.java:1624)
        at MetinSiniri.main(MetinSiniri.java:5)
```

Ne demek? String'de olmayan bir konuma erişildi. `"Java"`'nın uzunluğu 4, son karakterin indeksi 3'tür. `substring` için de aynı exception, sınırları anlatan farklı bir mesajla gelir.

Nasıl düzeltilir? Son karakter için `s.charAt(s.length() - 1)` yaz. `substring(bas, son)`'da `son` **hariçtir** ve `s.length()`'i geçemez. Boş String'de (`""`) `charAt(0)` bile bu hatayı verir; önce `isEmpty()` kontrolü yap.

`NullPointerException`

```
public class BosReferans {
    public static void main(String[] args) {
        String[] adlar = new String[3];
        adlar[0] = "Ali";
        for (String ad : adlar) {
            System.out.println(ad.toUpperCase());
        }
    }
}
```

Çıktı – program hata ile durur

```
ALI
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String.toUpperCase()"
because "ad" is null
        at BosReferans.main(BosReferans.java:6)
```

Ne demek? `null` olan bir referans üzerinden metot çağrıldı veya alana erişildi. Java'nın **yardımcı mesajı** (*helpful NullPointerException*) hangi çağrının yapılamadığını (`String.toUpperCase()`) ve hangi değişkenin `null` olduğunu (`ad`) tam olarak söyler. Burada `new String[3]` dizisinin elemanları başlangıçta `null`'dır; sadece ilkine değer verildi.

Nasıl düzeltilir? Nesneyi kullanmadan önce oluşturduğundan (`new`) veya değer atadığından emin ol. Sınıf alanlarında yapıcıda ilk değer vermeyi unutmak da sık bir nedendir. Gerekirse kontrol et: `if (ad != null) { ... }`.

`NumberFormatException`

```
public class SayiDegil {
    public static void main(String[] args) {
        System.out.println(Integer.parseInt("42") + 1);
        System.out.println(Integer.parseInt("12a") + 1);
    }
}
```

Çıktı – program hata ile durur

```
43
Exception in thread "main" java.lang.NumberFormatException: For input string: "12a"
        at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
        at java.base/java.lang.Integer.parseInt(Integer.java:565)
        at java.base/java.lang.Integer.parseInt(Integer.java:662)
        at SayiDegil.main(SayiDegil.java:4)
```

Ne demek? Sayıya çevrilmek istenen metin geçerli bir sayı değil. Mesaj, sorunlu metni tırnak içinde gösterir. `" 42"` (boşluklu), `"3.5"` (`parseInt` için) ve `""` (boş) da aynı hatayı verir.

Nasıl düzeltilir? Metni `trim()` ile temizle; kullanıcıdan gelen girdiyi `try-catch (NumberFormatException e)` içinde çevir ve hatalıysa tekrar iste.

`InputMismatchException`

```java
import java.util.Scanner;

public class YanlisTip {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Öğrenci sayısı: ");
        int n = sc.nextInt();
        System.out.println(n + " öğrenci kaydedildi.");
    }
}
```

Çıktı – program hata ile durur

```
Öğrenci sayısı: on iki
Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:977)
        at java.base/java.util.Scanner.next(Scanner.java:1632)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2297)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2251)
        at YanlisTip.main(YanlisTip.java:7)
```

Ne demek? `Scanner` bir `int` beklerken kullanıcı başka bir şey yazdı. Mesajda açıklama yoktur; yığın izinde `Scanner.nextInt` satırını görmen ipucudur. Türkçe bölge ayarında `nextDouble()`'a `3.5` (nokta ile) yazmak da bu hatayı verebilir (Bölüm 3).

Nasıl düzeltilir? Okumadan önce `sc.hasNextInt()` ile kontrol et; değilse `sc.next()` ile hatalı kelimeyi atıp tekrar sor. Ya da girdiyi `nextLine()` ile okuyup `Integer.parseInt`'i `try-catch` içinde çağır.

`ClassCastException`

```java
class Hayvan { }

class Kedi extends Hayvan { }

class Kopek extends Hayvan { }

public class YanlisCast {
    public static void main(String[] args) {
        Hayvan h = new Kedi();
        Kopek k = (Kopek) h;
        System.out.println("Dönüştürüldü");
    }
}
```

Çıktı – program hata ile durur

```
Exception in thread "main" java.lang.ClassCastException: class Kedi cannot be cast to class Kopek
(Kedi and Kopek are in unnamed module of loader 'app')
        at YanlisCast.main(YanlisCast.java:10)
```

Ne demek? Aşağı dönüşüm (downcasting) derleme sırasında kabul edildi, çünkü bir `Hayvan` referansı pekâlâ bir `Kopek`'i gösterebilirdi. Ama çalışırken nesnenin gerçekte bir `Kedi` olduğu görüldü ve `Kopek`'e çevrilemedi.

Nasıl düzeltilir? Cast etmeden önce `instanceof` ile kontrol et: `if (h instanceof Kopek) { Kopek k = (Kopek) h; ... }`. Çok sık cast yapıyorsan, metodu üst sınıfa koyup polimorfizmden yararlanmak genellikle daha iyi bir tasarımdır (Bölüm 11).

`ConcurrentModificationException`

```java
import java.util.ArrayList;
import java.util.List;

public class EsZamanli {
    public static void main(String[] args) {
        List<Integer> sayilar = new ArrayList<>(List.of(1, 2, 3, 4, 5));
        for (Integer s : sayilar) {
            if (s % 2 == 0) {
                sayilar.remove(s);
            }
        }
        System.out.println(sayilar);
    }
}
```

Çıktı – program hata ile durur

```
Exception in thread "main" java.util.ConcurrentModificationException
        at java.base/java.util.ArrayList$Itr.checkForComodification(ArrayList.java:1096)
        at java.base/java.util.ArrayList$Itr.next(ArrayList.java:1050)
        at EsZamanli.main(EsZamanli.java:7)
```

Ne demek? Bir koleksiyon for-each ile dolaşılırken aynı koleksiyondan eleman silindi (veya eklendi). Arka plandaki yineleyici (iterator), listenin "altından değiştirildiğini" fark edip durur.

Nasıl düzeltilir? Silme işi için `sayilar.removeIf(s -> s % 2 == 0);` kullan. Ya da açıkça bir `Iterator` alıp onun `remove()` metodunu çağır, veya silinecekleri ayrı bir listede toplayıp döngüden sonra `removeAll` ile sil.

`StackOverflowError`

Bu hata oluştuğunda yığın izi binlerce satır sürer: Aynı `at ...faktoriyel(...)` satırı defalarca tekrarlanır. Okunabilir olsun diye burada hatayı yakalayıp sadece adını yazdırıyoruz (gerçek programlarda `Error` yakalanmaz):

```java
public class Tasma {
    static int faktoriyel(int n) {
        return n * faktoriyel(n - 1);
    }

    public static void main(String[] args) {
        try {
            System.out.println(faktoriyel(5));
        } catch (StackOverflowError e) {
            System.out.println("Yakalandı: " + e.getClass().getName());
        }
    }
}
```

Çıktı

```
Yakalandı: java.lang.StackOverflowError
```

Ne demek? Çağrı yığını (call stack) doldu. Neredeyse her zaman **sonsuz özyinelemedir**: Metodun temel durumu (base case) yok ya da çağrılar temel duruma hiç ulaşmıyor (ör. `n - 2` ile tek sayıdan 0'a inmeye çalışmak).

Nasıl düzeltilir? Temel durumu ekle (`if (n <= 1) return 1;`) ve her çağrının temel duruma **yaklaştığından** emin ol. Yığın izinde tekrarlanan satır, hangi metodun kendini durmadan çağırdığını gösterir (Bölüm 15).

`IllegalFormatConversionException`

```
public class BicimHatasi {
    public static void main(String[] args) {
        double ortalama = 82.5;
        System.out.printf("Ortalama: %d%n", ortalama);
    }
}
```

Çıktı – program hata ile durur

```
Ortalama: Exception in thread "main" java.util.IllegalFormatConversionException: d !=
java.lang.Double
        at java.base/java.util.Formatter$FormatSpecifier.failConversion(Formatter.java:4662)
        at java.base/java.util.Formatter$FormatSpecifier.printInteger(Formatter.java:3200)
        at java.base/java.util.Formatter$FormatSpecifier.print(Formatter.java:3155)
        at java.base/java.util.Formatter.format(Formatter.java:2761)
        at java.base/java.io.PrintStream.format(PrintStream.java:1183)
        at java.base/java.io.PrintStream.printf(PrintStream.java:1081)
        at BicimHatasi.main(BicimHatasi.java:4)
```

Ne demek? `printf` biçim belirteci ile verilen değerin tipi uyuşmuyor. Mesajdaki `d != java.lang.Double`, "`%d` belirtecine `Double` verildi" demektir.

Nasıl düzeltilir? Ondalıklı sayı için `%f` veya `%.2f`, tam sayı için `%d`, metin için `%s` kullan. Belirteç sayısından az değer verirsen de `MissingFormatArgumentException` alırsın.

`IndexOutOfBoundsException` (ArrayList)

```
import java.util.ArrayList;

public class ListeSiniri {
    public static void main(String[] args) {
        ArrayList<String> liste = new ArrayList<>();
        liste.add("elma");
        liste.add("armut");
        System.out.println(liste.get(liste.size()));
    }
}
```

Çıktı – program hata ile durur

```
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index 2 out of bounds for length 2
        at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java:100)
        at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Preconditions.java:106)
        at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java:302)
        at java.base/java.util.Objects.checkIndex(Objects.java:365)
        at java.base/java.util.ArrayList.get(ArrayList.java:428)
        at ListeSiniri.main(ListeSiniri.java:8)
```

Ne demek? `ArrayList`'te olmayan bir indekse erişildi; dizilerdeki `ArrayIndexOutOfBoundsException`'ın liste karşılığıdır. Boş bir listede `get(0)` da bu hatayı verir.

Nasıl düzeltilir? Geçerli indeksler `0` ile `size() - 1` arasıdır. Erişmeden önce `isEmpty()` veya `size()` ile kontrol et.